

บันทึกภายในทีม

ความผิดพลาดทั้งหมด ที่เราทำระหว่างสร้าง Constistant และ Destrier

รวมทุกเรื่องผิดพลาด ตั้งแต่วันแรกจนวันชนะ — เก็บไว้ครบทุกข้อ เพื่อไม่ให้พลาด
ซ้ำอีก

รวบรวมจากไดอารี่ข้างทั้งหมด 70 ไฟล์ · ช่วงเวลา 2026-07-01 ถึง 2026-09-02 · ทั้งหมด 1054 เรื่อง

เอกสารนี้เกิดขึ้นหลังทีมชนะเลิศ STECON Construction Innovation Challenge 4 — และ
เหตุผลจริงที่ชนะ ไม่ใช่เพราะไม่มีอะไรผิดพลาด แต่เพราะเกือบทุกอย่างเคยพังมาแล้วอย่างน้อยหนึ่งครั้ง
ถูกจัดไว้ แล้วถูกแก้ ทุกข้อในนี้เก็บไว้ครบโดยตั้งใจ ไม่มีการตัดข้อที่ดูแย่ออก เพราะข้อที่ดูแย่ที่สุด คือ
ข้อที่ยังมีประโยชน์เหลืออยู่มากที่สุด

วิธีใช้เอกสารนี้

เอกสารนี้ไม่ได้มีไว้อ่านรวดเดียวจบ มีไว้เปิดหาตอนกำลังจะทำอะไรที่เคยพลาดมาแล้ว

- 1. ก่อนเริ่มงานที่เสียเงินหรือเสียเวลาเยอะ (เทรนโมเดล เข้าการ์ดจอ ย้ายข้อมูลชุดใหญ่) — อ่านหัวข้อ “กฎ 31 ข้อ” ก่อน เป็นเนื้อหาทั้งหมดที่บีบมาเหลือไม่กี่หน้า
- 2. ตอนมีอะไรพัง — ค้นหาการในเล่มนี้ก่อนจะเริ่มเดาสาเหตุ ความผิดพลาดจำนวนมากในนี้คือครั้งที่สองและสามของเรื่องเดิม
- 3. ตอนทำงานเสร็จ — ถ้าเจออะไรที่ยังไม่มีในเล่ม ให้เพิ่มเข้าไป รูปแบบตั้งใจให้เหมือนกันหมด: เกิดอะไรขึ้น · ต้นเหตุจริง · เสียหายอะไร · แก้ยังไง · ได้กฎอะไรกลับมา

ภาพรวมทั้งหมด

ระดับความรุนแรง	จำนวน
วิกฤต เสียเงิน เสียข้อมูล หรือเสียงานทั้งรอบ	147
สูง ตัวเลขผิดพลาดออกไป หรือเสียเวลาเป็นวัน	345
กลาง จับได้ก่อนเสียหาย แต่เป็นเรื่องจริง	415
ต่ำ เรื่องเล็กและเกือบพลาด	147
รวมทั้งหมด	1054

แบ่งตามหมวด

หมวด	ทั้งหมด	วิกฤต
การตรวจสอบ และ ความมั่นใจผิดๆ	81	10
พึงเสียบ ไม่มีใครรู้	21	7
วิธีวัดผล และ การประเมิน	94	23
คุณภาพข้อมูล ความถูกต้อง และ ข้อมูลหาย	142	26
สเปกข้อมูล และ ข้อตกลงที่ค่อยๆ เปลี่ยน	39	4
ก๊าดักการตั้งค่า สภาพแวดล้อม และ โลบรารี	87	18
เครื่อง การ์ดจอเช่า และ ค่าใช้จ่าย	124	7
ก๊าดักของเครื่องมือ และ พฤติกรรมของบริการภายนอก	58	2
การเทรนโมเดล และ วิธีทำ ML	75	18
บั๊กในโค้ด และ ตรรกะผิด	53	6

Git ความสะอาดของ repo และ ของที่ทำแล้วหาย	19	2
เอกสาร การตั้งชื่อ และ การสื่อสาร	32	3
กระบวนการทำงาน การวางแผน และ งานที่เสียเปล่า	229	21

กฎ 31 ข้อ

กลั่นมาจาก 147 เรื่องที่รุนแรงระดับวิกฤต เรียงตามว่ากันความเสียหายได้มากแค่ไหน ไม่ได้เรียงตามว่าข้อไหนฟังดูเท่

พิสูจน์ อย่าเดา

1. “รันผ่าน ไม่มี error” ไม่ได้แปลว่ามันทำงานถูก

หลังทุกขั้นตอนที่แปลงของ — merge, convert, quantize, export, ย้ายข้อมูล — ต้องวัดของที่ได้ออกมาจริงๆ เทียบน้ำหนักโมเดล นับจำนวนแถว อ่านไฟล์กลับมาดู ขั้นตอนที่ไม่ขึ้น error ยังไม่ได้พิสูจน์อะไรเลยสักอย่างเกี่ยวกับผลลัพธ์ของมัน

2. ตัวกันที่ยังไม่เคยเห็นมันจับอะไรได้ ไม่นับว่าเป็นตัวกัน

ก่อนจะเชื่อการตรวจสอบใดๆ ให้ป้อนของที่ผิดแน่ๆ เข้าไปแล้วดูว่ามันร้องจริงไหม การตรวจที่ของฟังผ่านไปได้ แยกว่าไม่ตรวจเลย เพราะมันสร้างความมั่นใจปลอมๆ ให้เรา

3. ห้ามบอกว่าเสร็จ จนกว่าจะรันจบทั้งเส้นด้วยข้อมูลจริงที่ใช้งานจริง

ไม่มี error ไม่ใช่หลักฐานว่าใช้ได้ ข้อมูลตัวอย่างที่เราเขียนเองมีข้อสันนิษฐานของเราฝังอยู่ และมันจะบังบั๊กตัวที่ทดสอบมีไว้จับพอดิ ให้สร้างข้อมูลทดสอบจากของจริงเสมอ

4. ตัวจำลองต้องเรียกโค้ดจริง ห้ามเขียนขึ้นใหม่ว่า “มันน่าจะทำงานแบบนี้”

ทางลัดทุกอันในตัวจำลอง จะกลายเป็นตัวเลขที่เกินจริงในผลลัพธ์ และจะมีคนเอาตัวเลขนั้น ไปตัดสินใจต่อ

5. เรนเดอร์สิ่งที่เครื่องเห็นจริงๆ ออกมาด้วยตาตัวเอง

ครอบรูปมาเทียบกัน 5 นาที ตีกว่านั่งคำนวณ token เป็นชั่วโมง วิธีนี้เคยช่วยประหยัดค่าการ์ดจอทั้งรอบมาแล้วหลายครั้ง

ห้ามฟังเจียบ

6. ข้อมูลที่ขาดหาย ห้ามแสดงผลออกมาเป็นศูนย์เด็ดขาด

ทุกเส้นทางที่คิดราคา ปริมาณ หรือคะแนน ต้องแยกให้ออกระหว่าง “วัดได้จริงว่าเป็นศูนย์” กับ “ไม่รู้” แล้วส่งเสียงดังกับอย่างหลัง เพราะบรรทัดที่เป็นศูนย์เจียบๆ บนหน้าจอ แยกไม่ออกจากบรรทัดที่ถูกต้องจริง

7. อะไรที่หึง ต้องบอกว่าหึง ห้ามหายไปเจียบๆ

ค่าที่ไม่รู้จัก ฟิลด์ที่อ่านไม่ออก แถวที่ถูกกรองทิ้ง รูปแบบที่ไม่คุ้น ทุกอันต้องบอกว่าหึงอะไรไปและเพราะอะไร ตัวอ่านข้อมูลที่คืนค่าว่างเปล่าเมื่อเจอรูปแบบแปลกๆ คือบั๊กข้อมูลหายชัดๆ

8. ทุกขั้นตอนที่ทำข้อมูลหายได้ ต้องมีการนับยอดกระทบกันที่ปลายทาง

นับจำนวนที่เข้า เทียบกับจำนวนที่ถูกใช้จริง แล้ว assert ว่าหมวดที่ควรมีของต้องไม่ว่าง คิวที่มีแต่คนนับ แต่ไม่มีใครเอาไปใช้ คือช่องทางข้อมูลหายที่ปลอมตัวมาเป็นความคืบหน้า

9. ห้ามเชื่อค่าตั้งต้นที่เราไม่ได้เลือกเอง

พิมพ์ค่าจริงออกมาและ assert ตอนเริ่มงานเสมอ — จำนวน token ต่อรูป ความยาว sequence ขนาดรูป timeout ค่า decoding ค่า default ของไลบรารีที่เราไม่ได้เลือก คือบั๊ก config ที่รื้อวันแพ่งที่สุดของมันอยู่

10. ขาวดีเกินคาด ให้ถือว่าเป็นอาการผิดปกติ

แรมใช้น้อยผิดปกติ เทสผ่านฉลุยผิดปกติ คะแนนสวยเกินจริง — ให้ไปตรวจก่อนดีใจ ทุกอันที่ว่ามาเคยชอนบั๊กจริงในโปรเจกต์นี้มาแล้วทั้งนั้น

วัดให้ตรงกับสิ่งที่อยากรู้

11. ห้ามรายงานคะแนนโดยไม่บอกเพดานที่เป็นไปได้จริง

เปอร์เซ็นต์ที่เท่ากันสองตัว อาจเป็นคนละปัญหากันสุดขีด ตัววัดที่มีเพดานแค่ 4% ไม่ได้กำลังวัดคุณภาพโมเดล แต่กำลังวัดคุณภาพการ label ของเราเอง

12. ตัวเลขความแม่นยำทุกตัว ต้องมี “วัดจากอะไร” ติดมาด้วยเสมอ

ชุดทดสอบต้องโหลดไฟล์ตัวเดียวกับที่ใช้งานจริง ถ้าตอนวัดกับตอนใช้โหลดคนละไฟล์กัน ตัวเลขนั้นกำลังอธิบายคนละระบบกัน

13. คะแนนเรื่องรูปแบบ ไม่ใช่คะแนนเรื่องคุณภาพ

JSON ถูกต้อง 100% ได้ ในขณะที่ความหมายถูกแค่ 0% ทุกครั้งที่วัดผล ต้องมีคนเปิดผลลัพธ์จริงดูด้วยตา และต้องรายงานตัววัดที่สนใจจริงแยกออกมาต่างหาก

14. ห้ามมีค่าเฉลี่ยโผล่อยู่ในคำถามเด็ดขาด

และต้อง assert ว่าป้อนข้อมูลต่างกันแล้วต้องได้คำตอบต่างกัน ถ้าตอบเหมือนกันหมดทุกอัน นั่นคือมันลอคคำถาม ไม่ใช่ความสามารถ

15. ต้องตรวจว่าชุดเทรนกับชุดวัดผลไม่ปนกัน ทุกครั้ง ทุกรอบ

ห้ามเชื่อชุดแบ่งข้อมูลของรอบก่อนโดยไม่ตรวจซ้ำ และต้องมีแหล่งเดียวที่ตัดสินว่าข้อมูลชิ้นไหนอยู่ชุดไหน รายชื่อที่ hardcode ไว้เป็นชุดที่สอง คือรอยร้าวที่รอเวลาปะทุ

16. ทดสอบเคสที่สำคัญที่สุดก่อน

ถ้าครบรอบแล้วความสามารถหลักที่เราอยากรู้ยังไม่ถูกทดสอบเลย แปลว่ารอบนั้นวัดผิดสิ่งไปทั้งรอบ

ความจริงต้องมีที่มาที่เดียว

17. แก่ schema ยังไม่จบ จนกว่าทุกคนที่อ่านมัน และตัวตรวจของมัน จะถูกแก้ไขใน commit เดียวกัน

คำว่า “validator ผ่านแล้ว” ไม่มีความหมายเลย ถ้าเราชี้ไม่ได้ว่าตัวไหนคือตัวที่ควรจับได้ก่อนที่จะแก้

18. เจอบั๊กแล้ว ต้อง grep หาทุกจุดที่เรียกใช้ ก่อนปิดงาน

ที่ที่ควรแก้คือฟังก์ชันกลางที่ทุกคนใช้ร่วมกัน ไม่ใช่ไฟล์ที่เราบังเอิญเปิดอยู่ตอนนั้น การแก้ฝาแฝดไปแค่ข้างเดียว คือบั๊กที่รอคิวของอีกข้างอยู่

19. ค่าเดียวกันที่เก็บไว้สองที่ จะเพี้ยนจากกันแน่นอน ให้ถือไว้ก่อนเลย

timeout ฝั่ง client กับฝั่ง server, config ที่ก๊อปกันมา, รายชื่อที่ hardcode ซ้ำ — ให้ดึงมาจากที่เดียว และ assert ความสัมพันธ์ระหว่างกันด้วย (เช่น timeout ฝั่ง server ต้องน้อยกว่าฝั่ง client เสมอ)

20. ค่าคงที่ทางวิศวกรรมทุกตัว ต้องมีมาตรฐานและเลขช้อยกำกับไว้

เพื่อให้ค่าที่ไม่ตรงมองเห็นได้ตั้งแต่ตอนอ่าน ไม่ใช่มารู้อีกหลายปีให้หลัง และเมื่อค่าไหนเปลี่ยน ต้องคำนวณทุกอย่างที่ขึ้นอยู่กับค่าของมันใหม่ในรอบเดียวกัน

เรื่องที่ย้อนกลับไม่ได้ ต้องหยุดให้สนิท

21. ไฟล์ที่อยู่บนเครื่องเรา ไม่ใช่ข้อมูลสำรอง

ก่อนเปิดหรือคืนเครื่องทุกครั้ง ต้องไปไล่ดูรายชื่อไฟล์ที่ปลายทางนอกเครื่องว่ามีอยู่จริง และต้องทดสอบปลายทางไว้ก่อน ไม่ใช่รอถึงเวลาที่ต้องใช้จริงค่อยรู้ว่ามันใช้ไม่ได้

22. การสำรองข้อมูลคือด่านบังคับที่ข้ามไม่ได้ ไม่ใช่โน้ตเตือนความจำ

และห้ามเอาไปแขวนไว้ในขั้นตอนที่ข้ามได้ ก่อนจะข้ามขั้นตอนไหนก็ตาม ต้องไล่ดูก่อนว่าขั้นตอนนั้นแบกอะไรไว้บ้าง

23. คำเตือนเรื่องข้อมูลหายถาวร ต้องพูดแยกออกมาต่างหาก เป็นภาษาคน พร้อมคำว่า “หยุด”

ห้ามซุกไว้ในรายการงานว่า “เดี๋ยวค่อยทำ” ถ้าผลที่ตามมาคือกินไม่ได้ ให้ปฏิเสธที่จะทำต่อ ดีกว่าทำต่อแล้วแปะโน้ตไว้เฉยๆ

24. งานที่กำลังรันอยู่ ให้ถือว่ามันจองทั้งเครื่องและทั้ง environment ไว้แล้ว

ห้าม pip install ห้ามใช้ interpreter รวมกัน ห้ามแทรกงานอื่นเข้าไป ให้แยก venv ออกมา หรือรอให้งานเดิมจบสนิทก่อน

ขนาดและขีดจำกัด ต้องมาจากของจริงที่วัดแล้ว

25. ทุกค่าเพดาน ขีดจำกัด และความยาว ต้องมาจากค่าสูงสุดที่วัดจากข้อมูลจริง

ห้ามมาจากตัวอย่างไม่กี่ชิ้น ห้ามมาจากค่ากลาง ห้ามมาจากการเดา และการตัดข้อมูลทั้งต้อง error ดังๆ เพราะการตัดเจี๊ยบๆ ทำลาย label ไปโดยไม่มีอะไรป้องกัน

26. ห้ามสรุปลักษณะข้อมูลทั้งหมด จากตัวอย่างชิ้นเดียว

ประโยคแบบ “ข้อมูลมันหน้าตาแบบนี้” ต้องมาจากสคริปต์ที่วัดทุกชิ้นแล้วรายงานค่าต่ำสุด ค่าสูงสุด และจำนวน

27. พิสูจน์เส้นทางขึ้นใช้งานทั้งเส้นบนโมเดลตั้งต้นก่อน แล้วค่อยจ่ายเงินเทรน

โมเดลที่ export ไม่ได้ โหลดไม่ได้ เสิร์ฟไม่ได้ มีค่าเท่ากับศูนย์ ไม่ว่าคะแนนจะสวยแค่ไหนก็ตาม

บทเรียนต้องกลายเป็นเครื่องจักร

28. ความผิดพลาดที่เกิดซ้ำ คือด่านอัตโนมัติที่ยังไม่มี ไม่ใช่โชคร้าย

พอเห็นอาการเดิมเป็นครั้งที่สอง ให้หยุดแก้เป็นรายครั้ง แล้วติดตั้งการตรวจถ่วงถ่วงก่อนรันแทน ในโปรเจกต์นี้บั๊กคลาสเดียวกันเกิดซ้ำถึง 3 ครั้ง ทั้งที่บทเรียนของมันถูกเขียนไว้เรียบร้อยแล้วเป็นตัวแทนสื่อ

29. บทเรียนที่จดไว้ แต่ไม่ได้เอาไปใช้ คือบทเรียนที่ยังไม่ได้เรียน

วันที่เขียนลงไดอารี่ว่า “X เป็นปัญหา” ให้ grep ทั้ง repo หา X แล้วแก้ไขให้หมดในวันเดียวกันนั้นเลย

30. โค้ดที่ไม่เคยรัน คือโค้ดที่ยังไม่ได้ตรวจ และมันเน่าไปตามเวลา

รันทุกสคริปต์อย่างน้อยหนึ่งครั้งกับข้อมูลเล็กๆ ก่อนเอาไปใช้กับงานที่เสียเงิน โค้ดที่ทิ้งไว้ไม่ได้รันนานๆ เสี่ยงกว่าโค้ดที่ยังไม่เคยมีอยู่เสียอีก

31. ห้ามเขียนต้นเหตุที่ยังพิสูจน์ไม่ได้

ต้องพิสูจน์ด้วยการอ่านโค้ดจริง หรือทำซ้ำให้เห็นสักรบรทัด ถ้าหลักฐานยังไม่พอ ให้เขียนตรงๆ ว่า “ยังไม่รู้ต้นเหตุ สงสัย A หรือ B” เพราะต้นเหตุผิดที่เขียนอย่างมั่นใจ พาทุกคนไปผิดทางกันหมด

การตรวจสอบ และ ความมั่นใจผิดๆ

บอกว่าเสร็จ/ถูกต้อง/ตรวจแล้ว ทั้งที่ยังไม่ได้ตรวจจริง

ทั้งหมด 81 เรื่อง · วิกฤต 10 · สูง 24 · กลาง 41 · ต่ำ 6

เรื่องที่ต้องอ่านให้ครบ (34)

วิกฤต 2026-07-09

Tool call รายงานว่า 'ถูกปฏิเสธ' แต่ .gitignore ถูกแก้ไปแล้วจริงๆ บนดิสก์

เกิดอะไรขึ้น: การพยายามอัปเดต 8 บรรทัดที่เกี่ยวกับ raw/ ใน .gitignore ถูกรายงานกลับมาจากระบบว่าเป็น tool use ที่ถูกปฏิเสธ แต่พอตรวจสอบรอบสุดท้ายกลับพบว่าเนื้อหาไฟล์ถูกเปลี่ยนเป็นชื่อโฟลเดอร์ใหม่ไปแล้วจริงๆ — ผลลัพธ์ที่รายงานไม่ตรงกับสภาพจริงบนดิสก์

ต้นเหตุจริง: สัญญาณ 'ปฏิเสธ' จากระบบเชื่อไม่ได้ว่าไม่มีการเขียนไฟล์เกิดขึ้น แต่กระบวนการของทิมกลับถือผลลัพธ์ของ tool เป็นข้อสรุป แทนที่จะไปตรวจตัวไฟล์จริง

เสียหายอะไร: ถ้าไม่ได้ตรวจสอบสุดท้าย .gitignore จะค้างซีไปชื่อโฟลเดอร์ที่ไม่มีอยู่แล้วอย่างถาวร ทำให้ข้อมูล 533 MB หลุดออกจากการ ignore แบบเสียๆ

แก้ยังไง: ไปอ่านไฟล์จริงบนดิสก์ คำนวณ .gitignore กลับเป็น raw/ แล้วยืนยันด้วยการ grep ทั้ง repo ได้ผลลัพธ์ 0 จุดสำหรับชื่อที่เล็กใช้

กฎที่ได้: อย่าเชื่อว่ารายงาน 'ถูกปฏิเสธ'/'ล้มเหลว' ของ tool แปลว่าไม่มีอะไรเปลี่ยน — ให้ไปอ่านไฟล์จริงกลับมาดูหลังทุกครั้งที่มีการเขียนล้มหรือถูกปฏิเสธ โดยเฉพาะก่อนจะสรุปว่าย้อนกลับสำเร็จแล้ว

วิกฤต 2026-07-13

ค่ากริดสมมติ 8.5 ที่ผิด ถูกยืนยันด้วยหลักฐานที่จริงๆ แล้วคือการอ้างวนกลับมาที่ตัวเอง

เกิดอะไรขึ้น: พอถูกถามว่ากริดสมมติ `3` ควรเป็น 7.6 ไม่ใช่ 8.5 หรือเปล่า Claude ตอบผิดว่ายืนยันแล้วว่าเป็น 8.5 โดยอ้างไจระยะในหน้า 06 กับช่วงคาน B4X ในหน้า 19 เป็นหลักฐานยืนยันอิสระ แต่ช่วงของ B4X มี `span\_source: "grid\_table"` แปลว่ามันถูกคำนวณย้อนกลับมาจากค่า 8.5 ในกริดมันนั่นเอง

ต้นเหตุจริง: เอาค่าที่คำนวณต่อมา (derived) ไปนับเป็นหลักฐานยืนยันอิสระ ทั้งที่ `confidence\_flags` ในไฟล์นั้นเองก็เขียนไว้แล้วว่า `uses\_dummy\_grid\_3\_from\_page00\_master` แต่ไม่มีใครดูตรงนั้นก่อนจะเอาไปนับเป็นแหล่งที่สอง

เสียหายอะไร: ค่ากริดที่ผิดซึ่งสืบทอดมาตั้งแต่รอบ t3/t4 ลามไปที่ช่วงคาน B4X สองตัว (เป็น 1.5 แทนที่จะเป็น 0.6) และคำตอบที่ผิดถูกยืนยันผ่านการถามซ้ำสองรอบ จนผู้ใช้ต้องบังคับให้ไปตรวจใหม่กับภาพผังบริเวณ

แก้ยังไง: ไปดูหน้า A-03 (ผังบริเวณ) ซึ่งระบุชัดว่าตัวอาคารยื่นเลยกริด 3 ออกไป 0.60 ม. แก่ `3` จาก 8.5 เป็น 7.6 และแก้ช่วงคาน B4X ทั้งสองตัวจาก 1.5 เป็น 0.6 พร้อมเขียนคำเตือนบันทึกเรื่องการอ้างวนไว้

กฎที่ได้: ก่อนจะนับค่าไหนเป็นหลักฐานยืนยันอิสระ ให้ดู `span\_source`/`confidence\_flags` ของมันก่อน — ค่าที่คำนวณมาจากกริดแม่ ไม่มีทางใช้ยืนยันกริดแม่ได้ และให้เขียนกติกาข้อนี้ลงใน schema ให้ชัด

เสาถูกทำโครงสร้างข้อมูลแบบคาน มีเหล็กบน/ล่าง ทำให้นับเหล็กแกนหลัก 4 เส้นกลายเป็น 8

เกิดอะไรขึ้น: ไฟล์ `หน้า21\_view2\_schedule.json` บันทึกเสา C1 ไว้ด้วย `main\_bar.top: 4` และ `main\_bar.bottom: 4` รวมเป็น 8 ทั้งที่ในแบบมีเหล็กมุมชุดเดียว 4 เส้น (4-Ø12mm) เสาไม่มีหน้าบน/ล่าง

ต้นเหตุจริง: เอาโครงสร้างสเปกของคานมาใช้กับเสาซ้ำๆ โดยไม่ได้ตรวจว่าแนวคิดมันย้ายมาใช้ได้จริงไหม บางไฟล์ถึงกับกลบอาการด้วยการเพิ่มช่อง `total\_main\_bar\_count` แทนที่จะไปแก้โครงสร้างให้ถูก

เสียหายอะไร: นับเหล็กแกนหลักของเสาเกินจริง 100% ทุกจุดที่เกิดขึ้น เป็นความผิดพลาดที่กระทบปริมาณเหล็กและต้นทุนโดยตรง เจอทั้งในบ้าน 1, บ้าน 2 (หน้า 29), บ้าน 3 (หน้า 35) และต่อมาเจออีก 6 ไฟล์ในเสาร์แรกของบ้าน 1/2/3

แก้ยังไง: เปลี่ยนเป็น `main\_bar: {count: 4}` ชุดเดียว ยืนยันทั้งโปรเจกต์บนบ้าน 2 แล้ววันที่ 2026-07-20 ทำการกวาดทั้งโปรเจกต์ เจอและแก้อีก 6 ไฟล์ที่มีบั๊กเดียวกัน แล้วยืนยันว่าไม่เหลือจุดไหนอีก

กฎที่ได้: อย่าเอาโครงสร้างสเปกของ element ชนิดหนึ่งไปใช้กับอีกชนิดหนึ่งโดยไม่ตรวจว่าแนวคิดมันย้ายมาได้จริง และเมื่อไหร่ที่เห็นช่องแก้ชดอย่าง `total\_main\_bar\_count` โพลมา ให้ถือว่ามันคือรายงานว่ามีบั๊กเชิงโครงสร้างที่ยังไม่ได้แก้ แล้วกวาดทั้งโปรเจกต์ทันที

แผนโครงสร้างของบ้าน 3 สกัดข้อมูลขาดไป 60% — หน้า 32 จาก 20 element กลายเป็น 32

เกิดอะไรขึ้น: การตรวจภาพแบบเต็มแผ่นของบ้าน 3 หน้า 31 (S-04) และ 32 (S-05) พบว่าหน้า 32 ขาดแนวคานแนวนอนทั้งแนวตามแถว D, ขาดรูปตัด B3X ที่แถว C, ขาดคานแนวตั้งสามตัว, ขาดคาน B1 ที่แนว B', ขาด BX และขาดจุดสังเกต S2 — จาก 20 element กลายเป็น 32 ส่วนหน้า 31 ต้องแก้ 7 จุด รวมถึงป้ายผิด (B3X→B3 สองที่, B2→B1, B3→B2) และการยึดกริดผิด (D1-C1→D1'-C1')

ต้นเหตุจริง: การสกัดข้อมูลรอบแรกไม่ได้กวาดทั้งแผ่นอย่างเป็นระบบ แต่บันทึก element ตามที่บังเอิญเจอ ทั้งแนวคานจึงหายไปโดยไม่มีใครสังเกต และการอ่านชื่อ mark ก็ไม่ได้ไปตรวจกับรูปตัดในสเปก หลายตัวจึงถูกตัดป้ายผิด

เสียหายอะไร: นับ element ขาดไป 60% บนแผนโครงสร้างหลัก เป็นช่องว่างการสกัดข้อมูลที่ใหญ่ที่สุดเท่าที่วัดได้ในโปรเจกต์นี้ และชิ้นส่วนที่ต้องใช้ต่อเรื่อง B' (คาน B1 ที่แนว B') ก็เป็นหนึ่งในตัวที่หายไปด้วย

แก้ยังไง: ตั้งมาตราส่วนพิกเซลที่เปรียบเทียบแล้ว (เส้นศูนย์กลางกริด x=1434.3px, กริด 2 x=1707.7px → 78.1 px/m, แนว E y=756) แล้วสกัดข้อมูลทั้งสองแผ่นใหม่ทั้งหมดโดยอิงมาตราส่วนนี้

กฎที่ได้: การสกัดข้อมูลจากผังโครงสร้าง ต้องกวาดอย่างเป็นระบบเทียบกับมาตราส่วนที่เปรียบเทียบแล้ว ไม่ใช่หยิบเก็บตามที่บังเอิญเจอ ให้ตรวจว่าทุกแนวแถวและแนวเสามีชิ้นส่วนต่อเนื่องครบ และตรวจทุก mark กับสเปกในรูปตัด ไม่ใช่อ่านแค่ป้าย

ประกาศว่าโมเดล "พร้อมใช้แล้ว" ทั้งที่ยังอ่านรูปไม่ได้เลย — ซึ่งเป็นงานทั้งหมดของโปรเจกต์

เกิดอะไรขึ้น: วันที่ 2026-07-21 มีการรายงานว่าโมเดลที่จนแล้วพร้อมใช้งาน แต่ GGUF ที่ได้มาอ่านได้แค่ข้อความ ส่วนที่อ่านรูป (mmproj) แตกไฟล์ออกมาไม่สำเร็จ ทั้งที่การอ่านแบบก่อสร้างคือจุดประสงค์เดียวของโปรเจกต์นี้

ต้นเหตุจริง: นิยามคำว่า "เสร็จ" ด้วยการที่ pipeline รันจบโดยไม่มี error แทนที่จะดูว่าของที่ส่งมอบทำหน้าที่หลักได้จริงไหม ไม่มีการทดสอบตั้งแต่ต้นจนจบ (ป้อนแบบจริง ได้คำตอบจริง) มาก่อนก่อนจะเคลมว่า "พร้อม"

เสียหายอะไร: เกิดสถานะ "เสร็จแล้ว" ปลอมๆ — เป็นที่มาของฉายา "DAY OF SHAME" ของทีม ที่ถูกอ้างถึงซ้ำๆ ในเซสชันต่อๆ มา ในฐานะความผิดพลาดที่กฎกระบวนการทั้งหมดหลังจากนั้นมีไว้เพื่อป้องกัน

แก้ยังง: เพิ่มการทดสอบแบบซ้อมบนเครื่องตัวเอง (Phase 0.3) — ดาวนี่โหลด mmproj ตัวทางมารมา รัน llama.cpp กับหน้าแบบจริงของบ้าน house01 แล้วยืนยันว่าอ่านข้อความไทย เส้นกริด และป้ายฐานรากได้ถูกต้อง ก่อนจะเคลมว่าพร้อม

กฎที่ได้: อย่าเพิ่งบอกว่าของพร้อมใช้ จนกว่าจะได้ลองใช้จริงตั้งแต่ต้นจนจบกับข้อมูลจริงที่ใช้งานจริง การไม่มี error ไม่ใช่หลักฐานว่ามันทำงานได้

บันทึกสันหลังคาหลักที่ไม่มีอยู่จริง เพราะแบบสองแผ่นดูเหมือนยืนยันตรงกัน ทั้งที่อ่านคนละเส้นที่หน้าตาคล้ายกัน

เกิดอะไรขึ้น: แบบ S-06 กับ A-07 ถูกถือว่ายืนยันตรงกันอย่างเป็นอิสระว่ามีสันหลังคาหลักบนก้อนอาคาร 11.4×11.4 แต่จริงๆ แล้วก่อนหน้านี้เป็นหลังคาปั้นหยาทรงสี่เหลี่ยมจัตุรัสที่มีเส้นสี่เส้นมาบรรจบกันที่จุดเดียว — ไม่มีสันเลย S-06 อ่านเส้นจันทันกลาง ส่วน A-07 อ่านเส้นตัด A-A สองเส้นที่หน้าตาคล้ายกันคนละอันซึ่งบังเอิญให้คำตอบตรงกัน

ต้นเหตุจริง: ยอมรับว่าแบบสองแผ่นตรงกันเท่ากับตรวจสอบแล้ว โดยไม่ได้เช็ควาทะทั้งสองแผ่นกำลังอ่านของชิ้นเดียวกันจริงไหม การอ่านผิดสองครั้งบนวัตถุคนละชิ้นที่ดันไปลงเอยที่คำตอบผิดอันเดียวกัน คือการยืนยันเท็จที่หนักที่สุดเท่าที่จะเป็นไปได้

เสียหายอะไร: มีการบันทึกชิ้นส่วนโครงสร้างที่ไม่มีอยู่จริง แถมยังได้คะแนนความมั่นใจสูงขึ้นจากการยืนยันเท็จนั้น — เป็นความผิดพลาดชนิดที่อันตรายที่สุด เพราะมันดูเหมือนผ่านการตรวจสอบมาแล้ว

แก้ยังง: อ่านแบบทั้งสองแผ่นใหม่เทียบกับรูปทรงจริง แล้วลบสันหลังคาออก บันทึกเป็นหลักการที่มีชื่อเรียกไว้ในไดอารี่

กฎที่ได้: การที่แบบสองแผ่นให้ข้อมูลตรงกัน จะนับเป็นการตรวจสอบได้ก็ต่อเมื่อพิสูจน์ได้ว่าทั้งสองแผ่นอ่านของชิ้นเดียวกัน การที่คำอ่านผิดสองอันตรงกันไม่ใช่การตรวจสอบ — ให้ระบุชื่อวัตถุจริงที่แต่ละแหล่งกำลังอ่านอยู่ก่อนเสมอ ก่อนจะถือว่ามันเป็นแหล่งอิสระต่อกัน

sanity check ของ GGUF นั้นโมเดลที่ไม่ได้ tune เลยก็ผ่านได้สบาย

เกิดอะไรขึ้น: export_gguf.py บรรทัด 107 กันการ merge สัมแบบเสียบๆ ด้วยการเช็คค่า output ขึ้นต้นด้วย '{' และมีคำใดคำหนึ่งใน "png"/"views"/"pattern"/"doc_page" มันขึ้นเครื่องหมายถูกสีเขียวเมื่อวันที่ 24 ก.ค. และทุกคนก็เชื่อมาตลอด แต่ตัว prompt เองสั่งให้โมเดลพ่นคำว่า "views" และ "pattern" อยู่แล้ว โมเดลไหนที่ทำตาม prompt เป็นก็ผ่านหมด

ต้นเหตุจริง: ตัวตรวจไปเช็คค่าที่ prompt ป้อนให้โมเดลเอง ไม่ใช่ค่าที่มีแต่การเทรนเท่านั้นที่ผลิตได้ มันถูกเขียนขึ้นโดยไม่ได้ถามตัวเองว่า 'ถ้าของมันพังจริงๆ ตัวนี้จะร้องไห้'

เสียหายอะไร: ความมั่นใจจอมปลอมใน GGUF ของ t01 นานถึง 5 วัน ความล้มเหลวชนิดที่ตัวกันนี้มิไว้ดักกลับหลุดไปโดยไม่มีใครรู้ ส่วนตัวกันของ t02 ที่เข้มกว่า (เทียบชื่อ key จริง) จับบักตัวเดียวกันได้ทันที ความต่างระหว่าง 'จับได้' กับ 'จับไม่ได้' คือคุณภาพของตัวกันล้วนๆ

แก้ยังง: ยังไม่แก้ — เป็นงานลำดับ 2 ของเซสชันถัดไป: เขียนตัวกันใหม่ให้เทียบชื่อ key จริงของ gold (png/doc_page/discipline/sheet_code) แบบเดียวกับตัวกันของ t02

กฎที่ได้: ตัวกันทุกตัวต้องเอาไปทดสอบกับ input ที่รู้อยู่แล้วว่าผิด ก่อนจะไว้ใจมัน ห้ามตรวจสอบด้วยข้อความที่ปรากฏอยู่ใน prompt เด็ดขาด — ให้ตรวจเฉพาะข้อความที่มีอยู่ในคำตอบสำหรับเทรนเท่านั้น ตัวตรวจที่ไม่มีทางล้มได้แยกว่าไม่มีตัวตรวจเลย เพราะมันผลิตความมั่นใจปลอมๆ ขึ้นมา

ประตูดตรวจว่าโมเดล tune แล้วหรือยังใน export_gguf.py ไปตรวจค่าที่ prompt ป้อนให้เอง โมเดลที่ไม่ได้ tune จึงผ่านฉลุย

เกิดอะไรขึ้น: export_gguf.py บรรทัด 107 ตัดสินว่าโมเดล 'tune แล้ว' ถ้าคำตอบขึ้นต้นด้วย { และมีคำใดคำหนึ่งใน "png", "views", "pattern", "doc_page" แต่ตัว prompt เองสั่งให้โมเดลพ่น "views" และ "pattern" อยู่แล้ว โมเดลไหนที่ทำตามคำสั่งเป็นก็ผ่านหมด ประตุนี้ขึ้นเขียวตั้งแต่ 24 ก.ค. และทุกคนก็เชื่อมาตลอด

ต้นเหตุจริง: ตัวตรวจถูกเขียนจากมุมมองว่า 'คำตอบที่ดีหน้าตาเป็นยังง' ไม่ใช่จากมุมมองว่า 'อะไรที่มีแต่โมเดลที่ tune แล้วเท่านั้นจะผลิตได้' ค่าที่แยกแยะได้จริง (png/doc_page) มีอยู่แค่ในข้อมูลจริงและไม่เคยถูกเอ่ยใน prompt เลย แต่การใช้ OR กับ 4 คำทำให้ค่าที่ prompt ป้อนให้ทำให้เสียใจผ่านได้

เสียหายอะไร: ไฟเขียวปลอมนี้บังความล้มเหลวเสียบๆ ของ merge_and_unload ไว้ถึง 5 วัน และทำให้กล้าเอา GGUF ไปใช้กับงานสกัดข้อมูลจริง ส่วนประตูของ t02 ที่เข้มกว่า (เทียบชื่อ key จริง) จับบักตัวเดียวกันได้ทันที — บักเดียวกัน ต่างกันแค่คุณภาพของประตู

แก้ยังง: วางแผนแก้แบบไม่มีค่าใช้จ่ายไว้แล้ว: เปลี่ยนประตูให้เทียบชื่อ key จริงของข้อมูลจริงแบบที่ t02 ทำ โดยตรวจเฉพาะค่าที่ไม่มีอยู่ใน prompt

กฎที่ได้: ออกแบบประตูตรวจสอบทุกตัวด้วยการถามว่า 'ถ้าของมันพัง ตัวนี้จะจับได้ไหม' ตัวตรวจที่ของพังๆ ผ่านได้ แยกว่าไม่มีตัวตรวจเลย เพราะมันผลิตความมั่นใจปลอมๆ ขึ้นมา

Format checker reported ALL CHECKS PASS while 345 files had a misspelled discipline value (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: tools/check_format.py returned "ALL CHECKS PASS" across the whole dataset even though 345 files carried `architecture` instead of the spec's `architectural`, plus 16 more out-of-vocabulary values. In total 485 defects across 423 files in all 49 houses were sitting undetected behind a green validator.

ต้นเหตุจริง: The schema spec was edited 3 times on 08-21 but the validator that enforces it was never updated in the same change. Spec and checker are two separate sources of truth with nothing binding them together, so the checker silently kept validating an obsolete contract.

เสียหายอะไร: 485 defects / 423 files / 49 houses shipped as "validated" training data. Every downstream consumer that filters on `discipline` was silently dropping or mis-routing those files. Required a full 485-point repair sweep across 4,366 files.

แก้ยังไง: Added 5 new gates to check_format.py (discipline vocabulary, `source\_image`, notes one-off key, structural roof-framing must be `plan`, gridmaster missing 3 arrays) and batch-repaired all 485 points; all 4,366 files re-parsed clean.

กฎที่ได้: A schema edit is not done until the validator that enforces it is edited in the same commit. Treat "validator passes" as meaningless unless you can point to the gate that would have failed the specific defect class you are worried about.

sanitize_elements() silently deleted every element with a negative level, wiping ground beams from the BOQ (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `sanitize\_elements()` in `worker.py` used the regex `\_NEG\_DIM\_KEYS = r"(\_mm|\_m|\_cm)\$"` to treat any key ending in a unit suffix as 'garbage' if its value was negative. But negative values are valid for `level\_m` (ground beams below datum, written on Thai drawings as "GB1(-0.50)"), `pos\_m` (grid lines before origin) and `level\_step\_mm` (stepped precast slabs for bathrooms). Whole elements were dropped before reaching BOQ, and the warning blamed the model ('generation หลุด') even though the model had answered correctly. Confirmed on real GT: house 07บ้าน_ใหญ่_2ชั้น_01 page 35 lost 3 of 11 elements.

ต้นเหตุจริง: A blanket denylist regex on key `*suffix*` rather than an allowlist of dimensions that genuinely cannot be negative. The validator encoded an assumption ('a measurement is never negative') that is false for elevation/offset fields, and the failure was silent-with-wrong-attribution because the warning text blamed model generation instead of the filter.

เสียหายอะไร: Ground beams disappeared entirely from BOQ on every affected house — a missing structural element in a real construction quantity takeoff. Also caused misdirected debugging effort, since the warning pointed at the model.

แก้ยังไง: Replaced the suffix denylist with an explicit allowlist of cross-section dimensions that truly cannot be negative (`width|height|thickness|depth|dia|spacing|length|cover`), plus a regression test. Verified by running old and new regex side-by-side: `level\_m`/`level\_step\_mm`/`pos\_m` flip from dropped to kept while `width\_mm`/`span\_length\_m`/`dia\_mm`/`depth\_mm` still drop.

กฎที่ได้: Validate data with an allowlist of fields you have positively confirmed a constraint for, never a denylist matched on name patterns. And when a filter discards data, its warning must name the filter as the actor — never blame an upstream component you did not verify.

สูง 2026-07-05

Eleven pages were extracted at low confidence and shipped into the dataset unverified (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Eleven pages were flagged as needing someone to zoom into the image and re-verify before real use: page 19 view2 (SI/SO/ST symbol meanings unknown), page 21 (B5/B5X top/bottom rebar split), 11, 14, 05, 06, 09, 35, 36 (possible wire-size mismatch between BTU 12,000 and 18,000), and 51/53 (BOQ row pairing uncertain).

ต้นเหตุจริง: Extraction proceeded to completion regardless of confidence, with low confidence recorded as metadata rather than acting as a gate. Combined with the absence of a zoom tool, uncertainty was the expected outcome rather than an exception.

เสียหายอะไร: Roughly a fifth of the pages in the house carry data that the team itself says should not be used as ground truth yet — but the files sit in the same folder, in the same format, as verified data. The verification pass was still outstanding at the end of the next day.

แก้ยังไม: Not fixed — listed as a pending item on both 2026-07-05 and 2026-07-06.

กฎที่ได้: Low-confidence records must be physically separated (a quarantine folder or an explicit status field consumers filter on), not just annotated. Anything below the confidence threshold cannot be allowed to sit in the same directory as accepted ground truth.

สูง 2026-07-06

Three of four dummy gridlines were written into the grid master at only 0.5-0.6 confidence and never verified (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The new grid master file recorded 4 dummy gridlines. Only 1' was high confidence (0.85, from actually seeing the full-height line on page 06). 3', A' and A'' were entered at 0.5-0.6 confidence, inferred from Gen 2's 'unnamed_extension' notes plus the positions of beams B4X/B3X/B5X — the printed dimension positions are real, but it was never confirmed that an actual beam runs at those positions.

ต้นเหตุจริง: Low-confidence inferences were promoted into a master reference file that every other page is meant to cite. A master file implies authority, so uncertainty at the source silently becomes certainty at every consumer.

เสียหายอะไร: Three gridlines of unverified existence became the canonical grid reference for the whole house, and page 19 view2 in round t2 was then extracted referencing them. Any error propagates to every page that cites the master.

แก้ยังไม: Not fixed. Flagged as needing comparison against the page 19/20 beam plan when those are redone; confidence scores were kept honest in the file rather than inflated.

กฎที่ได้: A master/reference file must contain only verified entries. Keep unverified candidates in a separate staging list until confirmed — the moment an inference is written into an authoritative file, its confidence score stops being read by anyone downstream.

สูง 2026-07-06

A full day of Label Studio work ended with nothing actually imported, published or reviewable (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The script, three XML configs and the task JSON (33 elements tasks, 1 material_list, 22 single) were all created and regenerated several times, but the diary records they were 'not yet imported into Label Studio Cloud for real' — no project created, nothing published, no collaborators invited. The receiving-side script label-studio-import-*.js to convert exports back into annotated/*.json does not exist at all.

ต้นเหตุจริง: The work was built outward from the generator without ever completing one thin end-to-end slice first. Each of the day's five fix rounds was triggered by a failure that a single early end-to-end attempt would have surfaced at once.

เสียหายอะไร: A whole day of tooling work produced zero reviewed records. The round trip is not just unfinished but unproven — the export→annotated direction has no code, so it is unknown whether reviewed data can even be got back out.

แก้ยังง: Not fixed — carried as pending items #1, #8 and #9 (import and confirm, write the import-side script, create and publish the real project).

กฎที่ได้: Build one thin end-to-end slice first — one task, imported, reviewed, exported back to disk — before generating dozens of tasks or polishing layout. Until the round trip is proven, none of the generator work is known to be usable.

สูง 2026-07-07

The 'overhang past the main grid' was assumed to be one thing; it was three unrelated systems (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Confusion about dimensions extending past the main grid was resolved by finding they are three genuinely separate systems: 1.50m ground-floor room extension (real dummy grid 3'/A'), 0.60/1.10m tie-beam corner, and 1.30m roof eave. Confidence in dummy grid A'' was lowered — an agent suspects it is not a grid line at all, just a ground-leveling area.

ต้นเหตุจริง: Several distinct offsets sharing the visual pattern 'a dimension beyond the last grid line' were collapsed into a single explanation because the grid master had no field for what an offset physically is.

เสียหายอะไร: A grid master carrying a possibly-fictitious grid line (A'') that would misplace every element referenced against it; repeated confusion across earlier rounds.

แก้ยังง: Grid master updated to model the three systems separately; confidence raised on 1'3'/A' and lowered on A''.

กฎที่ได้: Do not collapse similar-looking geometry into one explanation — record what each offset physically is, and lower confidence rather than keeping an unverified grid line at full confidence.

The t3 generator output was declared verified by comparing the UI against the generated task JSON — the same file that carried the bug (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: After running label-studio-tasks-makham.js on t3, the check performed was 'tested and confirmed page 19 (footing_plan + beam_plan) matched what Makham actually saw in Label Studio'. It passed. The next day a more thorough check found the generator had never been updated for the Gen 4 field shapes, so every beam's grid_ref and every main-rebar value was empty in that same output — the passing check had compared the UI to the broken file and found them consistent with each other.

ต้นเหตุจริง: Verification methodology compared two artifacts on the same side of the bug (generated JSON ↔ rendered UI) instead of comparing the end of the chain back to its source (raw JSON ↔ rendered UI). Consistency between two downstream artifacts proves nothing about correctness.

เสียหายอะไร: A whole day of confidence in the t3 Label Studio pipeline was false. If the t3 task set had already been imported to Label Studio Cloud, annotators would have been labeling beams with no grid reference and no rebar data, with nothing on screen saying data was missing.

แก้ยังไง: Fixed the next day: the generator was taught both the new (grid_ref_start/end/span_length_m, main_bar.top/bottom) and old field shapes, the XML configs gained the matching boxes, and task JSON was regenerated. A three-way source_image / disk / decoded-URL check was added afterwards.

กฎที่ได้: Verify against the source, not against an intermediate artifact. 'The UI shows what the file says' is not a test — pick a value you can see on the original drawing and trace it all the way to the screen.

เอาระยะสองช่วงที่ติดกันมาบวกกันเป็นชายคายื่น 1.50 ม. ที่ไม่มีอยู่จริง

เกิดอะไรขึ้น: ในหน้า page06 ของบ้านหลัง 2 เลข '0.60' กับ '0.90' ในแถวเส้นบอกระยะของแนว F ถูกอ่านเป็นชายคายื่นยาวต่อเนื่อง 1.50 ม. พอซูมเข้าไปดูถึงเห็นว่าเป็นคนละจุดกัน (0.60 = ด้านนอก, 0.90 = พ้นกันด้านใน) แค่มั้จตุรรมกันตรงเส้นกริด F เท่านั้น

ต้นเหตุจริง: เอาตัวเลขที่อยู่ติดกันในเส้นบอกระยะมาบวกกันโดยไม่เช็คว่าแต่ละตัววัดไปจบที่อะไร รูปแบบการอ่านผิดแบบเดียวกันนี้เคยทำให้ได้ค่า '3' ผิดในบ้านหลัง 1 มาแล้ว

เสียหายอะไร: เกิดแถบพื้นที่ 1.50 ม. ที่ถูกสร้างขึ้นมาลอยๆ ในผังพื้น ทำให้ต้องกำหนดขอบเขตห้องซักผ้า ห้องครัว การขอยห้องนอน 1 และห้องน้ำ 1 ใหม่หมด แถมยอหน้าค่าเดือนทั้งยอหน้าที่สรุปว่า "ไม่ต้องมี dummy grid" ก็ต้องเขียนใหม่เพราะผิด

แก้ยังไง: แยกออกเป็น 'F-F' x 1-2' (ห้องซักผ้า) กับ 'F'-E x 1-2' (ห้องครัว), ตรึงจุดแบ่งห้องนอน 1 ไว้ที่ 3' (6.4+1.30=7.7 พอตีเป๊ะ) แล้วเขียนยอหน้าค่าเดือนใหม่

กฎที่ได้: ห้ามเอาตัวเลขที่อยู่ติดกันในเส้นบอกระยะมาบวกกันโดยไม่ยืนยันก่อนว่าแต่ละตัวไปจบที่อะไร จุดปลายที่ใช้ร่วมกันทำให้ระยะสองค่าดูเหมือนต่อเนื่องกันทั้งที่วัดคนละของ

สูง 2026-07-20

เอาสเกลที่คาลิเบรตจากหน้า 31/32 มาใช้กับหน้า 33 โดยไม่เช็คจุดอ้างอิงของหน้า 33 เอง

เกิดอะไรขึ้น: ข้อสรุปที่ว่าแถบ D-C ในหน้า 33 ไม่มีส่วนต่อขยาย 1'/3' มาจากการเอาค่าคาลิเบรตของหน้า 31/32 มาใช้ซ้ำ ผู้ใช้แก้ไขให้ตรงๆ ว่าหน้า 33 มีส่วนต่อขยาย 1'/3' จริง และต้องเพิ่มคานสั้นอีก 4 ตัว (D1'-D1, C1'-C1, D3-D3', C3-C3')

ต้นเหตุจริง: การคาลิเบรตเป็นเรื่องเฉพาะของแต่ละแผ่นแบบ เพราะขึ้นกับจุดอ้างอิงและสเกลของแผ่นนั่นเอง แต่กลับถูกยกไปใช้ข้ามแผ่นเหมือนเป็นค่าคงที่ของทั้งโปรเจกต์

เสียหายอะไร: มีข้อสรุปเชิงลบที่ผิดถูกเขียนลงไป schema_note และ warnings ของไฟล์ คานหายไป 4 ตัว และต้องให้ผู้ใช้เป็นคนจับได้เอง

แก้ยังไง: ตรวจหน้า 33 ใหม่โดยเทียบกับจุดอ้างอิงของหน้านั้นเอง เพิ่มคานทั้ง 4 ตัวด้วย confidence 1 และแก้ schema_note/warnings ที่เคยระบุตรงกันข้าม

กฎที่ได้: หาค่าสเกลพิกเซลและจุดอ้างอิงใหม่ทุกแผ่นแบบ ห้ามยกค่าคาลิเบรตข้ามแผ่นเด็ดขาด — และเวลาข้อสรุปถูกกลับด้าน ต้องแก้ note กับ warnings ที่เคยยืนยันข้อสรุปนั้นด้วย ไม่ใช่แก้แค่ตัวข้อมูล

สูง 2026-07-24

Training scripts had never been read line-by-line against the real dataset's actual numbers (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The MAX_LENGTH bug had existed since before house 05 was added and survived every prior round, because nobody had ever compared the script's constants against the real measured properties of the dataset.

ต้นเหตุจริง: Script review consisted of skimming for obvious errors rather than cross-checking each configured limit against a computed statistic of the data it would process.

เสียหายอะไร: A latent answer-truncating bug shipped through at least one full paid tuning round undetected.

แก้ยังไง: Made Phase 0.4 a mandatory step: re-read the scripts seriously against real dataset numbers before every rental.

กฎที่ได้: Review configuration by computing the corresponding statistic from the real data and comparing, not by reading the number and judging whether it 'looks reasonable'.

สูง 2026-07-28

House 07 S-04: a position was eyeball-estimated at ~2.00 m when the drawing prints 1.50 m, and a dimension was measured from the wrong grid row (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The second CS2's distance from grid 3 had been recorded as an estimated ~2.00 m; the drawing prints 1.50 m. Separately the 1.60 m dimension at bottom right is measured from row B, not row A — which, once corrected, matches S-02's GB1X exactly.

ต้นเหตุจริง: An estimate was recorded in a field that reads as a measurement, with no marker distinguishing the two. The datum row for a dimension was assumed rather than traced to its extension line.

เสียหายอะไร: A 0.50 m position error (33% off) plus a wrong datum row; both feed element positions and grid resolution. The datum error is what had made S-02 and S-04 appear to disagree.

แก้ยังไง: Positions locked using the chains actually printed on the sheet; the corrected datum reconciled S-04 with S-02.

กฎที่ได้: Never write an estimate into a measurement field — if the chain isn't printed, mark it unresolved. Always trace a dimension back to its extension line to confirm which grid line is the datum.

สูง 2026-07-29

Phase 10 'GGUF passed' only tested that it emitted JSON and didn't crash (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: t01's Phase 10 acceptance test on 25 Jul checked (a) does it answer in JSON — passed, (b) does it hang or crash — passed. It never checked whether the content was correct. The doc even warned 'don't read this as proven complete', but over time everyone remembered only 'Phase 10 passed'.

ต้นเหตุจริง: Liveness was tested, correctness was not. And the caveat lived in prose next to a pass verdict, so the verdict survived in memory while the caveat did not.

เสียหายอะไร: A known-incomplete test hardened into a team belief that the GGUF was validated, which delayed discovery of the merge bug by days.

แก้ยังไง: not fixed — Phase 10 criteria still need a content-correctness gate added

กฎที่ได้: A test that only proves the thing runs must be named as such ('smoke test'), never recorded as a pass. Any caveat attached to a pass verdict must be encoded in the verdict itself (FAIL or PARTIAL), because prose caveats evaporate and the verdict does not.

สูง 2026-07-29

Phase 10 'GGUF verified' only tested that it emits JSON and does not crash — content was never checked (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The 07-25 Phase 10 GGUF test passed on two criteria: output is JSON, and no hang/crash.

Whether the content was correct was never tested. The document itself carried the caveat 'don't read this as proven complete', but over time everyone remembered only 'Phase 10 passed'.

ต้นเหตุจริง: Acceptance criteria measured liveness rather than correctness, and the written caveat eroded because it lived in prose next to a green checkmark instead of in the pass/fail criteria themselves.

เสียหายอะไร: A broken artifact carried a 'verified' label for four days and was used for real extraction work on house 01, producing bad data.

แก้ยังง: Reframed for future rounds: content correctness must be part of the GGUF acceptance test, using ground-truth key comparison.

กฎที่ได้: Acceptance criteria must test the capability you need, not that the process ran. And never park a caveat in prose beside a green result — encode it in the pass condition, or it will be forgotten.

สูง 2026-08-02

ประกาศ 'ผ่านทุกด้าน' ทั้ง 11 บ้าน แล้วการตรวจอิสระเจอ id หายไป ~500 ตัว, ฟอรัมแตกเป็น 5 แบบ และเนื้อหาถอยหลังซ่อนอยู่ข้างล่าง

เกิดอะไรขึ้น: รอบ 4 ปิดงานด้วย check_format.py รายงาน ALL CHECKS PASS ครบทั้ง 11 บ้าน และไฟล์ในโพลเดอร์ทำงานกับโพลเดอร์ raw ต่างกัน 0 ไฟล์ พอรอบ 5 ทำการตรวจอิสระที่ลงใจไม่สนตัวเช็คตัวนั้น แล้วเทียบชนิดข้อมูล JSON ของทุก key ใน 2,366 ไฟล์แทน — เจอ element ~500 ตัวไม่มี element_id, ไฟล์ BOQ ของบ้าน 04 อยู่ผิด pattern, ตารางราคาแบบ series ถูกเก็บไว้ 5 แบบต่างกัน, หน้าเครดิตแตกเป็น 4 pattern, ค่าที่ควรเป็นตัวเลขกลับเป็น string หรือ array และเจอตัวเลขราคาของบ้าน 05 ที่ถอยหลังไปจากของเดิมจริงๆ ผลคือตัวเช็คขยายจาก 14 เป็น 19 ข้อ

ต้นเหตุจริง: ตัวเช็คบังคับได้แค่กฎที่มีคนเขียนไว้แล้วเท่านั้น การเห็นไฟเขียวถูกอ่านว่า "ข้อมูลถูกต้อง" ทั้งที่จริงมันแปลได้แค่ "กฎ 14 ข้อที่รู้จักผ่านหมด" ส่วนความถูกต้องของชนิดข้อมูลและ id ไม่ได้อยู่ใน 14 ข้อนั้นเลย

เสียหายอะไร: ปิดงานทั้งเซสชันด้วยความเชื่อว่าชุดข้อมูลสะอาดและล็อกแล้ว ทุกอย่างในรอบ 5 เจอจะไหลเข้าไปเป็น label สำหรับเทรนโดยไม่มีใครรู้ รวมถึงตัวเลขราคาที่ผิด ซึ่งเคยมีบันทึกไว้ว่าแก้ไปแล้วด้วยซ้ำ

แก้ยังง: รันการตรวจแบบเทียบชนิดข้อมูลที่แยกจากตัวเช็คเดิม แล้วเอาสิ่งที่เจอกลับมาใส่เป็นกฎใหม่ 5 ข้อ (14 → 19)

กฎที่ได้: ตัวเช็คที่ผ่านคือหลักฐานเรื่องตัวเช็ค ไม่ใช่หลักฐานเรื่องข้อมูล ทุกครั้งที่จะเคลมว่า "ล็อกและตรวจสอบแล้ว" ต้องมีการตรวจอย่างน้อยหนึ่งครั้งที่ใช้วิธีคนละแบบกับเครื่องมือที่ให้คำเคลมนั้นมา

สูง 2026-08-21

Wrongly excluded site_plan/side_profile as grid-line sources, reasoning elevations have no grid axis — and made the same class of assumption twice (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The t03 pass design initially excluded site_plan and side_profile from the gridline subtask on the reasoning that elevations have no top-down grid axis. Makham produced a real elevation sheet (A-05, รูปด้าน 1/2) printing grid markers 1/2/3 and D/C/B/A with full dimension chains, same as a plan sheet. The exclusion was reverted. Later the same session, the diary notes this was the second time assuming side_profile/elevations were low-value proved wrong — they are the only place z-levels are printed at all.

ต้นเหตุจริง: A design decision was made from a mental model of what an elevation drawing contains rather than from opening real sheets in the dataset. The same untested assumption then recurred in a different form within one session.

เสียหายอะไร: Would have permanently excluded a genuine grid-line source from t03's gridline subtask, and (second occurrence) would have left the entire vertical axis unextractable. Both caught by Makham, not by any check.

แก้ยังไง: Reverted the exclusion; later added z_levels[], dimension_chains[] for the z axis, and unassigned_dimensions[] to the schema §4 after the second occurrence.

กฎที่ได้: Never exclude a drawing pattern from a data source based on what you assume it contains — open three real sheets of that pattern first; and when the same assumption class is disproven twice, treat it as a rule ('elevations carry data plans do not'), not two separate corrections.

สูง 2026-08-22

Answered a model-capability question from memory and got it wrong (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Makham asked whether Qwen could output pixel positions for beams to cross-check the grid master. The first answer was "not worth it", citing an in-repo paper doc (FLOORPLANVLM.md) that said 300K pixel-aligned samples plus RL were needed. Makham insisted on real web research, which found Qwen3-VL ships native 2D grounding (bbox in a 0-1000 normalized scale) in the base model — no fine-tuning at that scale required.

ต้นเหตุจริง: A single in-repo research doc was treated as the current state of the art. It compared a different model family and was never re-checked against vendor docs; capability claims about fast-moving VLMs go stale in weeks.

เสียหายอะไร: A genuinely viable QA direction (bbox↔gridmaster cross-check) was almost closed off entirely. The correct answer only surfaced because the user pushed back and demanded live research.

แก้ยังไง: Ran real web research, found native grounding + the GroundCount detection-then-VLM pattern, and wrote the corrected finding into tune_ai/t03/research_beam_grounding_vs_gridmaster.md.

กฎที่ได้: Never answer a "can model X do Y" question from memory or from one cached in-repo doc. Check the vendor's current model card/docs live before declaring a capability impossible.

สูง 2026-08-25

MAX_PIXELS 7,680 + MAX_LENGTH 47,104 written into the training script before the mandatory 5-step VRAM check was run (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Both values were changed in train_t03.py and documented as 'ทำจริงแล้ว', cutting free VRAM from 7.5GB to 4.0GB — while the diary itself records the rule that a short 5-step run must confirm the config before any full run, because a previous run OOM'd with 470MB left.

ต้นเหตุจริง: The change was made on a spreadsheet estimate of peak VRAM (90.9GB), not on a measured run. The estimate was produced by the same reasoning that had already been wrong once about image capacity.

เสียหายอะไร: Next full training round can OOM hours in and burn the round's GPU cost, at a moment when only \$8.91 credit remains.

แก้ยังไม: not fixed — flagged in the diary as ⚠️ 'ต้องรันทดสอบสั้น 5 step ยืนยันก่อนรันเต็มทุกครั้ง', not yet executed

กฎที่ได้: A config edit is not 'done' until the smoke test that validates it has run. Writing the value and validating the value are two separate tasks; only the second one prevents the burn.

สูง 2026-08-25

Only footings were scored against a true count — beams and columns shipped with eyeball verification only, yet the summary claims all three work (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The results table gives footings 14/14 and 12/12 against real counts, but beams are reported as bare counts ('17 แถบ', '19 แถบ') with no denominator and the note 'คลุมโครงคานเกือบครบ'. Columns are reported as 14 and 13 with no stated truth either (the 13 was later found to contain a duplicate). The day's summary nonetheless states the tool 'มาร์ค เสา/คาน/ฐานราก ได้จริง'.

ต้นเหตุจริง: Footings are countable discrete icons so a ground truth was easy; beams are variable-length runs with no obvious unit, so no ground truth was established and the gap was filled with visual inspection.

เสียหายอะไร: Beam detection accuracy is unknown — which matters most, because beams are the exact subtask (plan_beam 0%) the whole tool was built to help. A tool with unmeasured recall cannot be used as the 'did the AI miss something' cross-check proposed in section 6.2(ข).

แก้ยังไม: not fixed — no beam ground-truth count was produced

กฎที่ได้: The subtask you built the tool for is the one that needs a number, not the one that was easy to count. 'เกือบครบ' is not a measurement.

สูง 2026-08-25

The 08-24 roof_plan→plan fix was incomplete; 7 more files still hid the missing-roof-beam BOQ bug (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A fix made the previous day to retag `roof\_plan` as `plan` (which closes a bug where roof beams vanish from the BOQ) had only covered part of the dataset. Seven more files still carried the wrong pattern and had to be fixed again the next day.

ต้นเหตุจริง: The 08-24 pass fixed only the 11 houses being worked on that day rather than sweeping the whole dataset for the same defect class, so the same bug survived in the houses that were not in scope.

เสียหายอะไร: Roof beams were silently absent from the BOQ for those 7 files for an extra day; the same work had to be re-done as part of a second, larger sweep.

แก้ยังไง: Retagged the remaining 7 files `roof\_plan`→`plan` as part of the 485-point normalization sweep.

กฎที่ได้: When you find a defect class, grep the whole dataset for it and fix every instance in that pass. Fixing only the files in front of you guarantees the same bug comes back on the next file you open.

สูง 2026-08-25

A 22px "+" template poisoned the bank and produced 636 phantom footings on a page with none (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A harvested template from บ้าน 11 was a 22px "+" crosshair. It matched every line intersection on the drawing, ballooning footing detections to 636 on a page containing no footings at all.

ต้นเหตุจริง: The harvest step accepted any repeated icon-sized contour, and a crosshair is a universal drawing primitive rather than a distinguishing symbol. There was no minimum-size or distinctiveness gate on templates entering the bank.

เสียหายอะไร: Completely corrupted counts on affected pages; would have poisoned every measurement built on the bank if not caught.

แก้ยังไง: Removed the template and added a gate in `load\_templates`: skip any template under 28px with a warning.

กฎที่ได้: Any template smaller than a few dozen pixels is probably a drawing primitive, not a symbol. Gate template admission on minimum size and distinctiveness, and test every new template against a page that should score zero.

สูง 2026-08-25

Two tightly-cropped templates scored garbage as high as real matches, making a threshold impossible (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The solid-shaded 31px templates from บ้าน 05/33 scored 0.87–1.00 on genuine matches in their own house but garbage on other houses reached 0.90 — the score ranges overlap, so no threshold can separate them. A per-template table showed those 2 templates alone emitted 224 junk detections while the other 9 emitted zero.

ต้นเหตุจริง: The templates were cropped tight to the icon, discarding the surrounding white space. Real icons float alone on white; Thai text characters sit crowded together — the surrounding context is the actual discriminator, and cropping it away removed the only signal that separates them.

เสียหายอะไร: 224 false detections across the dataset, and an unfixable threshold until the root cause was addressed.

แก้ยังไง: Diagnosed with a per-template score table before changing anything, then fixed at the source: re-cropped with white margin around the icon (68px instead of 31px), removed the duplicate (05/33 are the same symbol with identical scores), and raised FOOTING_THRESH 0.60→0.72.

กฎที่ได้: When a threshold cannot separate signal from noise, the template is wrong — do not tune the threshold. Diagnose per-template before touching a global constant, and crop templates with their surrounding context, since context is often the real discriminator.

สูง 2026-08-25

Vast.ai credit on record (\$8.91) is not enough to finish even one H200 run (\$17.45) (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The full-resolution training path (MAX_PIXELS 16,384) requires an H200 at \$3.293/hr ≈ \$17.45 per 3-epoch run, but the account has \$8.91 of credit. A run started against that balance would die mid-training.

ต้นเหตุจริง: The remaining balance was never checked as part of planning a configuration change that requires a more expensive card.

เสียหายอะไร: The 16,384 path is completely blocked until money goes in. Had this not been checked first, a run would have been started and killed partway, burning both the spent credit and the time.

แก้ยังไง: Stated as a hard blocker in plain language to Makham immediately rather than filed as a to-do (per the Day of Shame lesson 3). Nothing was rented; no money spent this session.

กฎที่ได้: Check the account balance against the full cost of a planned run before proposing the configuration that requires it, and state a funding blocker out loud rather than burying it in a to-do list.

สูง 2026-08-25

MAX_PIXELS/MAX_LENGTH raised on an arithmetic estimate with the mandatory TEST_STEPS=5 confirmation not yet run (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: MAX_PIXELS was raised 5120→7680 and MAX_LENGTH 32768→47104, narrowing the VRAM margin from 7.5 GB to 4.0 GB. The 90.9 GB peak is an arithmetic estimate, not a measured value, and the mandatory TEST_STEPS=5 confirmation run was not performed.

ต้นเหตุจริง: The confirmation step requires a GPU rental that was not done in this session, so a configuration change shipped ahead of its own verification gate.

เสียหายอะไร: If the estimate is wrong, the next full run OOMs — the previous round OOM'd with only 470 MiB free, so a 4.0 GB estimated margin is not comfortable. A failed full run costs the whole rental.

แก้ยังไง: Documented as a mandatory pre-run gate (rule_of_tune Rule 4) with an explicit fallback order: cut gridline images 4→3 first (peak 89.2), then roll MAX_PIXELS back — never lower MAX_LENGTH alone without lowering MAX_PIXELS, which truncates labels mid-JSON (the t01 \$0.4 bug class).

กฎที่ได้: A memory-bound config change is not done until a short run confirms the peak on real hardware. Ship the change with the fallback order written down, and treat the confirmation run as a blocking gate, not a to-do.

สูง 2026-08-29

New span-scaling rule shipped into the schema without ever verifying a VLM can measure pixels accurately (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A new rule was adopted: unprinted span/dummy dimensions are measured in pixels against two grid lines of known pos_m and converted by proportion, with a new `span\_source: scaled\_from\_grid` enum value. It shipped across the schema, plan template + 4 rendered files, gridline, pass3, _common, and Constant's schema.js SPAN_SOURCE_VALUES — before anyone measured whether a VLM can read pixel distances accurately at all.

ต้นเหตุจริง: An appealing mechanism was designed and propagated across 8+ files based on it being conceptually correct, without a cheap feasibility probe first. The prompt even had to be hardened against the model faking measurements by pasting the label without the numbers.

เสียหายอะไร: If VLMs cannot measure pixels reliably, the entire enum value, prompt sections, worked examples and cross-repo schema change are wasted, and the model will have been trained to emit confidently-wrong scaled values.

แก้ยังไง: Not validated. Mitigations shipped: prompt forces the pixel numbers into the flag (`scaled\_between:1,3 P:380 U:142` — no P/U means it wasn't measured, so `unresolved` is required); `--hide-grid-lines` was added to infer_house_t03.py (eval only) to hide known pos_m on val pages and measure |measured – GT| directly without building a new dataset. Explicitly flagged that GPU round 1 must compare every scaled_from_grid value against GT before anyone trusts it.

กฎที่ได้: Probe a new model capability on a handful of examples before propagating a schema change across two repos. Design the falsification test at the same time as the mechanism — and force the model to show its work so a fabricated answer is detectable.

สูง 2026-08-31

Assumed a dry-run needs no HF token; push_to_hub actually calls init_hf_repo in SFTTrainer.__init__ (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The dry-run died with 401 in SFTTrainer.__init__ because push_to_hub=True creates the hub repo at construction time, not at the first save_steps. The standing assumption that a dry-run does not need a token was wrong.

ต้นเหตุจริง: The library performs a network/auth operation earlier in the lifecycle than the mental model assumed.

เสียหายอะไร: Without this discovery, a real run launched without HF_TOKEN would have died in the first second — or worse, been assumed safe until hour 15.

แก้ยังง: Dry-run now disables push automatically; documented that a real run without HF_TOKEN fails instantly rather than late.

กฎที่ได้: Verify when a library actually performs auth/network side effects rather than assuming they happen at first use — construction-time side effects are common and change what a dry-run must cover.

สูง 2026-09-01

Destrier (t05) is the surviving model but has never been run end-to-end on a real GPU (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Destrier is trained, souped and uploaded to HuggingFace, but its only recall measurement is a single ~43% run under forgiving name matching, and the full pipeline has never been executed once on a real GPU.

ต้นเหตุจริง: Verification was deferred behind the two failed branches; the surviving model inherited the 'good enough for now' status without ever passing a full-pipeline check.

เสียหายอะไร: The one model the project is relying on has an unconfirmed, single-sample, leniently-scored accuracy number and unknown end-to-end behaviour. Any downstream planning built on the 43% figure is built on sand.

แก้ยังง: Not fixed — explicitly recorded as outstanding: 'ยังไม่เคยรันครบทั้งสายพานกับการตรวจสอบจริงสักครั้ง'.

กฎที่ได้: A model is not validated until it has run the complete production pipeline at least once on real hardware, and its headline metric has been reproduced under strict (not forgiving) matching. State both conditions explicitly whenever quoting the number.

เรื่องที่เหลือในหมวดนี้ (47)

grid_ref ถูกเก็บในรูปแบบสำหรับเครื่องอ่าน ({D1D2,4}) ซึ่งคนตรวจอ่านไม่ออก

กลาง 2026-07-06

หน้าจอที่ให้คนตรวจ ต้องมีการแสดงผลแบบที่คนอ่านรู้เรื่องสำหรับทุกฟิลด์ที่เป็นรูปแบบเครื่อง โดยแสดงคู่กันไป (ไม่ใช่แทนที่) ค่าดิบที่แก้ไขได้ ถ้าคนตรวจกวาดตาแล้วอ่านฟิลด์นั้นไม่ออก เท่ากับฟิลด์นั้นไม่ได้ถูกตรวจเลย

ชิ้นส่วนคานย่อยของ mark เดียวกันกระจายกระจายในรายการตรวจ แทนที่จะถูกจัดกลุ่มไว้ด้วยกัน

กลาง 2026-07-06

ทุกครั้งที่การแก้ schema ทำให้จำนวนแถวเพิ่มขึ้นเป็นทวีคูณ ให้แก่การแสดงผลหน้าตรวจไปพร้อมกันในรอบเดียวกัน จัดกลุ่มแถวตามคีย์ที่คนใช้คิดจริง (คือ mark) และใช้การเรียงแบบ stable เพื่อให้ลำดับภายในกลุ่มยังคงความหมายเดิมไว้

ไฟล์ PNG ของ BOQ หนึ่งไฟล์ จริงๆ มีแบบสองแผ่นซ้อนกัน หมุน 90° และแยกซ้าย-ขวา

กลาง 2026-07-06

ให้ตรวจสอบสมมติฐาน 'หนึ่งหน้า = หนึ่งรูป' กับตัวอย่างของทุกชุดแบบก่อนเริ่มสกัดข้อมูล และบันทึกความแปลกของรูปแบบไฟล์ต้นทางไว้ในโน้ตของบ้านหลังนั้นทันที เพื่อไม่ให้คนถัดไปต้องมาค้นพบซ้ำ ถ้าสองคนเจอความแปลกอันเดียวกันแยกกัน แปลว่าคนแรกไม่ได้เขียนไว้ในที่ที่คนที่สองจะเห็น

สคริปต์ตรวจสอบรายงานว่าไฟล์รูปหายทั้ง 61 ไฟล์ ทั้งที่ตัว path ต่างหากที่เป็นบั๊ก ไม่ใช่ไฟล์

กลาง 2026-07-08

เวลาสคริปต์ตรวจสอบบอกว่าทุกอย่างพัง 100% ให้สงสัยสคริปต์ก่อนสงสัยข้อมูล และห้าม hardcode path แบบ shell ลงใน runtime ปกติบน Windows เต็ดขาด

ไม่เคยยืนยันเลยว่าโปรเจกต์ Material List ใน Label Studio ถูกสร้างขึ้นจริงหรือเปล่า

กลาง 2026-07-09

สถานะของระบบภายนอกต้องยืนยันจากระบบภายนอกนั่นเอง (เอา project id หรือ URL มา) ไม่ใช่จากสกรีนช็อตของไฟล์ในเครื่องที่แค่เกี่ยวข้องกับมัน

ชุด BOQ ของฐานรากแผ่กับฐานรากเสาเข็มที่ทำคู่ขนานกัน ถูกวินิจฉัยผิดว่าเป็นแบบพิมพ์ซ้ำ

กลาง 2026-07-12

ตัวเลขที่เหมือนกันบนแบบ BOQ สองแผ่น ไม่ใช่หลักฐานว่าซ้ำกันโดยตัวมันเอง ให้อ่านหัวแผ่นก่อนเสมอ (ฐานรากแผ่ vs ฐานรากเสาเข็ม) ก่อนจะฟันธงว่าซ้ำ และบันทึกความสัมพันธ์ไว้ในฟิลด์อ้างอิงไขว้ให้ชัด แทนที่จะลบหรือติดธงทิ้งไว้

เกือบจะลากค่าช่วงกริดที่เดามาไปใส่ WC1 โดยไม่ได้เช็คแบบขยายเฉพาะของมันก่อน

กลาง 2026-07-14

ก่อนจะลากการเปลี่ยนกริดไปใส่ห้องหรือ element ใดๆ ให้เช็คก่อนว่ามีแบบขยายเฉพาะของมันอยู่หรือเปล่า แบบขยายชนะการอนุมานที่ลากมาจากผังภาพรวมเสมอ

Two B4X spans reverted to the wrong dummy grid label after being fixed once

กลาง 2026-07-15

After any editing pass over a plan file, re-validate every span against the grid master. Two occurrences of the same mark with different spans is a regression signal — treat a self-inconsistency as a bug, never as data.

Tie beam recorded as RB1A when the sheet prints RB1

กลาง 2026-07-15

Where a mark changes along a run, zoom in on the boundary and check symmetry against the mirrored side — a mark boundary is exactly where a mislabel hides.

The pre-flight check passed all 5 of its checks and caught none of the things that actually broke the session

กลาง 2026-07-21

A pre-flight must assert the things that have broken before: free disk $\geq$ pipeline peak, exact pinned versions of transformers/torch (record and re-assert them), and a 1-example dry run through the real code path (image-processor setting included), not just imports.

The dataset rollup hash came out different from the previously noted number, causing a false data-changed scare — the formula had never been recorded

กลาง 2026-07-28

Never record a checksum, hash, or derived metric without the exact command or formula that produced it. An unreproducible verification number is worse than none — it manufactures false alarms.

Nearly 'fixed' eval_fields.py's missing max_seq_length, which would have broken cross-round comparability for no reason

กลาง 2026-07-28

Before fixing a suspected bug in shared measurement code, look for empirical evidence of whether it actually fires. A result that would be impossible if the bug existed is proof it doesn't — and an unnecessary fix to a shared ruler is worse than the bug.

Model support had been evidenced by a merged PR number rather than by the binary actually on the machine

กลาง 2026-07-28

Prove capability against the actual artefact on the actual machine — inspect the binary, read the real config file. A version or PR number is a hypothesis, not evidence.

The ~74GB VRAM requirement that drove machine selection was never measured — extrapolated from t01 and from a 5-step test run

กลาง 2026-07-28

When a full run succeeds, harvest its peak VRAM/RAM immediately; that is the only moment the real number is free to obtain. A number from a 5-step test is not a capacity figure.

มีคนสั่งทำลายเครื่อง GPU เข้าสองครั้งกลางคัน ก่อนที่จะदान์โหลดผลลัพธ์ลงมา เก็บข้อมูลหาย

กลาง 2026-08-20

ทำให้การทำลายเครื่องเกิดขึ้นไม่ได้ถ้ายังไม่ได้ sync โดยห่อขั้นตอนปิดเครื่องไว้ในสคริปต์ที่ rsync ผลลัพธ์ลงมาและนับไฟล์ในเครื่องยืนยันก่อน วินัยแบบนี้ไม่ควรฝากไว้กับความจำของคนที่กำลังเหนื่อย

การ import แบบครบวงจรถูกตรวจสอบด้วยการรัน adapter ใน Node เท่านั้น ไม่เคยผ่านหน้าจอบเอร์เซอร์จริง

กลาง 2026-08-21

การตรวจระดับ Node พิสูจน์ได้แค่เครื่องยนต์ ไม่ใช่ตัวผลิตภัณฑ์ วงจรจะยังไม่ถือว่าครบจนกว่าจะกด import จริงหนึ่งครั้งผ่าน UI จริง และได้อารีต้องบอกตรงๆ ว่าทำแบบไหนไป

เดโม 'ครบวงจร' เข้า Constaint ใช้ grid master ที่คนทำ ไม่ใช่ของโมเดล Qwen ไม่เคยรันหน้าเส้นกริดเลย

กลาง 2026-08-21

เวลาเดโม pipeline แบบครบวงจร ต้องบอกให้ชัดว่าขั้นไหนโมเดลทำจริง ขั้นไหนเอาข้อมูลคนมาแทน เพราะการเอาข้อมูลคนมาใส่แทนตั้งแต่ขั้นแรก ทำให้คำว่า 'ครบวงจร' ใช้ไม่ได้แล้ว ต่อให้ขั้นหลังจากนั้นจะจริงทั้งหมดก็ตาม

สร้างข้อเสนอความสามารถของโมเดลที่ไม่เคยพิสูจน์กับ adapter ที่ fine-tune แล้วจริงๆ

กลาง 2026-08-22

ก่อนออกแบบขั้นตอนใน pipeline โดยฟังความสามารถของโมเดลตัวตั้งต้น ให้อยอมจ่ายราวๆ \$0.10 ทดสอบก่อนว่า adapter ที่ fine-tune แล้วยังทำได้อยู่ไหม และให้เขียนข้อสันนิษฐานที่ยังไม่ทดสอบไว้ชัดๆ ในเอกสาร ไม่ใช่ซุกไว้

ข้อสันนิษฐานที่ยังไม่พิสูจน์ข้อที่สองในข้อเสนอเรื่อง grounding: chat template อาจไม่ปล่อย box token ออกมาเลยด้วยซ้ำ

กลาง 2026-08-22

การทดสอบความสามารถต้องรันผ่านเส้นทางการเลิร์ฟจริงทั้งเส้น (template + tokenizer + decode) ไม่ใช่แค่ถามว่า 'โมเดลรู้วิธีทำไหม' ให้ไล่รายชื่อทุกชั้นระหว่าง prompt กับผลลัพธ์ที่ parse แล้ว ก่อนจะบอกว่าการทดสอบนี้ถูก

ข้อสันนิษฐานของที่ปรึกษาที่ว่า 'ถ้าใช้ Qwen ตัวใหญ่ขึ้นสิ' มันหมดทางไปแล้ว

กลาง 2026-08-25

ก่อนรับข้อเสนอ 'ใช้โมเดลใหญ่ขึ้น' มาเป็นงานที่ต้องทำ ให้ตรวจสอบว่าขนาดไหนบ้างที่ดาวน์โหลดได้และเทรนได้จริง และให้คิดราคาของการเปลี่ยนโครงสร้างพื้นฐานทั้งหมด ไม่ใช่แค่ค่า GPU ต่อชั่วโมง

template ของฐานราก F2 ยิ่งผลบวกปลอม 3 จุดบนแปลนคาน ซึ่งไม่มีฐานรากอยู่เลยสักตัว

กลาง 2026-08-25

เมื่อตัวตรวจจับสับสนระหว่างสองคลาส ให้เลือกใช้เงื่อนไขเชิงเงื่อนไขที่พิสูจน์ได้ว่าไม่ทำอะไรกับเคสที่ถูกต้องอยู่แล้ว ดีกว่าไปขั่นเกณฑ์ให้แน่นขึ้น ซึ่งแค่แลกผลบวกปลอมเป็นผลลบปลอม ไม่ได้กำจัดอะไรสักอย่าง

pattern_recognition.py validated only on 2 houses from the same design series

กลาง 2026-08-25

State the diversity of your validation set alongside every accuracy number. Two samples from the same source is one sample of the source, not two.

Detection tool never tested on cover pages, sections or BOQ tables — could mark them with confident nonsense

กลาง 2026-08-25

A detector that is only valid on some input types must either detect and refuse the others, or must not be handed to users until it has been tested on every page type they will feed it.

The MAX_PIXELS fix shipped knowing 30% of images are still downsampled, and the largest 116 images still lose 67%

กลาง 2026-08-25

When a fix is partial, state the residual in the same sentence as the win. '32% better' and '30% of images still degraded' are the same measurement and must travel together.

Near-miss: the first fix considered for the F2 false positives would have silently deleted real footings

กลาง 2026-08-25

A global knob cannot fix a contextual error; it only trades a loud failure for a quiet one. When the wrong answer is wrong *because of where it is*, the fix is a rule about place, not a number.

"Just increase B" assumed model size is a free dial; the family is already at its open-weight ceiling

กลาง 2026-08-25

Before proposing a bigger model, enumerate the actual released open-weight sizes and price the hardware for each. "Bigger" is a hardware and pipeline decision, not a hyperparameter.

Translating the whole prompt to English scores ~8% worse than a native prompt — a trap the dictionary idea could have walked into

กลาง 2026-08-25

When adopting a technique from a meeting suggestion, search for its published failure mode as well as its success case, and pin down which specific problem it is supposed to solve before writing any of it.

First hypothesis wrong: F1 vs F2 footings differ in shape, not scale — multi-scale matching missed all F1s

กลาง 2026-08-25

Before choosing an invariance strategy (scale, rotation), look at the actual variants side by side. If they are different symbols, no amount of scaling will find them — you need a template per variant, exactly as the literature says.

F2 tall-rect footing template false-fired on beam plans

กลาง 2026-08-25

A shape-based detector needs a context filter, not just a higher threshold. Use a mutually-exclusive signal from the same page (here: beams and footings never coexist) to reject cross-context false positives.

Multi-scale matching on a tiny 28x34 column template inflated detections 14 → 21 with noise

กลาง 2026-08-25

Do not apply the same invariance strategy uniformly across templates. Small templates have too little signal for multi-scale — always validate a strategy change against a page with a known correct count before keeping it.

tpl_footing9 was too generic — leaked junk on other houses while scoring poorly on its own

กลาง 2026-08-25

Judge a new template by its worst effect across all houses, not its best effect on the one it came from. Prefer an honest gap over a template whose junk scores overlap its real scores.

Molmo-72B's HuggingFace-listed 293GB was fp32, not bf16 — nearly double the real weight size

กลาง 2026-08-25

Before using a model repo's listed size, divide bytes by parameter count to infer the dtype. A repo published in fp32 will read roughly 2x its real bf16 footprint.

InternVL models branded as non-Qwen actually wrap Qwen LLM backbones

กลาง 2026-08-25

Verify model lineage from the config's declared backbone, never from the brand name. A VLM's vision tower and its LLM often come from different organizations.

A research pass assumed the training run was 4-bit QLoRA when LOAD_IN_4BIT was already False (bf16)

กลาง 2026-08-25

Read the actual config file before reasoning about resource usage. Assumptions about your own setup are the easiest thing to verify and the most expensive thing to get wrong.

Pattern recognition validated on only 2 houses from the same drawing series

กลาง 2026-08-25

When your test set shares a common source, you are measuring within-source accuracy, not accuracy. State the source-diversity of the test set alongside every accuracy figure.

Two houses over-detect footings roughly 2x — likely a cross-series template matching an inset

กลาง 2026-08-25

Over-detection is as much a failure as under-detection and is easier to overlook because it looks like coverage. Always measure both directions and chase the over-detecting cases with the same energy as the misses.

Self-harvest can only recover element types already in the template bank — its real ceiling, found by measurement

กลาง 2026-08-26

A bootstrapping loop's ceiling is the classes it already has one seed for. Measure per-class recovery, not aggregate recovery, or the aggregate number hides that a whole class recovered nothing.

กฎลอกเครื่องหมายวรรคตอนแบบเหมารวมจะทำให้ ป. (ปลอก) กลายเป็น ป แล้วชนกับ ประตุ

กลาง 2026-08-29

กฎที่กวาดอักขระเหมารวมบนข้อความไทยต้องมีรายการยกเว้นที่สร้างจากการ grep การใช้งานจริงก่อน เคสอันตรายคือเคสที่ลอกแล้วยังได้ token ที่ดูเหมือนถูกต้อง

เคลมว่า Qwen3.5-122B-A10B เป็นโมเดลข้อความล้วน — จริงๆ มันมี mmproj และเป็น multimodal

กลาง 2026-08-30

ห้ามพูดว่า 'X ไม่รองรับ Y' โดยไม่เปิดแหล่งอ้างอิงจริงในจังหวะเดียวกัน — model card, รายการไฟล์ หรือชอรัสของไลบรารี

ยืนยันว่า 'เอา LoRA มาเฉลี่ยรวมกันไม่ได้' — ทั้งที่ PEFT มี `add\_weighted\_adapter` และเคสนี้เหมาะมาก

กลาง 2026-08-30

ทุกคำเคลมเชิงปฏิเสธความสามารถต้องมีแหล่งที่ตรวจแล้วรองรับก่อนพูดออกมา พูดผิดแบบนี้สองครั้งในวันเดียวคือรูปแบบพฤติกรรม ไม่ใช่ข้อผิดพลาด ให้ถือว่า 'X ทำ Y ไม่ได้' เป็นงานที่ต้องไปค้น ไม่ใช่เรื่องที่นึกเอาจากความจำ

เดาว่าหน้ารายการวัสดุ ปร.4 วางซ้อนบน-ล่าง จริงๆ มันวางเคียงกันและหมุน 90 องศา

กลาง 2026-08-31

ข้อสันนิษฐานเรื่องเลย์เอาต์ ต้องตรวจกับตัวอย่างจริงอย่างน้อยสองอันจากคนละแหล่งก่อนเอาไปใช้ทั้งชุด และบันทึกระดับความมั่นใจให้ตรงกับจำนวนที่ตรวจจริง

Label Studio <Style> className support is only documented for View/Choices, not Text — nearly applied unverified

ต่ำ 2026-07-06

When a tool's docs confirm a feature for some element types and are silent on others, treat the silence as 'unsupported' and fall back to the configuration already proven to work in this repo — do not extrapolate from partial documentation.

The backup was auto-flagged as failed because robocopy exit code 1 means success

ต่ำ 2026-07-10

Check a tool's exit-code semantics before trusting a red/green verdict — for robocopy specifically, anything under 8 is success, and the log's FAILED count is the real answer.

Duplicate column detection at grid (B,2) — 13 boxes for 12 real columns

ต่ำ 2026-08-25

Report detector counts against a hand-verified truth for every validation page, not just the headline page. An unverified count is not evidence, and small systematic over-counts hide inside "about right" numbers.

Molmo-72B's real bf16 footprint is ~147GB, not what '72B' implies — the model-size column and the GPU you must rent are different numbers

ต่ำ 2026-08-26

Screen candidates on bytes at the precision you will actually run, not on the marketing parameter count — and note it in every row, not just the one where someone happened to check.

Plan assumed grid master carried pixel coordinates — it only has pos_m

ต่ำ 2026-08-26

Open the actual data file before designing anything that consumes it. Five seconds of reading beats an afternoon of rework built on an assumed field.

Column icons on houses 01/02 were below the assumed 28px minimum and were missed until re-cropped with surrounding context

ต่ำ 2026-08-28

Size floors in a detector encode the first drawing series you saw. When a house yields zero candidates, treat that as a tooling bug to investigate, not as 'that house has none'.

พังเจียบ ไม่มีใครรู้

โค้ด/ไปป์ไลน์ที่พังหรือข้ามงานไปเจียบๆ ไม่มี error ให้เห็น ตัวเลขออกมาดูปกติแต่ผิด

ทั้งหมด 21 เรื่อง · วิกฤต 7 · สูง 7 · กลาง 6 · ต่ำ 1

เรื่องที่ต้องอ่านให้ครบ (14)

วิกฤต 2026-07-05

Agent เบื้องหลังค้างสองรอบ ถูกสั่งหยุดกลางคัน และหน้า 41-47 ของ BOQ ฐานรากแผ่ไม่เคยถูกสกัดเลย

เกิดอะไรขึ้น: Agent 4 ถูกส่งไปทำหน้าที่ 38-47 (BOQ ฐานรากแผ่) มันหยุดตัวเองกลางทางสองครั้ง ต้องปลุกด้วย SendMessage สองรอบ ทำเสร็จแค่หน้า 38, 39, 40 แล้วก็ถูกม้ามสั่งหยุดตอนกำลังทำหน้าที่ 41 หน้า 41-47 จึงไม่เคยถูกสกัดเลย — ทั้งในรอบ t1 และรอบ t2 (เพราะ t2 กำหนดขอบเขตไว้แค่หน้า 1-37)

ต้นเหตุจริง: Agent ที่รันท้ายกับงานซ้ำๆ จำนวนหน้าเยอะ หยุดตัวเองซ้ำๆ โดยไม่มีระบบ resume อัตโนมัติ และไม่มีการทำเครื่องหมายจุดเช็คต่อหน่วยงาน ที่จะให้คนอื่นมารับช่วงต่อที่หน้า 41 ได้สะดวกๆ ช่องว่างที่เหลืออกนอกขอบเขตรอบถัดไป เพราะ t2 กำหนดขอบเขตด้วยช่วงเลขหน้า ไม่ใช่ด้วย 'อะไรที่ยังขาดอยู่'

เสียหายอะไร: ข้อมูลจริง BOQ หายถาวร 7 หน้าของบ้าน 01 ข้ามการสกัดสองรอบเต็ม ค้างเป็นงานค้างตั้งแต่ 2026-07-05 ยังค้างอยู่จนจบ 2026-07-06 และทิ้งไฟล์ scratch กำพร้าไว้ด้วย

แก้ง่าย: ยังไม่ได้แก้ ขึ้นเป็นงานค้างข้อ 5 ท้ายวัน 2026-07-06: 'หน้า 41-47 (BOQ ฐานรากแผ่) ยังไม่ถูกสกัดในรอบไหนเลย'

กฎที่ได้: งาน agent ที่รันท้ายต้องแบ่งเป็นหน่วยงานเล็กๆ ที่ชัดเจน พร้อมมีเครื่องหมาย resume ต่อหน่วย และห้ามกำหนดขอบเขตรอบถัดไปด้วยช่วงเลขหน้า — ให้กำหนดจากส่วนต่างเทียบกับสิ่งที่เสร็จจริง ช่องว่างที่รู้อยู่แล้วจะได้ไม่หลุดออกนอกขอบเขตรอบถัดไปแบบเจียบๆ

วิกฤต 2026-07-28

บั๊กที่ 2: ค่าเริ่มต้น `resize='min'` ของ `collator` ตกไปที่ 512px เจียบๆ — ความละเอียดหายไป 25 เทา ขณะที่ VRAM ดูสบายๆ จนล่อลวงตา

เกิดอะไรขึ้น: vision collator ใช้ค่าเริ่มต้น `resize='min'` ซึ่งไปอ่านคีย์ `image_size` ของโมเดล แต่ Qwen3-VL ไม่มีคีย์ `image_size` เลย collator เลยตกไปใช้ 512px เจียบๆ — ความละเอียดหายไป 25 เทาจากที่ตั้งใจไว้ ส่วนการใช้ VRAM ก็ดูสบายๆ ซึ่งอ่านแล้วเหมือนงานกำลังไปได้สวย ไม่เหมือนงานที่พัง

ต้นเหตุจริง: ค่าเริ่มต้นของไลบรารีที่ลดคุณภาพลงเจียบๆ เมื่อหาคีย์ไม่เจอ บวกกับข้อเท็จจริงที่ว่าอาการเดียวที่สังเกตได้ (VRAM ต่ำ) ดันดูเหมือนความสำเร็จ และไม่มียะไรคอยตรวจว่าความละเอียดไปถึงโมเดลจริงๆ เป็นเท่าไร

เสียหายอะไร: จะเรนโมเดลด้วยรูปที่หยาบกว่าที่ตั้งใจ 25 เทา โดยที่ตัวชัดทุกตัวบนหน้าจาดูปกติดี บั๊กตัวเดียวกันนี้ที่ไม่มีใครจับได้ในฝั่ง t01 กลายเป็นตัวที่ทำให้การเปรียบเทียบ A/B ทั้งหมดใช้ไม่ได้ในภายหลัง

แก้ง่าย: บังคับ `resize='max'` ไปเลยตรงๆ พร้อมใส่ assertion ตรวจความละเอียดของ batch แรก

กฎที่ได้: อย่าไว้ใจค่าเริ่มต้นของไลบรารีที่ต้องไปหาค่าจากคีย์ที่มีหรือไม่มีก็ได้ ให้ assert ค่าที่ใช้จริงตั้งแต่ batch แรก — และให้ถือว่าการใช้ทรัพยากรน้อยผิดปกติเป็นสัญญาณอันตราย ไม่ใช่เรื่องน่าสบายใจ

The one guard against the project's most expensive bug was 4x too lenient and would have waved it through (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: train_t03.py's first assert is commented "ตัวอย่างแรก (ภาพเดียว)" and does ``_tok = (grid[:,1]*grid[:,2]).sum() // 4`` then ``assert _tok > 1000``. But ``train_ds[0]`` in the op04 dataset has 4 images and ``_sum()``` totals across all of them while the threshold is per-image. Simulation confirmed t01's exact failure level $(266 \text{ tokens/image}) \times 4 \text{ images} = 1,064 > 1,000$, so the assert passes.

ต้นเหตุจริง: The guard was written when the first example had exactly one image, and the dataset shape later changed to multi-image examples without the guard being revisited. The assert's own comment still documented the old assumption.

เสียหายอะไร: The single guard that exists to prevent a repeat of t01 (silently training at 266 tokens/image, the most expensive bug in the project's history) was inert. A t01-class disaster would have run to completion undetected, burning a full GPU rental.

แก้ยังไง: Divide by ``_gt.shape[0]`` so the threshold is genuinely per-image, print the configured cap alongside the measured value, and add a non-fatal warning when the measured value falls more than 25% below the cap. ``ast.parse`` clean; both branches simulated.

กฎที่ได้: A guard written for one data shape silently rots when the data shape changes. Periodically reprove your guards by simulating the exact failure they were written to catch — if you cannot make the assert fire, it is not protecting you.

t01 silently trained at the library default 512px / ~266 tokens per image and tanked accuracy (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Referenced in this session as the project's benchmark disaster: t01's MAX_PIXELS was never set explicitly, so the processor defaulted to 512px (~266 visual tokens/image) instead of the intended budget, and accuracy collapsed. Recorded as rule_of_tune #12.

ต้นเหตุจริง: A load-bearing training hyperparameter was left to a library default rather than being explicitly set and asserted, and nothing printed the effective value at run start.

เสียหายอะไร: An entire fine-tuning round (GPU rental plus wall-clock time) produced degraded results and had to be diagnosed after the fact — the most expensive bug in the project's history.

แก้ยังไง: MAX_PIXELS is now set explicitly in train_t03.py and guarded by an assert (which itself needed fixing this session — see the 4x-too-lenient guard).

กฎที่ได้: Set every accuracy-critical hyperparameter explicitly, print the effective value at startup, and assert it. A library default you did not choose is a silent config bug waiting for its first expensive run.

pass 0 mislabelled roof structure pages in 8 of 11 houses, silently deleting all roof beams from the BOQ (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The pass 0 page-classification stage assigned the wrong label to roof-structure sheets in 8 out of 11 houses. Because pass 1 routes images into folders based on pass 0's answer, those pages fell out of the whole conveyor and every roof beam vanished from the resulting BOQ.

ต้นเหตุจริง: pass 0 is a single point of failure with no downstream cross-check: a misclassified page is not flagged, it simply never appears again. Nothing reconciles 'pages in' against 'pages consumed' or checks that expected element types are present in the output.

เสียหายอะไร: Roof beams entirely missing from the bill of quantities for 8 houses — a silent quantity shortfall in the shipped output, not a visible error.

แก้ยังง: Documented as the most dangerous stage of the conveyor in pass0/README.md; the classification target was changed to answer with the destination subtask (plan_beam, section, ...) rather than a pattern, so pass 1 knows exactly where each image belongs.

กฎที่ได้: Any routing/classification stage that can drop data must have a reconciliation check downstream: count pages in versus pages consumed, and assert that expected element categories are non-empty. A misroute must raise an alarm, never just produce a smaller result.

ตัวนำเข้าไฟล์บนเว็บรู้จักแค่ชื่อ pattern ตามสเปก (`plan`/`gridline`) ซึ่งไม่มีอยู่ในไฟล์จริงสักไฟล์ — กดนำเข้าแล้วได้ 0 element แบบเนียนๆ

เกิดอะไรขึ้น: ลองจำลองว่าผู้ใช้จริงอัปโหลดบ้าน 01 ผ่านหน้าเว็บ แล้วพบว่า `raw-extraction-adapter.js` รู้จักแค่ชื่อ pattern `plan` กับ `gridline` ที่เขียนไว้ในเอกสารสเปก แต่ไฟล์จริงทั้ง 50 หลังใช้ชื่อ `beam\_plan` (85 ไฟล์), `footing\_plan` (60), `etc\_plan` (199), `roof\_frame\_plan` (46) รวม 390 ไฟล์ บวก `grid\_master` อีก 42 ไฟล์ ส่วนไฟล์ชื่อ `plan`/`gridline` มี 0 ไฟล์ พอกดป้อนนำเข้ากับข้อมูลจริงจึงได้ 0 element ไม่มีกริดเลย และไม่มีอะไรพ้อง

ต้นเหตุจริง: adapter ถูกเขียนตามเอกสารสเปก ไม่ได้เขียนตามข้อมูลจริง แล้วข้อมูลตัวอย่างสำหรับเทสในโปรเจกต์ก็ใช้ชื่อที่สเปกคิดขึ้นเองเหมือนกัน เทสจึงเขียวหมดทั้งที่ทางเดินจริงตายสนิท ข้อมูลตัวอย่างที่ตรงกับสเปกแต่ไม่ตรงกับข้อมูลจริง จับความต่างระหว่างสเปกกับความจริงไม่ได้เลย

เสียหายอะไร: ป้อนนำเข้า JSON ด้วยมือพังกับไฟล์จริงมาตลอดโดยไม่มีใครรู้ และทางเดินใหม่ของ Purson worker ก็จะสืบทอดค่าศูนย์เนียนๆ แบบเดียวกัน

แก้ยังง: แก้ที่ต้นเหตุ: เพิ่ม `canonicalPattern()` + `PATTERN\_ALIASES` ใน adapter แล้วให้จุดที่เทียบ `f.pattern` ตรงๆ ทั้ง 5 จุดใน 2 ไฟล์ รังผ่านตัวนี้หมด ป้อนนำเข้าด้วยมือที่มีอยู่เดิมก็ได้ผลทันที เพิ่ม `tests/raw-extraction-pattern-aliases.test.js` (4 เทส) หลังแก้: เรียกโมเดล 7 ครั้ง → 6 ไฟล์ → 24 element / 10 spec / 14 เส้นกริด npm test ผ่าน 1153/1153

กฎที่ได้: สร้างข้อมูลตัวอย่างสำหรับเทสจากข้อมูลจริงที่ใช้งานจริง ไม่ใช่จากเอกสารสเปก ถ้า fixture ใช้ค่าที่ไม่เคยปรากฏในข้อมูลจริงเลยสักครั้ง เทสนั้นไม่ได้พิสูจน์อะไรทั้งนั้น

Footings with no rebar size produced '(DBNull)' and \$0 steel with no warning flag — reading as if the footing genuinely has no reinforcement (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Footings F3/F4/F6 had no `main\_bar\_dia\_mm`, so `computeBOQ` emitted descriptions containing '(DBNull)' and rebar lines at \$0 with no `needs\_review` flag. On a real construction BOQ this reads as 'this footing has no reinforcement'.

ต้นเหตุจริง: Missing data flowed through the cost path unflagged. `computeBOQ` had no guard for an absent rebar diameter, so absence was rendered as a legitimate zero rather than as an unknown.

เสียหายอะไร: Dangerous for a real construction BOQ — three footings appeared to require no reinforcement. Exactly the kind of wrong number that ships.

แก้ยังไง: Added `needs\_review` / `main\_bar\_missing` flags and replaced the text with '(ไม่ทราบขนาดเหล็ก — ตรวจสอบแบบ)'. Verified it matches exactly F4/F6/F3.

กฎที่ได้: Missing input must never render as zero cost or zero quantity. Any cost path must distinguish 'measured as zero' from 'unknown' and flag the latter loudly — a silently-zero structural line is a safety issue, not a display issue.

npm test ran a single file, so an entire test suite had never been executed and was silently broken

(ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: In the Constistant repo, `npm test` was configured to run one file rather than the whole tests/ folder. When it was widened to the full folder, the bbs-shape-allowance test turned out to have never actually run and needed a DOM stub fix plus a float assertion fix before it would pass.

ต้นเหตุจริง: The test runner's scope was narrowed at some point (probably for speed or during a refactor) and never restored, so newly added test files entered the repo and rotted without anyone noticing they were not executing. Nothing reported the number of tests run against the number of test files present.

เสียหายอะไร: Unknown period of zero coverage on rebar shape allowance — a calculation that feeds cut lengths and steel weight. The broken test had to be fixed before it could pass, meaning it had been broken for as long as it had been ignored.

แก้ยังไง: Changed npm test to run the whole tests/ folder; fixed the DOM stub and the float assertion in the bbs-shape-allowance test; also merged the stray test/ folder into tests/. Result: 48/48 passing.

กฎที่ได้: The test command must always run the whole suite directory, never a hand-listed file. A test that is not run is worse than no test — it advertises coverage that does not exist. Assert that the file count and the executed count match.

สูง 2026-07-28

House 07 S-03: three structural members missing entirely from the staircase zone, and S1 slabs overcounted 4 vs 3 (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Clearing the staircase zone revealed three members that had never been extracted at all — B2 in the middle of bay 3-4, the 5th SP panel, and B6 at the ST2 edge — plus an S1 count of 4 where the real number is 3.

ต้นเหตุจริง: Congested drawing regions (a staircase, where many symbol types overlap) were passed over at normal zoom rather than being cleared exhaustively piece by piece; missing items produce no error signal, unlike wrong items.

เสียหายอะไร: Three real members and one over-counted slab in the quantity data — omissions understate the BOQ, and nothing downstream can detect an element that was never recorded.

แก้ยังไง: Zone fully cleared by opening the real images and zooming on every point; corrections appended with the reversal logged.

กฎที่ได้: Congested regions must be cleared exhaustively at full zoom, member by member. A missing element is invisible to every downstream check, so absence has to be hunted deliberately.

สูง 2026-07-28

Bug 3: max_seq_length defaulted to 2048, cutting through the middle of the image tokens, with no guard on the vision path (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: max_seq_length was left at its default of 2048. A single image is ~5,100 tokens, so the cut lands in the middle of the image token block. The audio path in the same codebase has a guard against this; the vision path has none.

ต้นเหตุจริง: A second, independent length limit existed alongside max_length, and only one of the two was being managed. The library protects one modality against this and not the other, so the asymmetry was invisible to anyone assuming parity.

เสียหายอะไร: Would have fed the model image token sequences severed mid-image — worse than truncating text, since a partial image block is meaningless.

แก้ยังไง: Corrected as part of the five-bug fix pass before renting.

กฎที่ได้: Enumerate every length/size limit in the stack, not just the one you know about. Where a library guards one code path and not another, assume the unguarded path is the one you are on.

สูง 2026-08-24

On house 08 page 20 the model emitted memorized boilerplate whose source_image pointed at a different house (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: t03 on the plan_beam page wrote only metadata and closed the JSON with no elements[], and the metadata itself referenced house 22 — the model pasted boilerplate from a similar-looking training example instead of reading the sheet.

ต้นเหตุจริง: SFT on near-identical JSON envelopes teaches the envelope; when the hard part (elements[]) fails, the model still has a high-probability header to emit, and nothing in the output shape forces the header to be consistent with the input image.

เสียหายอะไร: An output that is JSON-valid, passes grammar, and is 100% wrong including its own provenance field — the worst failure mode for a downstream consumer, since it looks like a real extraction of the wrong building.

แก้ยัง: not fixed — recorded as evidence that plan_beam 0% is memorization, not partial reading

กฎที่ได้: Add a mechanical check that a result's own metadata (source_image/house) matches the file it was generated from. That single assert catches memorized-boilerplate output that JSON validity and grammar both wave through.

สูง 2026-08-25

raw-extraction-adapter.js ADAPTED_PATTERNS still does not accept roof_plan — the bug can silently recur (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The roof-beam-missing bug was closed by fixing the data side (retagging files to `plan`), but the consumer code in Constaintant only reads `pattern:"plan"` and `ADAPTED\_PATTERNS` was never widened to accept `roof\_plan`. Found during the sweep, deliberately not fixed.

ต้นเหตุจริง: The fix was applied at the data layer only. The code-side gap that made the mistagging fatal in the first place (silently dropping unrecognized patterns instead of warning) was left in place, so the failure mode is dormant, not removed.

เสียหายอะไร: Any newly extracted house that gets tagged `roof\_plan` will have its roof framing silently disappear from the BOQ again, with no warning — exactly the original bug, re-armed.

แก้ยัง: Not fixed — recorded as a known open risk in the diary.

กฎที่ได้: When a bug is closed by cleaning data, ask whether the code would still fail silently on the same bad input. A silent-drop path is a bug even after the current data is clean; make it warn instead of vanish.

สูง 2026-08-25

Concrete/rebar strength values (fc_ksc, fy_*) had never reached any consumer in any house (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `fc\_ksc`, `fy\_main\_ksc`, `fy\_stirrup\_ksc` existed in the source `sections[]` of the JSON files but were never parsed into `notes{}` where consumers read them. 62 values across 26 files reached a consumer for the first time only during this session's sweep.

ต้นเหตุจริง: The extraction wrote the values into a free-form section array but no parsing step ever lifted them into the structured field the consumers actually read. The gap was documented in the spec (§4a) but nobody had acted on it.

เสียหายอะไร: Every downstream calculation depending on concrete/steel grade ran without the real drawing values for the entire project history up to this date.

แก้ยังไม: Parsed 62 values from the files' own `sections[]` into `notes{}`, keeping the literal printed text in `notes.parsed\_as`.

กฎที่ได้: Extracting a value is not the same as delivering it. For every field the spec defines, verify at least one consumer actually reads it end-to-end — a documented "never reaches consumer" note is an open bug, not a footnote.

สูง 2026-08-25

Images in Markdown silently never appeared in generated PDFs (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `tools/md\_to\_book\_pdf.py` writes the intermediate HTML to a temp directory, so every relative `![]` (...) image path resolved to nothing. Chromium skipped each missing image without raising an error, so PDFs shipped with 0 of 4 figures and no warning.

ต้นเหตุจริง: A deliberate workaround (temp dir, to dodge Chromium mishandling Thai/space paths on Windows) broke the relative-path assumption of image tags, and the renderer's failure mode for a missing image is silence rather than an error.

เสียหายอะไร: Teaching PDFs produced by this shared tool were figure-less; the defect was invisible until someone opened a PDF and looked. Unknown how many earlier documents shipped without their images.

แก้ยังไม: Added `absolutize\_images()` — rewrites relative `src` to a full `file://` URI via `Path.as\_uri()` (which also percent-encodes Thai folder names), leaves http/data/file/absolute alone, and prints a warning for a genuinely missing file. Verified 0 → 4 images embedded.

กฎที่ได้: When you relocate files to a temp dir, every relative reference in them breaks. And never accept a renderer that swallows missing assets — add an explicit warning so a missing file fails loudly instead of shipping a hole.

เรื่องที่เหลือในหมวดนี้ (7)

ฟิลต์ใหม่ **spec_source** กับ **spec_confidence_score** ไม่มีช่องให้แก่ คนตรวจเลยแก้ค่าไม่ได้

กลาง 2026-07-06

การเพิ่มฟิลต์ลง schema ยังไม่ถือว่าเสร็จ จนกว่าฟิลต์นั้นจะแก้ไขได้ในหน้าตรวจ ถึงรวมแบบอ่านอย่างเดียวมีไว้กันข้อมูลหาย ไม่ใช่ที่อยู่ของฟิลต์ที่ต้องให้คนแก้

Unresolved conflict between the section sheet and the beam plan on B4's additional-bar count

กลาง 2026-07-15

Flagging a two-source conflict instead of guessing is correct — but the flag needs an owner and a resolution path, otherwise it becomes permanent ambiguity in training data.

Dummy grid B' recorded as 10.2 when the real value is 10.1, cascading a wrong beam span

กลาง 2026-07-20

Derive every dummy grid position from at least two independent dimension chains and require them to agree before recording — a single chain has no error detection.

A typo printed on the drawing ('HITASHI') was silently auto-corrected to 'HITACHI' during extraction

กลาง 2026-07-28

Transcribe source documents verbatim, including obvious errors. Record the correction as a separate annotated field — never silently improve what the source says.

pass 2.5 ที่เก็บข้อมูลเองกู้คืนชนิดชิ้นส่วนที่คลั้งเทมเพลตกลางไม่มีเลยไม่ได้

กลาง 2026-08-29

บันทึกจุดบอดเชิงโครงสร้างของกลไกกู้คืนทุกตัวไว้ให้ชัด และเพิ่มการเฝ้าดูกรณีจุดบอดโดยเฉพาะ (หน้าที่ CV ตรวจไม่เจออะไรเลย) เพื่อส่งหน้าพวกนั้นให้คนตรวจ แทนที่จะปล่อยผ่านไปเงียบๆ

The completely rebuilt loading screen was never opened in a real browser

กลาง 2026-09-01

Never block UI verification on a real job. Add a dev flag or seeded fake progress state so any loading/waiting UI can be eyeballed on demand — otherwise it ships unseen.

1400px figures would have overflowed the A4 text column with no img CSS rule

ต่ำ 2026-08-25

Fixing a silently-dead code path exposes every unexercised bug behind it. After enabling a path for the first time, review the whole path, not just the line you changed.

วิธีวัดผล และการประเมิน

ตัววัดที่วัดผิดสิ่ง เทียบกันแบบไม่แฟร์ หรือออกแบบการวัดที่ตอบคำถามไม่ได้

ทั้งหมด 94 เรื่อง · วิกฤต 23 · สูง 33 · กลาง 31 · ต่ำ 7

เรื่องที่ต้องอ่านให้ครบ (56)

วิกฤต 2026-07-28

บักยอรูปเหลือ 512px แบบเนียนๆ ถูกแก้แค่ในสคริปต์ของ t02 ทำให้ t01 เทรนด้วยรูปที่เบลอกว่าประมาณ 14 เท่า

เกิดอะไรขึ้น: ระหว่าง Phase 0.4 เจอบักว่าถ้าไม่ระบุ resize="max" ไว้ ตัว collator จะยอรูปลงเหลือ 512px เนียนๆ แล้วแก้เฉพาะในสคริปต์เทรนของ t02 เท่านั้น ไม่มีใครย้อนกลับไปดูสคริปต์ของ t01 (ที่เขียนไว้ก่อนหน้านี้เป็นเดือน) ซึ่งมีข้อบกพร่องเดียวกัน พอวัดจริงยืนยันว่า t01 เทรนที่ประมาณ 266 visual token ต่อรูป ขณะที่ t02 อยู่ที่ประมาณ 3,796-5,100

ต้นเหตุจริง: แก้บักเฉพาะจุดที่เห็นมันตอนนั้น ไม่ได้แก้ทุกจุดที่ใช้เส้นทางโค้ดเดียวกัน ข้อบกพร่องนี้มองไม่เห็นเพราะมันมาจากค่าเริ่มต้นที่ไม่ได้ตั้ง เลยไม่มีบรรทัดโค้ดให้อามาเทียบ

เสียหายอะไร: การเปรียบเทียบ t01 กับ t02 ทั้งหมด — ซึ่งเป็นเหตุผลทั้งหมดที่เข้าเครื่องมาทำ t02 — กลายเป็นการเทียบที่ไม่ยุติธรรม ข้อสรุปที่ว่า "Qwen3.6 ชนะ Qwen3-VL" ต้องลดระดับลงเป็น "Qwen3.6 ที่ได้รูปเบลอกว่ามาก ก็ยังชนะ Qwen3-VL" ซึ่งไม่ได้ตอบคำถามที่ตั้งไว้แต่แรก

แก้ง่าย: ตรวจสอบด้วยการโหลดโมเดลจริงแล้วรัน collator ของ t01 กับรูปจริง จับ log ของไลบรารีเองได้ว่า "Model does not have a default image size - using 512" มะขามเลือกทางเลือก A คือยอมรับผลลัพธ์พร้อมใส่หมายเหตุกำกับไว้ทุกที่ แทนที่จะจ่ายเงินเทรน t01 ใหม่

กฎที่ได้: พอเจอบัก ให้ grep หาทุกจุดที่เรียกใช้แบบเดียวกันก่อนปิดงาน — การแก้ที่ซับซ้อนที่สุดคือแก้ในฟังก์ชันที่ใช้ร่วมกัน ไม่ใช่แค่ในไฟล์ที่บังเอิญกำลังอ่านอยู่ และให้พิสูจน์การวินิจฉัยด้วยการวัดจริง ไม่ใช่ด้วยการอ่านโค้ด

การทดลอง A/B ทั้งหมดใช้ไม่ได้: t01 เทรนด้วยภาพที่เบลอกว่าราว 14 เท่า เพราะ collator ถูกเรียกด้วยค่า default ล้วน และตารางเทียบตัวแปรไม่มีบรรทัดของมันเลย

เกิดอะไรขึ้น: หลังผลทั้งวันออกมาแล้ว การตรวจสอบด้วย 43 agent พบว่า train_qwen36.py บรรทัด 164 (ฝั่ง t01) เรียก collator โดยไม่ใส่ resize= เลย — ซึ่งเป็นบั๊กตัวเดียวกับที่เคยเจอและแก้ไปแล้วแต่แก้เฉพาะฝั่ง t02 วัดจริงบนเครื่องเช่า: t01 เทรนที่ราว 266 visual token ต่อภาพ ขณะที่ t02 อยู่ที่ราว 3,796 — ต่างกันประมาณ 14 เท่า ส่วนตารางเทียบตัวแปร 16 บรรทัดที่สร้างขึ้นมาเพื่อคุมตัวแปรโดยเฉพาะ กลับไม่มีบรรทัดของ collator เลย

ต้นเหตุจริง: ตารางเทียบตัวแปรเปรียบเทียบเฉพาะค่าที่ตั้งใจกำหนดเอง ค่าที่ไม่เคยพิมพ์ลงไปเลยจึงไม่มีบรรทัดให้เทียบ ทำให้ค่า default ที่ต่างกันมองไม่เห็นโดยโครงสร้างของการตรวจแบบนั้น — และบั๊กตัวที่ 2 ก็ถูกแก้เดินหน้าไปในสคริปต์ใหม่ โดยไม่มีใครย้อนกลับไปดูว่าสคริปต์ตัวเก่ามีบั๊กแบบเดียวกันหรือเปล่า

เสียหายอะไร: ข้อสรุปพาดหัวของวันนั้น ('Qwen3.6 ชนะ Qwen3-VL') ต้องถอนออก แล้วลดเหลือแค่ 'Qwen3.6 ถึงจะเห็นภาพเบลอกว่ามาก ก็ยังทำได้ดีกว่า' คำถาม A/B ที่เป็นเหตุผลของการทำรอบนี้ทั้งรอบยังไม่ได้คำตอบ การเทรนที่จ่ายเงินไปบวกกับการวิเคราะห์เพิ่มเติม ได้ผลเปรียบเทียบที่ตีความไม่ได้

แก้ยังง: พิสูจน์ด้วยการวัดจริง ไม่ใช่การอ่านโค้ด (โหลด Qwen3.6 แล้วสร้าง collator แบบเดียวกับ t01 ระบบพิมพ์ออกมาว่า 'Model does not have a default image size - using 512') ใสกลองเตือนตัวใหญ่ไว้ใน t02_workflow.md ที่เฟส 7 และ \$0.0 ก่อนคำว่า 'ชนะขาด' ออก และเพิ่ม Lesson 12 ลง rule_of_tune.md มะขามเลือกทางเลือก A (ยอมรับผลพร้อมค่าเตือน) แทนทางเลือก B (เทรน t01 ใหม่ด้วย resize='max' ราว \$0.60-1.80)

กฎที่ได้: ตารางคุมตัวแปรต้องใส่ทุก argument ที่ส่งเข้าไปในฟังก์ชันหรือคลาสที่ใช้ร่วมกัน โดยอ่านจากซอร์สจริง ไม่ใช่แค่ค่าที่เราเลือกจะเปลี่ยน ค่า default ที่ไม่ได้ตั้งก็เป็นตัวแปรเหมือนกัน และมันคือตัวที่พลาตาง่ายที่สุดเพราะไม่มีบรรทัดให้เทียบ อีกข้อ: เจอบั๊กในสคริปต์ตัวหนึ่งเมื่อไร ให้ไปไล่เช็คสคริปต์พี่น้องทุกตัวว่ามีรูปแบบเดียวกันไหมทันที

ตัวเลขพาดหัว 90% / 28.2% วัดมาจากของที่ไม่มีใครเอาไปรันจริง

เกิดอะไรขึ้น: ตัวเลขพาดหัวของทิม (JSON ถูกต้อง 90%, element recall 28.2%) มาจาก eval_fields.py ซึ่งโหลด bf16 + adapter ตอนรันด้วย load_in_4bit=False คือไม่มีขั้น merge และไม่มีการ quantize เลย แต่การใช้งานจริงรันด้วย GGUF ที่ merge แล้ว + Q4_K_M ซึ่งไม่เคยมีใครวัดความแม่นยำเลยสักครั้ง

ต้นเหตุจริง: การประเมินผลทำบนเส้นทางที่สะดวก (full precision, adapter ที่ยังไม่ merge) แทนที่จะทำบนเส้นทางที่ใช้งานจริง และสองเส้นทางนี้ต่างกันตรงสองขั้นตอนที่มีโอกาสทำคุณภาพตกมากที่สุดพอดี คือ merge กับ quantize

เสียหายอะไร: ตัวเลขคุณภาพที่ทีมยกมาอ้างบ่อยที่สุด อธิบายการตั้งค่าที่ไม่เคยถูกส่งมอบจริง การวางแผน ความมั่นใจ และการตัดสินใจไปต่อ/ไม่ไปต่อ ล้วนตั้งอยู่บนตัวเลขที่ใช้กับระบบที่รันอยู่จริงไม่ได้

แก้ยังง: แก้ไปบางส่วน — บันทึกความไม่ตรงกันนี้ไว้แล้วและระบุขอบเขตตัวเลขใหม่ชัดๆ ว่า 'ใช้ได้กับ adapter เท่านั้น' ส่วน GGUF ยังไม่มีตัวเลขความแม่นยำที่วัดมาเลย

กฎที่ได้: ตัวเลขความแม่นยำทุกตัวต้องแนบไว้เสมอว่า 'วัดจากอะไร' และต้องทำให้ชุดประเมินผลโหลดของชั้นเดียวกับที่ส่งมอบจริง ถ้า eval กับ production โหลดคนละไฟล์ ตัวเลข eval นั้นก็เป็นตัวเลขของโมเดลคนละตัว

ผ่านการเทรนเต็มรอบ export GGUF และใช้งานจริงแล้ว แต่การสกัดข้อมูลหลักเสริม — ซึ่งคือเป้าหมายจริง — ไม่เคยถูกทดสอบเลยสักครั้ง

เกิดอะไรขึ้น: ยังไม่เคยมีแบบโครงสร้าง (แผ่น S-*) แผ่นไหนที่ถูกสกัดข้อมูลจนครบเลย หน้า 07-11 ทำแค่จำแนกประเภทเท่านั้น กฎที่ใหญ่ที่สุดข้อเดียวใน prompt ('ต้องแยกหลักบน/หลักกลางเสมอ') ไม่มีหลักฐานทั้งฝั่งสนับสนุนและฝั่งค้าน

ต้นเหตุจริง: หน้าที่ใช้ประเมินผลและหน้าที่ใช้งานจริงถูกเลือกตามลำดับหน้า (01, 02, 03...) แทนที่จะเลือกตามความสำคัญต่อเป้าหมาย หน้าที่มีค่ามากที่สุดจึงไปอยู่ท้ายคิวเสมอและไม่เคยถูกทำถึง

เสียหายอะไร: ทุกคำกล่าวอ้างเกี่ยวกับความมีประโยชน์ของโมเดลต่อจุดประสงค์หลักของโปรเจกต์ ตั้งอยู่บนการวัดผลศูนย์ครั้ง งานหลายสัปดาห์ยังไม่ได้ตอบคำถามหลักเลย

แก้ยังไง: ยังไม่แก้ บันทึกไว้เป็นคำถามค้างที่ยังเปิดอยู่

กฎที่ได้: ทดสอบเคสที่มีค่ามากที่สุดก่อน ไม่ใช่เคสแรกตามลำดับ ถ้าจบครบหนึ่งรอบแล้วความสามารถหลักยังไม่ถูกทดสอบเลย แปลว่ารอบนั้นวัดผิดเรื่อง

t02 เห็นหน้า beam plan ตอนเทรนแค่ ~13 หน้า แล้วได้ element recall 11% บนหน้าพวกนั้น

เกิดอะไรขึ้น: พอับข้อมูลเทรนจริงของ t02 พบว่ามันเห็นหน้า beam plan แค่ ~13 หน้า เทียบกับ material_list ~158 หน้า และตารางประตู-หน้าต่าง ~24 หน้า ผลที่วัดได้ก็วิ่งตามจำนวนตัวอย่างจริงๆ เลย: ตารางประตู-หน้าต่าง 86% ส่วน beam plan 11% แกม beam plan ยังเป็น pattern ที่ยากที่สุดด้วย (~25 คานต่อหน้า แต่ละตัวต้องได้ grid ref ทั้งจุดเริ่มและจุดจบถูกต้อง)

ต้นเหตุจริง: สัดส่วนของชุดข้อมูลเทรนสะท้อนจำนวนหน้าดิบในแบบก่อสร้าง แทนที่จะถูกจัดสมดุลให้เอนไปทาง pattern ที่ยากที่สุด และมีค่าที่สุด pattern ที่ยากที่สุด + ตัวอย่างน้อยที่สุด = คู่ที่แย่ที่สุดเท่าที่จะเป็นไปได้ แล้วก็ไม่มีใครนับจำนวนตัวอย่างแยกราย pattern ก่อนเทรนเลย

เสียหายอะไร: beam plan ซึ่งเป็น pattern เดียวที่ BOQ พังพามากที่สุด ทำ recall ได้แค่ 11% การเดโมวงจรการสกัดข้อมูลทุกอย่างที่อยู่ปลายทางจึงติดคอขวดตรงนี้ grammar-constrained decoding แก่เรื่องรูปแบบได้ (JSON ที่ถูกต้อง 57.9%→96.8%) แต่ความแม่นยำไม่ขยับเลย

แก้ยังไง: ยังไม่ได้แก้ด้วยโค้ด เขียน dataset_sizing.md พร้อมตัวเลขจริง ตั้งเป้าไว้ที่ 25-30 หลัง (~99 หน้า beam plan) และสร้าง op04 ขึ้นมาให้ทำ annotation เฉพาะ pattern ของ Pass 2 เพื่อให้แรงงานไปลงที่ beam plan แทนที่จะไปลงกับ material list

กฎที่ได้: นับจำนวนตัวอย่างเทรนแยกตาม pattern ของหน้า ก่อนเทรน แล้วจัดสมดุลใหม่ให้เอนไปทาง pattern ที่ยากที่สุดและมีค่าที่สุด — ความสัมพันธ์ระหว่างจำนวนตัวอย่างกับความแม่นยำตรงและวัดได้จริง (13→11%, 24→86%) การสำรวจจำนวนแยกราย pattern จึงเป็นสิ่งที่ต้องทำก่อน fine-tune ไม่ใช่เครื่องมือไว้ชั้นสุดรสบทที่หลัง

บ้าน 08 ที่ใช้เป็น benchmark ดันอยู่ในชุดเทรนของ t03 — ถ้าไม่จับได้ การเทียบข้ามรอบจะไม่มีความหมายเลย

เกิดอะไรขึ้น: เลือกบ้าน 08 มาเป็น benchmark เทียบข้ามรอบเพราะมีผลของ t02 กับ base-Qwen อยู่แล้ว แต่บ้าน 08 อยู่ในชุด *เทรน* ของ t03 ในขณะที่ t02 ไม่เคยเทรนบนมันเลย ตัวเลขของ t03 จึงจะพุ่งขึ้นเพราะการท่องจำ และเทียบกันไม่ได้เลย

ต้นเหตุจริง: เลือกบ้านที่ใช้ทำ benchmark เพราะสะดวก มีผลเก่าอยู่แล้ว โดยไม่เช็คว่ารอบปัจจุบันมันตกอยู่ฝั่งไหนของการแบ่งชุดข้อมูล ไม่เคยมีใครเอาการเป็นสมาชิกของชุด train/val ไปตรวจไขว้กับแผนการเปรียบเทียบ

เสียหายอะไร: จับได้ก่อนที่จะเผยแพร่ผล — แต่ถ้าพลาดไป การเทียบ t02 กับ t03 ทั้งชุดจะใช้ไม่ได้เลย แล้วตัวเลข t03 ที่พุ่งขึ้นจะถูกนำเสนอเป็นความก้าวหน้าจริง

แก้ยังง: ย้ายบ้าน 08 ไปอยู่ฝั่ง val แล้วเทรนใหม่: train 887→854 (42 หลัง), val 229→262 (6 หลัง), อัตราส่วน 3.26:1 ตอนนี้บ้าน 08 ไม่เคยถูกเห็นโดยทั้งสามรอบจริงๆ แล้ว

กฎที่ได้: บ้านที่ใช้เป็น benchmark เทียบข้ามรอบ ต้องยืนยันว่ากันออกจากชุดเทรนใน *ทุกรอบ* ที่เอามาเทียบ ไม่ใช่แค่รอบปัจจุบัน เช็คก่อนเลือก benchmark ไม่ใช่เช็คทีหลัง

section 0% ส่วนใหญ่มาจากตัวชี้วัดที่พัง ไม่ใช่โมเดลที่พัง — AI ที่สมบูรณ์แบบก็ทำได้สูงสุดแค่ 4%

เกิดอะไรขึ้น: การประเมิน section เทียบสตริง element_id แบบตรงตัว แต่ ID ใน ground truth ของ section เป็นป้ายที่คนคิดขึ้นเอง (gate_front, door_lock_detail, SP_at_bathroom, purlin_rafter_connection, ห้องนอน) ซึ่งไม่ปรากฏบนแบบเลยสักที่ วัดเพดานได้: section 37% ในภาพรวม และแค่ 1 จาก 23 = 4% บนบ้าน 08 ซึ่งเป็นหลังที่ใช้ประเมินจริง ส่วน notes ยิ่งแย่กว่าที่ 2%

ต้นเหตุจริง: ID ใน ground truth ถูกสร้างขึ้นเพื่อความสะดวกในการจดบันทึกของคน แล้วถูกเอามาใช้เป็นกุญแจในการประเมินจริงๆ โดยไม่เคยเช็คเลยว่าข้อมูลที่ต้องใช้สร้าง ID พวกนั้นมีอยู่บนภาพต้นทางไหม

เสียหายอะไร: ตัวเลขพาดหัว 0% ถูกอ่านว่า 'โมเดลอ่าน section ไม่ได้' และกำลังจะพาไปสู่การไปแก้โมเดล/prompt ต่อ ซึ่งอาจกลายเป็นการเทรนเสียเงินอีกทั้งรอบเพื่อสิ่งที่ไม่มีโมเดลไหนทำได้ยุดี

แก้ยังง: เขียน measure_id_ceiling.py ขึ้นมาแยก ID ออกเป็นกลุ่มที่อ่านได้จากแบบ กับกลุ่มที่คนคิดขึ้นเอง แล้วคำนวณเพดานที่เป็นไปได้แยกตาม subtask ผลคือจัดประเภท section 0% ใหม่ว่าเป็นความล้มเหลวของการวัด ส่วน plan_beam 0% (เพดาน 88%) เป็นความล้มเหลวของโมเดลจริงๆ

กฎที่ได้: ก่อนจะไป optimize ตามตัวชี้วัดใดๆ ให้คำนวณเพดานที่เป็นไปได้ของมันเป็นบนชุดประเมินชุดนั้นๆ เสียก่อน ตัวชี้วัดที่มีเพดานอยู่ที่ 4% ไม่ได้กำลังวัดคุณภาพของโมเดล แต่กำลังวัดธรรมเนียมการทำ annotation ต่างหาก

Workflow doc claimed images were never downscaled; measurement showed 100% of them were (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `t03\_workflow.md` row 7 recorded "ภาพจริง 2339×1654 ต่ำกว่า cap 5,120 อยู่แล้ว = ไม่ย่อเลย". A direct measurement found 973/973 image instances (100%) exceed the cap — the smallest image in train.jsonl is 3307×2339 (7,554 tokens) and the largest 4631×3473 (15,707 tokens, losing 67% of its detail). No image of the recorded size exists in the current train set.

ต้นเหตุจริง: The earlier record was produced by looking at a single sample (`train\_ds[0]`) and generalising it to the whole dataset — the same failure mode already recorded as rule_of_tune #12, repeating.

เสียหายอะไร: The team believed image resolution was not a bottleneck and made downstream decisions on that belief. Every training image was in fact being downscaled, up to 67% detail loss on the largest pages.

แก้ยังไง: Built `measure\_capacity.py` (read-only, no GPU) that computes visual tokens for every image in the set from file dimensions, and recorded the contradiction in the workflow doc rather than quietly overwriting the old row.

กฎที่ได้: Never characterise a dataset from one sample. Any claim of the form "the images are size X" must come from a script that measures every item and reports the min/max/count over the cap.

Eval scored ids with raw exact string match, so trailing whitespace or case scored a correct answer as 0 (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `infer\_house\_t03.py:218` scored with `set(gt\_ids) & set(pred\_ids)` — exact string match with no normalization. A model answering `"B1 "` against GT `"B1"` scored zero. Self-check confirmed a fully correct model with stray whitespace/case scored 0% under the old scorer.

ต้นเหตุจริง: The scorer was written as a one-line set intersection with no normalization layer, treating model output as if it were canonical data rather than free-form text.

เสียหายอะไร: Every reported recall number across t02/t03 history understates the model's real capability by an unknown margin. Tuning decisions were made against a metric that punishes formatting noise as if it were a reading failure.

แก้ยังไง: Researched the standard first (KIE evaluation practice, KIEval arXiv:2503.05488, which reports exact and relaxed matching as separate columns), then added `norm\_id()` (strip / collapse whitespace / casefold) before intersection. Old metric kept as a column for cross-round comparability.

กฎที่ได้: Any string-comparison metric over model output needs an explicit normalization step, and the normalized and raw scores should be reported side by side. Check published evaluation practice for the task type before inventing a scorer.

Ground truth element_id mixed real printed marks with names the annotator invented, teaching the model to guess the unguessable (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: GT `element\_id` contained both marks genuinely printed on the drawing (`B1`, `B3X`, `C1`, `F1A`, `RB1`) and descriptive names the annotator invented to distinguish sub-details (`gate\_front`, `SP\_at\_bathroom`, `purlin\_rafter\_connection`, `door\_lock\_detail`, `ห้องนอน`, `ระดับพื้นอาคาร`). No model can produce those strings — there is nothing on the paper that says them. Measurement showed 300/455 section training examples (66%) teach the model to guess invented names, half of them 100% invented.

ต้นเหตุจริง: The schema's §0.2 actively instructed annotators to invent a descriptive name and forbade leaving the field blank, so the convention that made the field unlearnable was written into the spec itself.

เสียหายอะไร: Section id-recall pinned at 0% with a real ceiling of only 37% dataset-wide and 4% on the test house — so the number carried almost no information. Worse, 66% of section training examples were actively training the model to hallucinate names.

แก้ยังไง: After explicit approval, batch-changed 343 section files: 1,534 elements moved to `element\_id: null` with the original name preserved in a new `element\_name\_assigned` field (902 genuinely-printed marks kept, exactly matching the prior measurement). `strip\_assigned()` added to build_dataset_t03.py so the invented name never enters the training target; `pass2\_used/section.md` and schema §0.2 rewritten to say "mark present → copy it, no mark → null, never invent".

กฎที่ได้: A ground-truth field must only contain what is physically present in the input. If an annotator has to invent a value to fill a field, the field is teaching hallucination — use null and put the human-readable label in a separate, non-target field.

Harvest round 2 reported a perfect 44/44 that was a fake number from a loose threshold and one-sided check (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The second harvest round reported "44/44 ". It was fake: the threshold was loosened to 0.55 and the pass criterion checked only one direction (detected $\geq$ 70% of GT), so massive over-detection counted as success — house 27 has GT 20 and the tool detected 293, and still passed.

ต้นเหตุจริง: A pass criterion with only a lower bound. Combined with a loose threshold this rewards over-detection, since detecting everything guarantees you detect at least 70% of the truth.

เสียหายอะไร: A perfect-looking coverage number that was meaningless. The diary explicitly notes this is the same bug class as the section metric found earlier the same evening — the team produced two broken metrics in one day.

แก้ยังไง: Set HARVEST_THRESH = FOOTING_THRESH (0.72) and measured both directions ( 70–160% /  under /  over). Honest result: 23 correct / 19 under / 2 over.

กฎที่ได้: A pass criterion must be bounded on both sides. A one-sided "at least X%" check rewards over-detection and will report perfect coverage for a detector that fires on everything.

วิกฤต 2026-08-25

The schema itself instructed annotators to invent descriptive names and forbade leaving the field blank (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: ``primary_rawjson_schema.md`` §0.2 said the opposite of what was needed: "ตั้งชื่อบรรยาย ห้ามเว้น" — invent a descriptive name, never leave blank. This is the written instruction that created the 66%-invented-names training problem in the section subtask.

ต้นเหตุจริง: A schema rule was written to guarantee a non-empty field (nice for data completeness) without considering that the field is a model training target, so completeness was bought at the cost of teaching hallucination.

เสียหายอะไร: Root cause of the entire section-0% bottleneck: 300/455 training examples, 1,534 elements across 343 files. Required a full GT batch migration to undo.

แก้ยังไง: Rewrote §0.2 to the repo's existing convention ("not on the paper = null, not an invented name"), logged the change in ``primary_rawjson_schema_edit_log.md`` per rule 7, and updated ``pass2_used/section.md`` to teach the model the same rule.

กฎที่ได้: A schema rule that forbids null on a field the model must predict is an instruction to hallucinate. Check every required-field rule against the question "can this always be read off the input?" before writing it.

วิกฤต 2026-08-29

build_dataset was modified to put val houses (01-05) into train too — val stopped being held out (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A code comment records "2026-08-29 มะขามสั่ง: val houses (01-05) go in train too". `train.jsonl` is now 956 rows (727+229), val 229, test 33. Val is no longer a clean held-out set — the model sees the exam during training, so val numbers will be inflated by memorization and only measure fit.

ต้นเหตุจริง: A split change was made directly in code with only a comment, and it was discovered incidentally by a later session rather than announced. The consequence for metric interpretation was not recorded alongside the change.

เสียหายอะไร: Every val metric from this point on overstates real performance and cannot be compared to prior rounds' val numbers. Only test (house 08, held out from both) remains a valid cross-round benchmark — and that was itself later dissolved (see the TEST_HOUSES entry).

แก้ยังไง: Flagged in the diary with an explicit statement that val now measures fit only, and that test remains valid.

กฎที่ได้: A train/val split change is a measurement change and must be announced with its consequence for every metric, not left as a code comment. Never let contaminated val numbers be compared against clean historical ones.

House 08 pulled into training — the cross-round benchmark deliberately protected on 2026-08-24 is now gone (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `TEST\_HOUSES = set()` was set permanently and house 08 moved into train (987→1020 examples), reversing the 2026-08-24 decision that had specifically retrained a model to keep house 08 held out as a t02-vs-t03 benchmark.

ต้นเหตุจริง: Data volume was prioritized over measurement integrity. The earlier decision had cost a retrain (~\$0.35 plus restart) specifically to protect this benchmark, and that cost was not weighed against the 33 examples gained.

เสียหายอะไร: No held-out house remains that all rounds share, so no future model can be compared to t02 or t03 on clean ground. Combined with val contamination (above), there is now *no* uncontaminated evaluation set at all.

แก้ยัง: Not fixed — recorded as a deliberate reversal with the benchmark loss accepted.

กฎที่ได้: Before dissolving a held-out benchmark, check what it cost to establish and what comparison becomes impossible without it. 33 extra training examples is not worth the ability to measure progress.

ชุด validation ปนเปื้อน 100% — บ้านทั้ง 5 หลังที่ใช้วัดผลอยู่ในชุด train ด้วย ตัวเลข recall ทั้งหมดจึงวัดแค่การท่องจำ ไม่ใช่ความสามารถจริง

เกิดอะไรขึ้น: เขียนสคริปต์ดึงชื่อบ้านจาก path รูปใน train.jsonl เทียบกับ val.jsonl แล้วพบว่าบ้านทั้ง 5 หลังในชุด val (บ้าน_เล็ก_1ชั้น_01/_02, บ้าน_เล็ก_2ชั้น_01/_02/_03) อยู่ในชุด train ด้วยทั้งหมด เรื่องนี้เป็นการตัดสินใจที่ตั้งใจไว้ตั้งแต่ยุค t03 (val = บ้านที่คนตรวจแล้ว คุณภาพ GT สูงสุด เอาไว้ทดสอบว่าโมเดลตอบคำตอบที่รู้อยู่แล้วได้ไหม) แต่กลับถูกอ่านเป็นตัวเลขวัดการ generalize

ต้นเหตุจริง: ชุดแบ่งข้อมูลที่เราสร้างมาเพื่อจุดประสงค์หนึ่ง (เช็คว่ามีโมเดลตอบคำตอบที่รู้อยู่แล้วได้ไหม) ถูกเอาไปใช้ซ้ำเพื่ออีกจุดประสงค์หนึ่ง (วัดว่า generalize ได้แค่ไหน) ขำรุ่นโมเดลไปเรื่อยๆ โดยไม่มีใครกลับไปตรวจซ้ำว่าบ้านซ้ำกันไหมตอนที่จุดประสงค์เปลี่ยน

เสียหายอะไร: ตัวเลข recall ทุกตัวที่รายงานจาก val ก่อนหน้านี้สูงเกินจริงหมด และไม่ได้บอกอะไรเลยเกี่ยวกับแบบที่โมเดลไม่เคยเห็น แถมยังทำให้คำแนะนำเดิมที่ Claude เคยให้ไว้ ('ใช้ split เดียวพอ จะได้เก็บงบไว้ทำ eval') เสียน้ำหนัก จนต้องวางแผนใหม่กลางทางไปเป็น k-fold

แก้ยัง: เปลี่ยนไปใช้ k-fold จริงจาก manifest k=5 ที่มีอยู่แล้ว และตรวจสอบให้ครบก่อนเสียเงิน: fold0 = 815/205, fold1 = 816/204 ตรงกับ fold_manifest เป๊ะ, ไม่มีบ้านซ้ำกันเลยทั้งสอง fold, ไม่มีรูปหาย, จำนวน token ``<image>`` เท่ากับ len(images) ทุกแถว, สัดส่วน train:val = 4.00:1 พอดี

กฎที่ได้: ทุกครั้งที่เอาชุดแบ่งข้อมูลเดิมมาใช้ซ้ำ ต้องเขียนสคริปต์ตรวจว่าซ้ำกันในระดับ 'บ้าน' (ระดับตัวตนจริง ไม่ใช่ระดับแถว) และห้ามพูดตัวเลข recall ออกไปเลยถ้ายังไม่ได้พิสูจน์ว่าชุดวัดผลไม่ทับกับชุดเทรน

prompt ที่ใช้ทดสอบก่อนตัดสินใจ มีเฉลยปนอยู่ในตัวคำถาม โมเดลที่ตาบอดสนิทเลยผ่าน 3/3

เกิดอะไรขึ้น: prompt เวอร์ชัน v1 ที่ใช้เป็นด่านตรวจ vision มีตัวอย่างเขียนไว้ว่า 'เช่น F1, B2, C1, พ2' ซึ่งบังเอิญ F1 กับ พ2 อยู่ในเฉลยจริง โมเดลก็แค่ลอกตัวอย่างกลับมาทั้งต้น ตอบสตรึงเดียวกันเป๊ะทั้งกับแบบฐานรากและแบบ section แล้วสคริปต์ให้คะแนนก็นับว่าถูก อีกหน้าหนึ่ง 'ผ่าน' เพราะไล่จับ B1..B34 ไปเรื่อยๆ แล้วบังเอิญตัวแรกๆ ตรงกับมาร์คที่ถูก

ต้นเหตุจริง: ออกแบบข้อสอบโดยปล่อยเฉลยหลุดเข้าไปในคำถาม และตัวให้คะแนนใช้แค่การเช็คว่ามีสตรึงนั้นอยู่ไหม โดยไม่เช็คเลยว่าเป็นพุดที่ต่างกันต้องให้คำตอบที่ต่างกัน

เสียหายอะไร: โมเดลที่มองไม่เห็นอะไรเลยได้คะแนนเชียว 3/3 ถ้าเชื่อผลนี้จะพาทั้งคืนไปกับโมเดลที่อ่านอะไรไม่ออก ซึ่งจะเป็นการซ้ำรอย t04 แบบเป๊ะๆ

แก้ยังง: เขียน prompt ใหม่เป็น v2 (ตัดตัวอย่างออก, หน้าที่แตกต่างกันต้องให้คำตอบต่างกัน, ปฏิเสธคำตอบที่ไล่จับเรียงลำดับ) แล้วต่อด้วย v3 ที่ถามสามคำถามต่อหน้า และไม่เปิดช่องให้ตอบว่า 'อ่านไม่ออก' บันทึกไว้เป็น rule_of_tune บทเรียนที่ 16 ข้อ 1 และ 2

กฎที่ได้: ห้ามใส่คำหรือคำใดๆ ที่ปรากฏในเฉลยลงไป prompt ที่ใช้ทดสอบเด็ดขาด และต้องเขียนตรวจเสมอว่าอินพุตต่างกัน แล้วคำตอบต้องต่างกัน — คำตอบที่เหมือนกันเป๊ะข้ามอินพุต คือลายเซ็นของโมเดลที่ไม่เคยมองรูปเลย

เอา fold ของ t04 มาใช้ซ้ำ ทำให้ชุด validation ไม่มีแถว pass0 เลย จนวัดสมมติฐานหลักของรอบนั้นไม่ได้

เกิดอะไรขึ้น: หลังตัดสินใจใช้ fold ของ t04 เป็นแหล่งความจริงเดียว ปรากฏว่าชุด validation แทบไม่มีแถวของ pass0/pass2.4/pass3 เลย โดย fold1 ได้ 0/0/0 เพราะ t04 แบ่งบ้าน 40 หลังโดยไม่รู้ว่ามีแค่ 10 หลังที่มีข้อมูล pass0/2.4/3 (เฉพาะ pass0 มีแค่ 5 หลัง) บ้านพิเศษเหล่านั้นจึงตกไปอยู่ฝั่ง train หมด

ต้นเหตุจริง: เอาการแบ่งข้อมูลที่ออกแบบมาสำหรับชุดข้อมูลที่กระจายเท่าๆ กัน ไปใช้กับชุดข้อมูลที่มีบางชนิดหายากมาก โดยไม่มีการ stratify ตามความมีอยู่ของข้อมูลแต่ละชนิด

เสียหายอะไร: GPU ทั้ง 4 ตัวจะเรนไปแล้วได้ผลที่อ่านไม่ได้ เพราะไม่มีทางวัดได้เลยว่าโมเดลเรียนรู้ pass0/pass3 หรือเปล่า ซึ่งเป็นสมมติฐานทั้งหมดของรอบนี้ จับได้เพราะมั่วๆ ไม่ใช่เพราะทิม

แก้ยังง: เขียน build_house_folds() ที่กระจายบ้านแบบ deterministic ในระดับบ้าน (บ้าน pass0 5 หลัง กระจาย fold ละหลัง, บ้านข้อมูลเยอะที่ไม่ใช่ pass0 5 หลัง กระจาย fold ละหลัง, บ้านธรรมดา 30 หลัง fold ละ 6 หลัง) ทำให้ทุก fold มีชุด val ที่มีครบทั้ง 4 pass

กฎที่ได้: แบ่ง fold โดยกระจายตามการมีอยู่ของข้อมูลชนิดหายากที่เรากำลังจะวัดจริงๆ และก่อนกดรันทุกครั้ง ให้พิมพ์จำนวนข้อมูลแต่ละชนิดในชุด val ของแต่ละ fold ออกมาดู — ถ้าช่องไหนเป็นศูนย์ แปลว่ารอบนั้นตอบคำถามของตัวเองไม่ได้

การ probe หา VRAM กรณีแย่มากที่สุด ประเมินต่ำกว่าจุดสูงสุดจริงตอนเทรน เพราะไม่ได้สะสม fragmentation

เกิดอะไรขึ้น: probe แบบ SORT_LONGEST วัดจุดสูงสุดได้ 91.4 GB แล้วถูกใช้เป็นหลักฐานว่าความเสี่ยง OOM หหมดไปแล้ว พอเทรนจริง gpu1 (fold0) และ gpu4 (fold3) OOM ทั้งคู่ ส่วนอีกสองตัวที่รอดก็ค้างอยู่ที่ 93.7 และ 94.66 GB จากเพดาน 94.97 GB

ต้นเหตุจริง: probe รันแค่ 2 ตัวอย่างแบบแยกเดี่ยวบน process ที่เพิ่งเริ่มใหม่ แต่ตัวจัดสรรหน่วยความจำแบบแคชของ PyTorch สะสม fragmentation ไปเรื่อยๆ ทุก step และไม่เคยคืนที่ จุดสูงสุดจริงจึงเกินค่าที่ probe สั้นๆ วัดได้เสมอ

เสียหายอะไร: 2 ใน 4 fold ล้ม เสียเวลาประมาณ 1 ชั่วโมงไปกับการ probe ใหม่และรีสตาร์ททั้งสี่ตัว และต้องทิ้ง checkpoint ที่เทรนไปได้บางส่วน

แก้ยังไง: ลด MAX_PIXELS จาก 7680 เหลือ 6912 เท่ากันทุก fold แล้ว probe ใหม่ด้วย SORT_LONGEST TEST_STEPS=3 บน gpu1 ยืนยันว่าจุดสูงสุดลดจาก 91.4 เหลือ 80.6 GB ก่อนจะรีสตาร์ท

กฎที่ได้: probe สั้นๆ วัดได้แค่ 'พื้น' ไม่ใช่จุดสูงสุดจริง ต้องเผื่อ margin ให้ fragmentation อย่างน้อย 10% เหนือค่าที่ probe ได้ และต้องยืนยันด้วยตัวเลขจริงก่อนจะเชื่อว่าแก้ปัญหาหน่วยความจำได้แล้ว

คะแนนรูปแบบ JSON 96–100% กลบข้อเท็จจริงที่ว่าโมเดลกำลังตอบเรื่องบ้านคนละหลัง

เกิดอะไรขึ้น: การวัดผลของ t04 รายงานว่า JSON ถูกรูปแบบ 96–100% ทุกครั้ง ซึ่งดูเหมือนโมเดลสุภาพดี จนกระทั่งมีคนเปิดไฟล์คำตอบจริงมาอ่านด้วยตา ถึงพบว่าป้อนแบบฐานรากเข้าไป แต่มันตอบเป็นแปลนหลังคาของบ้านคนละหลังไปเลย

ต้นเหตุจริง: ตัวชี้วัดที่ใช้วัดแค่ 'รูปทรง' ของผลลัพธ์ (JSON ถูกต้อง มีฟิลด์ครบ) ไม่ได้วัดว่าเนื้อหาจริงหรือเปล่า โมเดลที่ท่องจำรูปแบบคำตอบแล้วฟันคำตอบจากชุดเทรนกลับมา จึงได้คะแนนเกือบเต็มทั้งที่ไม่เข้าใจอะไรเลย

เสียหายอะไร: ความล้มเหลวถูกปิดบังไว้ทั้งรอบการเทรน ทีมเชื่อว่าโมเดลใช้ได้ ทั้งที่มันตาบอดสนิท และยังเสียเวลากับเงินต่อไปภายใต้สัญญาเช่าเซิร์ฟเวอร์

แก้ยังไง: เปลี่ยนมาใช้วิธีเปิดไฟล์คำตอบจริงมาอ่านด้วยตา แทนที่จะเชื่อคะแนนรวม บันทึกไว้เป็นบทเรียนข้อ 2 และ 3 ในสรุป

กฎที่ได้: ห้ามตัดสินโมเดลจากตัวชี้วัดเรื่องรูปแบบ/โครงสร้างอย่างเดียว ทุกรอบการวัดผลต้องมีคนเปิดผลลัพธ์จริงมาอ่านด้วยตา — คะแนนรูปแบบเป็น 100% ได้ทั้งที่ความถูกต้องของเนื้อหาเป็น 0%

ด้านตัดสินใจราคา \$2 ตอนแรกขึ้นเขียวปลอม เพราะ prompt ที่ใช้ทดสอบมีเฉลยหลุดอยู่

เกิดอะไรขึ้น: การรันด้านตรวจก่อนเทรดครั้งแรกผ่าน พอไปสืบดูพบว่า prompt ที่ Claude เขียนใส่ตัวอย่างไว้ว่า 'เช่น F1, B2, C1, ผ2' แล้วบังเอิญ F1 กับ ผ2 เป็นคำตอบจริงในเฉลย โมเดลที่ตาบอดจึงแค่สะท้อนตัวอย่างกลับมา แล้วสคริปต์ให้คะแนนก็นับว่าถูก ส่วนอีกหน้าหนึ่งโมเดลไล่ับ B1, B2, B3 ... ไปจนถึง B34 แล้วสามตัวแรกบังเอิญตรงกับเฉลยพอดี

ต้นเหตุจริง: มีข้อผิดพลาดสองอย่างซ้อนกัน: (1) คนเขียน prompt ใส่ตัวอย่างที่เป็นค่าจริงลงใน prompt ที่มีไว้ทดสอบว่าโมเดลอ่านออกไหม ทำให้ข้อสอบปนเปื้อน (2) สคริปต์ให้คะแนนนับทุก token ที่ตรง โดยไม่เชื่อว่าเป็นการไล่ับหรือสะท้อนคำตอบกลับมา การเดาเรียงลำดับแบบสุ่มมั่วจึงได้คะแนนบางส่วนไปด้วย

เสียหายอะไร: เกือบเผาค่าเช่า GPU ทั้งคืน (ประมาณ \$60-70) ไปกับโมเดลที่มองไม่เห็นอะไรเลย ซึ่งจะเป็นการช้ำรอย t04 เป๊ะๆ จับได้เพราะมีคนสงสัยผลที่ขึ้นเขียวเท่านั้น

แก้ยังง: เขียน prompt ของด้านตรวจใหม่โดยเอาตัวอย่างออก แล้วรันซ้ำ คราวนี้โมเดลได้ 0/3 จึงหยุดสาขานี้ไป พร้อมตั้งกฎใหม่ว่า ห้ามใส่ค่าที่ปรากฏในเฉลยลงใน prompt ที่ใช้ทดสอบ

กฎที่ได้: prompt ที่ใช้ทดสอบต้องไม่มีตัวอย่างที่หยิบมาจากเฉลยเด็ดขาด และตัวให้คะแนนต้องปฏิเสธคำตอบที่แค่สะท้อนกลับหรือไล่ับเรียงลำดับ ถ้าด้านตรวจราคาถูกขึ้นเขียวแบบไม่คาดคิด ให้ถือว่าน่าสงสัยไว้ก่อน จนกว่าจะอธิบายได้ว่ามันผ่านเพราะอะไร

pass3 required exactly one grid_ref per element, so grouped records (count > 1) produced zero anchors (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Ground truth aggregates identical items into a single record with a list of refs (e.g. `F1 · count 11 · grid\_refs [11 points]`), but the code enforced `len(refs)==1` and skipped anything else — yielding 0 anchors even though CV had found 12 matching points exactly.

ต้นเหตุจริง: The code was written against the shape produced by the *first* simulation, in which Claude hand-wrote the model answers as one ref per element. That artificial shape never exercised the grouped form that real GT uses.

เสียหายอะไร: pass3 found no anchors on pages where perfectly good data existed. More importantly, the bug was completely invisible to the first simulation.

แก้ยังง: Handled multi-ref records. The bug was only exposed because Makham ordered the second simulation to use the real GT answers instead of Claude-authored ones.

กฎที่ได้: Never build a simulation whose inputs you author yourself — hand-written fixtures encode your own assumptions and mask exactly the bugs the simulation exists to find. Feed simulations real ground-truth data.

วิกฤต 2026-09-01

A validation gate passed green because the prompt itself contained the ground-truth answer (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The first round of gate checks showed green — but only because the prompt had the answer written into it. The evaluation was measuring the prompt, not the model.

ต้นเหตุจริง: Ground truth leaked into the evaluation prompt. No separation between the data used to construct the prompt and the data used to score the answer.

เสียหายอะไร: A false 'model works' signal that would have let a broken model through. Time lost on a measurement that meant nothing.

แก้ยังไง: Identified and documented as a trap in the teaching document.

กฎที่ได้: Before trusting any green eval, diff the prompt against the expected answer — if the answer text appears in the prompt, the score is meaningless. Build eval prompts from a separate code path than the scoring data.

วิกฤต 2026-09-01

The headline '191/191, 100%' pass3 result was measured on human-vetted pages; automatic CV on an unchecked house failed 5/5 (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: pass3 was validated at 191/191 leave-one-out accuracy — but on 18 pages a human had already eyeballed and confirmed had good anchors. Run against house02 with fully automatic CV on a house never checked before, all 5 structural pages failed: 2 footing pages rejected by the quality gate, 3 beam pages found 0 anchors.

ต้นเหตุจริง: The validation set was selected for quality, which silently converted the question from 'does pass3 work on real drawings?' to 'does the fit maths work on good anchors?'. The upstream anchor-detection step — the part that actually fails — was excluded from the measurement.

เสียหายอะไร: A headline accuracy figure that overstated real-world capability by a wide margin. Would have led to trusting pass3 in production.

แก้ยังไง: Re-measured honestly on unfiltered CV output on a fresh house and recorded the 0/5 result plainly alongside the 191/191.

กฎที่ได้: Never validate a pipeline stage on data that was pre-selected as clean by a human — you measure the wrong stage. Always report the end-to-end number on unfiltered input next to any curated-subset number.

Two full extraction rounds were produced but never systematically compared, so repeat-read consistency was never measured (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Round t2 was explicitly instructed to read the images fresh and independently and to report where it differed from t1, specifically to see how consistent repeated reads are. A systematic file-by-file t1 vs t2 comparison was never done — only the agents' own summary reports exist.

ต้นเหตุจริง: The comparison was the measurement that justified doing a second full round, but it was scheduled after the round rather than built into it, and the day's remaining time went to Label Studio tooling instead.

เสียหายอะไร: The cost of a whole second extraction round was paid without collecting its main deliverable — a real number for repeat-read agreement. Without it, per-field reliability is unknown and the t1/t2 conflicts (MEP grid, element vs elements) were found by luck rather than by a systematic diff.

แก้ยังไง: Not fixed — listed as 'should do a t1 vs t2 comparison summary for Makham to see the overall quality picture' and 'has not been done yet'.

กฎที่ได้: If a round of work exists to produce a measurement, build the measurement into the round's definition of done. A diff between two extraction rounds should be an automated script run at round close, not a summary task that competes with feature work for time.

Round 2 (t2) was not an independent re-read: six structural pages were copied verbatim from t1, and agents were handed t1 to diff against (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: t2 was set up as a second fresh extraction whose stated purpose included measuring how consistent repeated reads are. But pages 18, 21, 22, 23, 24 and 25 were copied straight from t1/ into t2/ with no re-read, and the agents doing pages 1-17 and 26-37 were explicitly instructed to report where their reading differed from t1 — i.e. they had access to round 1 while producing round 2.

ต้นเหตุจริง: Speed was prioritised over experimental hygiene, with the justification that those pattern types were 'unaffected by this round's schema update' — true for the schema, but the consistency measurement is about re-read agreement, which a copy trivially and falsely scores at 100%. Nobody separated 'pages that need a schema-driven redo' from 'pages needed as a blind control'.

เสียหายอะไร: Any t1-vs-t2 agreement number computed later would be inflated: six pages would show perfect agreement by construction, and the remaining pages' agreement was measured against a round that was not blind. Repeat-read reliability — the number that would justify trusting the extraction pipeline — cannot be derived from this data at all.

แก้ยังไง: not fixed — the copied pages were never re-read independently, and the systematic t1 vs t2 comparison was still listed as pending at end of day

กฎที่ได้: If a round is meant to measure consistency, every page in it must be re-read blind. Never copy files into a comparison set, and never show round 1 to the reader producing round 2 — keep a schema-redo pass and a blind-control pass as separate artifacts.

สูง 2026-07-07

Gen 1 extraction recorded concrete grade 240 ksc when the drawing says 210 ksc (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A conflict pending since July 5 over whether page 18 (S-01 notes) specifies $F_c' = 210$ or 240 kg/cm^2 . A fresh independent re-read during the t3 round returned a clear 210 ksc, matching Makham's own first-hand read on July 5. Gen 1's 240 was the wrong value.

ต้นเหตุจริง: The original Gen 1 pass recorded a structural constant from a single unverified read with no second source and no confidence flag, so a misread became a fact in the dataset.

เสียหายอะไร: A wrong concrete grade sat in candidate fine-tuning ground truth for two days; propagated into any downstream BOQ/strength assumption drawn from it.

แก้ยังไง: Re-read page 18 fresh; two independent reads agreed on 210 ksc, so Gen 1's 240 was overruled.

กฎที่ได้: Any single structural constant that governs the whole house (concrete grade, cover, lap length) needs two independent reads before it is treated as ground truth — one read is a hypothesis, not data.

สูง 2026-07-11

รายการ BOQ ของบ้านหลัง 2 หลายรายการถูกบันทึกที่ confidence 0.4-0.6 เพราะตัวหนังสือเล็กและหมุน 90 องศา

เกิดอะไรขึ้น: รายการในหมวดตกแต่ง สี และสุขภัณฑ์ ถูกสกัดออกมาที่ confidence 0.4-0.6 เพราะตัวหนังสือต้นฉบับเล็กมากและพิมพ์หมุน 90 องศา ตอนนี้อยู่ในชุดข้อมูลแล้วและติดธงไว้ว่าต้องมีคนไปชุมตรวจสอบก่อนใช้เป็น ground truth จริงจัง

ต้นเหตุจริง: การสกัดข้อมูลอ่านหน้ากระดาษตามที่เรนเดอร์มาเลย ไม่มีขั้นตอนเตรียมภาพ (หมุน/ครอบ/ขยาย) สำหรับบริเวณที่ต้นฉบับพิมพ์ตะแคงและตัวเล็ก

เสียหายอะไร: มีข้อมูล ground truth ก้อนหนึ่งที่มีโอกาสผิดมากกว่าถูก นอนอยู่ในชุดข้อมูลที่จะเอาไป fine-tune โดยที่การตรวจสอบยังไม่มีคิว

แก้ยังไง: ยังไม่ได้แก้ — บันทึกเป็นงานตรวจสอบค้างข้อ 2: ชุมเข้าไปตรวจรายการที่ confidence ต่ำ

กฎที่ได้: ถ้า confidence ตกเพราะวิธีเรนเดอร์หน้ากระดาษ ให้ไปแก้ที่การเรนเดอร์ (หมุน/ครอบ/ขยาย แล้วอ่านใหม่) แทนที่จะส่งแถวที่ติดธง confidence ต่ำออกไป และห้ามให้แถวที่ติดธงเข้าไปอยู่ในข้อมูลเทรนโดยไม่มีคิวตรวจ

สูง 2026-07-24

Local dry run used a 10.8GB stand-in, so it proved 'it runs' but never that it runs usably (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Phase 0.3 on 2026-07-24 validated local inference by downloading a small ~10.8GB test model, running llama.cpp against a real drawing page, and declaring 'ผ่าน'. The real artifact turned out to be 21.2GB — roughly double — and on the same 8GB-VRAM laptop it ran at 3.46 token/s, 13-21 minutes per page, with llama.cpp logging 'failed to fit params to free device memory ... abort'.

ต้นเหตุจริง: The dry run was sized for convenience, not for the artifact it was supposed to represent, and its pass criterion was binary ('does it run?') with no throughput threshold. A 10.8GB model fits VRAM far better than 21.2GB, so the CPU-offload bottleneck could not appear.

เสียหายอะไร: The team walked into the 2026-07-29 real run believing local execution was proven, and only then discovered a full house at ~18 min/page is impractical — forcing an unplanned 2-phase classify-then-extract redesign and a mis-set timeout that killed a healthy job.

แก้ยังง: Not fixed as a process rule; worked around on 2026-07-29 by splitting into a fast classify pass plus selective extraction (extract_house01_local.py).

กฎที่ได้: A dry run must use an artifact of the real size on the real hardware, and must assert a usable rate (sec/page), not just absence of a crash.

สูง 2026-07-24

The Phase 4 baseline eval was skipped, so the headline '3x better' figure compares against a different run's baseline (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Phase 4 (the untuned-baseline measurement) was skipped to save time on the recommendation of the workflow doc. The Phase 7 results were then reported as element recall 28.2% '↑ ↑ ↑ from 9.4%' and view exact match '70% ↑ from 60%' — numbers that come from the previous training round on different hardware and a different dataset size (226 train vs 315 train).

ต้นเหตุจริง: A baseline is treated as a cost rather than as the denominator of every claim in the run. Skipping it makes the improvement unmeasurable within the run, so a prior run's numbers get substituted silently.

เสียหายอะไร: The central quality claim of the whole tuning cycle rests on a cross-run comparison whose baseline was measured under different conditions — the improvement could be larger or smaller than reported and nothing in this run can tell.

แก้ยังง: not fixed — Phase 4 remains skipped and the comparison numbers stand as reported.

กฎที่ได้: Never skip the baseline in the run whose improvement you intend to quote. A baseline from another run is a different denominator, and mixing them makes every percentage in the report unfalsifiable.

The A/B parity table compared only explicitly-set values, so an unset default slipped through (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A detailed parity table was built to guarantee t01 and t02 differed only in the model. It listed learning rate, batch size, epochs, etc. It could not catch the resize difference because t02 set resize explicitly while t01 set nothing at all — there was no line to compare.

ต้นเหตุจริง: The comparison was constructed from remembered/intentional choices rather than from the full argument list of every shared function and class. Unset arguments are still variables, and are the easiest to overlook precisely because they are invisible.

เสียหายอะไร: Gave false confidence that the experiment was controlled, allowing an invalid conclusion to stand until a separate deep audit found it.

แก้ยังไง: Recorded as Lesson 12 in rule_of_tune.md: an A/B parity table must enumerate every argument passed to every shared function/class, read from source, not from memory of what was changed.

กฎที่ได้: Build A/B parity tables by diffing the effective arguments of every shared call (including defaults) read straight from source — never from the list of things you meant to change.

The t01 model was tried on house 01, which is inside its own training set (62 examples) — results look better than reality (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The tuned t01 model was run against house 01 pages. House 01 contributed 62 examples to the training set, so the model has memorised this material. The output is therefore not a measurement of generalisation.

ต้นเหตุจริง: The most convenient house to hand was used for a quality trial without first checking whether it was held out. Train/test separation was not enforced at the point of running an evaluation.

เสียหายอะไร: Caught and flagged in the diary before anyone quoted the numbers as performance — but had the caution not been written, in-sample results would have circulated as capability evidence.

แก้ยังไง: Explicitly labelled as usable as a smoke test only, never as a measure of generalisation.

กฎที่ได้: Before quoting any model output as a quality result, check whether the input is in the training set and label it. In-sample output is a smoke test; only held-out data measures capability.

สูง 2026-07-28

A healthy falling loss curve (1.795 → 0.18-0.21) accompanied a model that scored only 20.6% element recall (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: t02's training loss fell cleanly from 1.795 to ~0.18-0.21 and eval_loss reached 0.2076 — every indicator of successful learning. Phase 7 field-level measurement then showed JSON valid 65%, element recall 20.6%, losing to t01 on every metric.

ต้นเหตุจริง: Loss measures mathematical closeness to the target token sequence; it does not measure whether rebar numbers or part names were read correctly. Treating it as a quality signal conflates optimisation progress with task performance.

เสียหายอะไร: A model that looks trained by every training-time indicator but fails the actual acceptance criterion — the exact trap that makes a premature 'it worked' claim feel justified.

แก้ยังง: Phase 7 field-by-field measurement is the gate, not loss; recorded as lesson 4 of the old CLAUDE.md and reinforced in rule_of_tune.md.

กฎที่ได้: Never conclude anything about model quality from the loss curve. Only a field-level measurement against held-out data decides whether a round succeeded.

สูง 2026-07-28

Two contradictory sets of t01 headline numbers coexist in the same document, and the eval ran on only 20 of 88 val examples (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Section 14 (2026-07-21) records t01 as JSON valid 90% / view 60% / element recall 9.4%. The t02 comparison table (2026-07-28) records the same model as JSON valid 90% / view 70% / element recall 28.2%. The document never reconciles the 60-vs-70 and 9.4-vs-28.2 discrepancy, yet 28.2% is the figure quoted throughout as 'the number we trusted'. The eval output shown is also '0/20' and '0/170' — 20 examples out of the 88-line val set.

ต้นเหตุจริง: Evaluations were run at different times with different --limit values and possibly on different machines, and each run's headline was written into the diary without stating its sample size or superseding the previous one.

เสียหายอะไร: A reader (and the team itself) cannot tell which t01 number is authoritative; the recall figure that drove the entire 'Qwen3.6 beats Qwen3-VL' decision differs by 3x between two places in the same document, and may rest on ~23% of the validation set.

แก้ยังง: Not fixed — both numbers still stand side by side in the merged teaching document with no note explaining the difference.

กฎที่ได้: Every recorded metric must carry its sample size and run identity, and a re-measurement must explicitly supersede the old number rather than silently sitting next to it.

สูง 2026-07-29

The one capability the project exists for — rebar extraction — has never been tested (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: No structural (S-*) page has ever been fully extracted; pages 07-11 were only classified. The prompt's single biggest rule ('always split top/bottom rebar') therefore has no evidence for or against it, after multiple tuning rounds.

ต้นเหตุจริง: Testing followed page order and convenience rather than starting from the highest-value capability, so the core requirement kept being deferred behind cheaper pages.

เสียหายอะไร: After three tuning rounds and days of evaluation the team still cannot say whether the model can do its actual job.

แก้ยังไง: not fixed — no S-* page extracted yet

กฎที่ได้: Test the highest-value capability first, on day one, even with a rough harness. A quality programme that has not exercised the primary use case has measured nothing that matters.

สูง 2026-07-29

Three candidate causes still entangled — none tested one variable at a time (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Merge failure, resolution mismatch and quantization damage are all plausible contributors to the quality collapse, and the diary records that none has been tested separately to see which weighs heaviest.

ต้นเหตุจริง: All three changed between the measured-good configuration (bf16 + unmerged adapter) and the running configuration (merged + Q4_K_M + different image resolution) — they were varied simultaneously, so no single measurement can attribute blame.

เสียหายอะไร: No way to prioritise fixes or estimate how much quality each fix would recover; risk of retraining expensively to fix something quantization or a flag would have solved for free.

แก้ยังไง: not fixed — explicitly listed as still to do, one variable at a time

กฎที่ได้: Never change more than one variable between a measured-good configuration and a deployed one. When several have already changed, spend the cheap experiments to isolate them before spending money on a fix — attribution first, remediation second.

สูง 2026-07-29

The trusted 90% / 28.2% numbers were measured on a path that is not the shipped artifact (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: eval_fields.py runs full bf16 (load_in_4bit=False) with the LoRA adapter attached at runtime and no merge. The artifact actually in use goes through merge (the suspect step) and Q4_K_M compression. The two paths were never compared, yet the eval numbers were quoted as the product's accuracy.

ต้นเหตุจริง: The evaluation harness was built around the training artifact, and no one asked whether the evaluated object was the same object being deployed.

เสียหายอะไร: Months of decisions rested on a number that describes an artifact nobody runs; the shipped GGUF has never had a measured accuracy at all.

แก้ยังไง: Clarified in the diary that 90%/28.2% remain valid for the adapter only, and that the GGUF is unverified. The tuning work itself is intact — the adapter is safe on HuggingFace.

กฎที่ได้: Every accuracy number must be labelled with the exact artifact and path it was measured on, and the shipped artifact must be evaluated on its own path before any number is quoted for it.

สูง 2026-07-29

The 'do not switch to Qwen3-VL' verdict may be wrong on the path that actually ships — t02's training resolution matches llama.cpp's runtime exactly (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On 2026-07-28 the A/B concluded Qwen3.6 (t01) beats Qwen3-VL (t02) on every metric and recorded 'อย่าเปลี่ยนไปใช้ Qwen3-VL'. The next night it emerged that t02 was trained at ~3,796 visual tokens per image, which is exactly what llama.cpp feeds at inference, while t01 was trained at ~266 — so on the real local deployment path t02 may in fact be the better model.

ต้นเหตุจริง: The comparison was run on the bf16 + runtime-adapter path (the measurement path), not on the merged/quantized GGUF path (the deployment path), and train/inference token parity was never a comparison criterion.

เสียหายอะไร: A recorded, apparently settled model-selection decision is now unreliable — and the alternative it rejected is currently unusable anyway because the same MoE merge bug blocks its GGUF export, so the merge bug now blocks both rounds rather than only t02.

แก้ยังไง: Not fixed — flagged as 'ยังไม่ได้พิสูจน์' in the open-questions section.

กฎที่ได้: Judge candidate models on the path you will actually deploy, and treat train/inference input parity as a first-class comparison variable, not an afterthought.

สูง 2026-08-04

eval_fields.py ไม่เคยบันทึกว่า element ตัวไหนที่พลาด — บันทึกแค่จำนวน

เกิดอะไรขึ้น: สคริปต์วัดผลบันทึกผลระดับหน้าอย่างเดียว (เช่น 'element 9/14') ไม่เคยเขียนออกมาเลยว่า element id ตัวไหนที่พลาด ทั้งที่ตอนจับคู่ข้อมูลนั้นอยู่ในหน่วยความจำอยู่แล้ว ('pmap`/ge`/pe`)

ต้นเหตุจริง: ตัวประเมินถูกสร้างมาเพื่อให้ตัวเลขคะแนนพาดหัว ไม่ได้สร้างมาเพื่อช่วยวินิจฉัย รายละเอียดรายตัวถูกคำนวณแล้วโยนทิ้งไปเลย

เสียหายอะไร: วัด recall ได้แต่วิเคราะห์ไม่ได้ — ไม่มีทางรู้ว่าที่พลาดไปกระจุกอยู่ที่คาน ที่มาร์คจางๆ หรือที่ pattern ใด pattern หนึ่ง ทุกครั้งที่พยายามปรับปรุงคือการเดา และต้องรันใหม่ทั้งรอบแค่เพื่อขอรายชื่อ

แก้ง่าย: ยังไม่แก้ — ระบุว่าต้องเพิ่มบรรทัด log แค่บรรทัดเดียว (ไม่ใช่แก้ตรรกะ เพราะกฎของ t02 ห้ามแตะตรรกะการวัดผล) แล้วรันใหม่

กฎที่ได้: ตัวประเมินต้องพบนายชื่อสิ่งที่พลาดออกมา ไม่ใช่แค่คะแนน ตัวเลขที่ตกย่อยไม่ได้คือตัวเลขที่ปรับปรุงไม่ได้ — ให้ log สิ่งที่พลาดตอนจับคู่เลย เพราะตอนนั้นข้อมูลอยู่ในมืออยู่แล้ว

สูง 2026-08-20

แผนวัดผลรอบก่อนเลือกบ้าน 01/03/04 — ซึ่งอยู่ในชุด train/val หมุดแล้ว เท่ากับวัดการท่องจำ

เกิดอะไรขึ้น: ร่างแผนทดลอง (ทดลองถอดแบบ_01_03_04_workflow.md) เลือกบ้าน 01/03/04 มาทดสอบความสามารถในการ generalize ของ t02 พอ grep เทียบกับ train.jsonl/val.jsonl พบว่าทั้งสามหลังอยู่ในข้อมูลเทรน เลยเปลี่ยนไปใช้บ้าน 08/09/10/11 หลังยืนยันว่าไม่อยู่ในชุดเทรน

ต้นเหตุจริง: เลือกบ้านสำหรับวัดผลตามความสะดวก (บ้านที่เข้าใจดีและมีเฉลยดี) โดยไม่ได้เช็กกับ manifest ของชุดเทรนแบบอัตโนมัติ และไม่มีตัวกันอัตโนมัติที่ห้ามเอาบ้านในชุดเทรนมาใช้วัดผล

เสียหายอะไร: จับได้ก่อนเสียเงินค่า GPU ถ้าปล่อยให้รันไป มันจะให้ตัวเลข recall ที่ดูดีแต่จริง ๆ วัดการท่องจำ แล้วตัวเลขนั้นจะถูกเขียนลง SUMMARY.md และมีคนเชื่อ

แก้ง่าย: ยืนยันด้วย grep ว่าไม่อยู่ใน train.jsonl/val.jsonl แล้วเปลี่ยนไปใช้บ้าน 08/09/10/11 พร้อมเขียน ทดลองถอดแบบ_08_09_10_11_workflow.md เป็นเอกสารเตรียมก่อนรันตาม rule_of_tune §4 ส่วนร่างเดิมปล่อยทิ้งไว้

กฎที่ได้: ทุกครั้งที่จะรันวัดผล ต้อง grep บ้านที่จะใช้เทียบกับ train.jsonl/val.jsonl ก่อนเข้า GPU และเอกสารเตรียมงานต้องบันทึกการเช็กนี้ไว้ด้วย — การวัดผลบนข้อมูลเทรนให้ตัวเลขที่แย่กว่าการไม่มีตัวเลขเลย

สูง 2026-08-24

Initial val split was 10:1 — too small to measure anything, especially plan_beam (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The dataset was first built with val = house03 only, giving a 10:1 train:val ratio and just 4 plan_beam validation examples. Makham overruled it ("4/1 ดีกว่า").

ต้นเหตุจริง: The val set was sized by house count convenience rather than by how many examples the *bottleneck subtask* would get. With plan_beam as the whole point of the round, 4 val examples cannot distinguish signal from noise.

เสียหายอะไร: Any plan_beam metric off 4 examples would have been statistically meaningless; the round would have produced an unfalsifiable number.

แก้ยังไง: Rebuilt with val = houses 01-05 (all human-reviewed houses) → train 887 / val 229 = 3.87:1, plan_beam val 4→18. Bonus: val became 100% human-reviewed GT while train was pure op04 — a clean measuring stick.

กฎที่ได้: Size the val split by the example count of the subtask you are actually trying to improve, not by house count. And prefer putting human-reviewed GT in val so the ruler is clean.

สูง 2026-08-24

Controlled base-model comparison cancelled mid-run — no proof fine-tuning caused the improvement (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A Qwen3.6 base (untuned) run on the same job set was started to control variables, but cancelled before the first job even completed because Makham had class in the morning and time ran out.

ต้นเหตุจริง: The controlled baseline was scheduled last, after training and the tuned run, in a time-boxed overnight session. The most scientifically load-bearing measurement got the least slack.

เสียหายอะไร: t03's improvement over t02 cannot be attributed to fine-tuning — the comparison chart had to ship with the base column marked "n/a จริงๆ ไม่ใช่ยังไม่วัด". Still outstanding.

แก้ยังไง: Not fixed. Comparison image (เปรียบเทียบผลทูน_t03_2026-08-25.png) explicitly states the causal claim is unproven. Deferred to the next round.

กฎที่ได้: Schedule the controlled baseline run *first* or in parallel, not last. A results chart without its control column is a claim you cannot defend.

สูง 2026-08-24

The tuned-vs-untuned comparison ended up on 6 of 34 pages after three rounds of scope cuts (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Page scope was cut 34 → 22 → 12 on live orders, then Makham cut every page t01 couldn't finish, leaving 6 pages that both variants completed. The headline 'JSON valid 100%' and the tuned-vs-base result rest on those 6.

ต้นเหตุจริง: Scope was trimmed reactively around whichever pages stalled, so the surviving sample is exactly the set of pages the model found easy — a selection bias, not a random sample.

เสียหายอะไร: The tuned-vs-base conclusion is drawn from a self-selected easy subset; 'base is often faster' and the quality read are both on n=6 with the hard pages removed by construction.

แก้ยังไง: not fixed — result was reported as-is with page counts stated

กฎที่ได้: When you drop the runs that failed, you are not shortening the eval, you are changing what it measures. Record failures as failures in the comparison table rather than removing the page from the sample.

สูง 2026-08-25

Documented 'images are not downscaled' based on measuring a single sample (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The team had previously recorded as fact that images were not being downscaled. That claim came from measuring one image and generalizing to the whole set. It was wrong for 100% of the dataset.

ต้นเหตุจริง: Sampling of one, treated as a population measurement, and then written down as an established fact so nobody re-checked it. Identical to lesson #12 in rule_of_tune.md, which the team had already learned once in round t01.

เสียหายอะไร: A false 'this is fine' record blocked investigation of the real bottleneck for several training rounds and several hundred dollars of GPU time.

แก้ยังไง: Replaced the sample-based claim with a full-dataset measurement script that enumerates every image size and its token cost.

กฎที่ได้: Never write a dataset-wide claim into the record from a single sample. Measure all N, or label the claim explicitly as 'sample of 1, unverified'.

สูง 2026-08-25

plan_beam 0% and section 0% look identical on the scoreboard but have opposite causes (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Two subtasks both reported 0% recall. Ceiling analysis showed plan_beam has an 88% ceiling (the model genuinely fails) while section has a 4% ceiling on the evaluated house (the metric is impossible). Averaging or summarising them together would have produced the conclusion 'the model reads badly'.

ต้นเหตุจริง: Identical-looking numbers from different causes were being reported side by side with no ceiling column, so the scoreboard could not distinguish a model failure from a metric failure.

เสียหายอะไร: Risk of a wrong project-level conclusion and misdirected spend; also means that after any future fix, section's score cannot indicate whether the fix worked.

แก้ยังไง: Added a ceiling column per subtask to the reporting; classified each 0% by cause (model-broken vs. metric-broken) before deciding where to invest.

กฎที่ได้: Always report a metric next to its achievable ceiling. Two zeroes with different ceilings are two different diseases and must never be aggregated into one conclusion.

สูง 2026-08-25

The ID-ceiling problem is not section-only — notes can score at most 2%, and every subtask's scoreboard is uncorrected for its ceiling (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The measure_id_ceiling.py results show every subtask has a ceiling below 100%: notes 5 readable ids of 272 = 2%, section 37%, schedule 61%, plan_footing 75%, plan_slab 92%, plan_beam 88%. All rounds t01–t03 were scored and compared against a raw 100% scale, so 'plan_footing 33%' was actually 33 of an achievable 75, and notes was effectively unscoreable from the start.

ต้นเหตุจริง: The evaluation was built to compare element_id strings without anyone auditing what fraction of those ids physically exist on the drawings. section's 0% was loud enough to trigger the audit; the quieter subtasks never were.

เสียหายอะไร: Every cross-round comparison in the project's history is against a moving, uncorrected denominator. schedule '0% → 21.9%' and plan_footing '33.3% stuck' both mean something different once the ceiling is applied, and notes has been carried as a subtask that can never pass.

แก้ยังไง: not fixed — a ceiling-adjusted reporting column was proposed (safe, script-only) but not implemented

กฎที่ได้: When one metric turns out to measure the impossible, run the same ceiling check across every sibling metric in the same pass — the broken one is rarely alone.

สูง 2026-08-25

Every round's subtask scoreboard comes from a single evaluation house (บ้าน 08), whose section pages are fences and plumbing (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The section ceiling was computed 'เฉพาะบ้าน 08 ที่ใช้วัดผลจริง' — revealing that the numbers steering three training rounds are one-house measurements. House 08 is single-storey and its section sheets are almost all รั้ว/ประตู/ท่อน้ำ details, not beam/column cross-sections, which is why its ceiling collapses to 4% while the corpus-wide ceiling is 37%.

ต้นเหตุจริง: One house was chosen as the eval set, presumably for speed, and its representativeness was never checked against the corpus before its numbers were used to pick what to fix.

เสียหายอะไร: Round-over-round deltas (schedule 0→21.9%, plan_slab 0→20%) are single-sample readings that could move on house choice alone. The most expensive decision of the week — 'plan_beam is the target' — rests on this one house.

แก้ยังไง: not fixed — no multi-house eval set was created

กฎที่ได้: Before a number gets to choose what the next \$5 round trains on, check how many houses produced it. A one-house scoreboard measures the house as much as the model.

สูง 2026-08-25

The GT batch edit nulled some element_ids that are genuinely printed on the drawing (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The approved batch edit moved 1,534 section element ids to null based on the PRINTED_ID regex. Long ids that are in fact printed on the drawing — e.g. `เหล็ก WF 150x150x7x10` — were nulled as collateral, and the next dataset rebuild will train them as null.

ต้นเหตุจริง: A regex heuristic was used to drive a bulk edit of ground truth. It cannot distinguish a long printed specification string from an invented descriptive name.

เสียหายอะไร: A subset of genuinely-readable labels will be trained as null, teaching the model to skip values it could have read. Risk was explicitly accepted before the edit.

แก้ยังไง: Risk stated in one line to Makham before execution and accepted (att1235). Recovery is possible per-record because the original name is preserved in `element\_name\_assigned`; git was clean before the run as a backup, and the script is idempotent (re-run = 0 changes).

กฎที่ได้: When a heuristic drives a bulk ground-truth edit: state the error direction before running, keep the original value in a side field so it is reversible per record, ensure the script is idempotent, and start from a clean git state as the backup.

สูง 2026-08-25

343 GT section files were rewritten but train/val/test.jsonl were deliberately left stale — the dataset now disagrees with the ground truth (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The element_id convention change moved 1,534 elements to element_name_assigned across 343 section files in json_แก้ไขแล้ว/, but build_dataset_t03.py was not re-run, because rebuilding would overwrite the current train/val/test.jsonl that the existing t03 measurements are tied to. The dataset on disk and the GT it came from now describe different things.

ต้นเหตุจริง: The measurement history is pinned to a dataset file rather than to a dataset version/hash, so keeping comparability and keeping GT-consistency became mutually exclusive.

เสียหายอะไร: Any training or evaluation run started before the rebuild will silently use the old invented-name targets, exactly the problem the GT edit was made to remove; the inconsistency is invisible from the .jsonl alone.

แก้ยังไง: not fixed by design — rebuild deferred into the t03b workflow; recorded as a required first step of that round.

กฎที่ได้: Version datasets by hash/name so a GT fix does not force a choice between stale data and lost comparability — and mark a stale build loudly, not only in a diary line.

สูง 2026-08-30

`infer\_house\_t04.py --all-val` still read the original contaminated `val.jsonl` instead of the per-fold val file (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The eval script was written before k-fold existed, so `--all-val` loaded `val.jsonl` (the 5 contaminated houses) rather than `val\_fold{k}.jsonl`.

ต้นเหตุจริง: A downstream consumer was not updated when the upstream data layout changed from a single split to k-folds; the script had a hardcoded filename with no fold awareness.

เสียหายอะไร: Would have produced eval numbers on the known-contaminated split, reproducing exactly the inflated-recall problem the k-fold work had just been done to eliminate.

แก้ยังไง: Worked around by uploading `val\_fold0.jsonl` over the name `val.jsonl` on each rented machine after verifying the schema keys matched; the fold0 machine had never had the original val.jsonf so there was no risk of overwriting real data.

กฎที่ได้: When a data layout changes (single split → k-fold), grep every consumer for the old hardcoded filename before running any evaluation — an eval script silently reading the wrong file produces confident wrong numbers.

สูง 2026-08-30

Falling train_loss and 96-100% valid JSON were read as 'training is healthy, folds converge consistently' while the model was learning nothing but the schema (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: fold0 finished at train_loss 0.5503 with 'ลดต่อเนื่องตลอด ไม่มีสัญญาณ overfit'; fold2 finished at 0.5590 and was recorded as 'สัญญาณดีว่าทั้ง 4 folds converge สอดคล้องกัน'. Both readings were used as evidence the run was fine. The post-mortem showed the loss was falling because the model was memorising the JSON schema and the answers of houses it had already seen — it could not read the drawings at all (every image collapsed to one 256-token tile). The same misplaced comfort appears again when the 17773>8192 truncation warning was partly waved off with 'loss ที่ลดปกติดีเป็นสัญญาณบวก'.

ต้นเหตุจริง: train_loss and output-format validity are both blind to a preprocessing bug: a model that memorises schema plus seen answers drives loss down exactly the same way a model that learned to read does. No metric in the loop measured whether the visual input carried information.

เสียหายอะไร: Four full trainings were allowed to run to completion, and two were uploaded to HF and sha256-verified, on the strength of metrics that could not have detected the fault. Discovery was delayed until eval on held-out houses.

แก้ยังไง: Not fixed as a metric; replaced by the t05 rule to probe tokens-per-image before training. The false-positive reading is recorded in the root-cause section of the same day's entry.

กฎที่ได้: A converging loss curve is not evidence that the model can see. For a vision task, pair the loss with at least one input-side check (tokens per image, or a rendered sample at the resolution the model actually receives) before treating a run as healthy.

สูง 2026-08-30

Only aggregate recall numbers were tracked; nobody opened a raw output file until Makham demanded a root cause — the model was answering about a completely different house and sheet

(ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Eval was reported as plan_beam 6/6=0%, plan_footing 1/1=0%. Makham had to order 'หาสาเหตุก่อน มันไม่ควรแยกขนาดนั้น' before anyone opened an actual output file. Doing so immediately revealed the model was not producing near-miss element ids — it was answering with a plan of a different house and a different sheet entirely (fed a footing plan, it answered with someone else's roof plan). That qualitative fact, invisible in a recall percentage, is what pointed at a perception failure rather than an accuracy failure.

ต้นเหตุจริง: The eval harness reported a scalar (recall) and the workflow trusted it as the whole picture. A 0% score has many possible causes — near misses, format failure, total blindness — and the scalar cannot distinguish them, so diagnosis stalled at 'the score is bad' until a human forced a look at the raw text.

เสียหายอะไร: Root cause of the entire round's failure sat undiagnosed behind a number that had been available for hours; the diagnosis then took only three checks once actual outputs were read.

แก้ยังไง: Root cause found the same session by opening outputs, dumping image tokens from the training log, reading llamafactory's mm_plugin.py source, and resizing a real beam plan to 448×448 to look at it.

กฎที่ได้: When a metric reads 0% (or 100%), read the raw output before theorising. A scalar tells you something is wrong; only the artefact tells you what kind of wrong.

สูง 2026-08-31

t04 changed both model and precision at once, making the result uninterpretable (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The t04 round varied the model and the precision simultaneously, so when it failed there was no way to attribute the failure to either variable.

ต้นเหตุจริง: No experimental discipline of holding all but one variable fixed between rounds.

เสียหายอะไร: A failed round produced no usable signal about which change caused the failure.

แก้ยังไง: For this round the dataset content was made byte-identical across both arms deliberately, so any difference could only come from the model.

กฎที่ได้: Change exactly one variable per expensive run and freeze everything else — identical data across arms is what makes the comparison mean anything.

สูง 2026-08-31

The headline 11.8% strict recall badly understated real reading ability due to naming-convention mismatches (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Strict recall used exact string tuple equality with no normalization. The model read the footing page 100% correctly by position but scored 1/6 because it wrote mark ids as printed on the drawing ("F4,C1") while the GT convention splits them ("F4" + "C1"). Dummy grid lines were also named differently (1.5B vs 1'), and the footing page has no grid_ref_start/end at all.

ต้นเหตุจริง: A single strict metric designed for cross-round comparability was being read as a measure of capability.

เสียหายอะไร: Reading '11.8%' alone would have concluded the model can barely read drawings, when it read the entire footing page perfectly — a potentially major wrong strategic call.

แก้ยังไง: Added mark_base (88.2%) and position+mark_base (43.5%) metrics, and wrote an explicit interpretation summary so nobody reads the strict number alone.

กฎที่ได้: Never report a single strict metric as the capability number — pair every strict score with a convention-forgiving metric and a per-item breakdown, and state which one reflects real ability.

สูง 2026-08-31

The hand-made answer key for the benchmark house contained two errors, caught only by the adjudicator (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The op04 ground truth for บ้านไทยพอเพียง3 was built by two independent readers per page using different methods. The adjudicator, zooming into the images, found two disagreements were real errors in the key: a dummy gridline named 4' instead of 3', and the wrong y-orientation for beam B0.

ต้นเหตุจริง: Reading a construction drawing by eye is error-prone even for a careful reader; a single-reader key would have shipped both errors silently.

เสียหายอะไร: Two of 34 ground-truth items would have been wrong, corrupting the first real recall measurement of destrier — the number the whole round is judged by.

แก้ยังไง: two-independent-readers-plus-adjudicator process; the adjudicator zoomed and corrected both, then the whole key was re-checked once more against the images

กฎที่ได้: A hand-made evaluation key needs at least two independent readers and an adjudicator — the key is measurement apparatus, and an error in it is indistinguishable from a model error.

สูง 2026-09-01

Nearly reused t04's data fold for a different model — caught by Makham, not by the process (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: There was a plan to use t04's fold for training; Makham himself caught that this was wrong, with a confirming table of numbers.

ต้นเหตุจริง: Fold/split provenance was not tracked per model, so reusing another model's fold looked like a reasonable shortcut. It was only stopped by a human noticing, with no automated guard.

เสียหายอะไร: Would have contaminated train/val separation and produced misleading evaluation numbers. Caught before use.

แก้ยังไง: Avoided; documented as section 5 of the teaching document with a confirming numeric table.

กฎที่ได้: Folds are model-specific artifacts — record which model each fold belongs to and never reuse one across runs. A human catch is luck; add the provenance field.

สูง 2026-09-01

The \$2 gate's scoring script counted brute-force mark enumeration (B1...B34) as correct answers (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Besides the leaked-example problem, the blind t44 model passed one gate page by simply listing marks in sequence — B1, B2, B3 ... up to B34 — and the first three happened to match the ground truth. The scoring script counted those as hits, so a model that could not see the drawing at all scored as reading it.

ต้นเหตุจริง: The gate metric was recall-only: it checked whether GT marks appeared somewhere in the answer, with no precision penalty and no cap on how many candidates the model could emit. Any model that spams a dense enumeration of a predictable naming space (B1..Bn, C1..Cn) scores non-zero by construction.

เสียหายอะไร: Combined with the leaked-example bug this produced a false green on the go/no-go gate; acting on it would have burned the full \$60–70 training run on a blind model, repeating t04 exactly.

แก้ยังไม: The diary records the prompt-leak rule ('ห้ามใส่ตัวอย่างที่อยู่ในเฉลย ลงไปใน prompt ที่ใช้ทดสอบ') but records no precision/over-generation penalty in the scorer — the enumeration hole is described, not closed. Not fixed.

กฎที่ได้: A recall-only score is gameable by guessing everything. Any gate metric must penalise over-generation (precision, or a hard cap on emitted items) or a model that outputs the whole naming space always passes.

เรื่องที่เหลือในหมวดนี้ (38)

Baseline numbers were measured under a possibly wrong image resolution

กลาง 2026-07-21

Baseline and post-training evals must run under identical, asserted configuration — log and compare the effective settings in both runs, not just the scores.

Headline quality numbers (0%→90%) measured on only 20 of the 88 validation examples

กลาง 2026-07-21

Debug with `--limit`, but the number you write into the results table must come from a full validation pass; state the sample size next to every reported percentage.

Validation set drawn entirely from a single house (house03) — generalization across houses never measured

กลาง 2026-07-21

With few source documents, rotate the held-out house (cross-validation) or hold out pages from several houses, otherwise a per-house quirk is indistinguishable from a model weakness.

"element_id 100%" reported as a quality result, but it is circular if elements are matched by element_id

กลาง 2026-07-21

Never report exact-match on the field used as the matching key, and always state the matching rule and the n next to conditional metrics (here n=16).

t01 and t02 headline numbers were measured on different rented machines with possibly different library versions

กลาง 2026-07-28

Store the environment fingerprint with every metric, and evaluate all arms of a comparison on one machine in one session; a number without its measurement conditions cannot be compared to anything.

The model was judged on page types the pipeline already discards

กลาง 2026-07-29

Evaluate on the distribution the system actually serves. Order any diagnostic run by relevance to the real pipeline, not by file/page order, or you will spend your measurement budget on data you filter away.

The 'results are damn awful' verdict was nearly taken at face value — the root cause was only found by cataloguing what worked

กลาง 2026-07-29

Never diagnose from an aggregate score. Split every eval into what the model got right and what it got wrong, and read the *shape* of the split: an unnatural pattern (easy things wrong, hard things right) is a cause signal that a single percentage erases.

Every page the model has ever been run on in real use was its own training data — no held-out page has been tested

กลาง 2026-07-29

Keep at least one house (or a fixed page set) permanently out of training and run every production sanity check against it. A quality number measured on training data answers a different question from the one being asked — and here it accidentally became the key diagnostic only because the failure was in format, not content.

Model was judged on pages (index/notes/material_list/site_plan) that KEEP_PATTERNS already excludes

กลาง 2026-07-29

Draw the evaluation set from the production scope filter. Judging a model on inputs your pipeline discards produces a verdict you cannot act on.

Judged the model for emitting zero confidence fields that the prompt and dataset had deliberately stripped out

กลาง 2026-07-29

When scoring against ground truth, first subtract every field you deliberately removed from the prompt or training data — otherwise you audit your own scope decisions as model errors.

การทดสอบความสามารถในการทั่วไปของ t02 ไม่เคยทำจนจบ บ้านหลัง 10/11 ไม่ได้เริ่มเลย และหลัง 08 ก็ไม่ถึง เพดาน

กลาง 2026-08-21

ถ้าตัดขอบเขตงานลงจนการทดลองตอบคำถามของตัวเองไม่ได้แล้ว ให้ตัดป้ายว่าตัวเลขที่ได้เป็นค่าชั่วคราวทุกที่ที่เอาไปอ้าง ตัวเลขพาดหัวที่มาจากบ้าน 1.5 หลังจากท้วงแผนไว้ 4 หลัง ต้องบอกขนาดตัวอย่างติดไปด้วยทุกครั้ง

ตัวเลขที่คนทำเองก็ยังวางลงบนแบบได้ไม่ครบ (45/48 กับ 52/53)

กลาง 2026-08-21

ตรวจสอบตัวเลขก่อนจะเอาไปใช้เป็นไม้บรรทัด ให้รายงานอัตราที่ตัวเลขเองระบุตำแหน่งได้ควบคู่กับ recall ของโมเดล ไม่งั้นเรากำลังให้คะแนนโมเดลด้วยเลขที่ไม่ครบ โดยไม่มีใครรู้ว่าขาดไปแค่ไหน

Evaluation compares element IDs with exact string equality — whitespace and case cause false misses

กลาง 2026-08-25

Any string-equality scorer must normalise (trim, casefold, collapse internal whitespace) before comparing, and should log near-miss pairs so formatting failures are visible rather than silently scored as zero.

HoughCircles section-label detection scored a perfect 5/5 on one house and collapsed to 3/20 and 0/18 on others

กลาง 2026-08-25

A computer-vision technique that scores perfectly on one document is unvalidated. Test on at least three sources from different origins before reporting it as working — and write the failed attempts down so they are not repeated.

Non-Qwen model search returned InternVL3, which uses a Qwen backbone anyway

กลาง 2026-08-25

When the requirement is architectural diversity, verify the backbone in the model card/config — brand names hide shared foundations.

The 10-model alternative-VLM survey ranked candidates on spec sheets — only 1 of 10 had any real benchmark score

กลาง 2026-08-25

Put the hardest-to-fake criterion first in the screen. Licence and parameter count are free to read, which is exactly why they should not be what a survey sorts on.

Rebar-dot counting scored 1/5, but the test measured the hand-made crops, not the idea

กลาง 2026-08-25

If the input to an experiment was produced by hand and not verified, the result measures your hand, not the hypothesis. Mark such results inconclusive — recording a bad test as a disproof kills a good idea permanently.

The chars/2.2 token estimate driving MAX_LENGTH has never been validated against the real tokenizer

กลาง 2026-08-25

Any heuristic that becomes an input to a hard limit must be validated against ground truth once. Run the real tokenizer over the longest few examples and record the true ratio before trusting a margin under 10%.

Automatic template harvesting produced overwhelmingly garbage candidates — 45 of 57, then 71 of 79

กลาง 2026-08-25

When an automatic harvester's precision is under 15%, keep a mandatory human gate and never auto-promote. Ranking to speed up review is fine; ranking that decides is how poison templates get in.

เปิด xgrammar ให้ทุก pass ทำให้เทียบผล JSON ถูกต้องกับรอบก่อนหน้าไม่ได้เลย

กลาง 2026-08-29

ทุกครั้งที่เปลี่ยนค่าตอน inference ให้บันทึกไว้คู่กับตัวเลขที่มันกระทบ และเก็บ flag ไว้ปิดมันได้ เพื่อให้วัดผลกระทบได้จริง ไม่ใช่มานั่งเถียงกัน

การแบ่ง k-fold กระจายบ้าน 5 หลังที่คนตรวจแล้วไม่เท่ากัน — fold 0 กับ 3 ไม่ได้เลยสักหลัง

กลาง 2026-08-29

แบ่ง fold โดยถ่วงตามคุณภาพของ ground truth ด้วย ไม่ใช่ดูแค่จำนวนตัวอย่าง fold ที่วัดกับ label ที่ยังไม่มีคนตรวจ วัดคนละอย่างกับ fold ที่วัดกับ label ที่ตรวจแล้ว

สมมติฐานที่ว่า plan_beam พังเพราะหน้ามีผังสองแบบปนกัน ถูกพิสูจน์ว่าผิด — t03 อ่านผังคนไม่ได้จริงๆ

กลาง 2026-08-30

ทดสอบสมมติฐานของความล้มเหลวด้วยการแยกตัวแปรที่มันโห่ออกมาตัวเดียว (เคสนี้คือครอบหน้าให้เหลืออันเดียว) ก่อนจะสร้างวิธีใหม่ๆ ทยอยไป

ช่องว่างข้อมูลจริงสามจุดขวางการสร้าง dataset ของ t05 — ไม่มี GT ของ pass0 เลย, ไฟล์ CV sidecar มีแค่ 10 หน้า และไม่มีตัวจับคู่ cv_mark กับ GT

กลาง 2026-08-30

ก่อนวางแผนรอบเทรนที่พึ่งสัญญาณ input ใหม่ ให้ตรวจก่อนว่า ground truth และไฟล์ sidecar ที่ต้องใช้มีอยู่จริงทั้งชุดข้อมูล — นับให้เห็นตัวเลข อย่าเชื่อเอกสารแผน

Nearly set up a wrong conclusion: InternVL reads 3.3x fewer pixels than Qwen by architecture

กลาง 2026-08-31

Compute each candidate's architectural input ceiling before an A/B comparison and record it as an interpretation caveat in advance, so a ceiling effect is never mistaken for model quality.

CV detector found only a fraction of the boxes on ~20 pages, blocking pass3 auto-matching

กลาง 2026-08-31

When a downstream step keeps failing on a subset, check the detector's own recall on that subset before writing more matching logic — a matcher can never recover boxes that were never detected.

Two different final-loss figures were recorded on the same day, and the overfit verdict rests on the lower pair

กลาง 2026-08-31

Before using a loss number as evidence for a decision, state which metric it is and re-read it from the log; two numbers for one metric means neither is usable yet.

The merged soup model ships permanently unmeasured because all 40 houses were consumed as training data

กลาง 2026-09-01

If you plan to merge k-fold models, reserve a separate global holdout set from the start that no fold ever trains on. Failing that, label the merged artifact honestly as unmeasured — never borrow a fold's score to describe it.

Destrier's ~43% recall was measured with forgiving name matching, making the number not comparable to strict scores

กลาง 2026-09-01

Always report a strict-mode score next to any lenient/normalised score, and never quote a lenient number alone as 'the' accuracy. State the matching rule every time a recall figure is written down.

The first house simulation had Claude read the drawings itself, burning tokens for a less accurate result

กลาง 2026-09-01

To simulate a model, substitute its known-correct outputs (ground truth) — not another model's improvised answers. It is cheaper and strictly more faithful.

k-fold was designed as 5 groups but only 4 folds were run, so 8 houses sit in every training set and are never evaluated

กลาง 2026-09-01

If you declare k groups, run k folds or say plainly which group was dropped and why. A held-out set that is never held out is training data wearing an eval label.

The warning printed when elements were dropped blamed the model ('generation หลุด') for data our own sanitizer had deleted

กลาง 2026-09-01

An error message must name only what the emitting code can actually prove. A guessed cause in a log line is worse than no cause — it redirects the next debugger away from the real file.

Element result written with two different denominators in one cell, and precision never reported

ตำ 2026-07-21

For any extraction task, report precision and recall together (or F1) — a recall-only number hides hallucinated output and misdirects the next iteration.

Previously recorded that ERNIE-4.5-VL required ERNIEKit — actually an Unsloth notebook exists

ตำ 2026-08-25

Tag every tooling-support claim with the date and source it was checked against, and re-verify before using it to eliminate a candidate — the ecosystem changes weekly.

A 19-column detection count on บ้าน17 หน้า15 was reported without ever being verified against the drawing

ตำ 2026-08-25

Either verify a number or leave it out of the results table — an unverified figure listed beside verified ones gets quoted as if it were verified.

easyocr experiment failed — sheet_code 0/4 correct even after two rounds of tuning

ตำ 2026-08-29

Set a stop condition before starting a tool bake-off ("if round 1 is under X, stop"). A negative result is valuable and worth documenting — but budget one tuning round, not two, before accepting it.

'No truncation warnings' conclusion was based on a 40/1020 row sample, not full coverage

ตำ 2026-08-30

A dry-run sample cannot prove a dataset-wide property. For length/truncation questions, run a cheap CPU-only pass over 100% of rows instead of inferring from a 4% sample.

Naive 'take first N rows' eval sampling would have landed entirely inside one house because the jsonl is house-ordered

ตำ 2026-08-30

Never sample an ordered dataset sequentially — stratify by the grouping key (house/entity) and assert the sample covers every group before trusting the resulting metric.

A 2-anchor least-squares fit always yields residual 0, so a residual-based quality gate would have passed every bad fit

ตำ 2026-09-01

Before trusting a validation metric, ask in which regimes it can return a pass without any information — a metric that cannot fail is not a gate, it is decoration.

คุณภาพข้อมูล ความถูกต้อง และ ข้อมูลหาย

ข้อมูลผิด ขาด ซ้ำ ปนเปื้อน หรือหายถาวร รวมถึงข้อมูลที่กู้คืนไม่ได้

ทั้งหมด 142 เรื่อง · วิกฤต 26 · สูง 45 · กลาง 59 · ต่ำ 12

เรื่องที่ต้องอ่านให้ครบ (71)

วิกฤต 2026-07-02

โมเดลอ่าน main_bar_type สลับ DB↔RB ทั้งหน้า ทั้งที่สัญลักษณ์เดียวกันนี้อ่านถูกตอนเป็นเหล็กปลอมในหน้าเดียวกัน

เกิดอะไรขึ้น: ตอนลองเทียบ Claude กับ Qwen ครั้งแรก (บ้าน 01 จำนวน 4 หน้า) เจอว่า main_bar_type ถูกอ่านสลับ DB↔RB อย่างเป็นระบบในหน้าหนึ่ง ทั้งที่สัญลักษณ์ตัวเดียวกันในหน้าเดียวกันนั้นกลับอ่านถูกเมื่อเป็นเหล็กปลอม เรื่องนี้กลายเป็นบทเรียนข้อ 6 ใน No_touch_box/CLAUDE.md และยังถูกอ้างถึงจนถึงทุกวันนี้

ต้นเหตุจริง: โมเดลเดาชนิดเหล็กจากขนาดเส้นผ่านศูนย์กลางหรือจากบริบทรอบๆ แทนที่จะดูจากสัญลักษณ์ Ø (วงกลม) ตรงๆ สัญลักษณ์เดียวกันเลยออกมาคนละคำขึ้นกับว่ากำลังเติมช่องไหน ตอนนั้นสเปกยังไม่มีการเขียนไว้ว่า Ø แปลว่า RB เสมอ การเดาจึงไม่มีอะไรกัน

เสียหายอะไร: ชนิดเหล็กแกนหลักผิด ลามไปที่น้ำหนักเหล็ก รายการตัดเหล็ก (BBS) และค่าเหล็กใน BOQ ของทุก element ที่โดน และเพราะมันผิดอย่างเป็นระบบ (ไม่ใช่สุ่ม) มันจึงไม่หักล้างกันเอง ภายหลังต้องมารอบแก้เฉพาะเมื่อ 2026-07-06 ไล่เขียน main_bar.type ของทุก element ในหน้า 21 view1 จาก DB เป็น RB ใหม่ทั้งหมด

แก้ยังไง: บันทึกเป็นบทเรียนข้อ 6 ใน No_touch_box/CLAUDE.md แล้ววันที่ 2026-07-06 เขียนเป็นกฎตายตัวในสเปก ('สัญลักษณ์ Ø (วงกลม) = RB เสมอ ห้ามเดา DB จากขนาดเส้นผ่านศูนย์กลางเด็ดขาด') พร้อมแก้ไฟล์ที่ได้รับผลกระทบ

กฎที่ได้: อย่าปล่อยให้โมเดลเดาค่าที่เป็นประเภท (ชนิดเหล็ก, เกรด) จากค่าที่เป็นตัวเลขต่อเนื่อง (ขนาดเส้นผ่านศูนย์กลาง) ให้เขียนการแมปสัญลักษณ์→ค่าลงในสเปกเป็นกฎเด็ดขาด และถ้าสัญลักษณ์เดียวกันในหน้าเดียวกันอ่านถูกในช่องหนึ่งแต่ผิดในอีกช่องหนึ่ง นั่นคือ prompt/สเปกบกพร่อง ไม่ใช่ข้อจำกัดของการมองภาพ — ให้ไปแก้กฎ ไม่ใช่แก้ตัวอย่าง

วิกฤต 2026-07-05

กำลังอัปเดตคอนกรีตขัดกันเอง 210 กับ 240 ksc ระหว่างการอ่านรอบใหม่กับข้อมูล Gen 1 เก่า และไม่เคียดตัดสินว่าอันไหนถูก

เกิดอะไรขึ้น: อ่านหน้า 18 (โน้ต S-01) รอบใหม่ได้ concrete_strength_ksc = 210 ($F_c' = 210 \text{ kg/cm}^2$ เขียนชัดในภาพ) แต่ไฟล์ Gen 1 เก่า qwen-output/บ้าน_เล็ก_1ชั้น_01_หน้า18.json บันทึกไว้ 240 ทั้งสองค่าไม่มีใครตรวจสอบยืนยัน เรื่องนี้ยังค้างอยู่จนจบวันที่ 2026-07-06 และถูกยกไปเป็นงานค้างข้ามวันทั้งสัปดาห์

ต้นเหตุจริง: การสกัดข้อมูลสองรุ่นเขียนข้อเท็จจริงเดียวกันลงคนละไฟล์ โดยไม่มีขั้นตอนมาเทียบกัน และไม่มีแหล่งความจริงเดียวสำหรับค่าคงที่ระดับบ้าน ไม่มีอะไรใน pipeline ที่เอาค่าที่อ่านใหม่ไปเทียบกับค่าของรุ่นก่อน หรือหยุดงานเมื่อค่าไม่ตรงกัน ความขัดแย้งจึงนอนอยู่ในชุดข้อมูลได้เสียๆ ไม่มีกำหนด

เสียหายอะไร: ค่าคงที่ทางโครงสร้างหลักของทั้งบ้านคลุมเครือ การคิดราคาคอนกรีตใน BOQ/BBS หรือการคำนวณอะไรก็ตามที่ขึ้นกับกำลังอัด จะผิด 50% เพราะเรื่องนี้ concrete_grade จึงถูกปล่อยเป็น null ในทุก element ของคานที่ join ไว้ใน Gen 3.3 ข้อมูลสเปกที่ join แล้วจึงไม่ครบไปด้วย

แก้ยังไม: ยังไม่ได้แก้ ค้างเป็นงานค้างข้อ 3 ในรายการทำวัน 2026-07-06 ('ความขัดแย้ง concrete_grade 210 กับ 240 ksc') และเขียน concrete_grade เป็น null ทุกที่แทนที่จะเดา

กฎที่ได้: ค่าคงที่ระดับบ้าน (เกรดคอนกรีต, เกรดเหล็ก, ระยะหุ้ม) ต้องอยู่ในไฟล์แม่ไฟล์เดียวต่อบ้าน และต้องเทียบกับทุกรุ่นก่อนหน้าก่อนจะสกัดข้อมูลต่อ เจอความขัดแย้งเมื่อไหร่ให้ล็อกชองนั้นไว้ — เขียน null พร้อมติดธง ห้ามเดา — แล้วตั้งคนรับผิดชอบกับกำหนดเวลาให้เสร็จในวันเดียวกัน ไม่ใช่ปล่อยเป็นบรรทัดค้างที่ยกไปวันแล้ววันเล่า

วิกฤต 2026-07-09

คำสั่งให้เปลี่ยนชื่อโฟลเดอร์ raw/ เป็น 'ข้อมูลเก่าไม่ได้ใช้แล้ว' เกือบทำ pipeline ที่ใช้งานจริงพังใน 5 สคริปต์

เกิดอะไรขึ้น: มีการเข้าใจว่าโฟลเดอร์ raw/ เลิกใช้แล้ว เลยสั่งให้เปลี่ยนชื่อ พอ grep ก่อนลงมือ พบว่ามันมีขนาด 533 MB และถูกเขียน path ตายตัวไว้เป็นเส้นทางที่ยังใช้งานอยู่ใน run_pipeline.py, analyze_folder.py, build_document_map.py, pdf-processor.py และ upload-to-supabase-storage.js (รวม 7 จุด) สุดท้ายเปลี่ยนชื่อไปจริงพร้อมแก้ path ในโค้ด แต่ก็ถูกยกเลิกต้องย้อนกลับทั้งหมด

ต้นเหตุจริง: สถานะของโฟลเดอร์ถูกเดาจากชื่อและจากความจำ ไม่ใช่จากสิ่งที่โค้ดอ้างถึงจริง และชื่อกลางๆ อย่าง raw/ ก็ไม่ได้บอกอะไรเลยว่ายังใช้งานอยู่หรือเปล่า

เสียหายอะไร: ถ้าทำไปเลยจะทำให้ pipeline สกัดข้อมูลหลักพังทันทีบนไดเรกทอรีข้อมูล 533 MB แต่ที่เกิดขึ้นจริงคือเสียเวลาไปกับรอบเปลี่ยนชื่อ-แล้ว-ย้อนกลับ ทั้ง 5 สคริปต์ ไฟล์ .gitignore และเจอเหตุโฟลเดอร์ถูกล็อกอีกหนึ่งเรื่อง

แก้ยังไม: grep หา path ที่เขียนตายตัวทุกจุดก่อนลงมือ ถามก่อนทำต่อ แล้วย้อนชื่อโฟลเดอร์ สคริปต์ทั้ง 5 ตัว และ .gitignore กลับสู่สภาพเดิม จบด้วย git status ที่สะอาด

กฎที่ได้: ห้ามเปลี่ยนชื่อหรือย้ายโฟลเดอร์ข้อมูลโดยดูจากชื่อมันอย่างเดียว — ให้ grep หา path นั้นทั้ง repo ก่อน และให้ถือว่า 'ผู้ใช้ออกว่ามันเก่าแล้ว' เป็นสมมติฐานที่ต้องพิสูจน์ ไม่ใช่คำสั่งที่ทำตามได้เลย

ถ้ารันตัวสร้าง task ให้บ้านหลังที่ 2 จะไปทับไฟล์ผลลัพธ์ของบ้านหลังที่ 1 แบบเจียบๆ

เกิดอะไรขึ้น: ไฟล์ label-studio-tasks-makham.js เขียนผลลัพธ์ลงชื่อไฟล์ตายตัว (label-studio-tasks-makham-elements.json ฯลฯ) โดยไม่มีชื่อบ้านอยู่ในนั้นเลย ถ้ารันให้บ้านหลังที่ 2 มันจะเขียนทับไฟล์ task ของบ้านหลังที่ 1 ที่สร้างไปแล้ว (และอัปโหลดไปแล้ว) โดยไม่มีค่าเตือนใดๆ

ต้นเหตุจริง: สคริปต์ถูกเขียนขึ้นตอนที่มีบ้านหลังเดียว และไม่มีใครกลับมาทบทวนสัญญาเรื่องไฟล์ผลลัพธ์ตอนที่ workflow เปลี่ยนไปใช้โครงสร้างโฟลเดอร์แบบหลายบ้านที่มีเลขกำกับ

เสียหายอะไร: จับได้ตอนตรวจความพร้อมก่อนเริ่มบ้านหลังที่ 2 พอดี เลยไม่มีข้อมูลหาย แต่ก็เป็นตัวขวางจริงที่ทำให้บ้านหลังที่ 2 หยุดไปหนึ่ง session

แก้ยังง: เปลี่ยนชื่อไฟล์ผลลัพธ์ให้มีชื่อบ้านอยู่ด้วย รันบ้านหลังที่ 1 ซ้ำเพื่อยืนยันว่าได้จำนวนเท่าเดิม (34/39/26/99) แล้วลบไฟล์ชื่อตายตัวที่ค้างอยู่ทิ้ง

กฎที่ได้: สคริปต์ใดก็ตามที่ผลลัพธ์ผูกกับ entity ที่รับเข้ามา ต้องใส่ชื่อ entity นั้นในชื่อไฟล์ตั้งแต่วันแรก — path ผลลัพธ์ตายตัว บวกกับมิติข้อมูลใหม่ เท่ากับการเขียนทับแบบเจียบๆ

ตารางราคาบ้าน 10 แบบถูกอ่านผิดหลายแถวในสำเนาของทั้งห้าบ้าน ไม่ใช่แค่แถวเดียวที่เป็นขนวนให้ไปกวาด

เกิดอะไรขึ้น: การกวาดด้วยการซูม 8 เท่าเทียบตัวเลขอ้างอิง ซึ่งเป็นรอบที่ย้อน 'การแก้' ของบ้าน 10 นั้น ยังเจอความผิดพลาดในตารางเดียวกันอีกชุด: #1 แก้เป็น 1,020,000, #2 เป็น 1,460,000 (ราคาบ้าน 2 ของโปรเจกต์นี้เอง), #4 ราคาปูพื้นเป็น 1,824,000, ขนาดแบบบ้านหนึ่งเป็น $3/8=156$ และในสำเนาชุดที่ห้าของบ้าน 1 แก้ #8 ราคาปูพื้น 1,662,000→1,622,000 กับ #10 ราคาปูพื้น 2,235,000→2,035,000

ต้นเหตุจริง: เลขไทยถูกตัดสินที่ละตัวแบบโดดๆ ที่ระดับซูมสูง ซึ่ง ๑ กับ ๙ สับสนกันง่ายมาก แทนที่จะเทียบกับตัวเลขอ้างอิงในอี้นบนแผ่นเดียวกัน ความผิดพลาดนี้เป็นระบบ มันเลยเกิดซ้ำในทุกครั้งที่มีคนอ่านตารางเดียวกันนี้อ่างอิสระ

เสียหายอะไร: ตัวเลขราคาผิดหกจุดขึ้นไป กระจายอยู่ในไฟล์ของทั้งห้าบ้าน รวมถึงราคาบ้านของโปรเจกต์ตัวเองด้วย เจอได้ก็เพราะบังเอิญมี 'การแก้' ของแถวหนึ่งถูกตรวจซ้ำอีกรอบเท่านั้น

แก้ยังง: แก้ครบทั้งห้าสำเนาเมื่อ 2026-07-19 โดยเทียบตัวเลขทุกหลักกับตัวเลขอ้างอิงบนแผ่นเดียวกัน ตอนนีตารางตรงกันครบทั้งห้าบ้าน

กฎที่ได้: การอ่านตัวเลขต้องมีตัวเลขอ้างอิงในแผ่นเดียวกันเป็นตัวเทียบ เหมือนกับการวัดจากฟิกเชลต้องมีมาตราส่วนที่ปรับเทียบแล้ว อย่าตัดสินเลขไทยแบบโดดๆ ตอนซูมเด็ดขาด

ทำลาย instance ทั้งก่อนที่จะดาวน์โหลดไฟล์ผลลัพธ์ตัวไหนเลย — adapter, merged model และ GGUF หายหมด

เกิดอะไรขึ้น: มะขามสั่ง destroy instance บน Vast.ai ตอนที่ LoRA adapter ขนาด 7.5GB, merged model 66GB และ GGUF Q4_K_M 21.2GB ยังอยู่แค่บนเครื่องนั้นเครื่องเดียว การ destroy ของ Vast.ai เป็นการลบถาวร (ต่างจาก stop) ทั้งสามไฟล์จึงหายไปโดยกู้คืนไม่ได้

ต้นเหตุจริง: ใน workflow ไม่มีด่านกันเรื่องการสำรองข้อมูลเลย ผลลัพธ์ถูกนับว่า 'เซฟแล้ว' เพียงเพราะมันอยู่บนดิสก์ ส่วนการอัปโหลดขึ้น HuggingFace ที่ตั้งใจจะทำ ถูกเลื่อนออกไป (เพราะติดเรื่องยังไม่มี token) แทนที่จะถือว่ามันคือเงื่อนไขที่เปลืองงานนั้นเสร็จ

เสียหายอะไร: งาน train + merge + quantize ทั้งสิ้นสูญไปหมด — ทั้งค่าเช่า GPU ทั้งการอดนอนทั้งคืน ทั้งการขาดเรียน แลกกับผลลัพธ์ที่ตอนนี้ไม่มีอยู่แล้ว ต้อง train ใหม่ทั้งรอบตั้งแต่ต้น

แก้ยังไ: แก้ไม่ได้ — ไฟล์กู้คืนไม่ได้ สิ่งที่เหลือรอดมีแค่สคริปต์ที่แก้บั๊กแล้ว ชุดข้อมูล และบทเรียนที่จดไว้

กฎที่ได้: ไฟล์ที่อยู่บนที่เก็บของเครื่องเช่า ไม่นับว่าเป็นการสำรองข้อมูล ก่อนจะ stop หรือ destroy instance ใดๆ ต้องไปไล่ดูรายการไฟล์ที่ปลายทางนอกเครื่อง (HuggingFace/เครื่องตัวเอง) ให้เห็นกับตาว่าผลลัพธ์ทุกตัวไปถึงจริงแล้ว

ปิดเครื่องเช่าทั้งก่อนที่จะได้ก๊อป LoRA adapter, โมเดลที่ merge แล้ว และ GGUF ออกมา

เกิดอะไรขึ้น: เครื่อง Vast.ai ถูกปิดไปก่อนที่ผลงานของวันนั้น (LoRA adapter, โมเดลที่ merge แล้ว, GGUF) จะถูกดาวน์โหลดหรืออัปโหลดไปที่ไหนเลย พื้นที่เก็บข้อมูลบนเครื่องเช่าเป็นแบบชั่วคราว ทุกอย่างเลยหายไปแบบกู้คืนไม่ได้

ต้นเหตุจริง: ไม่มีด่านกันที่แข่งพอร์ระหว่าง "งานเสร็จ" กับ "ปิดเครื่อง" คำเตือนให้สำรองข้อมูลถูกพูดแบบเบาๆ ไม่ใช่เป็นขั้นตอนบังคับที่ต้องผ่านก่อน แถมยังไม่ได้เตรียมปลายทางสำหรับสำรองข้อมูล (HuggingFace) ไว้ล่วงหน้าด้วย

เสียหายอะไร: ค่าเช่า GPU การเทรน และการแปลง GGUF ของทั้งวัน ต้องทำใหม่หมดเพื่อให้ได้โมเดลที่ใช้ได้ สิ่งที่รอดมามีแค่สคริปต์ที่แก้บั๊กแล้วกับบทเรียนที่จดไว้

แก้ยังไ: กำหนดให้การอัปโหลดผลลัพธ์ขึ้น HuggingFace เป็นขั้นตอนหนึ่งของ pipeline อย่างชัดเจน และรอบถัดไปมีการทดสอบ HF write token ล่วงหน้าไว้แล้ว (Phase 0.1)

กฎที่ได้: การสำรองข้อมูลออกจากเครื่องเป็นขั้นตอนบังคับที่ต้องผ่านก่อน ไม่ใช่แค่คำเตือน — เครื่องชั่วคราวจะปิดได้ก็ต่อเมื่อพิสูจน์แล้วว่ามีความอยู่ในที่ถาวรจริง และปลายทางนั้นต้องทดสอบให้ผ่านก่อนเริ่มงานด้วย

วิกฤต 2026-07-24

แบบผังคานสองแผ่นของบ้านหลัง 5 ขาดคานไป 31 ตัว ซึ่งคนเป็นคนหามาให้ทั้งหมด ไม่ใช่ระบบสกัดหรือระบบตรวจ

เกิดอะไรขึ้น: ตอนจัดรูปแบบหน้า 32/33 ของบ้านหลัง 5 ใหม่ (ผังคานชั้น 1 และชั้น 2) ต้องเพิ่มคานเข้าไป 31 ตัว (16 + 15) ที่มขามส่งมาให้ตรงๆ บวกกับลบเส้นที่บันทึกไว้แต่ไม่มีอยู่จริงออกไป 3 เส้น และเพิ่มกริดสมมติใหม่ B'=12.2 เพื่อให้คานที่เพิ่มเข้ามาตัวหนึ่งมีที่ลง

ต้นเหตุจริง: การสกัดข้อมูลจากแบบสองแผ่นนี้ครั้งแรกจับคานได้แค่ส่วนน้อยของที่มีอยู่จริง และการตรวจทานรอบก่อนๆ ของบ้านหลังนี้ก็ไม่ได้ตรวจความครบถ้วนเทียบกับตัวแบบ

เสียหายอะไร: คานราวครึ่งหนึ่งบนแบบสองแผ่นของบ้านที่อยู่ในชุดเทรนหายไป จนกระทั่งคนมานั่งไล่นับเองด้วยมือ — ทั้งระบบสกัดและระบบตรวจอัตโนมัติได้ศูนย์คะแนนบนแบบคู่นี้

แก้ยังง: เพิ่มคาน 31 ตัวและลบเส้นที่ไม่มีอยู่จริง 3 เส้น เมื่อ 2026-07-24 แล้วตรวจสอบไขว้กับ page00 ไม่มี grid_ref ค้างเลย

กฎที่ได้: ถ้าคนต้องมานั่งไล่นับของที่หายไป 31 ตัวเอง แสดงว่าการสกัดข้อมูลจากแบบชนิดนั้นใช้ไม่ได้ ให้บันทึกว่าแบบชนิดนี้เป็นจุดอ่อนที่รู้ตัว แล้ววัดค่า recall ของมัน ดีกว่าไล่แก้ทีละบ้าน

วิกฤต 2026-07-28

เสนอข้ามเฟส 8 เพราะ t02 แพ้การเปรียบเทียบ — โดยไม่ทันสังเกตว่าการสำรองข้อมูล adapter ก็ถูกมัดรวมอยู่ในเฟสเดียวกันนั้น

เกิดอะไรขึ้น: หลังจาก t02 แพ้ทุกตัวชี้วัด ผู้ช่วยเสนอให้ข้ามเฟส export ไปเลยเพราะดูไม่มีประโยชน์แล้ว แต่การสำรองข้อมูล adapter ถูกมัดรวมอยู่ในเฟส 8 ด้วย ถ้าทำตามข้อเสนอั้นจริงๆ ทั้งรอบนี้จะไม่เหลือข้อมูลสำรองอะไรเลยสักอย่าง — ขำรอยความล้มเหลวครั้ง DAY OF SHAME เป๊ะๆ

ต้นเหตุจริง: คิดเรื่องเฟสหนึ่งจากหัวข้อของมัน ('export GGUF') โดยไม่ใส่ใจว่ามันทำอะไรอย่างอื่นอีกบ้าง การเอาขั้นตอนที่สำคัญต่อความปลอดภัย (การสำรองข้อมูล) ไปมัดรวมไว้ในขั้นตอนที่หน้าตาเหมือนของที่ข้ามได้ ทำให้ขั้นตอนความปลอดภัยนั้นถูกข้ามไปโดยไม่ตั้งใจ

เสียหายอะไร: จับได้ทันก่อนลงมือ ถ้าทำตามไปแล้ว การเทรนที่จ่ายเงินไป 85 นาฬิกาก็จะไม่เหลืออะไรกู้คืนได้เลย

แก้ยังง: รับสำรอง LoRA adapter (5.14GB) ด้วยมือขึ้นไปที่ Sicilian44/qwen3vl-30b-thai-rc นอกกระบวนการปกติของเฟส 8 แล้วตรวจสอบขนาดไฟล์ผ่าน HF API

กฎที่ได้: ห้ามเอาการสำรองข้อมูลที่สำคัญต่อความปลอดภัยไปมัดรวมไว้ในเฟสที่ข้ามได้ด้วยเหตุผลทางธุรกิจเด็ดขาด การสำรองข้อมูลต้องเป็นขั้นตอนของตัวเองที่ทำเสมอไม่มีเงื่อนไข และก่อนจะข้ามเฟสไหนก็ตาม ต้องไล่ดูให้ครบว่ามันทำอะไรบ้าง ไม่ใช่ดูแค่ชื่อ

ไฟล์ section ของบ้าน 10/11 ไม่มี elements[] เลย — แต่ละไฟล์ตั้งชื่อ array ขึ้นมาเอง

เกิดอะไรขึ้น: ไฟล์ `pattern=section` ในบ้าน 10 และ 11 ไม่มี `elements[]` เลย แต่ละไฟล์กลับตั้งชื่อ array ของตัวเองขึ้นมา (`beams`, `columns`, `slabs`, `footing\_types`, `details`, `sections`) โดยข้างในมีรูปทรงของ element ต่างกันไปอีก แค่เปลี่ยนชื่อ array เนี่ย จึงไม่พอ

ต้นเหตุจริง: spec อธิบายวิธีเขียนแต่ละ *ไฟล์* แต่ไม่เคยล็อก *รูปทรงของไฟล์* ไว้ ไม่มีกฎที่บอกว่า 'ตัวห่อต้องเป็น elements[] เสมอ' การสกัดข้อมูลจึงจัดระเบียบไฟล์ยังไงก็ได้ตามที่รู้สึกว่าเป็นธรรมชาติ โดยไม่ผิดกฎที่เขียนไว้เลย

เสียหายอะไร: ไฟล์ 30 ไฟล์เข้ากันไม่ได้เชิงโครงสร้างกับทุกระบบปลายทาง ต้องรื้อโครงสร้างใหม่ (ไม่ใช่แค่เปลี่ยนชื่อ) และต้องหยุดเพื่อขออนุมัติชัดเจนก่อน เพราะมันเปลี่ยนโครงสร้าง schema จริง

แก้ยังไง: ยูน array ที่ตั้งชื่อกันเองทั้งหมดเข้าไปใน `elements[]` พร้อม `element\_id`/`element\_type` แปลงหน่วยและรูปทรง เหล็กเสริม และเก็บค่าดั้งเดิมทุกค่าไว้ในฟิลด์ `\_printed\_as` ภายหลังล็อกด้วย \$0 FORMAT LOCK ตัวใหม่

กฎที่ได้: ต้องกำหนดรูปทรงของตัวห่อ (container) ไม่ใช่แค่กฎของฟิลด์ ถ้า spec ไม่เคยบอกว่า 'ไฟล์หน้าตาแบบนี้' การสกัดข้อมูลแต่ละครั้งจะแต่งโครงสร้างที่ดูสมเหตุสมผลขึ้นมาเอง และไม่มีอันไหนผิดตามกฎที่เขียนไว้เลย

แบบผังคานทุกแผ่นที่ตรวจในบ้าน 01-05 มีคานหายไปหมด (20→27, 22→36, 30→38, 5→21) — ส่วนบ้าน 06-11 ยังไม่เคยตรวจเลย

เกิดอะไรขึ้น: ตอนเอาผังคานของบ้าน 01-05 มาตรวจกับแบบจริง พบว่าทุกแผ่นมีคานจริงหายไปหมด กรณีหนึ่งสกัดได้ 5 ตัวทั้งที่มีอยู่จริง 21 ตัว ส่วนบ้าน 06 ถึง 11 ยังไม่เคยถูกตรวจเลย และต้องสันนิษฐานว่ามีข้อบกพร่องเดียวกัน

ต้นเหตุจริง: การสกัดคานตรวจไม่เจออย่างเป็นระบบ — คานที่ลากต่อข้ามหน้า คานที่วาดจางๆ หรือคานที่ป้ายชื่อไว้ด้านข้าง จะถูกข้ามไป และไม่มีอะไรมานับจำนวนคานที่สกัดได้เทียบกับการตรวจด้วยสายตา

เสียหายอะไร: เป็นข้อบกพร่องของข้อมูลที่เสียหายหนักที่สุดในชุดข้อมูลทั้งหมด การเทรนน label ที่คานหายไปคือการสอนให้โมเดลทำนายต่ำกว่าจริง ซึ่งเป็นตัวชี้วัดที่อ่อนที่สุดอยู่แล้ว และมีบ้านอีก 6 หลังที่แบกปัญหานี้อยู่ในปริมาณที่ยังไม่มีใครวัด

แก้ยังไง: บ้าน 01-05 แก้แล้วโดยการตรวจเทียบกับแบบ ส่วนบ้าน 06-11 ยังไม่ตรวจ ถูกตั้งธงซ้ำๆ ว่าเป็นงานค้างที่มีค่าสูงสุดมาแล้ว 4 เซสชัน

กฎที่ได้: ข้อบกพร่องประเภทตรวจไม่เจออย่างเป็นระบบที่พบในทุกบ้านที่สุ่มมาตรวจ ต้องสันนิษฐานว่ามีอยู่ในทุกบ้านที่ยังไม่ได้สุ่มด้วย และห้ามสร้างชุดข้อมูลสำหรับเทรนนผังคานที่ยังไม่ผ่านการตรวจสอบ — ตรวจให้เสร็จก่อน แล้วค่อยเทรน

วิกฤต 2026-08-02

คำสั่ง 'ตัด underscore ออกจาก element_id' จะทำให้ชิ้นส่วนที่ต่างกันจริง 17 ชิ้นยุบรวมเป็นชิ้นเดียว

เกิดอะไรขึ้น: ถ้าทำตามคำสั่งตรงๆ บันได_ST1 กับ `steps\_ST1` จะเหลือ `ST1` เหมือนกัน `B1\_خانพัก` (คานขานพัก) จะยุบรวมกับ `B1` ธรรมดา วัสดุขายคาสามชนิดและจันทันสามตัวแยกกันก็จะรวมกันหมด ตรวจพบการชนกัน 17 กรณี และย้อนคืน id ไป 37 ตัวใน 5 บ้าน

ต้นเหตุจริง: สั่งแปลงข้อความแบบเหมารวมโดยไม่ตรวจสอบว่าส่วนต่อท้ายนั้นมีความหมายที่แตกต่างกันจริงหรือเปล่า ซึ่งบางตัวมีจริงและเป็นการเข้ารหัสชิ้นส่วนโครงสร้างที่ต่างกันคนละตัว

เสียหายอะไร: เกือบทำข้อมูลหายแบบเสียๆ ไปแล้ว: ชิ้นส่วนโครงสร้างคนละตัวถูกรวมเป็น id เดียว คู่คืนไม่ได้นอกจากย้อนกลับไปด้วยแบบใหม่ จับได้ด้วยการตรวจการชนกันหลังการแปลง

แก้ยังง: เก็บส่วนต่อท้ายไว้ใน 17 กรณีที่ชนกัน พร้อมธงอธิบายเหตุผล ตรวจรอบสุดท้ายยืนยันว่าชนกัน 0 กรณี

กฎที่ได้: ต้องรันการตรวจการชนกันทั้งก่อนและหลังการแปลง id ให้เป็นมาตรฐานเสมอ ถ้าของสองอย่างที่แตกต่างกันแปลงไปเป็น id เดียวกัน แปลว่าการแปลงนั้นผิดสำหรับกรณีนั้น — คำสั่งให้ทำให้เป็นมาตรฐาน ไม่ใช่คำสั่งให้ทำข้อมูลหาย

วิกฤต 2026-08-02

element_type บานไปถึง 359 ค่าที่ไม่ซ้ำกัน จาก element ทั้งหมด 4,507 ชิ้น

เกิดอะไรขึ้น: การตรวจสอบพบว่า `element\_type` ซึ่งควรจะเป็นชุดค่าศัพท์ปิดเล็กๆ ได้สะสมค่าที่ไม่ซ้ำกันไปถึง 359 ค่า จาก element ทั้งหมด 4,507 ชิ้น

ต้นเหตุจริง: เป็น enum แบบข้อความอิสระที่ไม่มีการบังคับใดๆ ตลอด 11 บ้านที่สกัดมา ทุกครั้งที่เจอชิ้นส่วนที่ไม่คุ้น การสกัดก็แต่งชื่อประเภทที่ดูสมเหตุสมผลขึ้นมาเอง

เสียหายอะไร: การจัดกลุ่ม คิตราค่า หรือกรองข้อมูลด้วย element_type ไม่มีความหมายเลยทั้งชุดข้อมูล และโมเดลที่เทรนบนข้อมูลนี้จะเรียนรู้ label ที่ความหมายใกล้เคียงกัน 359 ตัว

แก้ยังง: เพิ่มรายการ element_type มาตรฐานใน §0.4 และให้ตัวตรวจบังคับใช้

กฎที่ได้: enum ไหนที่ไม่มีเครื่องหมายบังคับ จะบานไปเป็นหลายร้อยค่าแน่นอน ต้องจำกัดมันไว้ในตัวตรวจตั้งแต่วันแรก และค่าที่ไม่รู้จักต้องถือเป็นความล้มเหลวขั้นเด็ดขาด ไม่ใช่ปล่อยผ่านเสียๆ

วิกฤต 2026-08-02

element ราว 500 ชิ้นไม่มี element_id เลย — และอยู่ในบ้าน 01-07 ไม่ใช่บ้านใหม่ๆ

เกิดอะไรขึ้น: การตรวจสอบอิสระ (เทียบชนิดข้อมูล JSON ของทุก key ในไฟล์ทั้ง 2,366 ไฟล์ โดยจงใจไม่ใช้ตัวตรวจที่มีอยู่) พบ element ราว 500 ชิ้นที่ไม่มี `element\_id` ในบ้าน 01-07 ส่วนบ้าน 08-11 ครบถ้วนอยู่แล้ว ซึ่งตรงกันข้ามกับข้อสันนิษฐานที่ว่า 'ให้ยึดตามบ้าน 01-05'

ต้นเหตุจริง: มีสองสาเหตุ: หนึ่ง ไม่เคยมีการตรวจว่ามี `element\_id` อยู่หรือเปล่า และสอง การตรวจสอบที่เจอเรื่องนี้ทำได้เพราะมัน *ไม่* พังตัวตรวจที่มีอยู่ — ซึ่งตรวจได้แค่กฎที่มันรู้จักอยู่แล้วเท่านั้น

เสียหายอะไร: element ราว 500 ชิ้นเชื่อมโยงกับ spec หรือตารางรายการใดๆ ไม่ได้เลย และข้อสันนิษฐานเรื่องบ้านต้นแบบก็ถูกพิสูจน์ว่าผิดเป็นครั้งที่สองในวันเดียวกัน

แก้ยังง: เดิม id ย้อนหลังจากมาร์คที่พิมพ์บนแบบ/item_no/รูปแบบเชิงบรรยาย พร้อมธงต่อรายการที่บันทึกว่าแต่ละตัวได้มาอย่างไร

กฎที่ได้: ต้องตรวจสอบเป็นระยะด้วยวิธีที่ *ไม่* ใช้ตัวตรวจของตัวเอง เพราะตัวตรวจหาเจอได้แค่ข้อบกพร่องที่มีคนคิดไว้แล้ว การทำ type-profiling ทุก key ทั่วทั้งชุดข้อมูลจะเจอข้อบกพร่องที่ไม่มีใครคาดคิด

เนื้อหาถอยหลัง: ตารางราคาของบ้าน 05 ยังเป็นตัวเลขผิดชุดเดิม ทั้งที่ log บอกว่า 'แก้ครบทั้ง 5 ฉบับแล้ว'

เกิดอะไรขึ้น: รายการที่ 37 บันทึกว่าตัวเลขราคาที่ผิด (1,060,000/1,560,000) ถูกแก้ครบทั้ง 5 ฉบับแล้ว ตรวจสอบด้วยการซูม 8 เท่า แต่ฉบับของบ้าน 05 หลุดไป — มันยังเป็นตัวเลขผิดชุดเดิมอยู่ ตรวจสอบบ้าน 03/04 แล้วถูกต้อง มีแต่ฉบับของบ้าน 05 ที่ผิด

ต้นเหตุจริง: การแก้ถูกตรวจสอบราย *ประเด็น* ('ปัญหาเรื่องราคาแก้แล้ว') แทนที่จะตรวจราย *ฉบับ* ('ทั้ง 5 ไฟล์แสดงตัวเลขที่ถูกต้องแล้ว') log บันทึกว่าการตัดสินใจเสร็จแล้ว ไม่ใช่ว่าการลงมือทำเสร็จแล้ว

เสียหายอะไร: ตัวเลขเงินที่ผิด — คลาดเคลื่อนหลายแสนบาท — ค้างอยู่ในชุดข้อมูลเป็นเวลานาน ขณะที่ log อ้างว่าแก้แล้ว และนี่คือครั้งที่สองที่ความล้มเหลวแบบ 'log บอกว่าแก้แล้ว' กัดโปะเจตน์นี้

แก้ยังไง: ค้นหาตัวเลขตามข้อสรุปของรายการที่ 37 (เป็นการลงมือทำตามการตัดสินใจที่ตรวจสอบแล้ว ไม่ใช่การตัดสินใจใหม่) ส่วนตัวเลขเก่าเก็บไว้ใน `*_printed_as`` พร้อมธง RE-CORRECTED

กฎที่ได้: คำว่า 'log บอกว่าแก้แล้ว' ต้องพิสูจน์รายฉบับ ไม่ใช่รายประเด็น เมื่อการแก้ไขกระทบไฟล์ N ฉบับ การตรวจว่าเสร็จต้องนับให้ครบ N ฉบับที่ตรวจแล้ว — ถ้าน้อยกว่านั้นคือคำกล่าวอ้าง ไม่ใช่การแก้

roof_frame_plan ถูกแยกเป็นสอง pattern — คานหลังคาของ 8 ใน 11 หลัง ไม่เคยไปถึง BOQ เลย

เกิดอะไรขึ้น: ไฟล์โครงหลังคาทั้ง 12 ไฟล์จาก 11 หลัง ระบุ discipline: "structural" เหมือนกันหมด แต่ 4 ไฟล์เป็น pattern: "plan" ส่วนอีก 8 ไฟล์เป็น pattern: "roof_plan" ในขณะที่ `buildElements()` ของ Constant อ่านแค่ `pattern == 'plan'` คานหลังคาของ 8 ใน 11 หลังจึงหายไปจาก BOQ แบบเงียบๆ เจอตอนแยก subtask ของผังตามชนิดชิ้นส่วนโครงสร้าง

ต้นเหตุจริง: ชื่อ pattern ว่า 'roof_plan' อ่านแล้วดูเป็นที่ยู่ที่ถูกต้องของแผ่นโครงหลังคาอย่างชัดเจน แต่ในระบบจัดหมวดหมู่ roof_plan ตั้งใจให้ใช้กับแผ่นหลังคาฝั่งสถาปัตย์ ส่วนโครงหลังคาเชิงโครงสร้างต้องเป็น plan คนทำ annotation เลยใส่ตามสัญชาตญาณ แล้วก็ไม่มีใครตรวจว่า pattern กับ discipline ไปด้วยกันได้ไหม แถมผังที่เอาไปใช้ก็โครงทั้งเงียบๆ แทนที่จะเตือนว่าเจอ pattern ที่ไม่รู้จัก

เสียหายอะไร: เป็นบั๊กข้อมูลหายของจริง: ชิ้นส่วนโครงสร้างทั้งคานหายไปจาก BOQ ของบ้าน 8 ใน 11 หลังที่ทำ annotation ไว้ ความแม่นยำของ t03 ที่วัดเทียบกับ ground truth ชุดนี้ ก็เท่ากับวัดเทียบกับข้อมูลที่ไมครบ

แก้ยังไง: ยังไม่ได้แก้ในตัวข้อมูลจนจบ session — บันทึกไว้ว่าเป็นสิ่งที่ต้องแก้ บรรเทาไว้ก่อนด้วยการเขียน §1 pattern #12 roof_plan ใหม่ เพิ่มบล็อกเตือน '⚠️ roof_plan vs plan' ที่ตอกให้ชัดว่าโครงหลังคาเชิงโครงสร้างต้องเป็น plan และเขียนกับดักนี้ลงในสเกล op04 ด้วย

กฎที่ได้: อะไรก็ตามที่กรองข้อมูลตามค่าในระบบจัดหมวดหมู่ ต้องเตือนเมื่อเจอค่าที่ไม่รู้จัก ไม่ใช่ทั้งเงียบๆ และทุกคู่ discipline+pattern ควรถูกตรวจโดย check_format.py — ป้ายที่ดูสมเหตุสมผลแต่ผิด แล้วคนทำ annotation ใส่ซ้ำๆ กันทุกคน จะมองไม่เห็นเลยถ้าไม่มีกฎที่ตรวจข้ามฟิลด์

ป้าย `roof\_plan` ผิดบนโครงหลังคาเชิงโครงสร้างของ 8 หลัง ทำให้ BOQ ใน Constistant ข้อมูลหายจริงบนระบบใช้งานจริง

เกิดอะไรขึ้น: บ้าน 8 หลังมีแผ่นโครงหลังคาเชิงโครงสร้างถูกติดป้ายเป็น `roof\_plan` แทนที่จะเป็น `plan` ในขณะที่ pipeline ของ BOQ ใน Constistant ไม่ได้กิน `roof\_plan` ขึ้นส่วนพวกนั้นจึงถูกทิ้งจากการถอดปริมาณแบบเงียบๆ เป็นบั๊กข้อมูลหายบนระบบใช้งานจริง

ต้นเหตุจริง: มีค่าในชุดคำศัพท์ประเภทหน้าอยู่ใน ground truth ที่ไม่มีใครปลายทางรองรับ และไม่มีอะไรตรวจว่าทุกค่าที่ผู้ผลิตข้อมูลพ่นออกมามีคนรับหรือเปล่า ป้ายที่ผิดจึงกลายเป็นการทิ้งข้อมูลเงียบๆ แทนที่จะเป็น error

เสียหายอะไร: ยอด BOQ ในแอป Constistant ที่ใช้งานจริงต่ำกว่าความเป็นจริงในส่วนโครงหลังคาของบ้าน 8 หลัง โดยไม่มี error ให้ใครเห็นเลย

แก้ยังไง: เปลี่ยนป้ายจาก `roof\_plan` เป็น `plan` ทั้ง 8 หลัง โดยทำไปพร้อมกับรอบปรับ schema ให้เป็นมาตรฐาน

กฎที่ได้: ทุกค่าใน enum ที่ฝั่งผลิตข้อมูลพ่นออกมาได้ ต้องมีฝั่งรับที่ชัดเจน หรือไม่ก็ต้องมีการปฏิเสธแบบต่างๆ ค่าที่ปลายทางเมินเงียบๆ คือบั๊กข้อมูลหายที่รอวันปะทุ

beam plan ไม่เคยถูกตรวจว่าคานหายไหมเลยสักหลัง — คุณภาพ ground truth ที่ใช้เทรนยังไม่รู้เลย

เกิดอะไรขึ้น: การรีวิวเนื้อหาของบ้าน 06-11 เทียบกับแบบต้นฉบับยังค้างอยู่ และบันทึกไว้ว่า beam plan ไม่เคยถูกตรวจว่ามีคานหายไปไหม เลยสักหลังเดียว

ต้นเหตุจริง: แรงที่ใช้รีวิว ground truth พุ่งไปที่ความถูกต้องของ schema/รูปแบบ (ซึ่งเขียนสคริปต์ตรวจได้) แทนที่จะไปที่ความครบถ้วนของเนื้อหา (ซึ่งต้องเปิดแบบดูจริง) ground truth ของ subtask ที่เป็นคอขวดจึงไม่เคยถูกตรวจสอบเลย

เสียหายอะไร: เรากำลังเทรนโมเดลให้หิวเข้าหา ground truth ของ beam plan ที่ตัวมันเองอาจจะนับคานขาดอยู่ — แปลว่า recall 0% อาจจะบางส่วนเป็นการวัดเทียบกับ ground truth ที่ไม่ครบ แยกไม่ออกว่าโมเดลพังหรือ ground truth พัง

แก้ยังไง: ยังไม่ได้แก้ — ยกยอดไปเป็นงานค้าง

กฎที่ได้: ตรวจสอบ ground truth ของ subtask ที่กำลังจะไป optimize *ก่อน* ที่จะเสียเงินค่า GPU ไปกับการ optimize มัน ตัวเลข recall น่าเชื่อถือได้แค่เท่าที่ ground truth ของมันน่าเชื่อถือเท่านั้น

Schema revised 43 times in 5 weeks with zero retrofit of already-extracted houses (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Makham asked why the gridmaster files of house 20 and house 28 looked different from each other. Tracing it back revealed the primary schema had been edited 43 times over 5 weeks and older houses were never migrated forward, so the dataset contained many mutually incompatible generations of the same file type.

ต้นเหตุจริง: No migration step was attached to the schema-change process. Each spec edit was treated as a forward-looking rule for new extractions only, with no obligation to sweep existing files, so drift accumulated silently one revision at a time.

เสียหายอะไร: 49 houses of training data internally inconsistent; the previous sweep on 08-24 had only covered 11 houses, so 38 houses remained stale until this session. Model was being trained on a mixture of schema generations.

แก้ยังไง: Normalized both trees (json_แก้ไขแล้ว/ and rawjson_ยังไม่ได้แก้ไขโดนคน/) against the current spec in one pass — 485 fixes / 423 files / all 49 houses.

กฎที่ได้: Every schema revision must ship with either a retrofit script run over all existing data or an explicit written decision that old files stay on the old generation. Never leave "retrofit later" implicit — 43 unrecorded deferrals is what a silently forked dataset looks like.

A 1,220-line old-value→new-value log was sitting inside the training-data folder as a cross-house contamination channel (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `สิ่งที่ต้องแก้.md`, a 1,220-line log recording per-house old values and their corrections, was stored inside the training-data folder. A model extracting house 30 could read numbers belonging to house 01 out of that file and pick the wrong house's values.

ต้นเหตุจริง: A human working document was filed in the same directory as machine-consumed training data, with no rule separating "notes about the data" from "the data itself".

เสียหายอะไร: Live cross-house data-contamination risk across the whole training set — the model could be fed another house's dimensions as if they were the current one.

แก้ยังไง: Moved to `wait\_for\_thing/stale\_value\_logs/` with a README explaining why.

กฎที่ได้: Never put per-item human correction notes inside a folder a model reads. Working notes go in a separate tree; if a file lists concrete values from multiple subjects, it is a contamination vector by construction.

ชุด validation ที่รับต่อมา เป็นสับเซตของข้อมูลเทรน 100%

เกิดอะไรขึ้น: ตอนรวมชุดข้อมูล 4 pass เข้าด้วยกัน พบว่าไฟล์ validation ของ t04 ที่มี 229 แถว ซ้ำกับชุดเทรนทั้งหมด (id ซ้ำ 229/229) และบ้านสองหลังที่ t05 เลือกมาเป็น validation ก็อยู่ในชุดเทรนของ t04 ทั้งคู่

ต้นเหตุจริง: ชุดแบ่งข้อมูลถูกส่งต่อข้ามรอบโดยไม่มีใครตรวจ id ซ้ำอีกครั้ง แกรมรอบก่อนหน้าแบ่งข้อมูลทีละแถวแทนที่จะแบ่งทีละบ้าน บ้านหลังเดียวกันจึงไปโผล่ทั้งสองฝั่ง

เสียหายอะไร: ตัวเลข validation ทุกตัวที่ออกมาจาก split ชุดนั้นไม่มีความหมายเลย เพราะเป็นการวัดโมเดลด้วยข้อมูลที่มันท่องจำไว้แล้ว

แก้ยังง: หักชุด val เดิม แล้วแบ่งใหม่ทีละบ้านด้วยกฎเดียวกันทุก pass พร้อมเพิ่มการตรวจการปนเปื้อนของ val ลงใน smoke_4pass.py

กฎที่ได้: ทุกครั้งที่รับชุดข้อมูลต่อมาหรือรวมชุดข้อมูลเข้าด้วยกัน ต้องตรวจความถูกต้องของการแบ่งใหม่ทั้งระดับ id และระดับบ้าน — ห้ามเชื่อว่าไฟล์ val ของรอบก่อนสะอาดอยู่แล้วเด็ดขาด

ใส่ label แบบ batch ให้หน้า pass0 จำนวน 89 หน้า โดยตั้ง sheet_code เป็น null ทั้งที่เฉลยอ่านออกชัดเจน

เกิดอะไรขึ้น: ตอนใส่ label แบบ batch ให้หน้า material_list มีการตั้ง sheet_code กับ sheet_name เป็น null ทั้ง 89 หน้า ทั้งที่เฉลยแสดงค่าไว้ชัดๆ อย่าง 'ปร.4 (แผ่นที่ 1/23)' และ prompt ของ pass0 เองก็เขียนไว้ว่านี่คือโอกาสเดียวที่จะเก็บค่าพวกนี้ เพราะ pass ถัดๆ ไปจะถูกครอบห้วกระดาษออกไปแล้ว

ต้นเหตุจริง: สคริปต์ batch เขียนขึ้นมาเพื่อตอบคำถามเรื่อง layout (ครึ่งไหนเป็นแผ่นไหน) แล้วไม่เคยกลับไปดูฟิลต์อื่นที่จำเป็นเลย และไม่มีใครอ่านคำเตือน 'โอกาสเดียว' ใน prompt ซ้ำก่อนจะสร้าง label 89 อัน

เสียหายอะไร: จะสอนโมเดลให้ข้ามข้อความที่มันอ่านออกจริงๆ และทำให้เลขแผ่นของหน้าเหล่านั้นหายไปถาวร เพราะไม่มี pass ไหนหลังจากนี้เห็นหัวกระดาษอีกแล้ว จับได้เพราะมะขามถามว่า pass0 ทำ OCR ด้วยหรือเปล่า

แก้ยังง: เขียนใหม่ให้ฟิลต์ระดับบนสุดเก็บแผ่นที่เลขน้อยกว่า ส่วนแผ่นที่สองเก็บไว้ใน warnings[] เป็นข้อความที่อ่านได้ และทำให้สคริปต์รันซ้ำได้โดยไม่พัง ด้วยการหาหน้าเป้าหมายจากเฉลยโดยตรง ผลคือหน้า pass0 ที่มี sheet_code จริงเพิ่มจาก 63.2% เป็น 80.1%

กฎที่ได้: ตอนสร้าง label แบบ batch ต้องกลับไปอ่านข้อกำหนดใน prompt ให้ครบทุกฟิลต์ที่จำเป็น ไม่ใช่แค่ฟิลต์ที่กำลังทำอยู่ และฟิลต์ไหนที่ prompt บอกว่าเป็น 'โอกาสเดียวที่จะเก็บ' ห้ามตั้งเป็น null เด็ดขาดถ้ายังอ่านออก

วิกฤต 2026-08-31

รายชื่อ VAL_HOUSES ที่อาร์ดโค้ดไว้ ชนกับการแบ่ง fold ของ t04 ทำให้บ้านหลังหนึ่งรั่วข้าม pass

เกิดอะไรขึ้น: บ้าน_ใหญ่_1ชั้น_01 อยู่ฝั่ง train ตามรายชื่อ val ที่อาร์ดโค้ดไว้ แต่อยู่ฝั่ง val ใน fold0 ของ t04 ถ้าใช้ VAL_HOUSES แบบตายตัว บ้านหลังนี้จะอยู่ใน train ของ pass0 และอยู่ใน val ของ pass1 ภายในโมเดลตัวเดียวกัน

ต้นเหตุจริง: มีผู้มีอำนาจตัดสินใจการแบ่งข้อมูลสองเจ้าพร้อมกัน (รายชื่อบ้านที่อาร์ดโค้ด กับไฟล์ fold ของ t04) แล้วเอามาใช้กับโมเดลตัวเดียวกัน

เสียหายอะไร: ข้อมูลรั่วข้าม pass ซึ่งจะทำให้ตัวเลข validation สูงเกินจริงโดยไม่มีใครมองเห็น

แก้ยังไง: เลิกใช้รายชื่อ VAL_HOUSES แบบตายตัว แล้วให้การจับคู่บ้านกับฝั่ง train/val ของ t04 เป็นแหล่งความจริงเพียงแหล่งเดียวของแต่ละ fold ใช้เหมือนกันทุก pass

กฎที่ได้: ในหนึ่งโมเดล ต้องมีผู้กำหนดว่าบ้านหลังไหนอยู่ฝั่งไหนแค่เจ้าเดียว และใช้แบบเดียวกันกับทุก pass ของข้อมูล — การมีรายชื่ออาร์ดโค้ดเจ้าที่สองคือรอยร้าวที่รอวันเกิด

วิกฤต 2026-09-01

All slabs price at ฿0 because floor_area_sqm has no producer in the raw-extraction path (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Across every simulated house run, slab (พื้น) lines came out at ฿0. The cause was already documented in CLAUDE.md: `floor\_area\_sqm` has no producer anywhere in the raw-extraction path.

ต้นเหตุจริง: The extraction pipeline never produces the one field the slab BOQ formula needs. Compounded by `plan\_slab` having zero training examples in the entire system, so the model was never taught to answer slab plans at all.

เสียหายอะไร: Slab concrete, rebar and formwork all silently priced at ฿0 in every BOQ produced through the real extraction path — a systematically under-stated total on real construction quantities.

แก้ยังไง: Not fixed — recorded plainly as still broken.

กฎที่ได้: When a BOQ category consistently reads ฿0, treat it as a missing-producer bug, not an empty category. Trace every required field back to the stage that produces it, and if no stage does, the feature is not implemented regardless of how green the pipeline looks.

Multi-view pages queued for manual pass0 labelling were counted and printed but never entered the real training file (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `build\_pass0()` in `build\_t05\_night.py` reads only `pass0\_labels.jsonl`. The manual queue — which holds all multi-view pages, mostly footing+beam pairs — was read only to print a count. It never entered the actual training data, and nobody has resolved it (about 40 pages per house on average).

ต้นเหตุจริง: The manual queue was created as a holding area with no consumer wired to it. Reading it for a count gave the false impression it was integrated into the pipeline.

เสียหายอะไร: The single hardest and most valuable page category (multi-view pages — exactly the ones that keep failing at generation) has zero training signal. ~40 pages per house unlabelled.

แก้ยังไง: Not fixed — recorded as still pending.

กฎที่ได้: A holding queue that is only counted, never consumed, is a data-loss channel disguised as progress. Any queue must have a named consumer and an assertion that its items eventually leave it.

plan_slab has zero training examples anywhere in the system (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Checking the real `pass0\_train+val.jsonl` (171 rows) showed: plan_beam 14, plan_footing 7, also_gridline 32, and `**plan_slab = 0` across the entire system**.

ต้นเหตุจริง: Training-data coverage was never audited per subtask before training. A subtask existed in the taxonomy with no examples behind it, and nothing flagged the empty class.

เสียหายอะไร: The model can never answer slab plans — which is a second, independent cause of the \$0 slab BOQ problem. Any slab result from the model is untrustworthy by construction.

แก้ยังไง: Not fixed — recorded as needing added training data.

กฎที่ได้: Before any training run, print a per-class example count and fail the build on any class with zero (or near-zero) examples. A subtask with no training data is a lie in the taxonomy.

สูง 2026-07-05

deleteProject() did not delete Site/Foundation/manpower localStorage keys, orphaning project data (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A bug was found and fixed in project-store.js: `deleteProject()` was not removing the Site Investigate, Foundation Design and manpower-config localStorage keys, leaving orphaned data behind after a project was deleted.

ต้นเหตุจริง: The list of project-scoped storage keys was hand-maintained inside deleteProject(), while feature modules independently declared their own key strings. Every new project-scoped key had to be remembered in a second place, and it was not.

เสียหายอะไร: Deleted projects left residue in localStorage. Beyond the wasted space, orphaned Site/Foundation data can be picked up by a later project reusing an id, and it silently contradicts the user's belief that deletion removed everything.

แก้ยังไง: Fixed the deletion, and structurally: created js/shared/storage-keys.js as the single registry of every localStorage key, with PROJECT_SCOPED_KEYS derived automatically from the registry instead of hand-maintained, and fixed 14 feature modules that had been declaring key strings themselves.

กฎที่ได้: Any list that must stay in sync with a set of things (keys to delete, keys to sync, keys to migrate) must be derived from the registry, never hand-typed. A hand-maintained parallel list is guaranteed to drift the first time someone adds an entry in a hurry.

สูง 2026-07-06

Round 1 wrongly classified real structural gridlines on MEP pages as free-text labels (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On MEP pages 29, 35, 36, 37, grid circles marked 1/2/3/A/B/C/D appeared. Round t1 interpreted them as generic labels and tagged them `grid\_ref\_is\_free\_text\_not\_grid\_notation: true`. The independent round-2 agent read the actual printed dimension numbers and found they matched the house's main grid exactly, including dummy grids A/A" — meaning they are a real structural grid shared across disciplines, and t1 was wrong.

ต้นเหตุจริง: t1 judged the symbols by appearance and by discipline assumption ('MEP sheets don't carry structural grid') instead of verifying against the measurable evidence printed on the page — the dimension numbers. There was no central grid master file at the time of t1 to check against, so verification was not even possible.

เสียหายอะไร: Four pages of the dataset carried a wrong structural interpretation, and cross-discipline grid alignment was lost. At end of 2026-07-06 the conflict was still not decisively resolved ('need to decide which schema to treat as the standard'), so the dataset held two contradictory readings of the same pages.

แก้ยังไง: Not fully fixed. t2 used `grid\_source` referencing the central grid master file instead; the t1-vs-t2 conflict was left as pending item #4 with the note that t2 is likely more correct but the image should be re-checked before finalizing.

กฎที่ได้: Classify a symbol by the measurable evidence next to it (printed dimensions matched against the house grid master), never by which discipline's sheet it appears on. A grid master file must exist before the first extraction round, not be invented after one round has already guessed.

สูง 2026-07-06

Floor marker symbols SO/SI/SX/ST were classified as unknown_symbol when they are slabs (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The floor marker symbols SO/SI/SX/ST had been extracted as 'unknown_symbol'. Comparison against claude_output_03 confirmed that the `slab` element_type covers them, and t2 changed their element_type to 'slab'.

ต้นเหตุจริง: The extraction fell back to a catch-all 'unknown' bucket rather than looking up what the symbol legend on the drawing actually defines. A permissive unknown bucket removes the pressure to resolve the symbol.

เสียหายอะไร: Slab elements were invisible to any downstream consumer filtering by element_type — no slab area, no slab concrete, no slab rebar for these marks. Note the change was made under a 'refine as appropriate' mandate and was never formally confirmed by Makham, so the correction itself is unratified.

แก้ยังไง: Generation 3.1 point 3: confirmed `slab` covers SO/SI/SX/ST; element_type changed in round t2. Formal confirmation still outstanding.

กฎที่ได้: An 'unknown' classification is a defect report, not an answer — every unknown must be triaged against the drawing's own symbol legend before the round closes. And a schema change made under a broad 'refine as appropriate' mandate still needs explicit ratification, or the dataset carries an unsigned decision.

สูง 2026-07-06

Schema had no way to express an unnamed gridline, so beam B1 came out wrong in both the fresh extraction and Gen 2 (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Makham personally spotted on page 06 a wall/beam line running the full height of the building at 2.55 m from grid '1', with no grid-marker circle at all in the original drawing. This matched beam B1, which had been problematic in both the fresh self-extraction of page 19 and in claude_output_03 (Gen 2) — the same defect independently in two generations.

ต้นเหตุจริง: The schema assumed every structural line sits on a named grid. Real drawings contain unnamed gridlines, so extraction had nowhere to put B1's position and produced low-confidence or wrong output every time — a schema gap presenting as an extraction quality problem.

เสียหายอะไร: Beam B1's position was unreliable across two independent extraction generations. It also took a human eyeballing the drawing to notice, not any automated check. Once found, it required a new convention, a new master file, and re-extraction of the affected pages.

แก้ยังไง: Designed the Dummy Grid convention (prime notation appended to the nearest main grid: 1', A', 3', always stored with type: 'dummy', kept separate from type: 'named'), and created the central grid master file บ้านเล็ก_1ชั้น_01_หน้า00_gridline.json. Recorded as Generation 3.2.

กฎที่ได้: When the same element reads badly across two independent extraction attempts, stop blaming the reader and check whether the schema can even represent the truth. A recurring low-confidence element is a schema gap until proven otherwise.

สูง 2026-07-06

element_id read as 'S1' when the drawing says 'SI', with the sheet_name wrong on the same file (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: In mk_test page 23, element_id had been extracted as 'S1' when the correct value is 'SI'; the sheet_name on the same file was also wrong. Both were corrected with confidence_flags recording the source of the correction.

ต้นเหตุจริง: Visual confusion between the digit 1 and the capital letter I at the drawing's resolution, with no zoom tool available and no cross-check against the set of valid slab marks for the house (SO/SI/ST/SX), which would have rejected 'S1' immediately.

เสียหายอะไร: A wrong element id breaks the spec-join by element_id, so that slab's specification would silently attach to nothing or to the wrong element. Since the same file also had a wrong sheet_name, the error was not isolated.

แก้ยังง: Corrected element_id S1 → SI and fixed sheet_name in mk_test page 23, adding confidence_flags explaining the source of the correction.

กฎที่ได้: For character classes that look alike at drawing resolution (1/I, 0/O, 5/S, 8/B), validate every extracted id against the house's known mark vocabulary and reject anything outside it. Never accept a free-form id where a closed set exists.

สูง 2026-07-10

ชุดข้อมูล 101 ไฟล์ชุดเดียวกันมีอยู่สองโพลเดอร์ โดยไม่มีใครตัดสินว่าอันไหนคือตัวจริง

เกิดอะไรขึ้น: mk_test/t4/ ถูก copy (ไม่ใช่ move) เข้าไปที่ rawjson_.../01House_Small_1Story_01/ ทำให้ชุดข้อมูลเดียวกันอยู่สองที่ และไม่มีการตัดสินอย่างเป็นทางการว่าจะแก้ที่อันไหนต่อไป ตอนลงไล่ขั้นตอนการแก้เป็นตัวอย่างก็ต้องเดาคำตอบเอาเอง

ต้นเหตุจริง: การย้ายจากโพลเดอร์ทดสอบขึ้นไปเป็นของจริง ทำด้วยการ copy โดยไม่ปลดระวางต้นทาง แล้วเลื่อนการตัดสินใจเรื่องเจ้าของข้อมูลออกไป

เสียหายอะไร: การแก้ครั้งไหนก็ตามในอนาคตเสี่ยงจะได้ข้อมูล ground truth ชุดเดียวกันสองเวอร์ชันที่ขัดกัน ความเสี่ยงนี้ยังค้างอยู่ตอนจบวันและถูกยกยอดไปอีกหลายวัน

แก้ยังง: ยังไม่ได้แก้ — แคขยาย rule_of_tune.md ให้คุ่มครองโพลเดอร์ใหม่ด้วย และบันทึกความคลุมเครือนี้เป็นการตัดสินใจที่ยังค้างอยู่

กฎที่ได้: การเลื่อนชั้นจากทดสอบไปเป็นของจริง ต้องปลดระวางหรือแข่งขันต้นทางในการกระทำเดียวกัน — ข้อมูล ground truth ที่มีสองชุดใช้งานพร้อมกันคือบั๊กข้อมูลที่รอแค่การแก้ครั้งแรกเท่านั้น

สูง 2026-07-12

ส่วนต่างราคาฐานราก 45% ที่ผิดปกติ ถูกตีความว่าเป็นข้อมูลต้นฉบับจริง แทนที่จะทำให้อ่านใหม่

เกิดอะไรขึ้น: ตอนอ่านตารางราคา 10 หลังของบ้านหลัง 4 มีคนสังเกตเห็นความผิดปกติว่า บ้านหลังที่ 10 มีส่วนต่างราคาระหว่างฐานรากแผ่กับฐานรากเสาเข็มถึง ~45% ขณะที่บ้านหลังอื่นต่างกันแค่ 5-10% แต่ถูกบันทึกเป็นแค่ข้อสังเกตเกี่ยวกับเอกสารต้นฉบับแล้วปล่อยไว้ อีกหนึ่งสัปดาห์ต่อมา การกวาดเทียบตัวอักษรที่ 8 เท้าพบว่าตัวเลขของบ้านหลัง 10 อ่านผิดจริง (2,945,000/2,034,000 ทั้งที่ของจริงคือ 2,145,000/2,035,000) — ซึ่งคือต้นเหตุของส่วนต่าง 45% พลอมันนพอดิ

ต้นเหตุจริง: ทีมเคยสรุปไปแล้ว (จากความผิดปกติเรื่องฐานรากแผ่กับเสาเข็มของบ้านหลัง 6) ว่าตัวเลขแปลกๆ ในตารางนี้ "เป็นข้อมูลจริงจากต้นฉบับ" ความผิดปกติอันที่สองซึ่งเป็นคนละเรื่องเลยรับข้อสรุปนั้นมาใช้ต่อ แทนที่จะถูกตรวจด้วยตัวมันเอง — ค่าที่หลุดออกจากกลุ่มถูกมองเป็นแค่หมายเหตุ ไม่ใช่สัญญาณว่าอ่านผิด

เสียหายอะไร: ตัวเลขที่อ่านผิดนอนอยู่ในชุดข้อมูลของบ้านหลายหลังนานประมาณหนึ่งสัปดาห์ และสัญญาณเตือนล่วงหน้าที่ได้มาฟรีๆ ถูกทิ้งไป กว่าจะกู้คืนได้ก็ต้องรอการกวาดเทียบตัวอักษรทั้งหลายบ้านในวันที่ 19 ก.ค.

แก้ยังไ: แก้ทางอ้อมเมื่อ 2026-07-19 ด้วยการกวาดเทียบตัวอักษรอ้างอิงที่ 8 เท้าครบทั้ง 5 ชุด ซึ่งแก้หลังที่ 10 เป็น 2,145,000/2,035,000 (ส่วนต่างกลับมาที่ ~5%)

กฎที่ได้: ค่าที่หลุดจากรูปแบบของเพื่อนบ้านตัวเอง คือสัญญาณให้อ่านใหม่ ไม่ใช่แค่จุดโน้ต และอย่าปล่อยให้อ่านสรุปเก่าที่ว่า "ต้นฉบับมันแปลกจริงตรงนี้" กลายเป็นใบอนุญาตให้อ่านผิดอันที่สองซึ่งไม่เกี่ยวกันเลยผ่านไป

สูง 2026-07-14

ค่ากริด dummy ที่ได้มาจากการบอกปากเปล่า ถูกใส่ confidence_score 1 ทั้งที่ยังไม่ได้ตรวจสอบภาระ

เกิดอะไรขึ้น: กริด dummy 1"=3.45 ถูกเพิ่มเข้ามาตามคำสั่งโดยตรงและได้ confidence_score: 1 ตามธรรมเนียม ทั้งที่ในบันทึกเองก็เขียนว่ายังชี้จุดเส้นบอกภาระที่แน่นอนไม่ได้ 100% แล้วธรรมเนียมเดียวกันนี้ก็ถูกเอาไปใช้กับกริดเป็นชุดๆ ในบ้านหลัง 2, 3, 4 และ 5 ("ตามค่าที่มะขามให้มาโดยตรง confidence 1 ทั้งหมด")

ต้นเหตุจริง: ธรรมเนียมนี้เอา "มีคนบอกอย่างนี้" มาปนกับ "ตรวจสอบกับภาพแล้วอย่างอิสระ" ทั้งที่ confidence_score คือ field ที่คนใช้ข้อมูลปลายทางใช้ตัดสินว่าจะเชื่ออะไร ตอนนั้นมันบอกแค่ที่มาของข้อมูล ไม่ได้บอกว่าตรวจแล้ว

เสียหายอะไร: ค่าที่ตรวจสอบไม่ได้ ปนอยู่กับค่าที่วัดจริงจนแยกไม่ออกในชุดข้อมูล B'=12.5 และ B'=10.2 ที่ใส่ confidence 1 ทั้งคู่ ที่หลังพบว่าผิดทั้งคู่ (ที่ถูกคือ 12.2 กับ 10.1) — พิสูจน์ว่าคะแนนนี้ไม่ได้หมายความว่าอย่างที่มันบอก

แก้ยังไ: ยังไม่ได้แก้ — ธรรมเนียมนี้ยังใช้อยู่ ส่วนค่าที่ผิดถูกจับได้ที่ละตัวและแก้ที่หลังโดยเทียบกับแผ่นต้นฉบับ

กฎที่ได้: confidence_score ต้องบอกว่าหลักฐานแค่ไหน ไม่ใช่บอกว่าใครให้ตัวเลขมา ค่าที่คนบอกมาต้องมีงบอกที่มาของตัวเอง เพื่อให้การตรวจย้อนหลังหาเจอทุกตัว

สูง 2026-07-19

ความผิดพลาดแบบ B2X โพล์ซ้ำในบ้านหลัง 5 — ช่วงคานถูกบันทึกเป็น 'รอยเว้า F-E' แทนที่จะเป็นส่วนต่อขยาย F→F'

เกิดอะไรขึ้น: ตอนกำลังหาค่า dummy grid F'(14.40) ของบ้านหลัง 5 เจอการอ่านผิดในหน้า 32/33: สิ่งที่ยืนยันไว้ว่าเป็น 'รอยเว้า F-E' จริงๆ แล้วคือช่วงคาน B2X ที่ยื่นจาก F ออกไปถึง F' — ซึ่งเป็นการตีความผิดพลาดแบบเดียวกันเป๊ะกับที่เคยเจอและแก้ไปแล้วในบ้านหลัง 2

ต้นเหตุจริง: คานสั้นๆ ที่ยื่นเลยเส้นกริดหลักออกไป หน้าตาเหมือนรอยเว้าที่เว้าเข้าข้างในเป๊ะ ถ้ายังไม่มี dummy grid ที่มันยื่นไปถึงอยู่ใน grid master ตัว dummy grid กับการตีความคานมันขึ้นต่อกันไปมา การอ่านผิดเลยอยู่นิ่งๆ ได้จนกว่าจะมีฝั่งใดฝั่งหนึ่งถูกแก้

เสียหายอะไร: ความผิดพลาดประเภทเดียวกันหลุดเข้าไปในบ้านหลังที่สอง ถ้าไม่ได้ทำงานเรื่อง F' ก็คงไม่มีใครเจอ

แก้ยังไง: แก้แล้วเมื่อ 2026-07-19 ตอนเพิ่ม F'=14.40 พร้อมกับ 4'(9.6) สำหรับคานสั้น BX1 ที่ยื่นเลยกริด 4 ออกไป 0.60

กฎที่ได้: พอแก้บั๊กเรื่องการตีความในบ้านหลังหนึ่งได้แล้ว ให้ไล่กวาดแบบชนิดเดียวกันในบ้านหลังอื่นทันที — ความผิดพลาดแบบ B2X โพล์มาแล้วทั้งในบ้านหลัง 2 และ 5 และเจอห่างกันเป็นเดือนโดยบังเอิญทั้งคู่

สูง 2026-07-19

คาน B3X มีรายการ additional_bars ที่ไม่มีอยู่จริง — บันทึกเหล็กเสริมที่ไม่มีในแบบ

เกิดอะไรขึ้น: พอซูมเข้าไปที่ S-08 (หน้า 36) ของบ้านหลัง 5 แล้วนับจุดในรูปตัดจริง พบว่า additional_bars ที่บันทึกไว้ของ B3X ไม่มีอยู่ในแบบเลยแม้แต่จุดเดียว จึงลบรายการนั้นทิ้ง ไม่ใช่แก้ไข

ต้นเหตุจริง: additional_bars ถูกคัดลอกมาจาก label ที่พิมพ์ไว้ในแบบ โดยไม่เอาไปตรวจกับจุดเหล็กเสริมจริงในรูปตัด — ต้นเหตุเดียวกับบั๊กเรื่องระบุผิดหน้าคาน แต่ผลลัพธ์คือข้อมูลที่ถูกรสร้างขึ้นมามั่วๆ ไม่ใช่ข้อมูลที่วางผิดที่

เสียหายอะไร: มีเหล็กเสริมปลอมอยู่ในชุดข้อมูล ซึ่งแยกว่าเหล็กที่หายไป เพราะมันทำให้ปริมาณเหล็กสูงเกินจริง และสอนให้โมเดลเรียนรู้ที่จะสร้างเหล็กขึ้นมาเอง

แก้ยังไง: ลบทิ้งหลังจากซูมเข้าไปนับจุดในหน้า 36 แล้ว

กฎที่ได้: บั๊ก additional_bars ผิดหน้าคาน มีฝาแฝดคือเหล็กที่ไม่มีอยู่จริงเลย ทุกรายการ additional_bars ต้องยืนยันด้วยการนับจุด ไม่ใช่แค่เช็คว่าอยู่หน้าไหน

สูง 2026-07-20

ฐานรากซ้ำที่จุดกริดเดียวกัน ทำให้นับซ้ำในสองแผ่นแบบ

เกิดอะไรขึ้น: หน้า 29 (S-02 ฐานรากแผ่) ของบ้านหลัง 3 มี F0.90x0.90 ที่จุด B3 ซ้ำอีกตัวนอกเหนือจากรายการหลัก และหน้า 30 (S-03 ฐานรากเสาเข็ม) ก็มีบั๊กแบบเดียวกันเป๊ะ คือมี F5 ที่ B3 ซ้ำ

ต้นเหตุจริง: ไม่มีการตรวจว่า (mark, grid_ref) ต้องไม่ซ้ำกันภายในผังฐานรากเดียวกัน ฐานรากตัวเดียวกันที่ถูกบันทึกสองครั้งเลย รอดออกมาจากขั้นตอนสกัดข้อมูล

เสียหายอะไร: ฐานรากถูกนับซ้ำ ปริมาณและค่าใช้จ่ายบวมเกินจริงตรงๆ ในสองแผ่นแบบ และการที่มันโพล์แบบเดียวกันเป๊ะทั้งสองแบบ แปลว่ามันเป็นปัญหาเชิงระบบ ไม่ใช่เรื่องบังเอิญครั้งเดียว

แก้ยังไง: ลบตัวซ้ำออกทั้งสองที่ และตั้ง confidence_score=1 ให้หน้า 29

กฎที่ได้: ตั้งกฎตรวจสอบว่า (mark ของ element, จุดกริด) ห้ามซ้ำกันภายในแผ่นแบบเดียวกัน — ของซ้ำที่จุดกริดเดียวกันคือบั๊กเสมอ ไม่เคยเป็นข้อมูลจริง

หน้า 31 (S-04) ของบ้านหลัง 3 มี mark และจุดปลายคานผิด 7 จุด รวมถึงวางคานบนเส้นกริดที่ไม่มีคานอยู่จริง

เกิดอะไรขึ้น: การตรวจหน้า 31 ใหม่ทั้งแผ่นด้วยสเกลที่คาลิเบรตแล้ว แก้วข้อผิดพลาดไป 7 จุด: D1-C1 จริงๆ คือ D1'-C1' (ไม่มีคานแนวตั้งที่กริด 1 ในแถบ D-C ทั้งสองชั้น), คาน 2 ตัวที่ระบุว่า B3X จริงๆ เป็น B3 และมี B3X แยกต่างหากที่หายไปข้างๆ แต่ละตัว, C2-C3 เป็น B3 ไม่ใช่ B2, คานแนวตั้ง B1 ช่วง B2-A2 หายไปทั้งตัว และธง 'อาจเป็นช่องเปิด' 2 อันที่ผิดต้องปลดออก

ต้นเหตุจริง: การสกัดข้อมูลรอบแรกกำหนด mark จากความใกล้เคียง และสมมติว่าที่ไหนมีเส้นกริดตัดกันต้องมีคาน โดยไม่วัดเทียบกับสเกลที่คาลิเบรตแล้ว และไม่เช็คว่าเสาอยู่บนเส้นกริดพอดีหรือเยื้องออกไป

เสียหายอะไร: มี element โครงสร้างผิดหรือหายไป 7 จุดในแผ่นแบบเดียวของบ้านที่อยู่ในชุดเทรน mark ผิดแปลว่าสเปกปลายทาง ผิดด้วย เพราะสเปกอ้างอิงจาก mark

แก้ยังไง: แก้วครบทั้ง 7 จุดเมื่อ 2026-07-20 โดยใช้สเกลที่คาลิเบรตไว้ที่ 78.1 พิกเซล/เมตร ของแผ่นนั้น

กฎที่ได้: mark ในผังคานกำหนดจากความใกล้เคียงไม่ได้ ต้องมีสเกลที่คาลิเบรตแล้ว บวกกับการเช็คตรงๆ ว่า 'เส้นนี้มีคานอยู่จริงหรือเปล่า' ทีละแถบ ทีละชั้น

แผ่นแบบคานคอดินของบ้านหลัง 3 ขาดไป 6 element และมีการระบุระยะเยื้องผิด ซึ่งรวม 10 เทาแล้วพิสูจน์ว่าไม่จริง

เกิดอะไรขึ้น: หน้า 33 (S-06 คานคอดิน) เพิ่มจาก 17 เป็น 23 element หลังตรวจจากภาพเดิม: คานแนวตั้ง D2-C2 กับ B2-A2 หายไปทั้งตัว, คาน B'2-B'3 และ B3-B'3 ก็หายไป และ D1'-C1' เป็นการระบุผิด — รวม 10 เทาแล้วยืนยันว่าเสาอยู่บนกริด 1 พอดี ไม่มีระยะเยื้อง คานคอดินจึงเป็นวงต่อเนื่อง ไม่ได้ขาดตรงช่องบันได

ต้นเหตุจริง: เอารูปแบบระยะเยื้องที่เห็นใน S-04/S-05 มาสมมติว่าใช้กับ S-06 ได้เลยโดยไม่ตรวจกับแผ่นนั้นจริง แถบ 26% ของ element ในแผ่นนั้นไม่เคยถูกสกัดออกมาเลย

เสียหายอะไร: คานคอดินในแผ่นนั้นหายไปจากชุดข้อมูลถึงหนึ่งในสี่ และรูปแบบการต่อวางถูกบันทึกผิด

แก้ยังไง: ตรวจใหม่ทั้งหมดเทียบกับภาพต้นฉบับเมื่อ 2026-07-20 เพิ่ม element ครบทั้ง 6 ตัวและแก้วที่ระบุผิด

กฎที่ได้: รูปแบบระยะเยื้องที่เห็นในแผ่นหนึ่ง ต้องตรวจยืนยันใหม่ในทุกแผ่นที่จะเอาไปใช้ — แผ่นแบบของบ้านหลังเดียวกันต่างกันได้จริง

สูง 2026-07-28

Seven house-05 source files were corrected AFTER the dataset was built — including the gridmaster — so both t01 and t02 trained on known-wrong data (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The t02 dataset was copied byte-identical rather than rebuilt, because 7 files in house 05 (including หน้า00_gridline.json) had been edited after the dataset was originally built. Rebuilding would produce a different set and destroy comparability, so the corrections were deliberately excluded — meaning the training data still carries the pre-correction versions, including for the gridmaster example type the scripts flag as the most-missed topic.

ต้นเหตุจริง: No link between the corrected raw JSON and the built dataset — corrections to source files silently do not propagate, and there is no marker showing which built examples are stale.

เสียหายอะไร: Both t01 (28.2% recall) and t02 (20.6% recall) were trained on data known to contain corrected-away errors, and the dataset is now frozen: it can never be rebuilt without invalidating every A/B number measured against it.

แก้ยังไง: Not fixed — deliberately frozen. Documented in `tune_ai/t02/data_before_tune/DATASET_PROVENANCE.md` with checksum evidence and the reason rebuilding is forbidden.

กฎที่ได้: Record which source files a dataset build consumed and their hashes at build time. When a source file is later corrected, the build is stale — either rebuild before the comparison starts, or the whole experiment inherits the old errors permanently.

สูง 2026-07-29

Classification pass is itself wrong — elevations read as sections, 5 views claimed where 2 exist (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Pages 07/08/09 were classified as section/section/title/gridline/notes; the gold is side_profile, side_profile. The model also claimed 5 views on a sheet that has 2.

ต้นเหตุจริง: The same underlying model deficiency (likely untuned GGUF plus resolution mismatch) affects the classify pass, and no accuracy gate was placed on classification before making it the gatekeeper of the extraction pipeline.

เสียหายอะไร: Directly undermines the new 2-phase plan: wrong classification means needed pages get skipped and useless pages get extracted at 13-21 min each. Errors here are silent and expensive downstream.

แก้ยังไง: not fixed — recorded as a caveat

กฎที่ได้: When a cheap stage becomes the gate for an expensive stage, measure the cheap stage's accuracy first. A router that is wrong 30% of the time turns a saving into a loss.

Cross-reference sheet numbers (A-05, A-07, A-09) hallucinated into element marks (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On page 06 the model emitted 59 elements against a gold of 17 — inventing 12 columns, 8 doors and 4 windows that do not exist on the sheet, and naming them A-05/A-07/A-09. Those are sheet numbers: the drawing carries text like 'ดูรายละเอียดที่แผ่น A-05' (see detail on sheet A-05), which the model read as an element's name.

ต้นเหตุจริง: The model reads text correctly but has no model of what each piece of text is for; cross-reference callouts and element marks look alike lexically and nothing in the pipeline distinguishes them.

เสียหายอะไร: 3.5× element over-count on one page. If such output reached a BOQ it would price a dozen non-existent columns.

แก้ยังไง: not fixed

กฎที่ได้: Add a validation rule that rejects element marks matching the sheet-code pattern (letter-dash-digits) as marks, and cross-check element counts against the sheet's own schedule. Never let a lexical match become a semantic claim without a structural check.

Grid letters reversed and a fictitious per-intersection grid_table invented — every downstream span wrong (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Gold horizontal grid is D=0, C=4.0, B=6.0, A=9.5; the model answered A=0, B=1.5, C=5.0, D=9.5 — letters reversed and every interior distance wrong, only the 9.5 total correct. On page 06 it built a grid_table assigning a value to each grid *intersection*, which is not a real-world quantity, by shuffling the real drawing numbers (7.0, 9.5, 4.0, 3.5) into cells.

ต้นเหตุจริง: The model reproduced the right numeric set without the right structure, and the prompt instructs that spans must come from the grid table — so a plausible-looking but fabricated table silently becomes the basis for every span.

เสียหายอะไร: Every span computed downstream of the grid is wrong, with plausible-looking values that no eyeball check would flag.

แก้ยังไง: not fixed

กฎที่ได้: Validate structure, not just value presence: assert grid labels are monotonic in the drawing's own direction and that a table's cells correspond to quantities that physically exist. Fabricated-but-plausible numbers are the most dangerous kind, because they pass every 'is it filled in?' check.

สูง 2026-07-29

Wall in clay brick reported as ceiling in AAC block — a BOQ wrong at two levels (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On page 04 the gold records ผนัง (wall) built with มอญ (clay) brick; the model answered ฝ้าเพดาน (ceiling) built with มวลเบา (lightweight AAC) block — wrong on both the work category and the material.

ต้นเหตุจริง: No cross-field consistency constraint exists between work category and material, so an independently-wrong pair produces an internally coherent but entirely false line item.

เสียหายอะไร: If used for pricing this is wrong at two levels simultaneously: wrong trade and wrong unit rate. Exactly the class of error that ships a wrong BOQ total quietly.

แก้ยังไง: not fixed

กฎที่ได้: Enforce a materials-by-work-category whitelist at extraction time (AAC block is not a ceiling material) so implausible pairings fail rather than price. Any field pair that jointly determines money needs a joint validity check, not two independent ones.

สูง 2026-07-29

confidence_score / confidence_flags absent from all 6 output files (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The gold schema attaches confidence_score and confidence_flags to every element (e.g. 'ตัวหนังสือเล็กเกิน ต้อง zoom ตรวจสอบ' — text too small, needs zooming to verify). The model produced zero of them across all six files.

ต้นเหตุจริง: Same underlying schema-conformance failure — the model was not emitting the trained schema at all, and the confidence layer exists only in the trained targets.

เสียหายอะไร: No way at all to tell which parts of the output are trustworthy. The pipeline's entire mechanism for routing uncertain reads to human review is silently disabled.

แก้ยังไง: not fixed

กฎที่ได้: Treat the presence of the self-assessment layer as a hard output contract: reject any extraction with zero confidence fields rather than accepting it as complete. Output without uncertainty markers is not high-confidence output, it is unlabelled output.

สูง 2026-07-29

Page 06 returned 59 elements where the gold has 17 — and the same items were counted again in the roof view (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On the floor-plan page the model emitted 59 elements against a gold of 17. The excess was 12 columns + 8 doors + 4 windows that do not appear on that sheet at all, and the diary notes the fabricated set was then 'duplicated again in the roof view' — i.e. the same non-existent items were counted twice across two views of one page.

ต้นเหตุจริง: The model has no notion of 'this view already covered these items'; it re-enumerates per view and invents members to fill a schema slot rather than leaving it empty. Nothing downstream de-duplicates elements across views of the same sheet, and no count-vs-gold check ran before the file was written.

เสียหายอะไร: 3.5x element over-count on a single page. Any BOQ built on this would order roughly triple the columns/doors/windows for that floor; the double-counting across views means even a corrected per-view read would still double the quantity.

แก้ยังไง: not fixed — only measured against gold; the extraction was halted pending the LoRA-in-GGUF test

กฎที่ได้: Element counts are the cheapest possible validation signal: compare count-per-page against gold (or against a sanity bound) before writing output. And treat cross-view duplication as its own failure mode — a per-view extractor needs an explicit de-duplication step, or every multi-view sheet inflates quantities.

สูง 2026-07-29

Four pages of bad extraction were written into the real results folder before the pipeline was known to be broken (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Pages 03-06 were extracted with the suspect GGUF and saved into the real ผล/ output folder before anyone knew the artifact might contain no fine-tuning. Continuing would have filled the folder with wrong data.

ต้นเหตุจริง: Production extraction was started before the artifact had a content-correctness verification (see the weak sanity gate and the shallow Phase 10 test); no staging area separated trial output from accepted output.

เสียหายอะไร: Four contaminated result files that must be discarded or re-run; near miss on filling a whole house's folder with unusable data.

แก้ยังไง: Extraction halted pending the LoRA-presence test.

กฎที่ได้: Write model output to a staging area until the artifact has passed a content-correctness check; never let an unverified pipeline write directly into the dataset of record.

สูง 2026-07-29

Model invented a grid_table giving one length per grid intersection — a quantity that does not exist — poisoning every span downstream (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On page 06 the model emitted a grid_table structure with one numeric value per grid intersection. An intersection is a point and has no length; the model took the real dimensions on the sheet (7.0, 9.5, 4.0, 3.5) and scattered them into the wrong cells. Because PROMPT_SHORT states 'Span length comes from the grid table, not your own visual estimate', every span computed from that page is wrong.

ต้นเหตุจริง: The model produced a structurally plausible but physically meaningless object, and nothing in the pipeline validates that a grid table's cells correspond to spans between adjacent lines rather than to points.

เสียหายอะไร: Not just individual wrong numbers — the prompt makes the whole span chain depend on this table, so one invented structure invalidates every downstream quantity on the page, which then feeds BOQ.

แก้ยัง: Not fixed — recorded as a defect on 2026-07-29; no schema validation of grid_table exists in the extraction path.

กฎที่ได้: Validate that emitted structures represent real physical quantities, not just valid JSON — a table indexed by points instead of by spans passes every syntactic check and corrupts everything computed from it.

สูง 2026-08-01

Dummy grid 2'=5.2 added to house 04's grid master but never cascaded into the plan files (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A new x-axis dummy grid `2` (5.2) was inserted into house 04's gridline file between grids 2 and 3, at confidence 1 — but was not propagated into the pattern=plan files (footing/beam plans on pages 31–34 and others) that reference the grid.

ต้นเหตุจริง: The grid master and the plan files that consume it are edited independently, with no cascade step or checker enforcing that a change to the grid master reaches its dependants.

เสียหายอะไร: House 04's grid master and its plan files disagree; any element positioned relative to grid 2 or 3 on those plans is now referenced against a stale grid.

แก้ยัง: Not fixed — explicitly left pending as a cascade + source-image re-check task.

กฎที่ได้: A grid-master edit is not done until every dependent plan file is updated and re-checked against the source image. Add a checker rule: every grid referenced in the master must resolve in the plans, and vice versa.

สูง 2026-08-02

Houses 10/11 stored 65 rebar values as plain strings instead of schema §6 objects (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: 65 `main\_bar`/`stirrup` values across houses 10 and 11 were plain strings (as printed on the drawing) rather than the structured §6 objects the schema requires; they had to be converted.

ต้นเหตุจริง: No shape enforcement on field values — the schema said 'object' but the checker did not test it, so an extraction could write the printed text verbatim and pass.

เสียหายอะไร: 65 rebar specifications were unparseable by anything expecting structured counts/diameters — directly the fields the downstream BBS and BOQ calculations depend on.

แก้ยังง: Converted all 65 to §6 objects, keeping the original string in `printed\_as` on every one so nothing was lost.

กฎที่ได้: Enforce field `*shape*`, not just field presence. And when converting a value, always keep the original in a `printed_as` sibling — conversion is a lossy operation until proven otherwise.

สูง 2026-08-02

บ้าน 05 เดิมคำอธิบายต่อท้าย `element_id` (F1.30x1.30_E1) ทำให้จับคู่ข้ามหน้าไม่ได้

เกิดอะไรขึ้น: บ้าน 05 เขียน id ของ element โดยเติมคำต่อท้ายเข้าไปหลังมาร์คที่พิมพ์อยู่บนแบบ พอ id ไม่ตรงกับมาร์คที่ใช้ในหน้าอื่น การจับคู่ข้ามหน้าเลยพัง

ต้นเหตุจริง: ไม่เคยมีกฎเขียนไว้ว่า `element_id` ต้องเป็นมาร์คที่พิมพ์บนแบบเป๊ะๆ ห้ามมีอะไรเกิน พอ id แบบมีคำอธิบายดูให้ข้อมูลมากกว่า ก็เลยเขียนแบบนั้นไปเลย

เสียหายอะไร: element ของบ้าน 05 จับคู่ระหว่างหน้าผังกับหน้า schedule/section ไม่ได้ — ซึ่งการจับคู่นี้คือกลไกทั้งหมดที่ทำให้สเปกไปถึงตัว element

แก้ยังง: ลด `element_id` 82 ตัวให้เหลือแค่มาร์คที่พิมพ์บนแบบ แล้วเก็บของเดิมไว้ในช่อง `element_id_printed_as` ที่หลังล็อกเป็นกฎ §0.3

กฎที่ได้: หน้าที่เดียวของ id คือเอาไว้จับคู่ ข้อมูลที่เป็นคำอธิบายให้แยกไปอยู่ช่องของมันเอง ห้ามยัดใส่ id เด็ดขาด — id ที่ "ช่วยให้เข้าใจง่ายขึ้น" ก็คือ id ที่จับคู่ไม่ได้

สูง 2026-08-02

array ชื่อ `column_sections` แต่ข้างในเป็น section ของคานล้วน — cut ref ทั้ง 17 ตัวเป็นมาร์ค B

เกิดอะไรขึ้น: เจอ array ชื่อ `column_sections` ที่ข้างในไม่มีเสาเลยสักตัว — ค่า `cut_ref` ทั้ง 17 ตัวเป็นมาร์คคานหมด (B1, B2A, B3, R1, RB0, B4, B5, B6X)

ต้นเหตุจริง: ชื่อ array ตั้งตามหัวข้อบนหน้าแบบหรือตั้งตามที่เดาเอา ไม่ได้ตั้งตามสิ่งที่ข้างในมันอ้างถึงจริงๆ และไม่มีใครเช็คชื่อ array ตรงกับมาร์คข้างในหรือเปล่า

เสียหายอะไร: section ของคาน 17 ตัวถูกจัดประเภทเป็นเสา ถ้าไหลต่อไปข้างล่างมันจะถูกคิดราคา วางแผน และเสริมเหล็กแบบเสา แถมยังสอนโมเดลให้ติดป้ายคานผิดอีกด้วย

แก้ยังง: แปลงทั้ง 17 ตัวเป็น `element_type`: "beam" โดยเก็บป้าย cut เดิมไว้ใน `section_ref`

กฎที่ได้: อย่าเชื่อชื่อกล่องว่ามันบอกสิ่งที่อยู่ข้างในจริง ต้องตรวจ `element_type` เทียบกับ prefix ของมาร์คเสมอ — เจอมาร์ค B อยู่ใน array ของเสา ให้ถือว่าไม่ผ่านทันที

สูง 2026-08-07

แนวคานหน้า 18/19 ของบ้าน 14 ยังไล่ไม่ครบ — ปลอยไว้ที่ confidence ≤ 0.6

เกิดอะไรขึ้น: แนวคานบนหน้า 18 และหน้า 19 ของบ้าน 14 ยังไล่ไม่ครบ รายการถูกทิ้งไว้ที่ confidence ≤ 0.6 พร้อมติดธง `needs\_zoom\_review` แต่ตัวบ้านกลับถูกรายงานว่าเสร็จแล้ว

ต้นเหตุจริง: การไล่แนวคานบนฝั่งเป็นส่วนที่ยากที่สุดและทำอัตโนมัติได้น้อยที่สุดของการสกัดข้อมูล และ workflow ก็ยอมให้ปิดบ้านได้ทั้งที่ยังมีรายการคานความมั่นใจต่ำค้างอยู่ — เป็นปัญหาด้านเดียวกับที่บ้าน 01–05 คานหาย

เสียหายอะไร: มีบ้านอีกหลังเข้า corpus ทั้งที่รู้ว่าข้อมูลคานไม่ครบ ไปเพิ่มหนี้เรื่องคานหาย ซึ่งเป็นจุดบกพร่องที่ใหญ่ที่สุดของชุดข้อมูลอยู่แล้ว

แก้ยังไม: ยังไม่แก้ — ติดธง `needs\_zoom\_review` แล้วแบกเป็นงานค้างไว้

กฎที่ได้: คานคือจุดอ่อนที่รู้จักกันอยู่แล้วของทั้ง pipeline นี้ ถ้ารายการคานตัวไหน confidence ต่ำกว่าเกณฑ์ ให้ถือว่าบ้านหลังนั้นยังปิดไม่ได้ ไม่ใช่แค่ติดธงแล้วปลอยผ่านไป

สูง 2026-08-10

แบบโครงสร้างกับแบบสถาปัตยกรรมของบ้าน 19 ติดป้ายแกนกริดกลับด้านกัน ทั้งที่เป็นอาคารเดียวกัน

เกิดอะไรขึ้น: ในบ้าน 19 แบบโครงสร้างกับแบบสถาปัตยกรรมของอาคารเดียวกันติดป้ายแกนกลับด้านกัน — เป็นการสลับป้ายข้ามสาขาวิชาซีพจริงๆ ภายในชุดแบบชุดเดียว

ต้นเหตุจริง: แต่ละสาขาออกแบบทำแบบของตัวเองด้วยธรรมเนียมการติดป้ายกริดของตัวเอง แล้วชุดแบบถูกออกมาโดยไม่มีใครมาปรับให้ตรงกัน

เสียหายอะไร: element ไหนก็ตามในบ้านนี้ที่ต้องจับคู่ข้ามระหว่างแบบโครงสร้างกับแบบสถาปัตย์ จะถูกวางผิดแกนทันที ถ้าไม่มีใครตรวจเจอความขัดแย้งนี้แล้วแก้

แก้ยังไม: แก้ด้วยกฎลำดับความสำคัญว่า 'แบบโครงสร้างชนะ' และพอนตัวเช็คให้รับได้ทั้งสองแบบการวางแกน

กฎที่ได้: ธรรมเนียมกริดต่างกันได้*ภายใน*ชุดแบบชุดเดียว ไม่ใช่แค่ต่างกันระหว่างบ้าน ต้องตรวจหาความขัดแย้งของการติดป้ายแกนแยกตามสาขา แล้วใช้กฎลำดับความสำคัญที่เขียนไว้ชัดเจน แทนที่จะเหมาว่าอาคารหนึ่งหลังมีนิยามกริดชุดเดียว

สูง 2026-08-21

notes is the worst-drifted pattern: 55 files carry the same content under six different container keys

(ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: 55 notes files carry the same content under six container keys — notes (22), notes_sections (9), sections (8), spec_notes (3), notes_text (1), raw_text (1) — plus one-offs. The spec names sections[]; only 8 of 55 use it.

ต้นเหตุจริง: The notes pattern's container key was never enforced by check_format.py and the pattern is the least structured of the 16, so each extraction session invented a plausible key. Same failure class as roof_frame_plan and the discipline spelling split, but larger.

เสียหายอะไร: 85% of notes files are unreadable by any consumer keyed on the spec's name. The newly-defined \$4a path is empty in practice until they are rewritten.

แก้ยังไม: Not fixed. Migration of the 55 files to sections[] + notes{} recorded as pending.

กฎที่ได้: Container/key names must be enforced by the format checker, not merely named in the spec — for any pattern where the checker is silent, expect one key per session and audit key distribution across the dataset periodically.

สูง 2026-08-21

plan_column was designed as its own subtask but only 2 files exist across 11 houses (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The pass design split plan into plan_footing / plan_column / plan_beam / plan_slab, but counting the real data showed plan_column has only 2 files across 11 houses — columns almost never get their own plan sheet, appearing instead as markers on beam/footing plans and in column schedules. As a standalone subtask it would have zero training data.

ต้นเหตุจริง: The subtask split was derived from the structural element taxonomy rather than from a count of how many sheets of each type actually exist in the dataset. Designing categories before counting instances.

เสียหายอะไร: A subtask with 2 training examples would have been trained and evaluated as if it were real — producing a model branch that cannot work, and a metric nobody could interpret. Caught by counting before training.

แก้ยังไง: Not decided. Recorded as a blocking decision: merge plan_column into plan_beam, or move it to schedule. Also written into the op04 skill as a trap ('plan_column normally does not exist as a sheet — its absence is expected, not a miss').

กฎที่ได้: Count the real files per proposed category before creating it as a training subtask — a category with under ~20 examples is a metric generator, not a model; the 13-examples/11%-recall result already proved the floor.

สูง 2026-08-21

11 houses were annotated but t02 was actually trained on only 4 — most of the annotation effort never reached the model (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Dataset counting revealed that of the 11 fully annotated houses in `json\_แก้ไขแล้ว/` (1,180 files / 4,625 elements), t02's training run used only 4. That is why 35 beam-plan sheets existed on disk but the model only ever saw ~13 of them — the precise reason beam-plan recall came in at 11%.

ต้นเหตุจริง: The training set was assembled from a subset of available annotated houses and nobody re-checked coverage per page type before or after the run; the shortfall was only discovered by counting weeks later while sizing t03.

เสียหายอะไร: Roughly 7 houses' worth of human annotation (the most expensive input in this project) produced zero training value, and the resulting model shipped with 11% recall on the single most important page type.

แก้ยังไง: Not fixed — recorded in `tune\_ai/t03/dataset\_sizing.md` with a target of 25-30 houses; the next training run must use everything annotated.

กฎที่ได้: Before a training run, count examples per page type actually going into train.jsonl — not houses annotated. Unused annotation is indistinguishable from annotation never done.

สูง 2026-08-24

267 schema violations across 246 GT files had accumulated undetected (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A schema-normalize pass across all 11 houses of json_แก้ไขแล้ว found 267 changes needed in 246 files: discipline `architecture`→`architectural` (157 occurrences), 6 off-vocabulary values, container shape errors (element→elements, steel_members/void_bays/stair_details/reinforcement/sheet_index all sitting outside `elements`), stirrup_or_tie→stirrup, rebar_cover_m→cover_mm.

ต้นเหตุจริง: GT files were produced by many agents/reviewers over time with no schema validator running as a gate. The schema evolved and old files were never re-validated against it, so drift accumulated silently.

เสียหายอะไร: Every downstream consumer (training dataset builder, Constant BOQ) was reading inconsistent field names and container shapes; the training set carried the drift into the model.

แก้ยังไง: Normalized all 246 files, re-validated all 1,180 files with json.loads, logged in json_แก้ไขแล้ว/README.md § 2026-08-24.

กฎที่ได้: Run check_format/schema validation as a gate on every GT commit, not as a periodic cleanup campaign. Schema changes must trigger a repo-wide re-validation the same day.

สูง 2026-08-25

A truncated JSON file was broken in both data trees at once (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `31.../หน้า25\_plan\_footing.json` was truncated mid-file and unparseable in both `json\_แก้ไขแล้ว/` and `rawjson\_ยังไม่ได้แก้ไขโดนคน/`.

ต้นเหตุจริง: A write was interrupted or an output was saved incomplete, and the corruption propagated to both trees because the corrected tree was derived from the raw one without a parse check at the copy boundary.

เสียหายอะไร: One footing plan unusable in both trees; the F10 element data in it is permanently lost (no source to recover the missing values from).

แก้ยังไง: Closed the file so it parses, kept the complete F5 record, discarded the partial F10 fragment rather than guessing the missing values, and attached a warning.

กฎที่ได้: Parse-check every JSON file at the moment it is written and again at every copy boundary between trees. When a truncated record cannot be recovered, drop it with a warning — never fill the missing fields with a guess.

สูง 2026-08-26

House 07 footing-spread had wrong rebar counts and depths locally — anything computed from them needs recheck (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Merging house 07's footing-spread file revealed the local version had wrong real drawing data: F0.80 main_bar count was 8 when it should be 5; F1.00 and F1.50 `depth\_m` were 1.5 when they should be 1.2; F1.00 was missing its `main\_bar` block entirely. The teammate's numbers were correct.

ต้นเหตุจริง: Two people read the same drawing and got different numbers, with no reconciliation step. Nothing flagged the disagreement until git forced it.

เสียหายอะไร: Wrong rebar counts and footing depths were live in the GT. Any BOQ/BBS computation done off the old local numbers is wrong and needs re-running.

แก้ยังง: Merged per Makham's rule — teammate's real drawing numbers win, Makham's JSON shape/schema wins. Flagged that downstream computations need re-checking.

กฎที่ได้: When two people transcribe the same drawing, the data-owner's numbers win and the format-owner's schema wins — settle this rule in advance. And when GT numbers change, explicitly list what downstream results are now stale.

สูง 2026-08-28

16 of 40 grid masters were missing the z-axis entirely (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Only 24 of 40 grid masters had z-axis level data. Filling the gap took three different recovery routes: 6 houses reconstructed from existing elements, 2 dug out of prose text, and 8 required opening the original drawing images and reading them by hand. Z-chains were also verified closed on 9 houses.

ต้นเหตุจริง: z_levels was added to the gridmaster spec after many houses had already been extracted, and no back-fill sweep was run when the spec changed. The data existed in four different ad-hoc storage shapes across houses (Thai-label keys, prose "ระดับ... +X.XX", level_labels_as_printed, levels_m dicts).

เสียหายอะไร: 40% of houses had no vertical dimension available to any consumer; recovering it required manual image reading for 8 houses.

แก้ยังง: All 40/40 gridmasters now carry z_levels, harvested from each house's own reviewed per-page data. Z-chains checked closed on 9 houses.

กฎที่ได้: When a schema field is added, immediately run a coverage report across all existing data and schedule the back-fill. A field present in the spec but absent in 40% of records is worse than not having it.

สูง 2026-08-28

382 pages dumped into the catch-all `etc\_plan` bucket — 183 were real structural plans (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `etc\_plan` held 382 entries; 183 of them were actually classifiable structural plans (beam 82, footing 57, roof_frame 44). After the split, zero structural pages remained miscategorized.

ต้นเหตุจริง: A permissive catch-all bucket existed with no periodic audit. Extraction agents routed anything ambiguous into it, and because nothing failed, the bucket grew unchecked.

เสียหายอะไร: 183 structural plan pages were invisible to any consumer filtering on specific plan types — including the training dataset builder, which selects by subtask.

แก้ยังไง: Split `etc\_plan` 382 → 199, promoting 183 pages to beam/footing/roof_frame. Structural leftovers now 0.

กฎที่ได้: Audit catch-all buckets on a schedule and treat growth as a defect signal. A bucket named "etc" that holds hundreds of items is a classification failure, not a category.

สูง 2026-08-29

Relying on a prompt rule alone to stop the model deleting elements was rejected as untrustworthy (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: pass 3's prompt states the model must never delete an element from the confirmed inventory. The team explicitly did not trust the prompt to enforce this and wrote tools/merge_guard.py to reinsert any missing element as a flagged stub.

ต้นเหตุจริง: Prompt instructions are probabilistic requests, not constraints. Any invariant enforced only in natural language will be violated at some rate, and silently.

เสียหายอะไร: Prevented — but had merge_guard.py not been written, elements would have disappeared from the inventory with no trace, exactly like the pass 0 roof-beam loss.

แก้ยังไง: tools/merge_guard.py enforces the invariant in code and flags every reinserted stub for human review.

กฎที่ได้: Never enforce a data-integrity invariant with a prompt instruction alone. Enforce it in code after the model responds, and flag every violation rather than silently repairing it.

สูง 2026-08-29

The new `scaled_from_grid` rule has zero positive examples anywhere in the GT — the model is taught by prompt text alone (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Counting the GT to decide how to teach the new span-measurement rule found 113 unresolved cases across 20 files and only 2 `pos_m` nulls — meaning zero examples of a correctly measured ``scaled_from_grid`` value exist to train on.

ต้นเหตุจริง: The rule was invented after the GT was written; no correction pass has gone back to produce examples of it, and the fallback answer (unresolved) is already well represented.

เสียหายอะไร: Fine-tuning cannot teach a behaviour it has never seen an example of. The model is being asked to learn a measurement procedure from instructions in a supervised-imitation setup that only rewards copying output shape — the exact weakness the prompt work itself names.

แก้ยังไง: chose prompt-only teaching plus a P/U pixel-number requirement in the flag, with fallbacks recorded (CV code measures instead, or an intern fills 113 cases into GT)

กฎที่ได้: Before shipping a new output value, count how many GT examples of it exist. Zero means you are relying on instruction-following from a model trained to imitate, and the GPU round must be treated as a test of that gamble, not of the rule.

สูง 2026-08-30

Supabase service key leaked into the conversation transcript via automatic file diffs (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: While editing ``worker_config.json``, the Supabase service key value appeared in the chat because the system automatically sends file diffs to the assistant.

ต้นเหตุจริง: A live secret was stored in a file that the tooling automatically diffs and echoes into a conversation transcript — the secret's storage location was chosen without accounting for the tool's automatic diff behaviour.

เสียหายอะไร: A full-privilege service key is now present in a transcript. Still usable for testing, but must be revoked.

แก้ยังไง: Not yet done — flagged that the key should be revoked and regenerated after the work is finished.

กฎที่ได้: Never place a live secret in a file the agent tooling will read or diff; keep secrets in env vars or an untracked file the agent is told not to open, and rotate immediately if one ever appears in a transcript.

สูง 2026-08-30

Measured CV template coverage: only 23 of 44 houses come out exact — 19 under-detect and 2 produce false positives, and beams have no templates at all (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A direct survey of the real library (measured, not read off an old report) found 3,149 pages, a bank of only 17 templates (14 footing, 3 column), and harvest_report coverage of 'พอดี 23 / ขาด 19 / เกิน(FP) 2 หลัง'. Beams use no template bank whatsoever — beam detection is synthesised as a pair of lines from BEAM_GAP. So the CV layer that the t05 plan intends to feed the model as production-shaped input is itself wrong on roughly half the houses, and is weakest on exactly the element the model already scores 0% on (plan_beam).

ต้นเหตุจริง: The bank was grown opportunistically from whatever harvested cleanly rather than driven to coverage per house; beams were handled by a heuristic instead of templates because their shape does not match the icon-template approach. Coverage was never a release gate.

เสียหายอะไร: Any pass2.5/pass3 dataset built on this bank inherits 19 under-covered and 2 false-positive houses. Training on it would teach those errors into the model.

แก้ยังไง: Not fixed. `tools/review\_staging.py` built to speed the human review of 146 staged candidates, which is the path to more coverage, but the coverage numbers themselves are unimproved and the beam gap is unaddressed.

กฎที่ได้: Before a CV layer is promoted to 'production input' for training data, measure per-house coverage and treat under-detection and false positives as blockers — a detector that is wrong on half the corpus will teach that wrongness into whatever is trained on it.

สูง 2026-08-31

Dataset image paths pointed at a flat folder that only existed on the old training machine (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: pass1 and pass2.4 rows referenced images/<file>.png — a flattened directory that only ever existed on the t04 rental box, not in the repo.

ต้นเหตุจริง: Paths were baked at dataset-build time relative to a transient machine layout instead of the repo root.

เสียหายอะไร: Would have failed to find every image on a fresh rental; caught on free CPU before renting.

แก้ยังไง: Rewrote paths as image/<house>/<file>.png relative to the repo root.

กฎที่ได้: All dataset paths must be relative to a stable repo root, never to a rented machine's layout — and must be validated by actually opening every referenced file before renting hardware.

สูง 2026-08-31

Model emitted negative dimensions and corrupted JSON keys that reached the production output

(ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Real outputs contained height_mm:-500, spacing_mm:-200, corrupted keys such as "height_mm:-500,": ":null," and width-mn, and a beam labelled as a column.

ต้นเหตุจริง: Unconstrained free-form JSON generation with no grammar enforcement on field names or value ranges.

เสียหายอะไร: Physically impossible values could flow into quantity takeoff and pricing downstream.

แก้ยังไง: Added sanitize_elements() to worker.py: drops any element with a negative numeric dimension or a key containing ':' or ',', never repairs or guesses, and always records the drop count in warnings. Tested on real files, dropping 2 of 4 correctly.

กฎที่ได้: Validate model output against physical plausibility at the production boundary and drop rather than repair — and record every drop so silent data loss is visible.

สูง 2026-09-01

guessItemMaterialKey knew only 3 of 7 formwork types, so priced formwork lines matched nothing

(ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `guessItemMaterialKey` recognised only 3 of the 7 formwork types that actually exist in the price table. 'แบบหล่อ ฐานราก' (footing formwork) failed to match even though `formwork.footing` had a real price of ฿250/m² waiting in the table.

ต้นเหตุจริง: Two lists that must agree — the key-guesser's vocabulary and the price table's keys — were maintained separately and drifted apart.

เสียหายอะไร: Formwork cost silently missing from BOQ lines that had a real price available.

แก้ยังไง: Unified onto one table plus a test asserting every key the guesser can return has a real price, so key list and price list can never separate again.

กฎที่ได้: When two lists must agree, derive one from the other or add a test that pins their agreement. Never hand-maintain a lookup vocabulary against a separate data table.

สูง 2026-09-01

Only ~10 of 40 houses have pass0/2.4/3 annotations at all — pass0 has just 5 — so the newer passes cannot be evaluated under any split (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: When the t04 folds were audited, the underlying cause was not only a blind split but a dataset shortfall: only about 10 of the 40 houses carry pass0/2.4/3 data, and pass0 exists for just 5 houses. Every fold shows 204 pass1 eval files against 0 pass0 files.

ต้นเหตุจริง: Annotation effort tracked pass1 (which was mature) while pass0/2.4/3 were being designed; nobody counted per-pass house coverage before promising fold-level evaluation of those passes. 5 houses cannot be split into 4–5 folds and still leave eval houses anywhere.

เสียหายอะไร: Even a perfectly designed split cannot measure whether the model learned pass0 or pass3 — the capability being built is structurally unmeasurable with the current data. Any training run aimed at those passes ends with 'still don't know if it worked'.

แก้ยังไม: Not fixed — the diary records the count and the conclusion ('ถ้าใช้ fold เดิมต่อ = เทรนไปแล้วก็ยังไม่รู้ยอด'), but no annotation backfill is recorded.

กฎที่ได้: Before designing an eval split, count examples per capability, not per house. A capability with 5 labelled examples has no eval set at any k; the fix is annotation, not a better splitter.

สูง 2026-09-01

pass0 could auto-classify only 31 of 72 real pages — its plan-subtask training signal is a handful of examples (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The honest house02 simulation (driven from the real pass0_labels.jsonl instead of a derived mapping) showed pass0 resolving only 31/72 pages automatically, feeding just 24 real pass2 jobs. Checking the actual training file explained it: out of 171 rows the whole system has plan_beam 14, plan_footing 7, also_gridline 32.

ต้นเหตุจริง: pass0's training set was never balanced per subtask — the plan-page classes that matter most for the BOQ have single-digit-to-teens example counts, so more than half of a real house falls into the manual queue.

เสียหายอะไร: Over half of every house needs human labelling before extraction can even start; the automation rate quoted from earlier simulations was inflated because those simulations derived pass0 labels from filenames rather than from the model's real capability.

แก้ยังไม: not fixed — measured and recorded; requires more pass0 training data per subtask

กฎที่ได้: Measure a classifier's coverage on a whole unseen house, not on the pages it happened to be trained on — and count training examples per class before quoting an automation rate.

เรื่องที่เหลือในหมวดนี้ (71)

เอกสารเปรียบเทียบผลนารองเขียนไว้ชัดเจนว่าเป็นกราฟต์ยังไม่ตรวจ แต่ไปวางปนกับผลจริงจนคนเข้าใจผิดว่าเป็นข้อมูลอ้างอิงได้

กลาง 2026-07-02

สถานะ 'กราฟต์/ยังไม่ตรวจ' ต้องอยู่ในชื่อไฟล์หรือตำแหน่งโฟลเดอร์ ไม่ใช่แค่เขียนไว้ในหัวเอกสาร อะไรที่ยังไม่ผ่านการตรวจสอบต้องแยกไดเรกทอรีออกจากผลงานที่งานถัดไปอนุญาตให้เชื่อถือได้

จำนวนประตู/หน้าต่างในหน้า 06 เามาจากตาราง schedule แทนที่จะนับจากผังจริง

กลาง 2026-07-05

ถ้าค่าที่ได้มาจากแหล่งอื่นที่ไม่ตรงกับนิยามของฟิลด์นั้น ให้บันทึกแหล่งที่มาไว้ในฟิลด์ provenance (แบบเดียวกับที่ span_source ทำกับช่วงคาน) ไม่ใช่แค่เขียนบรรยายไว้เฉยๆ เพราะการสลับแหล่งข้อมูลแบบเสียบๆ จะดูไม่ต่างจากการอ่านที่ถูกต้องเลย

การสกัด Gen 3 รอบใหม่ทำผิดเรื่อง DB→RB ซ้ำอีก และผิดทุก element ในหน้า 21

กลาง 2026-07-06

จุดที่โมเดลพลาดซ้ำๆ ไม่ใช่ปัญหาของโมเดลอย่างเดียว ให้เขียนกฎที่ถูกต้องลงใน spec การสกัดข้อมูลไปเลย เพราะคนหรือ agent ที่เข้ามาอ่านสตๆ ก็จะมีมานิดแบบเดียวกัน ให้เขียนการ map สัญลักษณ์→ชนิดหลักเป็นกฎ และห้ามให้ขนาดเส้นผ่านศูนย์กลางมาบอกชนิดหลักเด็ดขาด

ไฟล์ stub ขนาด 0 ไบตันอนอยู่ในรีโป Training ทั้งหมด โดยทำทำเป็นเอกสาร schema

กลาง 2026-07-07

หลังย้ายไฟล์ข้ามรีโปทุกครั้ง ให้เช็คไฟล์ปลายทางมีขนาดไม่เป็นศูนย์และตรงกับต้นทาง ก่อนจะลบต้นฉบับทิ้ง

แบบ BOQ ต้นฉบับมีเลขที่ผิดจริงๆ และการอ่านแยกกันสองรอบก็ให้ผลผิดเหมือนกันทั้งคู่

กลาง 2026-07-08

บันทึกข้อผิดพลาดของเอกสารต้นฉบับไว้เป็น flag ในแถวนั้นให้ชัด เพื่อไม่ให้คนอ่านคนถัดไปมาไล่สืบซ้ำ สิ่งผิดปกติที่ยืนยันได้สองรอบคือข้อมูลเกี่ยวกับตัวแบบ ไม่ใช่ปักของคนอ่าน

แบบของบ้านหลังที่ 2 มีข้อผิดพลาดจริงหลายจุด ซึ่ง agent เลือกที่จะติดธงไว้แทนการแอบแก้เอง

กลาง 2026-07-10

การติดธงเมื่อเจอข้อขัดแย้งถูกต้องเสมอ และการเสียบๆ ปรับให้มันเข้ากันเองผิดเสมอ แต่ตรงพวกนี้ต้องถูกรวบรวมไว้ในคิวตรวจเดียว ไม่จั้นมันก็จะอยู่ในไฟล์ JSON เป็นร้อยไฟล์

แถว raw_json_data_log ของบ้านหลังที่ 2 ถูกลืม แล้วต้องมาเติมย้อนหลัง

กลาง 2026-07-11

บันทึกที่มาของข้อมูลตอนสร้างข้อมูล ไม่ใช่ตอนทำเสร็จ ชุดข้อมูลที่ยังไม่สมบูรณ์ยังต้องมีแถว log มากกว่าชุดที่เสร็จแล้วด้วยซ้ำ แค่ว่าเครื่องหมายไว้ว่ายังไม่สมบูรณ์

แบบของบ้านหลังที่ 3 มีบันไดติดป้ายผิด ชั้นติดป้ายผิด ลูกตั้งไม่ตรงกัน และสเปกแอร์หายไป

กลาง 2026-07-11

ตรวจไขว้ป้ายหรือขนาดกับแบบอีกแผ่นก่อนจะติดธง เพื่อให้ธงนั้นแยกออกว่า 'แบบผิด' หรือ 'เราอ่านผิด' และให้บันทึกสเปกที่ขาดหายไปเป็นช่องว่างที่ชัดเจน อย่าเดาค่าใส่ลงไปเด็ดขาด

ระดับชั้นที่ติดป้ายผิดในผังทอสุขภาพิบาลแนวตั้ง เกิดซ้ำทุกบ้าน และยังได้แค่ติดธงไว้ ไม่เคยแก้ไขจบ

กลาง 2026-07-12

ข้อบกพร่องที่โผล่ในเอกสารต้นทางของบ้านสามหลังติดกัน ต้องมีค่าตัดสินระดับโปรเจกต์บันทึกไว้ใน spec หนึ่งครั้ง ไม่ใช่ติดธงไล่ที่ละบ้าน

กริดหลอก A' ของบ้านหลังที่ 5 มีค่าระยะยื่นขัดกันสามค่าจากแบบโครงสร้างสามแผ่น และค้างอยู่ที่ค่าความมั่นใจ 0.65

กลาง 2026-07-13

เวลาแบบโครงสร้างขัดกันเองเรื่องระยะ คำตอบมักอยู่บนแบบสถาปัตย์หรือแบบงานระบบ ให้ขยายการค้นออกไปนอกสาขาก่อนจะติดธงว่าหาข้อสรุปไม่ได้

House 2's beam and tie-beam plans were missing beams that only surfaced during the dummy-grid cascade

กลาง 2026-07-19

A beam plan needs a completeness pass (every grid intersection accounted for), not just a label-reading pass. The dummy-grid cascade accidentally functioned as that pass — make it a deliberate step.

Warnings left in files after their issues were resolved, and one warning that was itself factually wrong

กลาง 2026-07-19

A warning is data that goes stale. Every fix must close the warnings it resolves, and a warning asserting a negative ('nothing here') needs the same evidence standard as a positive claim.

A beam recorded with a distance guessed from a box edge rather than a printed dimension

กลาง 2026-07-20

Only printed dimensions go into a dimension field. A value measured off the image must carry an explicit source flag, or it must not be recorded at all.

Beams that do not exist were recorded and had to be deleted on instruction — in house 3 and again in house 5

กลาง 2026-07-20

Over-prediction is a real failure mode in the human-corrected data too, not just in the fine-tuned model. A line without a mark and without a printed dimension is not a beam by default.

House 2's beam plan carried specs that disagreed with the section sheet they were supposed to come from

กลาง 2026-07-20

Splitting duplicated specs into specs{} solves duplication within a file but creates it across files. The section sheet must be the single spec source, with plan sheets referencing it, not copying it.

Two rooms given identical corner points but different names

กลาง 2026-07-29

Add a cheap post-extraction invariant: no two elements may share an identical point set. Self-contradiction is the easiest class of error to detect mechanically and the hardest to spot by reading.

Real Thai room names dumped into notes while element_id was invented as F1–F11

กลาง 2026-07-29

When output is wrong, check whether the right value is present in the wrong field before concluding the model cannot read it. Misplacement and misreading need completely different fixes.

'1 of 15' in the title block read as page number 15 — on two different pages

กลาง 2026-07-29

Parse X-of-Y counters explicitly rather than grabbing a nearby number, and assert page numbers are unique and within [1, total] across a document — a duplicate page number is a free, deterministic error signal.

Site plan extracted with not a single dimension — content-free output that still looked valid

กลาง 2026-07-29

Define per-pattern required fields (a site plan must carry dimensions; a schedule must carry rows) and fail the page if they are absent. 'Valid JSON' and 'contains an answer' are different acceptance criteria.

Thai construction terms mis-spelled into non-words

กลาง 2026-07-29

Validate extracted domain terms against a controlled vocabulary and flag near-misses. Spelling corruption on domain nouns is an early, cheap signal of a visual-input problem — treat it as a symptom worth tracing, not cosmetic noise.

Extraction manifest guessed the site-plan page range wrong (3 pages instead of 1)

กลาง 2026-07-30

Never trust a manifest's page ranges — open every page and classify from its own content before extracting. A section boundary is proven by what is printed on the page, not by page adjacency.

BOQ tables read by hand at 0.35–0.65 confidence because the table is rotated 90° with tiny type

กลาง 2026-07-30

Check page orientation and type size before extraction; rotate/upscale first, or route to the purpose-built extractor. Low confidence must always be recorded per-row, never averaged away.

House 10's price table says pile foundations are cheaper than spread footings — reverse of every other house

กลาง 2026-07-30

When a value contradicts a strong cross-house pattern, flag and escalate — never quietly 'fix' it to match the majority. Overwriting a real outlier is worse than carrying a flagged one.

The 'BOQ items 2-5 are identical between foundation systems' rule turned out to have a real exception — caught only because the sheets were still being read

กลาง 2026-07-30

'These sheets are always identical' is a hypothesis to test per set, not a licence to skip reading. Diff the files by content; never assume identity from a pattern seen on one or two houses.

House 11's ๗.4 BOQ tables read at 0.45–0.55 — worse than the rotated tables they replaced

กลาง 2026-07-31

Judge page readability by measured confidence on a sample, not by orientation or format. Sample 2–3 sheets and check confidence before committing to a 29-sheet set.

Dummy grids A'/D' paired with footing C0 have no supporting structure anywhere in the sheet set

กลาง 2026-07-31

An unexplained grid or mark stays flagged, never deleted. Deleting it destroys evidence that the source is incomplete; inventing its structure fabricates data.

House 07 used 3 pattern values that do not exist in the schema (detail/elevation/diagram)

กลาง 2026-08-02

Any closed vocabulary must be machine-enforced at write time. A rule that exists only in prose will be violated by the first plausible-sounding alternative.

Multi-level column schedules stored two incompatible ways — nested levels[] vs flat rows

กลาง 2026-08-02

Where the spec expresses a preference between two valid-looking shapes, the checker must reject the non-preferred one. A preference with no enforcement produces both shapes.

The identical series price table was stored 5 different ways in 5 houses

กลาง 2026-08-02

Content that repeats across houses must have one canonical representation decided once. Detect identical pages by hash and reuse the existing extraction rather than re-representing them.

The schema's specs{} block is populated in only 24 of 1,183 files — two houses have none at all

กลาง 2026-08-02

A field that matters to a downstream join needs a coverage number in the checker output, otherwise 2% coverage looks exactly like 100% coverage from the pass/fail line.

Rebar printed with a 'หยุดที่ L/8' / mid-span qualifier in houses 10-11 is still not merged into a bar face

กลาง 2026-08-02

Items that need the drawing open should be queued into a drawing-open session with an owner, not re-listed as pending at the end of every format session where they can never be done.

House 04's 10 BOQ files were stored as pattern=unknown with elements[] instead of a material list

กลาง 2026-08-02

'unknown' passing the pattern check is a hole, not a pass. The checker should flag any file whose content looks like a known pattern while being labelled unknown, at minimum as a NOTE.

Numeric fields were stored as strings and arrays across the corpus — ~170 level_m values alone

กลาง 2026-08-02

Lock the type of every field, not only its name and meaning — a spec that says what a field means but not what shape it is will collect five encodings of the same value.

Two new schema patterns were added with zero examples anywhere in the corpus

กลาง 2026-08-04

Mark a schema entry added without a real example as draft, and let the first real instance settle its shape. Also: a spec change and its checker change must land in the same commit, or the checker rejects the new spec.

บ้านหลัง 14 ไม่มีรหัสแผนแบบและไม่มีหน้าสารบัญเลย sheet_code เป็น null ทั้งหลัง

กลาง 2026-08-07

กฎที่เป็นจริงมาตลอด N หลังจากผู้จัดทำเจ้าเดียว คือธรรมเนียมของเจ้านั้น ไม่ใช่กฎสากล พอเจอชุดแบบจากแหล่งใหม่ ต้องกลับไปตรวจสอบข้อสันนิษฐานเชิงโครงสร้างใหม่ทุกข้อ และถ้าของนั้นไม่มีจริงๆ ให้บันทึกเป็น null พร้อมคำเตือน ห้ามแต่งค่าขึ้นมาเอง

บ้านหลัง 14-18 ใช้ธรรมเนียม png/doc_page ของ grid master สลับกันกับหลัง 01-13

กลาง 2026-08-09

ธรรมเนียมที่ทำตามๆ กันมาโดยไม่มีการเขียนไว้ สุดท้ายจะกลับหัวกลับหางแน่นอน ให้เขียนลง spec แล้วเพิ่มกฎในตัวตรวจทันทีที่เห็นบ้านสองหลังทำไม่เหมือนกัน

มีสามฟิลด์ที่ใช้งานมาตั้งแต่บ้านหลัง 06/11 แต่ไม่เคยอยู่ใน spec เลย

กลาง 2026-08-09

เพิ่มรายการ key ที่ไม่รู้จักเข้าไปในตัวตรวจ ฟิลด์ที่โผล่ในข้อมูลจริงแต่ไม่มีใน spec คือ spec ตกหล่นหรือไม่ก็พิมพ์ผิด ทั้งสองแบบต้องเห็นทันที ไม่ใช่มารู้เอาตอนผ่านไปสามหลังแล้ว

หน้า26.json เขียน spec.rebar เป็น array ของ string ซึ่งผิด schema 50.6

กลาง 2026-08-18

ถ้าเนื้อหาในแบบไม่พอดีกับฟิลด์ที่มีโครงสร้าง ให้ย้ายไปใส่ฟิลด์ข้อความอิสระที่เตรียมไว้ อย่าไปคลายชนิดข้อมูลของฟิลด์โครงสร้างให้หลวมลง และต้องรัน check_format.py กับทุกหลังก่อนบันทึกว่าเสร็จ

แปดหน้า BOQ ส่งออกไปเป็นแค่ stub อ้างอิง แทนที่จะเป็นการถอดข้อมูลจริง

กลาง 2026-08-18

การลดขั้นตอนจะยอมรับได้ก็ต่อเมื่อมันประกาศตัวเองในรูปแบบที่เครื่องอ่านได้ ให้เขียนเหตุผลลง warnings[] ในตัวไฟล์เลย ห้ามเขียนแค่ในไดอารี่ที่สคริปต์เทรนไม่มีวันอ่าน

มาร์คพื้น (SO/SI/ST) ไม่มี grid_ref เลยสักตัว พื้นจึงระบุตำแหน่งหรือวาดทับบนแบบไม่ได้เลย

กลาง 2026-08-20

'คาดไว้แล้ว' กับ 'ยอมรับได้' คนละเรื่องกัน ชนิดชิ้นส่วนที่ไม่มีกลไกไหนหาตำแหน่งให้ได้เลย ควรถูกบันทึกเป็นช่องโหว่ ไม่ใช่ปล่อยผ่าน เพราะทุกคนที่ใช้ข้อมูลต่อจะรับช่องโหว่นี้ไปด้วยทั้งหมด

ฟิลด์ discipline มีค่าเดียวกันเขียนสองแบบ: architectural (182 ไฟล์) กับ architecture (157 ไฟล์)

กลาง 2026-08-21

ฟิลด์ที่เป็น enum ทุกตัวต้องมีชุดค่าที่ปิดตายไว้ใน spec และมี assert ใน check_format.py ว่าค่าต้องอยู่ในชุดนั้น ตั้งแต่วันแรก ฟิลด์หมวดหมู่ที่ปล่อยให้พิมพ์อิสระจะแตกออกเป็นหลายแบบเสมอ และมันจะแตกแบบเงิบๆ เพราะยังไม่มีใครเอาไปใช้งานจะมาเป็น

สีไฟล์บันทึก rebar_cover_m เป็นหน่วยเมตร ซึ่งผิดกฎ \$0.5 ที่บอกให้ใช้ mm สำหรับขนาดชิ้นส่วน

กลาง 2026-08-21

บังคับเรื่องหน่วยใน check_format.py ด้วยช่วงค่าที่สมเหตุสมผลรายฟิลด์ ไม่ใช่เขียนไว้แค่ในเอกสาร เพราะความผิดพลาดเรื่องหน่วยเจ็บโดยธรรมชาติ ทั้งสองค่าเป็นตัวเลขที่ถูกต้องทั้งคู่ และระยะหุ้มที่ผิดไป 1000 เท่าคือตัวเลขความปลอดภัยของโครงสร้าง

ชื่อไฟล์เพี้ยนกันไปหมด (beam_plan / beam_plan_floor1 / beam_floor_plan / beam_plan_สุขา) เลยใช้ชื่อไฟล์ตัดสินใจส่งงานไม่ได้อีกต่อไป

กลาง 2026-08-21

ห้ามให้ความหมายอยู่แค่ในชื่อไฟล์ ถ้าค่าไหนสำคัญ ให้มันไปอยู่ในฟิลด์ พอชื่อไฟล์เพี้ยนไปทั้ง dataset แล้ว เครื่องมือที่อาศัยชื่อไฟล์ก็ตายถาวร ต้องเปลี่ยนไปจำแนกจากเนื้อหาแทน

หน้าแปลน 8 หน้าให้ตัวอย่างเทรน 0 ชิ้น เพราะไม่มีชิ้นส่วนไหนในนั้นตรงกับ subtask ของตัวเองเลย

กลาง 2026-08-24

บรรทัดที่เขียนว่า 'ข้ามและรายงานแล้ว' คือสิ่งที่ต้องทำต่อ ไม่ใช่เรื่องที่จบแล้ว ให้ส่งชื่อหน้าที่ถูกข้ามเข้าคิวรอตรวจ GT ไม่งั้นทุกครั้งก็ rebuild มันจะทิ้งหน้าเดิมๆ เจ็บๆ ตลอดไป

gridmaster ทั้ง 12 ไฟล์ยังไม่มี dimension_chains เลย ไม่เคยไล่ดูจากรูป และการคำนวณย้อนกลับก็ถูกปฏิเสธเพราะเป็นการปลอมที่มาของข้อมูล

กลาง 2026-08-24

การไม่ยอมแต่งที่มาของข้อมูลขึ้นมาเองนั้นถูกแล้ว แต่ช่องโหว่ที่บันทึกไว้ใน 12 ไฟล์ก็ยังเป็นช่องโหว่อยู่ดี ให้ตั้งงานไล่ดูจากรูปด้วยมือเป็นงานจริงในตาราง ไม่งั้นชั้นตอนถัดๆ ไปจะเข้าใจเองเสียๆ ว่าฟิลด์นี้มีข้อมูลครบแล้ว

ไฟล์ MEP/สถาปัตยกรรม/สัญญา/สัญลักษณ์ ราว 132 ไฟล์ยังมีโครงสร้าง container เพี้ยนอยู่ หลังจากรอบปรับ schema ให้เป็นมาตรฐาน

กลาง 2026-08-24

ถ้าการปรับให้เป็นมาตรฐานทำได้แค่บางส่วน ให้บอกว่าครอบคลุมก็เปอร์เซ็นต์ของ repo ไว้ข้างๆ บรรทัดผ่าน/ไม่ผ่าน เพราะ 'ตรวจผ่านทั้งหมด' วางคู่กับ '132 ไฟล์ยังไม่ได้แตะ' คือข้อเท็จจริงสองอย่างที่คนจะอ่านรวมเป็นอันเดียว

มยพ. 1103-64 was recorded on 08-24 as an unreadable scanned image; the real PDF has a text layer

กลาง 2026-08-25

"The PDF is a scan" is a property of one hosted copy, not of the standard. Search for another host before recording a source as unreadable and substituting a foreign standard.

25 gridmaster files still lack dimension_chains[] and unassigned_dimensions[]

กลาง 2026-08-25

Never fill a required field with an empty value to satisfy a validator — an empty array asserts a negative finding. Record the gap explicitly and queue the real work.

~76 notes files have no sections[] at all, so fc_ksc/fy_* can never be parsed for them

กลาง 2026-08-25

A parse-from-own-file fix only heals files already in the new shape; count and record the remainder explicitly, or the fix reads as complete when it covered half the set.

22 files had the wrong notes container shape and 20 more carried ad-hoc one-off keys outside notes{}

กลาง 2026-08-25

A schema with no closed key vocabulary invites improvised keys; the checker must reject unknown keys, not just validate known ones.

The one available non-BMA-series house has no plan_footing/plan_beam GT, so cross-series validation is blocked by missing data

กลาง 2026-08-25

Track GT coverage by drawing series, not just by house count — a dataset of 44 houses from one office validates nothing about a second office.

Four hard validation FAILs: BOM in 22 files, dummy grid, prose in grid_ref

กลาง 2026-08-28

Validate field `*format*`, not just presence — a `grid_ref` field should reject anything that isn't a grid identifier. And add a BOM check to the format checker; encoding bugs are invisible in every editor that hides them.

Two source drawings contradict themselves — recorded, not resolved

กลาง 2026-08-28

When source data contradicts itself, record both readings and flag it — never silently pick one. A guessed reconciliation is indistinguishable from real data once it's in GT.

Four recurring drawing-classification traps documented after repeatedly getting them wrong

กลาง 2026-08-28

Every time a classification mistake is made twice, write it into the spec as a named trap. A trap list is cheaper than re-deriving the same confusion on every house.

House 08 flagged as CV garbage — but GT is the more likely one that's wrong

กลาง 2026-08-28

When a detector disagrees with GT, check the drawing before assuming the detector is wrong. Log the direction of every disagreement so GT errors surface instead of being absorbed into the detector's error rate.

บ้านหลัง 51/52 (เข้าเป็น 39/40) เข้าชุด GT ทั้งที่ยังมีจุดกำกวมในแบบ 3 จุดที่ยังไม่เคลียร์

กลาง 2026-08-29

จุดกำกวมที่ติดธงแล้วยังหลุดเข้าชุด train คือการเดาแบบเงิบๆ ให้เลือกอย่างใดอย่างหนึ่ง: ติดธงนั้นเป็นฟิลด์บนตัว element เพื่อให้ปลายทางถ่วงน้ำหนักได้ หรือกันหน้าที่ติดธงออกจาก train จนกว่าจะมีคนชมดูแบบต้นฉบับ

เนื้อหาข้อความฝั่ง assistant มีสองรูปแบบปนกัน (list กับ string) ในชุดข้อมูลที่รวมแล้ว

กลาง 2026-08-31

จัดรูปแบบทุกฟิลด์ให้เป็นมาตรฐานเดียวกันตรงจุดที่รวมข้อมูล และใส่ assert เช็ค schema ไว้ใน smoke test แทนที่จะเชื่อว่าตัวสร้างข้อมูลต้นทางทุกตัวตกลงกันแล้ว

89 pages were auto-labelled from a rule verified by eye on only 2 pages

กลาง 2026-08-31

When one rule labels a whole corpus, verify it on a sample sized to the corpus, not to the time available — and re-check the batch when any bug is found in it.

RB1/RB2 appear twice with different sizes; BOQ silently uses only the first occurrence

กลาง 2026-09-02

If a mark is not guaranteed unique, first-match-wins is a coin flip. Detect duplicate marks with differing specs and flag them for review rather than silently picking one.

Page 14 eave detail references letter 'E' which does not exist on the grid, left unresolved

ต่ำ 2026-07-06

Separate structural grid coordinates from detail-callout labels in the schema so an ambiguous letter cannot silently become a gridline. And keep rewarding agents that stop and flag instead of guessing — this restraint protected the grid master.

House 6's price table shows spread footing costing more than pile foundation, in every independent read

ตำ 2026-07-11

When two independent reads agree on an implausible value, stop re-reading and escalate to a domain expert — repeated agreement means the question has moved from extraction to the source document.

A 200W water pump was filed under the wrong BOQ category in the source table

ตำ 2026-07-12

Category assignments read off a BOQ sheet need a plausibility check (does this item belong in this category?), not just faithful transcription.

The same CN contradiction reappeared in house 09 — it was a real source defect, not a misreading

ตำ 2026-07-30

Keep a cross-house anomaly index. When an oddity repeats across independent houses of the same series, classify it as a source defect and stop re-investigating it per house.

Two of house 10's BOQ sheets have an unclear work-category position, left flagged for zoom review

ตำ 2026-07-30

'100% done' should mean zero needs_zoom_review flags, or the phrase should be replaced with a count. A house signed off with open flags reads as finished to everyone downstream.

House 11's S-01 sheet index lists names that don't match the sheets themselves

ตำ 2026-07-31

A sheet's own title block outranks any index page. Never route a sheet by index metadata; read the sheet's title block and record any index disagreement as a source defect.

House 01's series price table is still unstructured raw text, flagged PENDING rather than converted

ตำ 2026-08-02

Flagging instead of guessing is right, but a PENDING flag needs a place it gets picked up from; three sessions later it was still only a line in a diary.

The same credits page was classified under four different patterns across houses

ตำ 2026-08-02

Byte-identical pages across houses are a free consistency check — a page that is the same file should never carry more than one label, and that is worth asserting in the checker.

The same physical quantity is named three ways — height_mm, height_m and depth_m

ตำ 2026-08-02

One quantity, one field name, one unit — enforced by the checker. Names that encode units invite a second name for the same thing.

18 precast plank layout sheets carry pattern:section despite plank_layout filenames — probably right, easy to 'fix' wrongly

ตำ 2026-08-21

When a correct labeling looks like a bug, write the justification into the files (warnings[] or a note field), not just the diary — a diary line will not stop the person doing a bulk cleanup six weeks later.

Three files had the literal strings 'grid'/'dummy' sitting in grid_ref instead of a real grid reference

ตำ 2026-08-25

Placeholder values are worse than empty ones — they pass presence checks and fail meaning checks; validate the format of every reference field.

151 images were downsampled for exceeding the pixel cap by only 0.13%

📅 2026-08-31

Give hard resolution caps a small tolerance band, and disprove a suspected root cause with measurements before acting on it — the DPI-inconsistency theory here was wrong.

สเปกข้อมูล และ ข้อตกลงที่ค่อยๆ เปลี่ยน

สองที่ที่ต้องตรงกัน (โครงสร้าง ชื่อ หน่วย) แล้วค่อยๆ ไม่ตรงกันโดยไม่มีใครรู้

ทั้งหมด 39 เรื่อง · วิภฤต 4 · สูง 10 · กลาง 21 · ต่ำ 4

เรื่องที่ต้องอ่านให้ครบ (14)

วิภฤต 2026-07-08

ตัวสร้าง task ไม่เคยถูกอัปเดตตาม schema Gen 4 ทำให้ทุกคนโชว์ grid_ref ว่างและไม่มีข้อมูลหลักเลยในหน้าจอ labeling

เกิดอะไรขึ้น: ไฟล์ label-studio-tasks-makham.js ยังอ่าน el.grid_ref กับ mainBar.count อยู่ ทั้งที่ Gen 4 แยกออกเป็น grid_ref_start/grid_ref_end/span_length_m และเปลี่ยน main_bar เป็นโครงสร้างซ้อน {top,bottom} ค่าจริงเลยตกไปอยู่ใน other_fields_json ที่อ่านได้อย่างเดียว ทำให้ grid_ref ของทุกคน และหลักแกนหลักของทุกคน/เสา ขึ้นว่างเปล่าใน Label Studio บั๊กนี้กระทบ t3 อย่างเจียบๆ มาตั้งแต่ต้น

ต้นเหตุจริง: มีการแก้ schema ในสเปกและในการสกัดข้อมูล โดยไม่ได้ไล่ตรวจว่าใครเป็นผู้ใช้ปลายทางบ้าง และตัวสร้าง task ก็ล้มแบบเจียบ — ช่องที่หาไม่เจอถูกเรนเดอร์เป็นกล่องว่าง แทนที่จะโยน error

เสียหายอะไร: ไฟล์ task JSON ของ t3 ที่สร้างไปแล้วทั้งหมด (และอาจ import เข้า Label Studio Cloud ไปแล้ว) แสดงข้อมูลหลักและ grid เป็นช่องว่างให้คนที่มา label เห็น ซึ่งเป็นสองช่องที่สำคัญที่สุดในชุดข้อมูล ต้องสร้างใหม่และ import ใหม่ทั้งหมด

แก้ยังไง: สอนตัวสร้างให้อ่านได้ทั้งรูปแบบใหม่และเก่า, ใส่ช่อง grid_ref_start/end/span_length_m และแยกหลักบน/ล่างลงใน XML ของ elements (จาก 3 ช่องเป็น 6 ช่อง) แล้วสร้าง task JSON ใหม่ พร้อมสุ่มตรวจกับคนปกติ (B1) และคนที่ไม่สมมาตร (B5, บน=2/ล่าง=3)

กฎที่ได้: การแก้ schema ยังไม่ถือว่าเสร็จจนกว่าผู้ใช้ข้อมูลปลายทางทุกตัวจะถูกอัปเดตในการแก้ครั้งเดียวกัน และตัวสร้างข้อมูลต้องล้มดังๆ เมื่อหาช่องที่ต้องการไม่เจอ ห้ามปล่อยกล่องว่างออกมาเด็ดขาด

วิกฤต 2026-08-25

standards-config.js carried wrong lap and cover values; real BBS cut lengths changed once corrected against มยผ. (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Reading มยผ. 1101-64 (31 pages) and 1103-64 showed the registry's values were wrong: deformed-bar lap was a single 48d instead of grade-split 30/36/45d, round-bar lap 36d instead of 40d, and covers 40/40/25 instead of beam 30 / column 35 / slab 20; the 180° hook had no 60mm minimum. A sample beam's main bar cut length moved 4998→4749 mm and its stirrup 952→1032 mm.

ต้นเหตุจริง: The registry had been populated from a different standard (วสท.) and never reconciled against the มยผ. documents it claimed to follow, with no per-value clause citation to make the mismatch visible.

เสียหายอะไร: Every BBS cut length and rebar quantity computed before this correction was wrong. Seven regression-pin test files had to be updated, each recomputed by hand.

แก้ยังไง: Migrated the whole registry to มยผ. values with per-value clause citations; values the standard does not define (top-bar 1.3×, SDM Ld) kept with the gap stated in the clause field. npm test 1249/1249 with all pins recomputed by hand.

กฎที่ได้: Every engineering constant must carry the standard and clause it came from, so a mismatch is visible on inspection rather than discovered years later. When constants change, recompute every regression pin by hand — never bulk-update pins to whatever the new code outputs.

วิกฤต 2026-09-01

worker.py wrote result files without the `pattern` field, so the adapter could not read the grid master and every beam lost its quantity (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The worker's output files omitted the `pattern` field entirely. Without it the adapter cannot recognise the grid-master page, so every beam ends up with `span\_source: 'unresolved'` — meaning zero beam quantities in the BOQ for the entire house. The training team's GT files already carried `pattern: "grid\_master"`; only the worker side had forgotten it.

ต้นเหตุจริง: The worker relied on the model to supply metadata that the worker itself already knows — it dispatched a specific subtask and therefore knew the pattern before the call was even made. Trusting the model's output for a field you already have is a needless dependency.

เสียหายอะไร: Would have caused a complete loss of beam quantities (the single largest BOQ category — ฿106,821 of ฿144,704 on house01) on the very first real run. Caught only because a full house was simulated end-to-end without a GPU.

แก้ยังไง: Added a `SUBTASK\_PATTERN` table and stamped the pattern from the worker side, not from the model response.

กฎที่ได้: Never ask the model for metadata you already possess. Any field determined by which subtask you dispatched must be stamped by the dispatcher, deterministically.

วิกฤต 2026-09-01

normalizeMainBar() only understood top/middle/bottom, so all column rebar came out null ('DBnull') (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `normalizeMainBar()` in `js/shared/schema.js` only read the nested top/middle/bottom rebar shape. Columns naturally answer flat — drawings write only '4-DB12' with no top/bottom split — so the flat `{count, dia_mm}` form yielded null and all column steel vanished from the house, surfacing downstream as 'DBnull'.

ต้นเหตุจริง: The normalizer was written against the beam data shape and assumed it was universal. Columns genuinely have a different, simpler natural shape on Thai drawings, and no fallback existed for it.

เสียหายอะไร: All column reinforcement missing from BOQ/BBS for every house; the wrong value surfaced as a visible 'DBnull' string rather than an error.

แก้ยังไง: Extended `normalizeMainBar()` to accept the flat `{count, dia_mm}` form.

กฎที่ได้: A normalizer must handle every shape the source domain naturally produces, not just the first one you built it for. When a field renders as literal 'null' text downstream, that is a normalizer gap, not a display bug.

สูง 2026-07-06

span_source field had been dropped from the spec and had to be added back after it caused a low-confidence extraction (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The `span_source` field ('grid_table' / 'local_dimension' / 'unresolved') for beam elements in the `plan` pattern was not in the Gen 3 spec. Extracting page 19 fresh hit a low-confidence problem precisely because the field was missing, and it was added back in Generation 3.1 as 'proven genuinely necessary'.

ต้นเหตุจริง: The Gen 3 schema was designed by simplifying Gen 2 without checking, field by field, why each dropped field had existed. A field that recorded provenance of a measurement was treated as noise until its absence made a measurement unverifiable.

เสียหายอะไร: Page 19 view2 (beam plan) came out at ~0.4 confidence and had to be completely re-extracted in round t2 with atomic segments plus `span_source`. A whole page of structural beam data was redone.

แก้ยังไง: Added `span_source` back into the spec for beam elements in the plan pattern (Generation 3.1 point 1) and re-extracted page 19 view2 in `mk_test/t2/`.

กฎที่ได้: When simplifying a schema, never drop a provenance field without writing down why it existed and what breaks without it. Provenance fields (where did this number come from) are the cheapest possible insurance against unverifiable data — keep them by default.

สูง 2026-07-06

Beam grid_ref was modelled as a rough aggregate count instead of atomic spans, losing per-span data (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The original Gen 3 rule stored a beam's grid_ref as a rough aggregate with a `count`. Comparison against claudes_output_03 showed this was wrong; the rule was changed so 1 entry = exactly 1 grid span, grouped by (element_id, span_length_m, span_source).

ต้นเหตุจริง: The schema modelled a beam mark as a single object because that is how it appears in the schedule, not as the set of physical segments it actually is on the plan. Aggregating before recording destroyed the per-span lengths needed downstream.

เสียหายอะไร: Every beam plan extracted under the old rule had to be rewritten. Page 19 view2 and page 20 view1 were both completely redone with atomic segments; page 20 went from ~0.5 confidence to 0.87 and yielded a complete set of 17 atomic segments that the aggregate form could never have expressed.

แก้ยังไง: Generation 3.1 point 2: grid_ref for beam changed to an atomic segment, one entry per grid span. Pages 19 and 20 re-extracted in mk_test/t2/.

กฎที่ได้: Record data at the atomic level it physically exists at, and let aggregation happen downstream. Aggregating during extraction is irreversible information loss — a count can always be derived from segments, segments can never be recovered from a count.

สูง 2026-07-06

Two agents spelled the same schema key differently (element vs elements), and the fix was to tolerate the drift rather than eliminate it (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: mk_test/t2/บ้านเล็ก_1ชั้น_01_หน้า29_floor_plan.json, produced by the MEP agent, used the key `elements` (plural) while the spec and every other file used `element` (singular). The Label Studio task script was written to check both names as a fallback instead of the data file being corrected.

ต้นเหตุจริง: Multiple agents were dispatched in parallel against a prose spec with no machine-checkable schema and no validation step on agent output. Nothing rejected a file whose top-level key did not match the spec, so drift landed silently in the dataset.

เสียหายอะไร: The dataset now contains two spellings of the same key, and every future consumer must know to handle both. The diary itself flags this as a signal that parallel agents will keep introducing this class of variation.

แก้ยังไง: Partially fixed — the task-generation script accepts both key names. The underlying data file was deliberately not corrected, and the choice between forcing exact field names vs staying tolerant was left open.

กฎที่ได้: Validate every agent's output against a machine-readable schema before it is accepted into the dataset. Making the reader tolerant of drift is a one-way ratchet: the second spelling never gets removed and every downstream tool inherits the branch forever.

สูง 2026-07-06

Gen 2 and the self-done extraction had swapped the meaning of the grid axis labels (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Comparing Gen 2 (claude_output_03) against the fresh extraction revealed that the two had swapped the meaning of the grid axes — the underlying data matched, only the labels were reversed. A clear convention was proposed: x_lines = horizontal/numeric, y_lines = vertical/alphabetic.

ต้นเหตุจริง: The axis naming convention was never written down, so each generation picked an interpretation independently. Because the data itself was identical, nothing downstream failed loudly enough to expose the disagreement.

เสียหายอะไร: Two datasets that look consistent but describe axes oppositely. Any consumer joining them by axis name would produce a transposed building. It was caught only by a deliberate manual comparison, not by any check.

แก้ยังไง: Proposed the explicit convention (x_lines=horizontal/numeric, y_lines=vertical/alphabetic) as Generation 3.1 point 9.

กฎที่ได้: Write down the meaning of every axis, direction and orientation convention in the spec, with an example, before the first extraction. Label-level disagreements where the data matches are the hardest class to detect — nothing crashes, the numbers just mean the opposite thing.

สูง 2026-07-07

Garden-fence marks C1/F/GB collide with the house's real structural marks (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Pages 16-17 (garden fence) reuse the marks C1, F and GB, which on the structural sheets mean the main column and the footing. Nothing in the data distinguishes a fence C1 from a structural C1.

ต้นเหตุจริง: Element identity in the schema is the mark string alone, with no scope/discipline qualifier — the drawing set reuses marks across unrelated sub-structures and the schema has no way to express that.

เสียหายอะไร: Merging the dataset would silently fuse a garden-fence post with a main structural column; still unresolved at the time of the entry.

แก้ยังไง: Not fixed — flagged as needing a decision on how to separate them before combining the dataset.

กฎที่ได้: Element ids must be scoped by their drawing/discipline context, not by mark alone; assume any mark string will be reused somewhere in a 60+ page set.

สูง 2026-07-07

schema แบบเก่า (ก่อน Gen4) เก็บเหล็กบน/เหล็กล่างแยกกันไม่ได้ — คาน B5 ที่มีบน 2 เส้น ล่าง 3 เส้น ถูกยุบเหลือตัวเลขเดียว

เกิดอะไรขึ้น: พออ่านหน้า 21 (S-04) ซ้ำอีกรอบ เห็นชัดจากภาพเลยว่าคาน B5/B5X มีเหล็กบน 2-Ø16mm และเหล็กล่าง 3-Ø16mm แยกกัน แต่ schema ตั้งแต่ Gen 3 ลงมาเก็บ main_bar เป็นก้อนแบบๆ `{count, dia_mm, type}` ไม่มีช่องแยกบน/ล่าง คานตัวนี้เลยบันทึกได้แค่ตัวเลขรวมตัวเดียว

ต้นเหตุจริง: ตอนออกแบบ schema แรกๆ คานตัวอย่างที่หยิบมาดูบังเอิญเป็นแบบสมมาตร (บนล่างเท่ากัน) เลยเก็บเหล็กหลักเป็นจำนวนเดียว — คือหน้าตาของ field ถูกออกแบบตามข้อสันนิษฐานที่มีต่อข้อมูล ไม่ได้ออกแบบตามโครงสร้างจริงในแบบ

เสียหายอะไร: คานทุกตัวที่สกัดด้วยรูปแบบแบบๆ นี้ เสียความต่างทางวิศวกรรมระหว่างเหล็กบนกับเหล็กล่างไปหมด — สองอย่างนี้ทำหน้าที่ต่างกัน (รับโมเมนต์ลบ vs โมเมนต์บวก) และกู้คืนจากตัวเลขรวมไม่ได้เลย พอต้องแก้ก็เลยลากไปเปลี่ยน schema ซึ่งไปหา Label Studio generator พังต่อ (บั๊ก main_bar ว่าง) และสร้างความเสี่ยงนับซ้ำที่รื้อหน้า 16/17 อีก

แก้ยังง: Gen 4 บังคับให้ทุก element ใช้โครงสร้างซ้อน `main_bar: {top:{...}, bottom:{...}}` โดยอ้างอิงหน้า 21 เป็นหลักฐานประกอบ

กฎที่ได้: ออกแบบหน้าตาของ field จากแบบที่ยากที่สุดเท่าที่หาได้ ไม่ใช่จากตัวแรกที่สุด field ที่ยุบข้อมูลจนเสียความต่างไปแล้ว จะแตกกลับคืนจากข้อมูลที่เก็บไว้ไม่ได้เลย ต้องกลับไปอ่านรูปใหม่ทุกใบ

สูง 2026-07-08

การบังคับให้แยกเหล็กบน/ล่าง ทำให้เกิดการนับซ้ำเมื่อแบบพิมพ์มาแค่บรรทัดเดียว

เกิดอะไรขึ้น: หน้า 16/17 (งานรื้อ) พิมพ์สเปกเหล็กของ C1 กับคานทับหลังมาบรรทัดเดียว ไม่ได้แยกบน/ล่าง แต่ schema บังคับให้กรอกทั้งสองช่อง ใครที่มาบวก `top.count + bottom.count` ทีหลังจะนับเหล็กชุดเดียวกันสองรอบ

ต้นเหตุจริง: กฎใน schema ถูกเขียนจากกรณีทั่วไป (คานที่ไม่สมมาตรมีเหล็กบนกับล่างแยกกันจริงๆ) แล้วตั้งเป็นช่องบังคับ โดยไม่เปิดทางให้บอกได้ว่า "ชั้นนี้มีเหล็กชุดเดียว ไม่ได้แยก"

เสียหายอะไร: ทุก element ที่มีสเปกบรรทัดเดียวมีความเสี่ยงนับเหล็กเกิน 2 เท่าแบบซ่อนอยู่ ปัญหานี้ยังไม่ถูกปิดที่ระดับ schema และยังถูกยกเป็นความเสี่ยงอยู่แม้ผ่านไปอีกรวัน

แก้ยังง: มี agent เพิ่ม field `total_main_bar_count` เข้ามาเป็นตัวกันไว้ก่อน แต่ช่องโหว่ของ schema ตัวจริงยังไม่ได้ปิด

กฎที่ได้: ช่องที่บังคับกรอก ต้องมีวิธีบอกว่า "ไม่มีข้อมูลนี้/ไม่เกี่ยวข้อง" ได้อย่างถูกกฎ ถ้าไปบังคับให้แยกในสิ่งที่ต้นฉบับไม่ได้แยก จะได้ข้อมูลที่หน้าตาดูจริงแต่บวกออกมาผิด

สูง 2026-07-10

สคริปต์นำ annotation กลับเข้าระบบไม่เคยถูกปรับให้เข้ากับ schema Gen4 และถูกทิ้งไว้หลายวัน

เกิดอะไรขึ้น: label-studio-import-annotations.js กับ -import-repeater-annotations.js ยังพูดภาษา schema ก่อน Gen4 อยู่ ถูกยกเป็นช่องว่างตั้งแต่ 7 ก.ค. ย้ายอีกวันที่ 8 ก.ค. และวันที่ 10-11 ก.ค. ก็ยังบอกซ์ดว่ายังไม่ได้แตะ แปลว่างานที่คน annotate ไว้ใน Label Studio ไม่มีทางกลับเข้าชุดข้อมูลได้เลย

ต้นเหตุจริง: การย้าย schema ทำครบแค่ขาไป (การสกัดข้อมูลและการสร้าง task) แต่ไม่ทำขากลับ และขาที่ขาดไปนั้นไม่เคยถูกจัดคิวให้ทำ

เสียหายอะไร: ข้อมูลบ้านสามหลังถูกส่งเข้า Label Studio ให้คน annotate โดยไม่มีทางเอาสิ่งที่แก้แล้วกลับมาได้ — งาน annotation ทั้งหมดไม่มีทางกลับบ้าน

แก้ยังไง: ยังไม่ได้แก้ — ค้างเป็นงานคาอยู่อย่างน้อยสัปดาห์

กฎที่ได้: ถ้าย้าย schema ให้ย้ายทั้งขาไปขากลับในการเปลี่ยนครั้งเดียวกัน — ทางส่งออกที่ไม่มีทางนำเข้ากลับ แปลว่างาน annotation ไม่มีที่ลง

สูง 2026-09-01

Footing rebar was missing only because SPEC_FIELDS lacked `dowel\_bar` — the join already worked and the data was already there (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The real root cause of the footing rebar loss was not a broken join. `buildSpecs()` in `raw-extraction-adapter.js` is already a generic element_id join, and the pile-cap detail pages (S-11/S-12) carry `element\_id` matching the plan pages exactly (F4/F6/F3) with width_mm/height_mm/dowel_bar all present. The only problem was that the field is called `dowel\_bar`, not `main\_bar`, and `SPEC\_FIELDS` did not list it.

ต้นเหตุจริง: The schema has no official field-set for footings (noted in the files' own `schema\_note`), so the training data used a domain-correct but unlisted field name. The adapter's field allowlist silently ignored it. First diagnosis blamed a missing join mechanism; the real cause was one absent field name.

เสียหายอะไร: All footing reinforcement lost across the corpus while concrete and formwork were correct all along — which made the failure look like a deep join problem rather than a one-line allowlist gap, and led to an initial misdiagnosis.

แก้ยังไง: `buildSpecs()` now falls back to `dowel\_bar` when `main\_bar` is absent (engineering-correct: dowel steel from the pedestal into the footing is the footing's main reinforcement, same shape, already parsed by `normalizeMainBar()`). F4/F6/F3 now yield 39.34/199.33/19.67 kg with status=ok, plus 2 tests.

กฎที่ได้: When data 'doesn't arrive', check the field allowlist before redesigning the join. And when the schema has no official field-set for a category, its field names will drift — grep the actual data files for the names in use rather than assuming the schema covers them.

Spread footings use element_id: null and share no key at all with the plan page (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Spread footings (ฐานรากแผ่, drawn by convention on S-09) use `element\_id: null` and are matched by size instead — but the plan page carries no width_mm/height_mm either, so there is literally no shared key between the two pages.

ต้นเหตุจริง: The drawing convention for spread footings omits the identifier the join relies on, and the schema provides no alternative correlation key. This is a genuine data-model gap, not a code defect.

เสียหายอะไร: Spread footings can never be joined to their detail pages; their reinforcement is unrecoverable with the current schema.

แก้ยังไง: Not fixed — declared out of scope and requiring a redesign, not another field addition.

กฎที่ได้: When two records must be joined, verify a shared key exists in the *real* data before building the join. If none exists, that is a schema design task — do not paper over it by adding fields one at a time.

เรื่องที่เหลือในหมวดนี้ (25)

คิด element_type ขึ้นมาใหม่ 9 ตัวกลางขั้นตอนสกัดข้อมูล แล้วเขียนลงชุดข้อมูลโดยไม่ผ่านการอนุมัติ spec

กลาง 2026-07-05

ถ้าจะขยายขอบเขต ต้องขยายคำศัพท์ที่ควบคุมไว้ (controlled vocabulary) ในการตัดสินใจครั้งเดียวกัน ตอนที่การสกัดข้อมูลต้องคิดคำใหม่ขึ้นมาเพื่อไปต่อได้ นั่นคือการตัดสินใจระดับ spec ที่ถูกทำโดยคนที่บังเอิญอยู่ตรงนั้น ให้หยุดแล้วขอให้เคาะให้ชัดก่อน ไม่งั้นมันจะถูกเขียนกระจายไปหลายสิบไฟล์

สร้าง grid master กลางขึ้นมาแล้ว แต่ไม่เคยย้อนไปแก้หน้าเดิมให้อ้างอิงมัน

กลาง 2026-07-06

การสร้างแหล่งความจริงเดียวเป็นแค่ครึ่งงาน อีกครึ่งคือย้ายข้อมูลเดิมออกจากสำเนาเก่าให้เสร็จในรอบเดียวกัน การมีธรรมเนียมสองแบบใช้งานอยู่พร้อมกัน แย่กว่ามีแบบเดียวที่ไม่ดี เพราะการแก้ไขจะไปถึงข้อมูลแค่บางส่วน

การ join สเปกทำให้แคตตาล็อก ทำให้ marker ของพื้นเหลือแค่ flag ชี้ทาง ไม่มีข้อมูลที่ join แล้ว

กลาง 2026-07-06

เวลาเปลี่ยนวิธีแทนความสัมพันธ์ระหว่างข้อมูล ต้องทำให้ครบทุกประเภท element ในรอบเดียวกัน หรือไม่ก็เขียนช้อยกเว้นไว้ใน spec ให้ชัด การเปลี่ยนแบบทำครึ่งๆ จะกลายเป็นเงื่อนไขแตกสาขาถาวรในทุกที่ที่ใช้ข้อมูลนั้น

เอกสาร schema รวมรุ่นที่ 4 ตกรุ่นภายใน 48 ชั่วโมงหลังเขียนเสร็จ

กลาง 2026-07-06

อย่าปล่อยให้ schema สองสายพัฒนาคู่ขนานกัน ให้ปรับให้ตรงกันอย่างต่อเนื่อง (รายวัน) หรือไม่ก็เลือกให้สายหนึ่งเป็นตัวหลักไปเลย การเอา schema ที่แตกกันมารวมทีเดียวก่อนใหญ่ จะได้เอกสารที่ล้าสมัยก่อนที่ใครจะได้เอาไปใช้

pattern type ชื่อ 'site_profile' จริงๆ หมายถึงรูปด้านของอาคาร ไม่ใช่ข้อมูลไซต์งาน

กลาง 2026-07-08

ทุกค่าใน enum ต้องมีคำนิยามเขียนไว้ตั้งแต่ตอนที่สร้างมันขึ้นมา ชนิดข้อมูลที่มีความหมายอยู่แค่ในชื่อ จะถูกใช้ผิดแน่นอน แล้วสุดท้ายก็ต้องมานั่ง migrate ที่หลัง

เพิ่ม pattern type ใหม่ 3 ตัวลง schema โดยยังไม่มีชุดฟิลด์ที่ตรวจสอบแล้ว

กลาง 2026-07-08

ถ้าจำเป็นต้องปล่อยชนิดข้อมูลใหม่ออกไปก่อนที่ฟิลด์จะนิ่ง ให้ทำเครื่องหมายทุกเรคอร์ดที่เขียนภายใต้ชนิดนั้น เพื่อให้การ migrate ที่หลังหำมันเจอ อย่าไปหวังพึ่งรายการค้างให้ช่วยจำ

Range grid_ref written horizontal-axis-first, contradicting the project's own existing convention

กลาง 2026-07-14

When inventing a new field format, derive it from the nearest existing convention and write it into the schema in the same change — an unwritten convention gets violated the first time a new shape appears.

Point-type grid_ref dashes stripped in data while the schema still documents the opposite

กลาง 2026-07-14

Never let the data lead the spec. If a convention change is worth applying to files, block on the spec decision first — a spec that contradicts the data is worse than no spec.

A footing plan carried both grid_source and a full inline grid copy, violating the spec's own no-duplication rule

กลาง 2026-07-14

Structural spec rules need a validator script, not just prose. Any rule phrased as 'never do X' should be a check that runs over every file before a house is declared done.

Identical beam specs repeated character-for-character on every occurrence of the same mark

กลาง 2026-07-14

Data that is per-mark belongs in a per-mark object, never repeated per instance. Duplicated identical values are a divergence bug waiting for its first edit.

Element-ordering rule was written in the wrong direction and had to be redone the next day

กลาง 2026-07-15

Confirm an ordering or sequencing rule against a concrete worked example before applying it to files — a directional rule applied backwards costs the full rewrite twice.

Series-wide promo pages were classified as title/unknown in every house until a 14th pattern was invented

กลาง 2026-07-15

A closed taxonomy needs an explicit escape hatch that is distinguishable from a default. Forcing a page into the nearest bucket hides the fact that the taxonomy is incomplete.

House 5's grid y-axis origin differed from every other house and had to be re-originated

กลาง 2026-07-19

Fix the grid-origin convention project-wide before extracting the first house. An inconsistent origin makes every derived position look wrong in a way that is easy to misdiagnose as a measurement error.

Stale '1'(2.25)' reference text left behind in files after the master value changed to 2.35

กลาง 2026-07-20

Cascading a value change must include free-text descriptions, notes and warnings — grep for the old numeric literal across the house, not just for the field name.

A beam sits at a different position on floor 1 and floor 2 beyond measurement tolerance, unresolved

กลาง 2026-07-20

When measurement finds a per-floor difference beyond tolerance, that is a data-model gap, not a measurement error — the schema needs per-floor grid values rather than a flag that defers the problem.

resolve_grid.py ของ King-try ขึ้น KeyError กับไฟล์ grid ปัจจุบันทุกไฟล์ หลังจาก schema ย้ายฟิลด์

กลาง 2026-08-20

เครื่องมือทุกตัวที่อ่าน schema ของ rawjson ต้องเอามารันกับไฟล์ปัจจุบันใหม่ทุกครั้ง ที่ schema ย้ายฟิลด์ และควร assert รูปร่างข้อมูลที่มีนาคาดหวังพร้อม error ที่บอกข้อขัดๆ แทนที่จะปล่อยให้ KeyError โผล่จากการไล่ key ซ้อนกัน

การแก้รูปซ้ำครั้งแรกทำได้แค่ลดโอกาสพัง และตัวกัน n-gram ก็ถูกเสี่ยงได้ง่ายๆ

กลาง 2026-08-21

การรับประกันเชิงโครงสร้าง (วงเล็บครอบคู่ รูปร่างตรง schema) ต้องใช้ข้อจำกัดที่ทำให้ผลลัพธ์แยกๆ เป็นไปไม่ได้ ไม่ใช่ค่ารับที่แค่ทำให้มันเกิดยากขึ้น และอย่าคิดว่าตัวกัน n-gram แบบตรงเป๊ะจะครอบคลุมลูปที่เปลี่ยนตัวกันไปเรื่อยๆ ได้

Candidate filenames read as "this is a footing" when they only meant "harvested from a footing page"

กลาง 2026-08-25

Never let a filename or field name imply a classification the code did not actually make. Name artifacts by their provenance when provenance is all you know.

Files printing both SD-40 and SD-50 on one page were left blank rather than resolved by guessing

กลาง 2026-08-25

When the source is genuinely ambiguous, leave the field empty and record why — never let a script pick for you. Repeated ambiguity of the same kind is a signal the schema shape is wrong, not the data.

ปล่อยช่องว่างของ schema ไว้: หน้าที่มีแผ่นแบบจริงสองแผ่นควรมี sheet_code แยกต่อมุมมอง

กลาง 2026-08-31

เมื่อข้อสันนิษฐานของ schema ถูกข้อมูลจริงพิสูจน์ว่าผิด ให้บันทึกช่องว่างนั้นไว้พร้อมผลกระทบที่จะตามมา เพื่อให้มีคนตัดสินใจ ไม่ใช่ลืมไปเฉยๆ

No decision yet on whether the model or the adapter should split combined marks like 'F4,C1'

กลาง 2026-08-31

Decide and document one canonical notation for element ids across the model, GT, and the ingesting adapter — and check the consuming code handles the model's actual output before scoring it wrong.

A block of SQL was living as a comment inside schema.js instead of in the database folder

ต่ำ 2026-07-05

Artifacts belong in the directory that owns their type, not next to the code that inspired them — a comment is not a storage location for another language's source.

The pattern taxonomy shipped with 9 types and no catch-all; `unknown` had to be bolted on during real extraction

ต่ำ 2026-07-08

Any closed classification applied to real-world documents needs an explicit 'does not fit' value from day one — otherwise the first unmatched item silently becomes a wrong classification.

House 2 pages 07/10 had grid row order written reversed in the original extraction

ต่ำ 2026-07-15

Ordering conventions must be in the schema before the first house is extracted; every convention settled late costs a retro-sweep across every house already done.

CV/pass3 sidecar files would have been fed into the raw-extraction adapter as unknown patterns

ต่ำ 2026-09-01

When a new stage writes files into a shared results directory, decide at the producer side whether each file is consumable output or a diagnostic sidecar — and give consumers an explicit allowlist rather than letting them warn about everything they don't recognize.

กับดักการตั้งค่า สภาพแวดล้อม และ ไลบรารี

ค่าตั้งต้น เวอร์ชัน หรือสภาพแวดล้อมที่ต่างกันนิดเดียว แล้วทำให้ผลลัพธ์เปลี่ยนไปเสียๆ

ทั้งหมด 87 เรื่อง · วิกฤต 18 · สูง 26 · กลาง 32 · ต่ำ 11

เรื่องที่ต้องอ่านให้ครบ (44)

วิกฤต 2026-07-21

รัน `pip install` ของ llama.cpp คู่ขนาน ทำให้ transformers กับ torch ถูกลดเวอร์ชันไต้หวัน training ที่กำลังรันอยู่

เกิดอะไรขึ้น: ขณะทำงาน training 3 epoch กำลังรันอยู่ มี tmux อีกหน้าต่างหนึ่งรัน `pip install -r requirements.txt` ของ llama.cpp ในสภาพแวดล้อม Python ตัวเดียวกัน มันลดเวอร์ชัน transformers จาก 5.5.0 เป็น 4.57.6 และ torch จาก 2.10.0+cu128 เป็น 2.11.0+cpu สคริปต์ training เลยพังตอนขั้นสุดท้ายที่สร้างตัวอย่างผลลัพธ์ ด้วย FileNotFoundError ที่ `modeling\_qwen3\_5\_moe.py`

ต้นเหตุจริง: ไม่มีการแยก virtual environment — งานสองงานใช้ interpreter เดียวกัน และปล่อยให้ requirements.txt ของโปรเจกต์อื่นไปหาเวอร์ชันแพ็คเกจที่ไม่ได้ pin ไว้ ซึ่งเป็นแพ็คเกจที่งานที่กำลังรันอยู่พึ่งพา

เสียหายอะไร: งานที่กำลังรันพังกลางคัน (โชคดีที่ adapter เพิ่งบันทึกเสร็จพอดี) torch เวอร์ชัน CUDA ถูกแทนที่ด้วยเวอร์ชัน CPU ล้วน ต้องซ่อมสามขั้นตอนนี้แล้วรัน verify_env.py ใหม่ กว่าจะวัดคุณภาพผลลัพธ์ได้

แก้ยังไง: รัน pip install transformers==5.5.0 (เวอร์ชันเป๊ะๆ จาก log ของรอบที่สำเร็จ) แล้ว pip install torch==2.10.0 --index-url .../cu128 เพื่อกู้เวอร์ชัน CUDA กลับมา จากนั้น verify_env.py ก็ผ่านอีกครั้ง

กฎที่ได้: ห้าม pip install อะไรบนเครื่องที่มีงานสำคัญรันอยู่เด็ดขาด — งานเสริมต้องใช้ venv หรือ container แยกเสมอ และให้จดเวอร์ชันแพ็คเกจที่ใช้ได้จริงจาก log ของรอบนั้นไว้ด้วย เวลาที่จะได้เป็นการ pin ไม่ใช่การเดา

วิกฤต 2026-07-21

CUDA OOM บนการ์ด 95GB เพราะตั้ง LORA_R=64 แล้วโดนคูณด้วย 256 experts ของ MoE

เกิดอะไรขึ้น: เทรนแล้วพังกลางทางด้วย CUDA out of memory — ใช้ VRAM ไป 94.87GB จาก 95GB เหลือว่างแค่ประมาณ 100MB ตอนนั้นตั้ง LORA_R ไว้ที่ 64

ต้นเหตุจริง: ประเมินหน่วยความจำที่ LoRA ใช้เหมือนกับว่าโมเดลเป็นแบบ dense ธรรมดา แต่ Qwen3.6-35B-A3B เป็น MoE ที่มี 256 experts ทำให้ LoRA adapter ถูกสร้างขึ้นมา 256 ชุด — ค่า rank ที่เลือกโดนคูณด้วยจำนวน experts ไม่ใช่แค่บวกเพิ่ม

เสียหายอะไร: การเทรนหยุดกลางคันบนเครื่องที่เสียเงินเช่า ต้องแก้ config แล้วเริ่มรันใหม่

แก้ยังไง: ลด LORA_R จาก 64 เหลือ 32 (งานวิจัยก็บอกด้วยว่า rank เล็กกว่าเหมาะกับชุดข้อมูล 226 ตัวอย่างมากกว่า)

กฎที่ได้: ถ้าเป็นโมเดล MoE ให้คิดขนาด LoRA rank เทียบกับจำนวน experts ไม่ใช่จำนวนพารามิเตอร์ — คำนวณ VRAM เป็น rank × experts ตั้งแต่ก่อนรันรอบแรก อย่ารอให้ OOM ก่อนแล้วค่อยคิด

pip install ในอีกหน้าต่างไปหับ transformers จนสคริปต์เทรนที่ยังรันอยู่พังคาที่

เกิดอะไรขึ้น: ระหว่างที่เทรน 3 epoch อยู่ ไปเปิด SSH อีกหน้าต่างแล้วรัน pip install เพื่อเตรียม llama.cpp ล่วงหน้า การติดตั้งครั้งนั้นไปอัปเดต/แทนที่ `transformers` ทำให้ไฟล์ที่ Unsloth ใส่มาให้รองรับ Qwen3.5-MoE หายไป สคริปต์เทรน — ซึ่งเซฟ LoRA adapter ไว้แล้วแต่ยังไม่จบ — เลยพังตอนขั้นสุดท้ายที่สร้างตัวอย่างผลลัพธ์ ด้วย error หาไฟล์ไม่เจอ

ต้นเหตุจริง: ทั้งสองหน้าต่างใช้ Python environment เดียวกันทั้งเครื่อง ไม่มีอะไรกันงานข้างเคียงออกจากงานหลักที่กำลังรันอยู่ และไม่มีใครคิดถึงความเสี่ยงเพราะรู้สึกว่ามันอยู่ "คนละหน้าต่าง"

เสียหายอะไร: ต้องซ่อม environment ก่อนถึงจะรัน eval ได้เลย ส่วน LoRA adapter รอดมาได้เพราะจังหวะพอดีล้วนๆ — ถ้าเกิดเร็วกว่านั้นแค่ไม่กี่นาที ต้องเทรนใหม่ทั้งรอบ (ทั้งเวลาและเงิน)

แก้ยังไง: ซ่อม environment ที่หลังแล้วรัน eval_fields.py ตั้งกฎใหม่ว่า ห้ามแตะ package ที่ใช้ร่วมกันขณะที่งานสำคัญกำลังรัน ถ้าจะทำงานข้างเคียงให้ใช้ virtual environment

กฎที่ได้: ให้ถือว่างานเทรนที่กำลังรันอยู่ล็อก Python environment ทั้งเครื่องไว้ งานติดตั้งอะไรก็ตามที่เตรียมล่วงหน้า ต้องแยกไปอยู่ใน venv ของตัวเอง หรือไม่ก็รอให้งานหลักจบสนิทก่อน

MAX_LENGTH=9216 ตัดคำตอบของตัวอย่างเทรน grid-master ทั้ง 5 ตัวทิ้งไปเสียบๆ

เกิดอะไรขึ้น: ค่า MAX_LENGTH ถูกตั้งมาจากตัวอย่างที่มีรูปเดียว แต่ในชุดข้อมูลมีตัวอย่างพิเศษแบบ "grid master" อยู่ 5 ตัวที่ใช้รูป 2-4 รูปพร้อมกัน ต้องการประมาณ 22,000 token — เกินขีดจำกัดไป 2.4 เท่า ส่วนที่เกินจะถูกตัดจากท้าย และท้ายนั้นก็คือคำตอบที่กำลังจะสอนโมเดล

ต้นเหตุจริง: คำนวณงบประมาณยาวลำดับจากตัวอย่างทั่วๆ ไป แทนที่จะดูจากเคสที่หนักที่สุด และการตัดข้อความก็เียบสนิทโดยการออกแบบ — ไม่มี error ไม่มีค่าเตือน บั๊กนี้มีมาตั้งแต่ก่อนเพิ่มบ้านหลัง 05 และเพิ่งเจอตอนอ่านสคริปต์เทียบกับจำนวน token จริงในชุดข้อมูล

เสียหายอะไร: ถ้าไม่เจอ โมเดลจะถูกเทรนด้วยคำตอบที่ถูกตัดขาด เฉพาะกับตัวอย่างชุดเดียวที่สอนเรื่องเส้นกริดทั้งอาคาร — ซึ่งเป็นหัวข้อที่สคริปต์เองบอกว่าผิบบอยที่สุด จับได้ก่อนรันรอบที่เสียเงิน

แก้ยังไง: เพิ่ม MAX_LENGTH จาก 9,216 เป็น 24,576 (มะขามเลือกเอง เพื่อรักษารายละเอียดของรูปไว้เต็มที่) ยอมรับความเสี่ยง OOM ที่สูงขึ้นในตัวอย่าง 5 ตัวนั้น

กฎที่ได้: ตั้งค่าความยาวลำดับจากตัวอย่างที่ยาวที่สุดจริงๆ ในชุดข้อมูล แล้วใส่ assert หรือ log ให้ร้องทุกครั้งที่มีตัวอย่างโดนตัด — การตัดแบบเสียบๆ ทำลาย label ทั้งโดยไม่แจ้งว่าพัง

วิกฤต 2026-07-24

MAX_LENGTH 9,216 มาจากตัวอย่างที่มีรูปเดียว เลย์ตัดคำตอบของตัวอย่าง **gridmaster** หลายรูปทั้ง 5 ตัวทิ้งเฉียบๆ

เกิดอะไรขึ้น: MAX_LENGTH ใน train_qwen36.py ถูกตั้งไว้ที่ 9,216 token ซึ่งเป็นตัวเลขที่คำนวณจากตัวอย่างที่มีรูปเดียว แต่ในชุดข้อมูลมีตัวอย่าง "grid master" อยู่ 5 ตัวที่ใช้รูป 2-4 รูปพร้อมกัน ต้องการถึงประมาณ 22,000 token — เกินไป 2.4 เท่าอะไรที่เกินจะถูกตัดจากท้าย และท้ายนั้นคือคำตอบ JSON ที่กำลังจะสอน

ต้นเหตุจริง: ตั้งค่าคงที่ตัวหนึ่งโดยอิงจากตัวอย่างทั่วไป แทนที่จะอิงจากค่าสูงสุดที่วัดได้จริงในชุดข้อมูล และตอนเทรนก็ไม่มีอะไรรายงานว่ามีการตัดข้อความเลย — มันตัดเฉียบๆ

เสียหายอะไร: ถ้าไม่แก้ ตัวอย่าง 5 ตัวเดียวที่สอนเรื่องเส้นกริดทั้งอาคาร (หัวข้อที่สคริปต์เองบอกว่าพลาตบอยที่สุด) จะถูกเทรนโดยคำตอบถูกตัดหายไป บั๊กนี้มีมาก่อนบ้านหลัง 05 และไม่เคยมีใครจับได้

แก้ยังไง: เพิ่ม MAX_LENGTH เป็น 24,576 (มะขามเลือกเพื่อรักษารายละเอียดรูปไว้เต็ม) ภายหลังพิสูจน์ด้วยการวัดจริง — แกวที่ยาวที่สุดคือ 22,515 token ไม่มีแกวไหนเกินเลย เหลือที่ว่าง 2,061 token

กฎที่ได้: ตั้งขีดจำกัดความยาว/ขนาดทุกตัวโดยอิงค่าสูงสุดที่วัดได้จริงในชุดข้อมูล ไม่ใช่จากตัวอย่างตัวไหน และทำให้การตัดข้อความโยน error ดังๆ ห้ามให้มันตัดเฉียบๆ

วิกฤต 2026-07-28

บั๊กที่ 1: ค่า **max_length 10240/13312** ของ **t02** จะตัดตัวอย่าง **gridmaster** เหลือแค่ **42%/39%** ตัดคำตอบ JSON ที่อยู่ท้ายทิ้ง

เกิดอะไรขึ้น: ค่า preset ของ t02 ยังใช้ max_length ที่ 10,240 กับ 13,312 อยู่ — ซึ่งเป็นค่าเก่าจากก่อนการตัดสินใจเมื่อ 2026-07-24 ตัวอย่าง gridmaster ต้องการประมาณ 22,500 token เลยจะถูกตัดเหลือ 42% กับ 39% และส่วนท้ายที่ถูกตัดก็คือคำตอบ JSON นั้นเอง

ต้นเหตุจริง: การตัดสินใจที่ทำกับสคริปต์หนึ่ง (train_qwen36.py เมื่อ 2026-07-24) ไม่เคยถูกส่งต่อไปยังสคริปต์พี่น้องของมัน สองไฟล์มีการตั้งค่าตัวเดียวกันโดยไม่มีกลไกอะไรคอยทำให้ตรงกัน

เสียหายอะไร: จะเป็นการทำบั๊กตัวเดียวกับที่เพิ่งแก้ไปสัปดาห์ก่อนซ้ำอีกครั้ง บนรอบที่แพงกว่าเดิม และเฉียบๆ ด้วย

แก้ยังไง: เปลี่ยนเป็น 24576 ในทุก preset พร้อมบันทึกชัดเจนว่าค่าเหล่านั้นเป็นค่าแปลกปลอม และถ้าเจอ OOM ให้ไล่แก้ตามลำดับ max_pixels → lora_r → ขยับไปการ์ดใหญ่ขึ้น ห้ามย้อนกลับมาลด max_length เด็ดขาด

กฎที่ได้: เวลาตัดสินใจเรื่องการตั้งค่าอะไรสักอย่าง ให้ grep หาไฟล์อื่นทุกไฟล์ที่มีการตั้งค่าตัวเดียวกันแล้วแก้ไขใน commit เดียวกันเลย การแก้ฝาแฝดแค่ตัวเดียว คือบั๊กที่รอวันที่ฝาแฝดอีกตัวจะออกโรง

วิกฤต 2026-07-28

เจอบั๊กอีกสามตัว จากการเอาค่า hyperparameter ของ t02 มาเทียบกับ t01 ทีละตัว — การตรวจที่ได้อารมณ์ว่าควรทำตั้งแต่แรก

เกิดอะไรขึ้น: พอเอาค่าตั้งของ t02 มาเทียบกับ t01 ทีละบรรทัดในช่วงท้ายๆ ก็เจอบั๊กเพิ่มอีกสามตัว: lora_dropout=0.05 (MoE ParamWrapper ไม่รองรับ dropout ที่ไม่ใช่ 0 — ยืนยันจาก error จริงของ t01), optim='adamw_8bit' (ค่าตรงเป๊ะที่ทำให้ t01 เจอ OOM ต้องใช้ paged_adamw_8bit) และไม่มี bf16=True เลย (โมเดลตระกูล MoE นี้ต้องตั้งค่าแบบ bf16 ตามที่ t01 พิสูจน์มา) ทั้งสามค่านี้เป็นค่าที่ t01 เคยพังมาแล้วและเลิกใช้ไปแล้ว

ต้นเหตุจริง: รอบใหม่เริ่มจากสคริปต์ต้นฉบับที่เก่าแล้ว แทนที่จะเริ่มจากรอบที่ผ่านการแก้บั๊กมาแล้ว และไม่มีการเอามา diff กับค่าที่รู้ว่าจะใช้ได้อย่างเป็นระบบ จนกระทั่งช่วงท้ายๆ ความรู้จากความล้มเหลวที่ได้มายากๆ อยู่แต่ในไฟล์ของ t01 เท่านั้น

เสียหายอะไร: ค่าที่รู้อยู่แล้วว่าพังสามตัวยังคงคาอยู่ตอนใกล้จะเข้าเครื่อง และทั้งสามตัวเคยทำให้เกิดความล้มเหลวจริงมาแล้วในรอบ t01

แก้ยังไง: แก้ทั้งสามตัว และทำตารางเทียบ 16 บรรทัดไว้ใน §0.4 ยืนยันว่าตัวแปรเดียวที่ยังต่างกันคือตระกูลของโมเดล

กฎที่ได้: เริ่มรอบใหม่ด้วยการเอา config ไป diff กับรอบล่าสุดที่รู้ว่าใช้ได้ ทีละ argument ก่อนทำอย่างอื่นทั้งหมด ค่าไหนที่รอบก่อนเลิกใช้ไปแล้ว คือค่าที่เคยทำให้พังมาแล้ว

วิกฤต 2026-07-28

transformers 5.5.0 ถอด ip.max_pixels/min_pixels ออกไปเลย ทำให้ Phase 4 พังบนเครื่องที่เสียเงินเช่า

เกิดอะไรขึ้น: การรันจริงครั้งแรกบนเครื่องเช่าพังทันทีด้วย AttributeError: property of 'Qwen2VLImageProcessor' object has no setter เพราะเวอร์ชันของ transformers ที่ pip ดึงมา (5.5.0) ถอด attribute max_pixels/min_pixels ออกไปแล้ว — ไม่ใช่ทำให้อ่านได้อย่างเดียว แต่ถอดออกเลย ทั้ง train_qwen3vl.py และ eval_fields.py ใช้เส้นทางที่ถูกถอดออกไปนั้น

ต้นเหตุจริง: เวอร์ชันไลบรารีเคลื่อนไปเรื่อยๆ ขณะที่โค้ดไม่เคยถูกรันเลยตั้งแต่วันที่เขียน ไม่ได้ pin เวอร์ชัน dependency ไว้ pip เลยไปเอาเวอร์ชันที่ใหม่กว่าทุกเวอร์ชันที่โค้ดเคยถูกทดสอบด้วย และ API ที่โค้ดพึ่งพาก็หายไปแล้ว

เสียหายอะไร: พังยับที่ Phase 4 บนเวลา GPU ที่เสียเงิน ทั้งที่การตรวจ Phase 0 ผ่านหมดทุกข้อ — พิสูจน์ว่า Phase 0 จับความล้มเหลวประเภทนี้ไม่ได้ เพราะมันรันบนเครื่องที่มีเวอร์ชันไลบรารีคนละชุด

แก้ยังไง: ไปอ่านซอร์สจริงของไลบรารีแทนที่จะเดา พบว่า ip.size['longest_edge'] / ['shortest_edge'] ยังใช้ได้อยู่ แล้วลบสองบรรทัดที่พึ่งออกจากทั้งสองไฟล์

กฎที่ได้: pin เวอร์ชันของ dependency ทุกครั้งที่จะรันงานที่เสียเงิน และเวลา API พัง ให้ไปอ่านซอร์สจริงของไลบรารี อย่าเดาว่าตัวแทนคืออะไร โค้ดที่นอนนิ่งไม่ได้รันมานาน เสี่ยงกว่าที่ตาเห็น — โค้ดไม่ขยับ แต่โลกข้างนอกขยับไปแล้ว

GGUF ของ t01 อาจไม่มีการ tune อยู่ในนั้นเลย — LoRA merge ล้มแบบเจียบๆ บน MoE ParamWrapper

เกิดอะไรขึ้น: t01 (tune ช่วง 21-24 ก.ค. แปลงเป็น GGUF แล้วใช้ในเครื่องมาตลอด) ให้ผลการสกัดข้อมูลที่ใช้ไม่ได้ พอไปดูพบว่า format/schema ผิดทั้งหมด (0 จาก 6 หน้า) ขณะที่การอ่านเนื้อหากลับดี — ซึ่งเป็นลายเซ็นของโมเดลฐานที่ไม่ได้ tune แล้วทำตาม prompt เท่านั้น ข้อสงสัย: merge_and_unload() ไม่เคย merge LoRA เข้าไปใน weight จริงก่อนแปลงเป็น GGUF

ต้นเหตุจริง: merge_and_unload() ของ peft/Unsloth merge LoRA แบบ ParamWrapper บนโมเดล MoE ผิด — พิสูจน์แล้วเมื่อวันที่ 28 ด้วยการวัด tensor ขนาด 201 ล้านค่าแล้วพบว่ามีเพียง 0.67% ที่เปลี่ยน โครงสร้างผ่านหมดและไม่มี error ขึ้นมาเลย ความล้มเหลวจึงเจียบสนิท และ t01 ก็ใช้ฟังก์ชันตัวเดียวกันบนโมเดลตระกูลเดียวกัน

เสียหายอะไร: ทุกข้อสรุปที่ได้จาก GGUF ของ t01 ตั้งแต่ 24 ก.ค. อาจเป็นเรื่องของโมเดลฐาน ไม่ใช่โมเดลที่ tune แล้ว ผลการสกัดข้อมูลบ้าน 01 จำนวน 6 หน้าปนเปื้อน งาน inference ในเครื่องทั้งหมดตั้งแต่วันที่ 24 มีค่าที่ไม่รู้ และเรื่องนี้บล็อกรอบ t01 และ t02

แก้ยังง: ยังไม่แก้ — ออกแบบการทดสอบไว้แล้วแต่ยังไม่ได้รัน: เอาหน้า 03 ไปรันกับ GGUF ของ Qwen3.6 ตัวฐานที่ไม่ได้ tune ด้วย prompt เดียวกัน แล้วเทียบกับ output ของ t01 (ราว 20 นาที ไม่มีค่าใช้จ่าย)

กฎที่ได้: ขั้นตอน merge/convert ที่ไม่ขึ้น error ยังไม่ได้พิสูจน์ว่ามันทำงานสำเร็จ หลังทุกการ merge weight ให้เอา tensor ที่ merge แล้วมาเทียบเชิงตัวเลขกับตัวฐาน (ดูว่าเปอร์เซ็นต์ของค่าที่เปลี่ยนไป) ก่อนจะแปลงหรือส่งมอบ — โครงสร้างผ่านไม่ได้แปลว่าพฤติกรรมถูก

t02 ยังไม่มี GGUF เลยสักตัว — บั๊ก MoE merge ตอนนับบล็อกการ tune ทั้งสองรอบ

เกิดอะไรขึ้น: การแปลง GGUF ของ t02 ถูกจับได้ว่าล้มเมื่อวันที่ 28 ด้วยตัวกันที่เข้มกว่า และยังไม่แปลงไม่ได้จนถึงตอนนี้ พอถึงวันที่ 29 ก็เข้าใจแล้วว่าบั๊ก MoE merge ของ peft/Unsloth ตัวเดียวกันนับบล็อก t01 ด้วย เท่ากับไม่มีรอบไหนเลยที่ได้ของที่ deploy ได้อย่างน่าเชื่อถือ

ต้นเหตุจริง: ข้อบกพร่องต้นทางเพียงจุดเดียวที่ยังแก้ไม่ได้ใน merge_and_unload() สำหรับ LoRA แบบ ParamWrapper บนโมเดล MoE และยังหาทางเลี่ยงไม่ได้

เสียหายอะไร: การ tune เติมรูปแบบสองรอบ ไม่ได้ของที่ส่งมอบให้รันในเครื่องได้เลย ที่น่าสังเกตคือ t02 เทรนที่ 3,796 token ต่อภาพ ซึ่งตรงกับที่ llama.cpp ป้อนให้พอดี มันจึงอาจเป็นโมเดลที่ดีกว่าสำหรับรันในเครื่อง และบั๊ก merge นี้แหละที่ทำให้ไม่มีทางรู้ได้

แก้ยังง: ยังไม่แก้ — มีทางเลือกที่ระบุไว้แล้ว: หาเส้นทาง merge อื่น หรือทิ้ง GGUF ไปเลยแล้วเสิร์ฟ bf16+adapter บนเครื่องเช่าแทน (เส้นทางนั้นรู้อยู่แล้วว่าได้ 90%)

กฎที่ได้: เมื่อข้อบกพร่องต้นทางเพียงจุดเดียวบล็อกลำดับ deploy ของทุกสิ่งที่เราผลิต ให้หยุดผลิตของแล้วไปแก้เส้นทาง deploy ก่อน อีกข้อ: ต้องมีโหมดเสิร์ฟสำรองที่รู้ว่าใช้ได้แน่ (bf16 + adapter ตอนรัน) เผื่อไว้ เพื่อไม่ให้เกิดการแปลงที่พึ่งบล็อกการส่งมอบทั้งหมด

ภาพเทรน 973 จาก 973 ภาพโดนยอลงหมด — คือ 100% ของ dataset และไม่มีใครรู้มาหลายรอบ

เกิดอะไรขึ้น: พอดีด้วยเครื่องมือที่ทำขึ้นมาเฉพาะ (measure_capacity.py) พบว่าทุกภาพในชุดเทรนเกินเพดาน MAX_PIXELS ที่ตั้งไว้ 5,120 visual token ขนาดที่พบบ่อยที่สุด (3309×2339 จำนวน 615 ภาพ) ต้องการ 7,558 token คือหายไป 32% ส่วนภาพที่ใหญ่ที่สุด (4631×3473) หายไป 67% แม้แต่ภาพที่เล็กที่สุดในชุดก็ยังเกินเพดาน

ต้นเหตุจริง: ตั้ง MAX_PIXELS ไว้ที่ 5,120 โดยไม่เคยคำนวณเลยว่าภาพจริงใน dataset ต้องการเท่าไร ต้นทุน token ของภาพหนึ่งภาพไม่เคยถูกคำนวณจากขนาดไฟล์จริงเลย จนกระทั่งวันนี้

เสียหายอะไร: การเทรนทุกรอบที่ผ่านมามีภาพที่ถูกทำให้เสียคุณภาพ เส้นคานและสัญลักษณ์เล็กๆ ถูกทำลายไปก่อนที่จะโมเดลจะได้เห็นด้วยซ้ำ ซึ่งเป็นเหตุที่เป็นไปได้โดยตรงของ plan_beam 0% เท่ากับหลายรอบที่จ่ายเงินไป รันอยู่บน input ที่เสียหาย

แก้ยังไง: เขียน measure_capacity.py ขึ้นมาคำนวณ token จากขนาดภาพจริง (patch 16 × merge 2 → 1 token = 32×32 px) ขยับ MAX_PIXELS เป็น 7,680 และ MAX_LENGTH เป็น 47,104 ทำให้สัดส่วนภาพที่ถูกยอลดจาก 100% เหลือ 30% โดยไม่ต้องเพิ่มฮาร์ดแวร์เลย (VRAM สูงสุด 87.5 → 90.9 GB บนการ์ด 96GB ตัวเดิม)

กฎที่ได้: คำนวณต้นทุนทรัพยากรจริงของ dataset จริงที่เราใช้ ก่อนจะตั้งเพดานอะไรก็ตาม เพดานที่ตั้งโดยไม่วัดข้อมูลก่อน คือค่าตั้งค่าที่ทำลายข้อมูลแบบเงียบๆ

Missing `crop\_to\_patches: true` collapsed every training image to one 256-token tile, so the 78B model never learned to read drawings (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: After 4 folds of InternVL3-78B QLoRA training finished, eval showed plan_beam 6/6 = 0% and plan_footing 1/1 = 0%. Opening the raw outputs (not just the recall number) showed the model answered with a completely different house and different sheet — fed a footing plan, it returned another house's roof plan. Investigation with three independent evidences: (1) counting image tokens in the training log dump = 768 tokens / 3 images = exactly 256/image, (2) reading `llamafactory/data/mm\_plugin.py` source showed tiling requires `crop\_to\_patches: true` which the yaml never set, (3) downscaling a real beam plan to 448×448 (what the model actually saw) made every beam mark, dimension and rebar note unreadable by eye.

ต้นเหตุจริง: The LLaMA-Factory training config was written by copying an official example without tracing the framework source for how many tokens per image it actually produces. InternVL3's dynamic tiling is opt-in via one flag; without it every image, no matter its original megapixels, is squashed into a single 256-token tile. Nobody probed the real token-per-image count before spending money on a full run.

เสียหายอะไร: ~\$63 and roughly 15-16 hours × 4 GPUs of training burned producing adapters that only learned the JSON schema (96-100% valid) and memorised answers for houses seen in training — 0% element recall on real reading. The whole 4-fold k-fold round, the new test house, and the planned soup were made worthless. Third occurrence of the same bug class (t01 Unsloth 512px fallback → t02 data_collator resize not compared → t04 crop_to_patches unknown).

แก้ยังไง: Root cause documented as rule 15 in `rule\_of\_tune.md`: before any real training on a new framework/model pair, trace the source for actual tokens-per-image and measure it with a CPU-only probe (`probe\_img\_tokens.py`) before paying for a full run. The fix itself (adding `crop\_to\_patches: true` and retraining) was not applied — direction deferred to Makham.

กฎที่ได้: Before paying for a full fine-tune on a new framework+model pair, run a CPU-only probe that prints the real tokens-per-image and the real pixel size the model sees, and eyeball a downscaled sample image yourself. If you cannot read it, the model cannot either.

t04 เทรนด้วยรูปแค่ 1 tile เพราะ LLaMA-Factory กับ transformers ตั้งค่า crop_to_patches สวนทางกัน

เกิดอะไรขึ้น: การเรน t04 (InternVL3-78B) ทั้งรอบให้ JSON ที่ถูกต้องแต่ recall 0% วันนี้ไล่เจอตันเหตุแล้ว: preprocessor_config ของโมเดลตั้ง crop_to_patches:false มาให้ ซึ่ง LLaMA-Factory ทำตามตอนเรน แต่ `InternVLProcessorKwargs\_defaults` ของ transformers ฮาร์ดโค้ดเป็น True ตอน inference ผลคือรูปที่ใช้เรนทุกใบยุบเหลือ tile เดียว 256 token ไม่ว่าต้นฉบับจะมีกี่ล้านพิกเซลก็ตาม

ต้นเหตุจริง: โลบารีสองตัวใน stack เดียวกันตั้งค่าเริ่มต้นของฟิลด์เดียวกันสวนทางกัน และไม่มีใครตรวจว่าค่าการแบ่ง tile ที่มีผลจริงเป็นเท่าไรบน batch ที่ใช้เรนจริง — config เขียนตามเอกสาร ไม่ได้เขียนจากจำนวน token ที่วัดได้จริง

เสียหายอะไร: เสียการเรนทั้งรอบที่ใช้ 4 GPU ไปเปล่าๆ (ทั้งเงินและเวลาเต็มวัน) โมเดลที่ได้ต้องทิ้ง และอาการที่เห็นก็ดูเหมือนปัญหาข้อมูล/pipeline อยู่หลายวัน

แก้ยังง: ตั้ง crop_to_patches: true ใน yaml ของ LF (LF เวอร์ชัน $\geq 0.9.3$, PR #7817) แล้วคุมจำนวน tile ทางอ้อมผ่าน image_max_pixels เพราะ max_patches=12 ถูกฮาร์ดโค้ดไว้ใน plugin ภายหลังตรวจสอบบนเครื่องเรนที่ mm_plugin.py:843 ยืนยันว่าค่านี้ทำให้แบ่งเป็น 13 tile / 3,482 token จริง

กฎที่ได้: อย่าไว้ใจค่าเริ่มต้นของขั้นตอนเตรียมข้อมูลที่โลบารีสองตัวอาจตั้งไม่ตรงกัน ก่อนจะจ่ายเงินเรนทุกครั้ง ให้พิมพ์จำนวน tile และจำนวน visual token ต่อบatch จริงออกมาดู แล้วเขียน assert ว่าตรงกับตัวเลขที่ตั้งใจไว้

การทำ quantization 4-bit ของ LLaMA-Factory ไปโดนส่วน vision ด้วย ทำให้ InternViT-6B ตาบอด

เกิดอะไรขึ้น: เจอปัญหาตัวที่สองซ่อนอยู่ใต้บั๊กเรื่อง tile: เอกสารของ InternVL เองเตือนไว้ว่า BNB 4-bit บน InternViT-6B จะให้ผลลัพธ์เพี้ยนและอ่านรูปไม่ออก ส่วน `quantization\_bit:4` ของ LF ก็ quantize ทั้งโมเดลโดยไม่มีการยกเว้น ViT ผ่าน modules_to_not_convert เลย

ต้นเหตุจริง: ตั้งค่า quantization ครอบทั้งโมเดลโดยไม่ได้ตรวจว่าตัว vision encoder หน 4-bit ได้ไหม ทั้งที่ผู้พัฒนาโมเดลเตือนไว้เองในเอกสาร แต่ไม่มีใครอ่านก่อนลงมือทำ arm นี้

เสียหายอะไร: arm ของ Voldemort (InternVL3-78B) ทั้งเส้นใช้ไม่ได้เลย ต้องตัดทิ้งหลังพิสูจน์ว่าโมเดลตาบอดสนิท ทั้งที่แก้ crop_to_patches ถูกต้องแล้ว

แก้ยังง: ไล่ดู source ของ quantization.py ยืนยันว่าไม่มีการยกเว้น ViT จริง แล้วตัด arm นี้ทิ้งหลังผ่านด่านตัดสินใจราคาประมาณ \$2 แล้วย้าย GPU 2 ตัวไปทำ k-fold ของ Courser แทน ไม่ต้องเผาทเรนทั้งรอบ

กฎที่ได้: ก่อนจะ quantize โมเดล vision-language ให้ไปอ่านข้อจำกัดเรื่อง quantization ของฝั่ง vision tower ที่ผู้พัฒนาระบุไว้ก่อน และต้องพิสูจน์ว่าโมเดลตั้งต้นยังอ่านหน้าจริงออกที่ความละเอียด bit นั้น ก่อนจะจ่ายเงินเรน

การถอดรหัสแบบ greedy ทำให้ recall เป็น 0% และพ่นข้อความหลอนหลายภาษายาว 16,363 ตัวอักษร

เกิดอะไรขึ้น: การวัด recall ครั้งแรกของ destrier ได้ 0/34 หน้า 26 ตอบ elements: [] พร้อม confidence 1.0 และติดธงว่าไม่มีฐานรากในแบบ ทั้งที่มีอยู่ชัดๆ 6 ตัว ส่วนหน้า 27 พ่นออกมา 16,363 ตัวอักษรใน 18 นาที: มีความคิดภายในหลุดออกมาทั้งที่ปิด thinking ไว้แล้ว ตามด้วยคำพ้องภาษาอังกฤษยาวเป็นพืด แล้วสลับไปภาษาจีนตรงวลี 'losing getting lost' → 迷路 ต่อด้วยสำนวนจีนเป็นชุด บทความให้กำลังใจ คำปฏิเสธความรับผิดชอบที่เขียนเอง แล้วพยายามกลับมาเขียน JSON ต่อก่อนจะ token หมด

ต้นเหตุจริง: ใช้ do_sample=False มาตลอด ทั้งที่ Qwen เขียนไว้ชัดๆ ว่า 'ห้ามใช้ greedy decoding' เพราะจะทำให้พ่นข้อความซ้ำไม่จบ แล้วยังไปซ้ำเติมด้วย repetition_penalty=1.15 กับ no_repeat_ngram_size=8 ซึ่งไปลงโทษเนื้อหาที่ซ้ำกัน โดยธรรมชาติของแบบก่อสร้าง โมเดลจึงถูกบีบให้หนีไปหาคำศัพท์ที่ไม่เกี่ยวข้องเลย

เสียหายอะไร: การวัดผลทั้งรอบได้ 0% และทำให้ดูเหมือนโมเดลอ่านอะไรไม่ออกเลย เสียเวลา GPU ที่จ่ายเงินไป 18 นาทีต่อหน้า

แก้ยังไง: แก้บรรทัดเดียวใน serve_purson.py: do_sample=True, temperature=0.7, top_p=0.8, top_k=20 แล้วเอา repetition_penalty กับ no_repeat_ngram_size ออก recall ชยับจาก 0% เป็น 11.8% แบบเข้มงวด / 43.5% แบบวัดตำแหน่ง+mark_base และข้อความภาษาจีนหายไปหมด

กฎที่ได้: อ่านและใช้คำการ generate ที่ผู้พัฒนาโมเดลประกาศไว้ก่อนจะรันวัดผลใดๆ และห้ามใส่ค่าลงโทษการซ้ำกับเนื้อหาเฉพาะทางที่ซ้ำกันเป็นเรื่องปกติอยู่แล้ว

t04 (Purson) เทรนด้วยรูปที่ถูกย่อเหลือ 448×448 แบบเสียบๆ เพราะไม่เคยตั้ง crop_to_patches

เกิดอะไรขึ้น: การ fine-tune t04 เสร็จแล้วได้ความแม่นยำในการอ่าน 0% ทั้ง plan_beam และ plan_footing ไฟล์ config ขาดบรรทัด 'crop_to_patches: true' ตัวโหลดข้อมูลจึงไม่ได้ซอยแบบเป็น patch แต่ย่อกระดาษ A1 ทั้งแผ่นเหลือรูปเดียวขนาด 448×448 วัดจาก log การเรนได้ 768 token สำหรับรูป 3 ใบ = 256 token ต่อรูปพอดี ทั้งที่แบบก่อสร้างจริงต้องใช้เป็นพันๆ token

ต้นเหตุจริง: การซอยรูปเป็น tile ต้องเปิดเองผ่าน flag ใน config ที่ทีมไม่รู้ว่ามีอยู่ และไลบรารีก็ไม่ส่งค่าเตือนหรือ error ใดๆ ตอนที่มันปิดอยู่ แล้วยังมีอะไรใน pipeline ที่ assert ว่า token ต่อรูปต้องไม่ต่ำกว่าเท่าไร ค่าเริ่มต้นที่มองไม่เห็นจึงทำลายตัวอย่างเรนทุกใบแบบเสียบๆ

เสียหายอะไร: เสียการเรนไปทั้งรอบ คิดเป็นค่า GPU หลายพันบาท บวกรอบวัดผลที่ตามมาอีก plan_beam กับ plan_footing recall เป็น 0% ทั้งคู่ โมเดลตายตั้งแต่ออกจากโรงงาน และเพิ่งมารู้ตอนจ่ายเงินไปแล้ว

แก้ยังไง: ไล่ต้นเหตุด้วยสามวิธีที่แยกจากกัน (นับ token จาก log, อ่าน source ของ mm_plugin.py, และลองย่อ beam plan จริงเป็น 448×448 แล้วยืนยันว่าอ่านไม่ออกจริง) แล้วตั้งกฎการใหม่เป็น rule_of_tune.md ข้อ 15: ก่อนเรนจริงทุกครั้ง ต้องวัด token ต่อรูปก่อน ซึ่งรันบน CPU ได้ ไม่เสียเงิน ไม่ต้องใช้ GPU

กฎที่ได้: ก่อนจ่ายเงิน fine-tune ทุกครั้ง ให้วัดและ assert จำนวน token ต่อรูปที่ตัวโหลดข้อมูลผลิตออกมาจริงบนตัวอย่างจริง อย่าไว้ใจการจัดการรูปแบบค่าเริ่มต้นของ pipeline ฝั่ง vision เพราะรูปที่ถูกย่อแบบเสียบๆ ไม่มี error ไม่มีเบาะแสอะไรเลย มีแต่ผลลัพธ์ขยะหลังจากที่จ่ายเงินไปแล้ว

วิกฤต 2026-09-01

t44 (Voldemort) ทำ quantization 4-bit โดนส่วน vision encoder ไปด้วย ทั้งที่ผู้พัฒนาเตือนไว้ชัดๆ

เกิดอะไรขึ้น: เพื่ออัด InternVL3-78B ให้ลง GPU ได้ เลยทำ quantization 4-bit ทั้งโมเดล รวมถึง vision tower ที่เป็น InternViT-6B ด้วย ทั้งที่เอกสารทางการของ InternVL เขียนไว้ตรงๆ ว่าการ quantize vision encoder จะ 'ให้ผลลัพธ์ที่ไม่มีคุณภาพและอ่านรูปไม่ออก' ผลคือโมเดลบรรยายแบบฐานรากภาษาไทยว่าเป็นเอกสารกลมๆ พื้นหลังสีชมพู มีโลโก้ Facebook/Apple/Barbie เขียนว่า 'Happy Birthday' และบรรยายแปลนคนว่าเป็นโลโก้ BB-8 จาก Star Wars

ต้นเหตุจริง: ทำ quantization ทั้งโมเดลในฐานวิธีอัดให้พอดีหน่วยความจำ โดยไม่อ่านเอกสารของโมเดลเองว่าส่วนไหนต้องคงความละเอียดเต็ม มอง vision tower เป็นแค่ก้อนน้ำหนักรอกก้อน ทั้งที่มันคือส่วนที่งานทั้งหมดพึ่งพาความคมชัดของมัน

เสียหายอะไร: โมเดลตาบอดสนิท จับได้ทีี่ด้านราคา \$2 ก่อนจะเทรนเต็ม ความเสียหายตรงๆ จึงน้อย แต่ก็เสียแรงไปทั้งสาขา (ทั้งการเลือกโมเดล ติดตั้ง และเช่าเครื่อง)

แก้ยังง: หยุดทันทีหลังด้านตรวจได้ 0/3 แล้วเลิกสาขาโมเดลนี้ไป

กฎที่ได้: อ่านคำแนะนำเรื่อง quantization ของผู้พัฒนาโมเดลเองก่อนจะ quantize อะไรก็ตาม และห้าม quantize vision encoder ในงานที่ต้องอ่านรูปเด็ดขาด เวลายัดให้พอดีหน่วยความจำ ให้เลือก quantize เป็นส่วนๆ และจดไว้ว่าส่วนไหนคงความละเอียดเต็มไว้

วิกฤต 2026-09-02

Server had its own 25-minute generation timeout that was never updated when the client's was raised, truncating JSON mid-answer (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Job f63bc71e finished 9/9 but 3 pages produced broken JSON (page_03_notes, page_12_plan_footing, page_13_notes). The real cause was that `serve\_purson.py` holds its own `PAGE\_TIMEOUT\_S = 25\*60`, entirely separate from `worker.py`'s client timeout which had been raised to 2700s the previous day. The server-side one is enforced as a `StoppingCriteria` that cuts generation mid-stream with no regard for whether the JSON is complete.

ต้นเหตุจริง: Two independent timeout definitions for the same operation, in two files, with only one of them updated. Worse, the server's mechanism truncates output silently rather than raising an error — so a cut answer looks like a model formatting failure rather than a timeout.

เสียหายอะไร: 3 pages of a paid GPU run destroyed; then a full re-run of the whole house (job 43323920). Wasted a complete GPU cycle plus debugging time chasing a suspected model problem.

แก้ยังง: Raised the server timeout to `40\*60` (2400s, deliberately below the client's 2700s so the client is never the one to give up first), restarted the server, resubmitted the house reusing the stored image paths rather than re-uploading.

กฎที่ได้: A timeout must be defined once. If client and server both need one, derive them from a single source and assert server < client. And any timeout that truncates output rather than erroring must stamp a 'was truncated' marker so the failure is not misattributed to the model.

สูง 2026-07-02

GPU rental template documented an instance, a model and an approval process that none matched what actually happened (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: No_touch_box/vast-template/ (README.md + onstart.sh) specified an RTX 3090 24GB instance defaulting to Qwen3-VL-8B-Instruct, deferring the real model choice to 'King's sign-off'. The actual fine-tuning (t01/t02) was run by Makham and Claude directly on a rented RTX PRO 6000 96GB, landing on Qwen3.6-35B-A3B.

ต้นเหตุจริง: Infrastructure and process were written up in advance of doing the work, then the work was done differently and the template was never updated or retired. It described an intended future rather than an observed reality, and nothing tied it to an actual run.

เสียหายอะไร: A runnable-looking template pointing at the wrong GPU class and the wrong model sat in the repo for a month. Anyone provisioning from it would have rented a 24GB card for a model that needed far more, burning rental money on an instance that cannot run the job. The 'King's sign-off' step also described an approval chain that was not being followed.

แก้ยังง: Archived on 2026-08-04 to wait_for_thing/No_touch_box/vast-template/ and flagged in ARCHIVE.md as safe to delete for real, since nothing in the repo still follows that process.

กฎที่ได้: Never leave a provisioning template in the repo that has not been used for a real run. Update it from the run that actually happened, or delete it — a stale infra template is an invitation to rent the wrong hardware and pay for it.

สูง 2026-07-05

Schema definitions scattered across 5 places, with dead factories and 3 mismatched pricing enum vocabularies (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The Data Schema / Data Flow review found: schema definitions spread across 5 different places, v2 factories in schema.js that are dead code, site/foundation/BBS-manual declaring their own object shapes outside schema.js, and 3 mismatched pricing enum sets.

ต้นเหตุจริง: schema.js was nominally the single source of truth, but nothing enforced it — feature modules could construct entity objects ad hoc, and there was no rule that a shape must come from a create* factory. Over time each module grew its own definition of the same concept.

เสียหายอะไร: Three different vocabularies for the same price-source concept means any consumer comparing them gets wrong answers; dead v2 factories mislead anyone reading schema.js about which shape is current; and shapes declared outside schema.js can never be validated centrally.

แก้ยังง: Documented in docs/DATA_SCHEMA_REVIEW.md with a 4-phase plan. Only Phase A was completed on this day; the factories and the canonical enum were left to Phases B-D.

กฎที่ได้: A 'single source of truth' that is only a convention is not one. Enforce it: never construct an entity object ad hoc, always through a create* factory, and add a test that fails when a shape is declared outside the schema module.

สูง 2026-07-05

Site, Foundation and Planner communicated by reading each other's localStorage keys directly, outside any contract (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The schema review found an informal Site → Foundation → Planner data flow in which modules read each other's localStorage keys directly, bypassing schema.js and the PIPELINE_EVENT mechanism that the rest of the app uses.

ต้นเหตุจริง: No explicit cross-module contract existed for this chain, so the fastest path to the data — reading the neighbour's storage key — became the actual integration mechanism. The formal path (factories plus events) only covered the pipeline chain.

เสียหายอะไร: Any rename of a storage key or change of shape in Site silently breaks Foundation and Planner with no compile-time or test-time signal, and no event fires so consumers cannot know when the data changed.

แก้ยังไง: Documented; the explicit Site → Foundation → Planner contract (createSiteInvestigation() / createFoundationDesign() factories plus getters and events) was scheduled as the first item of Phase B, not done on this day.

กฎที่ได้: Cross-module data must move through named getters/setters and an event, never by one module reading another's storage key. If a shortcut integration exists, write the contract for it before it acquires a third consumer.

สูง 2026-07-06

The whole existing Label Studio review flow was hard-wired to the Gen 1 schema and could not read Gen 3 data at all (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Both existing Label Studio variants (the whole-JSON-per-house 'Constistant01' and King's per-page Repeater 'Structural Review'/'BOQ Review') were tied to the Gen 1 schema — flat plan[]/section[]/schedule[] or categories[].items[] — and read from qwen-output/. Neither supported Gen 3's nested main_bar{}stirrup{}, the new pattern types, or files already split one-per-pattern/view.

ต้นเหตุจริง: The review tooling was built against a specific schema shape with the shape assumptions embedded in both the task generator and the XML configs, and it was not revisited across three schema generations. The schema evolved on one track and the tooling on none.

เสียหายอะไร: A complete rewrite of the review tooling was required in the middle of an extraction push: a new task generator plus three new XML configs, then five more rounds of fixes the same day. All Gen 3 data was unreviewable until this was done.

แก้ยังไง: Wrote label-studio-tasks-makham.js plus label-studio-makham-elements.xml, -material_list.xml and -single.xml, grouping files by their actual structure rather than by pattern name.

กฎที่ได้: Review/QA tooling is part of the schema, not downstream of it. When a schema generation is designed, budget the tooling update in the same plan — otherwise data accumulates faster than it can be verified and the review step becomes the bottleneck at the worst moment.

สูง 2026-07-21

เลือก LORA_R=64 โดยไม่คิดเรื่อง MoE expert 256 ตัว (และไม่คิดเรื่องชุดข้อมูลแค่ 226 ตัวอย่าง)

เกิดอะไรขึ้น: ตั้ง LoRA rank 64 / alpha 128 ไว้ตั้งแต่ต้น พอเป็นโมเดล MoE ตัว buffer ของ LoRA แต่ละ expert คุณ 256 เข้าไป VRAM 95GB เลยไม่พอ ลดเหลือ 32/64 แล้ว OOM หาย — และงานวิจัยที่เคยหาไว้ก่อนหน้านี้ก็บอกอยู่แล้วว่า rank ต่ำเหมาะกับชุดข้อมูล 226 ตัวอย่างมากกว่า

ต้นเหตุจริง: เลือก rank จากค่า default ทั่วไปโดยไม่คูณต้นทุนหน่วยความจำด้วยจำนวน expert ของโมเดล และไม่ได้เอาคำแนะนำเรื่องขนาดชุดข้อมูลที่ทีมจดไว้เองเมื่อ 2026-07-20 มาใช้

เสียหายอะไร: ช้อม TEST_STEPS ไป 4 รอบและ OOM พังหลายรอบบนเครื่อง \$1.056/ชม. กว่าจะเจอวิธีแก้จริง และค่าเดิมที่ตั้งไว้ยังน่าจะแย่กว่าในแง่คุณภาพด้วย (เสี่ยง overfitting และลืมความรู้เดิม)

แก้ยังไง: ลด LORA_R 64→32, LORA_ALPHA 128→64 (คงอัตราส่วนเดิม) มะขามเป็นคนตัดสินใจหลังจากได้ดูตัวเลือก 3 แบบ ตามกฎข้อ 2 ของ rule_of_tune.md

กฎที่ได้: สำหรับโมเดล MoE ให้คำนวณขนาด LoRA rank โดยคิดหน่วยความจำแบบ จำนวน expert × rank และตรวจ rank เทียบกับขนาดชุดข้อมูลก่อนรันจริงครั้งแรกที่เสียเงิน — งานวิจัยที่ตัวเองจดไว้แล้วนับเป็นข้อจำกัด ไม่ใช่แค่เชิงอรรถ

สูง 2026-07-24

Vast.ai's default 150GB disk would have reproduced the previous round's disk-full failures if left alone (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A screenshot of the Vast.ai rental page revealed the Container Size / Disk slider defaults to 150GB. The known requirement, learned from the 2026-07-21 round's repeated disk-full failures, is at least 300GB. Clicking Rent with the default would have hit the identical failure immediately.

ต้นเหตุจริง: A vendor default silently disagreed with a hard requirement the team had already learned the hard way, and that requirement lived only in a prior diary entry — not in any pre-rental check or the rental procedure itself.

เสียหายอะไร: Caught before renting, so zero cost this round — but the same default had already caused multiple disk-full failures during the 2026-07-21 round.

แก้ยังไง: Added to Phase 0.2 as an explicit item, and later hardened in verify_env.py as a check that fails when the whole volume is under 280GB ('forgot to drag the slider').

กฎที่ได้: Every vendor default that has ever bitten you must become an automated assertion, not a note to remember. Write the check into the environment verifier, not into a document.

สูง 2026-07-28

Training crashed immediately on library drift: `ip.max_pixels` is no longer settable in transformers

5.5.0 (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On the first real t02 run, the script died at once with `AttributeError: property of 'Qwen2VLLanguageProcessor' object has no setter`. The code set image resolution via `ip.max_pixels`, which the library's newer version made read-only.

ต้นเหตุจริง: `train_qwen3vl.py` had been written months earlier and never executed. pip installed a newer transformers on the fresh rented box; the library's API moved while the code sat idle. No version pin existed.

เสียหายอะไร: Failure occurred after the meter had started, on the very first paid step of the run.

แก้ยังไง: Read the library's actual source (not guessed) and switched to `ip.size["longest_edge"]`, deleting the old path entirely.

กฎที่ได้: Code that has never been executed is untested code, regardless of age. Either pin library versions in the environment spec, or smoke-run every script end-to-end locally before renting hardware.

สูง 2026-07-28

onstart.sh pip-installs every library unpinned — the actual root cause of the version drift was never fixed (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The Vast.ai on-start script installs triton, unsloth, trl, peft, accelerate, bitsandbytes and pillow via plain pip with no version pins. That is precisely the mechanism that installed transformers 5.5.0 on 2026-07-28 and crashed training at startup. The fix applied was to the calling code (use `ip.size["longest_edge"]`), not to the install step.

ต้นเหตุจริง: Every rental provisions a fresh, unpinned environment, so each new instance can silently differ from the one the scripts were last tested against; nobody added a lockfile or version pins after the drift bug.

เสียหายอะไร: Guarantees the same class of failure will recur on the next rental, and it recurs at the worst moment — right after the money starts being spent. It also means no two rentals are strictly comparable, which is the same doubt that forced the unplanned Phase 7.5 re-measurement.

แก้ยังไง: Not fixed — `onstart.sh` is still documented in Appendix A as installing unpinned packages.

กฎที่ได้: Pin every library version (or ship a lockfile) in the provisioning script; an unpinned environment on a rented machine turns a free, local, reproducible setup into a paid lottery.

สูง 2026-08-24

Two training variables silently drifted out of control between t01/t02 and t03 (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A parity table run before renting (rule_of_tune #12) found

`PYTORCH\_ALLOC\_CONF=expandable\_segments` had disappeared from t03 (t01 had it, added after an OOM that was 100MB short) and `FINETUNE\_VISION` had been hardcoded in t03 where t01/t02 had it as an env var.

ต้นเหตุจริง: train_t03.py was written fresh rather than copied, so hard-won environment fixes from prior rounds were not carried forward. There is no machine-checked contract between rounds — only the manual parity table.

เสียหายอะไร: Would have reproduced a known OOM (the exact one expandable_segments was added to fix) and made an A/B variable un-toggable at runtime. Caught before rental, so cost was zero.

แก้ยังไง: Restored both, with defaults byte-identical to the previous rounds.

กฎที่ได้: The parity table before renting is not optional. Every value in a fresh training script must be diffed call-site-by-call-site against the last working round, especially environment variables added to fix a past failure.

สูง 2026-08-25

Historical: t01 set MAX_PIXELS to 512px, blurring every image and invalidating the whole round's results (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: In training round t01, MAX_PIXELS was misconfigured so images were shrunk to 512px — extremely blurry. The entire round's evaluation numbers were therefore measured on garbage input.

ต้นเหตุจริง: A resolution parameter was set without validating what the model actually received; no visual sanity check of a rendered training sample was performed before the run.

เสียหายอะไร: A full training round's results were invalid and had to be discarded; recorded as lesson #12 in rule_of_tune.md — and the same class of error recurred on 2026-08-25.

แก้ยังไง: Documented as rule_of_tune.md lesson 12. The 2026-08-25 measurement pass is the systematic fix that should prevent a third occurrence.

กฎที่ได้: Before any training run, dump one preprocessed image exactly as the model will receive it and look at it with your eyes. A blurry sample invalidates every number the run produces.

สูง 2026-08-25

t03 OOM'd twice from raising numbers without sizing first, forcing a LoRA rank cut (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: In the previous round, MAX_LENGTH was raised to 32,768 without a VRAM sizing pass. It OOM'd, forcing LoRA rank down 32→16, and even then the eval step OOM'd again from fp32 logit upcasting at that same length. Last round OOM'd with only 470 MiB free.

ต้นเหตุจริง: Configuration values were raised on intuition instead of being decomposed against measured VRAM. Also, the eval path's fp32 logit upcast was not accounted for in the sizing at all — only the training step was considered.

เสียหายอะไร: Two failed GPU runs (rental time burned), a forced reduction in LoRA capacity (r 32→16) that degraded the round's ceiling, and a lost eval.

แก้ยังไง: This session decomposed peak VRAM from the one real measurement available (87.5 GB @ seq 32,768, r=16): weights bf16 74.0 + LoRA/grad/optim 5.7 + activation 7.8, with only the activation term scaling with sequence length. Raised MAX_PIXELS 5120→7680 / MAX_LENGTH 32768→47104 against that model instead of blind.

กฎที่ได้: Never raise a memory-bound constant without a decomposition of where the memory actually goes and which terms scale. Size on paper first, then confirm with a short TEST_STEPS run before committing to a full rental.

สูง 2026-08-25

Training MAX_PIXELS raised to 7680 while the inference script still sits at 5120 — a train/eval mismatch waiting to happen (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: train_t03.py was raised to MAX_PIXELS=7680, but `infer\_house\_t03.py` was deliberately left at 5120 to match the existing t03 adapter. It must be moved to 7680 before evaluating t03b, or the next round will be evaluated at a different resolution than it was trained at.

ต้นเหตุจริง: A resolution constant is duplicated across the training script and the inference script with no shared source, so raising one silently desynchronizes them.

เสียหายอะไร: If missed, t03b would be evaluated at the wrong input resolution, producing numbers that measure the mismatch rather than the model — and the mismatch would look like a model regression.

แก้ยังไง: Not fixed — a loud in-file comment was left on the infer script's MAX_PIXELS, plus an entry in the end-of-day mandatory-gate list for the next round.

กฎที่ได้: Constants that must match between training and inference belong in one shared config, not duplicated in two scripts. A comment is a reminder, not a guarantee — the durable fix is a single source both scripts import.

สูง 2026-08-25

Gridline training labels were being silently truncated mid-JSON at MAX_LENGTH=24,576 through the t01/t02 rounds (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: MAX_LENGTH was 24,576 in t01/t02. Gridline examples use 4 images and measure ~26,791 tokens, so their JSON answers were cut off mid-object during training. It was only found and raised to 32,768 in the t03 round; the earlier rounds trained on truncated labels. The same failure class is named again later in the day as the reason you must never lower MAX_LENGTH alone without lowering MAX_PIXELS ("t01 \$0.4 bug class").

ต้นเหตุจริง: Sequence length was set as a round number rather than measured against the longest real example in the dataset; nothing asserted that a label fit inside the budget, so truncation produced a plausible-looking run instead of an error.

เสียหายอะไร: Two whole tuning rounds (t01, t02) trained the gridline subtask on labels whose JSON was cut off — unknown but non-zero degradation of gridline accuracy, and any gridline metric from those rounds is untrustworthy.

แก้ยังไง: MAX_LENGTH raised 24,576 → 32,768 in t03, then → 47,104 later this day; the fallback order written into train_t03.py explicitly forbids lowering MAX_LENGTH alone.

กฎที่ได้: Measure the longest real example's token count before setting a sequence-length cap, and assert on it — a truncated label never raises an error, it just teaches the model garbage.

สูง 2026-08-29

MAX_PIXELS differed between training (7680) and inference (5120) (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Inference was running at MAX_PIXELS 5120 while training used 7680 — the two were fixed to match only on 2026-08-29, and then the model change made the fix moot for t04.

ต้นเหตุจริง: Train and infer scripts each carry their own copy of image-preprocessing constants with no shared source, so they drift whenever one is tuned.

เสียหายอะไร: Inference was feeding the model images at a different effective resolution than it was trained on — a silent quality degradation invisible in any metric that doesn't isolate it.

แก้ยังไง: Set infer MAX_PIXELS to 7680 to match train, alongside OUT_DIR/out-root renames outputs_t03→t04.

กฎที่ได้: Image preprocessing constants must live in one shared module imported by both train and infer. Any value that must match across scripts should not exist in two places.

สูง 2026-08-30

Decoder settings inherited from t03/Qwen forbade the repeated answers the ground truth actually requires (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The eval decoder carried ``no_repeat_ngram_size=8`` and ``repetition_penalty=1.15`` inherited from the t03/Qwen era. Real GT for construction drawings legitimately repeats marks many times (B2×10, RB1×24), so these settings actively suppressed correct output. Proven in isolation with a fixtest: removing them raised extracted elements from 0 to 10 (though recall stayed 0% because of the separate `crop_to_patches` root cause).

ต้นเหตุจริง: Generation hyperparameters were copied forward between model generations without re-asking whether the output distribution had changed. Anti-repetition penalties are a text-generation heuristic that is simply wrong for structured extraction where duplicates are semantically required.

เสียหายอะไร: Compounded the recall failure and would have masked or distorted any future measurement; on its own it zeroed out element extraction.

แก้ยังไง: Carried directly into t05: removed ``no_repeat_ngram_size=8`` and lowered ``repetition_penalty`` from 1.15 to 1.0.

กฎที่ได้: Never inherit decoder/generation hyperparameters across a model or task change without justifying each one against the target output format — anti-repetition penalties must be off for structured extraction whose ground truth repeats.

สูง 2026-08-30

AutoProcessor gave the model 13 image tiles at inference vs 1 tile at training — a 13× preprocessing mismatch (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: During the recall investigation it emerged that at inference time AutoProcessor tiled each image into 13 tiles, while training (missing ``crop_to_patches``) produced exactly 1 tile per image.

ต้นเหตุจริง: Training used LLaMA-Factory's `mm_plugin` path with tiling disabled, while inference used the plain HF AutoProcessor default with tiling enabled — two different preprocessing stacks were never compared against each other.

เสียหายอะไร: Even after any other fix, the model was being shown inputs in a form it had never been trained on, guaranteeing degraded output.

แก้ยังไง: Documented as a compounding factor in the root-cause report; the structural fix (make preprocessing identical on both sides) rolls into the t05 probe rule.

กฎที่ได้: Assert that the exact tensor shape / token count the model receives at inference equals what it received at training — print and compare both, don't assume two libraries agree.

สูง 2026-08-31

Three hyperparameters silently differed between arms because the yaml omitted them and took library defaults (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A parity check found weight_decay (0.01 vs HF default 0.0), optim (adamw_8bit vs default adamw_torch) and lora_dropout differed between the two arms because the yaml simply did not write them and inherited silent defaults. The same optim gap existed in t04 and nobody had noticed.

ต้นเหตุจริง: Parity was assumed from the values that were written, without enumerating the values that were omitted and resolving each library's default.

เสียหายอะไร: The two arms were not actually comparable; the same gap had already corrupted t04's comparability unnoticed.

แก้ยังไง: Wrote all three explicitly into both fold yamls and confirmed they parse.

กฎที่ได้: When comparing two training stacks, enumerate every hyperparameter each framework can default silently and write it explicitly on both sides — omission is a difference, not a match.

สูง 2026-08-31

Dry-run round 1: installed transformers did not know the qwen3_5_moe architecture (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Model loading failed because transformers 4.57.6 does not recognise qwen3_5_moe.

ต้นเหตุจริง: No pinned version window for the framework against the specific model architecture.

เสียหายอะไร: Would have failed at the first second of a paid full run had the dry-run not been done.

แก้ยังไง: Pinned transformers==5.5.0 (valid window [5.2.0, 5.5.0], upper bound from unsloth's own <=5.5.0 declaration).

กฎที่ได้: Pin the framework version against the exact model architecture and prove it loads in a dry-run before the full run.

สูง 2026-08-31

unsloth pulled a cu130 torch build onto a CUDA 12.8 image, breaking JIT kernel compilation mid-training (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Training crashed during JIT kernel compilation because nvrtc could not find libnvrtc-builtins.so.13.0 — the file existed but was not on the library path, since unsloth had installed a torch cu130 wheel on a CUDA 12.8 image.

ต้นเหตุจริง: Package manager resolved a CUDA build mismatched to the base image, and the failure only surfaces during in-training JIT compilation, not at import.

เสียหายอะไร: A crash that only appears once training is under way, i.e. after money has already been spent starting the run.

แก้ยังไง: Pointed LD_LIBRARY_PATH at nvidia/cu13/lib instead of downgrading torch.

กฎที่ได้: After installing an ML stack on a rented box, assert the torch CUDA build matches the image's CUDA version, and force at least one JIT-compiling step in the dry-run so this class of failure surfaces before the real run.

สูง 2026-08-31

Two instances rented two hours apart installed different peft versions, factorizing MoE layers transposed (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: During the merge, fold0's MoE-layer LoRA factors were oriented the opposite way from fold2/3 because gpu1 had peft 0.18.1 while the later machines got 0.20.0 — the rentals were two hours apart and unpinned.

ต้นเหตุจริง: No version pin on a library that decides an on-disk weight format; unpinned installs drift between machines rented minutes apart.

เสียหายอะไร: Adapters were silently incompatible for averaging; required writing a permutation converter and proving $\max|\Delta W_{\text{old}} - \Delta W_{\text{new}}^T| = 0.00e+00$.

แก้ยังไง: Wrote the permute converter, merged successfully, and recorded rule_of_tune Lesson 18: pin every library that writes a format.

กฎที่ได้: Pin the exact version of any library that determines an artifact's on-disk layout, and pin it identically across every machine in a parallel run.

สูง 2026-08-31

Loading the merged adapter with the older peft would produce silent garbage (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The merged destrier adapter follows peft 0.20 conventions; the machine's existing peft 0.18.1 would load it into silent garbage rather than erroring.

ต้นเหตุจริง: Format convention differences between peft versions fail silently rather than raising.

เสียหายอะไร: Any evaluation run on the wrong peft version would produce meaningless numbers that look valid.

แก้ยังไง: Created a dedicated venv with peft 0.20 for the smoke test.

กฎที่ได้: Pin and verify the exact library version an artifact was produced with before evaluating it — a version mismatch that loads silently is worse than one that crashes.

สูง 2026-08-31

GPU-A produced zero output for ~\$2.4 because its worker used a vanilla PeftModel with no generation guards (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: GPU-A's worker_page_raw_pratyad.py used vanilla AutoModelForImageTextToText + PeftModel — very slow on a 256-expert MoE LoRA — and set no repetition_penalty or no_repeat_ngram_size, unlike serve_purson.py. Pages 09 and 10 timed out at 25 minutes on both the normal and xgrammar attempts, producing no output at all.

ต้นเหตุจริง: Two divergent inference code paths in the same project, one of which never received the accumulated configuration knowledge from the other.

เสียหายอะไร: ~\$2.4 and ~2 hours of a rented GPU for zero results.

แก้ยังไง: Instead of renting more, wrote pure_power_endpoint.py to fire raw-power prompts at GPU-B's already-running endpoint, getting comparable results in 133-364s per page instead of 25+ minutes.

กฎที่ได้: Keep exactly one inference code path; if a second exists, diff its generation config against the canonical one before spending GPU time on it.

สูง 2026-08-31

The launcher menu started the wrong model because it inherited another file's default adapter (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Pressing menu option 1 started the server with --adapter Sicilian44/t03, an old model, because go.py called `up` without arguments and picked up presentation.py's default. Makham caught it.

ต้นเหตุจริง: Relying on another module's default value instead of passing the parameter explicitly.

เสียหายอะไร: A rented card was launched with the wrong model (instance 49410094), caught and swapped in time.

แก้ยังไง: Renamed the preset to destrier after the real adapter, made the menu always ask which model, always pass --model explicitly rather than relying on any other file's default, and display the currently loaded model at all times.

กฎที่ได้: Never let a launcher inherit a default from another file for a value that determines what is being run — pass it explicitly and display it in the UI.

สูง 2026-08-31

repetition_penalty and no_repeat_ngram_size — added as a safeguard — were themselves wrong for construction drawings (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: serve_purson.py ran with repetition_penalty=1.15 and no_repeat_ngram_size=8. Construction drawings are naturally repetitive (F7,C1 / F7,C1, repeated grid refs), so the model was penalised for producing the correct repeated content and was pushed to invent new words instead — feeding the runaway output.

ต้นเหตุจริง: Generic anti-repetition settings were copied in without asking whether the domain's correct output is repetitive; a known Qwen3-VL issue (#1611) documents exactly this.

เสียหายอะไร: Combined with greedy decoding, produced 0% recall and the 16,363-character multilingual runaway; both were removed to reach 11.8% strict / 43.5% position recall.

แก้ยังไง: removed both, switched to Qwen's recommended sampling (temperature 0.7 / top_p 0.8 / top_k 20)

กฎที่ได้: An anti-degeneracy knob is domain-specific: if the correct answer repeats by nature, penalising repetition is a bug, not a safeguard.

สูง 2026-09-01

pass3 anchored on `cv\_mark`, a field the model answers only 9% of the time (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: pass3 required elements carrying `cv\_mark` to form anchors. Destrier only saw `cv\_mark` in 92 of 1034 training examples (9%), so in practice the model almost never emits it — producing 0 anchors on every page and no measurement at all.

ต้นเหตุจริง: The design picked an anchor key without checking its frequency in the actual training data. A dependency on a model output was chosen from the schema, not from measured model behaviour.

เสียหายอะไร: pass3 would produce zero measurements on real runs despite the fit logic being correct.

แก้ยังไง: Added a fallback `anchors\_by\_shape()`: match 'points the model gave a grid_ref' to 'points CV saw' by spatial distribution shape (normalise both sets into [0,1], mutual nearest neighbour). Safe because a wrong pairing spikes the residual and `build\_transform` rejects it itself.

กฎที่ได้: Before depending on any model-emitted field, measure how often it actually appears in the training data. Model-optional fields need a fallback path, and the fallback should be self-checking (a bad match must fail loudly, not produce a plausible wrong number).

BOQ export used document.createElement for HTML escaping — unrunnable outside a browser and missing quote escaping (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `boq-export-variants.js` implemented `escapeHtml` via `document.createElement`, so none of the 4 export variants could run outside a browser (no tests possible, no server-side document generation). It also never escaped `"` or `\'`, because `textContent` doesn't — an injection hole if the output is placed in an HTML attribute.

ต้นเหตุจริง: Borrowing the DOM as an escaping utility couples pure string logic to a browser runtime, and inherits `textContent`'s escaping semantics (which cover text context only, not attribute context). The same attribute-escaping class of bug had already been fixed elsewhere in the codebase (`escAttr` in planner) but was not applied here.

เสียหายอะไร: Blocked all automated testing of exports and any server-side generation; latent XSS/attribute-breakout vector in exported documents.

แก้ยังไง: Replaced with pure string-replacement escaping and added `"` and `\'` to the escaped set.

กฎที่ได้: Escaping must be a pure string function, never DOM-based — and it must cover attribute context (`"` and `\'`), not just text context. When one instance of a bug class is fixed, grep for every other implementation of the same helper.

เรื่องที่เหลือในหมวดนี้ (43)

เอกสารของ pipeline รุ่นที่ตายไปแล้ว ยังค้างอยู่ในรีโปกเป็นเดือนหลังถูกแทนที่

กลาง 2026-07-01

เวลาเปลี่ยน pipeline ใหม่ ให้เก็บกวาดเอกสารของรุ่นเก่าไปพร้อมกันในการแก้ครั้งเดียวกัน โดยเฉพาะ quickstart เพราะคนใหม่เข้ามาจะอ่านอันนั้นเป็นอันแรกเลย เอกสารกู้คืนจาก git history ได้อยู่แล้ว ลบหรือย้ายไปเก็บได้เลยไม่ต้องสั่งลบ ดีกว่าปล่อยคู่มือที่พาคนหลงทางทิ้งไว้

URL รูปใน task ของ Label Studio ที่ generate ออกมาชี้ไปที่ raw.githubusercontent รูปเลยโหลดไม่ได้จนกว่าจะ push รีโปกขึ้นไปก่อน

กลาง 2026-07-06

เวลาตัว generator ปล่อย URL ภายนอกออกมา ให้มันลองยิงเช็คสักอันว่าเข้าถึงได้จริงก่อนจะบอกว่าสำเร็จ การไปแอบฟังเงื่อนไข 'ต้องมีคน push รีโปก่อน' คนที่เจอปัญหาจะเป็นคนตรวจ ไม่ใช่คนเขียน

ตัวไดอารี่งานเองมี path เก่าค้างอยู่ 4 จุด และมีงานที่ทำเสร็จแล้วยังค้างอยู่ในรายการรอทำ

กลาง 2026-07-06

จัดโครงสร้างไดเรกทอรีใหม่เมื่อไหร่ ให้ grep หา path เก่าใน log และเอกสารทันทีทุกครั้ง และเคลียร์รายการค้างให้จบทุกท้ายเซสชัน รายการค้างที่ไม่ตรงความจริงจะพาคนเข้าใจผิด แยกว่าไม่มีรายการเลย

สร้าง Label Studio ไปได้แค่ครึ่งวงจร ไม่มีอะไรแปลงผลตรวจที่ export ออกมากลับเข้าเป็น annotated/*.json

กลาง 2026-07-06

สร้างเส้นทางบางที่สุดที่วิ่งครบวงจรก่อน (เข้าหนึ่ง task ออกหนึ่งการแก้ไข) แล้วค่อยไปขัดผังการแสดงผล เครื่องมือตรวจงานที่ผลลัพธ์กลับเข้าระบบไม่ได้ ให้ข้อมูลที่แก้แล้วเป็นศูนย์ ต่อให้หน้าตาดีแค่ไหนก็ตาม

ไฟล์ label-studio-tasks.js ผัง Constant เป็นเวอร์ชันที่รู้อยู่แล้วว่าพัง มันอ่าน path qwen-output ที่แชร์กันอันเดียว แทนที่จะอ่านแยกตามบ้าน

กลาง 2026-07-07

ก่อนลบบะไรที่คิดว่าเป็น 'ไฟล์ซ้ำ' ให้ diff ทุกไฟล์ก่อน ไฟล์ที่เนื้อหาต่างกันไม่ใช่ไฟล์ซ้ำ มันคือการแก้ที่หายไป หรือบั๊กที่ฟื้นคืนชีพ และต้องรู้ให้ได้ก่อนว่าเป็นอันไหนก่อนจะกดลบ

config ของ Label Studio ทั้ง 5 ชุดปส่อยออกไปโดยฟิลต์ส่วนหัวเป็นข้อความอ่านอย่างเดียว คนตรวจเลยแก้ house/page/sheet/pattern ไม่ได้

กลาง 2026-07-07

ฟิลต์ไหนที่คนตรวจอาจต้องแก้ ต้องแก้ไขได้ ไม่ใช่แค่แสดงผล และเวลาทำให้ฟิลต์แก้ไขได้ ให้เอา label ไว้นอกช่องกรอก เพราะ label ที่ฝังอยู่ในกล่องจะกลายเป็นส่วนหนึ่งของข้อมูลที่ถูกลบทิ้ง

เครื่องที่ใช้ทำงานไม่มี Python เลย ทั้งที่สคริปต์หลักของ pipeline เป็น Python

กลาง 2026-07-10

ถ้าสคริปต์หลักของรีโปเขียนด้วยภาษาที่เครื่องทำงานรันไม่ได้ ก็ให้ติดตั้งภาษานั้นซะ หรือไม่ก็เลิกบอกว่าสคริปต์พวกนั้น 'ใช้งานอยู่จริง' การแก้โค้ดที่รันไม่ได้ คือการแก้ที่ไม่มีใครตรวจสอบได้

`lora\_dropout=0.05` incompatible with Unsloth's MoE LoRA wrapper

กลาง 2026-07-21

When fine-tuning an MoE model, validate every LoRA hyperparameter against the library's MoE code path — dense-model defaults do not transfer.

MAX_LENGTH set to 10,240 by estimate when stats.json said the real maximum was 8,890

กลาง 2026-07-21

Any capacity constant (sequence length, batch size, buffer size) must come from the measured dataset statistics you already produced — never from a mental estimate plus 'safety margin'.

Raising MAX_LENGTH to 24576 knowingly reintroduced OOM risk on a card that had only ~100MB headroom

กลาง 2026-07-24

When a fix increases resource pressure, decide and write down the fallback ladder before the paid run, and explicitly forbid the fallback that would undo the fix.

The MAX_LENGTH fix knowingly pushes the run toward OOM, with only ~100MB of 95GB spare in the previous round

กลาง 2026-07-24

When accepting a known risk, write down the ordered remediation ladder in advance and name the one action that is forbidden, so the wrong fix isn't reached for under pressure.

llama-server hung with 'non-consecutive token position' on the first inference run and had to be killed

กลาง 2026-07-28

Record the exact required launch flags for every serving binary alongside the model, and treat a hang with no output as a configuration failure to investigate immediately, not as slow progress to wait out.

--ctx-size 8192 cut inference output off mid-sentence because prompt plus image plus answer needs 8,960 tokens

กลาง 2026-07-28

Compute the context budget as prompt + image tokens + expected answer length before setting it, and treat mid-sentence truncation as a context-size symptom first, never as a model quality problem.

Qwen3-VL defaults to 16,384 visual tokens per image — 3.2x over budget — and this was only found by opening the real config on the rented machine

กลาง 2026-07-28

For every new model family, dump the real preprocessor/vision config on the actual machine and compare its defaults against your budget — vendor defaults are variables you did not set.

llama.cpp printed the exact remedy flag (--image-min-tokens 1024) and it was still not applied by end of session

กลาง 2026-07-29

Apply a zero-cost, vendor-named remedy the moment you read it, especially before running a comparison test — otherwise every subsequent measurement still carries the known confound and has to be redone.

CLAUDE.md เขียนว่า Structure Planner เป็นเท็บตายที่ไม่มีใครเรียก ทั้งที่ต่อสายไปเรียบร้อยแล้ว

กลาง 2026-08-18

ห้ามพูดว่าโค้ดตรงไหนถูกเรียกหรือไม่ถูกเรียกโดยอ่านจากเอกสารอย่างเดียว ต้อง grep หาจุดเข้าจริงก่อนเสมอ และโน้ตแนวว่า 'X ตายแล้ว/ไม่มีใครเรียก' ที่เก๋เกินไม่ก็สัปดาหฺ์ ให้ถือว่ายังไม่ได้พิสูจน์จนกว่าจะเช็คใหม่

หัวข้อ op4 ใน README กับ op04 SKILL.md เป็นคู่ที่ต้อง sync กันด้วยมือ ซึ่งยุ่งยากจะเพี้ยน

กลาง 2026-08-21

เนื้อหาที่รู้อยู่แล้วว่าซ้ำกันสองที่ ต้องมีเหตุผลจับความเพี้ยน ไม่ใช่โน้ตขอให้คนช่วยจำ ให้เพิ่มตัวตรวจว่าหัวข้อที่แชร์กันของสองไฟล์เหมือนกันปะ แบบเดียวกับที่โปรเจกต์นี้ป้องกันกฎอื่นๆ อยู่แล้ว

หน้า 22 ค้างเกิน 45 นาทีบน GPU เซา เพราะใช้ greedy decoding ล้วนๆ

กลาง 2026-08-24

เอาวิธีแก้ decoding ที่พิสูจน์แล้วติดไปกับ runner script ตัวใหม่ทุกตัวตั้งแต่ตอนสร้าง runner ตัวใหม่ห้ามส่งออกไปโดยไม่มี repetition_penalty และตัวกันค้างเด็ดขาด

คำแนะนำการจูนของทีปรึกษาชี้ไปที่ max_frames=768 ซึ่งเป็นค่าสำหรับวิดีโอที่โปรเจกต์นี้ไม่เคยใช้เลย

กลาง 2026-08-25

ก่อนแก้ค่า config ตัวไหน ให้ grep ในโค้ดเพื่อพิสูจน์ว่าเส้นทางโค้ดที่อ่านค่านั้นทำงานจริงในงานของเรา ปุ่มที่ไม่มีใครอ่านค่า ก็ไม่ใช่ปุ่ม

A second resolution-bound magic number (100px minimum beam run) is hardcoded with no CLI override, unlike the 21px gap beside it

กลาง 2026-08-25

When one magic constant is found to be scan-dependent, grep the same file for its siblings in the same commit. Constants of the same class fail together.

Template-comparison figure had F2 cropped off because the canvas height was hardcoded

กลาง 2026-08-25

Derive layout dimensions from the content, never hardcode them from one sample. And always open the produced artifact and look at it — rendering bugs are invisible to code-level checks.

BEAM_GAP=21px is calibrated to one scan batch's DPI and drawing scale

กลาง 2026-08-25

Any pixel-space constant must record the DPI and scale it was measured at, and be exposed as a knob. Silent zero-detection is the worst failure mode — add a sanity check that warns when a detector finds nothing on a page that should have something.

The FINETUNE_VISION A/B was never run because of a GGUF-export requirement that no longer applies

กลาง 2026-08-25

When a constraint is dropped, immediately re-list every decision that was made because of it — a lapsed blocker keeps blocking silently.

Adding the Thai glossary to every subtask pushed sequence length past MAX_LENGTH and would have forced a VRAM increase

กลาง 2026-08-26

Measure the sequence-length distribution before and after any prompt change, and exclude the subtasks a change cannot help. Apply prompt additions selectively, not uniformly.

prompt เรื่อง gridline โตขึ้นสะสม ~1.5k ตัวอักษรในวันเดียว เข้าใกล้เพดาน MAX_LENGTH

กลาง 2026-08-29

ตั้งงบตัวอักษร/token ต่อ prompt ของแต่ละ subtask ไว้ตายตัว แล้วเช็คทุกครั้งที่แก้ การปรับปรุงมันสะสมขึ้นเรื่อยๆ แต่เพดานไม่ได้ขยับตาม

จำนวน epoch เปลี่ยนสามรอบ (3→5→4→3) ด้วยเหตุผลที่ไม่ใช่เชิงเทคนิค ทำให้ต้องเริ่มเทรนใหม่สองครั้งและลบผลลัพธ์ทั้งสองรอบ

กลาง 2026-08-30

ลืตกค่า hyperparameter ที่กระທค่าใช้จ่ายให้หนึ่งก่อนกดรันงานที่ต้องจ่ายเงิน และตั้งค่าจากสัญญาณที่วัดได้ (loss curve / dry-run) ถ้าจำเป็นต้องเปลี่ยนหลังกดรัน ให้เปลี่ยนก่อนเผา GPU ไปมาก และบันทึกไว้ด้วยว่าไม่ใช่การตัดสินใจเชิงเทคนิค

cutoff_len / image_max_pixels / lora_rank ถูกปล่อยเป็นค่า default ที่ยกมาจากโมเดลคนละตระกูล โดยไม่ตรวจสอบ

กลาง 2026-08-30

ถ้ามีรายการที่ติดธงว่า 'ต้องตรวจสอบก่อนรันจริง' ให้ระบุให้ชัดว่าต้องวัดอะไรถึงจะถือว่าเคลียร์ — dry-run ที่เช็คแค่ VRAM กับ loss ไม่ได้ปลดคำเตือนเรื่องพารามิเตอร์ความละเอียดภาพ

ความยาว sequence เกิน model_max_length ของ tokenizer (17773 > 8192) — ไม่รู้ว่า label ถูกตัดหึ่งตอนเทรนหรือเปล่า

กลาง 2026-08-30

วัดการกระจายความยาว token ของ dataset เทียบกับ model_max_length ของ tokenizer และค่า cutoff_len ที่ตั้งไว้ ก่อนเทรน แล้วใส่ assert ว่าไม่มีตัวอย่างไหนล้นแบบเสียบๆ

onstart.sh ของ Courser สัม TORCH_CUDA_ARCH_LIST ที่ของ t02 มีอยู่แล้ว

กลาง 2026-08-31

สคริปต์ bootstrap ตัวใหม่ ให้ diff กับเวอร์ชันล่าสุดที่รู้ว่าใช้ได้ อย่าเขียนใหม่จากความจำ

dry-run รอบ 2: peft เรียก symbol ของ torchao ที่ torchao ถอดออกไปแล้ว

กลาง 2026-08-31

เมื่อ dependency สองตัวตีสัน ให้เช็ก่อนว่าตัวที่มีปัญหาถูกใช้งานจริงในคอนฟิกร์ใหม่ ก่อนจะไปใส่ pin เวอร์ชัน — บ่อยครั้งการถอดหึ่งเป็นวิธีที่เล็กลง

Only the new PYTORCH_ALLOC_CONF name was set; torch's own OOM message still names the old variable

กลาง 2026-08-31

When a library renames an environment variable, set both the old and new names until the old one disappears from the library's own messages.

Client sends temperature 0.7 instead of 0 — harmless today only because the server overrides it, catastrophic if the server ever changes

กลาง 2026-09-01

Never leave two components disagreeing about a critical parameter on the assumption that one currently overrides the other. Make both sides agree, so swapping either is safe.

Label Studio XML paste failed because the copy started at line 2, dropping the opening <View> root tag

ต้า 2026-07-06

For any hand-pasted config, publish the expected line count alongside it and check it after pasting. An XML 'extra content' error almost always means a missing or duplicated root tag, not a problem at the line the parser names.

Spec document header still claimed 2026-07-05 while it had been continuously edited through 2026-07-06

ต้า 2026-07-06

Put a 'last edited' line in every living spec and update it in the same edit that changes content. Never encode a date in a filename for a document that will keep evolving — the filename becomes a lie that cannot safely be corrected once it is referenced.

material_list.xml still carried a pre-Makham-era layout, out of step with the other configs

ต้า 2026-07-09

When N config files must behave alike, change them as a set and diff them against each other afterwards — hand-maintained siblings drift, and the drift is only visible side by side.

Header fields were laid out inside the pinned image column, eating the space reserved for the drawing

ต้า 2026-07-09

In a two-pane labeling UI, the pinned pane holds exactly one thing — the artifact being read. Anything else goes on the scrolling side.

The new production README shipped carrying a warning that the same session had already made false

ต้า 2026-07-10

When a session's code change invalidates a doc written earlier in that same session, fix the doc in the same commit. Do not rely on a later unrelated pass to catch it.

README commands used `python`, but the rented Docker image only has `python3`

ต้า 2026-07-21

On first login to any new rented image, verify the basics (`python`/`python3`, pip, CUDA, disk) and update the runbook before running anything billable.

eval_fields.py's --adapter default pointed at outputs/lora, which does not exist

ต้า 2026-07-28

Fix broken defaults, but never touch measurement code that both arms of a comparison depend on — changing the ruler mid-experiment destroys comparability.

n_gpu_layers was hard-set to 999, so llama.cpp aborted its own memory fitting instead of choosing a split

ต้า 2026-07-29

Do not force n_gpu_layers on a machine that cannot hold the model; let the runtime fit it. And read the abort/warning lines the loader prints before starting a multi-hour run — this log had the answer to both the speed problem and the resolution problem sitting in it.

The t03 spec-gap text was written into two READMEs at once and would have drifted — caught only during the folder reorg

တာ 2026-08-21

Apply the same anti-duplication rule to docs that is already applied to prompts: one topic, one file. If two files would carry the same paragraph, one of them should link instead.

Beam template gap of 21 px is hardcoded to one scan resolution and scale

တာ 2026-08-25

When a constant is measured in pixels from one scan, either derive it at runtime from the drawing scale or make it a required, documented parameter — never leave it as an invisible default that fails silently elsewhere.

Training script's FOLD default was empty, pointing at combined train/val files that had just been deleted

တာ 2026-08-31

When you delete a file, grep for every default that still resolves to it — a fallback to a deleted path is a landmine, not a safety net.

เครื่อง การ์ดจอเช่า และ ค่าใช้จ่าย

เครื่องเช่า โควตา เน็ต และเงินที่เสียไปโดยไม่ได้อะไร

ทั้งหมด 124 เรื่อง · วิกฤต 7 · สูง 54 · กลาง 46 · ต่ำ 17

เรื่องที่ต้องอ่านให้ครบ (61)

วิกฤต 2026-07-15

additional_bars ถูกบันทึกผิดพลาดหน้าของคาน — บั๊กตัวเดียวกันนี้เจอครบทั้งห้าบ้าน

เกิดอะไรขึ้น: เหล็กเสริมถูกบันทึกผิดพลาดหน้าซ้ำแล้วซ้ำเล่า: คาน B2 ของบ้าน 2 ที่ 'หยุดที่ L/8' บันทึกว่าอยู่ด้านบน แต่จริงๆ อยู่ด้านล่าง (2→4); การต่อทาบของ B2X บันทึกว่าอยู่กลางช่วง แต่จริงๆ อยู่ด้านบน (2→3); บ้าน 3 ที่ B3X และ B4; บ้าน 4 ที่ B4, B4A, B4X, B5; บ้าน 5 ที่ B2X, B3X, B4, B5 เจอได้ก็ต่อเมื่อซูมเข้าไปในรูปตัดแล้วนับจุดเหล็กจริง แทนที่จะเชื่อป้ายที่พิมพ์ไว้

ต้นเหตุจริง: การสกัดข้อมูลไปเชื่อข้อความบนป้ายและทิศทางที่เส้นขีดดูเหมือนจะชี้ไป แทนที่จะนับจุดเหล็กจริงในรูปตัด ตำแหน่งของป้ายบนแบบไม่ได้บอกอย่างน่าเชื่อถือว่าเหล็กอยู่หน้าไหน

เสียหายอะไร: การกระจายเหล็กบนคานหลักผิดในทุกบ้านที่รีวิว เป็นความผิดพลาดของปริมาณเชิงโครงสร้างโดยตรง บางกรณีจำนวนเหล็กของหน้าหนึ่งเกือบเป็นสองเท่าหรือเหลือครึ่งเดียว

แก้ยังไ: ซูมเข้าไปในภาพรูปตัดคานแล้วนับจุดเหล็ก จากนั้นรวม additional_bars เข้าหน้าที่ถูกต้องของ main_bar แล้วส่งต่อไปยัง `specs{}` ในผังคาน บันทึกเป็นกติกถาถาวรว่า ให้ตรวจ `additional\_bars[].position` เทียบกับเส้นชี้จริงเสมอ ห้ามเชื่อป้ายที่พิมพ์ไว้

กฎที่ได้: การระบุว่าเหล็กอยู่หน้าไหน ให้นับจุดในภาพรูปตัดที่ซูมแล้ว — ห้ามเชื่อป้ายที่พิมพ์หรือทิศที่เส้นขีดดูเหมือนจะชี้ไป และเมื่อบั๊กโผล่ในสองบ้าน ให้กวาดทุกบ้านทันที ไม่ใช่รอไปเจอทีละบ้าน

วิกฤต 2026-07-19

การ 'แก้' ตารางราคาข้ามบ้าน กลายเป็นการอ่านผิดเสียเอง ทำให้ไฟล์เสียไป 4 ไฟล์ก่อนจะย้อนกลับ

เกิดอะไรขึ้น: วันที่ 2026-07-16 การตรวจข้ามบ้านสรุปว่าค่าของบ้าน 3 (2,945,000/2,034,000) ถูก แล้ว 'แก้' ไฟล์ของบ้าน 2 (2,145,000/2,035,000) ให้ตรงกัน พอถึง 2026-07-19 ซูม 8 เท่าแล้วเทียบกับตัวเลขอ้างอิงบนแผ่นเดียวกัน พบว่าทั้งสี่สำเนาตรงกันที่ 2,145,000/2,035,000 — บ้าน 2 ถูกมาตลอด ที่ผิดคืออ่าน ๑ เป็น ๙ ตอนซูมเข้าไป

ต้นเหตุจริง: ตัวเลขถูกตัดสินแบบโดดๆ ที่ระดับซูมสูง แทนที่จะเทียบกับตัวเลขตัวเดียวกันที่รู้ค่าอยู่แล้วในที่อื่นบนแผ่นเดียวกัน การซูมสูงทำให้รูปตัวอักษรเสียคุณภาพและตัดบริบทรอบข้างที่ช่วยแยกแยะออกไป

เสียหายอะไร: ค่าที่ถูกต้องถูกเขียนทับด้วยค่าที่ผิดข้ามไฟล์ของหลายบ้าน แล้วต้องย้อนกลับใน 5 ไฟล์ และมีสองวันที่เชื่อตัวเลขที่ผิดอยู่ การกวาดรอบเดียวกันนี้เจอค่าที่ผิดจริงๆ อีก (#1, #2, #4, #8 และขนาดแบบบ้าน 3/8) ที่รอบก่อนหน้านามองข้ามไป

แก้ยังไ: ย้อนการแก้ของวันที่ 16 ก.ค. แล้วแก้ #1=1,020,000, #2=1,460,000, #4 ราคาปูพื้น=1,824,000, #8 ราคาปูพื้น 1,662,000→1,622,000, #10 ราคาปูพื้น 2,235,000→2,035,000 และยืนยันว่าตอนนี้ตารางตรงกันครบทั้งห้าบ้าน

กฎที่ได้: อย่าตัดสินเลขไทยแบบโดดๆ — ให้เทียบกับตัวเลขตัวเดียวกันที่รู้ค่าอยู่แล้วบนแผ่นเดียวกันที่ระดับซูมเท่ากันเสมอ และเมื่อการตรวจข้ามแหล่งพบว่าไม่ตรงกัน ให้ตรวจสอบทั้งสองฝั่งใหม่ก่อนจะไปเขียนทับฝั่งไหน

วิกฤต 2026-07-28

HuggingFace write token ถูกวางลงในแชท และจนจบวันก็ยังไม่ได้เพิกถอน

เกิดอะไรขึ้น: token ของ constistant-tune_t02 บน HuggingFace ซึ่งมีสิทธิ์ repo.write ถูกวางลงในบทสนทนา มีการจดไว้ว่าจะเพิกถอนหลัง Phase 8 แต่พอถึงตอนปิดไดอารี่ ก็ยังขึ้นสถานะว่ายังไม่ได้ทำ

ต้นเหตุจริง: ไม่มีช่องทางส่งข้อมูลกลับที่ปลอดภัยสำหรับการทำงานกับเครื่องเช่า credential ที่มีสิทธิ์เขียนเลยต้องเดินทางผ่านบทสนทนาที่ถูกบันทึกไว้ และการเพิกถอนก็ถูกจัดเป็นรายการที่ต้องทำ แทนที่จะทำทันทีตอนที่เลิกใช้ token แล้ว

เสียหายอะไร: credential ที่เขียนข้อมูลได้ ซึ่งเข้าถึงบัญชีที่เก็บ backup ชุดเดียวของ adapter ทั้งสองรอบ ถูกเปิดเผยอยู่ในบทสนทนา และยังใช้งานได้อยู่ตอนจบเซสชัน

แก้ยังไม: ยังไม่ได้แก้ — ยังอยู่ในรายการค้าง: "เพิกถอน HF token ที่วางในแชทวันนี้"

กฎที่ได้: ห้ามวาง credential ที่มีสิทธิ์เขียนลงในบทสนทนา ให้ส่งความลับผ่าน environment ของเครื่องเองหรือผ่านที่เก็บความลับ ถ้าเผลอเปิดเผยไปแล้ว ให้เพิกถอนทันทีที่งานนั้นเสร็จ ไม่ใช่รอตอนจบวัน

วิกฤต 2026-07-28

เฟส 8 export ล้ม: การ merge LoRA adapter เข้าไปในโมเดลฐานแบบ MoE ได้โมเดลที่ output schema ผิดไปคนละเรื่อง

เกิดอะไรขึ้น: export_gguf.py โหลดโมเดลฐาน + adapter แล้ว merge โดยไม่ error เลย แต่ sanity check จับได้ว่าผลลัพธ์ที่ merge แล้วพ่นออกมาเป็น page_number/title/project แทนที่จะเป็น schema จริง

(png/doc_page/discipline/sheet_code/views) สคริปต์วินิจฉัย 4 ตัวพิสูจน์ว่าตัว adapter เองไม่มีปัญหา (schema ตรง 100% ก่อน merge) รูปทรงและทิศทางของ delta_weight ถูกต้อง scaling factor ถูกที่ 2.0 แต่ค่า tensor หลัง merge เปลี่ยนไปแค่ 0.67% ของขนาดจากทั้งหมด 201 ล้านค่า และ _merge_lora() ของ Unsloth เองก็ไม่รองรับ MoE ParamWrapper เลย

ต้นเหตุจริง: เป็นข้อบกพร่องจริงในเส้นทาง merge ของ peft/Unsloth สำหรับ LoRA แบบ target_parameters บน MoE expert — การ merge ใส่ delta ที่แทบเป็นศูนย์ลงไปเจียบๆ แทนที่จะล้มให้รู้เรื่อง ไม่ใช่ปัญหาจากการเทรนหรือชุดข้อมูล และซ้ำเติมด้วยการที่ไม่มีการตรวจตั้งแต่ต้นทางว่าตัว merge รองรับสถาปัตยกรรมที่ใช้อยู่หรือเปล่า ก่อนจะตัดสินใจใช้เส้นทาง export นี้

เสียหายอะไร: t02 ไม่มี GGUF เลย รอบนี้จึงไม่ได้โมเดลที่รันในเครื่องได้สักตัว และการวินิจฉัยก็กินเวลาเครื่องที่จ่ายเงินไปไม่น้อยกับสคริปต์วินิจฉัยถึง 4 ชุด

แก้ยังไม: sanity check ของสคริปต์เองหยุดมันไว้ได้ก่อนที่จะ quantize แล้วอัปโหลดโมเดลขยะขึ้นไป มะขามตัดสินใจ: หยุด หลังทดสอบสมมติฐานหลักไปแล้ว 3 ข้อ ส่วน LoRA adapter ถูกสำรองไว้แยกต่างหากและพิสูจน์แล้วว่าใช้ได้

กฎที่ได้: ต้องวาง sanity check เรื่อง schema/output ไว้ทันทีหลังทุกขั้นตอน merge, quantize หรือ convert เพื่อไม่ให้ของที่เสียแบบเจียบๆ หลุดขึ้นไปเผยแพร่ได้ และต้องตรวจให้แน่ก่อนว่า toolchain ของ merge/export รองรับสถาปัตยกรรมโมเดลของเรา ก่อนที่จะเลือกใช้มันเป็นเส้นทางส่งมอบ

"รอยแผลแห่งความอาย" 2026-07-21 — ปิดเครื่องตอนงาน "ดูเหมือนเสร็จ" เสีย LoRA 7.5GB กับ GGUF 21GB ไปฟรีๆ

เกิดอะไรขึ้น: เคสนี้คือเหตุผลที่ทำให้ต้องมีด้านตรวจก่อนปิดเครื่องใน `op3` เลย — เครื่องดับไปตอนทำงาน *ดูเหมือน* จะเสร็จแล้ว แต่จริงๆ ยังไม่มีอะไรถูกเซฟลงไปสักอย่าง ผลคือ LoRA ขนาด 7.5 GB กับไฟล์ GGUF อีก 21 GB หายไปทั้งหมด

ต้นเหตุจริง: เอา "ดูเหมือนเสร็จ" มาใช้แทน "เสร็จจริง" — ระหว่างตอนทำงานจบกับตอนกดปิดเครื่อง ไม่มีจุดตรวจที่พิสูจน์ว่าเซฟ/commit แล้วจริงๆ เลย และไม่มีกฎที่ต้องพิมพ์สรุปให้ดูก่อนจะดับ

เสียหายอะไร: LoRA ที่เทรนเสร็จแล้ว 7.5GB กับไฟล์ GGUF ที่ export แล้ว 21GB หายเกลี้ยง เท่ากับเผาทั้งชั่วโมง GPU และการเทรนทั้งรอบทั้ง

แก้ยังไง: ออกแบบ `op3` ใหม่ให้มีด้านตรวจ 6 เดือนไซก่อนปิดเครื่อง: ไฟล์ต้องเขียนลงแล้วและ parse ผ่าน → `check\_format.py` ต้อง ALL PASS → มีแถวในสมุดบันทึกข้อมูล → `git commit` แล้ว → พิมพ์สรุปให้ผู้ใช้เห็นก่อน → แล้วค่อย `shutdown /s /t 120` ห้ามใช้ `/t 0` เด็ดขาด และถ้าขึ้นไหนไม่ผ่าน = ไม่ปิดเครื่องเลย

กฎที่ได้: อย่าปิดเครื่องเพราะ "ดูเหมือนเสร็จ" เด็ดขาด ทำตอนจบให้เป็นอัตโนมัติแต่ต้องมีด้านแข็งๆ — ผลลัพธ์ต้องตรวจแล้วเช็คผ่านแล้ว บันทึกแล้ว commit แล้ว และรายงานแล้ว — เกมต้องใช้เวลาหนึ่งให้ยกเลิกทันเสมอ ห้าม `/t 0`

Pass 2 เป็นโศกตายเป็นที่ยังเสียเงินอยู่ — ยิง API N ครั้งต่อรอบ ผลลัพธ์ไม่มีใครใช้ error ก็ถูกกลืน

เกิดอะไรขึ้น: Pass 2 เก็บ `layoutIds` มาแล้วไม่เคยเอาไปใช้เลย (Pass 3 ไปวนลูปบน id ของ Pass 1 แทน) เกม error ก็ถูก `catch` เปลาๆ กลืนหายไป มันเสียค่ายิง API N ครั้งทุกรอบที่รันจริง แล้วไม่ได้อะไรกลับมาเลย

ต้นเหตุจริง: มีขั้นตอนใน pipeline ที่ถูกแทนที่ไปแล้วแต่ไม่เคยถูกลบออก และบล็อก `catch` เปลาๆ ก็ปิดบังความจริงว่ามันพังหรือว่างงานอยู่ ไม่มีใครวัดว่า output ของมันถูกเอาไปใช้จริงหรือเปล่า

เสียหายอะไร: เผาเงินทุกครั้งที่รันจริงโดยไม่ได้ประโยชน์อะไร เกมเสียด้วย ที่แยกหน้านั้นคือ Pass 2 อาจเป็นตัวช่วยเพิ่ม recall ได้จริงถ้าต่อสายให้ถูก (มันจะจับ mark ที่มีในผังแต่ไม่มีในแผน section ได้) เท่ากับจ่ายเงินไปแล้วแต่ไม่ได้ประโยชน์ที่ควรได้

แก้ยังไง: ระบุตัวปัญหาและบันทึกไว้ ข้อเสนอคือให้เลือกอย่างใดอย่างหนึ่ง: ต่อสายให้มันทำงานจริง (อาจช่วยเรื่อง recall) หรือไม่ก็ลบทิ้งไปเลย

กฎที่ได้: อย่าปล่อยให้มี `catch` เปลาๆ เด็ดขาด และให้ไล่ตรวจทุกจุดที่ยิง API แบบเสียเงินว่าผลลัพธ์ถูกเอาไปใช้จริงไหม — ขั้นตอนใน pipeline ที่ไม่มีใครอ่าน output ของมัน ถ้าไม่ใช่บั๊กก็คือบิลค่าใช้จ่าย

requeue_stale() used claim time as heartbeat, so every real job qualified as dead while still running

(ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `requeue_stale()` filtered on `claimed_at`, which is written once when a worker grabs a job and never refreshed. Real jobs run 75–80 minutes while `STALE_PROCESSING_MIN = 45`, so every genuine job entered the 'stale' condition mid-flight. If a second worker started (Makham double-clicking GO.bat, or opening a new window because the job looked stuck), it would flip the running job back to `'pending'` and re-claim it.

ต้นเหตุจริง: Confusing 'when work started' with 'when work was last alive'. There was no heartbeat concept at all — the liveness check read a timestamp that by definition never updates, so the timeout threshold could never be satisfied by a healthy job.

เสียหายอะไร: Double GPU billing for the same house, plus two workers writing over each other's results. Never observed in the wild only because a second worker had not yet been started concurrently.

แก้ยังไง: Switched the filter to `updated_at`, which the `purson_jobs_touch_updated_at` trigger refreshes on every `set_progress` write — so a job still reporting progress can never be considered dead.

กฎที่ได้: A staleness check must read a field that a healthy process actively refreshes. Before shipping any timeout, ask 'what updates this timestamp while the job is alive?' — if the answer is 'nothing', the check is wrong by construction.

แก้ config หั้ววันแต่แก้แค่ในไฟล์บนเครื่อง โปรเจกต์ Label Studio ตัวจริงยังใช้ config เก่าที่พังอยู่

เกิดอะไรขึ้น: ที่แก้ไปทั้งวัน (แก้ scroll, ย้าย header, แบ่ง 50/50, ออกแบบ hdrbar ใหม่) แก้ในไฟล์ XML บนดิสก์อย่างเดียว แต่โปรเจกต์ Elements และ Single (271482, 271493) ที่สร้างไว้แล้วบน Label Studio Cloud ไม่เคยถูกวาง config ใหม่ทับ ของจริงเลยยังมีบั๊กเนื้อหาหายอยู่

ต้นเหตุจริง: config ของ Label Studio ต้อง deploy ด้วยการ copy-paste เข้าหน้าเว็บด้วยมือ และในขั้นตอนการทำงานไม่มีอะไรผูก "แก้ไฟล์" เข้ากับ "deploy" เลย

เสียหายอะไร: บั๊ก scroll ที่เป็นต้นเรื่องของงานทั้งวัน ยังพังอยู่จริงสำหรับคนทำ annotation ทั้งที่ประกาศว่า "แก้แล้ว" และถูกยกยอดไปเป็นงานค้างของวันถัดไป

แก้ยังไง: ยังไม่ได้แก้ในรอบนี้ — บันทึกลงงานค้างข้อ 1: เอา config ล่าสุดไปวางทับในโปรเจกต์ที่สร้างไว้แล้ว

กฎที่ได้: config อะไรก็ตามที่ deploy ด้วยมือ ให้ถือว่า "แก้ไฟล์แล้ว" กับ "ระบบที่รันอยู่เปลี่ยนแล้ว" เป็นคนละเรื่องกัน และห้ามรายงานว่าแก้เสร็จจนกว่าจะไปตรวจสอบในเครื่องมือตัวจริงแล้ว

ข้อมูลหลักเสริมที่เสี่ยงสุดของบ้านหลัง 2 และ 3 ยังติดธง confidence ต่ำ และยังไม่มีการไปชมดู

เกิดอะไรขึ้น: บ้านหลัง 2 ส่วนที่อ่านแล้วไม่มั่นใจเต็มร้อยถูกติดธงไว้แทนที่จะเอา ได้แก่ ส่วนต่อขยายลงใต้เลยกริด C ในผังคาน/คานคอดิน ตำแหน่งของ RB2 ที่ไม่ได้อยู่บนกริดหลัก และแถวหลังคา/คานคอดินในตารางเสาที่ดูเหมือนช่องว่างหรือลูกศร บ้านหลัง 3 ก็เจอแบบเดียวกัน คือตำแหน่งพื้นที่ส่วนต่อขยายหน้าต่างโค้ง RB2 ที่ออกนอกกริดหลักอีกครั้ง และจำนวนเหล็ก dowel ที่มีเครื่องหมาย '+' โพลมาสองครั้ง ทุกอันได้ confidence_score ต่ำพร้อมธง needs_human_review_zoom_required

ต้นเหตุจริง: ความละเอียดของภาพและธรรมเนียมการเขียนแบบ (ลูกศรที่แปลว่า "ต่อจากชั้นล่าง" แถวตารางที่ดูเหมือนว่างเปล่า) เกินกว่าจะอ่านได้แน่นอนที่ระดับการซูมที่ใช้อยู่ และยังไม่มีการจัดคิวให้ไปชมตรวจรายการที่ติดธงเหล่านั้น

เสียหายอะไร: นี่คือส่วนโครงสร้าง — ส่วนที่บอกไว้เองว่าเสี่ยงสุดและต้องอ่านเองแทนที่จะให้ agent อ่าน ด้วยเหตุผลนี้พอดี ตำแหน่งและจำนวนเหล็กที่ติดธงแต่ยังไม่ได้ตรวจ นอนอยู่ในโพลเดอร์ของจริงของบ้านหลัง 2 และ 3 และเป็นตัวเลือกที่จะกลายเป็น ground truth สำหรับ fine-tune

แก้งัง: ยังไม่ได้แก้ — บันทึกธงไว้ ระบุว่าตรวจยังไม่ค้างอยู่ แต่ในรายการงานค้างวันที่ 11 ก.ค. ระบุรายการ BOQ ของบ้านหลัง 2 ที่ confidence ต่ำ ไม่ได้ระบุรายการโครงสร้างเลย

กฎที่ได้: ติดธงแทนการเอาเป็นสิ่งที่ถูกแล้ว แต่ธงที่ไม่มีคิวตรวจก็คือการเอาที่แค่เลื่อนออกไป ธงในส่วนโครงสร้างต้องมีรอบตรวจที่ระบุชัดเจนจะเรียกบ้านหลังนั้นว่าเสร็จ และรายการงานค้างต้องเขียนมันลงไปจริงๆ

ลิมิต session ของบัญชี Claude ฆ่า agent ทำ BOQ ทั้งสองตัวกลางทางบนบ้านหลังที่ 3

เกิดอะไรขึ้น: ทั้ง agent ทำ BOQ ฐานรากแผ่ (หน้า 64-74) และ agent ทำ BOQ ฐานรากเสาเข็ม + ผังบริเวณ (หน้า 75-89) ถูกหยุดโดยลิมิต session ของบัญชี ซึ่งรีเซ็ตตอนตี 2 เวลาไทย ตัวแรกขาดไฟล์ page74_2 หนึ่งไฟล์ ตัวที่สองขาดส่วน B ทั้งหมด (หน้า 86-89)

ต้นเหตุจริง: ส่ง agent ห้าตัวออกไปพร้อมกันโดยใช้บ session ของบัญชีร่วมกัน โดยไม่ได้เช็คก่อนว่าเหลือบเท่าไร และไม่มิงบ retry ต่อ agent

เสียหายอะไร: บ้านหลังที่สองติดกันที่ค้างไม่เสร็จเพราะ agent ถูกฆ่า ต้องมาซ่อมด้วยมือ และถ้าส่ง agent ใหม่ก็จะชนลิมิตเดิมอีก เพราะมันยังไม่รีเซ็ต

แก้งัง: ซ่อมด้วยการอ่านหน้าที่ขาดเองแทนที่จะส่ง agent ใหม่ และเขียนบทเรียนลงบันทึกไว้ว่า ให้เช็คก่อนว่า agent ไปได้ถึงไหน แล้วค่อยอ่านส่วนที่ขาดด้วยมือ

กฎที่ได้: ให้ถือว่าลิมิต session ของบัญชีเป็นทรัพยากรที่ใช้ร่วมกัน เช็คที่เหลือเท่าไรก่อนกระจาย agent ขนานกัน และเวลา มีตัวไหนถูกฆ่า ให้วัดก่อนว่ามันผลิตอะไรออกมาได้บ้าง แล้วค่อยตัดสินใจว่าจะซ่อมมือหรือส่งใหม่

สูง 2026-07-12

ส่ง agent เบื้องหลัง 5 ตัวพร้อมกัน ขนลิมิต session ของบัญชีกลางทางซ้ำแล้วซ้ำอีก

เกิดอะไรขึ้น: ส่ง agent เบื้องหลัง 5 ตัวขนานกันเพื่อทำส่วนที่เหลือของเอกสาร สามในห้าตัว (สถาปัตยกรรม, BOQ ฐานรากแผ่, BOQ ฐานรากเสาเข็ม) ถูกลิมิต session ของบัญชีหยุดกลางทาง เรื่องเดียวกันนี้เกิดมาแล้วกับบ้านหลัง 2 และ 3

ต้นเหตุจริง: รูปแบบการกระจาย agent ขนกันเกิน session ที่ใช้ร่วมกันเร็วกว่าที่งานของบ้านหลังเดียวจะเสร็จ ขั้นตอนการทำงานถูกออกแบบมาโดยไม่มีการคิดขยับว่าบัญชีรับ agent ได้กี่ตัวจริงๆ

เสียหายอะไร: ต้องทำงานสกัดข้อมูลใหม่ด้วยมือประมาณ 55 หน้า กระจายอยู่ในสามส่วน และเสียหายแบบเดิมซ้ำทุกหลังตั้งแต่หลังที่ 2 เป็นต้นมา

แก้ยังไม: แทนที่จะส่ง agent ใหม่ (ซึ่งจะแพงไปอีก) เลย์อ่านหน้าที่ขาดเอง วิธีกู้คืนที่เชื่อได้คือ ไปดูไฟล์ที่ถูกเขียนลงดิสก์จริง ไม่ใช่ดูข้อความสุดท้ายที่ agent รายงาน

กฎที่ได้: ห้ามเชื่อข้อความสุดท้ายของ agent ที่ถูกตัดกลางทางว่าเป็นจุดเช็คพอยต์ — ให้ diff ไฟล์บนดิสก์กับรายการหน้าที่คาดว่าจะได้เสมอ และกำหนดจำนวน agent ที่กระจายขนานกันตามงบ session จริง ไม่ใช่ส่งเต็มพิกัดทุกครั้ง

สูง 2026-07-16

regex ที่ใช้ตัดชั้ดกลาง ไปทำ sheet_code พังหมดเจียบๆ

เกิดอะไรขึ้น: regex ที่ใช้ตัดชั้ดกลางออกจาก grid ref แบบจุด ดันไปแมตช์กับค่า `sheet\_code` ด้วย ทำให้ 'S-04' กลายเป็น 'S04' ในทุกไฟล์ที่มันรันผ่านในบ้านหลัง 3

ต้นเหตุจริง: regex แมตช์รูปแบบ ตัวอักษร-ขีด-ตัวเลข ที่ไหนก็ได้ในไฟล์ แทนที่จะจำกัดขอบเขตไว้เฉพาะฟิลด์ grid_ref ที่ตั้งใจจะแก้จริงๆ

เสียหายอะไร: ฟิลด์หนึ่งพังไปทั้งบ้านแบบลูกหลง ต้องไล่หาและย้อนคืนทีละไฟล์ ทีหลังพอรันกับบ้านหลัง 5 ก็ต้องเช็คการ normalize ด้วย perl เป็นพิเศษเพื่อยืนยันว่า "sheet_code รอด"

แก้ยังไม: ไล่เช็คและแก้ sheet_code กลับคืนในทุกไฟล์ที่โดน แล้วการ normalize แบบยกชุดครั้งต่อๆ มาก็คตรวจสอบความถูกต้องของ sheet_code หลังรันทุกครั้ง

กฎที่ได้: จำกัดขอบเขต regex ที่รันยกชุดให้อยู่แค่ฟิลด์ JSON ที่ต้องการจริงๆ ห้ามรันกับข้อความดิบทั้งไฟล์ — และต้อง diff ผลลัพธ์ดูตัวอย่างสักชุดเพื่อหาการแมตช์ลูกหลง ก่อนรันจริงทั้งบ้าน

สูง 2026-07-19

B2X ถูกอ่านเป็นคาน 4 ตัวแนวตั้ง ทั้งที่จริงเป็นช่วงหนึ่งของคานแนวนอน

เกิดอะไรขึ้น: ในไฟล์ `หน้า25\_beam\_plan.json` ของบ้านหลัง 2 B2X ถูกบันทึกเป็นคานแนวตั้ง 4 ตัว พอซูม 3 เท่าถึงเห็นว่ามันคือช่วงกลางของคานแนวนอนบนแนว E/D ในช่วงส่วนต่อขยายที่ยื่นออกไป 1.00 ม. ถึง `1'"/4'`

ต้นเหตุจริง: เดาแนววางคานจากตำแหน่งของ mark ในแบบ แทนที่จะไล่ตามแนวเส้นของตัวคานจริงๆ แกมคานแนวนอนบนแนว E/D รอบๆ ก็หายไปหมด เลยไม่เหลือบริบทที่จะให้เห็นแนววางที่ถูกต้อง

เสียหายอะไร: แนววางผิดไป 4 ตัว บวกกับคานแนวนอนที่หายไปทั้งต้นอีก 6 ตัว (E1-E2, E2-E3, E3-E4, D1-D2, D2-D3, D3-D4) และ label ที่ grid2/3 ผิด (B2 ควรเป็น B1)

แก้ยังไม: สกัดข้อมูลใหม่ที่ซูม 3 เท่า เพิ่มคานแนวนอนที่หายไปทั้งหมดและคานแนวตั้งที่ `1'"/3'"/4'"/F` แล้วแก้ label ของ mark ให้ถูก

กฎที่ได้: ไล่ตามแนวเส้นของคานทุกตัวก่อนบันทึกแนววาง และเช็คด้วยว่าคานในแต่ละแถวต่อเนื่องกันครบไหม — ช่องว่างในแถวคือสัญญาณว่ามีของหาย ไม่ใช่ช่องที่ว่างเปล่าจริง

สูง 2026-07-20

onstart.sh ทำให้เครื่องเช่าเครื่องแรวนลูปพัง เพราะ **pip upgrade** ชนกับ **pip** ที่ลงมาจาก **apt**

เกิดอะไรขึ้น: คำสั่ง `pip install --upgrade pip` ใน `onstart.sh` ชนกับ `pip` ที่ติดตั้งมาจาก `apt` แล้ว error ตัว `set -euo pipefail` เลยฆ่าสคริปต์ทั้งตัว ทำให้เครื่องเช่าเครื่องแรวนลูปพังไปเลย

ต้นเหตุจริง: เอาคำสั่งที่ไม่จำเป็น (แค่ทำเพื่อความสะดวก) ไปใส่ในสคริปต์โหมดเข้มงวดที่ไม่ยอมให้อะไรพลาดได้เลย บนเครื่องที่ `base image` ติดตั้ง `pip` มาทาง `apt` อยู่แล้ว

เสียหายอะไร: เครื่อง GPU ที่เช่ามาถูกเผาทิ้งไปกับการรวนลูปพัง ต้องเช่าเครื่องที่สอง (id 45406175) มาใหม่

แก้ยังง: เติม `|| true` ต่อท้ายคำสั่ง `pip upgrade` แล้วเครื่องที่สองรันผ่านตลอด ยืนยันจาก log แล้ว

กฎที่ได้: ในสคริปต์ provisioning ที่ใช้ `set -euo pipefail` คำสั่งที่ไม่จำเป็นทุกตัวต้องมี `|| true` และบนเครื่องเช่า สคริปต์ตอนบูตพังเท่ากับเสียเงิน ให้ซ่อมรันบนเครื่องถูกๆ ก่อนขึ้นเครื่องแพง

สูง 2026-07-21

เช่า GPU แพงมาก่อน แล้วนั่งติ๊กสคริปต์สตาๆ บนเครื่องนั้น

เกิดอะไรขึ้น: เช่า RTX PRO 6000 96GB ราคา \$1.056/ชม. มาก่อน ทั้งที่ยังไม่เคยพิสูจน์เลยว่าสคริปต์เทรน/ประเมินผลรันได้จริงไหม แล้วก็นั่งไล่บั๊ก 4 ตัวกับ OOM อีก 4 รอบไปโดยที่มีเตอร์เดินอยู่

ต้นเหตุจริง: ในกระบวนการไม่มีขั้นตอนซ่อมบนเครื่องถูกๆ เลย สมมติเอาว่าสคริปต์ถูกอยู่แล้วโดยไม่ได้ตรวจสอบ ที่รันจริงครั้งแรกเลยกลายเป็นเครื่องที่แพงที่สุดไปด้วย

เสียหายอะไร: เสียเวลาหลายชั่วโมงที่ \$1.056/ชม. ไปกับการติ๊กแทนที่จะเทรน มะขามหักตรงๆ ว่ามันเปลืองเงินชัดๆ ("ขนาดแชตบอทยังรู้เลยว่าเปลืองเงิน") และต้องเติมเงินอีก \$10 (รวมประมาณ \$19)

แก้ยังง: บันทึกลงหน่วยความจำการเป็นกฎประจำ: เช่าเครื่องถูกๆ มาทดสอบสคริปต์ก่อน แล้วค่อยเช่าเครื่องแพง พร้อมเพิ่มสวิตช์ `TEST_STEPS=5` ไว้ซ่อมใน `train_qwen36.py`

กฎที่ได้: ห้ามติ๊กบน GPU ระดับพรีเมียม ให้พิสูจน์ทุกสคริปต์ตั้งแต่ต้นจนจบบนเครื่องที่ถูกที่สุด (หรือ dry-run เล็กๆ ในเครื่องตัวเอง) ก่อน การดแพงมีไว้สำหรับงานที่รู้แล้วว่ารันผ่านเท่านั้น

สูง 2026-07-21

ดิสก์เต็มซ้ำๆ ระหว่าง merge/GGUF และเต็ม 100% พอดีตอนแปลงไฟล์ค้างอยู่

เกิดอะไรขึ้น: ดิสก์ 150GB ของเครื่องเต็มซ้ำแล้วซ้ำอีกระหว่างขั้นตอน `merge`→`f16`→`quantize` ต้องลบ checkpoint เก่า 44GB, HF cache 67GB, pip cache 5.4GB และไฟล์ระหว่างทาง `q8_0/f16` ทั้งกลางคัน และมีจังหวะหนึ่งดิสก์เต็ม 100% พอดีตอนที่แปลง `f16` เสร็จ ทำให้ต้องลบ `merged/` แบบฉุกเฉิน

ต้นเหตุจริง: ไม่ได้คำนวณงบพื้นที่ดิสก์ก่อนเริ่ม ทั้งที่ไฟล์ระหว่างทาง (โมเดล 70GB + `f16` 66GB + quantized 21GB + cache) มันเกิน 150GB ชัดๆ อยู่แล้ว ส่วน `verify_env.py` ก็เช็คแค่ GPU/VRAM/ไลบรารี ไม่ได้เช็คพื้นที่ดิสก์ว่าง

เสียหายอะไร: งานสะดุดหลายรอบและต้องลบไฟล์แบบฉุกเฉินภายใต้ความกดดันเรื่องเวลา — และการลบ `merged/` แบบตกใจนี้แหละที่ทำให้ทีหลังสกัด `mmproj` ไม่ได้

แก้ยังง: ลบ cache / checkpoint / ไฟล์ระหว่างทางไปเรื่อยๆ ตามที่พื้นที่หมด แล้วแปลงต่อจนจบ

กฎที่ได้: คำนวณพื้นที่ดิสก์สูงสุดที่ทั้ง pipeline ต้องใช้ก่อนเช่าเครื่อง เลือกเครื่องที่มีที่เหลือเผื่อ และเพิ่มการเช็คพื้นที่ว่างเข้าไปใน `verify_env.py` — และห้ามเรียกคืนพื้นที่ด้วยการลบไฟล์ที่ยังใช้ไม่เสร็จ

สูง 2026-07-21

Instance rented with 150GB disk for a pipeline that needs well over 200GB of intermediates (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The rented machine had only 150GB of storage, but the merge/GGUF chain needs the 70GB source model + checkpoints (44GB) + merged model (66GB) + f16 GGUF (~70GB) + the Q4_K_M output (21GB) at overlapping times. The whole conversion was run as a series of emergency deletions: old checkpoints (44GB), the HF cache (67GB), the pip cache (5.4GB), intermediate q8_0/f16 files — and the disk still hit exactly 100% mid-conversion.

ต้นเหตุจริง: Disk size was never sized against the downstream pipeline when choosing the instance — only GPU/VRAM was checked (verify_env.py checks VRAM, not free disk). The whole merge/quantize chain was planned only after the machine was already rented.

เสียหายอะไร: Hours of firefighting during the conversion; forced the emergency deletion of `merged/`, which is the direct reason the vision component (mmpoj) could no longer be extracted.

แก้ยังไง: Deleted files reactively at each failure until the conversion squeezed through.

กฎที่ได้: Size the instance's disk against the *largest simultaneous footprint* of the whole pipeline (source + merged + f16 + quantized), not just the model size, and pick disk at rent time — it cannot be grown later. Add a free-disk assertion to verify_env.py.

สูง 2026-07-21

Rented instance disk filled up repeatedly during GGUF export (150GB was too small) (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: During merge and GGUF conversion the instance ran out of disk multiple times, forcing manual deletion of old checkpoints, cached base-model files and intermediate conversion artifacts mid-process.

ต้นเหตุจริง: Disk was sized for training only. Nobody added up the peak simultaneous footprint of base model + merged model + f16 GGUF + quantized GGUF before renting.

เสียหายอะไร: Repeated interruptions and manual cleanup during a paid session; risk of deleting something still needed.

แก้ยังไง: Deleted unused artifacts as the run progressed; on the next rental the required disk was pre-computed at 300GB.

กฎที่ได้: Size instance disk from the peak concurrent artifact footprint of the whole pipeline (train + merge + export + quantize), not just the training step.

สูง 2026-07-24

Debugging done on a rented GPU while the hourly meter was running, instead of on free local hardware (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: During the 2026-07-21 round, problems were hit after renting and were then fixed live on the paid instance at ~30 baht/hour, rather than having been caught beforehand for free on the local machine.

ต้นเหตุจริง: No separation between checks that require the rented GPU and checks that don't. Every verifiable-for-free item (API token validity, account balance, disk defaults, script correctness, model-runs-locally) was deferred until after the expensive resource was already ticking.

เสียหายอะไร: Rental hours burned on diagnosis rather than training; unquantified but sufficient that the team built an entire Phase 0 regime to stop it.

แก้ยังไง: Phase 0 created: token test, balance check, local dry run, script re-read, machine selection — all completed before the Rent button is pressed.

กฎที่ได้: Sort every pre-flight check into 'needs the paid resource' vs 'doesn't'. Everything in the second bucket must be green before the first satang is spent.

สูง 2026-07-24

Training script hangs at its epilogue holding 83GB of VRAM, causing the next run to fail OOM (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: After the short test (`TEST_STEPS=5`) finished, the process refused to exit and hung holding 83GB of VRAM. It had to be killed by hand. Because it wasn't, the first Phase 6 run died with an OOM-adjacent meta-tensor error — not enough VRAM was free at restart. The hang recurred on every subsequent run, at the demo-generate epilogue after saving.

ต้นเหตุจริง: The demo-generate epilogue after saving hangs (later traced to the same reasoning-mode template problem that broke evaluation), and the process holds its CUDA context until killed.

เสียหายอะไร: One wasted full training start that died OOM; every subsequent run needed a manual kill, on billed GPU time.

แก้ยังไง: Kill the process by hand after the adapter is saved — it does not affect the training result. The underlying cause is the ``enable_thinking`` template mismatch fixed later.

กฎที่ได้: Always verify VRAM is actually released before starting the next run on a rented GPU — a held context makes the next run's OOM look like a model-size problem when it is a cleanup problem. And treat a script that will not exit as a bug to root-cause, not a quirk to work around.

สูง 2026-07-24

Deleting vision_config to skip mmproj conversion broke the main model save with triple-nested tensor names (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Unsloth auto-detects the model as a VLM and tries to convert mmproj even though that was deliberately skipped; the conversion fails on the too-new architecture and brings the whole export down. The first fix attempt — temporarily deleting `model.config.vision_config` — broke something different instead, producing tensor names nested three levels deep (`language_model.language_model.language_model...`).

ต้นเหตุจริง: The fix targeted a config field that affects how the main model is saved, not only mmproj conversion. The blast radius of the config change was not traced before making it.

เสียหายอะไร: A broken export requiring another cycle of debugging and re-running on billed GPU time; each re-run added to the disk-full problem.

แก้ยังไง: Monkey-patched `unsloth.save.convert_to_gguf` to force `supported_vision_archs=set()` (forced, not defaulted, since the kwarg is always present), triggering Unsloth's own built-in `is_vlm=False` fallback — which does not touch the main model save at all.

กฎที่ได้: Before mutating a shared config object to disable one code path, trace every consumer of that field. Prefer patching the specific function whose behaviour you want over removing state that many paths read.

สูง 2026-07-24

Export failed with 'No space left on device' from 147GB of stale artifacts left by repeated re-runs (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The export hit `'No space left on device'`. Cleanup recovered 147GB: old checkpoints duplicating the real LoRA adapter (33+11GB), pip cache (3GB), and a stale `outputs_qwen36/gguf/` directory (66GB).

ต้นเหตุจริง: Every failed export re-run wrote a fresh full-size artifact set without cleaning up the previous one, and there was no disk-space check between attempts.

เสียหายอะไร: A failed export on billed GPU time, plus the whole cleanup cycle; on a 150GB default disk this would have hit far sooner.

แก้ยังไง: Deleted stale checkpoints, pip cache, and the stale gguf directory, recovering 147GB.

กฎที่ได้: Any script that writes multi-GB artifacts must clean the previous run's output before starting, and check free space against the expected output size first. On a rented machine, disk exhaustion mid-export is money spent for nothing.

สูง 2026-07-24

Running llama-cli without --ctx-size reserved a 262k-token KV cache and left 0.1GB of RAM free (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The first local test used `llama-cli --n-gpu-layers 999` with no `--ctx-size`. RAM dropped to 0.1GB free out of 31.4GB; `llama-cli` had committed 34.96GB of virtual memory — more than all installed RAM — because the model supports a 262,144-token maximum context and reserved a KV cache for it. The user spotted it first.

ต้นเหตุจริง: `--ctx-size` defaults to the model's declared maximum, and this model's maximum is enormous. The KV cache scales with context, so the default reserves far more than any single-page task needs.

เสียหายอะไร: The workstation was pushed to the edge of freezing while in use. The identical mistake recurred the next day at ctx-size 12288 (0.45GB free), requiring another emergency kill.

แก้ยังง: Killed the process (21GB of RAM returned), re-ran with `--ctx-size 8192` (sufficient for one page plus the answer), using ~20GB and stable.

กฎที่ได้: Always pass an explicit `--ctx-size` sized to the actual task — never let it default to the model's maximum. On a shared workstation, check free RAM during the first run of any new inference command.

สูง 2026-07-25

Rented GPU instance left possibly running with destroy unconfirmed (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Backups were verified in two independent places and the green light was given to destroy the rented instance, but it was never confirmed that the destroy actually happened. Phase 11 stayed unticked and the instance had to be assumed still running at ~\$0.111/hr.

ต้นเหตุจริง: The teardown handoff had no confirmation step — 'green light given' and 'instance destroyed' were treated as the same event by everyone except the diary.

เสียหายอะไร: Ongoing GPU charges of unknown duration; the entry explicitly ties this to a prior 'DAY OF SHAME' incident on the 21st when backups had not been done before teardown.

แก้ยังง: Not fixed — recorded honestly as unconfirmed, with the instance assumed running until the user confirms.

กฎที่ได้: Teardown is not done until the provider confirms it — check the instance list, do not accept 'I gave the green light' as completion. Money keeps burning on the gap between approval and action.

House 07 EE-03/04: switches (solid dots) systematically counted in with light fixtures; one whole group missing and one row assigned to the wrong room (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Across the electrical sheets, solid-dot switches were being counted as light fixtures — systematically, not once. Recounting gave floor 1 = 29 fixtures (group b had been missing entirely) and floor 2 = 19 (row d had been assigned to the wrong room).

ต้นเหตุจริง: Symbol classification was done by visual similarity rather than against the sheet's own legend, and the error was systematic because the same wrong rule was applied everywhere. The missing group and the misassigned row show the count was never reconciled per room against the legend.

เสียหายอะไร: Fixture counts wrong on both floors in both directions — inflated by counted switches, deflated by a missing group — feeding electrical quantities and costs.

แก้ยังไง: Full recount by symbol type against the legend; correct totals recorded as 29 and 19.

กฎที่ได้: Count each symbol type separately against the sheet's legend and reconcile per room, never by lumping visually similar marks. A systematic classification error corrupts every sheet it touches, so fix the rule and recount everything, not just the instance found.

House 07 M-02/03: plan adjacency read as physical stacking — CU pair, CEF-3's room, and CU-2/01's roof all placed wrong (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Three mechanical placement errors from the same cause: the CU pair was recorded as stacked when it is lined up along the wall, CEF-3 was placed in the stair hall instead of the bathroom, and CU-2/01 was placed on the tiled wing of S-05 instead of the metal-sheet roof of the S-04 frame.

ต้นเหตุจริง: A 2D plan projects 3D arrangement onto one surface; two symbols drawn close together were interpreted as physically stacked, and room/roof assignment was inferred from proximity in plan rather than traced to the enclosing boundary or the structural frame beneath.

เสียหายอะไร: Three equipment items located wrongly — affecting which structure carries them (a metal-sheet roof and a tiled wing are different load cases entirely) and which room's services they belong to.

แก้ยังไง: Corrected by re-reading against the room boundaries and the S-04/S-05 frame layout; logged as one of three recurring error classes.

กฎที่ได้: Never infer 3D arrangement or containment from 2D proximity. Trace equipment to its enclosing room boundary and to the structural frame drawn beneath it, on the structural sheet, before recording a location.

สูง 2026-07-28

Vast.ai's listed RAM of 76GB was misleading — the delivered machine had 188GB — and RAM had never been a selection criterion at all (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The candidate machine's listing showed 76GB RAM, below the new $\geq 80\text{GB}$ criterion, forcing three layers of mitigation to be designed (verify_env warning, adapter-to-HF before GGUF, a 64GB swap plan). The machine actually delivered had 188GB. Separately, t01 had never had a RAM criterion at all — that round's machine simply happened to have enough.

ต้นเหตุจริง: Two failures at once: the vendor's advertised RAM figure does not reflect what is delivered, and the resource that actually kills the bf16 30B merge (host RAM, $\sim 60\text{GB}$, and it dies with a silent Killed) had never been part of machine selection because it had never yet been the binding constraint.

เสียหายอะไร: Planning and mitigation work done against a false number; and a latent gap — any earlier round could have been silently killed by RAM with no criterion in place to prevent it.

แก้ยังไง: RAM $\geq 80\text{GB}$ set as a criterion; verify_env.py extended to read RAM from /proc/meminfo; guarded in three layers including a documented swap remedy that avoids retraining.

กฎที่ได้: Verify advertised specs on the delivered machine before relying on them, and derive selection criteria from every resource that can kill the job — including the ones a previous round survived only by luck. A silent 'Killed' means host RAM, not GPU.

สูง 2026-07-28

The rented GPU machine was left running at the end of the session, still billing at \$1.161/hour (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: At the close of the day the machine had not been shut down. It is blocked behind a Phase 11 hard block requiring an explicit instruction from Makham, so it continues to bill until that instruction arrives.

ต้นเหตุจริง: The shutdown gate correctly requires human authorisation (to prevent destroying unbacked-up work), but there is no mechanism ensuring that authorisation is actually requested and obtained before the session ends — so the safety rule converts directly into open-ended spend.

เสียหายอะไร: Continuous billing at \$1.161/hour on an idle machine for an unbounded period after all useful work finished and all data was confirmed safe.

แก้ยังไง: Not fixed — listed under Pending, with the note that all data is already safe whenever shutdown does happen.

กฎที่ได้: A human-authorisation gate on shutdown must be paired with a deadline or dead-man's switch. Once backups are verified, an idle metered machine is pure loss — request the authorisation explicitly before ending the session, never leave it as a to-do.

สูง 2026-07-28

Option B (retraining t01 at resize='max') was declined, leaving the round's original question permanently unanswered (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Faced with the invalidated comparison, two options were offered: accept with warnings, or retrain t01 at matched resolution for ~\$0.60-1.80 more. Option A was chosen. The A/B question the whole round existed to answer therefore remains open, and answering it later requires redoing t01's training first.

ต้นเหตุจริง: The decision was made after the fact, when the sunk cost of the day was already visible and the incremental cost looked separate from it — rather than the far larger cost of the round having produced no answer.

เสียหายอะไร: A full paid tuning round plus a day of analysis produced no answer to its own research question; the outstanding work is now larger than the \$0.60-1.80 it would have cost to close it on the spot.

แก้ยังไง: Not fixed — recorded under Pending: to answer 'which family is genuinely better' fairly, t01 must first be retrained with resize='max'.

กฎที่ได้: When an experiment is found invalid, weigh the remediation cost against the entire cost already sunk, not against zero. A round that answers nothing has already spent far more than the fix.

สูง 2026-07-29

21GB model on an 8GB card — CPU offload made local inference ~20 hrs/house (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Q4_K_M was chosen specifically so the model would run on the laptop, but 21GB still exceeds 8GB VRAM. llama.cpp aborts fitting ('failed to fit params to free device memory ... abort'), roughly two-thirds of the model runs on the CPU, generation drops to 3.46 t/s, and a full page takes 13-21 minutes → ~20 hrs/house × 5 houses.

ต้นเหตุจริง: The quantization target was picked to shrink the file, not to satisfy a measured VRAM budget. Nobody computed 'does 21GB fit in 8GB' before committing to the local-inference plan.

เสียหายอะไร: The entire local extraction plan is economically unworkable at this speed, and the quantization quality hit was accepted for a fit that never actually happened.

แก้ยังไง: not fixed — GPU rental proposed but correctly deferred until the LoRA question is settled

กฎที่ได้: Before choosing a quantization level, write the arithmetic down: model size vs available VRAM, with headroom for context. If it does not fit, quantizing further or renting is a decision to make up front — not something to discover from an abort message mid-run.

สูง 2026-07-29

Almost rented a GPU to fix 'slow' when the real problem was 'wrong' (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A GPU rental ($\geq 24\text{GB}$, \$0.2-0.5/hr) was proposed to speed up the extraction run. It was caught in time that renting only addresses speed, and if the LoRA is genuinely absent from the GGUF, the rented machine produces identical garbage $10\times$ faster.

ต้นเหตุจริง: Reflex to treat the visible symptom (slowness) before the diagnosis of the invisible one (quality) had completed.

เสียหายอะไร: Money not burned — this one was caught before spending. Cost would have been rental fees plus a second round of unusable output.

แก้ยังไม: fixed — rental explicitly blocked until the base-model comparison test is run

กฎที่ได้: Never buy capacity to fix a quality problem. Order the work by 'what changes the decision': a free 20-minute diagnostic always precedes any paid step, because a wrong diagnosis just makes the failure faster and more expensive.

สูง 2026-07-29

Rented Vast.ai instance from the 28th still not shut down (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Shutting down the rented Vast.ai instance was a hard block on the 28th (Phase 11) and is still listed as carried over and not done on the 29th.

ต้นเหตุจริง: Teardown lives in a prose checklist at the end of a work session, so it is the first thing lost when a session ends abruptly or attention moves to the next problem.

เสียหายอะไร: Continuous hourly billing for a machine doing no work, for more than a day.

แก้ยังไม: not fixed — still listed as carried over from the 28th

กฎที่ได้: Instance teardown is not a checklist item, it is an automated deadline: set a self-destruct/auto-stop on the instance at rental time so forgetting costs nothing. Never let 'shut it down' depend on a human remembering after the interesting work is over.

สูง 2026-07-29

Renting a GPU was on the table as a response to bad output quality, which would have burned money for nothing (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: With extraction slow and bad, renting a GPU was a natural next move. It was explicitly blocked: renting fixes speed, not correctness, and if the LoRA is genuinely absent from the GGUF the rented run produces the same garbage, faster.

ต้นเหตุจริง: Two distinct symptoms (slow, wrong) tempt one expensive remedy. Without root-causing 'wrong' first, spend gets allocated against the symptom that is easiest to price.

เสียหายอะไร: Avoided — the free 20-minute base-model comparison was ordered first. Had it been skipped, GPU rental money would have produced identical bad output.

แก้ยังไง: Priority order set: (1) free base-model comparison to confirm LoRA presence, (2) free fix of the export sanity gate, (3) only then decide anything costing money. Explicitly forbade continuing house 01 extraction meanwhile.

กฎที่ได้: Never spend money on a symptom you have not root-caused. Run every free diagnostic first, and stop producing output into the dataset while the pipeline is under suspicion.

สูง 2026-08-01

5 commits with ~2.8GB of drawing PNGs sat unpushed for a long time because GitHub caps a push at 2GB (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The Training repo had 5 old commits totalling ~2.8GB of house-drawing PNGs that would not push. The cause was GitHub's 2GB per-push cap. They had to be split into ~500MB batches and pushed one chunk at a time.

ต้นเหตุจริง: Large binary assets were committed straight into git with no size awareness, and the push failure was not diagnosed when it first happened — it was simply left stuck for weeks.

เสียหายอะไร: ~2.8GB of work existed only on one laptop for an extended period with no remote backup; then a whole session spent splitting and pushing chunks.

แก้ยังไง: Split the history into ~500MB pushes and pushed sequentially until complete; verified the diff against the source matched 100% afterwards.

กฎที่ได้: Diagnose a failing push the day it fails — an unpushed commit is unbacked-up work. Keep pushes under 2GB and keep bulk binary assets out of git (or in LFS) from the start.

สูง 2026-08-01

ku_project_jop cannot be pushed at all — a 2.1GB zip is baked into commit history (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The `ku\_project\_jop` repo cannot be pushed. A 2.1GB zip ('data from รุ่นพี่') was committed, then deleted in a later commit — but the blob remains in history, so every push still carries it. History has to be rewritten to strip it.

ต้นเหตุจริง: A large archive was committed without checking its size, and the 'fix' was `rm` + commit, which does not remove the blob from history — a very common misunderstanding of how git stores data.

เสียหายอะไร: An entire repo has been unpushable and unbacked-up; the fix (history rewrite) is destructive and was still pending a decision at the time of writing.

แก้ยัง: Not fixed — waiting on confirmation to rewrite history and strip the blob.

กฎที่ได้: Deleting a large file in a later commit does NOT shrink the repo. Never commit multi-GB archives; if one lands, rewrite history immediately (filter-repo/BFG) before more commits pile on top of it.

สูง 2026-08-02

การสั่งปิดเครื่องของ op3 รู้จักแค่บ้านของตัวเอง — มันจะฆ่างานอื่นบนเครื่องไปด้วย

เกิดอะไรขึ้น: บันทึกลงไปเป็นข้อจำกัดตั้งแต่ตอนสร้าง op3 ว่าห้ามใช้ op3 ตอนที่มีอย่างอื่นรันอยู่บนเครื่อง (งานเทรน งานอัปโหลด หรือ agent ตัวอื่น) เพราะด้านตรวจของมันเป็นแค่บ้านของตัวเองเสร็จหรือยัง

ต้นเหตุจริง: ตัวตรวจว่างานเสร็จมองแค่งานชิ้นเดียว แต่สิ่งที่มันสั่งให้ทำ (ปิดเครื่อง) มีผลกับทั้งเครื่อง

เสียหายอะไร: เป็นอันตรายที่ซ่อนอยู่: ถ้ารัน op3 ตอนงานเทรนหรืองานอัปโหลดกำลังวิ่งอยู่ งานนั้นจะพังแบบเดียวกับเหตุการณ์ Mark of Shame ครั้งแรกเป๊ะ

แก้ยัง: เขียนเป็นข้อห้ามถาวรไว้ในนิยามของ op3 แทนที่จะบังคับด้วยโค้ด

กฎที่ได้: ถ้าตัวตรวจแบบมองเฉพาะจุดไปสั่งการกระทำที่ทำลายทั้งระบบ ต้องเลือกอย่างใดอย่างหนึ่ง: จำกัดขอบเขตของการกระทำ หรือไปตรวจสอบสถานะทั้งเครื่อง ข้อห้ามที่เขียนไว้เฉยๆ อ่อนกว่าโค้ดที่เช็คว่ามีงานอื่นรันอยู่ไหมก่อนปิดเครื่อง

สูง 2026-08-20

เช่าเครื่อง Vast.ai ที่มีดิสก์แค่ 78GB เพราะปล่อยแถบเลื่อนไว้ที่ค่าเริ่มต้น — โดนกับดักนี้เป็นครั้งที่ 3

เกิดอะไรขึ้น: เครื่องแรกที่เช่ามารัน inference ของ t02 มีดิสก์แค่ 78GB ซึ่งเล็กเกินไปสำหรับงานนี้ เพราะแถบเลื่อนขนาดดิสก์ของ Vast.ai ตั้งค่าเริ่มต้นไว้ต่ำและไม่ได้เลื่อนขึ้น บันทึกรบว่าเป็นครั้งที่ 3 แล้วที่โดนกับดักเดิม

ต้นเหตุจริง: ฟอรัมเช่าเครื่องของ Vast.ai ตั้งค่าดิสก์เริ่มต้นไว้ต่ำกว่าที่งาน VLM ระดับ 30B ต้องการ และไม่มี checklist ก่อนเช่าที่บังคับให้เปลี่ยนค่านี้ ขั้นตอนที่ต้องทำด้วยมือขั้นเดิมเลยถูกลืมทุกครั้งตอนรับ

เสียหายอะไร: เสียค่าเช่าไปรอบหนึ่ง บวกเวลาที่ใช้ไปกับการหาว่าปัญหาคืออะไร ทำลายเครื่อง แล้วเช่าใหม่ รวมแล้วโดนแบบเดียวกัน 3 ครั้ง ทั้งที่เลี่ยงได้

แก้ยัง: ทั้งเครื่องนั้นไป สุดท้ายเช่าเครื่องที่สาม (ดิสก์ 158GB) แล้วใช้งานได้

กฎที่ได้: เขียนตัวเลขขั้นต่ำของ disk/VRAM/RAM ลงในเอกสาร workflow เตรียมงานเป็น checklist ที่ต้องติ๊กให้ครบก่อนกดเช่า — กับดักที่โดนซ้ำ 3 ครั้งคือ checklist ที่ขาดไป ไม่ใช่ความสะเพร่า

สูง 2026-08-20

อ่านราคา GPU ผิดเป็น \$0.058/ชม. ทั้งที่จริง \$1.792/ชม. — ผิด 30 เท่า จับได้ตอนวางแผนรันไป ~11 ชม.แล้ว

เกิดอะไรขึ้น: ราคาของเครื่องที่ใช้งานได้ถูกบันทึกว่า \$0.058/ชม. แต่ราคาจริงคือ \$1.792/ชม. ความผิดพลาด 30 เท่านี้เพิ่งสังเกตเห็นตอนประเมินว่างานจะใช้เวลาราว 11 ชั่วโมง / \$20 — คือหลังจากวางแผนทั้งหมดบนฐานต้นทุนที่ผิดไปแล้ว

ต้นเหตุจริง: อ่านตัวเลขราคาจากหน้ารายการของ Vast.ai โดยไม่เทียบกับอัตราคิดเงินจริงของเครื่องที่กำลังรันอยู่ และไม่ได้ตรวจสอบความสมเหตุสมผล ('\$0.058/ชม. สำหรับ RTX PRO 6000 96GB เป็นไปได้จริงหรือ') กับตัวเลขที่ถูกจนผิดปกติ

เสียหายอะไร: การวางแผนต้นทุนทั้งเซสชันผิดไป 30 เท่า จับได้ก่อนที่จะบานปลาย แต่ทุกการตัดสินใจเรื่องขอบเขตงานก่อนหน้านี้ที่ตั้งอยู่บนตัวเลขเศรษฐศาสตร์ที่ผิด

แก้ยังไง: แก้เป็น \$1.792/ชม. แล้วหลังจากนั้นต้องตัดขอบเขตงานลงเรื่อยๆ เมื่อเห็นต้นทุนและเวลาจริง

กฎที่ได้: อ่านอัตราค่าเช่าต่อชั่วโมงจากเครื่องที่กำลังรันอยู่ ไม่ใช่จากหน้ารายการ และตรวจสอบความสมเหตุสมผลด้วย — การ์ดเรือธง 96GB ราคาต่ำกว่า \$0.10/ชม. คือการอ่านผิด ไม่ใช่ของถูก แล้วเขียนอัตราที่ยืนยันแล้วลงในเอกสาร workflow ก่อนเริ่มงาน

สูง 2026-08-20

SSH dropped mid-nohup after a restart, so the old job was killed and the new one never started (ยัง

ไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: After a mid-run reorder, the old batch process was killed and a new one launched via nohup — but the SSH session dropped during the launch, so the new process never actually started. Nothing was running. Caught by checking pgrep afterwards rather than trusting the launch command's exit code.

ต้นเหตุจริง: nohup over a fragile SSH session gives no reliable confirmation that the process survived the disconnect; the command's exit code reports the shell's success at forking, not the job's continued existence. The kill-then-relaunch ordering left a window where zero jobs were running.

เสียหายอะไร: Silent dead time on a \$1.792/hr instance under a slipping 1am deadline — caught only because it was verified, not assumed.

แก้ยังไง: Detected via pgrep and the process relaunched. No tooling change recorded.

กฎที่ได้: Never trust a launch command's exit code on a remote box — always confirm with pgrep/ps after launching, and prefer tmux/screen over nohup so a dropped SSH cannot orphan the launch; start the new job before killing the old where possible.

สูง 2026-08-21

The real cause of the post-grammar failures: lm-format-enforcer emits trailing commas that Python's json rejects (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: After grammar-constrained decoding was added, the first run still produced unparseable JSON. The actual root cause (found only after reading the code layer by layer) was that `lm-format-enforcer` treats `{ "a":1,}` — a trailing comma before the closing brace — as valid, while Python's json parser rejects it. Two components disagreed on the definition of 'valid JSON'. Proven in one line: deleting the extra comma from a 'broken' file made it parse immediately.

ต้นเหตุจริง: A grammar backend was adopted without checking that its notion of valid JSON matches the consumer's (Python json.loads). The grammar was working correctly the whole time; the two specs of 'correct' simply differed.

เสียหายอะไร: Wasted a debugging cycle on rented GPU time, produced a confidently wrong entry in SUMMARY.md blaming Unsloth, and nearly caused the entire grammar approach to be abandoned as non-functional.

แก้ยังไง: Switched the default backend to `xgrammar` (which cannot emit a trailing comma at source) with `lmfe` demoted to fallback; recorded as rule_of_tune lesson 13.

กฎที่ได้: When a constraint layer and a parser disagree, verify both definitions of 'valid' against each other before blaming either — and prefer the backend whose grammar makes the invalid output physically impossible rather than merely unlikely.

สูง 2026-08-24

Vast.ai price misread by ~20x for the second time — web display shows storage rate, not GPU rate (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The Vast.ai web page showed \$0.056/hr; DLPerf/\$ math implied ~\$1.16/hr. The real bill for the morning instance was \$5.43 with the GPU at \$1.1067/hr. The \$0.056 figure is the *storage* rate. This was flagged as "same class as the 30x misread on 2026-08-20" — the same mistake had already been made four days earlier.

ต้นเหตุจริง: The web UI puts a storage rate in a position that reads as the instance price, and the team kept sourcing price from the UI instead of the API. The 2026-08-20 occurrence produced no rule, so it repeated.

เสียหายอะไร: Budget planning was wrong by ~20x across at least two rental decisions; the bill went unchecked for most of a day while decisions were made on a false price.

แก้ยังไง: Checked the real bill via vastai CLI (GPU \$1.1067/hr, storage 0.24, DL 0.48, total \$5.43). New standing rule: read price only from `vastai search offers` (`dph\_total`), never the web display.

กฎที่ได้: Never source a rental price from a vendor web UI. Read it from the API/CLI field that bills you (`dph\_total`), and when a pricing surprise happens once, write the rule down the same day or it will repeat.

สูง 2026-08-24

Training survived but per-epoch eval OOM'd — accelerate upcasts the whole logits tensor (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: TEST_STEPS=5 passed cleanly (3,778 visual tokens/image, seq 21,445 < 32,768, ~55s/step, loss 1.86) and then the run died at eval time on the rented GPU.

ต้นเหตุจริง: `evaluate` → `convert\_to\_fp32`: accelerate wraps forward and upcasts the entire output, materializing logits at seq 21k × vocab 152k × 4 bytes ≈ 13GB on top of 6.5GB bf16, asking for 24.72GB. Training never hits this because Unsloth uses a fused loss that never materializes logits. t01 escaped it (512px images, short seqs) and t02 escaped it (30B left headroom); t03 was the first run combining a 35B model with full-resolution images.

เสียหายอะไร: A full training run died on paid GPU time after passing every pre-flight check, requiring a restart.

แก้ยังไง: Set `eval\_strategy="no"`, kept `save\_strategy="epoch"` so three checkpoints remain for retrospective measurement.

กฎที่ได้: A training smoke test that skips the eval step is not a smoke test. Peak memory at eval can exceed training peak by 2-3x because eval materializes logits; size VRAM for the eval path, not the train path.

สูง 2026-08-25

A widely-quoted \$0.67/hr GPU price was a misread of the interruptible min_bid column — the second time this exact trap was hit (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A "\$0.67/hr RTX PRO 6000" figure in circulation was checked against vast.ai's live API and refuted — the cheapest real offer is \$1.01/hr. The \$0.67 was likely a misread of the `min\_bid` (interruptible) column. The diary notes this is the same trap as an earlier `\$0.056` storage-vs-GPU price misread.

ต้นเหตุจริง: Marketplace listings show several price columns (on-demand, interruptible min_bid, storage) and the cheapest visible number gets quoted without checking which column it belongs to. The same misread had already happened once and no rule was written to prevent it.

เสียหายอะไร: Budget planning against a price 40% below reality. Repeating a known trap means the first occurrence produced no durable lesson.

แก้ยังไง: Queried `vastai search offers` directly (not web-scraped, not an aggregator) and recorded the real on-demand rates: RTX PRO 6000 96GB \$1.01–1.11/hr, ×2 \$2.00–2.19/hr, H200 141GB \$3.29/hr.

กฎที่ได้: Always read GPU prices from the vendor's own live API and name which column the number came from (on-demand vs interruptible vs storage). When a trap repeats, write it into the rules file, not just the diary.

สูง 2026-08-25

Documented framework support did not match reality — GLM-4.6V has an open LoRA-blocking bug, InternVL3-78B fails to load in Unsloth (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Checking framework support against real GitHub issues rather than documentation claims found that GLM-4.6V has an open, unresolved LoRA-blocking bug (hiyouga/LLaMA-Factory#9635, `NotImplementedError: get\_input\_embeddings`), InternVL3-78B does not load in Unsloth (two open issues #3401, #2341), and Pixtral-Large / Molmo-72B / NVLM-D-72B have no mainstream framework confirmed working at all.

ต้นเหตุจริง: Framework documentation lists models as supported based on intent or partial support, not on a verified working fine-tune run. Trusting the docs would have produced a false candidate list.

เสียหายอะไร: Would have committed GPU rental to a model that cannot be LoRA-fine-tuned at all. Two of the surviving finalists still require either a different framework (LLaMA-Factory, not Unsloth) or a different training mode (4-bit QLoRA, not bf16 LoRA) — a real pipeline change, not a drop-in swap.

แก้ยังไง: Verified support against open GitHub issues and confirmed third-party runs (e.g. Llama-4-Scout via a Fixstars blog report on 8×H100) before shortlisting.

กฎที่ได้: Never accept a framework's model-support table as proof. Search its open issue tracker for the exact model plus your training mode, and require evidence of a real completed run before committing rental money.

สูง 2026-08-29

A wrong --cv-dir path silently degrades arm 2.4a into arm 2, invalidating the experiment and burning GPU time (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `apply_arm()` looks for a sidecar hint file; if it cannot find one it returns the original prompt with `hint_used: false` rather than erroring. A mistyped `--cv-dir` therefore produces a full run where the treatment arm is silently identical to the control arm.

ต้นเหตุจริง: The no-hint-found path is intentionally non-fatal (correct for houses genuinely lacking CV data) but there is no distinction between 'legitimately absent' and 'path is wrong'. The failure surfaces only as a flag buried in the results.

เสียหายอะไร: An entire experimental run can produce results that cannot answer the question it was run for, consuming GPU rental for nothing.

แก้ยังไง: Documented as a mandatory pre-flight check: verify `hint_used` is true for pages that have sidecars before continuing the run.

กฎที่ได้: Before any paid run, assert that the experimental treatment is actually being applied — check the flag on a sample row first. Silent degradation to control is worse than a crash because you pay full price for a null result.

สูง 2026-08-30

`hf upload <repo> <folder> --exclude 'pattern'` silently uploaded zero files while logging success-looking output (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Uploading the fold2 adapter with `--exclude 'checkpoint-\*/\*'` produced logs reading 'Start hashing 19 files / Processing Files (0/0) / New Data Upload 0.00B/0.00B' even though all files existed. Not a token or permission problem — `whoami` was fine and a single-file upload test succeeded.

ต้นเหตุจริง: Not fully determined; suspected interaction between `--exclude` and the large-folder incremental sync logic in the HF CLI. Vendor tool reports apparent success while transferring nothing.

เสียหายอะไร: A backup that appeared to have happened had not. If not caught, an adapter would have been believed safe on HF and the GPU destroyed, losing the weights.

แก้ยังง: Workaround that works 100%: copy only the top-level files (excluding `checkpoint-\*/\*`) into a separate staging folder and upload that folder without using `--exclude` at all. 842MB uploaded successfully. Recorded so the same trap is not hit on fold3/fold4.

กฎที่ได้: Never trust an upload tool's exit status — verify the uploaded byte count / file list on the remote side (and sha256 after pulling back) before deleting or destroying the source.

สูง 2026-08-30

Image tarball uploaded only half (514MB/1005MB), tar extract failed silently, and the training data ended up with 0 images (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On the fold1 instance, the images tarball transferred only 514MB of 1005MB. `tar` extraction then failed silently, leaving 0 image files, even though the logs of the preceding step looked normal.

ต้นเหตุจริง: No integrity check between upload and extraction — the pipeline assumed a transfer that returned without an error had transferred the whole file, and tar's failure on a truncated archive was not surfaced.

เสียหายอะไร: A fold would have trained on a dataset with no images at all if not caught. Required re-uploading and restarting that fold's setup.

แก้ยังง: Re-uploaded, and added a mandatory `stat` size comparison between local and remote before every extract. The pattern was applied to fold2/fold3 from the start and both passed first try.

กฎที่ได้: Always compare file size (or checksum) local vs remote before extracting or consuming a transferred archive; a transfer command exiting 0 is not evidence the bytes arrived.

สูง 2026-08-30

Scaled to 4 parallel GPUs with \$49.75 credit against a \$63 job — instances would have been stopped mid-training (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: After expanding from 2 to 4 folds and renting 2 more cards, total projected cost was ~\$63 while the account held \$49.75. The four instances burned \$4.29/hr, leaving roughly 10 hours before the credit ran out mid-run.

ต้นเหตุจริง: The rental scale-up decision was executed before checking that the credit balance covered the full projected runtime; cost was computed after committing rather than as a gate.

เสียหายอะไร: Risk of all four training runs being killed partway through, losing up to ~14 hours of GPU work across 4 cards. Averted only because Makham topped up (\$76.63) in time.

แก้ยังไง: Reported to Makham immediately with the deadline; credit topped up manually and auto top-up enabled as a second layer.

กฎที่ได้: Before starting any long unattended GPU run, compute $\text{total_hours} \times \text{hourly_burn}$ and refuse to start unless credit covers it plus a buffer — treat the balance check as a hard gate, not a follow-up.

สูง 2026-08-31

fold1 still OOM'd at step 68/252 even at the reduced MAX_PIXELS that the other three folds survived (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: After the uniform reduction to 6912, three folds completed but fold1 died at step 68 of 252, leaving only checkpoint-50 on HuggingFace.

ต้นเหตุจริง: Per-fold sample composition differs, so a globally-set memory ceiling is not equally safe on every fold — the margin left was still too thin for fold1's longest sequences.

เสียหายอะไร: Only 3 of 4 folds available; the model soup had to be merged at $k=3$ instead of $k=4$.

แก้ยังไง: Decided not to rerun (same settings would likely OOM again, and lowering only fold1 would break cross-fold parity); merged $k=3$ with equal $1/3$ weights, noting all 40 houses are still covered.

กฎที่ได้: Size memory headroom against the worst fold, not the average, and if one fold must differ, accept losing it rather than breaking cross-fold parity.

สูง 2026-08-31

Hours of inference were run on gpu1, which was ~20x slower than needed because its torch build did not match the image's CUDA (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: smoke_destrier and the whole pure-power queue ran on gpu1 at ~20 minutes per page, forcing a 25-minute timeout rule, an xgrammar retry system, and cutting the test from 65 pages to 8. Only when GPU-B was rented later did it become clear that a machine with torch 2.11+cu128 matching CUDA 12.8 runs about 20x faster.

ต้นเหตุจริง: gpu1 carried torch cu13.0 on a CUDA 12.8 image (the same mismatch that blocked flash-linear-attention), so everything fell back to slow torch paths — and nobody compared its throughput against a known-good machine before committing a long queue to it.

เสียหายอะไร: Most of a day of inference at 20x cost in wall-clock time, a whole timeout/retry subsystem written to work around it, and the pure-power test reduced from 65 pages to 8.

แก้ยังไง: GPU-B's working config (torch 2.11+cu128 on CUDA 12.8) saved to results/gpuB_working_config/ as the rental recipe for next time

กฎที่ได้: Benchmark one page on a new rental against a known-good baseline before queueing hours of work onto it; a 20x slowdown reads as "the model is slow" instead of "this box is misconfigured".

สูง 2026-09-01

Eight real bugs sat undetected in the Constistant app until a whole-house run was simulated without a GPU (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On 1 Sep the team simulated running a whole house through the Constistant application without renting a GPU, and immediately found 8 real bugs, all of which were then fixed.

ต้นเหตุจริง: The application side had never been exercised end-to-end on a full house — testing had been per-module, so integration-level defects accumulated invisibly until the first whole-pipeline dry run.

เสียหายอะไร: Eight latent bugs would have surfaced during an expensive live GPU run instead, wasting rental time on debugging application code rather than on model work.

แก้ยังไง: All 8 fixed the same day; details recorded in that day's diary.

กฎที่ได้: Run the full pipeline end-to-end in cheap simulation mode (no GPU, no rental) before every expensive live run. Integration bugs are invisible to per-module tests and are far cheaper to find on a laptop than on a rented card.

สูง 2026-09-01

Job sat pending 7+ minutes with zero GPU activity because the SSH tunnel was never actually started (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A job submitted from the web stayed in `pending` for over 7 minutes with no GPU activity and no error message at all. Investigation showed `presentation.py up` had never actually opened the SSH tunnel, so `localhost:8000` was unreachable and `worker.py` had nowhere to send requests.

ต้นเหตุจริง: `presentation.py up` reported success without verifying the tunnel was actually listening. A silent partial-startup: the orchestration command returned OK while its most critical side effect (the tunnel) had not happened, and nothing downstream checked reachability before accepting jobs.

เสียหายอะไร: 7+ minutes of a paid GPU rental burned on a job doing nothing, plus debugging time during a live demo session; the job had to be manually reset to `pending` and `worker.py` restarted.

แก้ยังไง: Tunnel started manually, orphaned job reset to `pending`, `worker.py` restarted; GPU activity then confirmed. Later hardened with `ServerAliveCountMax=6` and `ExitOnForwardFailure=yes` on the tunnel.

กฎที่ได้: A startup command must verify its critical side effect before reporting success — `presentation.py up` should probe `localhost:8000` and fail loudly if the tunnel is not listening. Never let a job enter `pending` when its transport is unproven.

สูง 2026-09-01

pass3 shipped with only a synthetic self-check whose answers were known in advance (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: pass3 (`pass3\_measure.py`) was written with a self-check using a fabricated 50 px/m grid whose correct answer was already known. No real drawing had ever been run through it — real px/m and residual figures were completely unseen. The entry itself flagged this as the top pending item.

ต้นเหตุจริง: The self-check validated the arithmetic against data the test itself constructed, so it could only prove 'the fit function fits a perfect fit'. It could not detect whether real drawings even produce enough valid anchors — which turned out to be the actual failure mode (see the cv_mark and count>1 bugs).

เสียหายอะไร: A whole measurement stage was believed working while its dominant real-world failure modes (0 anchors, wrong scale, multi-ref elements) were entirely untested — and three separate real bugs were later found in it.

แก้ยังไง: Closed later the same day with `validate\_pass3\_real.py` using real human-verified pairs and a leave-one-out methodology: 191/191 correct, fit succeeding on 15/18 pages, error 0.004–0.093 m.

กฎที่ได้: A self-check on synthetic data you generated proves the code runs, not that it works. Before claiming a measurement stage is done, run it on real artifacts already sitting on disk — here the real data was available locally and required no GPU at all.

สูง 2026-09-01

worker.py had no retry anywhere, so a momentary network blip killed operations mid-job (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `update\_job`, `download\_image` and `call\_purson` all had no retry logic. Any transient network failure during a 75–80 minute GPU job broke the operation outright.

ต้นเหตุจริง: The code was written assuming a stable local network, but it runs against a rented remote GPU over an SSH tunnel — an inherently flaky path, as the HuggingFace stalls the next day proved.

เสียหายอะไร: Any network hiccup wasted the remaining GPU time on a job that could no longer report or fetch.

แก้ยังง: Added `\_retry()` with 5/15/30/60s backoff wrapping `update\_job`, `download\_image` and `call\_purson`.

กฎที่ได้: Every network call that crosses a rented-machine boundary needs backoff retry from the first version. The cost of a dropped call is measured in GPU-minutes, not milliseconds.

สูง 2026-09-01

Naively retrying on ReadTimeout would have made the GPU run two generations at once (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Adding retry to `call\_purson` created a new hazard: retrying on `ReadTimeout` means the model is still generating while a second identical request is sent, so the GPU does two jobs simultaneously.

ต้นเหตุจริง: 'Retry on failure' is not safe for non-idempotent long-running operations. A read timeout says nothing about whether the server is still working — it only says the client stopped waiting.

เสียหายอะไร: Would have halved effective GPU throughput and doubled cost per page. Caught during design of the retry itself.

แก้ยังง: ReadTimeout is explicitly not retried; only connection failures and 5xx are.

กฎที่ได้: Classify each error before retrying: connection-refused and 5xx are safe to retry, read-timeout on a long generation is not. Never apply a blanket retry to a non-idempotent expensive call.

สูง 2026-09-01

No automatic tunnel reconnection — a fully dead tunnel requires a human to retype a command (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: If the tunnel dies completely, nothing restarts it; a person must run `python presentation.py tunnel` by hand. The worker will retry and wait, but the job stalls until a human notices.

ต้นเหตุจริง: Retry was added to the client calls but not to the transport itself, so the resilience layer stops one level short of the actual failure point.

เสียหายอะไร: An unattended overnight run can stall indefinitely on a dead tunnel while GPU billing continues.

แก้ยังง: Not fixed — recorded as pending.

กฎที่ได้: Resilience must extend to the transport, not just the calls over it. If a run can be unattended, every layer it depends on needs self-recovery — otherwise the GPU bills while nothing happens.

สูง 2026-09-01

The full pass0→pass3 pipeline had never once been run end-to-end against a real GPU (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: After a full day of simulation work proving every stage, the entry states plainly that the whole belt has never been run with a real GPU — everything proven was 'all stages except the model itself'.

ต้นเหตุจริง: GPU cost pressure pushed all verification into CPU simulation. Valuable, but it structurally cannot catch model-interaction faults (timeouts, truncation, malformed JSON) — which is exactly what broke on the very next day.

เสียหายอะไร: The first real GPU run the next day immediately hit three broken pages from server-side timeouts — a failure class no simulation could have surfaced.

แก้ยังไง: Not fixed that day; the real run happened 2026-09-02 and immediately found new bugs.

กฎที่ได้: Simulation proves your code, never the model interaction. Budget at least one cheap full end-to-end GPU run per major pipeline change — the failures unique to that boundary (timeouts, truncation, malformed output) are invisible to every mock.

สูง 2026-09-02

No checkpointing meant any failure forced restarting an 80-minute GPU job from zero (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Until this change, a job that failed part-way had to be restarted from the beginning — re-running every model call already completed. This is what made the page-12 failure cost a full house re-run.

ต้นเหตุจริง: Results were written only at job completion, so all intermediate work was discarded on any failure. On a 75–90 minute paid job this makes any single-page failure catastrophic.

เสียหายอะไร: Full house re-runs on paid GPUs for single-page failures (directly caused the job 43323920 re-run).

แก้ยังไง: `save\_checkpoint()` writes `result` after pass0/gridline/each page of pass2/pass2.5; `claim\_next\_job()` also fetches `result` so `run\_house\_extract` skips any pass already checkpointed — real resume, not just saving. Web side remembers the last job id per project and offers a '🔄 ทำต่อ' button that PATCHes the existing job back to pending rather than creating a new one.

กฎที่ได้: Any paid long-running job must checkpoint after every expensive unit of work, and resume must actually skip completed units — 'we save results' is not resume.

สูง 2026-09-02

Renting a GPU took 7 attempts — 2 stuck loading, 3 SSH key propagation lag (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Getting a working rented GPU for the house09 run required 7 attempts: 2 instances stuck loading, 3 rejected the SSH key with 'Permission denied (publickey)' despite the key being correct (propagation lag).

ต้นเหตุจริง: vast.ai provisions the instance before the SSH key has propagated, so the first connection attempt fails on a perfectly good host. The team's script connected immediately and treated the failure as terminal, so every lag event was misdiagnosed as a bad host and answered with destroy-and-re-rent.

เสียหายอะไร: Five wasted rentals and a large amount of setup time; each destroy/re-rent cycle costs both money and minutes. This was a repeat of the same symptom seen on 2026-09-01.

แก้ยังไง: Root-caused as propagation lag and fixed in `presentation.py` with 6 retries over 60s before the first mkdir/scp, instead of destroying and re-renting every time.

กฎที่ได้: Distinguish 'host is bad' from 'host is not ready yet'. Give a freshly provisioned instance a bounded retry window (60s) before destroying it — destroy-and-re-rent as a first response to a publickey error burns money on healthy hosts.

สูง 2026-09-02

HuggingFace download stalled with 4 repeated timeouts while pulling a 70GB model onto a \$2.05/hr rental (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On a successfully started \$2.05/hr instance, the HuggingFace network stalled badly — 4 repeated timeouts on different files while downloading the 70GB model. Makhm stopped mid-way rather than continue paying for a bad connection and asked to pick the machine himself.

ต้นเหตุจริง: Model weights are pulled from HuggingFace onto every fresh rental, so the whole setup depends on a third-party CDN's throughput to that specific datacenter. No local cache, mirror, or pre-baked image exists.

เสียหายอะไร: Paid rental time burned entirely on a failed download; the rental was abandoned with no work done.

แก้ยังไง: Rental abandoned. The incident directly produced the 'attach to a card you rented yourself' feature (GO.bat menu item 7 / `presentation.py attach <id>`).

กฎที่ได้: Downloading 70GB from a third party on every rental makes cost hostage to their CDN. Bake the weights into a snapshot/image or mirror them to storage in the same region — and abandon a rental early once the download is clearly degraded, rather than paying while retrying.

Job reported 'success 9/9' while three of its output pages contained invalid JSON (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Job f63bc71e finished and reported ความสำเร็จ 9/9, but three pages (page_03_notes, page_12_plan_footing, page_13_notes) had corrupt/truncated JSON. Nothing between generation and the 'done' status validated that each page's output actually parsed.

ต้นเหตุจริง: The worker's completion status counts pages generated, not pages that produced valid parseable JSON — the server-side generation timeout cuts mid-token and the worker still marks the page done.

เสียหายอะไร: A green 9/9 status that a user would trust; the corruption was only found by opening raw text by hand, and three pages of an ~87-minute paid GPU run were silently unusable.

แก้ยัง: Found by manual inspection, then recovered with a purpose-written `salvage\_json.py` bracket-depth walker; no validity gate was added to the worker itself

กฎที่ได้: A pipeline that emits JSON must `json.loads()` every page before it may report success — 'the model returned bytes' is not the same as 'the page is usable', and a false green is worse than a loud failure.

เรื่องที่เหลือในหมวดนี้ (63)

ไฟล์ -all.json ที่ไว้อ้างอิงอย่างเดียว กับไฟล์ config.xml ถูกอัปโหลดเข้าแท็บ Data Import ของ Label Studio

กลาง 2026-07-07

ไฟล์ที่ห้าม import เด็ดขาด ต้องไม่ใช่ชื่อขึ้นต้นเหมือนหรืออยู่โฟลเดอร์เดียวกับไฟล์ที่ import ได้ ให้ตั้งชื่อแบบ *-REFERENCE-DO-NOT-IMPORT หรือแยกไปอยู่โฟลเดอร์ของมันเอง

การรวม 3 โปรเจกต์ Label Studio เป็นอันเดียวด้วย visibleWhen จะพังเพราะบั๊กของตัวเครื่องมือที่รู้กันอยู่แล้ว

กลาง 2026-07-09

ค้น issue tracker ของผู้พัฒนาก่อนจะออกแบบระบบทับบนพีเจอร์แสดงผลแบบมีเงื่อนไข และให้เลือก config ตรงไปตรงมา 3 ชุด ดีกว่า config ฉลาดๆ ชุดเดียวที่แสดงผลผิดผิดได้

ข้อความจั่วที่หมุน 90 องศาบนหน้า BOQ สก๊อตออกมาได้ค่าความมั่นใจต่ำ แล้วปลอยผ่านไปโดยคิดว่าต้องมาตรวจ

กลาง 2026-07-12

หน้าไหนมีข้อความหมุนหรือตัวเล็กมาก ให้หมุนและซูมเป็นรอบเฉพาะก่อนสก๊อตข้อมูล อย่ายอมรับการจับคู่ที่ค่าความมั่นใจต่ำแล้วเลื่อนไว้อีก เพราะธงที่เลื่อนไว้จะกองสะสมและไม่มีใครมาเคลียร์

สคริปต์สร้าง task ของ Label Studio ทั้งไฟล์ที่รูปทรง JSON ไม่รองรับไปเสียๆ

กลาง 2026-07-12

ตัวแปลงข้อมูลที่ข้ามอะไรไป ต้องบันทึกว่าข้ามอะไรและเพราะอะไร จำนวนที่ไม่ตรงกันแบบเสียๆ คือบั๊กข้อมูลหาย และให้ตัดสินตั้งแต่ครั้งแรกที่เจอว่าจะแก้ที่ schema หรือแก้ที่สคริปต์ อย่าลากช่องว่างนี้ไปที่ละบ้านๆ

Git push rejected with 403 because the account lacked collaborator access on the org repo

กลาง 2026-07-20

Verify push access on day one of a repo, not at the first push. A permissions gap turns 'commit the work' from a five-minute task into a blocked one.

Repeated 'Jupyter connection refused' failures chasing an access method the setup never supported

กลาง 2026-07-20

Verify the access path is actually provisioned before debugging connection failures — and prefer SSH/scp/tmux, the method that works everywhere, over a notebook server that has to be stood up.

Days lost fighting Jupyter web UI 'connection refused' before switching to SSH

กลาง 2026-07-21

Before renting infrastructure, read the vendor's own docs and adopt the workflow that serious practitioners use (SSH/tmux for GPU rental); do not burn hours proving a convenience UI unusable.

Baseline eval re-launched itself and burned GPU time unnoticed

กลาง 2026-07-21

After any long/expensive job reports completion, check ``ps`/`nvidia-smi`` that it has not restarted itself before walking away.

Vast.ai's rent form defaults disk to 150GB — the exact value that caused disk-full failures

กลาง 2026-07-24

Every provider default on a rent/provision form is a decision you are making by omission — write the required non-default values into the pre-rental checklist and verify them on the confirmation screen.

Vast.ai's disk slider silently defaults to 150GB and must be dragged to 300 by hand

กลาง 2026-07-24

Record vendor UI defaults that must be overridden into the pre-rental checklist. A default that silently undersizes the machine costs a re-rent, not just a resize.

Export sanity check raised its token limit three times before the check itself was recognised as wrong

กลาง 2026-07-24

After the second time raising a limit fails, stop raising it and question the assertion instead. A sanity check should prove the smallest sufficient thing, not reproduce a full production output.

Export reported 'file not found' although conversion and quantization had genuinely succeeded

กลาง 2026-07-24

Never assume a library's output path — list the directory or use the returned path. And when an expensive step reports failure, check whether the artifact exists before re-running it.

The GPU selected in the Phase 0.5 pre-flight was not the one rented, and the first machine actually rented was switched away from

กลาง 2026-07-24

Pre-flight should fix the machine *criteria* (VRAM, disk, reliability, max \$/hr), not a specific listing id. Selecting an id hours in advance guarantees it is stale and hides better prices.

The exact RAM-exhaustion mistake from Phase 10 was repeated the next day at a higher ctx-size

กลาง 2026-07-25

When raising a resource parameter to fix a different problem, re-check the constraint that parameter was originally tuned against. Handle truncation by re-running the one affected page at higher ctx, not by raising it globally.

House 07 SN-04: inspection chambers counted as 8 when there are 11

กลาง 2026-07-28

Every count must come from an exhaustive zoomed sweep with a recorded per-region tally, not a single visual pass over the whole sheet.

Running the 21GB model locally made the working machine unusable for anything else, forcing the run to be stopped after 2 of 61 pages

กลาง 2026-07-28

Estimate duration and resource footprint before starting a long GPU job, and never run one on a machine someone needs interactively. Make every long batch job resumable from the start.

Bug 5: the training script's epilogue hangs while still holding VRAM

กลาง 2026-07-28

Any script that runs on metered hardware must have every optional convenience step behind an off switch, and must release resources before it can possibly hang.

A GPU load was stacked on top of a job already running — recorded the same day as permanent Lesson 11

กลาง 2026-07-28

Two things: never start a second GPU workload against a card already running a real job; and when a failure is serious enough to become a permanent rule, write down the incident, not only the rule.

GPU showed 100% utilisation while drawing only 33.7W — the card was waiting, not working

กลาง 2026-07-29

Never diagnose a bottleneck from utilisation percentage alone. Cross-check power draw and memory residency; 100% util at low wattage means the device is starved, and the fix is upstream of it.

HuggingFace token constistant-tune_t02 still live after the job it was issued for ended

กลาง 2026-07-29

Issue job-scoped credentials with an expiry that outlives the job by hours, not with unlimited life plus an intention to revoke. If the platform supports no expiry, revocation goes in the same automation that tears down the instance.

เครื่องเช่า Vast.ai เครื่องที่สอง SSH ต่อไม่ติดเลย

กลาง 2026-08-20

เพื่อไว้เลยว่าแต่ละ session จะเจอเครื่องเช่าเสียหนึ่งถึงสองเครื่อง และให้เช็ค SSH กับ nvidia-smi ภายในสองนาที่แรกของการเช่าทุกครั้ง ถ้าประกาศเดิมให้เครื่องเสียสองครั้งติด ให้เปลี่ยนประกาศ ไม่ใช่กดเข้าซ้ำที่เดิม

t02 ไม่มี GGUF ที่ใช้ได้เพราะ export สัมเหลว ทำให้ต้องรัน inference ด้วย transformers/PEFT ดิบๆ บน GPU เช่า

กลาง 2026-08-20

การ export ที่สัมเหลวไม่ใช่เรื่องเล็ก มันเปลี่ยนต้นทุนของการทดลองทุกครั้งต่อนั้น ให้แก้ export ให้ได้ ไม่งั้นก็ต้องเขียนในแผนตรงๆ ว่าการทดสอบทุกครั้งต่อนี้คือค่าเช่า GPU ที่ต้องจ่ายจริง

รอบก่อนหน้า OOM ทั้งที่ยังเหลือ VRAM วางอยู่ 470 MB

กลาง 2026-08-25

รัน smoke run 5 step แล้วอ่านค่า peak VRAM จริงก่อนเริ่มงานเทรนเต็มรูปแบบที่เสียเงินเสมอ โดยเฉพาะหลังจากแก้พารามิเตอร์อะไรก็ตามที่กระทบหน่วยความจำ

ต้องลดขนาด LoRA ลงครึ่งหนึ่งกลางแผน เพื่อให้ MAX_LENGTH 32,768 พอดีกับการ์ด 95GB และก็ยังพอดีแบบเนียนฉิว

กลาง 2026-08-25

คำนวณงบ VRAM แยกเป็นส่วนๆ (น้ำหนักคงที่ vs สถานะที่เทรน vs activation) ก่อนเลือกความยาว context กับ rank ของ LoRA ไปพร้อมกัน ไม่ใช่ไปหั่นอะไรที่หั่นง่ายที่สุดตอนที่รันแล้วมันไม่ยอมเริ่ม

เหลือเครดิตแค่ \$8.91 ไม่พอจบแม้แต่รอบเดียวบนเส้นทางอัปเกรดไป H200

กลาง 2026-08-25

เช็คยอดเครดิตเทียบกับต้นทุนรวมที่คาดว่าจะใช้ทั้งรอบ ก่อนกดเริ่ม งานที่ตายตอน 60% เพราะเครดิตหมด เสียเงินเท่ากับงานที่ไม่เคยเริ่ม แต่ได้เงินคืนน้อยกว่า

Pixtral-Large evaluated in depth for capability, then disqualified on licence in one line

กลาง 2026-08-25

Screen model licences BEFORE evaluating capability. Licence is a binary gate; capability is a ranking. Never rank candidates that have not passed the gate.

MiniMax M3 carries the same hidden commercial-use licence trap as Pixtral-Large, and it was left unresolved

กลาง 2026-08-25

After a licence trap bites once, close every candidate carrying the same shape of trap in the same session — leaving one 'to read later' is how the same hour gets spent twice.

The PRINTED_ID classifier is a regex heuristic that misclassifies genuinely-printed Thai labels

กลาง 2026-08-25

When a heuristic drives a published number or a data edit, state its error direction in the same sentence as the number, and make the edit reversible per record so the heuristic's misses can be recovered.

The cheapest big-VRAM offer (8× RTX 4090, 384GB at \$2.99/hr) would have been a bad buy for an MoE model — no NVLink

กลาง 2026-08-25

For MoE, read interconnect bandwidth before \$/hr and total VRAM — the cheapest big-VRAM offer is often the one whose cards cannot talk to each other.

ตั้ง auto top-up ของ Vast.ai ไว้ที่ `< \$0.00` — มันเติมเงินหลังยอดหมดแล้ว ไม่ใช่ก่อน

กลาง 2026-08-30

ตั้ง threshold เติมเงินอัตโนมัติให้สูงกว่าศูนย์อย่างน้อยเท่ากับค่าใช้จ่ายหลายชั่วโมง (เช่น อัตราเหมาเงิน × 4 ชม.) และห้ามคิดว่า การเติมที่ threshold ศูนย์จะปกป้องงานที่รันทิ้งไว้ได้

เช่าเครื่องที่แสดงสัญญาณผิดปกติตั้งแต่ตอนสร้าง — 26 นาทีต่อมามันหลุดออฟไลน์

กลาง 2026-08-30

ถ้าเครื่องเช่ามีอะไรผิดปกติตั้งแต่ตอน provision (ไม่มี direct port, เน็ตช้า, คะแนนความน่าเชื่อถือต่ำ) ให้ทำลายแล้วเช่าใหม่ทันที ค่าเช่าใหม่ถูกกว่าการที่เครื่องพังตอนรันไปแล้ว 10 ชั่วโมงเยอะ

`vastai destroy instance` ถาม y/N แบบโต้ตอบ พอร์นในสคริปต์มันเลิกไปเสียๆ — การดออาจยังเปิดอยู่ทั้งที่คิดว่าคืนไปแล้ว

กลาง 2026-08-30

หลังสั่งทำลาย/รี้อทุกครั้ง ให้ถามฝั่งผู้ให้บริการซ้ำว่าเหลือ instance กี่ตัว และต้องเป็นศูนย์เท่านั้น ห้ามใช้ 'คำสั่งรันแล้วไม่ error' เป็นหลักฐานว่าทรัพยากรถูกคืนแล้ว

การ evaluate เติมรูปแบบใช้เวลา 7-8 ชั่วโมงต่อ fold — พอๆ กับเวลาเทรน แต่ไม่ได้ตั้งบไว้

กลาง 2026-08-30

ตั้งบ GPU-hours ของขั้นตอน evaluate รวมไว้ในประมาณการเดียวกับการเทรน วัดเวลาต่อตัวอย่างตอน smoke test แล้วคูณออกมาก่อนตัดสินใจรัน eval เติม

vLLM รองรับ MoE LoRA จริง แต่ `is\_3d\_lora\_weight` ประกาศผิดได้ แล้วมันจะพ่นผลขยะออกมาเจียบๆ โดยไม่มี error

กลาง 2026-08-30

ทุกครั้งที่เปลี่ยน inference stack ให้รันหน้าที่รู้คำตอบอยู่แล้วเป็น regression แล้ว diff กับผลอ้างอิงก่อนเชื่อ — ในงาน ML serving 'ไม่มี error' ไม่ใช่หลักฐานว่าถูกต้อง

อิมเมจของ vast.ai ไม่มี `/workspace` (redirect log ตอน onstart เลยลัมเจียบๆ) และ torch อยู่ใน `/venv/main` ไม่ใช่ python3 ของระบบ

กลาง 2026-08-30

ห้าม redirect log ตอน bootstrap ลงโพลเดอร์ที่ยังไม่ได้สร้างในสคริปต์เดียวกัน และให้ระบุ path ของ interpreter ให้ชัดเจน (which python, path ของ venv) แทนที่จะเดาว่า `python3` มี dependency ครบ

อิมเมจ vastai/pytorch สร้าง authorized_keys ด้วย permission ผิด ทำให้ SSH เข้าไม่ได้เลย

กลาง 2026-08-31

เลือกใช้อิมเมจตัวเดียวที่พิสูจน์แล้วว่า SSH เข้าได้ อ่าน log ของ vastai เพื่อดู error จริงของ sshd แทนการเดา และถ้าเครื่องใหม่เข้าไม่ได้ ให้ทำลายแล้วเช่าใหม่ อย่าเปลี่ยนเป็นโปรเจกต์ซ่อมเครื่อง

27 steps of fold1 training had to be thrown away when MAX_PIXELS changed mid-round

กลาง 2026-08-31

Validate resolution and memory settings before the first real step — any mid-run change to a data-shaping parameter invalidates everything trained before it.

flash-linear-attention could not be installed because CUDA toolkit 12.8 clashed with torch cu13.0

กลาง 2026-08-31

Match the torch CUDA build to the image's CUDA toolkit at rental time — a mismatched pairing silently costs an order of magnitude in speed, not just optional features.

A watcher process lost its SSH connection mid-operation, leaving the old queue running work that had been cancelled

กลาง 2026-08-31

Never run a multi-step kill-and-relaunch over a single SSH session; issue one command at a time, and re-check real machine state (ps) before believing any log.

A GPU was rented after Makham had said to stop

กลาง 2026-08-31

On a stop instruction, cancel queued and in-flight provisioning steps explicitly and confirm zero instances, rather than assuming nothing further will start.

Rented vast.ai instance rejected a correct SSH key for minutes, forcing destroy-and-re-rent

กลาง 2026-09-01

When a freshly rented host misbehaves and your own config verifies clean, destroy and re-rent immediately — do not spend more than a couple of minutes debugging someone else's provisioning. Verify the fingerprint first so you know it is theirs, not yours.

A single failed status poll aborted the whole web flow while the job was still running fine

กลาง 2026-09-01

Polling errors and job errors are different things. A status-check failure must never terminate the work it is observing.

SSH tunnel had no keepalive or forward-failure detection, so it could die silently

กลาง 2026-09-01

Any SSH tunnel that a pipeline depends on must run with keepalive and ``ExitOnForwardFailure=yes`` — a tunnel that fails silently is worse than one that never started.

All network-resilience work was verified only by simulated tests, never against a real network drop on a live GPU run

กลาง 2026-09-01

Resilience code proven only by mocks is a hypothesis. Schedule one deliberate failure-injection run (pull the network for 60s mid-job) on a cheap rental before relying on it.

Checkpoint/resume shipped with unit tests only, never exercised against a real failed GPU job

กลาง 2026-09-02

A recovery path proven only by unit tests is unproven. Deliberately kill a job mid-run once on a cheap instance and verify resume actually skips the completed passes.

A self-selected vast.ai offer accepted the rental then reported resources unavailable and queued the state change

กลาง 2026-09-02

Treat 'state change queued' as a dead host, not a delay — abandon immediately. A suspiciously cheap offer that will not start is the vendor's problem, and waiting on it costs more than re-renting.

Drag-resizable columns were requested but Label Studio's config language has no splitter

ต้า 2026-07-09

Before promising a UI capability inside a third-party config DSL, confirm the tag exists in that DSL — seeing the behaviour in the vendor's own chrome proves nothing about what your config can do.

Initial \$9.49 GPU budget (~9 hours) ran out mid-session, needing a \$10 top-up

ต้า 2026-07-21

Budget a rented GPU at 2–3× the estimated job time for a first run, and set a balance alert — an instance that dies for lack of credit mid-training loses everything on it, the same way a destroy does.

Repeatedly retried Jupyter on the rented GPU before switching to SSH

ต้า 2026-07-21

For any rented-GPU session, connect by SSH + tmux from the first minute; treat notebook UIs as a debugging convenience, never as the control channel for a multi-hour job.

A HuggingFace 404 was briefly taken as an upload failure when the repo was simply private

ต้า 2026-07-24

Verify remote artifacts through the authenticated API, not an unauthenticated browser tab. A 404 on a private repo means 'not logged in' far more often than 'not uploaded'.

House 07 A-30/31: headboard placed on the wrong wall and a sofa recorded as a bed in the dressing room

ต้า 2026-07-28

Resolve every symbol against the legend and its containing room before recording it, and state orientation relative to a named datum (grid line or compass), never 'the wall it looks like it's on'.

The two diaries disagree on what t01's machine cost — one of the recorded prices must be wrong

ต้า 2026-07-28

Record the machine ID, hourly rate and final bill of the machine actually rented, not the one selected during planning. Cost comparisons across rounds are worthless otherwise.

Assumed similar model size means similar training time — t02 took 2.9x longer than t01 with identical hyperparameters

📅 2026-07-28

Never budget rental hours from parameter count — measure throughput on a short test run of the actual model before committing to a full run.

Phase 11 (destroy the rented instance) still not executed — an open billing exposure

📅 2026-07-29

Pair the 'never destroy without confirmation' rule with a 'confirm backup and decide within N hours' rule — an unbounded hold on a metered resource converts a safety rule into a money leak.

Deployed-site login failed with 'Invalid login credentials' — cause could not be distinguished

📅 2026-08-20

When an auth error is deliberately ambiguous by design, do not debug the deployment — verify account existence directly at the provider first, and note that demo paths that need no login are the faster way to unblock a demo.

Vendor-side breakage: GLM-4.6V cannot be LoRA-trained, NVLM-D-72B cannot even run on vLLM

📅 2026-08-25

For any candidate model, verify a working train path and a working serve path exist today before counting it as a candidate — and check the vendor's own issue tracker, not just the model card.

Used `--max\_steps 5` for a LLaMA-Factory CLI override; the parser breaks — it wants `key=value` with no dashes

📅 2026-08-30

Check a framework's own example invocation for CLI override syntax before running it on a billing machine — one failed launch on a rented GPU is pure waste.

Plan assumed two separate rentals (cheap card for dry-run + big card for training); measurement showed one 96GB card covered both

📅 2026-08-30

Measure actual peak VRAM in a 5-step dry-run before planning multi-GPU or multi-rental strategies — estimated requirements routinely over-provision.

Starting the server before onstart's pip install finished would fail the unsloth import and leave a health check that never goes green

📅 2026-08-30

Never start a service that depends on a provisioning script until you have positively confirmed provisioning finished — poll for the installer process to exit, don't sleep and hope.

HF token copied onto the fold3 card was never verified before that host went offline — the whole copy had to be redone on the replacement

📅 2026-08-30

Verify immediately inside the same operation, not as a follow-up — and do not invest setup work in a host that showed an abnormal signal at provision until it has proven stable.

A rented instance's SSH proxy could not bind its forwarding port

📅 2026-08-31

Rent, connect, and verify SSH end to end before uploading anything — a provider-side networking fault is cheapest to discover while the machine is still empty.

gpu3 failed to start with CUDA out of memory right after tmux kill-server, despite an empty GPU

๓๓ 2026-08-31

After killing a GPU process, poll nvidia-smi until the memory is actually released before launching the replacement, rather than launching immediately.

A 90-minute auto-shutdown was armed at session start and had to be cancelled mid-session; a stale timer from a previous run was already pending

๓๓ 2026-09-01

An armed auto-shutdown must be disarmed by the thing that finishes the work, not by whoever happens to notice the clock — and a session should verify no stale timer is pending before it starts anything long.

กับดักของเครื่องมือ และ พฤติกรรมของบริการภายนอก

เครื่องมือหรือบริการภายนอกที่ทำงานไม่เหมือนที่เอกสาร/ชื่อ/ผลลัพธ์บอกไว้

ทั้งหมด 58 เรื่อง · วิกฤต 2 · สูง 14 · กลาง 32 · ต่ำ 10

เรื่องที่ต้องอ่านให้ครบ (16)

วิกฤต 2026-07-24

แตกไฟล์ mmproj ไม่สำเร็จเพราะเครื่องมือยังไม่รองรับโมเดลนี้ แล้วเพิ่งมารู้ตอนงานเกือบเสร็จ

เกิดอะไรขึ้น: ตอน export GGUF วันที่ 2026-07-21 แตกไฟล์ mmproj (ตัวแปลงรูปเป็นตัวเลข) ออกมาไม่ได้ — โมเดลใหม่เกินกว่าที่เครื่องมือจะรองรับ ผลคือได้ GGUF ที่อ่านได้แต่ข้อความ และเรื่องนี้เพิ่งเจอตอนขั้นสุดท้ายของ pipeline

ต้นเหตุจริง: คิดเอาเองว่าเครื่องมือรองรับสถาปัตยกรรมโมเดลใหม่เอี่ยมนี้ โดยไม่ได้ตรวจสอบ แคมจัดตรวจยังไม่อยู่ปลายสุดของ pipeline ที่ยาวและแพง แทนที่จะอยู่ตอนต้นซึ่งตรวจฟรี

เสียหายอะไร: การจนท้วงรอบได้ผลลัพธ์ที่ใช้งานตามจุดประสงค์ไม่ได้เลย และปัญหานี้ค้างอยู่ตั้งแต่ 07-21 ถึง 07-24

แก้ยังไ: พบว่า finetune_vision_layers=False แปลว่าฝั่งที่อ่านรูปเหมือนกับโมเดลตั้งต้นทุกไบต์ เลยดาวน์โหลด mmproj (899MB) ที่ผู้ผลิตเผยแพร่เองมาจับคู่กับ GGUF ที่จนท้วงได้แล้ว ไม่ต้องแตกไฟล์เองเลยสักนิด

กฎที่ได้: ตรวจสอบก่อนเทรนว่าเครื่องมือรองรับสถาปัตยกรรมโมเดลใหม่จริงไหม อย่าไปตรวจตอน export และถ้าพิสูจน์ได้ว่าส่วนไหนไม่ได้ถูกแตะระหว่างเทรนเลย ก็เอาเวอร์ชันที่ผู้ผลิตเผยแพร่มาใช้ ดีกว่าสร้างใหม่เอง

วิกฤต 2026-08-02

ตัวตรวจเวอร์ชันแรกจะยุบกลุ่มฐานราก 4 กลุ่มที่มี confidence_flags ต่างกันทิ้งไปเลย

เกิดอะไรขึ้น: ตัวตรวจฐานรากเข้าเวอร์ชันแรกตัดสินใจว่าเข้าจาก 'confidence_score' กับ 'note' เท่านั้น มันจะยุบกลุ่ม 4 กลุ่มที่แต่ละรายการมีเหตุผลใน 'confidence_flags' *ต่างกัน* ทิ้งไป รวมถึงรายการที่ติดธง 'inferred_from_repeating_module' ด้วย

ต้นเหตุจริง: ญญแสำหรับตัดข้อมูลเข้าถูกสร้างจากฟิลด์ที่นี้กออก ไม่ใช่จากทุกฟิลด์ที่แยกแยะสองรายการออกจากกันได้จริง

เสียหายอะไร: จะทำลายบันทึกว่าตำแหน่งฐานรากไหนอ่านมาจากแบบจริง กับตำแหน่งไหนอนุมานเอา — เป็นการสูญเสียที่มาของข้อมูลแบบกู้คืนไม่ได้ แคมถูกนำเสนอในรูปของการ 'แก้ไข' ที่ดูสะอาดตา

แก้ยังไ: ตอนนี้ตัวตรวจนับ 'confidence_flags' ด้วยแล้ว ส่วนอีก 11 กลุ่มที่เหลือถูกบันทึกไว้ใน \$0.10 ว่าเป็นช้อยกเว้นที่อนุญาตเพื่อไม่ให้มีใครมา 'แก้' มันทีหลัง

กฎที่ได้: ญญแสำหรับตัดข้อมูลเข้าต้องรวมทุกฟิลด์ที่แยกแยะสองรายการออกจากกันได้จริง โดยเฉพาะฟิลด์ที่บอกที่มาของข้อมูล และต้องบันทึกช้อยกเว้นที่ตั้งใจไว้ใน spec ไม่งั้นการทำงานรอบหน้าจะมา 'เก็บกวาด' มันทิ้ง

Lack of a zoom/crop tool made the new extraction rounds less accurate than the older Gen 2 that had real image zooming (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Page 19 view2 (beam plan) still had low-confidence segments in round t2 (B1, top B3, B3X/B5X in the extension zone below A). The diary states this is 'better than t1 (structure is clearer) but still not as accurate as Gen 2, which had real image zooming', because there is no real zoom/crop tool available.

ต้นเหตุจริง: Tooling capability was lost between generations — the Gen 2 workflow could zoom into the image, the Gen 3 hand/agent workflow could not. Nobody rebuilt the zoom/crop capability before starting two full extraction rounds that depend on reading fine detail.

เสียหายอะไร: A measurable accuracy regression against a previous generation across two rounds of work, on exactly the pages (beam plans) that carry the most valuable structural data. Multiple pages across both days were listed as needing someone to zoom in and re-verify.

แก้ยังไม: Not fixed. Recorded as a known limitation; page 19 view2 confidence improved but remains below Gen 2.

กฎที่ได้: Before starting a new extraction generation, inventory the capabilities the previous generation had and confirm each one still exists. A rewrite that quietly loses a tool ships worse data while looking like progress — and a crop/zoom step is a prerequisite for fine-detail drawing reads, not an optimization.

agent เบื่องหลังถูกฆากกลางทางทั้งที่ยังไม่ได้เขียนไฟล์เลยสักไฟล์ ทำให้ BOQ 10 หน้าว่างอยู่สองวัน

เกิดอะไรขึ้น: Agent 4 (BOQ ฐานรากแผ่ หน้า 49-58 ของบ้านหลังที่ 2) ถูกหยุด — จะโดนสั่งหยุดเองหรือ session ปิดก็ไม่เคยยืนยันได้ — ตอนที่ยังตรวจรหัสรายการอยู่ และยังไม่ได้เขียนไฟล์ออกมาสักไฟล์ บ้านหลังที่ 2 เลยค้างอยู่ที่ 88 ไฟล์ ครอบคลุม 62 จาก 72 หน้า และค้างแบบนั้นจนต้องมาทำมือเสร็จวันที่ 11 ก.ค.

ต้นเหตุจริง: agent ถูกออกแบบให้เก็บงานทั้งหมดไว้ในหน่วยความจำแล้วค่อยเขียนตอนจบ แทนที่จะเขียนทีละหน้าเมื่อทำเสร็จ พอโดนฆากกลางทางงานเลยหายหมด และไม่มีใครคอยตรวจว่าเสร็จหรือยัง ช่องว่างเลยเจอตอนมา validate

เสียหายอะไร: บ้านหลังหนึ่งขาดข้อมูลไป 10 หน้าเป็นเวลาสองวัน ทำให้งาน label-studio กับการบันทึก data log ของบ้านหลังนั้นติดขัด และต้องกลับมาทำใหม่ที่ละหน้าด้วยมือ

แก้ยังไม: อ่านหน้า 49-58 ใหม่เองทีละหน้าในวันที่ 11 ก.ค. จนบ้านหลังที่ 2 ครบ 107/107 ไฟล์ และ 72/72 หน้า

กฎที่ได้: agent ที่ทำงานสกัดข้อมูลยาวๆ ต้องเขียนลงดิสก์ทันทีที่ทำเสร็จเป็นชิ้นๆ ห้ามเก็บไว้เขียนตอนท้าย และต้อง diff ไฟล์ที่ได้จริงกับรายการหน้าที่คาดว่าจะได้เสมอ ก่อนจะประกาศว่าบ้านหลังนั้นเสร็จ

สูง 2026-07-28

Whether Unsloth can export GGUF for this MoE VL model at all was never established — and a known corrupted-GGUF issue existed upstream (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: At session close it remains unconfirmed whether Unsloth can export GGUF for this MoE VL model, with issue #3899 already reporting corrupted GGUF for Qwen3-VL-2B. Also unconfirmed: the real VRAM requirement of bf16 LoRA 30B-A3B, which was only estimated from t01 at ~74GB and judged from a TEST_STEPS=5 run.

ต้นเหตุจริง: The delivery path's feasibility was assumed rather than checked against upstream issue trackers before committing a paid round to it, and a critical resource figure was extrapolated from a 5-step test rather than measured on a full run.

เสียหายอะไร: The round's headline deliverable (a runnable GGUF) was blocked by a limitation that a pre-flight search of the upstream issue tracker would likely have surfaced.

แก้ยังไง: Not fixed — recorded as unconfirmed, with the note that continuing means waiting for a new peft/unsloth release or writing the merge ourselves.

กฎที่ได้: Before committing paid compute to a delivery path, search the toolchain's issue tracker for your exact model family. And never treat a figure extrapolated from a 5-step test as a confirmed resource requirement.

สูง 2026-07-29

REQUEST_TIMEOUT_S=1200 killed genuinely-running work at page 03 (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The timeout was calibrated from pages 01-02, which are the cover and the table of contents — the lightest pages in the book. Page 03 needed 1,256s and was killed at 1,200s. After raising it to 3,600 the page completed normally.

ต้นเหตุจริง: The threshold sample was the first-encountered case, not the worst case. Pages 01-02 are structurally the cheapest content in any drawing set, so any budget derived from them under-covers everything else.

เสียหายอะไร: A restart of the run and lost wall-clock time on a job already running at ~13-21 min/page; briefly created a false impression that the job was hung.

แก้ยังไง: fixed — REQUEST_TIMEOUT_S raised 1200 → 3600, page 03 then completed in 1,256s

กฎที่ได้: Calibrate any timeout/threshold against the heaviest plausible case, never against whatever ran first. If the sample size is 2 and both samples are the cheapest items in the set, that is not a sample.

กฎ §0.1 ที่เพิ่งเขียน กว้างเกินไป — เดือน 106 ไฟล์ แต่ 104 ไฟล์ถูกอยู่แล้ว

เกิดอะไรขึ้น: ร่างแรกของกฎ §0.1 ห้าม array ชื่อ `sections` หรือ `columns` ตัวชี้คใหม่เลยขึ้นเดือน 106 ไฟล์ พอเปิดดูจริงพบว่า 104 ไฟล์ถูกต้องอยู่แล้ว — คำว่า `sections` ในไฟล์ `index` หมายถึงหัวข้อของเอกสาร ส่วน `columns` ใน BOQ หมายถึงหัวคอลัมน์ของตาราง มีแค่ 2 ไฟล์เท่านั้นที่ผิดจริง

ต้นเหตุจริง: กฎไปห้ามที่*ชื่อ* แทนที่จะห้ามตาม*บริบท* โดยลืมนึกว่าคำเดียวกันมีความหมายต่างกันในไฟล์คนละแบบ

เสียหายอะไร: ตัวตรวจที่เพิ่งสร้างมีอัตราเตือนผิด 98% ถ้าเชื่อมั่นโดยไม่ตรวจ ไฟล์ที่ต้อง 104 ไฟล์จะโดน "แก้" จนกลายเป็นผิด

แก้ยังไง: แก้ที่ตัวสเปกกับตัวเช็ค (ไม่ใช่แก้ข้อมูล) ให้ห้ามตามบริบทของ pattern แล้วแก้เฉพาะ 2 ไฟล์ที่ผิดจริง

กฎที่ได้: เวลาตัวตรวจใหม่ขึ้นเตือนเป็นพรวด ให้เปิดดูของจริงก่อนแก้อะไรทั้งนั้น — โอกาสที่ตัวตรวจผิดมีมากกว่าที่ไฟล์ 104 ไฟล์จะผิดพร้อมกัน และให้ห้ามตามบริบท ไม่ใช่ห้ามตามชื่อ

check_format.py ฮาร์ดโค้ดว่า 'ตัวอักษร = แถว, ตัวเลข = คอลัมน์' — จริงกับ 18 บ้าน แต่ผิดกับบ้านที่ 19

เกิดอะไรขึ้น: ตัวตรวจ grid_ref ใน `tools/check\_format.py` เหมาเอาว่าตัวอักษรคือแถวและตัวเลขคือคอลัมน์เสมอ แต่แบบโครงสร้างของบ้าน 19 หักกลับกัน — ใช้ตัวอักษรกับแกนนอนและตัวเลขกับแกนตั้ง ข้อมูลที่ถูกต้องเลยไม่ผ่านตัวเช็ค

ต้นเหตุจริง: เอาข้อตกงที่สังเกตว่าเป็นจริงกับ 18 บ้านติดกันมาฮาร์ดโค้ดเป็นกฎสากล ตัวอย่าง 18 หลังจากผู้เขียนแบบไม่ก็รายดูเหมือนเป็นข้อพิสูจน์

เสียหายอะไร: ด้านตรวจจะปฏิเสธข้อมูลที่ถูกต้องของบ้าน 19 แล้วจะมีแรงกดดันให้ไป "แก้" บ้านให้ตรงกับตัวเช็ค — เท่ากับทำลายข้อมูลที่สกดมาถูกต้องเพื่อสนองกฎที่ผิด

แก้ยังไง: แก้ตัวตรวจให้รับได้ทั้งสองแบบการวางแกน แล้วรันใหม่ทั้ง 19 บ้านโดยไม่มีอะไรถอยหลัง

กฎที่ได้: อย่าฮาร์ดโค้ดข้อตกงที่สังเกตมาเป็นกฎตรวจสอบ ไม่ว่าจะมตัวอย่างรองรับก็ตัวก็ตาม เวลาด้านตรวจไม่ผ่านกับข้อมูลชุดใหม่ ให้ถามก่อนว่าด้านตรวจผิดหรือเปล่า — ตรงกัน 18 บ้านคือรูปแบบ ไม่ใช่กฎธรรมชาติ

สูง 2026-08-21

check_format.py has no page-completeness check, so a 38-page partial house passes as legitimately finished (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Designing op04's partial-house mode required reading check_house() and confirming that check_format.py skips _-prefixed files and has no page-completeness check at all — so a folder holding 38 of 107 pages passes validation legitimately, indistinguishable from a finished house.

ต้นเหตุจริง: The format checker validates per-file shape only. Completeness is a property of the folder, which nothing owns — and the extraction log records houses as done without recording scope.

เสียหายอะไร: Any partial house in the dataset is silently counted as complete, inflating apparent dataset size and misleading anyone sizing a training run. The house-24 stub pages from 08-18 have a related version of the same problem.

แก้ยังไง: Introduced a required _scope.json marker keeping three separate lists (subtasks_done / subtasks_absent_from_this_house / deliberately_skipped) — the last two kept distinct because collapsing them destroys the only signal saying whether a page is missing or merely unwanted. Log rows for an op04 run must carry op04 / pass2-only.

กฎที่ได้: A validator that checks per-item shape must be paired with a completeness marker at the collection level — and never collapse 'absent from source' with 'deliberately skipped', because that distinction is the only thing that tells a later reader whether data is missing or unwanted.

สูง 2026-08-26

GLM-4.6V passes every criterion on paper but cannot be LoRA-trained due to an unfixed upstream bug (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: GLM-4.6V has an MIT licence, the right size, and would have been the second choice — but LLaMA-Factory issue #9635 raises NotImplementedError and nobody has fixed it, so LoRA training is impossible.

ต้นเหตุจริง: Documented framework support does not mean working framework support. A model can be listed as supported while the actual code path throws.

เสียหายอะไร: Would have been discovered only after renting a GPU and attempting the run, had the team not checked the issue tracker during evaluation.

แก้ยังไง: Caught during catalogue research; the criterion 'มีเครื่องมือเทรน LoRA ที่ยืนยันว่าใช้ได้จริง (ไม่ใช่แค่เอกสารอ้าง)' was applied explicitly.

กฎที่ได้: Verify training-framework support by finding someone who actually ran it, or by checking the issue tracker for open blockers — never by reading a support matrix. Documentation claims are not evidence.

สูง 2026-08-28

CV template scan detected footings well but found ~0 columns and beams — the experiment had to be cut back (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Running pass 1.5+2.5 on 25 real plan pages across the 5 val houses showed footings detected reliably (9–20 points/page) but columns and beams at essentially zero. The night's 3-arm experiment had to be narrowed to `plan\_footing` only.

ต้นเหตุจริง: The template bank had exactly 1 column template and a single hardcoded BEAM_GAP value of 21px. One template cannot cover multiple drawing series, and one gap value is blind to any series using a different spacing.

เสียหายอะไร: The 3-arm CV-hint experiment could not test the subtask it was built for (plan_beam), on the night it was scheduled.

แก้ยังไง: Same night: expanded the column bank 1→7 templates (hand-picked from a 78-candidate montage) and BEAM_GAP 21 → BEAM_GAPS (16,21) measured across 33 houses. cv_val was rescanned and both plan_footing and plan_beam became runnable.

กฎที่ได้: Measure detector coverage across the whole corpus before scheduling an experiment that depends on it. A single hardcoded geometric constant will always be blind to half a heterogeneous dataset.

สูง 2026-08-31

LLaMA-Factory only warns (never errors) when a media file referenced via media_dir is missing (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Reading LF source confirmed that the join in converter.py:58 emits a warning, not an error, for a missing image file.

ต้นเหตุจริง: Vendor library chose a permissive failure mode for missing media, so a broken dataset can train to completion silently on partial input.

เสียหายอะไร: A run could complete looking healthy while having trained on missing images; risk mitigated only by a manual smoke check.

แก้ยังไง: Verified all 1,416 path references resolve through media_dir with 0 missing, and made a smoke run mandatory before every training launch.

กฎที่ได้: Assume media-loading libraries warn rather than fail; always run an explicit 'open every referenced file' pass before paying for a training run.

สูง 2026-08-31

The training monitor produced SSHFAIL on every line for 7 hours and never alerted (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: watch4.sh was set to check every 10 minutes and wake on CRASHED or ALL_4_DONE. It returned SSHFAIL on every single line from the very first round for 7 hours, never obtaining real data, and never exited because SSHFAIL was not classified as CRASHED.

ต้นเหตุจริง: The monitor was launched without verifying its first output, and its failure mode ('cannot connect') was treated as a pass-through rather than an alert condition.

เสียหายอะไร: Seven hours of unmonitored paid training; the two folds that OOM'd were found by manual check, not by the monitor.

แก้ยังไง: Recorded as rule_of_tune Lesson 17: a monitor's first round of output must be eyeballed before walking away, and 'cannot connect' must be an alerting condition.

กฎที่ได้: Never trust a monitor you have not watched produce one real reading, and make connection failure an alert, not a silent skip.

สูง 2026-08-31

pkill -f <script name> over SSH matched and killed the SSH session itself — twice in one day (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Because the SSH command line contains the script name being searched for, `pkill -f serve\_purson.py` matched its own session and exited 255 before the following commands ran. First occurrence: the GPU-A queue would not die and required finding PIDs 3411/3412 directly. Second occurrence: a chained pkill; mv; nohup left no server running at all, undetected until ps -C python was checked and found empty.

ต้นเหตุจริง: Pattern-matching process killers search the full command line, which includes the invoking command itself.

เสียหายอะไร: An unnoticed window with no inference server running — any job fired at the machine during it would have hit nothing.

แก้ยังไง: New rule: never pkill by script name over SSH; use explicit PIDs or separate commands, and always verify real process state afterwards.

กฎที่ได้: Never kill processes by a pattern that appears in your own command line; kill by PID and always confirm the intended process state afterwards rather than trusting the command returned.

สูง 2026-08-31

A wait-for-pip loop matched its own command line, so the inference server never started while the GPU billed (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The launcher used ``while pgrep -f 'pip install' > /dev/null; do sleep 10; done && nohup ... serve_purson.py``. The bash process's own command line contains the string 'pip install', so pgrep always found itself and the condition was permanently true. On instance 49411642, pip finished at about minute 3 but the loop kept spinning for ~17 more minutes with 2 MiB GPU usage and no purson.log line — it looked exactly like a 70GB model load in progress.

ต้นเหตุจริง: Same self-matching class as the pkill bug: a pattern search over command lines that includes the searcher.

เสียหายอะไร: ~17 minutes of paid GPU time doing nothing, disguised as normal startup; the third occurrence of this exact bug class in a single day.

แก้ยังไง: Changed the pattern to `'[p]ip install'` so the bracket makes the regex not match the literal string in its own command line; proved both patterns against a real regex before committing.

กฎที่ได้: Any pgrep/pkill/ps-grep pattern must be written so it cannot match the process issuing it (bracket trick), and a wait loop must be proven to terminate on a real machine before it is trusted.

สูง 2026-08-31

The 'command searches its own command line' bug class occurred three times in a single day (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `pkill -f` killing its own SSH session (twice) and `pgrep -f` matching its own wait loop (once) are the same defect class, hit three times within one day across different scripts.

ต้นเหตุจริง: No standing rule existed about pattern-matching over process command lines, so each instance was fixed locally without generalizing.

เสียหายอะไร: An unnoticed window with no server running, ~17 minutes of idle billed GPU, plus repeated debugging time.

แก้ยังไง: Recorded the class explicitly with two concrete mitigations: use PIDs instead of name patterns, and bracket the first character of any pattern.

กฎที่ได้: Treat any pgrep/pkill/ps-grep by name as unsafe by default — use PIDs, or bracket the pattern, and generalize the fix to every script the moment the class is identified.

เรื่องที่เหลือในหมวดนี้ (42)

IDE สร้างไฟล์ขนาด 0 ไบต์ขึ้นมาที่ path เดิมของเอกสารที่ย้ายไปแล้ว ทำให้ดูเหมือนงาน 24KB หายไป

กลาง 2026-07-06

หลังย้ายไฟล์ ให้ปิดแท็บ editor ทุกอันที่ยังชี้ path เดิม แล้วตรวจให้แน่ใจว่า path เดิมหายไปจริงก่อนลบเซสชัน ไฟล์ 0 ไบต์ที่ path ที่เคยถูกต้อง แยกว่าไฟล์ที่หายไปเลย เพราะมันอ่านออกมาเหมือนข้อมูลพัง ไม่ใช่เหมือนการย้าย

TextArea ที่ตั้ง rows > 1 จะแอบเพิ่มปุ่ม '+Add' แบบหลายค่าให้เอง ซึ่งจะทำให้ฟิลด์ค่าเดียวพัง

กลาง 2026-07-06

อ่านเอกสารของเครื่องมือก่อนจะไปแต่งหน้าตา widget ของมัน คั่นก่อน อย่าเอา และในเครื่องมือ annotation โดยเฉพาะ property อะไรก็ตามที่เปลี่ยนฟิลด์ค่าเดียวให้กลายเป็นหลายค่าได้ ถือเป็นความเสี่ยงต่อความถูกต้องของข้อมูล ไม่ใช่แค่เรื่องเลยเอาต์

ต้องวาง config ก่อน import task เสมอ ไม่งั้นต้องลบ task ที่พังไปแล้วทั้งหมด

กลาง 2026-07-06

ใน Label Studio การแก้ config ไม่ย้อนไปมีผลกับของเก่า ให้แก้และเซฟ labeling config ให้เสร็จก่อน import ทุกครั้ง และลบ task ที่พังหรือค้างอยู่ทิ้ง ไม่ใช่ import หับลงไป เขียนลำดับการกู้คืนนี้ไว้ข้างๆ ตัวเครื่องมือเลย ไม่ใช่ไว้แค่ใน log ของเซสชัน

Label Studio สลับเลยเอาต์ไม่ได้ เลยแก้ด้วยการก๊อป config เป็นคู่แฝด freeze-L/freeze-R ทำให้ทุกการแก้ที่หลังต้องแก้ไข 4-5 รอบ

กลาง 2026-07-06

ถ้าเครื่องมือบังคับให้ต้องมี config ซ้ำหลายชุด ให้ generate ชุดต่างๆ ออกมาจากต้นทางเดียว แทนที่จะนั่งก๊อปมือ ไม่งั้นทุกการแก้ที่หลังจะกลายเป็นงานแก้ N ไฟล์ และสำเนาจะค่อยๆ เพิ่มขึ้นไปคนละทาง

'Permission denied' ตอนเปลี่ยนชื่อโฟลเดอร์กลับ ดูเหมือนข้อมูลเสียหาย แต่จริงๆ คือมีอีกโปรแกรมเปิดค้างไว้

กลาง 2026-07-09

บน Windows 'Access denied' ตอนเปลี่ยนชื่อโฟลเดอร์ แปลว่ามี process เปิดค้างอยู่ ไม่ได้แปลว่าข้อมูลพัง พิสูจน์ด้วยการลองเขียนไฟล์เข้าไปข้างใน ปิดโปรแกรมที่จับไว้ แล้วลองใหม่ ก่อนจะไปติตรว่าเรื่องใหญ่

ทุกรอบของการสกัดข้อมูลถูกตรวจสอบด้วยสคริปต์ที่เขียนขึ้นชั่วคราวแล้วทิ้ง

กลาง 2026-07-10

ตัวตรวจที่ตัดสินใจว่าชุดข้อมูลครบหรือยัง ต้องเป็นสิ่งที่ commit เข้ารีโปและมีแฮชของตัวเอง ตัวตรวจแบบเขียนสดทิ้ง คือของอีกชิ้นที่ยังไม่ถูกตรวจสอบมาขวางระหว่างเรากับความจริง

Built llama.cpp with CUDA three times unsuccessfully — CUDA was never needed

กลาง 2026-07-21

Before fighting a build, ask whether that build option is needed at all — convert/quantize steps are CPU-only, so build the minimum configuration the task actually requires.

Converted to q8_0 first to save disk; llama-quantize refuses to requantize from q8_0

กลาง 2026-07-21

Always quantize from the full-precision source (f16/bf16); check a tool's supported input formats before inventing a space-saving shortcut.

PowerShell's own interpretation mangled \$PATH and broke the working terminal window

กลาง 2026-07-21

When a shell environment starts behaving wrongly (PATH, env vars), open a brand-new window instead of repairing in place; and never let PowerShell interpret '\$VAR' syntax that is meant for the remote bash shell — quote it or write it into a script file.

Training script printed its success line but kept running a ~15-minute sample-generation step, making a busy job look finished

กลาง 2026-07-21

Judge completion by the shell prompt returning or by 'ps', never by a log line; and put the final success banner at the true end of a script, after every optional/bonus step.

Typed the full-training command into a busy terminal, then panicked that full_train.log did not exist

กลาง 2026-07-21

Never judge a job finished from console text — wait for the shell prompt to return, or verify with a process check. When a log file is missing, check whether the command ever started before assuming it failed.

VSCode auto-commit swept the remaining split files back into one 2.4GB chunk mid-push

กลาง 2026-08-01

Turn off editor auto-commit/auto-stage before any manual git surgery (history rewrite, staged splitting, rebase). An IDE writing to the index behind you will silently undo your staging.

First id-warning pass was too broad — flagged 1,059 legitimate ids and polluted 205 files with noise

กลาง 2026-08-02

Test a flagging heuristic on a sample and read the hits before running it over the corpus. A warning system that cries wolf gets ignored, which is worse than having no warnings.

A known footgun: a bulk regex written for grid refs also matches sheet_code values

กลาง 2026-08-02

Scope a bulk edit by field path, never by value pattern, when two different fields share a text shape. Make the safety structural, not a matter of being careful.

บั๊กใน skill wrapper ของ op01/op03 ที่สร้างไว้วันที่ 08-05 ต้องมาแก้เอาวันที่ 08-09

กลาง 2026-08-09

สำเนาของเอกสารต้นฉบับต้องมีตัวตรวจว่ามันเพี้ยนไปจากต้นทางหรือยัง ไม่งั้นมันจะแยกทางกันแน่นอน ทางที่ดีกว่าคือให้ skill อ้างอิงไฟล์ต้นฉบับจริงๆ แทนที่จะก๊อปปี้เนื้อหาไปไว้อีกที่

โปรโตไทป์ overlay อาร์ดโค้ด ELEMENT_TYPES ไว้แค่ฐานรากกับเสา ทั้งคานทั้งหมด

กลาง 2026-08-20

ห้ามตรึงรายการชนิดข้อมูลไว้ตามที่ตัวอย่างทดสอบขึ้นเดี๋ยวนี ให้ดึงจาก schema แทน และเลือกตัวอย่างทดสอบที่มีคลาสยากที่สุด อยู่ด้วย ไม่ใช่คลาสที่ง่ายที่สุด

reportlab จัดรูปตัวอักษรไทยไม่ได้ ต้องรื้อการสร้าง PDF ไปใช้ Edge แบบ headless แทน

กลาง 2026-08-21

งานออกเอกสารภาษาไทย (หรือภาษาที่มีการประกอบตัวอักษรซับซ้อน) ต้องใช้ตัวจัดรูปตัวอักษรจริง คือ renderer ของเบราว์เซอร์ และให้ตรวจสอบด้วยการดึงข้อความกลับออกมาจาก PDF ที่ได้ เพราะไลบรารีที่ทำการจัดรูปเสียๆ จะให้ผลลัพธ์ที่รู้าพังก็ต่อเมื่อ เปิดดูด้วยตาเท่านั้น

agent ของ op04 ชน limit ของ API session แล้วทิ้งไฟล์ GT ค้างครึ่งๆ กลางๆ ระหว่างเขียน

กลาง 2026-08-24

เขียนไฟล์ GT แบบ atomic (เขียนไฟล์ชั่วคราวแล้วค่อยเปลี่ยนชื่อ) และบันทึกสถานะเสร็จรายไฟล์ ตัวสร้าง dataset ต้องข้ามไฟล์ ที่เสียแล้วรายงาน ห้ามเดาเนื้อหาซ่อมให้เอง

Column and beam detection ship on one hand-cropped template and one code-synthesized template respectively — neither ever harvested

กลาง 2026-08-25

When one class of a multi-class detector is validated far more than the others, say so beside the numbers — otherwise a single well-tested class lends unearned credibility to the rest.

Unsloth does not work with InternVL3-78B, and Llama-4-Scout on Unsloth is 4-bit only

กลาง 2026-08-26

Record, for every candidate model, which specific training framework works and at which precision. A framework that only supports 4-bit is a red flag for any vision task — that constraint is what blinds the vision encoder.

คลังเทมเพลตสายังขาดอีก 13 หลัง — ช่องว่างย่อยและพื้นที่ทั้งคลาสยังไม่มีใครครอบคลุม

กลาง 2026-08-28

ตัวตรวจจับที่เรียนรู้เองไม่มีทางเจอชุดที่มันไม่เคยเห็นมาก่อน ต้องกันเวลาไว้ป้อนตัวอย่างเริ่มต้นด้วยมือสำหรับชุดที่ยังไม่ครอบคลุม และรายงานความครอบคลุมเป็นรายหลัง เพื่อให้ "ไม่มีค่าโง่เลย" มองเห็นได้ ไม่ใช่เสียหายไป

subagent ชน spend limit ทั้ง session — งานสัปดาห์ op04 ต้องทำมือทั้งหมด

กลาง 2026-08-29

ใช้คงบ/โควตาก่อนตัดสินใจลงงานที่ใช้ agent ขนานกันหลายตัว และเขียนแผนสำรองแบบทำทีละตัวพร้อมเวลาที่ต้องใช้ไว้ล่วงหน้าเสมอ

คลังเทมเพลต CV สร้างจากบ้าน 40 หลังนี้เท่านั้น — ลูกค้าใหม่ที่เขียนแบบคนละสำนักอาจไม่แมตช์อะไรเลย และการเก็บข้อมูลเองก็เริ่มไม่ได้ถ้าไม่มีเมล็ดตั้งต้น

กลาง 2026-08-30

ระบบจับรูปแบบ/เทมเพลตที่เทรนจากข้อมูลภายในต้องมีเส้นทางเริ่มต้นสำหรับธรรมเนียมที่ไม่เคยเจอ ออกแบบ UX สำหรับการป้อนเมล็ดพร้อมกับตัวจับคู่ ไม่ใช่ค่อยมาทำทีหลัง

สคริปต์ตรวจที่เขียนเองอ่านชื่อไฟล์ผิด ผิดแล้วสรุปว่ากล่อง CV ไม่มีพิกัด

กลาง 2026-08-31

เมื่อตัวตรวจรายงานว่า 'ข้อมูลขาด X' ให้พิสูจน์ก่อนว่าตัวตรวจอ่านไฟล์ถูกต้องกับข้อมูลที่รู้ว่าดีอยู่แล้ว — ตัวตรวจที่ฟังหน้าตาเหมือนข้อมูลที่ฟังปะ

ลูป poll สถานะ instance เรียก python3 ซึ่งบน Windows เป็น alias ของ Store ที่ใช้ไม่ได้ มันเลยวนไม่จบ

กลาง 2026-08-31

พิสูจน์ว่ารอบแรกของลูป poll ได้ข้อมูลจริงก่อนจะเดินจากไป และห้ามคิดว่า python3 มีอยู่บน shell ของ Windows

กับดัก Windows สองอัน: ไฟล์ manifest ที่เขียนด้วย Python ได้ CRLF และ tar อ่าน D:/ เป็นชื่อเครื่องปลายทาง

กลาง 2026-08-31

บน Windows ให้เขียนไฟล์ manifest ที่ใช้ข้ามระบบด้วย newline='\n' เสมอ และใช้ path แบบ MSYS (/d/) กับ tar ห้ามใช้ path ที่ขึ้นต้นด้วยตัวอักษรไดรฟ์

Visual-token warning compared against the global cap, so it fired on every legitimately small image

กลาง 2026-08-31

An alarm's threshold must be computed per item from what that item should legitimately achieve — a global constant baseline turns a detector into noise.

PEFT's add_weighted_adapter cannot merge MoE adapters using target_parameters

กลาง 2026-08-31

Verify that the intended library API supports your specific architecture before designing the final assembly step of an expensive pipeline.

Naming the merged adapter 'destrier' would have hidden it in a subfolder with no files at repo root

กลาง 2026-08-31

Check how a serialization library maps names to paths before choosing a name, and verify the published artifact's file layout, not just that the upload succeeded.

New rental template had no /workspace and plain pip installed into the wrong Python

กลาง 2026-08-31

On any new rental template, verify the working directory exists and resolve the absolute path of the Python that owns torch before installing anything — never rely on bare pip/python.

presentation_state.json kept a stale instance id after the card was returned

กลาง 2026-08-31

Reconcile any local record of a billed cloud resource against the provider on every read, and bias toward over-warning when the provider cannot be reached.

The merge script would have uploaded all four source fold adapters alongside the merged one

กลาง 2026-08-31

Before pushing a model, enumerate what is actually attached to it — a push uploads state, not intent.

Two git config commands pasted back-to-back with no separator produced 'no action specified'

ต้า 2026-07-06

Hand over multi-command snippets one command per line, or explicitly joined with `;`. And keep the rule that an agent explains git config rather than executing it — the safety protocol held correctly here.

scp -P vs ssh -p port flag case mismatch caused failed upload attempts

ต้า 2026-07-21

Write the exact working command (with flags) into the runbook the first time it succeeds; never assume two related CLI tools share flag casing.

Assumed Unsloth could be used to run the model locally for real use — it needs the full 70GB model and a 74GB+ card

ต้า 2026-07-24

Training-time tooling and inference-time tooling are different programs with different hardware assumptions. Confirm which tool owns each phase before planning around it.

The pattern vocabulary lives in two places — extend the spec without the checker and every new file fails instantly

ต้า 2026-08-04

Derive the count from the list rather than writing the number in prose, and treat the checker constant as part of the spec — one change, one commit, or the gate fails on correct data.

Overlay click-to-accept toggle only console.logged — nothing was saved

ต้า 2026-08-20

A UI affordance that discards its own output is worse than a missing one; either persist the decision or disable the control until the write path exists.

The house 01 overlay render — the strongest proof produced all week — was a scratch script never added to the repo

ต้า 2026-08-20

The moment a script's output is quoted as evidence, the script is a deliverable: commit it. Re-deriving the same calibration three times costs more than a one-line `git add`.

House 09 หน้า28 overlay skipped — grid circles incomplete on the printed sheet, calibration impossible

ต้า 2026-08-21

Any calibration method that depends on a printed feature needs a documented fallback (manual two-point calibration, or reuse of a sibling sheet's scale) — otherwise the sheets that most need checking will be exactly the ones that cannot be checked.

graphify CLI not installed — documented workflow step silently skipped

ต้า 2026-08-26

A workflow that depends on a machine-local CLI must document the install and check for it, not fail with 'command not found' and get skipped. Prefer repo-checked-in tooling over per-user PATH dependencies.

PaddleOCR needed two environment workarounds before it would run at all

ต้า 2026-08-29

Record vendor-tool environment workarounds in the runbook the moment you find them — the next person on a different machine will hit the same two walls and re-derive them from scratch.

measure_capacity.py's --split was locked to a fixed choice list, blocking measurement of the fold files

ต้า 2026-08-31

Don't hard-code a choices list for a filename fragment — it turns a data-layout change into a tool outage right when the measurement matters.

การเทรนโมเดล และ วิธีทำ ML

การเทรน แบ่งข้อมูล prompt รวมโมเดล และการเรียกใช้โมเดล ที่ไม่ได้ทำอย่างที่ตั้งใจ

ทั้งหมด 75 เรื่อง · วิกฤต 18 · สูง 34 · กลาง 21 · ต่ำ 2

เรื่องที่ต้องอ่านให้ครบ (52)

วิกฤต 2026-07-24

MAX_LENGTH ที่คิดจากตัวอย่างรูปเดียว จะตัด label ของตัวอย่าง gridmaster ทุกตัวทิ้ง

เกิดอะไรขึ้น: `MAX\_LENGTH=9216` ถูกคำนวณจากตัวอย่างที่มีรูปเดียว แต่ตัวอย่าง gridmaster หักหัว (บ้านละหนึ่งตัว) ใช้รูป 2-4 รูป และต้องการประมาณ 22,067 token จริงๆ ถ้าไม่แก้ ตัวเทรนจะตัด label ของตัวอย่างพวกนั้นทิ้งไปทั้งหมด

ต้นเหตุจริง: งบประมาณยาวลำดับถูกคำนวณจากตัวอย่างค่ากลาง ไม่ใช่ค่าสูงสุด และตอนตั้งค่าก็ไม่ได้นึกถึงตัวอย่างที่มีหลายรูป

เสียหายอะไร: จับได้ตอนอ่านทบทวนใน Phase 0.4 ก่อนจะเข้า GPU จริง ถ้าปล่อยผ่านไป ตัวอย่างที่สำคัญที่สุดเชิงโครงสร้างหัวตัวจะถูกเทรนด้วยลำดับที่ถูกตัดจน label หายไป โดยไม่มี error ขึ้นเลย

แก้ยังไง: เพิ่ม `MAX\_LENGTH` เป็น 24576 ยอมรับความเสี่ยง OOM ที่สูงขึ้นในตัวอย่าง 5 ตัวนั้น แทนที่จะลดความละเอียดของรูป

กฎที่ได้: ตั้งงบประมาณยาวลำดับจากตัวอย่างที่ยาวที่สุด ไม่ใช่ค่ากลาง และวัดแยกตามประเภทของตัวอย่างด้วย (รูปเดียว vs หลายรูป) การตัด label แบบเสียๆ เป็นบั๊กตอนเทรนที่แย่ที่สุด เพราะไม่มี error ขึ้น แต่ได้โมเดลที่แย่ง

วิกฤต 2026-07-24

JSON ที่ถูกต้องเหลือ 0% เพราะ chat template ตอน inference ปิดโหมด reasoning ที่ตอนเทรนไม่เคยใช้

เกิดอะไรขึ้น: การประเมินรอบแรกใน Phase 7 ได้คะแนน JSON ถูกต้อง 0% — แยกว่าโมเดลตั้งต้นที่ยังไม่ได้จูนอีก พอไล่ดูพบว่าโมเดลเขียนคำอธิบายความคิดยาวๆ นำหน้าทุกคำตอบ กินงบประมาณ token จนไปไม่ถึง JSON เลย

ต้นเหตุจริง: เรียก `apply\_chat\_template(add\_generation\_prompt=True)` โดยไม่ใส่ `enable\_thinking=False` ตอนเทรนใช้ `add\_generation\_prompt=False` และไม่เคยใส่แท็ก `` เลย แต่ตอน inference ค่าเริ่มต้นใส่ `\n` เปล่าๆ เข้าไป ทำให้โมเดลเข้าใจผิดคิดว่าวิเคราะห์ สรุปคือ template ตอน inference ไม่ตรงกับตอนเทรน

เสียหายอะไร: ผลลัพธ์ออกมาดูเหมือนการ fine-tune ทำโมเดลพัง แต่หลังแก้แล้ว ตัวเลขจริงคือ JSON ถูกต้อง 90%, view ตรงเป้า 70% (เดิม 60%) และ element recall 28.2% (เดิม 9.4% ดีขึ้น 3 เท่า) — จริงๆ แล้วการ fine-tune ได้ผลมาตลอด

แก้ยังไง: ใส่ `enable\_thinking=False` ในทั้งสามไฟล์ (`eval\_fields.py`, `train\_qwen36.py`, `export\_gguf.py`) แล้วรันใหม่ผ่านทันที

กฎที่ได้: chat template ตอน inference ต้องทำงานเหมือนตอนเทรนทุกไบต์ — เอามา diff กันตรงๆ เป็นข้อหนึ่งใน checklist เลย และถ้าตัวเลข eval ออกมาแยแบบหายนะ มักจะเป็นเพราะชุดทดสอบตั้งค่าไม่ตรงกันมากกว่าโมเดลพัง ตรวจสอบก่อนสรุปว่าเทรนล้มเหลว

เกือบเอาผลลัพธ์ของโมเดลที่แม่นแค่ 28% ป้อนกลับเข้าโพลเดอร์ที่ใช้เป็น ground truth สำหรับเทรน

เกิดอะไรขึ้น: มีคนตั้งคำถามว่าจะใช้โมเดลที่จนแล้วสร้าง raw JSON ของ `บ้าน\_ใหญ่\_1ชั้น\_01` ลงในโพลเดอร์ที่ใช้งานจริงเลยไหม แต่ `rawjson` ยังไม่ได้แก้ไขโดนคน/ คือ ground truth ที่ป้อนเข้าไปเทรน การเอาผลลัพธ์ที่แม่นแค่ 28% และยังไม่ผ่านการตรวจ มาใส่ตรงนั้นเท่ากับวางยาตัวเอง

ต้นเหตุจริง: ยังไม่ได้ตัดสินใจล่วงหน้าว่าร่างที่ AI สร้างจะไปอยู่โพลเดอร์ไหน และโพลเดอร์ ground truth ก็เป็นปลายทางที่นึกถึงเป็นอันดับแรกอยู่แล้วเวลาพูดว่า "raw JSON ของบ้านหลังหนึ่ง"

เสียหายอะไร: จับได้ก่อนเกิดขึ้นจริง ถ้าปล่อยไป ชุดข้อมูลเทรนจะปนเปื้อนด้วยข้อผิดพลาดของโมเดลเองที่แม่นแค่ 28% เกิดวงจรป้อนกลับที่ทำให้หุ้รอบกั้ตไปแย่งเรื่อยๆ

แก้ยังง: ตกลงกันว่าผลลัพธ์เป็นแค่ร่างที่รอคนตรวจ ให้เขียนลงโพลเดอร์แยกต่างหาก ห้ามเขียนลงโพลเดอร์ ground truth เด็ดขาด

กฎที่ได้: ห้ามเขียนผลลัพธ์ของโมเดลลงในโพลเดอร์ที่ใช้เทรนโมเดลนั่นเอง บังคับด้วยการแยก path และถ้าเป็นไปได้ให้ใส่ตัวกันการเขียนไว้ด้วย — พลาดเซฟผิดที่ครั้งเดียว ปนเปื้อนหุ้รอบเทรนหลังจากนั้นทั้งหมด

merge_and_unload() สัมเหลวเจียบๆ กับ LoRA แบบ MoE ParamWrapper — ค่าใน tensor เปลี่ยนแค่ 0.67%

เกิดอะไรขึ้น: ตอน merge LoRA adapter เข้าโมเดลตั้งต้นก่อนแปลงเป็น GGUF พังกั้ชันทำงานจนจบโดยไม่มี error แต่พอวัดด้วยมีอบน tensor ที่มี 201 ล้านค่า พบว่ามีค่าที่เปลี่ยนไปแค่ 0.67% — แปลว่าโมเดลที่ merge แล้วมีพฤติกรรมเกือบเหมือนโมเดลตั้งต้นที่ยังไม่ได้จนเลย เรื่องนี้ทำให้ GGUF ของ t02 ติดขัด และสงสัยอย่างหนักว่าน่าจะทำให้ GGUF ของ t01 เสียเจียบๆ ไปด้วย

ต้นเหตุจริง: กลไก ParamWrapper ของ peft ซึ่งเป็นวิธีที่ LoRA เกาะเข้ากับ experts ของ MoE ไม่ได้ถูกจัดการอย่างถูกต้องโดย merge_and_unload() ความล้มเหลวแบบนี้คือโครงสร้างผ่านแต่พฤติกรรมพัง — ไม่มี exception รูปร่างข้อมูลถูกต้อง แต่เนื้อในผิด

เสียหายอะไร: หุ้การตัดสินใจที่ตามมาจาก GGUF ของ t01 กลายเป็นเรื่องน่าสงสั้ยหมด บั้กนี้ตอนนีขวางทั้งสองรอบการเทรนไม่ใช่แค่ t02 และเป็นไปได้ว่าไฟล์ที่จะเอาไปใช้งานจริงไม่มีผลของการ fine-tune อยู่เลยแม้แต่นิดเดียว

แก้ยังง: ยังไม่ได้แก้ ออกแบบการทดสอบฟรีที่ใช้เวลาประมาณ 20 นาทีไว้แล้ว (รันหน้า 03 ด้วย GGUF ตั้งต้นที่ยังไม่จน แล้วเทียบกับผลของ t01) เพื่อยืนยัน และต้องรันก่อนจะจ่ายเงินเพิ่มอีก

กฎที่ได้: หลังทำขั้นตอน merge/convert/quantize หุ้ครั้ง ให้ตรวจสอบด้วยการวัดค่าน้ำหนัก (ก็เปอร์เซ็นต์ของค่าที่เปลี่ยนไป) และวัดพฤติกรรมกับตัวอย่างที่กันไว้ — อัยายอมรับ "มันรันจบโดยไม่มี error" เป็นหลักฐานว่าการแปลงมีผลจริง

วิกฤต 2026-07-29

0 จาก 6 หน้าที่ใช้ schema ที่เทรนไว้ — พณชื่อ key ออกมา 3 แบบสลับกันมั่วๆ

เกิดอะไรขึ้น: ข้อมูลจริงใช้ png / doc_page / discipline / sheet_code แต่โมเดลพ่นออกมาเป็น 'page', 'page_number' และ 'page_number'+ 'sheet_id' สลับกันไปทั้ง 6 หน้า รวมสามรูปแบบ ไม่ตรงกับข้อมูลจริงสักแบบ ทั้งที่หน้า 03-06 เป็นข้อมูลเทรนของมันเองที่เห็นมาแล้ว 3 epoch

ต้นเหตุจริง: format คือสิ่งที่ fine-tune เรียนรู้ได้เร็วที่สุด การที่มันทำ format พังบนหน้าที่ทองมาแล้ว แต่กลับทำเนื้อหาๆ ได้ถูก คือลายนิ้วมือของโมเดลที่ไม่เคยได้รับการ tune เลย — นั่นคือการ merge สัมแบบเสียบๆ นั่นเอง

เสียหายอะไร: output นี้ระบบปลายทางที่คาดหวังสัญญาณรูปแบบข้อมูลจะอ่านไม่ได้เลย แต่ที่สำคัญกว่านั้นคือความผิดพลาดนี้เองที่เปิดโปงต้นเหตุจริงออกมา

แก้ยังไม: ยังไม่แก้ — จะหายก็ต่อเมื่อปัญหา LoRA/merge ถูกแก้

กฎที่ได้: ให้ดูรูปทรงของความผิดพลาด ไม่ใช่ดูอัตราความผิดพลาด อาการ 'ผิดในเรื่องง่าย แต่ถูกในเรื่องยาก' คือทองคำสำหรับการวินิจฉัย ซึ่งคะแนนรวมแบบภาพรวมจะกลบมันมิดสนิท

วิกฤต 2026-07-29

output ของโมเดลตรงกับ schema ที่เทรนไว้ 0 จาก 6 หน้า และใช้ชื่อ key ถึง 3 สไลด์

เกิดอะไรขึ้น: ไม่มีสักหน้าเดียวจาก 6 หน้าที่ใช้ key ตามข้อมูลจริง (png / doc_page / discipline / sheet_code) โมเดลใช้ "page", "page_number" และ "page_number"+"sheet_id" สลับกันไปมาในแต่ละหน้าอย่างไม่มี ความสม่ำเสมอ

ต้นเหตุจริง: เป็นหลักฐานหนักแน่นว่าการ fine-tune ไม่ได้อยู่ในของที่เรีนอยู่จริง เพราะ format ของ output คือสิ่งแรกที่ fine-tune ล็อกได้ ภายในไม่กี่ร้อย step โมเดลที่ไม่ได้ tune แล้วได้รับ prompt ที่บรรยาย schema จะมีพฤติกรรมแบบนี้เปะ — มันทำตามคำศัพท์ใน prompt และไม่รู้จัก key ที่มีอยู่แต่ในคำตอบสำหรับเทรน

เสียหายอะไร: ทุกไฟล์ที่สกัดออกมาใช้งานปลายทางไม่ได้ถ้าไม่แปลงชื่อ key ก่อน และความไม่สม่ำเสมอระหว่างหน้าทำให้แม้แต่สคริปต์แปลงก็ยังเชื่อถือไม่ได้

แก้ยังไม: ยังไม่แก้ ข้อสังเกตนี้กลายเป็นเบาะแสหลักที่ชี้ไปยังสมมติฐานว่า LoRA หายไป

กฎที่ได้: การที่ output ทำตาม format ได้ถูก คือสัญญาณที่ถูกที่สุดและเร็วที่สุดว่า fine-tune ถูกโหลดเข้ามาจริง — ให้เช็คตั้งแต่หน้าแรก และถ้า format พังทั้งที่เนื้อหาดี ให้ถือว่าเป็นบักของการ deploy ไม่ใช่ปัญหาคุณภาพโมเดล

วิกฤต 2026-07-29

โมเดลเห็นภาพหลอน พณชิ้นส่วนออกมา 59 ชิ้น ในหน้าที่ข้อมูลจริงมีแค่ 17

เกิดอะไรขึ้น: ในหน้า 06 โมเดลพ่น element ออกมา 59 ชิ้น เทียบกับข้อมูลจริงที่มี 17 ชิ้น โดยแต่งเสาชั้นมาเอง 12 ต้น ประตุ 8 บาน และหน้าต่าง 4 บาน ซึ่งไม่มีอยู่ในแบบแผนนั้นเลย

ต้นเหตุจริง: เมื่อไม่มี fine-tune ทำงานอยู่ (ตามที่สงสัย) โมเดลจะเติม schema ที่ถูกขอมาด้วยรายการที่ดูสมเหตุสมผล แทนที่จะจำกัดตัวเองอยู่แค่สิ่งที่มียูบนหน้านั้นจริงๆ กฎ 'ใช้ null ห้ามเดา' ใน prompt เป็นแค่คำสั่งที่พูดออกไป ไม่ได้ถูกเรียนรู้

เสียหายอะไร: การถอดปริมาณงานอะไรก็ตามที่สร้างบน output นี้จะสูงเกินจริงไปมหาศาล และไฟล์ที่สกัดของบ้าน 01 ก็ปนเปื้อนไปแล้ว

แก้ยังไม: ยังไม่แก้ การสกัดข้อมูลบ้าน 01 ถูกหยุดไว้ก่อนจนกว่าคำถามเรื่อง LoRA หายจะได้คำตอบ

กฎที่ได้: ใส่การตรวจนับจำนวน element เทียบกับข้อมูลจริง (หรือเทียบกับการรันรอบสอง) ก่อนจะยอมรับผลการสกัดข้อมูล การพ่นเกินอันตรายนพๆ กับการตกหล่น และตรวจจับด้วยเครื่องได้ง่ายกว่าการนั่งไล่ดูด้วยตา

วิกฤต 2026-07-29

โมเดลกลับหัวกลับหางตัวอักษรของเส้นกริด และแต่งระยะห่างภายในขึ้นมาเองทั้งหมด บังเอิญได้แค่ระยะรวมที่ตรง

เกิดอะไรขึ้น: ข้อมูลจริงของกริดคือ D=0, C=4.0, B=6.0, A=9.5 แต่โมเดลตอบ A=0, B=1.5, C=5.0, D=9.5 — ตัวอักษรกลับด้าน ระยะห่างภายในผิดทุกช่วง มีแค่ผลรวม 9.5 ที่ถูก และยังพ่น grid_table ที่ให้ค่าหนึ่งค่าต่อจุดตัดกริดหนึ่งจุด ซึ่งเป็นปริมาณที่ไม่มีอยู่จริงในทางกายภาพ

ต้นเหตุจริง: ตัวเลขจริงจากแบบ (7.0, 9.5, 4.0, 3.5) ถูกสลับเปลี่ยนลงช่องต่างๆ แทนที่จะอ่านตามตำแหน่งจริง โมเดลสร้างตารางที่ดูสมเหตุสมผลขึ้นมาแทนที่จะยอมรับว่ามันแก้เรขาคณิตตรงนี้ไม่ได้

เสียหายอะไร: prompt กำหนดชัดว่าช่วง (span) ต้องมาจากตารางกริด ดังนั้นการคำนวณช่วงทุกอย่างที่อยู่ถัดจากนี้จึงผิดหมด ซึ่งเท่ากับทำลายปริมาณงานโครงสร้างที่เป็นเหตุผลของการมีอยู่ของทั้งโปรเจกต์

แก้ยังไง: ยังไม่แก้ ตั้งธงไว้ว่าเป็นข้อบกพร่องของ output ระดับรุนแรงสูงสุด

กฎที่ได้: ต้องตรวจ output ของกริดเชิงโครงสร้างก่อน (พิกัดต้องเรียงทางเดียว ตัวอักษรต้องเรียงตามแบบ ผลรวมสะสมต้องสอดคล้องกัน) ก่อนจะเอาไปคำนวณช่วงใดๆ — ผลรวมที่ถูกไม่ใช่หลักฐานว่าระยะห่างถูก

วิกฤต 2026-08-04

การเทรนรอบ t02 เสียเปล่าทั้งรอบ — Qwen3-VL-30B-A3B export เป็น GGUF ไม่ได้ แคมแพท t01 ทุกตัวชี้วัด

เกิดอะไรขึ้น: workflow ของ t02 ถูกตีตรา  SUPERSEDED: Qwen3-VL-30B-A3B export เป็น GGUF ไม่ได้เลย (เป็นบั๊กตอน merge MoE ใน `ParamWrapper.merge()` ของ MoE experts) และโมเดลที่ได้ก็แพท t01 ทุกตัวเลขที่วัด สุดท้ายทั้งสายนี้แล้วถอยกลับไปใช้ Qwen3.6

ต้นเหตุจริง: เลือกโมเดลแล้วเทรนไปเลย ทั้งที่ยังไม่เคยพิสูจน์เส้นทางใช้งานจริงครบวง (merge → export GGUF → รัน inference บนเครื่องนี้) จุดที่ตันอยู่ตรงขั้น export ซึ่งกว่าจะรู้ก็เทรนเสร็จไปแล้ว

เสียหายอะไร: การ fine-tune หนึ่งรอบเต็ม — ทั้งค่าเช่า GPU ทั้งเวลาสร้าง dataset ทั้งเวลาเทรน — ได้โมเดลที่เอาไปใช้ไม่ได้และแย่กว่าตัวเดิม ขาดทุนล้วนๆ

แก้ยังไง: ตีตรา t02 เป็น SUPERSEDED พร้อมบันทึกเหตุผลไว้ เก็บไฟล์ไว้เป็นหลักฐานว่าทำไม แล้วกลับไปใช้ Qwen3.6-35B-A3B ที่เส้นทาง merge/GGUF พิสูจน์แล้วว่าใช้ได้จริงบนเครื่องนี้

กฎที่ได้: พิสูจน์เส้นทางใช้งานจริงให้ครบ (merge → export → รันบนเครื่องจริง) กับโมเดลตัวตั้งต้น ****ก่อน**** ที่จะหุ้มเทรนหนึ่งรอบลงไป โมเดลที่ export ไม่ได้ = มีค่าเป็นศูนย์ ไม่ว่าจะคะแนนจะสวยแค่ไหน

เทรนแบบหน้าเดียวจบในที่เดียว แต่ของจริงรัน 5 pass ข้ามหน้า — โมเดลที่ tune มาแทนได้อย่างมากแค่ pass เดียว

เกิดอะไรขึ้น: ไล่ดูตั้งแต่ต้นจนจบแล้วพบว่า การเทรนสอนงานแบบ "หนึ่งภาพ → JSON ครบของหน้านั้น จบในที่เดียว" แต่ pipeline ที่ใช้งานจริงเป็นแบบ 5 pass และต้องอ้างอิงข้ามหน้า โมเดลที่ tune มาไม่เคยถูกสอนให้เชื่อมโยงข้ามหน้าเลย ดีที่สุดคือแทนได้แค่ Pass 1 ไม่ใช่ทั้ง pipeline แถมยังทำให้ค่า element recall ถูกจำกัดโดยโครงสร้างด้วย เพราะ ground truth สร้างจากหลายแผ่นแต่โมเดลเห็นแค่แผ่นเดียว

ต้นเหตุจริง: ออกแบบรูปแบบงานเทรนจากรูปแบบของข้อมูล (หนึ่งไฟล์ต่อหนึ่งหน้า) แทนที่จะออกแบบจากงานจริงที่มันต้องไปทำ และไม่มีใครเอา input/output ของการเทรนไปเทียบกับ pass ของ production ก่อนเริ่มเทรนเลย

เสียหายอะไร: fine-tune ไปสองรอบเต็ม (t01, t02) ได้โมเดลที่ไม่รู้จะเอาไปเสียบตรงไหน แถมตัวเลข recall ที่เป็นตัวชูโรงก็เป็นค่าที่โครงสร้างงานทำให้ไปไม่ถึงอยู่แล้ว ตัวเลขจึงไม่ใช่แค่ต่ำ แต่หลอกด้วย

แก้งัง: ตั้งเป็นกฎตายตัวว่าแผ่น t03 ต้องระบุให้ชัดก่อนเริ่มว่าจะไปแทน pass ไหน แล้วออกแบบ t03 เป็นสถาปัตยกรรม 2 pass ที่แมปตรงกับ pass จริงแบบ 1:1

กฎที่ได้: ก่อนเทรน ต้องบอกให้ได้ก่อนว่าโมเดลจะไปแทนขั้นตอนไหนของงานจริง แล้วทำ input/output ตอนเทรนให้เหมือนขั้นตอนนั้นเป๊ะ รอบที่วัดผลสอยแต่เอาไปใช้ที่ไหนไม่ได้ = รอบที่เสียเปล่า และตัวชี้วัดที่รูปแบบงานทำให้ไปไม่ถึง = ตัวชี้วัดที่พัง

Pass 3 ของงานจริงสั่งให้โมเดลกระแะช่วงเอาเอง — ขัดกับกฎ §4 ของ schema ตรงๆ

เกิดอะไรขึ้น: schema §4 เขียนไว้ว่ากระแะช่วง "ให้โค้ดคำนวณจากกริดเท่านั้น ห้ามให้โมเดลประมาณกระแะเด็ดขาด" แต่ `qt\_buildCountPrompt` กลับสั่งให้โมเดลจัดกลุ่ม element "ตามความยาวช่วง (เมตร)" ซึ่งก็คือให้กระแะจากภาพนั่นเอง

ต้นเหตุจริง: prompt ที่ใช้งานจริงกับ schema ของการสกัดข้อมูลเขียนกันคนละสาย ไม่มีใครเอามาตรวจไขว้กัน และกฎใน schema ก็ไม่เคยถูกบังคับใช้กับฝั่ง prompt เลย

เสียหายอะไร: ความยาวช่วง ซึ่งไหลตรงเข้าไปคิดปริมาตรคอนกรีต ความยาวเหล็ก และราคา กลายเป็นค่าที่ได้จากโมเดลเพ่งพิศเซลเอา ไม่ได้มาจากตารางกริด ปริมาณที่ผิดจึงไหลลงไปทุกการคำนวณข้างล่าง

แก้งัง: บันทึกไว้ว่าเป็นความขัดแย้งกันเอง แล้วสถาปัตยกรรมของ t03 ย้ายการคำนวณระยะช่วงไปอยู่ในโค้ดล้วนๆ (ชั้น merge) ไม่มี AI เข้ามาเกี่ยวเลย

กฎที่ได้: ตรวจ prompt เทียบกับสเปกข้อมูลด้วย ไม่ใช่ตรวจแค่โค้ดเทียบสเปก prompt ก็คือการนำกฎไปใช้อย่างหนึ่ง — prompt ที่ขอค่าที่สเปกห้ามโมเดลผลิต ก็คือการละเมิดสเปก

วิกฤต 2026-08-04

รูปแบบ output ของ Pass 1 เก็บเหลืบน/ล่างไม่ได้ — ต่อให้อ่านถูกเป็ก็โดนบีแบนตอนเขียนลงอยู่ดี

เกิดอะไรขึ้น: ของจริงมี `main\_bar\_count` ซองเดียวแบนๆ แต่ schema §6 กำหนดว่าต้องแยกเป็น `main\_bar{top, middle, bottom}` ต่อให้โมเดลอ่านมาถูกต้องสมบูรณ์แบบ ข้อมูลก็ถูกยุบทั้งตอนเขียนลงอยู่ดี

ต้นเหตุจริง: โครงสร้างข้อมูลฝั่ง production มีมาก่อนที่ schema จะกำหนดเรื่องแยกหน้าเหลื แล้วก็ไม่เคยถูกขยายให้รองรับ ข้อมูลหายตรงจุดที่เขียนลง ซึ่งเป็นจุดที่การวัดความเอนของตัวโมเดลเองมองไม่เห็น

เสียหายอะไร: ข้อมูลว่าเหลือยู่หน้าไหนถูกทำลายทุกครั้งทีสกด ไม่ว่าโมเดลจะดีแค่ไหน และทำให้การไป tune Pass 1 ไม่มีความหมายเลยจนกว่ารูปแบบ output จะเปลี่ยน

แก้ยังง: บันทึกไว้เป็นเงื่อนไขที่ต้องทำก่อน: ทั้ง prompt และรูปแบบ output ต้องเปลี่ยนก่อน การไป tune Pass 1 ถึงจะเริ่มมีค่า

กฎที่ได้: เช็คก่อนว่า schema ปลายทางเก็บสิ่งที่โมเดลผลิตออกมาได้จริงไหม ก่อนจะไป tune เพื่อสิ่งนั้น การไปพัฒนาตัวอ่าน ให้เก่งขึ้นทั้งที่ตัวเขียนตัดข้อมูลทั้ง = เสียแรงเปล่า ขยายปลายทางให้รับได้ก่อน

วิกฤต 2026-08-04

t01 เทรนบนภาพที่เหลืแค่ ~266 visual token ต่อภาพ — เบลอกว่าที่ตั้งใจ 14 เท่า เพราะลืมใส่ resize="max"

เกิดอะไรขึ้น: รอบเทรน t01 ใช้ visual token ประมาณ 266 ต่อภาพ น้อยกว่าที่ตั้งใจไว้ราว 14 เท่า เพราะไม่เคยตั้งค่า `resize="max"` เลย เรื่องนี้ถูกระบุว่าเป็นหนึ่งในสองอย่างที่รู้าพังและต้องแก้ก่อนจะเริ่มรอบ t03

ต้นเหตุจริง: รับค่า default ของการเตรียมภาพมาใช้เลยโดยไม่เคยเช็คความันเหลื token ต่อภาพเท่าไร และในชุดตั้งค่าการเทรน ก็ไม่มีอะไรรายงานหรือ assert จำนวน visual token เลย

เสียหายอะไร: เทรน fine-tune ทั้งรอบบนภาพที่ความละเอียดต่ำจริงๆ — บนงานที่ต้องอ่าน mark กับตัวเลขระยะเล็กๆ บนแบบก่อสร้าง ซึ่งความละเอียดคือหัวใจทั้งหมด ตัวเลขทุกตัวของ t01 จึงถูกกดต่ำเพราะเรื่องนี้ และเพดานของโมเดลถูกกำหนดโดยค่า default ตัวเดียว

แก้ยังง: ยังไม่ได้แก้ — ระบุเป็นหนึ่งในสองอย่างที่ต้งแก้ให้ได้ก่อน t03 (อีกอย่างคือสร้าง dataset ใหม่จาก 11 หลังแทนที่จะเป็น 5 หลัง)

กฎที่ได้: assert จำนวน visual token ต่อภาพจริงๆ ก่อนเริ่มเทรน แล้ว log ไว้ด้วย บนงานที่ต้องอ่านรายละเอียดเล็กๆ ค่า default ของการเตรียมภาพตัวเดียวทำให้เสียทั้งรอบได้แบบเสียๆ

VLM ยุดคานจริง 48 ตัวเหลือ 2 รายการใน JSON แล้วไม่มีอะไรจับได้เชิงกลไกเลย

เกิดอะไรขึ้น: บนหน้า 26 ของบ้าน 09 โมเดลพ่นคานออกมา 2 ตัวใน JSON ทั้งที่ในแผ่นมี 48 ตัว ความผิดพลาดนี้จะเห็นก็ต่อเมื่อมีคนมานั่งเพ่ง JSON เอง ไม่มีการตรวจอัตโนมัติไหนเลยที่เอาจำนวน element ที่ได้ไปเทียบกับอะไรสักอย่าง

ต้นเหตุจริง: เป็นอาการนับผิดของ VLM ที่มีบันทึกไว้แล้ว — cross-modal attention ที่ไม่สมดุลทำให้ตัวถอดรหัสเชื่อ text prior มากกว่า visual token และความแม่นยำตกฮวบเมื่อวัตุเกินราว 4 ชิ้น ข้ำเติมด้วยการที่ไม่มีแหล่งนับอิสระอีกแหล่งไว้เทียบเลย pipeline เลยไม่มีทางรู้ตัว

เสียหายอะไร: ground truth / output ของโมเดลผิดแบบเสียบๆ บน subtask ที่สำคัญที่สุด (beam plan) และตรวจไม่ได้เลยถ้าไม่มานั่งรื้อที่ละหน้าด้วยมือ

แก้ยังง: เสนอไว้ (ยังไม่ได้ทำ) ให้มี pass ตรวจไขว้เพื่อ QA: ขอ output เป็น bbox มาพร้อมกับ grid_ref แล้วแปลง grid_ref→pixel ด้วย gridmaster ที่มีอยู่แล้ว แล้วให้ระบบเตือนอัตโนมัติเมื่อจำนวน bbox ไม่ตรงกับจำนวนใน elements[]

กฎที่ได้: output ของ VLM ที่เป็นจำนวนนับ ต้องเอาไปตรวจไขว้กับแหล่งนับอิสระอีกแหล่งเสมอ (classical detection, output แบบ bbox หรือตารางที่พิมพ์อยู่บนแบบ) ห้ามเชื่อมั่นลอยๆ เด็ดขาด

plan_beam recall ยังคงอยู่ที่ 0% — เป้าหมายหลักทั้งหมดของรอบ t03 ทำไม่สำเร็จ

เกิดอะไรขึ้น: ผลของบ้าน 08 ในรอบ t03: JSON ถูกต้อง 32/33 (97%), id-recall รวม 10/81 = 12.3% (ขึ้นจาก 3.5% ของ t02) โดย schedule 21.9% และ plan_slab 20% ดีขึ้นจริง แต่ plan_beam ได้ 0/15 = 0% เท่ากับ t02 เป๊ะ บนหน้า 20 โมเดลพ่นออกมาแค่ metadata แล้วปิด JSON เลยโดยไม่มี elements[] สักตัว แกม metadata ยังชี้ไปที่บ้านผิดหลัง (`source\_image` อ้างถึงบ้าน 22) คือมันแปะข้อความสำเร็จรูปที่ห้องจามาจากตัวอย่างที่คล้ายกัน

ต้นเหตุจริง: ตัวอย่างเทรน plan_beam 149 ตัวอย่างยังไม่พออยู่ดี และความล้มเหลวนี้เป็นเรื่องเชิงสถาปัตยกรรม (การนับที่หลอน) ไม่ใช่ปัญหาที่ขึ้นถอดรหัส grammar แก่เรื่องไวยากรณ์ได้ แต่ไม่ได้แก้เรื่องการดึงเนื้อหากลับมา — โมเดลผลิต JSON ที่ถูกต้องแต่ข้างในไม่มีอะไรเลย

เสียหายอะไร: เป้าหมายหลักของรอบนี้ล้มเหลว ที่แย่กว่านั้นคือ ตัวเลข JSON ที่ถูกต้องขึ้นสูง ในขณะที่ตัวชี้วัดเป้าหมายยังเป็นศูนย์ ซึ่งเป็นหน้าตาของผลลัพธ์ที่คนอ่านผิดเป็นความก้าวหน้าได้ง่ายมาก

แก้ยังง: ยังไม่ได้แก้ บันทึกไว้ตรงๆ ทางที่ระบุไว้ว่าจะไปต่อคือ เพิ่มข้อมูลผ่าน op04 หรือใช้แนวทาง grounding/bbox↔gridmaster และระบุชัดว่าไม่ใช้การไปแต่งคำตอนถอดรหัสอีกแล้ว

กฎที่ได้: ตัวเลข JSON ที่ถูกต้องขึ้นได้ ในขณะที่ตัวชี้วัดที่สำคัญจริงๆ ยังเป็นศูนย์ ให้รายงานตัวชี้วัดเป้าหมายแยกออกมาเสมอ และอย่าปล่อยให้ชัยชนะระดับรูปแบบมาแทนที่ชัยชนะระดับเนื้อหาในบทสรุปเด็ดขาด

วิกฤต 2026-09-01

Purson model died because a single missing line (crop_to_patches) shrank every drawing to a 256-token stamp — third occurrence of the same bug class (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The Purson model failed because `crop\_to\_patches` was missing — one line — so an A1-size drawing was downscaled to a stamp at 256 tokens per image. Documented with three layers of evidence, and explicitly noted as the third time this same bug class had occurred.

ต้นเหตุจริง: Image preprocessing silently degraded rather than erroring: the pipeline accepted a drastically under-resolved image without complaint. There was no assertion on token count or effective resolution per image, so the same class of failure had recurred twice before undetected.

เสียหายอะไร: A whole fine-tuned model was worthless. Training compute and rental cost fully burned. Third repeat of the same failure class.

แก้ยังไง: Root-caused and documented in `workmen's\_diary/สอนมะขาม\_ทำไม\_Purson\_กับ\_Voldemort\_พัง\_2026-09-01.md`.

กฎที่ได้: Add a hard assertion on tokens-per-image / effective resolution before any training run starts — a bug class that has now recurred three times needs a guard, not another root-cause writeup.

วิกฤต 2026-09-01

Voldemort model was blinded by 4-bit quantisation of the ViT — reported seeing Barbie and BB-8 in construction drawings (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Voldemort failed for a different reason than Purson: image tiling was correct, but 4-bit quantisation destroyed the ViT to the point of blindness. It answered that it saw Barbie and BB-8 in a construction drawing.

ต้นเหตุจริง: Quantisation applied uniformly across the model including the vision tower, where 4-bit precision is not survivable. No per-component precision policy and no visual sanity check after quantising.

เสียหายอะไร: A second fine-tuned model rendered useless — more burned training/rental cost.

แก้ยังไง: Root-caused and documented in the teaching document.

กฎที่ได้: Never quantise the vision tower to 4-bit. After any quantisation, run a trivial visual sanity probe ('what is in this image?') before spending money on evaluation or training.

The house01 simulation was unintentionally cheating in two ways, making its results not representative of Destrier (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: When Makham asked whether the JSON used for the house01 simulation could really be trusted as Destrier-equivalent, two forms of unintentional cheating were found: (1) it mapped `pattern` → subtask directly (e.g. etc_plan → plan_slab always) whereas real pass0 must inspect element_type; (2) it returned the whole GT file back verbatim, including fields real training strips (`element\_name\_assigned`) and element types the subtask model was never taught to answer.

ต้นเหตุจริง: The simulator was built to produce plausible output rather than to replicate the real pass0/dataset-filter code paths. It bypassed `build\_dataset\_t03.py`'s per-subtask element_type filtering entirely, so it modelled a far more capable system than actually exists.

เสียหายอะไร: The impressive house01 numbers (62 files, 153 elements, \$144,704 BOQ) reflected an idealised system, not the real one. A rebuild (`sim\_house\_from\_labels.py`) showed reality: pass0 only auto-answers 31 of 72 pages, and only 24 real pass2 jobs result.

แก้ยังไง: Built `sim\_house\_from\_labels.py` following real `pass0\_labels.jsonl` and mirroring `build\_dataset\_t03.py::PLAN\_SUBTASKS` filtering exactly.

กฎที่ได้: A simulator must import or exactly mirror the real code paths, not re-implement 'what it probably does'. Any shortcut in a simulator becomes an overstatement in its results — and someone will quote those numbers.

ถ้าแบ่ง train/val แบบสุ่มรายหน้า ข้อมูลจะรั่วข้ามฝั่ง

เกิดอะไรขึ้น: มีการตั้งว่าการแบ่ง train/val สำหรับ fine-tune ต้องแบ่งตามบ้าน ไม่ใช่สุ่มเลือกทีละหน้า เพราะหน้าจากบ้านหลังเดียวกันใช้กริด mark และสเปกเหมือนกัน

ต้นเหตุจริง: หน่วยตัวอย่างตามธรรมชาติของชุดข้อมูลคือ 'หน้า' การสุ่มแบ่งแบบไม่คิดอะไรเลยจึงเป็นการแบ่งรายหน้า ซึ่งทำให้แบบที่แทบจะเหมือนกันจากบ้านหลังเดียวกันไปอยู่ทั้งสองฝั่งของการแบ่ง

เสียหายอะไร: จับได้ก่อนที่จะใช้จริง ถ้าแบ่งรายหน้าจะได้ค่า validation ที่สวยเกินจริง ซึ่งจะบังปัญหาการ generalise ของโมเดลไว้

แก้ยังไง: แบ่งตามบ้านตามที่สั่ง (train = 01+02+04, val = 03) และแนะนำให้ใช้ 4/1 หรือ 5-fold ไปก่อน แล้วค่อยเปลี่ยนเป็น 80/10/10 เมื่อมีบ้านที่ label แล้ว 8-10 หลัง

กฎที่ได้: แบ่งข้อมูลสำหรับประเมินผลตามหน่วยที่ใหญ่ที่สุดที่ข้อมูลข้างในสัมพันธ์กัน (ในที่นี้คือ 'บ้าน') ไม่ใช่ตามหน่วยตัวอย่าง ถ้า val set รั่ว จะได้ตัวเลขที่คุณจะเชื่อทั้งที่ไม่ควรเชื่อ

สูง 2026-07-21

Tuned model still finds only 9.4% of drawing elements — recall unfixed (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Post-tuning evaluation showed a big win on format (JSON valid 0%→90%, view exact match 0%→60%) but element recall of only 16/170 (9.4%), i.e. the model still misses the great majority of rebar/beam/column items.

ต้นเหตุจริง: The training set is small (226 examples from 3 houses) and the run used a reduced LoRA rank; format is far easier to learn than exhaustive element enumeration, so a headline 90% can mask the metric that actually matters for takeoff.

เสียหายอะไร: The tuned model is not yet usable for quantity takeoff; a further tuning round is required — and the artifacts that would have supported it were subsequently lost.

แก้ยังง: Not fixed — recorded as the target for the next tuning round (more data / more epochs / prompt adjustment).

กฎที่ได้: Judge a fine-tune on the metric that carries the business value (element recall), not the easiest-to-move metric (JSON validity); report both together so a headline number never implies readiness.

สูง 2026-07-21

GGUF shipped text-only because the vision projector (mmpoj) could not be exported (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The GGUF conversion succeeded for the language part but the mmpoj (image encoder projection) could not be split out — llama.cpp itself warned it only supports certain models. The resulting artifact could not read drawings, which is the entire purpose of the project.

ต้นเหตุจริง: The export path's support for this very new model family was never verified before the end of the pipeline; the vision half of the deliverable was assumed to follow the text half.

เสียหายอะไร: The day's deliverable was unusable for the actual task; discovered only at the very end of the run.

แก้ยังง: Three days later realized the fix was free: since training froze vision layers (finetune_vision_layers=False), the upstream vendor's mmpoj (899MB) is byte-equivalent and can simply be downloaded and paired with the tuned GGUF.

กฎที่ได้: Verify the hardest/most novel step of an export pipeline first, on a throwaway model, before spending a full training run on it — and check whether a frozen component can simply be reused from upstream instead of re-derived.

สูง 2026-07-24

Fine-tuned model reaches only 28.2% element recall while over-predicting 78 elements (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: After the reasoning-mode fix, the measured results on `--limit 20` were JSON valid 90%, view exact match 70%, element recall 28.2%, element_id/count exact 100%, element_type exact 87.5% — with 78 elements over-predicted.

ต้นเหตุจริง: The training set is small (315 train examples from five small houses) and the element-level task is the hardest part of the schema; the model both misses most elements and hallucinates extras.

เสียหายอะไร: The model cannot be trusted to produce production raw JSON — over 70% of elements missing plus fabricated ones. Directly drove the decision that its output is a human-reviewed draft only.

แก้อย่างไร: Not fixed — recorded honestly as the measured state; more labelled houses is the path forward.

กฎที่ได้: Report the weakest metric alongside the headline improvement. A 3x recall improvement that lands at 28% is still unusable, and over-prediction must be measured as its own number, not folded into recall.

สูง 2026-07-25

Proposed multi-pass call split would have silently discarded the fine-tuning advantage (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A proposal to split extraction into five sub-calls like the production Constant pipeline was analysed and rejected: the model was trained on one call per page, so the sub-tasks (survey ID, counting) were never seen during training and would be handled by base Qwen — implicitly giving up the tuning benefit. It would also cost ~15 minutes per page just re-processing the image five times.

ต้นเหตุจริง: The inference calling pattern was being designed independently of the training data format; a fine-tuned model only carries its advantage on the task shape it was trained on.

เสียหายอะไร: Caught in analysis before implementation — would have spent hours per house to get base-model quality at 5x the image-processing cost.

แก้อย่างไร: Concluded: to use the tuned model, one call per page (fix truncation with a right-sized ctx, accept slowness); to go multi-pass, either use the existing cloud pipeline or retrain with multi-pass-formatted data.

กฎที่ได้: Training format must match the calling pattern. Changing how a fine-tuned model is invoked silently reverts it to base-model behaviour, with no error to tell you.

สูง 2026-07-28

House 07 EE-05/06: four TV outlets missed entirely, despite a 4-way splitter on the same sheet stating the count (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Four TV outlet symbols (⊙) had not been extracted at all. The same sheet carries a 4-way splitter diagram whose fan-out matches the four outlets exactly — the evidence was already on the page.

ต้นเหตุจริง: Extraction read symbols in isolation without using the sheet's own internal consistency checks (a splitter's port count constrains the number of outlets it feeds).

เสียหายอะไร: Four fixtures missing from the electrical takeoff, understating quantities and cost.

แก้ยังไง: Outlets recovered and cross-verified against the splitter diagram.

กฎที่ได้: Use a sheet's own internal arithmetic as a completeness check — splitter ports, panel schedules, circuit counts all constrain how many downstream items must exist. If the counts disagree, something was missed.

สูง 2026-07-29

Model trained on 512px images (~266 visual tokens) but served 3,796 tokens/image — a 14.3× mismatch (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: llama-server's log reports $2048 + 1748 = 3,796$ visual tokens per image at inference, while the collator bug shrank training images to $512\text{px} \approx 266$ tokens. The model was asked at inference to read inputs 14.3× denser than anything it practised on.

ต้นเหตุจริง: The collator silently used a default image size because the model has no default declared; training resolution was therefore never a deliberate choice, and nobody compared it against what the serving stack would feed in.

เสียหายอะไร: A large fraction of tonight's poor extraction quality is plausibly attributable to this alone; knowledge learned at training resolution is largely unusable at serving resolution.

แก้ยังไง: not fixed — mitigation identified (--image-min-tokens 1024, free to try); a retrain with `resize="max"` is the fallback

กฎที่ได้: Log and assert the exact preprocessed input shape at both training and inference, and fail loudly if they differ. Train/inference mismatch is invisible in both logs individually — it only appears when the two numbers are put side by side.

สูง 2026-07-29

Quantized to Q4_K_M despite the vendor's own warning, then never re-measured (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: t01_workflow.md records that Unsloth warns this model family 'loses quality abnormally when quantized' — which is precisely why bf16 training was mandated and 4-bit banned. At deployment time the model was compressed 70GB → 21GB Q4_K_M anyway (an 8GB laptop cannot hold 70GB) and accuracy was never re-measured after compression.

ต้นเหตุจริง: A constraint was enforced rigorously for training and then silently dropped for inference, because nobody treated 'putting it to use' as a stage that could change quality.

เสียหายอะไร: An unknown share of tonight's quality collapse may be quantization damage, and there is no measurement to separate it from the merge bug or the resolution mismatch.

แก้ยังไง: not fixed — needs a controlled A/B against the unquantized model

กฎที่ได้: A quality constraint applies to the whole pipeline, not the training step. When you knowingly violate one out of necessity, book the re-measurement as a required task in the same breath — an unmeasured deliberate compromise becomes an unknown a week later.

สูง 2026-07-29

Train/inference mismatch: model trained on ~266 visual tokens/image, used at 3,796 (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: llama-server logs showed 3,796 visual tokens per image at inference, while the collator bug had trained the model at ~266 — a 14.3× difference between what it learned on and what it is shown in production.

ต้นเหตุจริง: The training-side image pipeline (Unsloth collator default) and the inference-side pipeline (llama.cpp) were configured independently and never compared. The training side was silently downscaling.

เสียหายอะไร: Whatever the model learned may not transfer at all at production resolution; a plausible secondary cause of the poor real-world extraction quality on 07-29.

แก้ยังไง: Not fixed. Identified --image-min-tokens 1024 as a free first thing to try; retraining with resize="max" is the fallback.

กฎที่ได้: Assert that inference-time preprocessing matches training-time preprocessing exactly — capture the actual token/pixel counts on both sides and compare numbers, not settings.

สูง 2026-07-29

Quantized to Q4_K_M against the vendor's own warning, and never re-measured accuracy afterwards (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Unsloth warns that this model family degrades abnormally when quantized — which is exactly why training was forced to bf16 with 4-bit forbidden. Yet for deployment the model was compressed 70GB → 21GB as Q4_K_M (necessary for an 8GB laptop) and accuracy was never measured again after compression.

ต้นเหตุจริง: The vendor caveat was applied to the training phase where it was first read, and not carried forward to the deployment phase where it applies just as much. The compression was driven by a hardware constraint, and the constraint was allowed to substitute for a quality decision.

เสียหายอะไร: The artifact actually in daily use has unknown accuracy; a plausible contributor to the poor 07-29 output that could not be ruled in or out.

แก้ยังไม: Not fixed. Recorded as a secondary cause; re-measuring the quantized artifact remains outstanding.

กฎที่ได้: A quality caveat about a model applies to every phase, not the phase you first read it in. Any lossy transform on the shipping artifact requires a re-run of the accuracy eval on that exact artifact.

สูง 2026-07-29

Model read sheet references (A-05, A-07, A-09) as column marks (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The drawing contains cross-references such as 'see detail on sheet A-05'. The model treated A-05 / A-07 / A-09 as the names of columns and emitted them as element marks.

ต้นเหตุจริง: The model reads Thai text accurately but does not understand the function of the text it reads — a semantic-role failure, not an OCR failure. It reads a pointer as a name.

เสียหายอะไร: Fabricated structural elements carrying sheet numbers as marks, which would flow into BBS/BOQ as real items.

แก้ยังไม: Not fixed. Documented as evidence that the problem is comprehension, not vision.

กฎที่ได้: Validate extracted marks against the project's real mark vocabulary (and reject anything matching the sheet-code pattern) — the model can read text perfectly and still misclassify what it is.

สูง 2026-07-29

Output contradicted itself: two rooms assigned the identical four grid points (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: F6 was given grid points ["C-2", "C-3", "D-3", "D-2"] and named 'เตรียมอาหาร'; F7 was given the same four points in a different order and named 'นอน 2'.

ต้นเหตุจริง: Nothing in the pipeline cross-checks emitted entries against each other; the model generates each entry locally with no global consistency constraint.

เสียหายอะไร: Two mutually exclusive room definitions occupy the same footprint — an error that silently propagates into area and finish takeoffs.

แก้ยังไม: Not fixed.

กฎที่ได้: Run cheap internal-consistency assertions on model output (no two areas share an identical point set, no duplicate marks with different attributes) — these catch failures no per-field accuracy metric will.

สูง 2026-07-29

confidence_score and confidence_flags were emitted 0 times across all 6 files (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The ground-truth schema carries confidence_score and confidence_flags marking passages the reader could not resolve (e.g. 'text too small, needs zoom'). The model produced none across six files.

ต้นเหตุจริง: Those fields exist only in the training answers, never in the prompt text — so a model without the fine-tune loaded has no way to know they should exist. Their total absence is itself corroborating evidence for the missing-LoRA hypothesis.

เสียหายอะไร: There is no way to tell which parts of the extraction to trust and which to re-check, so every value must be treated as unverified — removing the human-review targeting the schema was designed to provide.

แก้ยังไง: Not fixed. Used as evidence in the LoRA-presence diagnosis.

กฎที่ได้: Fields that appear only in training answers and never in the prompt are the best available fine-tune-presence detector — check for them explicitly in the artifact sanity gate.

สูง 2026-07-29

Model swapped both work category and material type: wall/clay brick became ceiling/AAC block (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On page 04 the ground truth is a wall (ผนัง) built of clay brick (อิฐมอญ). The model output a ceiling (ฝ้าเพดาน) built of autoclaved aerated block (อิฐมวลเบา).

ต้นเหตุจริง: The model matched surface tokens without binding material to its host element; nothing validates that a material is legal for the work category assigned.

เสียหายอะไร: Wrong on two dimensions at once — if used for pricing, both the work category and the unit rate would be wrong, so the BOQ error compounds rather than cancels.

แก้ยังไง: Not fixed.

กฎที่ได้: Cross-validate material against work category with a legality table before any extracted line reaches pricing — a two-dimensional error produces a confidently wrong number, not an obviously wrong one.

สูง 2026-08-04

มีคำศัพท์ประเภทหน้าแบบ 6 ชุดที่เข้ากันไม่ได้ ตอบคำถามเดียวกัน — และตอนนี้มี 2 จุดที่ข้อมูลหายจริง

เกิดอะไรขึ้น: คำถามว่า "หน้านี้เป็นหน้าประเภทไหน" ถูกตอบด้วยคำศัพท์ 6 ชุด (schema §1 มี 14/16 คำ, Pass 0 มี 2, CLASSIFY_SHEET/sheet_type มี 5, drawing_type มี 3, classifyPageType() มี 7, schema-v2 ที่ฝึกไว้มี 6) และมี 2 จุดที่ทำข้อมูลหายอยู่ทุกวันนี้: `wz-step1.js:224` ขุด 5 คำให้เหลือ 2 (general_notes/schedule_table/unknown กลายเป็น section_detail หหมด) และ `drawing-index.js:205-208` ฮาร์ดโค้ด section_detail หักคำที่อ่านมาได้จริง

ต้นเหตุจริง: แนวคิดเดียวกันถูกสร้างแบบแยกกัน 6 ที่ตามเวลาที่ผ่านไป โดยไม่มีเจ้าของที่เป็นตัวจริง ผู้ใช้งานใหม่แต่ละรายก็นิยามคำศัพท์ที่ตัวเองต้องการขึ้นมาเองในที่ของตัวเอง

เสียหายอะไร: ข้อมูลการจำแนกหน้าถูกทำลายทุกครั้งที่รันใน 2 จุดที่รู้อยู่ — ประเภทหน้าที่อ่านมาถูกต้องกลับถูกเขียนทับด้วยคำที่ฮาร์ดโค้ดไว้ แกม Constantant ยังเก็บผลลัพธ์แบบ 16 คำไม่ได้เชิงโครงสร้าง จึงเอาตัวจำแนกที่ดีกว่ามาใช้ไม่ได้เลย

แก้ยังง: ยังไม่แก้ — ตัดสินว่าตอนนี้ยังไม่คุ้ม เติมเฉพาะ 2 pattern ที่ขาดจริงๆ เข้า schema ส่วนการขยาย `sheet\_type`/`drawing\_type` บันทึกรู้ว่าเป็นเงื่อนไขที่ต้องทำก่อนถ้าจะมี Pass 0 ตัวใหม่

กฎที่ได้: หนึ่งแนวคิด หนึ่งชุดคำศัพท์ หนึ่งเจ้าของ และห้ามฮาร์ดโค้ดหักคำที่เพิ่งอ่านมาได้ — ถ้าผลที่อ่านมาถูกทั้ง ก็ไม่ควรเสียเงินเสียเวลาไปอ่านตั้งแต่แรก

สูง 2026-08-21

The completed end-to-end loop delivered 21 elements but 0 spans and 0 specs — model quality, not platform code (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: After both adapter bugs were fixed, the Qwen export of house 09 imported successfully and yielded 21 elements including real structural ones (C1 ×8, C2 ×12, B2 ×18, footings, steel trusses) — but span 0/21 and specs 0. The model never emitted grid_ref_start/grid_ref_end (it packs a whole sheet into one string, "F-F x 1'-2', F-F x 2'-3', ..."), and no section or schedule page of the house was ever run through it. Ground truth for the same house through the same adapter gives 87 elements / 55 specs / 18 spans.

ต้นเหตุจริง: Two compounding causes: the model does not emit per-element grid refs on beam plans (the 11%-recall problem, itself caused by ~13 training examples), and the run scope never covered section/schedule pages, which are the only source of specs.

เสียหายอะไร: The 'complete loop' demo produces elements with no geometry and no material specs — the BOQ can count things but cannot price or size them. Honest gap named rather than papered over.

แก้ยังง: Not fixed — recorded as model quality, not platform code. Recommendation: run this house's section/schedule pages through Qwen to at least populate specs and let the BOQ price steel.

กฎที่ได้: When demonstrating an end-to-end loop, state the delta against ground truth through the same code path (21 vs 87 elements, 0 vs 18 spans) — a loop that runs is not a loop that works, and comparing against ground truth through the identical adapter is the only way to separate platform bugs from model quality.

สูง 2026-08-21

On beam plans the model collapses a whole sheet into ~2 zone-notation elements instead of per-beam rows (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Overlay comparison on house 09 หน้า26 showed the model reducing a sheet of 48 real beams to 2 elements, packing the entire page into one string of zone notation ("F-F x 1'-2', F-F x 2'-3', ...") whose boxes float free of any real beam line; หน้า27 could not be placed at all. This is a distinct failure mode from the fence-detail hallucination and from JSON parse failure — the JSON here is valid, the content is structurally wrong.

ต้นเหตุจริง: Too few beam-plan training examples (~13) to teach one-row-per-segment output, so the model falls back on summarising the sheet in a compressed notation it has seen in captions/schedules.

เสียหายอะไร: Even after grammar decoding made 96.8% of pages parse, beam spans remain unusable — the platform import produced span 0/21 against ground truth's 18/87. Beam lengths never reach the BOQ.

แก้ยังไง: Not fixed — deferred to t03's per-pattern subtask split plus more beam-plan training examples.

กฎที่ได้: Valid JSON is not correct content. Measure element-level recall with a visual overlay against ground truth, not parse rate, before declaring an extraction run successful.

สูง 2026-08-24

t01 found 5 beams / 0 columns on a page with 16 beams in GT (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On house 11 page 22, the tuned t01 model reported 5 beams and 0 columns, while the sheet visibly carried 13+ beam labels plus columns and GT records 16 beams. Base and tuned produced the same 6 usable pages, and base was often faster (p21: 121s vs 410s) — likely just less verbose, not smarter.

ต้นเหตุจริง: The same architectural counting hallucination documented on 2026-08-22, now confirmed on our own model and our own data. Fine-tuning did not cure it.

เสียหายอะไร: Confirms the tuned model cannot be trusted for element counts; the tuned-vs-untuned comparison produced no quality win, only a confirmation of the failure mode.

แก้ยังไง: Not fixed — recorded as live confirmation of the 2026-08-22 research; the fix direction (detection-first / GroundCount hint) remains unbuilt.

กฎที่ได้: Do not read a speed difference between base and tuned as a quality signal. Always compare element counts against GT, and treat count accuracy as a separate metric from JSON validity.

Model returned an empty element list with a header copied from a different house (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On hard beam-plan pages the model output only header fields and `"elements": []` — no beams at all. Worse, the header pointed to the wrong house (claimed house 22 while reading house 08).

ต้นเหตุจริง: The model had memorized the output form from training examples and was reproducing it from memory instead of actually looking at the image in front of it — a memorization/hallucination failure, aggravated by the image being downscaled so heavily that the beams were not legibly present.

เสียหายอะไร: plan_beam recall pinned at 0%; the wrong house code in the output would silently corrupt downstream data if any pipeline trusted the header field.

แก้ยังไง: Root-caused as a 'cannot see the object' problem rather than a 'reads the number wrong' problem; led to the pattern-recognition tool and the MAX_PIXELS fix. The hallucinated header itself is not yet guarded against.

กฎที่ได้: When a model outputs a well-formed but empty/mismatched record, treat it as memorization of the template, not as a parsing bug. Add a validation check that the header identity in the output matches the file actually fed in.

Thai-reading ability was never a screening criterion in the model search — both surviving candidates are untested on Thai (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Eight models were screened on size, licence and trainability. Only after the shortlist was down to two (GLM-4.6V, Llama-4-Scout) does the diary note that nobody has tested how well either reads Thai — on a project whose entire input corpus is Thai construction drawings.

ต้นเหตุจริง: Thai capability is the hardest criterion to check (no leaderboard, needs a real trial run), so it was deferred — and deferring it meant it never filtered anything.

เสียหายอะไร: Both survivors could fail on the one requirement that cannot be worked around, making the whole survey's shortlist provisional. Adopting either would risk a full training round spent discovering the model cannot read the language.

แก้ยังไง: not fixed — noted as an open unknown

กฎที่ได้: The requirement that would disqualify a candidate outright belongs in the first filter, not the footnote. A shortlist screened on everything except the deal-breaker is not a shortlist.

สูง 2026-08-26

InternVL3-78B and InternVL3.5-241B look like non-Qwen alternatives but use Qwen as their language backbone (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The search criterion was explicitly 'not Qwen, by brand and by backbone'. InternVL3-78B and InternVL3.5-241B pass the brand test but were found to use Qwen2.5-72B and Qwen3-235B respectively as their brain, so both were disqualified.

ต้นเหตุจริง: Model branding does not reveal architecture lineage; a different vendor name can wrap the exact backbone you were trying to avoid. Only reading the model card's architecture section reveals it.

เสียหายอะไร: Would have selected an apparent alternative that carries the identical dependency and identical failure modes as the incumbent — defeating the whole purpose of the search.

แก้ยังง: Both explicitly disqualified in the catalogue with the reason recorded ('ข้างในใช้ Qwen2.5-72B เป็นสมอง').

กฎที่ได้: When evaluating a model to avoid a specific dependency, check the actual backbone in the architecture section, never the brand name. Record the backbone in your comparison table as a first-class column.

สูง 2026-08-26

Several shortlisted models carry licences that forbid or ambiguously restrict commercial use (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Pixtral-Large ships under the Mistral Research License with no direct commercial use (a separate licence must be bought); MiniMax M3 has nested conditions where fine-tuning and deploying may itself count as commercial; NVLM-D-72B is CC-BY-NC-4.0.

ต้นเหตุจริง: Open weights are routinely conflated with open licences. Restrictions are in the licence text, not in the availability of the download.

เสียหายอะไร: Any of these, if adopted without reading the licence, would have created legal exposure discovered only after the model was embedded in a commercial product.

แก้ยังง: Each licence read and its restriction recorded per row in the catalogue table, with commercial-use marked // explicitly.

กฎที่ได้: Read the full licence text for every candidate model before any spend, and record the commercial-use verdict as a mandatory column. 'Downloadable' never implies 'usable commercially'.

Llama-4-Scout was selected as the only surviving candidate without anyone testing whether it can read Thai (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Llama-4-Scout passed every catalogue criterion (size, licence, confirmed LoRA training) and was named the sole survivor — but the catalogue records that nobody has tested how well any candidate reads Thai, and recommends a zero-shot Thai-OCR test before renting a GPU.

ต้นเหตุจริง: The selection criteria covered size, licence and trainability but omitted the single most task-critical capability: reading Thai construction drawings. The criteria were about feasibility, not fitness for purpose.

เสียหายอะไร: A GPU-rental decision was about to be made on a model whose core required capability was completely unmeasured — the same class of mistake as training t04 without checking image resolution.

แก้ยังไง: Not fixed — flagged as a recommendation ('แนะนำ zero-shot Thai-OCR test ก่อนตัดสินใจลงทุนเช่า GPU จริง') rather than executed.

กฎที่ได้: Include the task-critical capability in the selection criteria itself, and run a free zero-shot test of it before any rental decision. Feasibility criteria (size/licence/tooling) do not measure whether the model can do the job.

CV hint blocks must be barred from containing dimension numbers or they teach the model to violate its own rule (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The pass 2.4 hint contract explicitly forbids putting any dimension numbers into the hint appended to the prompt, because doing so would teach the model to break the standing rule 'never guess dimensions from the image' — with the team's own hand.

ต้นเหตุจริง: An auxiliary channel (the CV hint) can silently contradict a prompt rule if its contents are not constrained. Helper data injected into a prompt is training signal, not neutral context.

เสียหายอะไร: Caught at design time as a contract rule rather than after shipping, so no measured damage — but the failure mode would have been model-wide dimension fabrication that is very hard to attribute.

แก้ยังไง: Written into pass2.4_hint/prompt.md as an explicit prohibition on the hint block's contents.

กฎที่ได้: When injecting auxiliary/tool-derived data into a prompt, explicitly enumerate what it must NOT contain. Auxiliary context that contradicts a stated rule will override the rule.

สูง 2026-08-30

The repo's import fixtures used pattern names that appear in zero real files, so the whole test suite stayed green while the real import path produced nothing (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `raw-extraction-adapter.js` only recognised the spec's pattern names `plan` and `gridline`. Real data across all 50 houses uses `beam\_plan` (85), `footing\_plan` (60), `etc\_plan` (199), `roof\_frame\_plan` (46) and `grid\_master` (42) — `plan`/`gridline` appear in 0 files. The bug survived indefinitely because the fixtures in the repo were written to the spec names too, so every test passed against a code path no real file ever takes. It surfaced only when a real house was pushed through the pipeline in a simulated-user run.

ต้นเหตุจริง: Fixtures were authored from the spec document rather than sampled from the real data corpus, so the test suite validated the adapter against its own assumptions. Spec names and produced names had diverged and nothing compared the two.

เสียหายอะไร: Pressing the real import button produced 0 elements and no grid, silently, for an unknown length of time — the manual-import button was affected too, not just the new Purson path. 1149 tests were green throughout.

แก้ยังไง: `canonicalPattern()` + `PATTERN\_ALIASES` added in the adapter, applied at all 5 comparison sites across 2 files; alias list matched to `build\_dataset\_t03.py`; `tests/raw-extraction-pattern-aliases.test.js` (4 tests) added; suite 1153/1153.

กฎที่ได้: A fixture invented from a spec tests the spec, not the system. At least one test per ingestion path must use a byte-real sample taken from the actual corpus, or a green suite means nothing.

สูง 2026-08-31

Model emitted training-set filenames when reading a brand-new house — overfit memorization (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Output for a completely new house contained `house\_001\_หน้า00\_gridline.json` and `image/house\_001/...` — filenames memorized from the training corpus. Final training losses of 0.10-0.13 are below Unsloth's stated overfit threshold of 0.2.

ต้นเหตุจริง: Too many epochs and too little house diversity relative to model capacity, producing memorization rather than generalization.

เสียหายอะไร: Output for unseen houses is contaminated with fabricated references to training data, which would corrupt any downstream consumer.

แก้ยังไง: Not fixed this round — flagged as a required change for the next training round (reduce epochs, increase house diversity).

กฎที่ได้: Track training loss against the overfit threshold during the run, and test on a genuinely out-of-corpus house before declaring a model usable — memorized filenames in output are a definitive overfit signal.

สูง 2026-08-31

All six SP precast slab panels were missed entirely on the beam plan (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On the scored beam page the model produced no SP element at all — all six precast floor panels were absent, alongside other errors (row B all labelled B2 when B3 should appear, B3 in the wrong orientation, B0 classified as slab instead of beam).

ต้นเหตุจริง: Unknown — flagged as a likely systematic gap rather than noise, since a whole element class disappeared rather than individual instances.

เสียหายอะไร: Precast slabs are required for BOQ; a whole element class missing means quantities would be silently short.

แก้ยังไง: Not fixed — listed as the second-highest priority next action, to determine whether it is a systematic gap.

กฎที่ได้: When an entire element class is missing rather than individual instances, treat it as a systematic gap and investigate before shipping — silent zero for a BOQ-relevant class is worse than a wrong number.

สูง 2026-08-31

Model returned zero elements with confidence 1.0 on a page with six clearly drawn footings (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: In the greedy round, page 26 (pile-cap plan) came back as elements: [] with confidence 1.0 and a flag claiming footing marks were present but no footings were drawn — while six footings were plainly visible.

ต้นเหตุจริง: Nothing in the schema or serving path ties the confidence field to evidence; the model is free to assert full confidence in an empty result, and downstream code has no rule that empty-plus-confident is suspicious.

เสียหายอะไร: A confidently empty page is worse than an unknown one — a downstream consumer would take it as "this plan has no footings" and drop 6 real elements from the takeoff.

แก้ยังไง: not fixed for the confidence field itself; the underlying emptiness was fixed by switching off greedy decoding (page 26 then read 100% correct)

กฎที่ได้: Treat high confidence on an empty result as a validation failure, not as data — a model that says "nothing here, certainly" needs a guard.

สูง 2026-08-31

Model invented column positions by inferring them from beam intersections, breaking the explicit no-guessing rule (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: In the page-27 output the model emitted columns with its own comment "Column location inferred from beam intersections" — deriving element positions that were not read from the drawing, which the prompt rules forbid.

ต้นเหตุจริง: The no-guessing rule exists in the prompt but nothing at generation or ingest time detects or rejects an inferred element; the model even labelled the violation and it still passed through.

เสียหายอะไร: Fabricated structural elements would enter the takeoff as if read from the drawing; caught only by reading the raw output by hand.

แก้ยังไง: not fixed — no validator rejects self-declared inferences

กฎที่ได้: When the model announces it is guessing, that is a machine-readable signal — build a check for it instead of relying on the prompt rule alone.

สูง 2026-09-01

t04 memorised training-set answers instead of learning to read drawings, and this was mistaken for competence (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Because the input images were unreadable, the model could only learn the shape of the expected answer and recall the answers of houses it had seen during training. On any unseen house it fabricated output immediately.

ต้นเหตุจริง: Unreadable inputs plus a loss signal that still rewards producing the right-looking JSON means memorisation is the only learnable strategy. The training setup made cheating the optimal policy.

เสียหายอะไร: An entire model whose apparent capability was pure train-set recall — worthless on new houses, which is the only case that matters in production.

แก้ยังไง: Diagnosed by testing on unseen houses and by inspecting actual outputs rather than aggregate scores.

กฎที่ได้: Always hold out unseen data and check performance there specifically. If a model scores well on seen data and collapses on unseen data, suspect the inputs are unusable and it is memorising labels, not learning the task.

สูง 2026-09-01

t04's k-fold split was made blind to per-pass data coverage, leaving zero pass0 eval houses in every fold (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The 40 houses were split into folds without checking that only about 10 houses carried pass0/pass2.4/pass3 data (pass0 existed for only 5 houses). The random split pushed almost all of those special houses into the training side. Counting the files showed every fold had 204 pass1 eval houses but 0 pass0 eval houses, and only 0–9 for pass2.4 and 0–2 for pass3.

ต้นเหตุจริง: The split was stratified on house count only, treating all houses as interchangeable, when in fact the data is sparse and unevenly distributed per pass. Nobody counted per-pass file coverage before splitting.

เสียหายอะไร: Even after a successful training run it would have been impossible to measure whether the model learned pass0 or pass3 at all. Reusing these folds would have meant training and still not knowing the answer.

แก้ยังไง: Caught by Makham directly ('อย่าเอา fold ของ t04 ไปใช้') and confirmed by counting the actual files; the t04 fold layout was rejected for reuse.

กฎที่ได้: Before splitting data into folds, count coverage per sub-task and stratify on the sparse dimension, not just on the total item count. Verify every fold has non-zero eval examples for every capability you intend to measure.

สูง 2026-09-01

Model-soup merge nearly done without a shared LoRA A initialisation (same-basin requirement) (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The plan to average 4 fold models into one 'model soup' at 0.25 weight each only works if every model starts from the same initialisation. The team came close to merging models that did not share a common LoRA A init — recorded as a near-miss in the 30 Aug diary.

ต้นเหตุจริง: The same-basin precondition for weight averaging is a silent mathematical requirement, not something any tool enforces. Merging models from different initialisations produces a valid-looking file that is numerically meaningless.

เสียหายอะไร: Would have produced a merged model that looked fine but was effectively noise, with no error at any stage to reveal it.

แก้ยังไง: Caught before merging; the shared LoRA A initialisation requirement was documented as a lesson in the 30 Aug diary.

กฎที่ได้: Before averaging any set of model weights, verify programmatically that all members share the same initialisation. Weight merging has silent preconditions — assert them, do not assume the training script preserved them.

สูง 2026-09-02

A page containing both footing and beam plans failed generation at both attempts even with a 40-minute ceiling (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Even after raising the timeout to 40 minutes, page 12 (footing + beam on one sheet) still failed in both views. Reading the raw text showed two distinct failures: for the footing the model had already produced all 7 elements correctly and then looped repeating the same explanatory sentence endlessly; for the beam it was genuinely still producing real content (64 segments) and got cut 144 characters from the end.

ต้นเหตุจริง: Content density on a single sheet exceeds what one generation call can complete. Two different failure modes were being lumped together: a degenerate repetition loop (which more time cannot fix) and a genuine length overflow (which more time only barely fixes). No repetition penalty / loop detection existed, and no per-sheet content splitting.

เสียหายอะไร: Repeated failures across multiple houses (house09 page 9 hit the same thing), each costing a re-run or manual salvage. Not solvable by timeout tuning.

แก้ยังไง: Recovered with a purpose-written `salvage\_json.py` (bracket-depth walker finding a safe cut point and closing the JSON, without inventing values). Both files recovered fully.

กฎที่ได้: Distinguish 'ran out of time' from 'stuck in a repetition loop' — they need different fixes (splitting vs a repetition penalty/stop condition). For dense multi-view sheets, crop into separate views before generation rather than raising the timeout again.

สูง 2026-09-02

House09 page 9 (beam plan + roof truss overlaid) truncated again — the same dense-page failure recurring (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: During the house09 run, page 9 (beam plan plus roof truss, two views overlaid) was cut off mid-generation — the identical failure mode as house2's page 12, on a different house.

ต้นเหตุจริง: The dense multi-view sheet problem was never fixed at the source (no cropping), only made survivable by salvage. So it recurred on the very next house.

เสียหายอะไร: Another manual salvage step in a run that was supposed to be timed as a clean end-to-end benchmark; 22 elements had to be recovered by script.

แก้ยังไง: Recovered with `salvage\_json.py` (all 22 elements).

กฎที่ได้: A failure that recurs on the next dataset is not an edge case — it is the normal case. Fix dense multi-view sheets at the cropping stage before running more houses through the same trap.

เรื่องที่เหลือในหมวดนี้ (23)

Tried to quantize from an already-quantized GGUF; had to redo the conversion from f16

กลาง 2026-07-21

Always keep and convert from the full-precision export; never derive one quantization from another.

Nearly used the production app prompt, which targets a different schema than the model was trained on

กลาง 2026-07-25

Pin the exact prompt a model was trained with next to the model artifact. When multiple prompt families exist for different schemas, an unlabelled prompt choice is a silent correctness bug.

Planned to run the tuned model on a big house, a building type entirely absent from training

กลาง 2026-07-25

State the in-distribution boundary of a model before anyone runs it on new data, and plan dataset coverage against the building types you actually intend to process — not against whatever was labelled first.

Proposed stuffing the entire 6k-token schema document into the inference prompt

กลาง 2026-07-25

For a fine-tuned model the prompt is part of the trained artifact. Any prompt that differs from the training prompt — richer or poorer — discards part of the fine-tune.

Time spent evaluating SAM 3 / Grounding DINO / YOLO-World before realising they are the wrong tool class

กลาง 2026-08-04

Write the task down as an input→output contract before shortlisting models. If the required output is an identifier joined to a table value, it is not a detection problem no matter how visual it looks.

t02 bug 1: the token budget was computed too early, against numbers that then changed

กลาง 2026-08-04

Do not compute a derived plan number until its inputs are frozen. Sequence planning so budget/sizing comes after dataset finalisation, never before.

โค้ดที่ปรับให้เร็วขึ้นด้วย text layer ของ PDF ไม่เคยทำงานเลย เพราะ PDF ทุกไฟล์เป็นเส้น vector ไม่มีตัวอักษรสักตัว

กลาง 2026-08-04

ก่อนสร้างโค้ดปรับความเร็วบนข้อสันนิษฐานเรื่องรูปแบบไฟล์ ต้องเอาไฟล์จริงมาลองก่อนว่าจริงไหม แล้วใส่ log ไว้ด้วยว่าสิ่งที่ปรับความเร็วนั้นได้ทำงานจริงหรือเปล่า ของที่ประหยัดได้แต่ไม่มีใครวัด ก็เท่ากับไม่ได้ประหยัดอะไรเลย

pattern สามคลาสมีตัวอย่างน้อยมาก แค่ 13, 12 และ 5 ไฟล์จากบ้าน 12 หลัง

กลาง 2026-08-04

ก่อนตัดสินใจว่าจะเทรนแยกเป็นรายคลาส ให้นับตัวอย่างต่อคลาสก่อน คลาสที่มีแค่ไฟล์เดียวต่อบ้าน ต้องวัดด้วยจำนวนบ้าน ไม่ใช่จำนวนหน้า ถึงจะรู้ว่าพอเทรนได้หรือยัง

Pass 3 จูนไม่ได้ เพราะ schema เก็บคานแยกทีละช่วง แต่ Pass 3 ต้องการคานที่จัดกลุ่มตามความยาว

กลาง 2026-08-04

ก่อนวางแผนจูน pass ไหน ให้เช็คก่อนว่าสร้าง dataset ในรูปแบบผลลัพธ์ของ pass นั้นได้จริงจากข้อมูลที่เก็บไว้หรือเปล่า ถ้ารูปแบบไม่ตรงกัน นี่คือการ dataset ไม่ใช้งานเทรน

โมเดลตัดตัวอักษรนำหน้าของ element_id ทั้งเป็นระบบในหน้าตาราง (D1 กลายเป็น 1)

กลาง 2026-08-20

ถ้าต้องคลายเกณฑ์การจับคู่ให้หลวมลง eval ถึงจะให้คะแนนได้ ความหลวมนั้นคือสิ่งที่ค้นพบในตัวเอง ให้บันทึกไว้ว่าเป็นข้อบกพร่องของโมเดล แล้วไปเช็คข้อมูลเทรนสอนมันแบบนั้นหรือเปล่า อย่าไป normalize ทั้งเสียๆ

โมเดลมโนรายละเอียดรัวในหน้า 16/17 ออกมาเป็นเสา ฐานราก และคานคอดิน

กลาง 2026-08-20

การสกัดข้อมูลรวดเร็วโดยไม่มีด้านคัดชนิดหน้า จะทำให้โมเดลแต่งโครงสร้างขึ้นมาบนแผนใดก็ตามที่ดูเหมือนงานโครงสร้าง ให้จำแนกชนิดหน้าก่อนสกัด และให้ถือว่าของที่มโนขึ้นมาอันตรายกว่าของที่อ่านตกไป เพราะมันจะไหลไปคิดเงินใน BOQ

ยังมีหน้าหนึ่งที่ใช้ **grammar-constrained decoding** แล้ว โมเดลลูปอยู่ข้างใน **string**

กลาง 2026-08-21

grammar-constrained decoding ปิดได้แค่ความพึงเชิงโครงสร้าง ให้เก็บกลไก parse แล้ว retry ไว้สำหรับที่เหลือ และอย่าคาดหวังว่า no_repeat_ngram จะจับลูปที่หมุนเปลี่ยน token ไปทีละตัวได้

Three pattern-recognition approaches on section sheets all failed; one looked perfect on the first house tested

กลาง 2026-08-25

A CV heuristic that works perfectly on the first sample proves nothing — test cross-source before believing it, and write failed experiments down with their numbers so the next person does not spend the same day.

NVLM-D-72B cannot be run at all — even vLLM cannot serve it, as NVIDIA itself admits

กลาง 2026-08-26

Confirm both a permissive licence AND a working inference path before shortlisting any model. Published weights prove neither.

The Thai→English glossary could teach the model to translate labels it is forbidden to translate — mitigated only by where it sits in the prompt

กลาง 2026-08-26

When a new prompt block contradicts an existing rule, prompt adjacency is a hope, not a guard. Add an explicit check on the next eval: do verbatim label fields still match GT character-for-character?

อักขระ Markdown ที่ตกค้างอยู่ใน prompt ทำให้โมเดลหลอน

กลาง 2026-08-29

prompt คือ input ของโมเดล ไม่ใช่เอกสารให้คนอ่าน ลอกอักขระจัดรูปแบบออกก่อนส่งเสมอ และแยกฉบับที่คนอ่านออกจากข้อความ prompt ตัวจริงที่ใช้งาน

กฎข้อ 4 ของ prompt pass3 ห้ามคานแนวนอนใช้ cv_mark ร่วมกัน ซึ่งผิด

กลาง 2026-08-31

เขียนกฎที่คุยกับตัวตรวจจับแยกตามคลาสเรขาคณิตของชิ้นส่วน (จุด vs เส้น) และดูก่อนว่าตัวตรวจจับตีกรอบแต่ละคลาสนี้ยังไ้ก่อน จะเขียนเงื่อนไขว่าห้ามซ้ำ

The pass system misclassified beams B1/B2/B2A/B4 as columns where the raw-power run got it right

กลาง 2026-08-31

A subtask hint that improves recall can also bias classification — check element-type accuracy separately from recall when evaluating a guided pipeline.

enable_thinking=False did not stop chain-of-thought from leaking into the JSON answer

กลาง 2026-08-31

A config flag that turns off a channel is not a guarantee about content; validate the output shape rather than trusting the flag.

Beam-plan reading errors beyond the missing slabs: row B all labelled B2, B3 drawn on the wrong axis, B0 classified as a slab

กลาง 2026-08-31

Record per-item error modes, not just an aggregate recall number — "row B all B2" and "B3 on the wrong axis" point at different training fixes than a low score does.

pass3 finds zero CV anchors on steel C-channel drawings — an entire drawing style it can never measure

กลาง 2026-09-02

Before claiming a measurement stage 'works', name the drawing classes it was built from — a CV vocabulary trained on one construction style returns an honest zero on another, and zero coverage is easy to mistake for a bug or for a pass.

Near-miss: translating the whole prompt set to English would have cost ~8% accuracy

ต้า 2026-08-25

For cross-lingual prompting, insert a term glossary rather than translating the prompt wholesale — and confirm which concrete problem a proposed change is meant to fix before building it.

The Thai→field dictionary was placed too late in the prompt and had to be moved to the top

ต้า 2026-08-29

Put definitions, vocabularies and constraints at the top of a prompt, before the task and the data. Prompt ordering is a functional parameter, not a formatting choice.

บั๊กในโค้ด และ ตรรกะผิด

บั๊กธรรมดา: สูตรผิด เงื่อนไขผิด นับเกิน/ขาด regex ผิด หน้าจอพัง

ทั้งหมด 53 เรื่อง · วิกฤต 6 · สูง 14 · กลาง 23 · ต่ำ 10

เรื่องที่ต้องอ่านให้ครบ (20)

วิกฤต 2026-07-28

บั๊กที่ 4: สิม `enable_thinking=False` อีกแล้ว — บั๊กตัวเดียวกับที่เคยทำให้ t01 รัดได้ 0/3

เกิดอะไรขึ้น: `train_qwen3vl.py` ไม่มี `enable_thinking=False` ไดอารี่ระบุตรงๆ ว่าเป็นบั๊กตัวเดียวกับที่ทำให้ t01 รัดได้ 0 จาก 3 คือเป็นความล้มเหลวที่เคยรู้แล้วในอดีต ถูกนำกลับมาใหม่ในไฟล์ใหม่แบบเป๊ะๆ

ต้นเหตุจริง: ความล้มเหลวที่เคยวินิจฉัยไว้แล้ว ถูกแก้แค่ในสคริปต์เดียว และไม่เคยถูกบันทึกเป็นค่าที่ต้องมีใน config ที่ใช้ร่วมกัน ใน template หรือในตัวตรวจ — พอเขียนไฟล์ใหม่จากศูนย์ ก็เลยทำซ้ำแบบเดิมเป๊ะ

เสียหายอะไร: จะได้การวัดผลที่ได้ศูนย์อีกรอบ เสียทั้งเงินค่าเช่าและเวลาวิเคราะห์ที่ตามมา จับได้ก่อนเข้าเครื่อง

แก้ยังไง: เพิ่ม `enable_thinking=False` เข้าไป

กฎที่ได้: บั๊กทุกตัวที่เคยทำให้ได้คะแนนศูนย์ ต้องกลายเป็น assertion ในตัวตรวจ environment ไม่ใช่แค่แก้ในไฟล์เดียว ถ้าความล้มเหลวเก่ากลับมาได้แค่เพราะเขียนสคริปต์ใหม่ แปลว่ามันยังไม่เคยถูกแก้จริง

วิกฤต 2026-08-21

greedy decoding ที่ไม่มีตัวกันการวนซ้ำ ทำให้โมเดลวนจนชน token cap พังทั้งหน้า

เกิดอะไรขึ้น: หน้า beam plan ของบ้าน 08 (หน้า20, S-03) parse ไม่ผ่าน ต้นเหตุคือ `run_house_batch.py` กับ `infer_t02.py` ใช้ greedy decoding ล้วน (`do_sample=False`) โดยไม่มีตัวกันการวนซ้ำ โมเดลเลยวนพ่นคำจน B2 ปลอมๆ เข้าไปเกือบทุกช่อง กริดจนชน `max_new_tokens` แล้วไม่เคยปิด JSON ได้หน้านั้นมีคำจริง 31 ตัว หายหมดทั้ง 31 ตัวเพราะวงเล็บไม่เคยปิด

ต้นเหตุจริง: การถอดรหัสแบบ autoregressive จะยังเสริมตัวเองเมื่อเจอเนื้อหาซ้ำๆ — ทุกครั้งที่ซ้ำ หลักฐานที่จะซ้ำอีกยิ่งมากขึ้น จน stop token ไม่มีวันชนะ greedy decoding ที่ไม่มีทั้งบงกชโทษและตัวบังคับโครงสร้าง จึงพังแน่นอนบนหน้าที่มี element เยอะ และ JSON เป็นแบบได้หมดหรือไม่ได้เลย ผลถูกบางส่วนจึงมีค่าเป็นศูนย์

เสียหายอะไร: หน้าที่มี element เยอะ ซึ่งก็คือแผนโครงสร้างที่มีค่าที่สุดพอดี ให้ผลใช้งานไม่ได้เลยสักหน้า ทั้งอัตรา JSON ที่ถูกต้อง 57.9% และ element recall 11% ส่วนใหญ่มาจากบั๊กตัวนี้ แค่หน้า20 หน้าเดียวก็มี element ใน ground truth ที่นับไม่ได้ 31 ตัว

แก้ยังไง: ใส่ `repetition_penalty=1.15` กับ `no_repeat_ngram_size=8` ลงในทั้งสองสคริปต์ (เป็นการกำบางส่วน คือ "ขอร้อง" ไม่ใช่ "ห้าม") แล้วคิดถัดมาก็เปลี่ยนไปใช้ grammar-constrained decoding ผ่าน `xgrammar` ซึ่งดัน JSON ที่ถูกต้องจาก 57.9% ขึ้นเป็น 96.8%

กฎที่ได้: ยารันงานที่ต้องได้ output แบบมีโครงสร้างด้วย greedy decoding ล้วนๆ โดยไม่มีตัวบังคับโครงสร้างเด็ดขาด — ให้ใช้ grammar-constrained decoding (`xgrammar`) เป็นค่าตั้งต้นสำหรับ output ที่เป็น JSON เพราะบงกชโทษแบบอ่อนๆ แค่นี้ทำให้วนยากขึ้น แต่มีแค่ grammar เท่านั้นที่ทำให้หวังเลิกค้างเป็นไปไม่ได้

parseGridMaster อ่านไฟล์กริดรุ่นปัจจุบันไม่ได้เลยสักไฟล์ตั้งแต่ 2026-08-02 — ทุกการ import ได้กริดว่างเปล่าแบบเงียบๆ

เกิดอะไรขึ้น: parseGridMaster ของ Constistant อ่าน f.x_lines / f.y_lines ที่ระดับบนสุด แต่ schema ย้ายมันเข้าไปอยู่ในอ็อบเจกต์ grid{} ที่ซ่อนอยู่ข้างในแล้ว พอเราข้ามทั้งสองไฟล์เตอร์ของ dataset พบว่าไฟล์กริด 37 จาก 37 ไฟล์เป็นแบบซ่อน และ 0 ไฟล์เป็นแบบระดับบนสุด ทุกการ import raw-extraction ตั้งแต่วันที่ปรับโครงสร้างนั้นจึงได้กริดว่างเปล่า แล้วมารีคทุกช่วงว่า unresolved แบบเงียบๆ

ต้นเหตุจริง: การย้ายไฟล์ใน schema ผังข้อมูล ไม่เคยถูกส่งต่อไปยังฝั่งแพลตฟอร์มที่เราไปใช้ และเวลาหา key ไม่เจอ ฝั่งที่ใช้ก็คืนค่าว่างแทนที่จะโยน error ความล้มเหลวโดยสิ้นเชิงจึงแยกไม่ออกจาก 'บ้านหลังนี้ไม่มีกริด'

เสียหายอะไร: เงียบมาตั้งแต่ 2026-08-02: ทุกการ import raw-extraction ในแพลตฟอร์มได้ข้อมูลกริดเป็นศูนย์ เลยได้ช่วงคานที่แก้ได้เป็นศูนย์ตามไปด้วย ความยาวคานจึงไม่เคยไปถึง BOQ เลยแม้แต่หนึ่งเดียว

แก้ยังไง: แก้ให้อ่าน grid{} แบบซ่อนก่อน แล้วค่อยไปอ่านระดับบนสุดสำหรับไฟล์เก่า พิสูจน์กับข้อมูลจริงแล้ว (รัน adaptRawExtraction บนบ้าน 09 ที่เป็น ground truth ช่วงคานขยับจาก 0/87 เป็น 18/87 พร้อมค่าจริง) เพิ่ม regression test ใหม่ 4 ตัวใน tests/raw-extraction-grid.test.js npm test ผ่าน 1122/1122

กฎที่ได้: parser ที่คืนค่าว่างเมื่อเจอโครงสร้างที่ไม่รู้จัก ต้องเตือนแทน — และทุกครั้งที่ย้ายไฟล์ใน schema ต้อง grep หาทุกจุดที่เราไปใช้ แล้วเขียน regression test ที่ใช้ไฟล์จริงรุ่นปัจจุบัน ไม่ใช่ข้อมูลตัวอย่างสำหรับเทสที่เขียนมือแล้วอัปเดตไปพร้อมกับ parser

parseGridRef รับแต่รูปแบบ grid ref ที่มีขีดคั่น ทั้งที่สเปกกำหนดให้ไม่มีขีด — ช่วงคานเลยเป็น null ทุกตัว

เกิดอะไรขึ้น: parseGridRef ทำ trimmed.split('-') แล้วปฏิเสธทุกอย่างที่ไม่ได้ออกมาเป็น 2 ส่วนพอดี แต่สเปก §0.8 กำหนดให้ใช้รูปแบบไม่มีขีด ("C1" ไม่ใช่ "C-1") และข้อมูลจริงทั้งหมดก็ใช้แบบนั้น ช่วงคานทุกตัวในทุกการ import จึงเป็น null

ต้นเหตุจริง: parser เขียนขึ้นตามข้อสันนิษฐานเรื่องรูปแบบ ('grid ref คือตัวอักษร-ขีด-ตัวเลข') ซึ่งขัดกับกฎ §0.8 ของสเปกเอง แล้วก็ไม่มีเทสไหนใช้ค่าจริงที่ตรงตามสเปก แคมเวลาพึ่งมันคืน null ไม่ได้โยน error เรื่องเลยไม่เคยโผล่ขึ้นมา

เสียหายอะไร: ช่วงคานทุกตัวในทุกการ import raw-extraction ได้ค่า null — เป็นบั๊กเงียบตัวที่สองจากสองตัวที่รวมกันแล้วทำให้เราคาดคิดของคานไม่เคยไปถึง BOQ เลย

แก้ยังไง: ให้รับ ^([A-Za-zก-ฮ]{1,3})*(\d+)?\$ ควบคู่ไปกับรูปแบบมีขีดแบบเดิม ส่วน ref ที่นำหน้าด้วย ~ ซึ่งเป็นค่าโดยประมาณยังถูกปฏิเสธถูกต้องเหมือนเดิม ตรวจสอบกับข้อมูลจริงแล้ว (ช่วงคาน 0/87 → 18/87) พร้อม regression test ใหม่อีก 2 ตัว

กฎที่ได้: เขียนเทสของ parser จากตัวอย่างมาตรฐานในสเปก ไม่ใช่จากรูปแบบที่เราเดาเอาเอง — และทำให้การ parse ไฟล์ที่จำเป็นไม่ผ่านมันดังขึ้นมา (เตือน/นับจำนวน) แทนที่จะคืน null ซึ่งแยกไม่ออกจากกรณีที่ข้อมูลไม่มีอยู่จริงๆ

Dedup by NCC similarity broke real detection coverage in 4 houses — similarity ≠ interchangeability (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Running dedup on the already-promoted footing bank removed tpl_footing13/15/17 (each scoring 0.80–1.00 against templates 7/6/16). Real coverage then broke: houses 04, 05, 17 and 26 dropped out of green. Templates that look near-identical at 64x64 NCC behave differently under multi-scale sliding-window matching on a full page.

ต้นเหตุจริง: NCC on a resized 64x64 crop measures appearance similarity, which is not the same property as "produces the same matches on a full-page multi-scale sweep". The dedup metric was a proxy for the property that actually mattered.

เสียหายอะไร: Four houses lost detection coverage. Had this been applied without a coverage check, the loss would have shown up much later as unexplained detection regressions.

แก้ยังไง: Added a second gate to `--bank --apply`: measure real coverage before and after using house_pages() + match_bank via the same path as harvest_templates.py. If any house drops out of green, the whole kind is cancelled and files auto-restored (tested — caught footing, restored 14 to 14 automatically). Column dedup was verified safe with real match_points before applying: 7 templates → 3, coverage unchanged at 16/30 houses.

กฎที่ได้: When deduplicating anything that feeds a detector, validate against the detector's real output, not an appearance-similarity score. Build the before/after coverage check with auto-rollback into the tool itself.

Dataset builder still filtered old pattern names after a rename — plan_footing/plan_beam/gridline were 0 examples for a full day (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: build_dataset_t03.py still filtered on the old pattern names "plan" and "gridline". After the 28 Aug rename, plan_footing, plan_beam and gridline vanished entirely from train/val.jsonl — 0 examples each — and nobody noticed until a prompt-hygiene sweep stumbled into it. After the fix: gridline 40 / plan_footing 100 / plan_beam 125.

ต้นเหตุจริง: A rename in one place (page-type vocabulary) was not propagated to the consumer's filter, and the builder reports total example count rather than per-subtask counts, so "0 plan_beam" looked like a normal successful build.

เสียหายอะไร: The three most important subtasks — including plan_beam, the entire point of the round — were completely absent from the training data. Any training run in that window would have been worthless for those subtasks and the reason would have been invisible.

แก้ยังไง: Fixed the filter. Verified 257 real rows in train+val.jsonl.

กฎที่ได้: A dataset builder must assert a minimum example count per subtask and fail loudly on zero. A silent empty filter after a rename is the most expensive class of bug there is — it makes an entire training run meaningless with no error.

สูง 2026-07-09

คอลัมน์ที่ปักตรึงไว้ใน Label Studio ตัดเนื้อหาทิ้งเจียบๆ แทนที่จะให้เลื่อนดูได้

เกิดอะไรขึ้น: คนทำ annotation แจ้งว่าแถบซ้ายเลื่อนไม่ได้ ตรวจแล้วพบว่า `.stickycol`` ไม่มี `max-height` และไม่มี `overflow-y` เลย เนื้อหาส่วนที่สูงเกินจอจึงไม่ใช่แค่เลื่อนไปไม่ถึง แต่หายไปเลย โดยไม่มีอะไรบอกว่ามีของหายอยู่

ต้นเหตุจริง: คอลัมน์แบบ sticky ถูกจัดสไตล์บนข้อสันนิษฐานว่าเนื้อหาจะขนาดคงที่ และไม่เคยทดสอบกับเนื้อหาที่สูงกว่าจอ ส่วนค่าเริ่มต้นของ CSS overflow ก็คือตัดทิ้งแบบเจียบๆ

เสียหายอะไร: คนทำ annotation มองไม่เห็นบางส่วนของแบบที่ต้องมา label — เกิดบนโปรเจกต์จริงตัวแรกที่สร้างใน Label Studio Cloud และเป็นเหมือนกันทั้ง 5 ไฟล์ config

แก้ยังไง: เพิ่ม `max-height:96vh; overflow-y:auto; padding-right:6px`` ให้ `.stickycol`` ในไฟล์ XML config ทั้งห้าไฟล์

กฎที่ได้: แถวที่ปักตรึง/sticky ทุกอันต้องใส่ `max-height` กับ `overflow-y:auto` ตั้งแต่แรก เพราะโหมดฟังโดยค่าเริ่มต้นของมันคือข้อมูลหายไปเจียบๆ ไม่ใช่เลยเอาต์แตกให้เห็น

สูง 2026-07-19

RB2 ถูกบันทึกที่ตำแหน่งเดา confidence 0.4 ซึ่งกลายเป็นผิดทั้งหมด

เกิดอะไรขึ้น: RB2 ในผังคานาคอดินของบ้านหลัง 2 ถูกบันทึกไว้ที่ B3-A3 ด้วย confidence 0.4 — คือเดาล้วนๆ ที่จริงมันอยู่ที่ C4-B4 (1.8) + B4-A4 (3.15) ผังเสาแนว 4 ส่วนเลข 2.50 ที่เอาไปแปะกับ RB2 จริงๆ แล้วเป็นของ dummy grid B'

ต้นเหตุจริง: เอาค่าที่เดาแบบ confidence ต่ำมาเขียนลงเป็นข้อมูลจริง แทนที่จะปล่อยเป็น null แล้วติดธงไว้ และเอาระยะที่อธิบายไม่ได้ (2.50) ไปแปะกับ element ที่ใกล้ที่สุดที่ดูเข้าเค้า แทนที่จะปล่อยไว้ว่าไม่รู้ว่าเป็นของอะไร

เสียหายอะไร: คานาคอดินถูกบันทึกผิดฝั่งของอาคาร ระยะหนึ่งถูกยกให้ผิด element ซึ่งทำให้เจอ dummy grid B' ซ้ำไปด้วย

แก้ยังไง: แก่ RB2 เป็น C4-B4 + B4-A4 แล้วเอาเลข 2.50 ที่วางออกมาไปหาค่า B' = A-2.50 จากนั้นเพิ่ม RB1A ที่ B'3-B'4

กฎที่ได้: ตำแหน่งที่ confidence 0.4 คือการเดา และการเดาที่เขียนลงในฟิลด์ตำแหน่ง ปลายทางแยกไม่ออกเลยว่าเป็นข้อมูลจริงหรือเดามา ให้ปล่อยเป็น null พร้อมติดธงให้ชัด และห้ามเอาระยะที่อธิบายไม่ได้ไปแปะกับ element ที่บังเอิญอยู่ใกล้ที่สุด

สูง 2026-07-21

ตัวกัน `isinstance(ip.size, dict)` เป็น false เจียบๆ ทำให้การตั้งค่าความละเอียดภาพถูกข้ามไปโดยไม่มี error

เกิดอะไรขึ้น: โค้ดกัน `if isinstance(ip.size, dict)` ที่เพิ่มเข้าไปตอนแก้บั๊ก max_pixels เป็น False เสมอ ค่าความละเอียดที่ตั้งใจไว้ที่ 5,120 visual token เลยไม่เคยถูกใช้ โมเดลรันด้วยค่า default 16,384 ไปเจียบๆ

ต้นเหตุจริง: `ip.size` เป็น `SizeDict` ซึ่งรองรับ `[\"key\"] = value` ได้ แต่ไม่ได้สืบทอดมาจาก `dict` โค้ดกันไปเช็คการสืบทอดแทนที่จะเช็คพฤติกรรม และเส้นทางที่ลึ้มเหลวคือข้ามไปเจียบๆ ไม่ใช่ error

เสียหายอะไร: ทั้งการเรนและการประเมินผลรันด้วยความละเอียดภาพที่ผิดโดยไม่มีอาการอะไรให้เห็น ทำให้ค่า baseline ปนเปื้อนอย่างน้อยหนึ่งรอบ และเสียรอบตึกเพิ่มบนเครื่องที่เสียเงิน

แก้ยังไง: เอาเงื่อนไข isinstance ออก แล้วเขียนค่าลงไปตรงๆ ยืนยันด้วยสคริปต์เล็กๆ บนเครื่องที่อ่านค่าที่ได้กลับมาดู (ไม่ต้องโหลดโมเดลเต็มตัว)

กฎที่ได้: ห้ามเอาการเช็ค type มากับการเขียนค่า config ถ้ามันข้ามไปเจียบๆ ได้ ให้เขียนค่าลงไปเลย แล้วอ่านกลับมา assert ว่าตรงกับที่ตั้งใจ การไม่ทำอะไรเจียบๆ อันตรายกว่าการฟังโครมๆ

สูง 2026-08-20

Unsloth's from_pretrained silently ignores the subfolder kwarg, so the LoRA adapter could not be loaded (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The t02 adapter files live at lora-adapter/ inside the HF repo, not at root. Unsloth's from_pretrained does not forward the subfolder kwarg to its internal AutoConfig/PeftConfig probes, so loading failed. Traced by reading unsloth/models/loader.py source rather than guessing.

ต้นเหตุจริง: Unsloth's loader accepts **kwargs at the top level but does not thread them through to the config-probe calls it makes internally — a silent kwarg drop, not an error. The caller had no way to know the argument was ignored.

เสียหายอะไร: Blocked the entire inference run until diagnosed; would have been easy to misattribute to a broken adapter upload or a bad HF repo layout.

แก้ยังไง: Pre-download with snapshot_download(allow_patterns=[...]) and point from_pretrained at the resulting local path. Applied to both run_house_batch.py and infer_t02.py.

กฎที่ได้: When a wrapper library ignores an argument, read its source rather than theorising — and prefer resolving remote artifacts to a local path yourself (snapshot_download) instead of trusting a wrapper to forward path kwargs it never documented.

สูง 2026-08-24

The MAX_LENGTH assert checked "most images" instead of the actual longest sequence and was blind to a real overflow (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: After op04 expanded the dataset from 452 to 1,116 examples, one example was found to exceed MAX_LENGTH. The existing assert only checked the example with the most images, which is not the longest sequence. Fixing the assert to check the top 3 by image+prompt+GT combined revealed real sequences of 30,368 / 28,008 / 28,035 — only 7% margin under 32,768, and the original assert could not see any of them.

ต้นเหตุจริง: The guard used a proxy metric (image count) for the thing it was actually protecting (total token length). Prompt text and GT length were excluded from the calculation entirely.

เสียหายอะไร: Silent truncation of training examples, or an OOM mid-run, would have been the first symptom. This is the same bug class already documented as t01 §0.4.

แก้ยังไง: Rewrote the assert to compute image + prompt + GT tokens and check the top 3 longest; also asserts visual-token count on the first batch to catch silent 512px resizing.

กฎที่ได้: A guard must measure the exact quantity it protects, never a proxy. When adding data to a dataset, recompute the length distribution — do not trust a guard tuned on the old distribution.

สูง 2026-08-24

First house-08 inference run died on both of its first two jobs, from two unrelated JSON bugs (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The t03 inference run on house 08 failed 2/2 on its first jobs: (1) gridline output had genuinely broken structure — missing the opening `{` of an array element and a corrupted `pox\_M` field name; (2) notes output had `"confidence\_score": 0.` (decimal point with no digit) plus a second bug in the same file, `{item\_no":` missing the opening quote on a key name.

ต้นเหตุจริง: Grammar (xgrammar) was only attached to plan_beam per the standing rule, leaving every other subtask generating free-form JSON. gridline additionally has the fewest training examples (45) and is the hardest task.

เสียหายอะไร: A paid inference run wasted on 2 jobs before anyone noticed; a regex-patching approach would have been an endless whack-a-mole.

แก้ยังไง: Decided ☒ to switch to `--grammar-all` (force xgrammar on every subtask, not just plan_beam), killed the running process at 2/33 and re-ran the whole set (~\$0.4).

กฎที่ได้: If a structural constraint is correct for one subtask it is almost certainly correct for all of them — apply it globally by default. Never fix malformed model output with per-pattern regex patches.

สูง 2026-08-28

BEAM_GAP hardcoded to a single 21px value was blind to half the drawing library (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Beam detection used one hardcoded gap of 21px. Measuring 33 houses revealed two distinct series: the กรมโยธา series at ~16px and the house-17 series at ~21px. The single value could only ever see one of them.

ต้นเหตุจริง: The constant was derived from whichever house was open when the detector was written, and never validated against the corpus. Drawings from different issuing bodies use different line spacings.

เสียหายอะไร: Beam detection was structurally impossible on roughly half the library, for as long as the constant existed.

แก้ยังไง: Replaced with BEAM_GAPS = (16, 21), measured across 33 houses.

กฎที่ได้: Any geometric constant derived from a drawing must be measured across the whole corpus, and stored as a set of series values rather than a single number. Different issuing bodies produce different conventions.

สูง 2026-08-28

Column match threshold of 0.65 produced garbage detections in 10 of 30 houses (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: At threshold 0.65 the column detector produced garbage in 10 of 30 houses. A sweep raised it to 0.75, cutting garbage houses to 1. Two small templates (t6/t7) still sprayed 49–96 garbage points per page onto other drawing series at 0.75; a new rule (small template + ink coverage < 0.15 → require 0.85) fixed them with zero garbage and full true-positive retention.

ต้นเหตุจริง: A single global threshold cannot serve templates of different sizes and ink densities — a small, sparse template matches noise almost anywhere. Thresholds were set by intuition rather than by sweeping across the corpus.

เสียหายอะไร: Up to 96 false detections per page would have been fed to the model as "hints", actively poisoning the pass the hints exist to help.

แก้ยังไง: Swept 30 houses to set 0.75, added the small-template/low-ink → 0.85 rule, and placed the rule in match_bank as the single decision point so analyze and harvest cannot drift apart. Result: columns matching GT went 4→16 houses, val 01–05 all green.

กฎที่ได้: Sweep detection thresholds across the full corpus, and make the threshold a function of template properties (size, ink density) rather than a global constant. Put the rule in exactly one place so callers cannot drift.

สูง 2026-08-28

Union-find dedup created transitive chain drift, merging two crops of completely different words (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: dedup_templates.py first used union-find (transitive grouping) at $NCC \geq 0.8$. Files A~B scored 1.0 and B~C scored 0.80, so A was grouped with C even though A~C is only 0.737 — and the pair it merged turned out to be two text crops of different words, not icons at all.

ต้นเหตุจริง: Similarity is not transitive. Union-find over a similarity threshold builds chains where the endpoints are arbitrarily dissimilar, bridged by an intermediate file.

เสียหายอะไร: Would have silently discarded genuinely distinct templates, and the reported similarity score would have been a lie (the reported score was never the actual pair score).

แก้ยังไง: Rewrote as greedy — each file is compared only against already-kept representatives, never through another file. Guarantees every reported score is a real pairwise score.

กฎที่ได้: Never use transitive clustering on a similarity threshold. Compare against retained representatives only, so the score you report is the score you measured.

สูง 2026-08-29

Human reviewer prose leaks into the model's prompt via the grid master JSON block (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: All prose prompt files scanned clean of forbidden characters, but the *assembled* prompts for plan_footing/plan_beam/plan_slab append a real grid master JSON block from GT — and that block carries `source` and `confidence\_flags` fields containing human reviewer explanations full of `.` , `\_` , `—`. Those reach the model.

ต้นเหตุจริง: The hygiene scan checked the prose `.md` source files, not the assembled prompt that the model actually receives. The grid-master block is injected at build time from GT, so it bypasses the source-file scan entirely, and no field whitelist limits what gets pasted in.

เสียหายอะไร: The exact characters being systematically removed from prompts to reduce hallucination are still reaching the model, plus irrelevant reviewer commentary the model must ignore. Not yet fixed.

แก้ยังไม: Not fixed — flagged as a separate concern awaiting Makham's ruling on whether to strip source/confidence_flags and keep only id/pos_m/type.

กฎที่ได้: Scan the assembled artifact the model actually sees, not the source templates. When injecting real data into a prompt, whitelist the fields the model needs — never paste the whole record.

สูง 2026-08-30

Known-since-t03 and still unfixed: JSON grammar constrains structure only, so generation can loop forever inside a string value — that is what hung the server for 7+ hours (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: xgrammar enforces that output is valid JSON but places no constraint on the content of a string value, so a request can enter an endless repetition inside a string and never terminate. This behaviour was already known from t03. It was still unfixed when `serve\_purson.py` was put on a rented card, and one such runaway request is exactly what blocked the event loop and made the whole server unreachable for 7+ hours, including the trivial `/v1/models` health check.

ต้นเหตุจริง: A known generation-level failure mode was carried forward without either a mitigation (max-token cap that actually fires, stop condition, repetition guard) or an architectural containment, so the first time it met a single-threaded blocking server it took the whole process down.

เสียหายอะไร: A rented GPU billed for 7+ hours while producing nothing; two planned test runs (including the `test\_browser\_path.py` end-to-end with a real user JWT) never completed and the instance had to be torn down with `presentation.py` down.

แก้ยังไม: Not fixed. The recorded next step addresses only the amplifier (wrap `generate()` in `asyncio.to\_thread()` or a separate queue thread); the runaway generation itself is still open, as is why `PAGE\_TIMEOUT\_S` never fired.

กฎที่ได้: A known failure mode carried into production is a scheduled outage. Grammar-constrained decoding guarantees shape, never termination — pair it with a hard token cap and a timeout that is proven to actually fire.

สูง 2026-08-31

The $1/\sqrt{k}$ adapter-averaging formula was measured and found wrong — norms blew up 3x to 16x (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The planned $1/\sqrt{k}$ averaging was measured against real weights: because the folds point in nearly the same direction, cross terms add constructively and $\|\Delta W\|$ came out 3.0x (attention) and 16.3x (MoE) too large.

ต้นเหตุจริง: A scaling formula was adopted from theory that assumes near-orthogonal contributions, without measuring the actual alignment between folds.

เสียหายอะไร: Would have produced a badly mis-scaled merged model; caught by measuring rather than assuming.

แก้ยังไง: Switched to exact rank-axis concatenation (r 16->48, alpha 32->96), measured ratio 1.000 on both layer types — equivalent to PEFT's combination_type="cat" done manually.

กฎที่ได้: Measure the norm ratio of any merge/averaging formula against real weights before shipping it — theoretical scaling assumes independence that fine-tuned folds do not have.

สูง 2026-09-01

pass3 double-counted points into the BOQ across three separate layers (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Points that had already been matched, points used as anchors, and points belonging to a different subtask were each re-added as new elements into the BOQ. Fixing it layer by layer took the duplicate count from 48 → 33 → 7 → 0.

ต้นเหตุจริง: No single ledger of 'points already accounted for'. Each consumption path removed points from its own view but not from a shared consumed-set, so a point could legitimately pass three different 'is this new?' checks.

เสียหายอะไร: Up to 48 phantom elements inflating the BOQ. Required three successive rounds of fixing — work redone twice.

แก้ยังไง: Tracked consumed points across all three layers until the duplicate count reached 0.

กฎที่ได้: When several code paths can each claim the same item, keep one shared consumed-set rather than per-path filters. And when a de-dup fix reduces but doesn't eliminate duplicates, keep going — a partial fix here means another claiming path exists.

สูง 2026-09-01

If the network died while reporting a job failure, the exception escaped the loop and killed the entire worker (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: In ``main()``, a network error during the 'report job failed' call threw an exception that propagated out of the ``while`` loop, terminating the worker process entirely.

ต้นเหตุจริง: The error-reporting path was itself unprotected — the code that handles failures could fail in a way nothing handled. Additionally the done and failed paths shared code, so a 'failed' stamp could overwrite an already-completed job.

เสียหายอะไร: A single network blip during error reporting would take down the worker completely, leaving the rented GPU idle and billing.

แก้ยังไง: Separated the done and failed paths and forbade stamping ``failed`` over a job that already completed.

กฎที่ได้: The failure-reporting path is the one that must never throw. Wrap it, and never let done/failed share a write path where the failure branch can clobber a success.

เรื่องที่เหลือในหมวดนี้ (33)

ค่ายาวๆ ที่ไม่มีที่ตัดคำสั้นออกนอกกล่องในหน้าตรวจของ Label Studio จนอ่านข้อมูลไม่ได้

กลาง 2026-07-06

อย่าปล่อย config ของ UI งาน annotation ออกไปโดยไม่เคาะลงเรนเดอร์กับข้อมูลจริงสักครั้ง และให้ถือว่าทุกค่าที่เครื่องสร้าง (flag แบบ snake_case, ก่อน JSON, id) เป็นข้อความที่ตัดบรรทัดเองไม่ได้ ให้ตั้ง break-word กับฟิลต์พวกนี้เป็นค่าเริ่มต้นไปเลย

เติม xml-stylesheet เปล่าๆ เพื่อซ่อนคำเตือนของเบราว์เซอร์ แล้วผลลัพธ์และ สุดท้ายต้องย้อนกลับทั้งหมด

กลาง 2026-07-09

อย่าไปแก้เรื่องความสวยงามของเส้นทางการใช้งานที่ไม่ได้รองรับตั้งแต่แรก ให้บอกไปตรงๆ ว่าเส้นทางนั้นไม่รองรับ การหาทางเลี่ยงให้ปัญหาที่ไม่ใช่ปัญหา สร้างปัญหาจริงขึ้นมาได้

findProductionDir() แอบหายไปใช้ mk_test/t2 ซึ่ง schema เก่ากว่าสองรุ่น เวลาหาโฟลเดอร์บ้านไม่เจอ

กลาง 2026-07-10

การหายไปใช้ข้อมูลเก่าต้องส่งเสียงดัง หรือไม่ก็อย่ามีเลย ถ้าไม่เจอ input ที่ขอมมา ให้ล้มพร้อมบอก path ที่ไปหามา อย่าแอบเอาชุดข้อมูลเก่ากว่าที่ยัง parse ผ่านมาใช้แทนเจียบๆ

eval_fields.py crashed: `ip.max\_pixels` became a read-only property in the installed transformers

กลาง 2026-07-21

Pin library versions used by training/eval scripts, and validate every model-config write against the exact library version installed on the rental image before the paid run.

Raw template matching reported 235 detections for 14 real footings

กลาง 2026-08-25

Any sliding-window or score-map detector needs NMS before its output can be counted. Never report a raw above-threshold pixel count as an object count.

Self-harvest threshold set on the wrong side — seed 0.85 could never fire

กลาง 2026-08-26

Never pick a similarity threshold without first plotting the score distribution of real positives and real negatives. A threshold above the positive distribution is a silent no-op, not a strict filter.

Two Set-of-Mark label-overlap bugs that unit tests could not see

กลาง 2026-08-26

Anything whose output is an image must be eyeballed before it ships. Unit tests prove the data is right; only looking proves the render is readable.

`--pass25` wrote only `\*25` sidecar files while `apply\_arm()` reads the plain ones

กลาง 2026-08-28

Producer and consumer of a file-naming convention must share one constant. When two passes emit similar artifacts, verify the consumer actually finds the producer's output — an empty read is not an error.

prompt ของ pass3 สั่งให้ตอบฟิลด์ `stirrup\_tie\_count` ซึ่งไม่มีอยู่ใน schema

กลาง 2026-08-28

ชื่อฟิลด์ที่เขียนใน prompt ถือเป็นส่วนหนึ่งของสัญญา schema ให้เพิ่มตัวตรวจว่าทุกชื่อฟิลด์ที่ prompt ขอ ต้องมีอยู่จริงใน schema — เป็นตัวกันประเภทเดียวกับที่ใช้เช็คอักขระต้องห้ามอยู่แล้ว

เทมเพลตเสา t6/t7 พ่นผลตรวจจับขยะ 49–96 อันต่อหน้า ที่ threshold 0.75 ที่เพิ่งปรับขึ้น

กลาง 2026-08-28

threshold เป็นค่าประจำเทมเพลตแต่ละตัว ไม่ใช่ค่าคงที่ตัวเดียวใช้ทั้งระบบ ก่อนปล่อยเทมเพลตใหม่ให้กวาดทดสอบทั้งคลัง และดูจำนวนขยะแยกรายเทมเพลต ไม่ใช่ดูแค่ยอดรวมผ่าน/ไม่ผ่าน

OCR กรอภาพโดยใช้สัดส่วนของ PDF ต้นทาง ทั้งที่ภาพเป็น PNG ที่จัดเรียงใหม่แล้ว — กรอปผิดกรอบไปเลย

กลาง 2026-08-29

คำนวณพิกัดกรอปจากภาพตัวที่โค้ดจะอ่านจริงเท่านั้น ห้ามคำนวณจากรูปแบบต้นทางก่อนหน้า และให้ดูผลกรอปหนึ่งใบด้วยตา ก่อนรันทั้งชุด

อภิธานศัพท์เกือบถูกฝังไว้ในไฟล์ที่ถูก generate เท่านั้น — เรนเดอร์รอบหน้ามันจะหายเจิบๆ

กลาง 2026-08-29

ห้ามแก้ไฟล์ที่ถูก generate เด็ดขาด ให้ใส่หัวข้อความ 'generated — do not edit' ไว้ในทุกไฟล์ผลลัพธ์ เพื่อให้เพลอทำพลาดแบบเจิบๆ ไม่ได้

อภิธานศัพท์ถูกฉีดเข้าไปสองรอบ — จาก _common.md ครั้งหนึ่ง และจาก prompt ของแต่ละ subtask อีกครั้ง

กลาง 2026-08-29

เวลาที่ย้ายเนื้อหาจากแบบใช้ร่วมกันมาเป็นแบบแยกรายชิ้น ให้ลบจุดฉีดของเดิมในการแก้ครั้งเดียวกัน แล้วพิสูจน์ด้วยการนับจำนวนที่ปรากฏในไฟล์ผลลัพธ์จริง ไม่ใช่ด้วยการอ่านโค้ด

`subprocess(shell=True)` บน Windows ทำให้ `>` ใน `gpu\_ram>=90` กลายเป็นการ redirect ไฟล์ — `vastai search` คินค่าว่างโดยไม่มีเบาะแส

กลาง 2026-08-30

ห้ามใช้ `shell=True` กับ argument ที่แทรกค่าเข้าไป ให้ส่งเป็น argv list เสมอ บน Windows นี่ไม่ใช่เรื่องสไตส์การเขียน มันทำให้ argument ที่มี `>`, `<`, `&` หรือ `|` เพี้ยนแบบเจิบๆ

`serve\_purson.py`: request ที่ไม่มีรูปพังด้วย IndexError ที่ `images[0]` และ `return dict, 500` ถูก FastAPI ตีความเป็น list ไม่ใช่ HTTP status

กลาง 2026-08-30

ตรวจ payload ของ request ที่ทุกขอบเขตของเซิร์ฟเวอร์ และยืนยันธรรมเนียมการคืน error ของ framework นั้นๆ (FastAPI ต้องใช้ `JSONResponse`) — status code ผิดทำให้ทุกความล้มเหลวกลายเป็นความสำเร็จปลอมที่ปลายทาง

load_split() หาไฟล์ภาพโดยอิงตำแหน่งของสคริปต์ แทนที่จะอิงรากของ repo

กลาง 2026-08-31

ยึดการหาไฟล์ทุกจุดกับค่าคงที่ที่ชี้รากของ repo อย่าพึ่ง working directory ของ process หรือตำแหน่งของสคริปต์ตัวเอง

The batch-check guard raised NameError on an already-deleted variable and reported failure after passing

กลาง 2026-08-31

Test the guard on the success path too, and treat a guard that cries wolf as a priority bug — a distrusted guard is worse than no guard.

UI progress bar could never reach 100% when a job included a gridline page

กลาง 2026-08-31

Ensure test fixtures cover the case that changes a computed total — a fixture where the special case is absent will pass an off-by-one forever.

UI button relabelled 'Read with Gemini' while the backend actually calls Qwen

กลาง 2026-09-01

Never ship a user-visible label that you know is false while waiting on a decision — revert to a provider-neutral wording ('Read with AI') until the backend actually matches the name.

Progress UI hid the bar while waiting and never said which pass was running

กลาง 2026-09-01

For any long-running job, the UI must always show a state — 0% plus the current step name beats a hidden bar. An invisible progress indicator on a 75-minute job actively causes duplicate-run bugs.

Adding pass3 made the ETA jump back to hours right when the job was nearly finished

กลาง 2026-09-01

Anything that reasons about pipeline stages must derive from the ordered step list, never from hand-written per-step conditionals — otherwise every new stage silently corrupts the older ones.

3D structure model rendered ceiling fans, drain pipes and power outlets as structural boxes

กลาง 2026-09-01

Reuse the existing filter list rather than consuming raw collections — and when you drop items, report what you dropped instead of dropping silently.

Beam-only plan pages can never be measured — a beam's box centroid sits on no grid ref

กลาง 2026-09-01

Geometric anchoring must respect element topology — a point element and a linear element cannot share the same 'where is it' rule. Check the assumption against each element class before generalising.

The Single project's header stacked 5 fields as full-width rows, pushing the actual content off-screen

ต้า 2026-07-09

Screenshot every annotation config at a real working resolution before handing it to annotators — header chrome that eats a third of the viewport is a throughput cost, not a cosmetic one.

GPU showed 100% utilisation at only 33.7W — the real bottleneck was CPU offload

ต้า 2026-07-29

Never read GPU utilisation alone — pair it with power draw and memory-fit logs. And when generation is the cost, reduce tokens emitted rather than chasing hardware.

House 12 footing plan reports 13 columns where 12 exist — one duplicate box survives NMS

ต้า 2026-08-25

Record known off-by-one detection errors explicitly rather than rounding them away; any downstream consumer comparing counts needs to know the tool has a small over-count bias.

Junk detections sitting 4px outside a beam band escaped the footing-in-beam filter

တာ 2026-08-25

A geometric exclusion filter needs a tolerance margin sized to the detector's own positional jitter. An exact-bounds filter always leaks at the edge.

Variable ``h`` shadowed in the mining script — house name became a box height

တာ 2026-08-28

Do not reuse single-letter names across scopes in scripts that mix strings and numbers. A one-character name is not worth the class of bug it enables.

`cv_scan --demo self-check` broke on our own format change ("`4.`" → "`4`")

တာ 2026-08-29

Write self-check assertions against semantic content, not literal formatting. When running a repo-wide character sweep, re-run every self-check afterwards — the sweep will break some of them.

Accidentally introduced a forbidden ``**`` while writing the very prompt cleanup that removes them

တာ 2026-08-29

When a rule bans characters that are also your editor's default syntax, the automated scan is not optional — run it after every single edit, including edits made by the rule's own author.

A hand-written selftest assertion was itself wrong, briefly implicating ``strip_fence`` as buggy

တာ 2026-08-30

When a new test fails on ported, previously-working code, suspect the test before the code — verify the expected value against the original implementation's real output.

bash reported 'unbound variable' for arguments that were passed, due to local declarations on one line under `set -u`

တာ 2026-08-31

Declare each ``local`` on its own line in bash, especially under `set -u`, and confirm shell bugs with `bash -x` rather than reasoning about them.

Three files wrote `window.x` directly with no guard, making them impossible to unit test

တာ 2026-08-31

Follow the repo's existing guard convention for global attachments so every module remains importable in a test environment.

Git ความสะอาดของ repo และ ของที่ทำแล้วหาย

commit สาขา การ merge และงานที่ทำเสร็จแล้วแต่ไม่ได้เก็บ หรือของที่ลบไปแล้วกลับมา

ทั้งหมด 19 เรื่อง · วิกฤต 2 · สูง 3 · กลาง 6 · ต่ำ 8

เรื่องที่ต้องอ่านให้ครบ (5)

วิกฤต 2026-07-10

มีคน **force-rewrite** ประวัติทั้งหมดของ repo **Constistant** ทำให้ต้อง **hard reset** ฝั่งเครื่องตัวเอง

เกิดอะไรขึ้น: เกิดเหตุ git ประวัติแตกแยกในวันเดียวกันก่อนหน้านี้: ประวัติทั้งหมดของ repo **Constistant** ถูก **force-rewrite** ที่ฝั่ง upstream ทำให้ main ในเครื่องต้อง reset ให้ตรงกับ origin/main มีการตัดสินใจว่าไม่เกี่ยวกับงาน raw_json เลยจึงใจไม่บันทึกรายละเอียดไว้

ต้นเหตุจริง: การ **force-push** ไปเขียนประวัติที่เผยแพร่ไปแล้วทับ ทำให้ clone อื่นๆ ทุกตัวแตกจากมัน และไม่มีบันทึกว่าการแจ้งหรือประสานงานกันไว้เลย

เสียหายอะไร: ต้อง **hard reset main** ในเครื่อง และ commit ที่มีเฉพาะในเครื่องบน branch นั้นเสียหายทั้งหมด รายละเอียดของเหตุการณ์ไม่เคยถูกจดไว้ ต้นเหตุจริงจึงยังไม่มีใครรู้จนถึงตอนนี้

แก้ยังไง: reset main ในเครื่องให้ตรงกับ origin/main

กฎที่ได้: อย่า **force-push** branch ที่ใช้ร่วมกันเด็ดขาด และถ้าเกิดขึ้นแล้ว ให้บันทึกว่าอะไรถูกเขียนทับและงานในเครื่องอะไรหายไปบ้าง — คำว่า 'ไม่เกี่ยวกับงานวันนี้' นี่แหละคือวิธีที่ทำให้ต้นเหตุจริงหายไป

วิกฤต 2026-07-11

อ่านหน้า **50-51** พร้อมกันแล้วค่อยเขียนจากความจำ ทำให้ข้อมูลสลับกันไปมาระหว่าง **BOQ** สามแผ่น

เกิดอะไรขึ้น: หน้า 50 และ 51 ของบ้านหลังที่ 2 ถูกอ่านพร้อมกัน แล้วค่อยมาเขียนทีหลัง ผลคือข้อมูลโครงหลังคาเหล็กจากหน้า 50 (แผ่นที่ 3/19) ถูกเขียนลงไฟล์งานมุงหลังคา (แผ่นที่ 5/19) ส่วนข้อมูลจริงของแผ่น 5/19 ไม่เคยถูกเขียนลงไปเลย และแผ่น 6/19 (งานฝ้าเพดาน) ถูกทิ้งวางเพราะเข้าใจว่าอ่านไม่ออก ทั้งที่ข้อมูลมีอยู่ แต่ถูกเก็บผิดไฟล์

ต้นเหตุจริง: ตาราง **BOQ** หลายตารางที่หน้าตาคล้ายกันถูกจำไว้ในหัวข้ามการอ่านหลายหน้า ก่อนที่จะเขียนอะไรลงดิสก์สักอัน ตัวตนของหน้าจึงหายไประหว่างการอ่านกับการเขียน

เสียหายอะไร: ไฟล์ข้อมูลจริงผิดพร้อมกันสามไฟล์ — ไฟล์หนึ่งมีข้อมูลของอีกหน้า ไฟล์หนึ่งวางทั้งที่ควรมีข้อมูล และอีกไฟล์ถูกทำเครื่องหมายผิดๆ ว่าอ่านไม่ได้ ทั้งหมดนี้อยู่ในชุดข้อมูลที่ตั้งใจเอาไปใช้ **fine-tune**

แก้ยังไง: กลับไปอ่านหน้า 50 และ 51 ใหม่ทีละหน้าตั้งแต่ต้น เขียนทั้งสามไฟล์ใหม่ให้ถูก และใส่คำเตือนอธิบายสาเหตุไว้ในแต่ละไฟล์ที่แก้

กฎที่ได้: อ่านหน้าเดียวแล้วเขียนไฟล์ของหน้านั้นให้เสร็จก่อนอ่านหน้าถัดไป — ห้ามอ่านรวดเดียวหลายหน้าแล้วค่อยเขียนจากความจำ โดยเฉพาะตารางซ้ำๆ อย่าง **BOQ** ที่แต่ละหน้าหน้าตาเหมือนกัน

สูง 2026-07-07

An entire pipeline folder existed as a stale duplicate in a second repo for 5 days (ยังไม่ได้แปล — ต้นฉบับ
อังกฤษ)

เกิดอะไรขึ้น: Constant/No_touch_box/ turned out to be a full copy of the Training repo's pipeline folder frozen at a July 2 snapshot — missing mk_test/, the Label Studio Makham configs, and carrying an older CLAUDE.md. One of its files (label-studio-tasks.js) was a pre-bugfix version still pointing at the old single-path qwen-output location.

ต้นเหตุจริง: Work was propagated between the two repos by copying folders instead of designating one owner repo per artifact, so the copy silently stopped receiving updates the moment the original moved on.

เสียหายอะไร: Five days of divergence; a deprecated buggy script sitting on disk where someone could have run it; a multi-round cleanup (July 7 twice, plus still-unfinished remainder) spent just proving the duplicate had no unique content.

แก้ยังไง: Dified each file against its Training-repo twin (3 identical byte-for-byte, 1 clearly older), then deleted rule_of_tune.md and the 4 Label Studio files; the rest of the folder was left pending a decision.

กฎที่ได้: Every artifact has exactly one owning repo. If a file must be visible from two repos, link or reference it — never cp it, because a copy is a fork that nobody remembers to merge.

สูง 2026-07-10

งานสัปดาห์ (ตั้งแต่รอบ t4) ไม่ได้ commit เลย และยังคงค้างอยู่ถึงวันที่ 11 ก.ค.

เกิดอะไรขึ้น: ตั้งแต่รอบ t4 เป็นต้นมาไม่มีการ commit เลยสักครั้ง งานค้างวันที่ 9 ก.ค. เขียนว่า "ยังไม่ได้ commit อะไรเลยตั้งแต่รอบ t4" วันที่ 10 ก.ค. ย้ำอีกว่า "ค้างมาหลายวันแล้ว" และวันที่ 11 ก.ค. ก็ยังอยู่ในรายการ ครอบคลุมทั้งการเขียน schema ใหม่ การแก้ generator ไฟล์ XML หาไฟล์ raw JSON สามหลัง และเหตุการณ์เปลี่ยนชื่อโฟลเดอร์ raw/

ต้นเหตุจริง: การ commit ถูกเลื่อนไปรบกวนจนจบงานทีเดียว ขณะที่แต่ละวันก็ใส่ของใหม่ที่ไม่เกี่ยวข้องกันเข้ามาเรื่อยๆ และมีอุบัติเหตุอย่างน้อยหนึ่งเรื่อง (การเปลี่ยนชื่อ raw/) ที่ไปแตะ working tree เดียวกัน

เสียหายอะไร: งานสกิดข้อมูลที่ทำใหม่ไม่ได้ทั้งหลายวัน อยู่แค่ใน working tree ที่ยังไม่ commit ในรีโปที่ประวัติเพิ่งถูกเขียนทับแบบ force ในช่วงเดียวกันนั้นพอดี และ commit ที่เกิดขึ้นก็จะปนทั้งอุบัติเหตุเปลี่ยนชื่อ การแก้ config และข้อมูลบ้านสามหลังรวมกันมั่วไปหมด

แก้ยังไง: ยังไม่ได้แก้ — ยังอยู่ในรายการงานค้างตอนจบวันที่ 11 ก.ค.

กฎที่ได้: commit ทุกครั้งที่จบรอบงาน โดยขอบเขตจำกัดแค่อรอบนั้น — งานค้างที่ยังไม่ commit ทั้งเสียหายก็ไม่ได้ และรับประกันได้เลยว่า commit ที่ออกมาสุดท้ายจะรื้อวไม่ไหว

Work committed to `main` was invisible to Makham, who was working on the diverged `Presentation-demo` branch (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The house09 results were committed to `main`, but Makham works on `Presentation-demo`. He went looking for the house09 files through GO.bat and could not find them, because the two branches have separate histories with a different merge-base and were never merged.

ต้นเหตุจริง: Two long-lived branches diverged with no merge plan and no agreement about which is the working branch. Output was committed to a branch the actual user was not on, with nothing surfacing the mismatch.

เสียหายอะไร: Makham could not find deliverables he had just asked for; required a manual copy of all 17 files to `D:\000download\09` outside git as a workaround, and left an unresolved merge decision hanging.

แก้ยังไง: Files copied out to `D:\000download\09` as a backup outside the repo; the merge itself was recorded as an open decision.

กฎที่ได้: Agree on one working branch before committing deliverables, and if branches must diverge, decide the merge point immediately — do not let 'we'll merge later' persist while output lands where the user cannot see it.

เรื่องที่เหลือในหมวดนี้ (14)

t3 บันทึกความชันหลังคาไว้ 9 องศา แต่ในแบบเขียนว่า 6

กลาง 2026-07-08

ค่าใดที่ปรากฏบนแบบมากกว่าหนึ่งแผ่น ต้องอ่านจากแผ่นที่ขีดที่สุดแล้วตรวจไขว้กัน และการสกัดข้อมูลรอบใหม่ต้องห้ามให้เห็นคำตอบของรอบก่อนเด็ดขาด

หน้าที่คาตจากสารบัญว่าเป็นเกณฑ์ผังบริเวณ กลับเป็นอย่างไรไปคนละเรื่อง

กลาง 2026-07-10

สารบัญบอกได้แค่โครงสร้าง ไม่ได้บอกเนื้อหา ทุกหน้าต้องจำแนกจากตัวหน้าเอง และต้องอนุญาตให้ agent เฝียกับแผนที่มันได้รับมาได้

House 7's price misread as 1,572,000 instead of 1,528,000

กลาง 2026-07-16

Small-print numeric tables need a cross-source check as a standard step, not an occasional one — the cross-house tie-break technique found three digit errors in two rounds and should run on every house.

House 07 S-05: BS1's position recorded as 1.1 m below grid C when it is ~2.1 m

กลาง 2026-07-28

Sanity-check every offset against the enclosing bay dimension — an offset that does not fit inside its bay, or that leaves an implausible remainder, is a digit error.

12 commits ของ King-try (โปรโตไทป์ overlay, hooks, skills, agent) ค้างเติง ไม่ได้ cherry-pick อะไรเลย เลื่อนการตัดสินใจข้าม session ไปเรื่อยๆ

กลาง 2026-08-20

กึ่งที่แยกออกไปแล้วมีงานที่เราอยากได้อยู่ข้างใน คือทรัพย์สินที่เสื่อมค่าลงเรื่อยๆ ให้ cherry-pick commit ที่ต้องการใน session เดียวกับที่รีว ไม่งั้นต้องกลับมารีวใหม่ทั้งหมด แล้วช่องว่างก็จะยิ่งถ่างออก

Output correctness is enforced by post-hoc filtering rather than by grammar at generation time

กลาง 2026-08-31

Constrain generation with a schema grammar at source rather than filtering downstream; filtering removes garbage but never recovers the information that was lost.

A killed agent left a scratch file in the ground-truth folder that was never cleaned up

ต้า 2026-07-05

Write scratch and progress files to a separate scratch directory, never into the data directory. Cleanup that only happens on the agent's happy path is not cleanup — assume abnormal termination as the default case.

Repo accumulated empty scaffolding folders, empty files, temp scripts and duplicate test directories

ต้า 2026-07-05

Do not create scaffolding before the code that fills it, never write temp files to the repo root, and never allow two directories whose names differ only by a plural — that pair silently splits tooling scope.

The cancelled migration script was left on disk with no owner and no decision

ต้า 2026-07-08

When output is discarded, delete or quarantine the tool that produced it in the same action — leaving it runnable is an invitation to recreate the mistake.

An 11-file trash/ holding folder sits inside the repo, not gitignored

ต้า 2026-07-09

Decide a quarantine folder's git status at the moment you create it — the default is that it gets committed, and a large uncommitted backlog makes that easy to miss.

The only copy of the working 61-page inference driver lives in a scratchpad, not in the repo with the results it produced

ต้า 2026-07-28

A script that produced committed results is a work product; commit it next to them, especially when the run is explicitly resumable.

An older superseded generation of pipeline scripts is still on disk with nobody deciding whether it is dead

ต้า 2026-08-02

When cleaning up, sort by live-versus-dead, not by which feature a file belonged to. A file's owner script is the test, and it should be checked once and written down.

The Constant accept/reject checklist work (preview/save split + UI) was left uncommitted

ต้า 2026-08-20

Commit the reusable half before showing the questionable half. A rejected feature should not be able to take a passing refactor down with it.

Dead TARGET/ELEMENT_TYPES substitution logic lived on in the dataset builder

ต้า 2026-08-29

Delete the machinery when you delete its reason to exist. Dead substitution code in a data pipeline is worse than dead code elsewhere — it implies a transformation is happening when nothing is.

เอกสาร การตั้งชื่อ และการสื่อสาร

เอกสารที่บอกไม่ตรงความจริง ชื่อที่ทำให้เข้าใจผิด และคำเตือนที่ไม่ถูกพูดให้ดังพอ

ทั้งหมด 32 เรื่อง · วิกฤต 3 · สูง 7 · กลาง 10 · ต่ำ 12

เรื่องที่ต้องอ่านให้ครบ (10)

วิกฤต 2026-07-21

รู้ยู่แล้วว่ามีความเสี่ยงข้อมูลหายถาวร แต่ไม่เคยพูดออกมาเป็นคำเตือนแบบบล็อกงาน

เกิดอะไรขึ้น: ความเสี่ยงที่ว่าการ destroy instance จะลบผลลัพธ์ทั้งถาวรนั้น รู้กันตั้งแต่ตอนแปลง GGUF เสร็จ แต่ถูกพูดถึงแค่ในฐานะรายงานเบาๆ ('ยังไม่ได้อัปเดต HuggingFace เต็มค้อยทำ') ไม่เคยถูกพูดเป็นคำเตือนชัดๆ ว่า 'ห้ามปิดเครื่องก่อนสำรองข้อมูล'

ต้นเหตุจริง: การสื่อสารความเสี่ยงที่ย้อนกลับไม่ได้ ถูกยัดรวมอยู่ในรายการสิ่งที่ต้องทำ แทนที่จะยกระดับเป็นคำเตือนที่บล็อกงาน งานที่เลื่อนไว้กับคำเตือนความเสี่ยงทำลายล้าง ถูกจัดเป็นข้อความประเภทเดียวกัน

เสียหายอะไร: เป็นเหตุโดยตรงที่ทำให้ข้อมูลสูญทั้งหมดตามข้อข้างบน และถูกบันทึกเป็นกฎ 'Mark of Shame' ใน rule_of_tune.md

แก้ยังง: เพิ่มกฎถาวรว่า ความเสี่ยงใดๆ ที่ทำลายข้อมูลถาวรได้ ต้องพูดออกมาเป็นคำเตือนแบบบล็อกงานที่แยกออกมาชัดเจน

กฎที่ได้: แยกคำเตือนความเสี่ยงทำลายล้างออกจากรายการงานทั่วไป — พูดออกมาจริงๆ ด้วยภาษาที่เข้าใจง่าย และพูดทันที ก่อนถึงจังหวะที่ความเสี่ยงนั้นจะเกิดจริง

วิกฤต 2026-07-24

ไฟล์หายไปบน GPU ที่เข้ามา เพราะคำเตือน "ยังไม่ได้สำรองข้อมูล" ถูกพูดแบบเบาๆ แทนที่จะเป็นคำสั่งหยุด

เกิดอะไรขึ้น: วันที่ 2026-07-21 งานบนเครื่องเช่าหายหมด บันทึกชั้นสุดในไดอารี่ระบุชัดเจนไม่ใช่บั๊กในโค้ด แต่เป็นเพราะผู้ช่วยพูดถึงเรื่องไฟล์ที่ยังไม่ได้สำรองแบบผ่านๆ เบาๆ ไม่ได้ยกขึ้นมาเป็นคำเตือนแบบบังคับหยุด เลยไม่มีใครลงมือทำอะไรก่อนเครื่องจะหายไป

ต้นเหตุจริง: การสื่อสารความเสี่ยงไม่มีระดับความรุนแรง สถานการณ์ที่อาจทำลายข้อมูลถาวร (งานที่ยังไม่ได้สำรองบนเครื่องเช่าชั่วคราว) ถูกพูดด้วยน้ำเสียงเดียวกับรายงานความคืบหน้าทั่วไป คนอ่านเลยแยกไม่ออกว่าอันไหนคือ "แจ้งให้ทราบ" อันไหนคือ "หยุดเดี๋ยวนี้"

เสียหายอะไร: งานบนเครื่องเช่าหายหมด และต้องใช้วันที่ 2026-07-24 ไปกับการสร้างและรันระบบตรวจสอบก่อนบิน Phase 0 เพื่อ กันไม่ให้เกิดซ้ำ — เสียเวลาทำงานกระบวนการเต็มๆ หนึ่งวันแบบไม่ได้วางแผนไว้

แก้ยังง: กลายเป็นกฎข้อ 4 ใน rule_of_tune.md และเป็นที่มาของ checklist "Phase 0" ก่อนเข้าเครื่องใน RETUNE_WORKFLOW.md ต้องตรวจสอบให้แน่ใจว่าสำรองข้อมูลแล้วก่อนจะทำลายอะไรที่เป็นของชั่วคราว

กฎที่ได้: อะไรก็ตามที่ทำให้งานหายแบบกู้ไม่ได้ ต้องพูดเป็นคำสั่งหยุดชัดๆ ใช้คำว่า STOP ห้ามพูดแบบผ่านๆ ถ้าผลที่ตามมา ย้อนกลับไม่ได้ ให้ปฏิเสธที่จะทำต่อไปเลย ดีกว่าเอ่ยถึงมันแค่ผ่านๆ

เส้นทาง notes → Constistant ไม่ได้อะไรเลย: ตัว adapter อ่านไฟล์ที่ schema ไม่เคยกำหนดไว้

เกิดอะไรขึ้น: raw-extraction-adapter.js อ่าน notes.fc_ksc / fy_main_ksc / fy_stirrup_ksc / cover_mm เพื่อสร้าง extractedNotes แต่ primary_rawjson_schema.md ไม่เคยกำหนดโครงสร้างนั้นไว้เลย ตัว fc_ksc ไม่มีในสเปกเลยสักที และมีอยู่ในไฟล์ notes จริงแค่ 1 จาก 55 ไฟล์ สรุปคือสเปกคอนกรีตและเหล็กระดับทั้งโครงการไม่เคยไปถึง pipeline เลยสำหรับบ้านแทบทุกหลัง

ต้นเหตุจริง: ฝั่งที่เอาข้อมูลไปใช้เขียนขึ้นตามโครงสร้างไฟล์ที่ "หวังว่าจะมี" แทนที่จะเขียนตามสเปก แล้วสเปกก็ไม่เคยถูกอัปเดตให้ตรงกัน ไม่มีอะไรเช็คว่าไฟล์ที่ adapter อ่านมีอยู่จริงใน schema ใหม่ เส้นทางข้อมูลทั้งเส้นเลยอ่านไฟล์ที่ไม่มีอยู่และคืนค่าว่างไปเรื่อยๆ โดยไม่ error สักครั้ง

เสียหายอะไร: ชั้นคุณภาพคอนกรีต ชั้นคุณภาพเหล็ก และระยะหุ้ม — ซึ่งเป็นสเปกวัสดุระดับทั้งโครงการ — ไปถึง pipeline ของ BOQ ได้อย่างมาก 1 จาก 55 ไฟล์ notes คือแทบไม่เคยไปถึงเลย และเสียบมาตั้งแต่วันที่เขียน adapter ตัวนี้

แก้ยังไง: มะขามเลือกทาง "ไปกำหนดไฟล์ในสเปกให้ตรง" แทนที่จะแก้ฝั่งที่เอาไปใช้ เพิ่ม §4a ใหม่: มีชั้นแบบซ้อนที่บันทึกสิ่งที่พิมพ์อยู่บนแบบจริงๆ (concrete{}, steel{round_bar{},deformed_bar{}}, cover{default_mm,by_condition[]}) บวกกับ alias แบบแบนทางเดียวอีก 4 ตัว (fc_ksc/fy_main_ksc/fy_stirrup_ksc/cover_mm) เพื่อให้ adapter ไม่ต้องแก้อะไรเลย ส่วนการย้ายข้อมูลไฟล์เดิม 55 ไฟล์ยังค้างอยู่

กฎที่ได้: ทุกไฟล์ที่ adapter อ่าน ต้องมีอยู่จริงในสเปก — เพิ่มเทสที่ตรวจไว้ว่าไฟล์ที่ adapter อ่านมีใน schema ใหม่ เพราะ adapter ที่อ่านไฟล์ที่ไม่มีอยู่จะพังแบบเสียบๆ ด้วยการคืนค่าว่าง ซึ่งหน้าตาเหมือนกับ 'บ้านหลังนี้ไม่มี notes' เป๊ะ

ย่อ schema แล้วทำกฎการทำงานจริงหล่นหายไป 3 ข้อโดยไม่รู้ตัว

เกิดอะไรขึ้น: พอเอา primary_rawjson_schema.md มาเทียบกับต้นฉบับ 219 บรรทัดที่ละห้วงข้อ พบว่ามีกฎจริงหายไป 3 ข้อ ไม่ใช่ประวัติที่ตั้งใจตัดออก คือ ขอบเขตงานอัตโนมัติ (run_pipeline.py สกัดอัตโนมัติแค่ 6 จาก 13 pattern), กฎ atomic-segment (คานต้องรายงานแยกทีละช่วงกริด ห้ามรวมยอดมาก่อน) และคำถามที่ยังค้างเรื่อง element_id ซนกันข้ามไฟล์ section

ต้นเหตุจริง: การย่อทำด้วยการเขียนใหม่ ไม่ได้ไล่ทีละห้วงข้อจากต้นฉบับ ของที่หายเลยมองไม่เห็นจนกว่าจะเอามา diff กันจริงๆ

เสียหายอะไร: schema ตัวที่ใช้งานจริง — ไฟล์ที่กำหนดการสกัดข้อมูลของบ้านทุกหลังต่อจากนี้ — ขาดทั้งกฎ atomicity ของคานและขอบเขตงานอัตโนมัติ ถ้าไม่ได้เอามา diff บ้านทุกหลังหลังจากนั้นจะถูกสกัดด้วยข้อกำหนดที่ไม่ครบ

แก้ยังไง: เทียบทีละห้วงข้อกับต้นฉบับฉบับเต็ม แล้วใส่กฎทั้งสามข้อกลับเข้าไป

กฎที่ได้: ย่อสเปกเสร็จแล้วต้อง diff กลับกับต้นฉบับทีละห้วงข้อ และพิสูจน์ว่าไม่มีกฎการทำงานข้อไหนหาย — งานเขียนใหม่เชื่อไม่ได้ว่ามันเก็บสิ่งที่มันไม่ได้พูดถึงไว้ครบ

สูง 2026-07-21

บอกว่า GGUF 'พร้อมใช้แล้ว' ทั้งที่มันอ่านภาพไม่ได้เลย

เกิดอะไรขึ้น: รายงานว่า GGUF ที่ quantize แล้วพร้อมใช้งาน ทั้งที่ส่วน mmpoj/วิชันหายไป คือมันทำสิ่งเดียวที่โปรเจกต์นี้มีอยู่เพื่อทำ (อ่านแบบก่อสร้าง) ไม่ได้เลย

ต้นเหตุจริง: รายงานสถานะจากการที่ pipeline รันจบ ('แปลงเสร็จแล้ว') แทนที่จะรายงานจากความสามารถ ('ไฟล์ที่ได้ทำงานได้จริงไหม') ทำให้เคลมเกินจริง

เสียหายอะไร: ทำให้เข้าใจผิดว่ามีของที่ใช้งานได้จริงแล้ว ซึ่งมีส่วนทำให้ตัดสินใจปิดเครื่องไป

แก้ยังไ: แก้ในไดอารี่แล้ว และเพิ่มเป็นกฎ: ห้ามบอกว่าผลลัพธ์พร้อมใช้ ถ้ามันยังทำหน้าที่หลักของมันไม่ได้

กฎที่ได้: รายงานความพร้อมโดยวัดจากหน้าที่หลักของสิ่งที่สร้าง และต้องพิสูจน์ด้วยการรันจริงตั้งแต่ต้นจนจบ ไม่ใช่วัดจากการที่ขั้นตอน build/convert จบด้วย exit code 0

สูง 2026-07-21

Assistant warned too weakly about the missing backup and declared the model ready while it could not read images (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The 07-21 loss was attributed not to a code bug but to two communication failures: the backup warning was phrased mildly instead of as a stop-the-line alert, and the artifact was described as ready to use although its vision path was known not to work.

ต้นเหตุจริง: Severity of a warning was not matched to the irreversibility of the action it guarded; and 'ready' was reported against the steps completed rather than against the actual project goal (reading drawings).

เสียหายอะไร: Directly enabled the irreversible loss of a day's work, and set a wrong expectation about what the GGUF could do.

แก้ยังไ: Established Phase 0 pre-rental checks and the rule that documented checklists must be actually executed, not merely written.

กฎที่ได้: Scale warning strength to irreversibility — anything that destroys data gets a blocking confirmation. And never report a deliverable as ready against completed steps; report it against the capability the user actually needs.

สูง 2026-07-28

Workflow files for different tuning rounds shared the same name, so a previous round's  could be mistaken for this round's completion (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Both rounds' workflow documents were called RETUNE_WORKFLOW.md-style names. Reading a ticked checkbox from an earlier round recreates the belief that the current round's work is finished — the exact mechanism behind the DAY OF SHAME incident.

ต้นเหตุจริง: Round identity was carried only by directory path, not by filename or document header, so a file opened out of context gave no signal about which round its checkmarks belonged to.

เสียหายอะไร: A live risk of a false 'finished' conclusion — the same failure class that already cost the team a lost round.

แก้ยังไง: git mv to t01_workflow.md / t02_workflow.md preserving history; both headers now state the round, the status, and a warning that a  belongs to whichever round it is in; the naming convention added as a rule.

กฎที่ได้: Name per-round artefacts with the round in the filename and restate the round in the document header. A status marker must carry its own scope, since it will be read out of context.

สูง 2026-08-21

The vertical axis had no field anywhere in the schema despite levels being printed on every elevation and section (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: z-levels such as +3.75 ระดับหลังคาน, +0.60 ระดับพื้นชั้น 1 and ± 0.00 ระดับอ้างอิง are printed on every elevation and section in the set, yet the schema had no field for them at all — x_lines/y_lines are plan axes only. Discovered when Makham instructed 'make the grid master record every value it finds on every page'.

ต้นเหตุจริง: The schema's grid model was built entirely from plan-view thinking (two horizontal axes) and nobody checked it against a section or elevation sheet. Compounded by the earlier assumption that elevations were low-value.

เสียหายอะไร: Every house extracted to date recorded zero vertical levels. Floor heights and beam soffit levels — needed for any volume or 3D reconstruction — were never captured, across the whole dataset.

แก้ยังไง: Added z_levels[] under grid{} in primary_rawjson_schema.md §4, extended dimension_chains[] to the z axis, added unassigned_dimensions[] as the catch-all, and added two §0 checklist items so check_format.py enforces it.

กฎที่ได้: Audit the schema against one sheet of every drawing type before locking it — an entire missing axis is the kind of gap that survives 22 houses because nothing ever asks for it.

Teaching PDF shipped a wrong claim that raising MAX_LENGTH requires renting a bigger card (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The teaching document written at 15:00 said §2.3 "MAX_LENGTH เพิ่มได้ แต่ต้องเช่าการ์ดใหญ่ขึ้น = จ่ายแพงขึ้น". The VRAM decomposition done an hour later showed 79.7 of 87.5 GB does not scale with sequence length at all, so +32% resolution fits the existing card with 4.0 GB spare and no extra cost. The document had gone stale inside the same session.

ต้นเหตุจริง: A conclusion was written into a durable document before the measurement that would decide it had been done, and the document had no mechanism linking it back to the numbers it depended on.

เสียหายอะไร: A factually wrong cost claim in a document Makham may show his professor — it would have argued for spending money that is not needed.

แก้ยังไง: Corrected §2.3 immediately rather than deferring, and added §2.3.1/§2.3.2 with the real 100%-downscale measurement and the VRAM decomposition. PDF regenerated 12 → 19 pages, verified by extracting text back out and checking six markers.

กฎที่ได้: Do not write a conclusion into a shareable document ahead of the measurement that decides it. If a doc is published before the data lands, put a dated "unmeasured" marker on the claim so it cannot be read as fact.

Five BBS minimum-rebar values were slated to cite มยพ. 1103-64, which contains no design minimums at all (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Ton ordered the BBS fallback's third gate (assumeRebarFromRules) to cite มยพ. 1103 for accuracy. Reading all 20 pages of the real document showed มยพ. 1103-64 is a materials and workmanship standard (มอก. / cutting / bending / hooks / lapping / testing) and contains none of the five design-minimum reinforcement values.

ต้นเหตุจริง: The standard was assumed to cover design minimums based on its subject area (rebar) without anyone having read its scope. Rebar materials standards and rebar design standards are different documents.

เสียหายอะไร: Would have shipped five values falsely attributed to a Thai standard that does not contain them — a citation error in engineering output.

แก้ยังไง: Kept the ACI values and flagged `dpt1103: 'not\_found'` on every row, with a Thai label appended to each `label\_th` making the non-attribution explicit in the UI.

กฎที่ได้: Read a standard's scope before citing it. A standard covering the same material can cover a completely different aspect of it — and a false citation is worse than an honest foreign one.

เรื่องที่เหลือในหมวดนี้ (22)

Label Studio ขึ้น 'undefined' แล้วไล่หาบั๊กในโค้ด ทั้งที่ต้นเหตุจริงคือ task ที่ import เข้าไปเป็นข้อมูลเก่า

กลาง 2026-07-06

ระบบไหนที่ข้อมูลถูก import เข้ามาเป็น snapshot (task ของ Label Studio, fixture ที่ seed ไว้, cache) ให้เช็คก่อนว่าฝั่งที่ใช้ข้อมูลถืออันเก่าอยู่หรือเปล่า ก่อนจะไปอ่านโค้ดฝั่งที่สร้างมันแม้แต่บรรทัดเดียว ตรวจไฟล์จริงบนดิสก์ก่อน คำสั่งเดียวจบและตัดปัญหาทั้งกลุ่มนี้ออกได้เลย

onstart.sh's runbook gave a wrong instruction and referenced a file that does not exist

กลาง 2026-07-28

Re-validate every operational runbook against the current code before the rental starts. A runbook step that names a file must be checked against the filesystem, since it will only be discovered wrong while the meter runs.

The diary states export_gguf.py was adapted for t02 in section 5 and still unadapted in the Pending list

กลาง 2026-07-28

Reconcile the pending list against the session's own completed work before closing a diary entry. A pending item that contradicts the body is worse than no list, because it will be trusted.

เส้นบอกระยะที่ไม่มีอะไรวางอยู่บนนั้น (ระยะจากขอบถึงเส้นกริดแรก) ถูกกฎ dummy grid ทิ้งไปเสียบๆ

กลาง 2026-08-21

ฟิลต์ใน schema ต้องนิยามตามสิ่งที่แบบพิมพ์มาจริง ไม่ใช่ตามที่ผู้ใช้ข้อมูลตอนนี้อต้องการ และค่าอะไรก็ตามที่ตัวสก็ดตัดสินใจไม่บันทึก ต้องไปลง array เก็บของเหลือพร้อมค่าเตือน ห้ามหายไปเสียบๆ

material_list → boqSeed[] ไม่มีทางเขียนลงที่เก็บ BOQ จริงของ Constistant เลย

กลาง 2026-08-21

อย่าส่งฝั่งผู้ผลิตข้อมูลออกไปโดยไม่ฝั่งผู้ใช้ ถ้าจะเลื่อนการต่อท่อออกไปก่อน ให้บันทึกไว้ตรงที่ตัวผู้ผลิตอยู่ เพื่อไม่ให้คนถัดไปเข้าใจผิดว่า 'มีผลลัพธ์ออกมาแล้ว' เท่ากับ 'ผลลัพธ์ถูกเอาไปใช้แล้ว'

site_plan.element_type ไม่มีชุดค่ามาตรฐาน เจอ 10 ค่าที่ต่างกันในบ้าน 5 หลัง

กลาง 2026-08-21

ปิดชุดค่าของฟิลต์หมวดหมู่ทุกตัวก่อนที่ค่าต่างๆ จะแตะเลขสองหลัก 10 ค่าจากบ้าน 5 หลังคือฟิลต์ที่ไม่เคยมีใครดูแลเลย และทุกหลังที่เพิ่มเข้ามาจะทำให้การย้ายข้อมูลในอนาคตแพงขึ้นอีก

't04_workflow.md' เก่าค้ำหน้า — หัวไฟล์ยังเขียนว่า 'ไม่เคยเข้า GPU' ทั้งที่ 2 จาก 4 fold เทรนใกล้เสร็จแล้ว

กลาง 2026-08-30

อัปเดตเอกสารทำงานทันทีที่การตัดสินใจเปลี่ยน ไม่ใช่รอตอนจบ session แผนเก่าที่อ่านแล้วเหมือนยังเป็นปัจจุบัน อันตรายกว่าไม่มีแผนเลย

Copying commands out of a PDF mangled Thai vowels, so cd failed

กลาง 2026-08-31

Never put Thai characters inside a command line intended to be copied, and verify programmatically that command blocks in a manual are ASCII-only.

A trailing explanatory comment copied along with a command crashed argparse

กลาง 2026-08-31

Keep explanations off command lines entirely, and where an operator must run a multi-step procedure, replace typed commands with a numbered menu that eliminates the whole class of copy-paste errors.

Makham still has not been taught the vast.ai workflow — a standing handoff item deferred again

กลาง 2026-09-02

A handoff deferred once will be deferred every time. Schedule the walk-through as the first task of a session, not the last, or it never happens.

Assumed Unsloth was a service requiring a login/account

ตำ 2026-07-21

For every dependency, state explicitly whether it is a local library or a hosted service needing credentials — credential prerequisites should be listed only against the components that truly have them.

Teaching doc states the LoRA adapter is "a few hundred MB"; the real file was 7.5GB

ตำ 2026-07-21

Teaching/summary docs must quote the real measured numbers from the run they describe; cross-check every figure against the log before publishing, especially sizes that drive backup decisions.

Teaching doc states a LoRA adapter is 'a few hundred MB'; the real t01 adapter is 7.5GB

ตำ 2026-07-21

On an MoE model, every per-adapter size estimate must be multiplied by the expert count; correct the intuition in the doc rather than letting the dense-model figure sit next to the real one.

rule_of_tune.md item 4 pointed at a file path that had been moved

ตำ 2026-07-28

Reference documents by role ('that round's own workflow'), not by a fixed path, so a rename can't silently break the rule. And re-check every cross-reference in the same commit as a rename.

A correct token count of 5,100 would have been read as a failure because the documentation stated the expected value as 5,120

ตำ 2026-07-28

Document expected values as the number the real computation actually produces, including rounding artefacts — and state why. An idealised expected value manufactures false alarms.

Grid master hit a dummy-grid naming case the spec had no rule for, decided ad hoc

ตำ 2026-08-18

When a real drawing forces a naming decision the spec does not cover, record it in the file AND raise it as a spec amendment the same day — an undocumented precedent only in one file's metadata will be silently contradicted by the next house.

House 01 หน้า19 is recorded as a footing/column plan on 08-18 and a beam plan on 08-20 — same page, two sheet types

ตำ 2026-08-20

When logging a measured result, copy the page's recorded `pattern`/`sheet\_code` verbatim into the diary rather than describing it; a proof whose subject is ambiguous cannot be reused.

roof_plan subtask had no input row in the pass-design table despite being a grid-master source

ตำ 2026-08-21

Hand-built dependency tables need a mechanical consistency check — if any subtask of a class declares an input, assert all subtasks of that class do, rather than relying on reading the table row by row.

Pass 2 was defined as 'the 7 patterns Constant consumes' but the traced adapter reads only 6 — soil_boring_log is not among them

ตำ 2026-08-21

When a scope is justified by 'the code consumes this', name the consuming module per pattern; one traced module is not the whole platform, and one extra entry in a justified list is how scope quietly inflates.

The operator asked about 'apply_arm.py', a file that does not exist — the arm switcher is a function buried inside the inference runner

ต้า 2026-08-29

If a knob decides how an expensive run behaves, its location must be guessable from its name. A README pointer chain is fine for stages; it is not fine for the one function an operator must trust at 2am.

Diary entries written in Thai despite the diary's own rule requiring English

ต้า 2026-09-01

A written convention with no enforcement point will drift. Either put the language rule in the entry template header where the writer sees it, or accept the language and update the README — do not leave the rule and the practice disagreeing.

A parallel English translation folder fell out of date and had to be deleted

ต้า 2026-09-01

Never maintain a translated duplicate of a living document. Pick one language and write it that way from the first keystroke — translate-later never happens.

กระบวนการทำงาน การวางแผน และ งานที่เสียเปล่า

ลำดับการทำงาน ขอบเขต การทำซ้ำ และแรงที่หุ่มนำไปผิดที่

ทั้งหมด 229 เรื่อง · วิกฤต 21 · สูง 74 · กลาง 87 · ต่ำ 47

เรื่องที่ต้องอ่านให้ครบ (95)

วิกฤต 2026-07-06

บั๊ก Label Studio ตัวเดิมซ้ำอีกรอบ: ยัดตัวแปร \$ หลายตัวลงใน Text/Header ค่าเดียว ทำให้ import ล้ม 10 จาก 10 งาน

เกิดอะไรขึ้น: ตอน import ไฟล์ label-studio-tasks-makham-elements-structural.json เข้า Label Studio Cloud ล้มทั้ง 10 งาน ด้วยข้อความ `"sheet_code $sheet_name — $pattern: $view_title" key is expected in task data` สาเหตุคือเอาตัวแปร \$field ตั้งแต่ 2 ตัวขึ้นไปยัดใน `value=` ของ Text/Header อันเดียว บั๊กตัวนี้เคยเจอมาแล้วเมื่อ 2026-07-02 และจดไว้ใน No_touch_box/CLAUDE.md แล้วด้วย แต่ตอนเขียน XML config ของ Gen 3 ใหม่กลับไม่ได้เอามาใช้ พอ grep ดูเจอจุดที่ผิด 7 จุด กระจายใน 5 ไฟล์ XML

ต้นเหตุจริง: บทเรียนที่เขียนไว้แล้วไม่ถูกหยิบมาดูตอนกำลังเขียน config ใหม่ เพราะมันอยู่ลึกในหัวข้อย่อยของ Label Studio ใน CLAUDE.md ไม่ได้อยู่ในรายการ 'Key Lessons' ระดับบนสุด เลยไม่มีอะไรบังคับให้เช็คตอนเขียน config แบบเดียวกันขึ้นใหม่ อีกอย่างคืออาการของบั๊กเปลี่ยนรูปไประหว่างสองครั้ง (ครั้งแรก 2 ก.ค. เป็น 'undefined' ตอนรันใน Repeater ส่วน 6 ก.ค. เป็น error ตอน validate ตอน import ที่ระดับบนสุดของ task) เลยไม่มีใครจำได้ว่าเป็นบั๊กตัวเดียวกันจนกว่าจะไล่หาเจอ

เสียหายอะไร: import ล้ม 10 จาก 10 งาน ต้องไล่หาและเขียนใหม่ 7 จุดใน 5 ไฟล์ XML (แยกทุกค่าที่มีหลายตัวแปรออกเป็น Text หนึ่งแท็กต่อหนึ่งตัวแปร) แล้วต้องลบ task ที่ import พังไปแล้วทั้ง วาง config ใหม่แล้ว import ใหม่ตามลำดับที่ถูกต้อง งานตั้งค่า Label Studio ต้องทำใหม่ทั้งรอบ

แก้ยังไง: แยกทุกค่าที่มีหลายตัวแปรเป็น `<Text>` หนึ่งแท็กต่อหนึ่งตัวแปร ครบทั้ง 5 ไฟล์ XML ตรวจซ้ำด้วย grep แล้วยกกฎนี้ขึ้นเป็นข้อ 8 ของหัวข้อ 'Key Lessons (อย่าทำพลาดซ้ำ)' ระดับบนสุดใน No_touch_box/CLAUDE.md พร้อมใส่คำเตือนไว้ใน SESSION_HANDOFF_2026-07-06.md

กฎที่ได้: ห้ามใส่ตัวแปร \$field เกินหนึ่งตัวใน value ของ Text/Header ใน Label Studio เด็ดขาด — หนึ่งตัวแปรต่อหนึ่งแท็กเสมอ และในเชิงโครงสร้าง: บทเรียนที่เก็บไว้ได้หัวข้อย่อยของเครื่องมือ จะไม่มีใครอ่านตอนเขียน config ใหม่ ดังนั้นบั๊กที่เกิดซ้ำต้องถูกยกขึ้นไปอยู่ในรายการ Key Lessons ระดับบนสุดทันทีที่มันซ้ำ ไม่ใช่ปล่อยไว้ตรงที่เขียนครั้งแรก

วิกฤต 2026-07-07

BOQ รอบ t1 บันทึกระเบียงปูพื้น 137/294 ตร.ม. ทั้งที่บ้านมีแค่ 70 ตร.ม.

เกิดอะไรขึ้น: Agent เบื้องหลังที่กลับไปอ่านหน้า 51 ใหม่จับได้ว่าการสักรอบ t1 กรอกปริมาณกระเบื้องปูพื้นไว้ 137 กับ 294 ตร.ม. ซึ่งเป็นไปไม่ได้ทางกายภาพสำหรับบ้านที่มีพื้นที่รวม 70 ตร.ม. (ระบุไว้ในหน้า 60) ค่าที่ถูกต้องของหมวด 2.3 อยู่ในช่วง 21-47

ต้นเหตุจริง: ตัวเลข BOQ ถูกคัดลอกที่ละช่องโดยไม่มีการตรวจสอบความสมเหตุสมผลเทียบกับหน้าอื่นในเอกสารเดียวกันเลย ไม่มีอะไรในกระบวนการที่เอาปริมาณไปเทียบกับพื้นที่ของตัวอาคารเอง

เสียหายอะไร: ปริมาณที่ผิดราว 4-6 เท่าหลุดไปอยู่ในชุดข้อมูล t1 และรอดมาได้จนกว่าจะมีการตรวจสอบหลังบังเอิญมาเจอ

แก้ยังไ: Agent แก่ปริมาณให้เป็นค่าที่ถูกต้อง พร้อมเขียนอธิบายว่าทำไมค่าเดิมถึงเป็นไปไม่ได้

กฎที่ได้: ทุกปริมาณที่สกัดออกมาต้องผ่านการตรวจสอบความสมเหตุสมผลข้ามหน้าอย่างน้อยหนึ่งครั้ง (เทียบพื้นที่ จำนวน หรือยอดรวม) ก่อนจะรับเข้ามา ตัวเลขที่เป็นไปไม่ได้ทางกายภาพต้องตกทันที ไม่ใช่คัดลอกตามต้นฉบับอย่างซื่อสัตย์

วิกฤต 2026-07-14

การแก้ทั้งหมดจากการรีวียูแค่ในโฟลเดอร์สำเนาที่แก้แล้ว ส่วนต้นฉบับที่เอาไป tune ไม่เคยถูกซิงค์เลย

เกิดอะไรขึ้น: ทุกจุดที่แก้ให้บ้าน 1 (กริดสมมติ D/1", การอ้างอิงจุดแบบไม่มีขีด, การเรียงลำดับ element ใหม่, การแยกสเปกออกมา) มีอยู่แค่ใน `json\_แก้ไขแล้ว/` ส่วนต้นฉบับ `rawjson\_ยังไม่ได้แก้ไขโดนคน/01.../` ซึ่งเป็นชุดที่เอาไป tune จริง ยังเป็นของเก่าทั้งหมด การซิงค์กลับไม่เคยเริ่มเลยข้ามทุกรายการต่อจากนั้น

ต้นเหตุจริง: สร้างโฟลเดอร์สำเนาที่แก้แล้วขึ้นมาเพื่อกันไม่ให้ต้นฉบับเสีย แต่ไม่ได้กำหนดขั้นตอนหรือเงื่อนไขการซิงค์กลับไว้พร้อมกัน สองฝั่งเลยห่างกันไปเรื่อยๆ ไม่มีที่สิ้นสุด

เสียหายอะไร: ชุดข้อมูลที่ใช้ train จริงยังมีค่าที่รู้อยู่แล้วว่าผิด (กริดสมมติผิด จำนวนเหล็กผิด การอ้างอิงมีขีด) ในขณะที่ข้อมูลที่แก้ถูกเลื่อนอนอยู่เฉยๆ ไม่ได้ใช้ ทุกวันที่รีวิวเพิ่ม ระยะห่างก็ยิ่งกว้างและความเสี่ยงตอนซิงค์ก็ยิ่งสูง

แก้ยังไ: ยังไม่ได้แก้ — 'การซิงค์กลับยังไม่เริ่มเลย' ค้างอยู่ในรายงานงานค้างมาจนถึง 2026-07-25

กฎที่ได้: เวลาแยกข้อมูลออกมาเป็นสำเนาเพื่อปกป้องต้นฉบับ ให้กำหนดเงื่อนไขการรวมกลับไว้ในการเปลี่ยนแปลงครั้งเดียวกันเลย สำเนาที่แยกออกมาโดยไม่มีแผนซิงค์ ก็คือแหล่งความจริงที่สองดีๆ นี่เอง

วิกฤต 2026-07-19

ป้ายเหล็กแกนหลัก '3-Ø16' ของคาน B5 หายไปจากข้อมูลทั้งต้น ในสองบ้านแยกกัน

เกิดอะไรขึ้น: ในบ้าน 4 ข้อมูลของ B5 ทั้งป้าย '3-Ø16' ไปเฉยๆ เหลือแค่ 2 เส้น ทั้งที่ควรเป็น 3+2=5 แล้วก็เจอความผิดพลาดแบบเดียวกันเป๊ะที่ B5 ของบ้าน 5 (ล่าง 3→5, เหล็ก L/8 หายไปทั้งชุด)

ต้นเหตุจริง: เมื่อคานหนึ่งมีทั้งป้ายหลักและป้ายเสริม L/8 คนอ่านอ่านเจอแค่ตัวหนึ่ง อีกตัวไม่เคยถูกบันทึกเลย และไม่มีการตรวจสอบไขว้เอาจำนวนเหล็กที่บันทึกไว้ไปเทียบกับภาพรูปตัด

เสียหายอะไร: นับเหล็กล่างของคานหลักขาดไป 60% ในสองบ้าน เป็นการขาดเหล็กเชิงโครงสร้างในข้อมูลโดยตรง

แก้ยังไ: แก้เป็น 5 ทั้งสองที่ หลังจากซูมเข้าไปในรูปตัดแล้วนับจุด และยังพบว่า 'additional' ของ B3X ในบ้าน 5 เป็นข้อมูลผี จึงลบทิ้ง

กฎที่ได้: คานที่มีป้ายเหล็กมากกว่าหนึ่งป้าย ต้องบันทึกให้ครบทุกป้ายอย่างชัดเจน — นับจุดในรูปตัดแล้วกระหนยอดกับผลรวมของทุกป้ายที่พิมพ์ไว้ก่อนบันทึก และต้องตรวจสอบข้อมูลผีด้วย: รายการใน additional_bars ที่ไม่มีจุดเหล็กในรูปตัดรองรับ คือข้อมูลที่ถูกรูขี้เ็นมา

train_qwen3vl.py มีบั๊กติดมา 5 ตัว เพราะเขียนไว้ตอนชุดข้อมูลยังมี 221 ตัวอย่าง และไม่เคยถูกรันเลยสักครั้ง

เกิดอะไรขึ้น: การอ่าน train_qwen3vl.py ก่อนบินเจอบั๊กคนละแบบ 5 ตัว ไฟล์นี้เขียนไว้ตั้งแต่ตอนที่ชุดข้อมูลยังมี 221 ตัวอย่าง (ตอนนี้ 403 แถว) และไม่เคยถูกรันเลยสักครั้งเดียว

ต้นเหตุจริง: เขียนโค้ดล่วงหน้าเพื่อการรันในอนาคต แล้วปล่อยให้มันเนาไปในขณะที่ทั้งชุดข้อมูลและไลบรารีรอบตัวเปลี่ยนไปเรื่อยๆ ไม่เคยมีอะไรรันมันเลย ก็เลยไม่เคยมีอะไรทำให้ความล้มเหลวโผล่ออกมา

เสียหายอะไร: บั๊กห้าตัวจะเข้าไปอยู่ในการเทรนที่เสียเงินพร้อมกันหมด — รวมถึงสองตัวที่ทำข้อมูลเทรนเสียเจียบๆ แทนที่จะพัง

แก้ยังไง: แก้ทั้งห้าตัวก่อนเข้าเครื่อง และภายหลังยังพบอีกว่าไฟล์นี้ยังฟังตอนรันจริงด้วยบั๊กตัวที่หก ซึ่งเป็นเรื่องเวอร์ชันของไลบรารี (ดูรายการเรื่อง transformers 5.5.0)

กฎที่ได้: โค้ดที่ไม่เคยรัน คือโค้ดที่ยังไม่ได้ตรวจสอบ และมันเสื่อมสภาพลงเรื่อยๆ ตามที่ไลบรารีและข้อมูลเปลี่ยนไป ให้รันทุกสคริปต์อย่างน้อยหนึ่งครั้ง — แม้จะรันกับข้อมูลตัวอย่างเล็กๆ ก็ตาม — ก่อนจะไว้ใจให้มันไปรันงานที่เสียเงิน

คำแนะนำที่ยึดถือกันมาว่า 'อัปโหลด adapter ก่อนเสมอ' นั้นผิด — โค้ดจริงอัปโหลด GGUF ก่อน แล้วค่อย adapter ทีหลัง

เกิดอะไรขึ้น: แนวปฏิบัติเรื่องความปลอดภัยของการสำรองข้อมูลที่ใช้กันอยู่บอกว่า adapter จะถูกอัปโหลดขึ้นก่อน แต่พออ่านโค้ด export_gguf.py จริงๆ กลับตรงกันข้าม: GGUF ขึ้นก่อน adapter ขึ้นทีหลังสุด และเพราะการตรวจสอบความถูกต้อง (sanity check) ล้มก่อนที่ขึ้น GGUF จะเสร็จ เส้นทางเฟส 8 จึงไม่ได้สำรองอะไรไว้เลยสักอย่าง

ต้นเหตุจริง: เชื่อกันรับประกันด้านความปลอดภัยจากคำอธิบายโค้ดที่จ๋าๆ แทนที่จะไปตรวจสอบกับโค้ดจริง และความเชื่อนี้เป็นตัวรับน้ำหนักทั้งหมด — เพราะมันคือเหตุผลที่ทำให้ทุกคนไว้ใจว่าเฟส 8 เป็นเส้นทางสำรองข้อมูลได้

เสียหายอะไร: ทั้งรอบนี้อยู่ในความรู้สึกปลอดภัยจอมปลอมว่ามีข้อมูลสำรองแล้ว พอบวกกับการที่ sanity check ของเฟส 8 ล้มด้วยเส้นทางเฟส 8 จึงจะไม่ได้สำรองอะไรเลยแม้แต่ชิ้นเดียว

แก้ยังไง: ตรวจสอบกับโค้ดจริงแล้ว และเปลี่ยนกฎเป็น: การสำรองข้อมูลต้องทำแยกต่างหากเสมอ ห้ามพึ่งเฟส 8

กฎที่ได้: การรับประกันด้านความปลอดภัยทุกข้อต้องพิสูจน์กับโค้ดจริง ไม่ใช่กับความจำของตัวเอง และขั้นตอนสำรองข้อมูลที่ไม่อยู่ใน pipeline ที่อาจหยุดกลางคันได้ ไม่นับเป็นการสำรองข้อมูล — ต้องทำแยกและทำทุกครั้งไม่มีเงื่อนไข

ต้นเหตุจริงของความยุ่งเรื่อง format ทั้งหมด: spec ไม่เคยลือกรูปร่างของไฟล์ ล็อกแค่ฟิลด์

เกิดอะไรขึ้น: หลังใช้เวลาทั้งวันแก้ format ก็เจอต้นเหตุจริง: spec อธิบายวิธีเขียนแต่ละฟิลด์ แต่ไม่เคยลือกรูปร่างของไฟล์ ซึ่งเป็นเหตุผลว่าทำไมแต่ละบ้านถึงตั้งชื่อ array เองได้ เก็บหลักเสริมเป็น string ได้ หรือสะกดหลักปลอมเป็น `tie\_bar` ได้ โดยไม่ผิดกฎที่เขียนไว้เลยสักข้อ

ต้นเหตุจริง: spec ถูกเขียนแบบค่อยๆ เติมทีละกฎในระดับฟิลด์ โดยไม่มีสัญญาเชิงโครงสร้างระดับบนสุด อีสระเชิงโครงสร้างจึงไม่มีอะไรมาจำกัดตั้งแต่ต้น

เสียหายอะไร: บ้าน 11 หลังที่มีโครงสร้างแตกต่างกันไปคนละทาง ต้องตามแก้ย้อนหลังหลายร้อยจุดข้ามหลายเซสชัน

แก้ยังไง: เพิ่ม §0 FORMAT LOCK ตัวใหม่ไว้บนสุดของ spec (10 กฎย่อย: ตัวห่อ, ฟิลด์ที่ element ต้องมี, element_id = มาร์คที่พิมพ์บนแบบ, รายการ element_type มาตรฐาน, หน่วยเป็น mm จำนวนเต็ม, หลักเสริมต้องเป็น object, printed_as, สัญกรณ์ grid_ref, ห้ามแต่ง pattern เอง, checklist ปิดท้าย) และเขียน `tools/check\_format.py` เพื่อบังคับตรวจสอบครบทั้ง 14 ข้อ โดย exit 1 เมื่อไม่ผ่าน

กฎที่ได้: spec ของข้อมูลต้องลือกรูปร่างของตัวห่อก่อน แล้วค่อยกำหนดความหมายของฟิลด์ และทุกกฎต้องตรวจด้วยเครื่องได้ชุดข้อมูล 1000 หลังไม่มีทางตรวจด้วยมือไหว

material_list กินไป 435 ไฟล์ = 37% ของแรง annotation ทั้งหมด แล้วให้ element เป็นศูนย์

เกิดอะไรขึ้น: พอนับ dataset จริง (11 หลัง 1,180 ไฟล์ 4,625 element) พบว่า material_list กินไป 435 ไฟล์ = 37% ของงาน annotation ทั้งหมดที่ทำมา แต่ผลิต element ได้ = 0 เพราะมันเป็น pattern แบบตาราง ไม่มีตำแหน่งของ element เลย ถ้าทำ annotation เฉพาะ pattern ที่ Constant ใช้จริง บ้านหนึ่งหลังจะลดจาก ~107 หน้าเหลือ ~38 หน้า

ต้นเหตุจริง: ขอบเขตงาน annotation ถูกกำหนดว่า 'ทุกหน้าของทุกหลัง' โดยไม่เคยวัดว่า pattern ไหนสร้างมูลค่าให้ปลายทางบ้าง สัญชาตญาณที่ว่าต้องทำให้ครบทุกหน้า ไม่เคยถูกเอาไปเทียบกับสิ่งที่ปลายทางอ่านจริง

เสียหายอะไร: ประมาณ 37% ของแรงคนที่ทำ annotation มาทั้งหมด ไม่ได้ element เลยสักตัว และถ้าเดินตามเป้าที่จะเพิ่มอีก 20 หลัง จะกลายเป็นต้องทำ 2,140 หน้าแทนที่จะเป็น ~750 หน้า คือเสียแรงเปล่า 65%

แก้ยังไง: สกิล op04 ตัวใหม่ข้าม material_list เป็นค่าตั้งต้น พร้อมเขียนหลักฐาน 435 ไฟล์/37%/element เป็นศูนย์ ไว้ในตัวสกิลเลย เขียน dataset_sizing.md พร้อมตัวเลขจริง และข้าม Pass 3 ทั้งหมดเป็นค่าตั้งต้นด้วย

กฎที่ได้: ก่อนจะขยายงาน annotation ให้ใหญ่ขึ้น ให้นับก่อนว่าแต่ละ pattern ผลิต element ได้กี่ตัวต่อไฟล์ แล้วตัด pattern ที่ให้ผลเป็นศูนย์ทิ้ง — วัดจาก dataset ที่มีอยู่ในมือแล้ว ดีกว่าเดาว่าทำให้ครบทุกหน้าคือทางที่ปลอดภัยที่สุด

สรุปผิดว่า Unsloth กลืน grammar constraint ไป แล้วเอาข้อสรุปนั้นไปเขียนลง SUMMARY.md

เกิดอะไรขึ้น: หลังจากกรอบแรกที่เปิด grammar แล้วยังได้ JSON ที่ parse ไม่ได้ ก็รีบสรุปทันทีว่าตัวโหลดของ Unsloth คงทั้ง grammar processor ไป แล้วเอาข้อสรุปนั้นไปเขียนลง SUMMARY.md เลย ซึ่งผิด พอไล่อ่านโค้ดที่ละชั้นก็พบว่า Unsloth ส่งต่อทุกอย่างครบ และ grammar ทำงานอยู่ตลอด ต้นเหตุจริงคือ lm-format-enforcer ถือว่าคอมมาท่าย ({ "a": 1, }) เป็นของถูกต้อง แต่ json ของ Python ไม่ถือ พิสูจน์ได้ด้วยบรรทัดเดียว: ลบคอมมาท่ายออก ไฟล์ที่ว่า "พัง" ก็ parse ได้ทันที

ต้นเหตุจริง: เลือกผู้ต้องสงสัยที่ฟังดูเข้าท่า แทนที่จะไปอ่านเส้นทางโค้ดจริง แล้วเอาการเดาไปบันทึกเป็นข้อค้นพบ ไม่ใช่เป็นสมมติฐาน เพราะเรื่อง "สองไลบรารีนิยามคำว่า JSON ที่ถูกต้องไม่ตรงกันแบบเงิบๆ" มันเป็นคำอธิบายที่ฟังไม่มันเท่า "ตัวหอกกลืนมันไป" ผู้ต้องสงสัยที่ผิดเลยนะ

เสียหายอะไร: ต้นเหตุที่ผิดแต่เขียนอย่างมั่นใจ นิ่งอยู่ใน SUMMARY.md ของโปรเจกต์ พร้อมจะพาคนอ่านคนถัดไป (หรือตัวเราเอง อีกสามเดือนข้างหน้า) ไปผิดทาง ในไดอารี่บันทึกว่านี่คือบทเรียนที่แพงที่สุดของคืนนั้น และกลายเป็นกฎ rule_of_tune ข้อ 14

แก้ยังงัย: ไล่หาต้นเหตุจริงด้วยการอ่านโค้ดที่ละชั้น เปลี่ยน backend ของ grammar ตัวตั้งต้นไปเป็น xgrammar (ซึ่งไม่มีทางพ่นคอมมาท่ายออกมาได้ตั้งแต่ต้นทาง) โดยมี lm-format-enforcer เป็นตัวสำรอง แก่ SUMMARY.md ให้ถูก และบันทึกเหตุการณ์นี้เป็น rule_of_tune ข้อ 14

กฎที่ได้: อย่าเขียนต้นเหตุลงเอกสารจนกว่าจะพิสูจน์ได้ด้วยการอ่านโค้ดจริงหรือทำซ้ำได้ด้วยบรรทัดเดียว — ถ้าหลักฐานยังไม่พอ ให้เขียนว่า 'ยังไม่รู้ต้นเหตุ สงสัย A หรือ B' ข้อสรุปที่ผิดแต่เขียนอย่างมั่นใจในเอกสาร อันตรายกว่าการไม่มีข้อสรุปเลย

Two subtasks both reading 0% recall were nearly conflated as the same problem (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: plan_beam and section both reported 0% id-recall. They are different diseases: plan_beam sits under an 88–93% achievable ceiling (a genuine model failure), while section's ceiling on the test house is 1/23 = 4% (a perfect model scores at most 4%, so the metric measures almost nothing).

ต้นเหตุจริง: Recall was reported as a bare percentage with no accompanying ceiling, so two structurally different failures produced an identical-looking number.

เสียหายอะไร: Conflating them would have aimed an entire tuning round (and its GPU rental) at a subtask where the model cannot possibly score, while leaving the real failure untouched.

แก้ยังงัย: Built `measure\_id\_ceiling.py` to compute the achievable ceiling per subtask and reported ceiling next to actual for all six subtasks; split reporting into `recall` and `recall\_printed` in the eval script.

กฎที่ได้: Never report a score without its achievable ceiling. Two identical percentages can be opposite problems, and the ceiling is what tells them apart.

The "downscaling causes plan_beam's 0% recall" hypothesis was concluded from token math alone and then disproven visually (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: An earlier research pass concluded from token arithmetic that image downscaling was the cause of plan_beam's 0% recall. A direct visual test — cropping the most-downscaled real training image (บ้าน_ใหญ่_1ชั้น_01 หน้า15) at native resolution and at the actual scale the model receives at MAX_PIXELS=7680 — showed grid bubbles, beam marks, column marks and dimension text equally legible in both.

ต้นเหตุจริง: A causal claim was made from an indirect proxy (token count) without ever looking at what the model actually sees. Token loss and legibility loss are not the same thing.

เสียหายอะไร: The team was about to buy image resolution (and consider an H200 rental) to fix a problem resolution does not cause. The real correlate found instead is GT JSON label length: plan_beam's answer is 2.3× longer than plan_footing's from the same sheet at the same resolution.

แก้ยังไง: Ran the visual comparison on real training data (no GPU needed), recorded the contradiction, and reframed the working theory as "the model fails to produce one long internally-consistent JSON blob in a single completion", with a recommended next step of splitting plan_beam into per-grid-zone sub-answers merged by code.

กฎที่ได้: Never conclude a perception failure from token math. Render what the model actually receives and look at it — a five-minute crop-and-compare beats an hour of arithmetic and can save a GPU rental.

Scaled straight to 4 parallel full trainings (~60 GPU-hours, ~\$63) before ever cheaply validating that the model could read a drawing at all (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The round went from 'dry-run 5 steps passed' straight to 4 folds × 306 steps running in parallel on 4 rented cards, ~15-16 h each. Every one of the 4 ran to completion and produced an adapter, and only after all 4 finished and eval scored 0% on plan_beam/plan_footing did the investigation find that `crop\_to\_patches: true` was missing, so no image was ever legible at training time. The entire round's GPU spend produced 4 adapters that cannot read drawings. The corrective rule written for t05 says it plainly: train one fold first, do not start with 4 parallel folds.

ต้นเหตุจริง: The dry-run only proved the plumbing (VRAM fits, loss is a number, checkpoint saves) — it never proved the model was seeing readable pixels. There was no cheap gate between 'config parses / 5 steps run' and 'commit 60 GPU-hours across 4 cards'. Scale-out was decided on schedule pressure ('เวลาจริงลดครึ่ง 29→~16 ชม.') and budget-permission ('อย่าสนงบ ผมจ่ายเอง'), not on a validated config.

เสียหายอะไร: ~\$63 of GPU rental plus ~60 GPU-hours and roughly 24 h of wall-clock across two days, producing 4 unusable adapters. Also forced 4 separate HF uploads, 4 sha256 verifies, 4 destroys, and an eval round on top.

แก้ยังไง: Not fixed for t04 (money already spent). Carried into t05 as an explicit rule: single fold first, plus a CPU-only image-token probe before any paid training run.

กฎที่ได้: Parallel scale-out multiplies whatever the config does — including the bug. Before committing more than one card, run one cheap end-to-end check that proves the model perceives the input (probe tokens per image, or eyeball a resized sample), not just that the training loop executes.

วิกฤต 2026-08-26

Repo-wide JSON pretty-print pass ran over 343 files including 16 a teammate had just finished checking (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Commit `be9ad80` ("แก้ไขระบบ") ran a repo-wide JSON pretty-print pass touching 343 files under json_แก้ไขแล้ว, including 16 files across houses 24, 26, 27 that Ark had committed only moments earlier. Discovered days later while tracing house 42's history.

ต้นเหตุจริง: A broad automated pass was run without first checking `git log` for other people's fresh commits in the same paths. No ownership or freshness check gated the operation.

เสียหายอะไร: Ark's just-finished work was reformatted underneath him without warning. Required a deep semantic diff (custom Node script, key-order-normalizing, treating element_id:"name" vs element_id:null+element_name_assigned as equivalent) across all 343 files to prove no data was lost, plus asking Ark to freeze push/pull and back up his local copies, plus a written incident note and formal apology.

แก้ยังไง: Verified 0 real data differences — only formatting changed. Published an incident note + Thai apology letter as an artifact, apologizing for the process (not for data loss, since there was none).

กฎที่ได้: Before any repo-wide automated pass, run `git log --since` on the target paths and check whose work you are about to touch. If someone else committed there recently, ask first. A format-only change is still a change to someone else's working state.

วิกฤต 2026-08-29

Model switch to InternVL3-78B invalidated the entire Unsloth toolchain — and the model had been rejected three days earlier (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Makham directed t04 to use OpenGVLab/InternVL3-78B. Unsloth does not support InternVL3 (issues #3401/#2341), so train_t03.py, infer_house_t03.py and onstart.sh all became unusable and were marked 🚫. A single 96GB card is also insufficient (LoRA bf16 needs $\geq 2 \times H100$ -class, or an untested single-card QLoRA 4-bit attempt). This same model had been rejected in the 2026-08-26 catalog on the grounds its backbone is Qwen2.5-72B anyway.

ต้นเหตุจริง: The model catalog evaluated model *quality* (OmniDocBench 83.76, license) but not *toolchain compatibility* or *hardware fit*. Framework support and VRAM requirements were not selection criteria, so the cost of a switch was invisible until after the decision.

เสียหายอะไร: Three working scripts became reference-only. Requires selecting a new framework, a new GPU strategy, a dataset-format converter (Unsloth-style jsonl $\rightarrow$ new framework format), rewritten train+infer scripts, and a fresh xgrammar/tokenizer probe. Also invalidated same-day work: MAX_PIXELS 5120 $\rightarrow$ 7680 and OUT_DIR t03 $\rightarrow$ t04 fixes were made just before the model change landed.

แก้ยังไง: Marked all three scripts 🚫 with headers and kept them as Qwen-era reference. Wrote t04_Purson/t04_workflow.md as a Phase 0 doc naming every gap. Framework subsequently decided: LLaMA-Factory (ms-swift held in reserve). Prompts/dataset/GT/CV sidecars are plain text and carry over — only the jsonl envelope needs converting.

กฎที่ได้: Make framework support and single-card VRAM fit hard gates in the model-selection catalog, alongside benchmark and license. A model your training stack cannot load is not a candidate, however good its scores.

เอา `model.generate()` หนีบล็อกไปใส่ใน `async def` ทำให้คำขอเดียวที่หลุดลูปแวนเชิร์ฟเวอร์ inference ทั้งตัวนาน 7 ชั่วโมงกว่า

เกิดอะไรขึ้น: ยิงคำขอพร้อมกัน 2 อัน (เทส e2e กับ arm 2.4a) เข้า `serve_purson.py` หลังจากนั้นเซิร์ฟเวอร์หยุดตอบทุกอย่างนานกว่า 7 ชั่วโมง แม้แต่ `/v1/models` ซึ่งเป็น endpoint เช็คสุขภาพที่ง่ายที่สุดก็ไม่ตอบ ตรวจแล้ว GPU ยังมีชีวิต (utilization นิ่งที่ 11-14%, process state R, ไม่มี error Xid/ECC) ต้นเหตุคือ `model.generate()` ถูกเรียกตรงๆ ในตัวจัดการที่เป็น `async def` พอมีคำขอหนึ่งตกลงไปในลูปพนข้อความซ้ำ (ปัญหาที่รู้จักกันตั้งแต่ t03 — grammar คุณได้แค่โครงสร้าง JSON ไม่ได้คุมเนื้อหา) มันก็บล็อก event loop ทั้งเส้น

ต้นเหตุจริง: เอาฟังก์ชันที่ทำงานยาวและบล็อก (ใช้ CPU/GPU หนัก) ไปวางตรงๆ ใน coroutine ของ `asyncio` ซึ่ง `async def` ของ FastAPI ไม่ได้ช่วยให้ทำงานพร้อมกันได้เลยถ้าข้างในเป็นการเรียกแบบบล็อก เพราะ event loop เป็น thread เดียว generate ที่ค้างอันเดียวจึงทำให้คำขออื่นอดหมด รวมถึง health check และดูเหมือนว่าตัวเช็ค timeout เองก็โดนด้วย

เสียหายอะไร: เสีย GPU เข้าไปเปล่าๆ กว่า 7 ชั่วโมงที่ราคา \$1.070/ชม. ระหว่างที่เซิร์ฟเวอร์เข้าถึงไม่ได้เลย และต้องปิดเครื่องด้วย `presentation.py down` ก่อนที่เทสเส้นทาง browser กับโมเดลจริงจะเสร็จ ณ สิ้นวันยังแก้ไม่เสร็จ และถูกระบุว่าเป็นตัวขวางอันดับหนึ่งก่อนพรินต์

แก้ยังง: ยังไม่ได้แก้ แต่บันทึกวิธีแก้ไว้แล้ว: ห่อ `generate()` ด้วย `asyncio.to_thread()` หรือทำคิว worker thread แยก เพื่อไม่ให้คำขอเดียวล้มเซิร์ฟเวอร์ทั้งตัวได้

กฎที่ได้: ห้ามเรียกฟังก์ชันโมเดล/IO ที่บล็อกตรงๆ ใน `async def` เด็ดขาด — ต้องใช้ `asyncio.to_thread()` หรือคิว thread/process เสมอ และก่อนจะเชื่อว่าเซิร์ฟเวอร์ใช้ได้ ต้องยิงคำขอพร้อมกัน 2 อันโดยให้อันหนึ่งกินเวลานาน เพื่อพิสูจน์ว่ามันรับพร้อมกันได้จริง

บั๊กเรื่องความละเอียดรูป/จำนวน token ตัวเดิม ทำให้เรนฟ้งเป็นรอบที่สามติด (t01 → t02 → t04) เพราะสองรอบแรกไม่เคยตั้งขั้นตอนตรวจการไว้เลย

เกิดอะไรขึ้น: t01 ฟ้งเพราะ Unsloth ถอยไปใช้ 512px เอง, t02 ฟ้งเพราะ data_collator ย่อรูปแล้วไม่เคยมีใครเทียบกับที่ตั้งใจไว้, t04 ฟ้งเพราะไม่รู้ว่ามี `crop_to_patches` อยู่ สาม framework คนละตัว สามกลไกคนละแบบ แต่อาการเดียวกันเป๊ะ: โมเดลถูกป้อนรูปที่เล็กเกินกว่าจะอ่านออกโดยไม่มีอะไรฟ้อง กว่าจะมีการเขียนกฎขึ้นมา (rule_of_tune.md ข้อ 15) ให้ตรวจจำนวน token ต่อรูปจาก source แล้วรัน probe บน CPU ก่อนจ่ายเงินเรน ก็ต่อเมื่อเกิดเรื่องเป็นครั้งที่สามแล้ว

ต้นเหตุจริง: หลัง t01 กับ t02 มีการแก้บั๊กเฉพาะตัวไป แต่ไม่มีใครเพิ่มขั้นตอนตรวจการที่ใช้ได้กับทุก framework ลงในเช็คลิสต์ก่อนเรน พอเปลี่ยน framework/โมเดลใหม่ ความผิดพลาดเดิมก็กลับมาทางประตูใหม่ และต้องนั่งหาต้นเหตุใหม่ทุกครั้งหลังจากที่เงินหมดไปแล้ว

เสียหายอะไร: เสียการเรนไป 3 รอบเพราะข้อผิดพลาดชนิดเดียวกัน เฉพาะ t04 รอบเดียวคิดเป็นประมาณ \$63 และ GPU ประมาณ 60 ชั่วโมง

แก้ยังง: บันทึกเป็น rule_of_tune.md ข้อ 15: ก่อนจะเรนจริงบน framework/โมเดลคู่ใหม่ ต้องไล่อ่าน source เพื่อดูจำนวน token ต่อรูปจริงๆ แล้วยืนยันด้วย probe ที่รันบน CPU อย่างเดียว ไม่ต้องใช้ GPU

กฎที่ได้: บั๊กที่กลับมาซ้ำซ้ำรอบ แปลว่าเช็คลิสต์ขาดข้อ ไม่ใช่ดวงช่วย พออาการเดิมโผล่มาเป็นครั้งที่สอง ให้หยุดแก้เฉพาะคสนั้น แล้วเพิ่มด่านตรวจก่อนเริ่มงานแบบถาวรแทน

วิกฤต 2026-08-31

rsync resume ไม่ได้ทำอะไรเลยแบบเงียบๆ: ไปเช็คเครื่องมือผิดเครื่อง แล้ว error ถูกกลบด้วย exit 0 กับ `| tail`

เกิดอะไรขึ้น: scp ไป gpu4 ค้างอยู่ที่ 83% เลยพยายาม resume ด้วย rsync --partial --append-verify แต่ไปเช็ค `which rsync` แค้นเครื่องปลายทาง ไม่ได้ใช้บน Git Bash เครื่องตัวเองซึ่งไม่มี rsync ติดตั้ง คำสั่งจึงล้มด้วย 'command not found' แต่ task รายงาน exit 0 เพราะ echo ที่ต่อท้ายทำงานสำเร็จ แล้ว `| tail -2` ก็ตัดข้อความ error ที่ไฟล์บน gpu4 จึงเป็นแค่ scp ที่ถูกฆ่ากลางทาง

ต้นเหตุจริง: ความผิดพลาดอย่างซ้อนกัน: เช็ความีเครื่องมือผิดเครื่อง, อ่านค่า exit code ของคำสั่งรวมแทนที่จะอ่านของคำสั่งที่สนใจจริงๆ, และตัด output ที่มีข้อความความล้มเหลวอยู่ทิ้ง

เสียหายอะไร: gpu4 เกือบเหวไปโดยที่รูปหาย 60 ใบแบบเงียบๆ (md5 ไม่ตรง 2aa1039d... เทียบกับ 4b765a37..., แตก tar ไม่ได้ ไม่ครบ มีรูปแค่ 824 จาก 884 ใบ) จับได้เพราะมีการตรวจ md5 เป็นขั้นตอนบังคับเท่านั้น

แก้ยังไง: ส่งไฟล์ใหม่แล้วตรวจ md5 ให้ตรง พร้อมบันทึกกฎสองข้อไว้ให้ชัด

กฎที่ได้: เช็ความีเครื่องมือหรือเปล่าบน 'เครื่องที่จะรันคำสั่งนั้น' ไม่ใช่เครื่องปลายทาง และห้ามตัดสินว่าสำเร็จจาก exit code ของคำสั่งที่ต่อกันหลายตัว หรือจาก output ที่ถูกตัดมาแค่บางส่วน — ต้องอ่าน stderr ตัวจริงและยืนยันด้วย checksum

วิกฤต 2026-08-31

ปัญหา decoder ถูกบันทึกและทดสอบไว้ตั้งแต่ 2026-08-30 แต่ไม่เคยเอาไปแก้ใน `serve_purson.py`

เกิดอะไรขึ้น: `no_repeat_ngram_size=8 + repetition_penalty=1.15` ถูกระบุไว้แล้วตั้งแต่ 30 ส.ค. ว่าเป็นปัญหาซ้ำเติมของ t04 พร้อมผลทดสอบว่าเอาสองตัวนี้ออกแล้วจำนวน element ขยับจาก 0 เป็น 10 แต่ `serve_purson.py` ไม่เคยถูกอัปเดต ข้อบกพร่องเดิมจึงทำให้วันนี้ได้ 0% และหล่นออกมาเป็นภาษาจีน

ต้นเหตุจริง: บทเรียนถูกเขียนลงบันทึกประจำวัน แต่ไม่ได้เอาไปแก้ในทุกไฟล์ที่มีโค้ดตัวเดียวกันอยู่

เสียหายอะไร: เสียการวัดผลไปทั้งรอบ เพื่อทำซ้ำความล้มเหลวที่วินิจฉัยไว้ตั้งแต่เมื่อวานแล้ว

แก้ยังไง: แก้ใน `serve_purson.py` แล้ว พร้อมบันทึกกฎว่าต้องกระจายการแก้ไขไปที่

กฎที่ได้: เมื่อบันทึกประจำวันเขียนว่า 'X เป็นปัญหา' ให้ grep ทั้งโปรเจกต์หา X แล้วแก้ทุกจุดในวันเดียวกันเลย — บทเรียนที่จดไว้แต่ไม่ได้เอาไปใช้ คือบทเรียนที่ยังไม่ได้เรียนรู้

วิกฤต 2026-08-31

ผ่านไปอีกวันหลังเทรนเสร็จ โดยไม่มีใครรู้ว่าโมเดลอ่านออกจริงหรือเปล่า

เกิดอะไรขึ้น: เทรนเสร็จ รวม adapter แล้วอัปโหลดขึ้นเรียบร้อย กว่าจะมีคนวัด recall จริงก็วันถัดไป แล้วปรากฏว่าได้ 0% ภายใต้อาคาร generate ที่ส่งขึ้นไปจริง

ต้นเหตุจริง: pipeline การเทรนไม่มีขั้นตอนวัด recall แบบอัตโนมัติเลย การตัดสินว่าสำเร็จดูจากค่า loss กับดูว่ามีไฟล์ผลลัพธ์อยู่บน HuggingFace เท่านั้น

เสียหายอะไร: เสียเวลาไปเต็มวันและจ่ายค่า GPU เพิ่ม ไปกับโมเดลที่ไม่มีใครรู้ความสามารถจริง และตามที่ตั้งค่าไว้คือศูนย์

แก้ยังไง: ระบุเป็นสิ่งที่ต้องเพิ่มในรอบถัดไป: ให้มีการวัด recall อัตโนมัติอยู่ในรอบการเทรนเลย

กฎที่ได้: ทุกรอบการเทรนต้องจบด้วยการวัด recall ระดับงานจริงแบบอัตโนมัติบนบ้านที่กันไว้ไม่ให้โมเดลเห็น ก่อนจะประกาศว่าเสร็จ — ค่า loss กับการอัปโหลดสำเร็จ ไม่ได้พิสูจน์ความสามารถอะไรเลย

ปัญหาความละเอียดรูปโพลเป็นรอบที่สามติดกัน ทั้งที่มีบทเรียนมาแล้วสองครั้ง

เกิดอะไรขึ้น: t01 (Unsloth ถอยไป 512px), t02 (data_collator ย่อรูปโดยไม่มีใครเทียบ) และ t04 (ไม่รู้ว่ามี crop_to_patches) ล้วนเป็นปัญหาชนิดเดียวกัน คือโมเดลเรนด้วยความละเอียดต่ำกว่าที่ตั้งใจไว้มากแบบเสียวๆ บรรทัด config ของ t04 เขียนโดย Claude เมื่อ 2026-08-29 ทั้งที่มีบทเรียนมาแล้วสองครั้ง

ต้นเหตุจริง: บทเรียนก่อนหน้านี้ถูกบันทึกเป็นเรื่องเล่าเกี่ยวกับพารามิเตอร์ตัวใดตัวหนึ่ง แทนที่จะแปลงเป็น assert บังคับก่อนรันที่ครอบคลุมปัญหาทั้งชนิด

เสียหายอะไร: เสียการเรนทั้งรอบที่ใช้ 4 GPU บวกเวลาเต็มวัน และต้องทิ้งโมเดล — เป็นการซ้ำรอยครั้งที่สามของปัญหาชนิดเดียวกัน

แก้ยังง: รอบนี้เพิ่ม preflight_seq_lengths() และตรวจนับ tile/token จริงบนเครื่องเรน ปัญหาชนิดนี้จึงมีตัวตรวจที่รันได้จริงคุมอยู่แล้ว

กฎที่ได้: ความล้มเหลวชนิดไหนที่เกิดซ้ำ ให้แปลงเป็น assert บังคับก่อนรันที่รันได้จริง — บทเรียนที่เขียนเป็นข้อความบรรยายพิสูจน์แล้วว่ากันการเกิดซ้ำไม่ได้ถึงสามครั้ง

ปัญหาชนิด 'รูปถูกย่อแบบเสียวๆ' ตัวเดียวกัน ฆ่าการเรนไปแล้ว 3 รอบ (t01, t02, t04)

เกิดอะไรขึ้น: t01: Unsloth ย่อรูปเป็น 512px เสียวๆ โดยไม่บอกใคร t02: ตัวแพ็คข้อมูลย่อรูป แล้วไม่มีใครเทียบขนาดก่อน-หลังเลย t04: ไม่มีใครรู้ว่ามีสวิตช์ crop_to_patches อยู่ สามกลไกคนละแบบ แต่ล้มเหลวเหมือนกันเป๊ะ และไม่มี error โพลมาสักครั้ง

ต้นเหตุจริง: หลังเกิดสองครั้งแรก ไม่มีการติดตัวกันถาวรไว้เลย แต่ละรอบถูกมองว่าเป็น config ผิดครั้งเดียวจบ แทนที่จะมองว่าเป็นความล้มเหลวชนิดที่เกิดซ้ำได้ ถ้าไม่มีการวัดบังคับก่อนรัน framework หรือตัวโหลดข้อมูลตัวใหม่ก็จะพาค่าย่อรูปเริ่มต้นของตัวเองกลับมาอีก

เสียหายอะไร: เสียการเรนไป 3 รอบ ได้โมเดลที่ใช้ไม่ได้ทั้งหมด แคมค่าใช้จ่ายทบขึ้นเพราะทุกรอบต้องนั่งไล่หาต้นเหตุใหม่ราวกับเป็นเรื่องใหม่

แก้ยังง: บันทึกเป็น rule_of_tune.md ข้อ 15 — บังคับวัด token ต่อรูปก่อนเรน ทำบน CPU ฟรีๆ — ซึ่งเปลี่ยนจากการค้นพบเฉพาะกิจซ้ำๆ ให้กลายเป็นด่านตรวจถาวร

กฎที่ได้: พอลความล้มเหลวแบบเดิมโพลมาครั้งที่สอง ให้หยุดแก้เฉพาะเคส แล้วติดตัวกันอัตโนมัติแบบถาวรแทน บั๊กที่เกิดซ้ำถึงสามครั้งคือกระบวนการที่ขาดไป ไม่ใช่ config ผิดสามครั้ง

สูง 2026-07-06

Four schema open questions were written into the doc but never actually put to the decision maker

(ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Generation 3.1 opened 4 new questions — asymmetric top/bottom main_bar, formal confirmation of additional_bars[], multi-level column schedules (separate `level` field vs embedded in element_id), and precast plank fields (dowel_bar/topping_mesh/topping_thickness_min_mm/level_step_mm). The diary states plainly they had 'still not been asked' to Makham as of the end of July 6, despite him being present and working in the same session all day.

ต้นเหตุจริง: Questions were recorded into a document instead of being asked in the live conversation. Writing a question down feels like progress and removes the pressure to actually ask it, especially when the person is right there and work is flowing.

เสียหายอะไร: Four schema decisions remained unmade while extraction continued around them, meaning more data was produced under assumptions that may later be reversed. All four rolled forward as pending item #2 at the end of the day.

แก้ยังไง: Not fixed — explicitly recorded as 'still not asked as of today (July 6) either — genuinely still pending'.

กฎที่ได้: If the decision maker is in the session, ask the blocking question in the session — do not file it. A question written into a doc is a question that will be asked days late, or never, while data keeps being produced under the wrong assumption.

สูง 2026-07-06

The pending list grew to 9 items with several rolling forward untouched across two full working days

(ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Every entry in the 2026-07-06 diary ends with 'All prior pending items remain the same', and the day closed with 9 open items. Several — pages 41-47 BOQ, the 210 vs 240 concrete grade conflict, the empty 0-byte file — had already been open since 2026-07-05 and were repeated verbatim in six consecutive entries without action.

ต้นเหตุจริง: The pending list was append-only with no owner, no size limit and no triage step. Each session started new work rather than closing old, and the ritual of restating the list substituted for acting on it.

เสียหายอะไร: Two of the rolled-forward items are data-correctness blockers (missing BOQ pages, ambiguous concrete grade) that gate the usability of the entire house dataset, while the day's effort went into review tooling instead.

แก้ยังไง: Not fixed — the 9-item list was carried into SESSION_HANDOFF_2026-07-06.md as the starting point for the next session.

กฎที่ได้: Cap the pending list and triage it at the start of each session, not the end: anything repeated three sessions running gets done, dropped, or assigned an owner and a date. Restating a list is not tracking it — a pending item with no owner is a decision to never do it.

สูง 2026-07-06

Schema revved three times in one day (3.1 → 3.2 → 3.3) after extraction was already finished, forcing a full 37-page re-extraction plus hand-edits (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Pages 1-60 were extracted on 2026-07-05 under Gen 3. The next day the schema gained span_source + atomic beam segments (3.1), the dummy-grid convention + page-00 grid master (3.2), and spec-join-into-plan (3.3). Each change invalidated already-written data: a whole second round (t2) covering pages 1-37 was ordered and run, page 19 view2 and page 20 view1 were rewritten from scratch, and two more t2 files had beam spec hand-joined into every element afterwards.

ต้นเหตุจริง: The schema was still being designed while bulk extraction was already running in parallel — the open questions from 2026-07-05 (and four more opened on 07-06) were never closed before the data was produced, so every design decision landed as rework on existing files instead of as a constraint on new ones.

เสียหายอะไร: Roughly a full day of extraction output (pages 1-37, ~60 files) had to be redone, plus hand-editing of individual JSON files, plus regeneration of every downstream Label Studio task file. Pages 38-60 were left on the older schema, so the house's dataset is now internally inconsistent across page ranges.

แก้ยังไง: t2 round created for pages 1-37; pages 38-60 left at t1 schema and never brought forward

กฎที่ได้: Freeze the schema before bulk extraction, or extract one deliberately small pilot set until the schema stops moving. A schema change after N pages are written costs N pages of rework every time — batch the open questions and answer them all before dispatching agents.

สูง 2026-07-07

Grid master carries dummy grid A'' which may not be a real grid line at all, just a ground-leveling area (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: While resolving the overhang confusion on house 1, confidence went UP on dummy grids 1'/3'/A' but DOWN on A'' — the agent suspected A'' is not a real grid line but a ground-leveling area that was mistaken for one. A'' stayed in the grid master anyway.

ต้นเหตุจริง: A dimension line marking a slab/ground level was read as a structural grid line during an earlier extraction round, and the grid master was built before anyone cross-checked each dummy grid against a second sheet.

เสียหายอะไร: The grid master drives grid_ref/grid_ref_start/grid_ref_end for every element in the house. A fabricated grid line means every element positioned against it carries a coordinate that does not exist on the drawing — and this one shipped into t3, t4, and the production 01 folder unresolved.

แก้ยังไง: not fixed — confidence was lowered and the suspicion recorded, but A'' was never removed or confirmed.

กฎที่ได้: A dummy grid must be confirmed by at least two independent sheets before it enters the grid master, the same bar later applied on house 3 (where 3 agents independently confirmed 1'/3'). A low confidence score is not a substitute for resolving it — grid master errors propagate to every element in the house.

สูง 2026-07-10

ไม่เคยตัดสินใจว่าจะแยกโปรเจกต์ต่อบ้าน หรือรวมทุกบ้านไว้โปรเจกต์เดียว จนบล็อกการ import อยู่หลายวัน

เกิดอะไรขึ้น: ตอนเช็คความพร้อมของบ้านหลังที่ 2 เจอตัวบล็อกสองเรื่อง เรื่องชื่อไฟล์ชนกันแก้เสร็จวันเดียวกัน ส่วนอีกเรื่อง — "ยังไม่ตัดสินใจว่า Label Studio จะแยกโปรเจกต์ต่อบ้าน หรือรวมทุกบ้านไว้ในโปรเจกต์เดิม" — ไม่เคยถูกตัดสินใจเลย และไฟล์ซ้ำแบบเดิมในรายการงานค้างของทุกบันทึกหลังจากนั้นจนถึง 11 ก.ค.

ต้นเหตุจริง: การตัดสินใจที่บล็อกงานถูกบันทึกเป็นแผนงานค้าง แทนที่จะบังคับให้เคาะให้จบ แล้วก็ทำงานอ้อมมันไปเรื่อยๆ บ้านหลัง 2 กับ 3 ถูกสกัดข้อมูลและสร้างไฟล์ task เสร็จ ทั้งที่ยังไม่รู้ว่าจะไปลงที่ไหน

เสียหายอะไร: พอถึงวันที่ 11 ก.ค. มีไฟล์ task ของบ้านครบสามหลัง (1, 2, 3) แต่ของหลัง 2 กับหลัง 3 ยังไม่ถูก import เข้า Label Studio เลยสักหลัง จุดประสงค์ทั้งหมดของการสกัดข้อมูล — เอาแบบไปวางตรงหน้าคน annotate — ยังไม่เกิดขึ้นเลยสำหรับสองในสามหลัง

แก้ยังไง: ยังไม่ได้แก้ — ยังเปิดค้างอยู่ในรายการงานค้างของบันทึกล่าสุด

กฎที่ได้: การตัดสินใจที่บล็อกขั้นตอนส่งมอบ ต้องบังคับให้เคาะก่อนจะขยายขั้นตอนการผลิต ผลงานสามหลังที่ไม่มีทีไป แยกว่าหลังเดียวที่เดินจนจบทาง

สูง 2026-07-12

งานสกัดข้อมูลหลายสัปดาห์ไม่ได้ commit ลง git เลย

เกิดอะไรขึ้น: ในบันทึกหลายวันติดกัน งานค้างประจำเขียนว่า "commit งานทั้งหมด (ยังไม่ได้ commit อะไรเลยตั้งแต่รอบ t4)" — raw JSON หัวหลังบวกกับการแก้ schema และกฎต่างๆ ค้างอยู่โดยไม่มีการติดตามใน git

ต้นเหตุจริง: การ commit ถูกมองเป็นงานแยกที่ค่อยทำทีหลัง ไม่ใช่ขั้นตอนหนึ่งในการทำงานต่อบ้านหนึ่งหลัง เลยถูกเลื่อนไปเรื่อยๆ ตามหลังงานสกัดข้อมูล

เสียหายอะไร: ไฟล์ JSON ที่ทำมือหลายร้อยไฟล์ ไม่มีประวัติเวอร์ชันและไม่มีจุดกู้คืน ถ้าเผลอเขียนทับหรือรัน regex กวาดผิด ก็กู้ไม่ได้เลย

แก้ยังไง: ยังไม่ได้แก้ในช่วงบันทึกเหล่านี้ — ยังค้างอยู่ในรายการงานค้าง

กฎที่ได้: commit ทุกครั้งที่จบบ้านหนึ่งหลัง ก่อนจะเริ่มหลังต่อไป ข้อมูลที่ทำมือและทำใหม่ไม่ได้ ถ้าไม่มี commit ก็ห่างจากการหายไปแค่สคริปต์ฟังตัวเดียว

การถูกฆ่าเพราะลิมิต session ทั้งไฟล์ JSON ที่เขียนไม่จบไว้ ซึ่งการนับจำนวนไฟล์จับไม่ได้

เกิดอะไรขึ้น: ไฟล์ `หน้า54\_roof\_floor\_plan.json` ขาดวงเล็บปีกกาปิด `}` ของ object `grid\_dimensions\_as\_printed` เพราะ agent ถูกตัดหลังจากเปิด object ซ้อนแล้วแต่ยังไม่ทันปิด ไฟล์นั้นมีอยู่และจำนวนหน้าก็ตรง มีแต่การ validate ด้วย JSON.parse เติมรูปแบบเท่านั้นที่จับได้

ต้นเหตุจริง: การเช็คความครบใช้วิธี "จำนวนไฟล์เท่ากับจำนวนหน้า" แต่การเขียนที่ถูกขัดจังหวะจะได้ไฟล์ที่มีอยู่แต่ใช้ไม่ได้ ซึ่งการเช็คแบบนั้นมองไม่เห็น

เสียหายอะไร: ไฟล์ที่ใช้ไม่ได้จะทำให้สคริปต์ label-studio และทุกอย่างที่กินข้อมูลจากนั้นพัง เจอได้เพราะบังเอิญไปรัน validate เท่านั้น

แก้ยังไง: เขียนสคริปต์ node แบบใช้ครั้งเดียวเพื่อหาตำแหน่งที่ปีกกาหายจริงๆ (ตัว parser บอกแค่ว่าพังที่ EOF) แล้วใส่ `}` ลงตำแหน่งที่ถูกต้อง จากนั้น validate ใหม่ทั้ง 138 ไฟล์

กฎที่ได้: จำนวนไฟล์ไม่ใช่การเช็คความครบ ให้รัน JSON.parse กับทุกไฟล์ที่ผลิตออกมาก่อนประกาศว่าชุดนั้นเสร็จเสมอ และเวลา parser บอกว่าพังที่ EOF ให้เขียนโปรแกรมหาตำแหน่งที่วงเล็บไม่สมดุลจริงๆ แทนที่จะเดาจากข้อความ error

ใช้โซ่ระยะซ้อนกันหลายชั้นบนผังพื้นเป็นหลักฐานหลัก แทนที่จะใช้ผังบริเวณ

เกิดอะไรขึ้น: ขอบเขตอาคารถูกคิดจากโซ่ระยะที่ซ้อนกันบนผังพื้น ($0.60+0.90=1.50$) แทนที่จะดูจากแผ่น A-03 ซึ่งเป็นผังบริเวณและระยะย่นชายคาไว้ตรงๆ ว่า 0.60 ม.

ต้นเหตุจริง: ไม่มีการกำหนดลำดับความน่าเชื่อถือของหลักฐานสำหรับคำถาม "ขอบอาคารจริงอยู่ตรงไหน" คนอ่านเลยใช้แผ่นที่เปิดอยู่ตรงหน้า และโซ่ระยะบนผังพื้นก็คลุมเครือว่าตัวเลขไหนคือขอบอาคารจริง

เสียหายอะไร: เป็นเหตุโดยตรงที่ทำให้ค่าของกริด `3` ผิด และยังทำให้อ่านผิดอีกจุดบนบ้านหลัง 2 หน้า 06 ที่เอา '0.60' กับ '0.90' มาบวกกันเป็นชายคายื่นต่อเนื่อง 1.50 ม. อย่างผิดๆ

แก้ยังไง: กำหนดว่าแผ่น A-03 (ผังบริเวณ) เป็นหลักฐานหลักสำหรับขอบเขตอาคาร เวลาตรวจ dummy grid ที่ยื่นนอกนอกกริดหลัก

กฎที่ได้: กำหนดลำดับความน่าเชื่อถือของหลักฐานแยกตามประเภทคำถาม สำหรับขอบเขตอาคารและระยะย่น ผังบริเวณชนะโซ่ระยะบนผังพื้น และระยะสองตัวที่ติดกันโดยใช้จุดร่วมกัน ไม่ได้แปลว่าเอามารวมกันได้โดยอัตโนมัติ

สูง 2026-07-13

dummy grid ตัวอื่นของบ้านหลัง 1 ไม่เคยถูกตรวจหาข้อบกพร่องอ้างอิงวนกลับที่ทำให้กริด 3' ผิด

เกิดอะไรขึ้น: หลังจากพบว่ากริด 3' ผิดเพราะหลักฐานที่ใช้ "ยืนยัน" มันคือคานที่มี span_source เป็น grid_table (คืออ้างอิงวนกลับไปที่ตารางกริดตัวเอง) บันทึกก็ระบุว่า A' กับ A'' มีรูปแบบการพึ่งพาแบบเดียวกัน — คาน B3X/B5X ก็ใช้ span_source: grid_table เหมือนกัน — และควรตรวจต่อ แต่การตรวจนั้นไม่เคยเกิดขึ้น

ต้นเหตุจริง: การแก้ทำเฉพาะกรณีเดียวที่คำถามพาไปเจอ ไม่ได้ทำกับทั้งกลุ่มที่การวินิจฉัยชี้ไว้ และไม่มีการรันสคริปต์กวาดหรือ grep หาค่า span_source ทั่วทั้งบ้าน

เสียหายอะไร: ยังมี dummy grid อีกสองตัวในบ้านหลังที่เป็นแหล่งข้อมูล fine-tune ที่ยังไม่ได้ตรวจกับความผิดพลาดรูปแบบเดียวกันนี้ ซึ่งเคยทำให้ได้ค่า 8.5 ม. ผิด และช่วงคาน 1.5 ม. ผิดมาแล้ว

แก้ง่าย: ยังไม่ได้แก้ — บันทึกเป็นงานค้างข้อ 1 และไม่ได้กลับมาทำในบันทึกวันหลังๆ

กฎที่ได้: เมื่อระบุได้แล้วว่าต้นเหตุจริงเป็นรูปแบบ (field ที่คำมาจากการอนุมาน ไม่ได้มาจากการวัด) ให้ grep หาทุกที่ที่รูปแบบนั้นโผล่ทันที อย่าปิดงานที่กรณีเดียวที่มีคนแจ้งเข้ามา

สูง 2026-07-14

อ้างกฎเรื่องการขออนุญาต แล้วแก้ไฟล์ในข้อความเดียวกันโดยไม่รอคำตอบ

เกิดอะไรขึ้น: Claude บอกว่ากำลังอ้าง `rule\_of\_tune.md` ข้อ 7 ก่อนจะแก้ `primary\_rawjson\_schema.md` แล้วก็แก้ทันทีในข้อความเดียวกันเลย ผู้ใช้จับได้และถามว่าทำไมถึงอ้างกฎแล้วไม่หยุดจริง เรื่องนี้เกิดมาแล้วสองครั้ง

ต้นเหตุจริง: การอ้างกฎถูกเข้าใจว่าเท่ากับทำตามกฎแล้ว ตัวกฎบอกแค่ทำให้เรียกอะไรขึ้นมา แต่ไม่ได้บอกว่าการเรียกมันขึ้นมาแปลว่าต้องจบเทิร์นและรอคำตอบ

เสียหายอะไร: สเปก — เอกสารกลางที่สำคัญที่สุดของโปรเจกต์ — ถูกแก้สองครั้งโดยไม่ผ่านการอนุมัติที่กฎกำหนด และต้องเขียนกฎใหม่ขึ้นมาปิดช่องโหว่

แก้ง่าย: บันทึกเป็นกฎข้อ 9 ใหม่ใน `rule\_of\_tune.md`: การอ้างกฎแปลว่าต้องหยุดและถามจริงๆ ห้ามอ้างกฎแล้วแก้ต่อในข้อความเดียวกัน

กฎที่ได้: ประตูป้อนอนุญาตเปิดได้ด้วยคำตอบเท่านั้น ไม่ใช่ด้วยการประกาศ ถ้าเอ่ยชื่อกฎที่ต้องขออนุมัติ ให้จบเทิร์นตรงนั้นเลย

สูง 2026-07-15

คานเจ็ดตัวหายไปจากหน้า 19 จนกว่าผู้ใช้จะส่งรูปที่ชัดกว่ามาให้

เกิดอะไรขึ้น: คานเจ็ดตัว (`B2 D'2-D'3`, `B4 B1-B2`, `B4 B2-B3`, `B2 B1''-A1''`, `B2 C3'-B3'', `B3 A1''-A3'', `B3 A1''-A3'`) หายไปจากผังคานทั้งหมด จนกระทั่งผู้ใช้ส่งรูปหน้า 19 แบบเต็มที่ชัดกว่ามาให้ จำนวน element ขยับจาก 20 เป็น 27

ต้นเหตุจริง: การสกัดข้อมูลรอบแรกทำจากรูปที่ความละเอียดไม่พอกับความหนาแน่นของเส้นบนแผ่นนั้น และไม่มีการเช็คความครบที่จะสังเกตได้ว่าคานหายไปทั้งหมด

เสียหายอะไร: นับคานขาดไป 26% บนแผ่นโครงสร้าง — แปลว่าปริมาณงานทุกอย่างที่คิดต่อจากนั้นผิดไปหมด และการสกัดข้อมูลไม่ครบแบบเดียวกันนี้โผล่อีกครั้งแบบหนักกว่าเดิมที่บ้านหลัง 3 หน้า 31/32

แก้ง่าย: เพิ่มครบทั้ง 7 ตัวด้วย confidence 1 หลังตรวจกับรูปที่ชัดกว่า

กฎที่ได้: สกัดข้อมูลจากแบบโครงสร้างด้วยความละเอียดที่พิสูจน์แล้วว่าพอกับความหนาแน่นของเส้นบนแผ่นนั้น และให้เทียบจำนวน element กับแหล่งอิสระอื่น (ผังคานคอดิน รูปตัด หรือตาราง) ก่อนจะบอกว่าแผ่นนั้นสกัดครบแล้ว

สูง 2026-07-15

บ้านหลัง 1 ตรวจสอบวิธีที่อ่อนกว่าบ้านหลัง 2 เลยเหลือ error ที่ยังไม่เจอค้างไว้

เกิดอะไรขึ้น: ตอนตรวจบ้านหลัง 1 ใช้แค่ลำดับของ label เป็นหลัก ไม่ได้ซูมเข้าไปนับจุดหลักเสริม ซึ่งเป็นเทคนิคที่ทำให้เจอบั๊ก B2/B2X ในบ้านหลัง 2 มีการบันทึกไว้แล้วว่าบ้านหลัง 1 น่าจะยังซ่อนความผิดพลาดแบบเดียวกันอยู่

ต้นเหตุจริง: วิธีตรวจถูกพัฒนาให้ดีขึ้นระหว่างทำโปรเจกต์ แต่ไม่มีใครวางแผนย้อนกลับไปตรวจซ้ำบ้านที่ตรวจด้วยวิธีเก่าที่อ่อนกว่า

เสียหายอะไร: บ้านหลัง 1 ตรวจไม่ครบและรู้ทั้งรู้ ยังมี error เรื่องหลักเสริมที่ยังไม่เจอค้างอยู่ในชุดข้อมูลที่จะเอาไปเทรนโมเดล

แก้ยังไม: ยังไม่ได้แก้ — บันทึกไว้เป็นงานค้างว่า "ควรพิจารณากลับไปตรวจบ้านหลัง 1 ใหม่ด้วยวิธีซูมแล้วนับจุด"

กฎที่ได้: พอปรับวิธีตรวจให้ดีขึ้นเมื่อไหร่ ให้ย้อนกลับไปตรวจซ้ำทุกอย่างที่เคยตรวจด้วยวิธีเก่าทันที ไม่งั้นคุณภาพของชุดข้อมูลจะไม่เท่ากันเสียๆ ขึ้นอยู่กับว่าส่วนไหนถูกตรวจตอนไหน

สูง 2026-07-20

เพิ่มฟิลต์กลบเกลื่อนบั๊กหลักเสา แทนที่จะแก้บั๊กจริง ทำให้มันซ่อนอยู่ในอีก 6 ไฟล์

เกิดอะไรขึ้น: ตอนไล่กวาดทั้งโปรเจกต์เพื่อหาบั๊กหลักบน/ล่างของเสา พบว่ามันซ่อนอยู่ในเสารั้ว C1 อีก 6 ไฟล์ (บ้าน 1 หน้า 16/17, บ้าน 2 หน้า 20/21, บ้าน 3 หน้า 26/27) ซึ่ง "เคยถูกกลบเกลื่อนไปด้วยการเพิ่มฟิลต์ `total\_main\_bar\_count` แทนที่จะแก้ไขบั๊กจริงๆ"

ต้นเหตุจริง: ตอนแรกทีเจอร่ายอตรมผิด แทนที่จะแก้โครงสร้างที่ทำให้ยอผิด กลับเพิ่มฟิลต์ใหม่มาชดเชยเพื่อบันทึกยอดที่ถูกไว้ โครงสร้างที่ผิดเลยยังอยู่และหยุดแสดงตัวว่าเป็นบั๊ก

เสียหายอะไร: บักรอดไปแบบไม่มีใครเห็นในอีก 6 ไฟล์ ซ้ำ 3 บ้าน จนกระทั่งมีการไล่กวาดทั้งโปรเจกต์แบบตั้งใจถึงเจอ

แก้ยังไม: แก้ทั้ง 6 ไฟล์ให้เหลือ `main\_bar.count` ตัวเดียว เพิ่ม `specs.CN` ที่หายไปหน้า 31 แล้วไล่กวาดทั้งโปรเจกต์ซ้ำเพื่อยืนยันว่าไม่เหลืออีก

กฎที่ได้: ห้ามเพิ่มฟิลต์ชดเชยมาบันทึกคำตอบที่ถูกไว้ข้างๆ โครงสร้างที่ผิดเด็ดขาด — ให้แก้โครงสร้าง ฟิลต์กลบเกลื่อนเปลี่ยนบั๊กที่มองเห็นให้กลายเป็นบั๊กที่ซ่อนตัว และการันตีว่ามันจะแพร่ต่อ

สูง 2026-07-21

วินิจฉัย OOM ผิดว่าเป็นเรื่องความยาว sequence กับ allocator เปลืองที่ แก้ไป 3 อย่างแล้วไม่เปลี่ยนอะไรเลย

เกิดอะไรขึ้น: แก้ไป 3 อย่าง (MAX_LENGTH 10240→9216, PYTORCH_ALLOC_CONF=expandable_segments=True, adamw_8bit→paged_adamw_8bit) แล้วรอบ 3 ก็ยัง crash ด้วย OOM แบบเดิมแทบไม่ต่างกัน (เหลือว่าง 98.25MB เทียบกับ 96.25MB) ต้นเหตุจริงคือ buffer ของ LoRA delta ที่อยู่ใน `forward\_native\_grouped\_mm` ซึ่งถูกคูณด้วย MoE expert 256 ตัว

ต้นเหตุจริง: เลือกวิธีแก้จากสมมติฐานที่ฟังดูเข้าเค้า แทนที่จะอ่านจุด crash จริงใน traceback การลดเพดานที่ตัวอย่างทดสอบไม่เคยไปถึงอยู่แล้ว มันช่วยอะไรไม่ได้ตั้งแต่แรกโดยหลักการ

เสียหายอะไร: เสียเวลา GPU ที่จ่ายเงินไปทั้งรอบ (fix-upload-run รอบ 3) ไปกับสิ่งที่ไม่ใช่ต้นเหตุ แถมยังทำให้เจอวิธีแก้จริงช้าลงอีก

แก้ยังไม: อ่าน traceback ที่ละบรรทัดจนเจอ `lora\_delta = torch.empty(...)` ในเส้นทางของ MoE expert แล้วหันไปจัดการที่ LoRA rank แทน

กฎที่ได้: อ่าน traceback ให้ถึงบรรทัดที่จองหน่วยความจำจริงๆ ก่อนจะไปแก้ค่าอะไรทั้งนั้น ถ้าวิธีแก้ที่คิดไว้อธิบายตัวเลขที่เห็นในเชิงกลไกไม่ได้ แปลว่ามันไม่ใช่วิธีแก้

สูง 2026-07-21

ลบ `merged/` ทิ้งก่อนที่จะสกัดส่วนที่ใช้อ่านภาพ (mmpoj) ออกมา

เกิดอะไรขึ้น: ลบโมเดลเต็มที merge แล้วทิ้งเพื่อเอาพื้นที่ดิสก์ ทั้งที่ยังต้องสกัดส่วน mmpoj (ตัวอ่านภาพ) ออกมาจากมันอยู่ ผลคือ GGUF ที่ได้อ่านได้แต่ข้อความ ทั้งที่โปรเจกต์นี้มีอยู่เพื่ออ่านแบบก่อสร้างโดยเฉพาะ

ต้นเหตุจริง: การลบไฟล์ถูกขัดด้วยความกดดันเรื่องพื้นที่ดิสก์เฉพาะหน้า โดยไม่มีเช็คลิสต์ว่าไฟล์แต่ละก้อนยังต้องถูกสกัดอะไรออกมาบ้างก่อนลบ

เสียหายอะไร: GGUF อ่านภาพไม่ได้เลย เส้นทางฝังวิชวลค้างไว้ไม่จบ และถ้าจะแก้ต้องสร้างโมเดล merged ขนาด 66GB ขึ้นมาใหม่

แก้ยังง: ยังไม่ได้แก้ — บันทึกไว้เป็นปัญหาที่ยังค้าง ตัว llama.cpp เองก็เตือนว่า --mmpoj "ใช้ได้แค่กับบางโมเดล" และยังไม่ได้ตรวจสอบว่ารองรับ Qwen3.6 ไหม

กฎที่ได้: ก่อนลบไฟล์ระหว่างทางก้อนใหญ่ทุกครั้ง ให้ใส่รายการขั้นตอนปลายทางทุกชั้นที่ยังต้องใช้ไฟล์นั้น แล้วยืนยันว่าแต่ละชั้นรันไปแล้วจริง

สูง 2026-07-21

ไม่เคยไปขอ token ของ HuggingFace เลย การสำรองข้อมูลตามแผนเลยทำไม่ได้

เกิดอะไรขึ้น: แผนคืออัปโหลดไฟล์ที่ tune เสร็จขึ้น HuggingFace แต่ไม่เคยสร้าง HF token ไว้ การอัปโหลดเลยถูกเลื่อนไปเรื่อยๆ และไฟล์ก็ยังอยู่แค่บนเครื่องเช่าตอนที่เครื่องถูกทำลายทิ้ง

ต้นเหตุจริง: credential ที่จำเป็นสำหรับขั้นตอนสำรอง/นำของออก ถูกทิ้งไว้ทำทีหลังสุด แทนที่จะเตรียมให้เสร็จก่อนเริ่มรันงานที่เสียเงิน

เสียหายอะไร: ตาข่ายนิรภัยชิ้นเดียวของงานทั้งชิ้น ใช้ไม่ได้พอดีในจังหวะที่ต้องการใช้

แก้ยังง: ยังไม่ได้แก้ในเซสชันนี้ — ใส่ไว้เป็นงานค้าง ("อัปโหลด qwen36-tuned-Q4_K_M.gguf ขึ้น HuggingFace — ต้องมี HF token ก่อน")

กฎที่ได้: เตรียมและทดสอบ credential ทุกตัวที่เส้นทางนำของออกต้องใช้ (HF token, key ของที่เก็บไฟล์) ให้เสร็จก่อนเข้า GPU — และลองอัปโหลดไฟล์เล็กๆ ดูจริงสักครั้งเพื่อพิสูจน์ว่าเส้นทางนั้นใช้ได้

สูง 2026-07-21

Whole merge→f16→quantize GGUF pipeline was run before checking whether llama.cpp supports this model's vision at all (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Hours were spent on merge_lora.py, three failed CUDA builds, repeated disk-full recoveries, an f16 conversion (~70GB) and a Q4_K_M quantization — and only at the end was it discovered that the resulting GGUF is text-only, and that llama.cpp's own docs warn `--mmap` "will only work for some models", with Qwen3.6 being new enough that it may not be supported at all.

ต้นเหตุจริง: Feasibility of the deliverable's core requirement (reading drawings = vision) was never verified before committing to the pipeline. The check that mattered most was left until last, when it should have been a 10-minute doc/issue search first.

เสียหายอะไร: An entire evening's compute, disk thrashing and effort produced a 21.2GB artifact that cannot perform the one job the whole project exists for; also lost with the instance.

แก้ยังไง: Not fixed — Makham decided to stop and rely on running through Python/Unsloth on a rented GPU instead of the GGUF route.

กฎที่ได้: Before committing to a multi-hour export/conversion pipeline, verify the *hardest* requirement first (here: does this tool support this architecture's vision component?). Order feasibility checks by risk, not by pipeline sequence.

สูง 2026-07-24

A pre-flight checklist existed as a document but had never actually been run (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The Phase 0 checks were written down but treated as reading material rather than as steps to execute. Rule 4 had to be added specifically to say the checks must be genuinely performed, not merely present in a document.

ต้นเหตุจริง: Documentation was conflated with execution. Writing the check felt like doing the check, and nothing in the workflow forced an actual pass/fail result to be recorded per item.

เสียหายอะไร: Every latent problem the checklist would have caught (disk default, MAX_LENGTH bug, token validity) survived into paid runtime.

แก้ยังไง: rule_of_tune.md Rule 4 rewritten: before every real rental, Phase 0 checks must actually be run; a results table with  per item was added.

กฎที่ได้: A checklist with no recorded per-item result is decoration. Require an artefact (output, number, screenshot) proving each item was executed, not just read.

สูง 2026-07-24

Phase 0 pre-rental checks existed only as a document and were not actually being run (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Rule 4 had to be added to `rule\_of\_tune.md` stating that before every real GPU rental, the Phase 0 checks must actually be run, not just sit there as a document.

ต้นเหตุจริง: A checklist was written but never made a blocking gate, so it was read rather than executed — which is how the MAX_LENGTH bug survived until a serious 0.4 re-read.

เสียหายอะไร: Documented checks that had not been executed; the MAX_LENGTH truncation bug and the disk-slider default were both only found once the checks were genuinely run.

แก้ยังไง: Added Rule 4 making Phase 0 execution mandatory before any GPU rental, then genuinely executed 0.1 through 0.5 (HF token tested, balance checked, local dry run on the real mmproj and a test model, script re-read, machine selected).

กฎที่ได้: A pre-flight checklist that is not a blocking gate is documentation, not a check. Make it produce evidence (a token test result, a real dry-run log) before money is spent.

สูง 2026-07-24

The mmproj blocker that killed the 21st was self-inflicted — the vision layers were frozen, so the vendor's own mmproj was always usable (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The 21st's round was declared broken because mmproj extraction failed (tooling did not support the model), leaving a GGUF that could not read images. On the 24th it emerged that the training script sets finetune_vision_layers=False — the image-reading half of the tuned model is byte-identical to the base model, so the vendor's published mmproj (899MB) could simply be downloaded and used. No extraction was ever required.

ต้นเหตุจริง: Nobody checked what the training script actually trains before assuming a custom mmproj had to be produced. The team debugged a tool for something the pipeline never needed.

เสียหายอะไร: The entire mmproj-extraction effort on the 21st, and the resulting 'model can't read images' failure of that round, were avoidable. The fix on the 24th was a 899MB download.

แก้ยังไง: Downloaded the vendor mmproj, verified it live in the local dry run (llama.cpp read Thai text, grid 1/2/3 x A/B/C/D, footing marks F1/C1 correctly off house01 page19).

กฎที่ได้: Before debugging a conversion/export tool, read the training config to see whether the component you are trying to export was ever modified. A frozen component can be taken from the vendor unchanged.

สูง 2026-07-24

First GPU rental had no pre-flight check at all — every bug was discovered while the meter was running (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The 2026-07-21 rental went straight from 'rent' to 'train'. Every problem that day (OOM, disk full, mmpj unsupported, environment clobbered, files lost) was found while paying ~30 THB/hour. Only afterwards was a 'Phase 0' pre-rent checklist invented, and the diary explicitly records that having a checklist document is not the same as running it.

ต้นเหตุจริง: No process existed requiring free, local verification (token, balance, disk sizing, local dry run, script re-read) before committing money; the checklist was written only after the failure that motivated it.

เสียหายอะไร: An entire paid session's worth of debugging happened on the clock, and the day's output (LoRA, merged model, GGUF) was lost anyway — the work had to be redone on 2026-07-24/25.

แก้ยังไง: Became Rule 4 in rule_of_tune.md and Phase 0 in t01_workflow.md; on 2026-07-24 Phase 0 was actually executed and immediately caught two real problems (the 150GB disk default and the MAX_LENGTH truncation bug) before any money was spent.

กฎที่ได้: Everything checkable for free on your own machine must be finished before the rental clock starts — and the rule must require running the checklist, not merely having one.

สูง 2026-07-24

Identical house 5 page36 rebar fixes appear in both the 07-19 and 07-24 entries — the work was either redone or double-logged (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The 07-19 entry records fixing house 5's S-08 page36: B2X top→3, B3X phantom deleted, B4 bottom→5, B5 bottom→5. The 07-24 entry records the same four fixes on the same page with the same numbers (B2X main_bar.top 2→3, B3X phantom additional_bars deleted, B4 bottom 2→5, B5 bottom 3→5).

ต้นเหตุจริง: Either the fixes were lost (house 5's folder was 'freshly copied+normalized' from raw during this period, which would overwrite earlier corrections) and had to be redone, or the same session's work was written into two days' entries. Both possibilities are process failures with the same cure.

เสียหายอะไร: Either a session of point-counting review work was repeated, or the diary double-counts progress — in both cases the record of what is genuinely fixed cannot be trusted for this house.

แก้ยังไง: not fixed — the duplication is visible in the two entries and was never reconciled.

กฎที่ได้: Re-copying a house folder from raw silently destroys corrections already made in the corrected copy. Copy-from-raw must be a one-time bootstrap, guarded by a check that no corrections exist yet.

สูง 2026-07-28

A script written long ago and never run was treated as ready (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: train_qwen3vl.py was assumed usable because it existed and had been reviewed, but had never actually been executed until the paid t02 run.

ต้นเหตุจริง: 'Written and reviewed' was conflated with 'known to work'. Nothing in the surrounding world (library versions, model repo layout) is frozen while a script waits.

เสียหายอะไร: Debugging a stale-API failure on paid GPU time instead of on a free local dry run.

แก้ยังง: Fixed the API call; the incident was recorded as a general lesson about dormant code.

กฎที่ได้: Before any paid run, execute every script at least once — even against a tiny model or a 5-step limit. Untouched code decays because its dependencies move.

สูง 2026-07-28

House 07 S-02: GB1X was read as a full C4→B4 span when it is a 1.60 m stub, which also wrongly flagged a footing as off-grid (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: GB1X was extracted as spanning C4 to B4 (that span is actually a second GB1). It is really a 1.60 m stub below B-4. The consequence propagated: footing F1 at B-4 had been recorded as off-grid when it sits exactly on the grid.

ต้นเหตุจริง: A member label was matched to the nearest plausible span without reading the printed dimension chain, and the derived grid position was then trusted rather than re-derived from the corrected geometry.

เสียหายอะไร: Wrong span length feeds beam quantity takeoff directly; the false off-grid flag would corrupt grid resolution for that footing downstream.

แก้ยังง: Corrected append-or-rewrite style with the reversal recorded and the original warning preserved, never deleted.

กฎที่ได้: Never infer a member's span from label adjacency — read the printed dimension chain. And when a geometry fact is corrected, re-derive everything computed from it, not just the field that was wrong.

สูง 2026-07-28

Fourteen of nineteen house-07 plan files needed fixes, all reducible to three recurring error classes that had no checklist entry (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Only 5 of 19 plan files were clean. The 14 that needed fixing collapse into three repeating classes: counting different symbol types mixed together, reading plan adjacency as physical stacking, and look-alike lines masquerading as structural members. None of the three was in the extraction checklist.

ต้นเหตุจริง: Errors were being fixed individually as they were found, with no step that generalised a found error into a rule applied to every other sheet. The same three mistakes therefore recurred across a whole house.

เสียหายอะไร: 74% of the house's plan files carried at least one error; the same three classes will recur on every future house until the checklist covers them.

แก้ยังไง: The three classes were written up to be folded into op2's Stage 3 checklist.

กฎที่ได้: When the same error appears twice, stop fixing instances and add it to the checklist as a class. Every extraction pass should end by asking which of the found errors is a class, not an incident.

สูง 2026-07-28

export_gguf.py was still entirely t01's — pointing at Qwen3.6 paths and at t01's HF repo, which Phase 7.5 depended on (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The t02 export script had been copied without adaptation: paths, repo, and size range all still targeted Qwen3.6. Running it as-is could have overwritten the HF repo qwen36-thai-rc, which Phase 7.5 needed intact to re-measure t01. It also loaded 62GB of weights before checking that ADAPTER_DIR even exists.

ต้นเหตุจริง: Scripts were carried between rounds by copying rather than by parameterisation, so every hard-coded round-specific value became a latent cross-round hazard. No guard protected a repo other work depended on.

เสียหายอะไร: Near-loss of t01's published adapter repo, which would have made the Phase 7.5 re-measurement impossible; plus wasted 62GB loads on missing-adapter runs.

แก้ยังไง: Path/repo/size tied to the same MODEL_SIZE used during training; an existence check for ADAPTER_DIR added before loading weights; any HF_REPO ending in /qwen36-thai-rc explicitly blocked.

กฎที่ได้: Parameterise round-specific values instead of copying scripts, and add an explicit refusal guard around any destination another round's results depend on. Check cheap preconditions before expensive loads.

สูง 2026-07-29

Model-load warnings that named the exact bug were printed and read by nobody — twice (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: llama.cpp printed 'Qwen-VL models require at minimum 1024 image tokens to function correctly ... try adding --image-min-tokens 1024'. Nobody read it. This is the second occurrence of the same pattern: the collator earlier printed 'Model does not have a default image size - using 512' and that was missed too.

ต้นเหตุจริง: Warnings appear interleaved with hundreds of startup lines and no step in the workflow requires anyone to read them before a long run begins.

เสียหายอะไร: Both halves of the resolution bug were self-reported by the tooling and both were missed, costing a full training round and a night of extraction before the cause was found by inference instead of by reading.

แก้ยังไม: not fixed — the lesson is recorded but no mechanical gate exists yet

กฎที่ได้: Make warning review a required step, not an act of attention: grep the load log for WARN/W: and print the hits to the top of the run before any long job starts. Anything the tool tells you for free is the cheapest bug report you will ever get.

สูง 2026-07-29

Six pages of known-bad extraction written into the project output folder before validation (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Four more pages (03-06) were extracted and written to tune_ai/t01/ผล/ before anyone checked quality; only afterwards was the output found unusable and further running blocked to stop the folder filling with wrong data.

ต้นเหตุจริง: The run proceeded straight to bulk production with no quality gate on the first outputs — validation happened because Makham looked at the results, not because the process required it.

เสียหายอะไร: Six contaminated output files that must now be identified and discarded; had the run not been stopped, ~55 more would have joined them.

แก้ยังไม: partially — further running halted until the root cause test is done; the bad files remain on disk

กฎที่ได้: Gate every bulk run on a validated first sample: extract 1-2 items, diff against gold, and only then release the batch. Write provisional output to a quarantine path until it passes, so bad data can never be mistaken for good.

สูง 2026-07-29

Phase 10's own written caveat ('don't read this as proven complete') decayed into 'Phase 10 passed' within days (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The Phase 10 GGUF test document on the 25th explicitly warned that its result should not be read as proof of completeness — it only checked that the model emitted JSON and did not hang. Four days later everyone, the diary's author included, remembered only the green 'Phase 10 passed', and that memory was used to justify running the artifact at full scale.

ต้นเหตุจริง: The caveat lived in prose beside the status, not in the status itself. A pass/fail marker with a warning attached loses the warning the moment anyone summarises it — and it will be summarised, repeatedly, by both people and later documents.

เสียหายอะไร: The GGUF was treated as validated for four days and used for a whole night's extraction run that produced six pages of unusable output.

แก้ยังไง: not fixed as a process — the caveat was rediscovered only by re-reading the original doc during root-cause tracing

กฎที่ได้: A caveat written next to a green checkmark is not a caveat. Encode the limitation in the status string itself ('Phase 10: JSON-shape only, content unverified'), so it survives being quoted, summarised, and remembered.

สูง 2026-07-29

The MoE merge bug was proven on t02 on the 28th and never re-applied to t01 — a full night of extraction was spent before anyone re-asked (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: On the 28th the team measured, hands-on, that `merge_and_unload()` silently mis-merges a `ParamWrapper` LoRA on this MoE model (only 0.67% of 201M values changed). t01 was built with the same function, on the same model family, with the same `ParamWrapper` — but nobody went back and re-questioned t01's already-shipped GGUF. Instead t01 was run at full scale on the 29th, and the suspicion only surfaced hours later while tracing why the output was unusable.

ต้นเหตุจริง: The bug was recorded as 't02's problem' rather than as a property of the toolchain. No step in the process asks 'which other artifacts were produced by the same broken function?' when a root cause is found.

เสียหายอะไร: A whole overnight session (00:30-03:30) plus the GPU-hours of six ~20-minute page extractions spent producing output from an artifact that was already known-suspect on the previous day. Six bad pages were written into the project output folder in the meantime.

แก้ยังไง: not fixed — the base-model comparison test that would confirm it was still pending at the end of the session

กฎที่ได้: When a root cause is found in one artifact, immediately list every other artifact built by the same code path and re-open each one. A toolchain bug is never scoped to the artifact that happened to expose it.

สูง 2026-07-29

Model-loader warnings that named both root causes were printed and ignored — twice (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: llama.cpp printed 'Qwen-VL models require at minimum 1024 image tokens to function correctly ... try adding --image-min-tokens 1024'. Nobody read it. This was the second occurrence of the identical pattern — the Unsloth collator had earlier printed 'Model does not have a default image size - using 512', also unread, which was the 14× resolution bug.

ต้นเหตุจริง: Startup warnings scroll past among hundreds of lines and are treated as noise. No step in the workflow required reading them before starting a long run.

เสียหายอะไร: Both root causes of the resolution problem were sitting in the logs the whole time; days were spent on other hypotheses instead.

แก้ยังไง: Adopted as a rule: read the loader warnings every time before launching a long job.

กฎที่ได้: Grep the startup log for WARN/W before starting any expensive run, and pipe loader output to a file that is reviewed — a warning nobody reads is the same as no warning.

สูง 2026-07-29

Killed a healthy 1,256-second job because the 1,200s timeout was calibrated on the two lightest pages (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A 1,200-second timeout was derived from the runtimes of pages 01-02. Page 03 needed 1,256 seconds and was cut off mid-work. GPU utilisation at 100% proved the job was alive, not hung.

ต้นเหตุจริง: Two failures at once: the calibration sample was pages 01-02 — the cover and table of contents, the lightest pages in the whole set — and 'no output' was read as 'hung' when a generating model is silent by nature.

เสียหายอะไร: A real in-progress extraction was destroyed, and had the GPU check not been done, the rest of the night would have gone into debugging a non-existent hang.

แก้ยังไง: Verified liveness with nvidia-smi (GPU at 100% = working; 0% = actually hung) before concluding anything.

กฎที่ได้: Calibrate timeouts against the heaviest expected case, not the first case you happen to run; and before declaring a silent job dead, check an independent liveness signal (GPU utilisation, growing output file).

สูง 2026-07-29

The decisive clue was the shape of the errors, not their quantity — an aggregate score would have hidden it (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The model performed well on hard content (TIS numbers 8/8, floor levels 7/8, rooms 8/8) while failing completely on the easy part (output format 0/6). That inversion is unnatural for a fine-tuned model and is what pointed at the missing-LoRA hypothesis.

ต้นเหตุจริง: Initial reaction was to treat the result as 'bad quality' and look for model or resolution causes. Only inspecting the error pattern per category revealed that the failure profile matched an untuned model following a prompt.

เสียหายอะไร: Had the team only looked at the aggregate score, the real root cause would never have surfaced and time would have gone into retraining or renting more GPU.

แก้ยังไง: Diagnosis by error shape; recorded as one of the six lessons of the night.

กฎที่ได้: When results are bad, categorise the errors before measuring them — 'wrong on the easy things, right on the hard things' is a deployment/plumbing signature, not a model-quality one.

สูง 2026-07-29

The classify phase itself is inaccurate — elevations read as sections, 5 views claimed where 2 exist (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Pages 07/08/09 were classified as `section` when ground truth is `side\_profile`, and one page was reported as having 5 views when it really has 2.

ต้นเหตุจริง: The 2-phase pipeline's cheap classify pass was never accuracy-tested on its own; it was treated as a routing convenience rather than as a gate whose errors are unrecoverable.

เสียหายอะไร: Misrouting: pages that should be extracted get skipped and pages that should be skipped get extracted. Errors here are invisible downstream because a skipped page simply produces no data.

แก้ยังไง: Not fixed. Recorded as a caveat on interpreting the night's results.

กฎที่ได้: A cheap routing/classification stage needs its own accuracy measurement against ground truth — a routing error deletes work silently, so it is more dangerous than a noisy extraction error.

สูง 2026-07-29

A written 'do not read this as proven' caveat decayed into 'Phase 10 passed' in everyone's memory, including the assistant's (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The Phase 10 documentation explicitly warned 'don't read this as proven complete', but over the following days the caveat was forgotten and the phase was remembered — by Makham and by the assistant alike — simply as passed. That false memory is what let the unverified GGUF be used for real production extraction on 2026-07-29.

ต้นเหตุจริง: The status label (a passing phase) and the qualification (prose warning next to it) live in different places and decay at different rates; humans and agents recall the label, not the paragraph.

เสียหายอะไร: Four pages of bad extraction were produced and written to the real results folder on the strength of a phase everyone believed was verified.

แก้ยังไง: Not fixed — no mechanism proposed to make a phase's status carry its own limits.

กฎที่ได้: A caveat in prose beside a green status will be forgotten; encode the limit into the status itself (e.g. 'Phase 10: smoke-only, content unverified'), never into a sentence nobody rereads.

สูง 2026-07-30

The 'is the LoRA actually inside t01's GGUF?' check was carried over unresolved for a second session (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: A red-flagged item from the previous night — verify that the LoRA is genuinely baked into t01's exported GGUF — was still not done at the end of this session, which went entirely to annotation.

ต้นเหตุจริง: Annotation work (op1 houses) always has a clear finish line and crowds out verification tasks that have no deadline; there is no mechanism forcing a red-flagged carry-over to be scheduled.

เสียหายอะไร: An unverified model artifact stayed in use as the project's reference GGUF; if the merge silently failed, every downstream comparison against t01 is measuring the base model.

แก้ยังไง: Not fixed this session — re-flagged as pending for the next one.

กฎที่ได้: A carry-over flagged ● must be scheduled as the first item of the next session, not appended to the end. Never treat an unverified model artifact as a baseline.

สูง 2026-08-02

The canonical schema contradicted the fix-log in 4 places — and the spec was wrong every time (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `primary\_rawjson\_schema.md` was checked against `สิ่งที่ต้องแก้.md` and found to contradict it in 4 places (point-ref dashes, dummy grid naming, column main_bar top/bottom, point-element merge rule). In all 4 the spec was the wrong one — the data had already moved on.

ต้นเหตุจริง: Decisions were being recorded in the fix-log as they were made but never propagated back into the spec, so the spec silently decayed into describing an older generation of the data.

เสียหายอะไร: Anyone following the spec (a new house, a new agent, a prompt writer) would have produced data contradicting 11 houses of existing work.

แก้ยังไง: Applied 5 fixes to the spec so it matches the decided rules, and logged each in `primary\_rawjson\_schema\_edit\_log.md`.

กฎที่ได้: A decision is not made until it is written into the spec, not just the fix-log. Periodically diff spec against decision-log — the spec is the artifact people follow.

สูง 2026-08-02

Houses 06–11 had never been copied into the reviewed-copy folder at all (571 files) (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `json\_แก้ไขแล้ว/` — the reviewed working copy — contained only houses 01–05. Houses 06 through 11 had never been placed there, so 571 files had never entered the review workflow.

ต้นเหตุจริง: Extraction (op1) writes to the raw folder; getting a house into the reviewed folder was a separate manual step with nothing tracking whether it had been done per house.

เสียหายอะไร: Six houses of extracted data sat outside the review/normalisation pipeline entirely and were invisible to every quality pass run on the reviewed copy.

แก้ยังไง: Copied all 571 files in fresh from raw.

กฎที่ได้: Every pipeline stage needs a per-house completion ledger. A house is not 'done' at extraction — track its state through every downstream stage explicitly.

สูง 2026-08-02

การแก้รอบ 2 ที่ตัดชิดออก และเฉพาะไฟล์ที่*ชื่อ*มีคำว่า 'footing' — ตกไป 809 ref

เกิดอะไรขึ้น: รอบก่อนหน้าแก้ point ref ที่มีขีด (`C-1` → `C1`) แต่แก้เฉพาะไฟล์ที่ชื่อไฟล์มีคำว่า 'footing' ไฟล์ผ้งอื่นๆ ในบ้าน 06–11 ยังมีขีดอยู่หมด รวม 809 point ref ใน 87 ไฟล์ที่ต้องมาแก้กันใหม่ทีหลัง

ต้นเหตุจริง: ตอนแก้กำหนดขอบเขตด้วยชื่อไฟล์ แทนที่จะกำหนดด้วยชื่อช่องข้อมูล ปัญหาจริงมันอยู่ในช่องตระกูล `grid\_ref` ทุกที่ที่มีนโพล ไม่ได้อยู่ในไฟล์ที่ชื่อแบบใดแบบหนึ่ง

เสียหายอะไร: งานที่รายงานว่า "แก้เสร็จแล้ว" ครอบคลุมข้อมูลที่มีปัญหาแค่เศษเสี้ยว สุดท้ายต้องรื้อทำใหม่ทั้ง corpus

แก้ยังไง: เขียน ref ใหม่ทั้ง 809 ตัวในทุกไฟล์ โดยกำหนดขอบเขตด้วยชื่อช่อง (`grid\_ref`/`grid\_ref\_start`/`grid\_ref\_end`/`grid\_refs`) แทนชื่อไฟล์

กฎที่ได้: เวลาแก้ข้อมูล ให้กำหนดขอบเขตด้วยช่องที่มีมันอยู่ ห้ามใช้ชื่อไฟล์ แล้วพิสูจน์ว่าแก้ครบจริงด้วยการนับจำนวนที่ยังผิดอยู่ทั้ง corpus ไม่ใช่เชื่อว่าสคริปต์รันแล้วจบ

สูง 2026-08-02

คำสั่ง 'ทำตามบ้าน 01-05' ผิด — เรื่องคำเกินใน grid ref บ้าน 01-05 คือตัวที่แย่มากที่สุด

เกิดอะไรขึ้น: คำสั่งที่ยึดกันมาคือให้บ้านรุ่นหลังทำตามบ้าน 01-05 แต่เรื่องคำว่า `grid`/`dummy` ที่ซ้ำซ้อนใน grid ref บ้าน 05 ผิดอยู่ 127 จุด ส่วนบ้าน 06 ผิด 0 จุด — บ้านต้นแบบกลายเป็นตัวที่แย่มากที่สุด ไม่ใช่ดีที่สุดใน

ต้นเหตุจริง: บ้าน 01-05 ถูกยกเป็นมาตรฐานเพราะเก่าที่สุดและถูกรีวิวก่อนมากที่สุด แต่ "เก่าที่สุด" แปลว่ามันเกิดก่อนกฎหลายข้อ และสะสมของเก่าที่ผิดกฎไว้มากที่สุดด้วย

เสียหายอะไร: ถ้าทำตามคำสั่งจริงๆ จะเป็นการเอาข้อผิดพลาดของบ้าน 01-05 ไปแพร่ใส่บ้านรุ่นใหม่ที่สะอาดกว่า

แก้ยัง: ยึดตาม*กฎที่เขียนไว้* (ข้อ 5) ซึ่งบ้าน 01-05 เป็นคนเขียนเอง แทนที่จะยึดตาม*สภาพไฟล์*ของมัน — กฎถูกอยู่แล้ว แค่ไม่เคยถูกเอาไปใช้ให้ครบ

กฎที่ได้: ให้ยึดกฎที่เขียนไว้ ไม่ใช่ยึดสภาพของบ้านต้นแบบ ข้อมูลที่เก่าที่สุดมักปนเปื้อนของเก่ามากที่สุด อย่าปล่อยให้มีนกลายเป็นมาตรฐานโดยปริยาย

สูง 2026-08-02

งาน Label Studio ถูกยกเลิกและลบทั้งหมด — 51 ไฟล์ 22MB พร้อมขั้นตอนใน pipeline ที่สร้างรอบมันไว้

เกิดอะไรขึ้น: Label Studio ถูกยกเลิกทั้งต้น สิ่งที่ลบไป: ตัวสร้าง task, สคริปต์ import ทั้ง 2 ตัว, ไฟล์ XML ของโปรเจกต์ 3 ไฟล์, ไฟล์ task JSON ที่สร้างไว้ 21MB สำหรับ 10 บ้าน, PDF mindmap ของ flow การรีวิวก่อน, สคริปต์อัปเดตที่มีไว้สร้างไฟล์ task ของ LS อย่างเดียว และขั้นตอนใน workflow op1 ที่เรียกใช้มันทั้งหมด

ต้นเหตุจริง: สร้าง track เครื่องมือทำ annotation ทั้งชุดแล้วเอาไปรันกับ 10 บ้านก่อนที่จะตัดสินใจว่าจะใช้มันจริงหรือเปล่า คำถามซ้ำๆ ("เราต้องเก็บ annotation จากคนจริงๆ ไหม") ถูกตอบหลังจากสร้างเครื่องมือเสร็จแล้ว ไม่ใช่ก่อน

เสียหายอะไร: งานสร้างเครื่องมือหลายสัปดาห์ บวกกับขั้นตอนรัน `label-studio-tasks-makham.js` ที่ละบ้าน 10 บ้าน กลายเป็นต้นทุนจม แล้วยังต้องเขียน workflow op1 ใหม่เพื่อตัดขั้นตอนพวกนั้นออก

แก้ยัง: ย้ายทุกอย่างไปพักไว้ใน `wait\_for\_thing/` ด้วย `git mv` ก่อน (ก็คิดได้) แล้วค่อยลบหลังยืนยัน ส่วนด้านตรวจหลังสัปดาห์ของ op1 เปลี่ยนไปใช้ `python tools/check\_format.py` แทน

กฎที่ได้: ตัดสินใจให้จบก่อนว่าต้องใช้ workflow นี้จริงไหม แล้วค่อยสร้างเครื่องมือและเอาไปรัน 10 บ้าน และเวลาจะฆ่า track ไหน ให้ย้ายด้วย `git mv` ก่อนเสมอ ห้ามใช้ `rm` — เพื่อมีของที่จัดประเภทผิดจะได้ถูกกลับมาได้

สูง 2026-08-02

ไฟล์ที่ยังใช้งานอยู่ 2 ไฟล์ถูกจัดผิดว่าเป็นของ Label Studio — ถ้าลบ manifest.json ไปสคริปต์ 2 ตัวจะพัง

เกิดอะไรขึ้น: ตอนเก็บกวาด Label Studio ไฟล์ `manifest.json` ถูกย้ายเข้าโฟลเดอร์รอบ เพราะสคริปต์ import ของ LS เขียนลงไฟล์นี้ แต่เจ้าของจริงคือ `pdf-processor.py` กับ `qwen-processor.js` ซึ่งอ่านไฟล์นี้แต่ไม่ได้เป็นคนสร้าง ส่วน `review.html` ก็ถูกจัดผิดเหมือนกัน — มันคือ UI รีวิวแบบ local ที่มีมาก่อน LS ไม่ได้เกี่ยวกับ LS เลย

ต้นเหตุจริง: ดูความเป็นเจ้าของจาก "สคริปต์ไหนเขียนลงไฟล์นี้" แทนที่จะดูจาก "สคริปต์ไหนพึ่งพาไฟล์นี้" พอเครื่องมือที่กำลังจะลบไปแตะไฟล์นั้น เลยเหมาเอาว่าไฟล์นั้นเป็นของมัน

เสียหายอะไร: ถ้าลบ `manifest.json` ไปจริง `pdf-processor.py` กับ `qwen-processor.js` จะพังทั้งคู่ในครั้งถัดไปที่มีคนรัน — เป็นการพังเงียบๆ ที่จะไม่มาทีหลัง รอดมาได้เพราะใช้วิธีย้ายไปพักแทนการ `rm` ตรงๆ

แก้ยัง: กู้ทั้ง 2 ไฟล์กลับไปไว้ที่ `No\_touch\_box/` แล้วแก้การจัดประเภทของรอบ 6 ในบันทึกให้ถูกต้อง

กฎที่ได้: ก่อนลบไฟล์ ให้ grep หาทุกตัวที่อ่าน*ไฟล์นั้น ไม่ใช่แค่ตัวที่เขียน ไฟล์ที่สคริปต์เปิดด้วย `open(..., 'r')` แต่ไม่เคยสร้างเอง คือ dependency ไม่ใช่ของที่ออกมา

สูง 2026-08-02

โพลเดอร์ทำงานที่รีวิแล้วกับโพลเดอร์ raw ที่เป็นข้อมูลจริง ไม่เคยถูก sync กันเลยตลอดประวัติโปรเจกต์

เกิดอะไรขึ้น: รอบ 4 ของวันที่ 2026-08-02 ทำสิ่งที่บันทึกเรียกเองว่า "การ sync กลับครั้งแรกในประวัติศาสตร์ของโปรเจกต์" — ดัน 362 ไฟล์เข้า rawjson ยังไม่ได้แก้ไขโดนคน/ ก่อนหน้านั้นทุกการแก้ไขทำใน json_แก้ไขแล้ว/ มีอยู่แค่ในโพลเดอร์นั้นทีเดียว ส่วนโพลเดอร์ที่ถูกกำหนดให้เป็นข้อมูลจริงยังเก็บเวอร์ชันที่ยังไม่แก้ อยู่ คำว่า "ยังไม่ได้เริ่ม sync กลับเลยสักบ้าน" โพลเป็นรายการค้างในบันทึก 3 รอบติดกันกว่าจะได้ทำจริง

ต้นเหตุจริง: มีโพลเดอร์ 2 ชุดที่ดูแลกันโดยไม่มีขั้นตอน sync ที่ชัดเจน และไม่มีตัวเช็คความทั้งสองตรงกันใหม่ workflow มีแต่การเขียนไปข้างหน้าลงโพลเดอร์ที่รีวิแล้วอย่างเดียว

เสียหายอะไร: ตลอดช่วงก่อนหน้านั้น อะไรก็ตามที่อ่านโพลเดอร์ raw — รวมถึงตัวสร้างชุดข้อมูล — อ่านข้อมูลก่อนแก้ไขทั้งหมด ทั้งที่ทีมเชื่อว่าแก้ไขแล้ว พอถึงเวลา sync จริงต้องออกค่าเตือน Rule 2 เติมรูปแบบและขออนุญาตอย่างชัดเจน เพราะมันไปแตะโพลเดอร์ที่ถูกป้องกันไว้เป็นจำนวนมาก

แก้ยังไง: sync 362 ไฟล์เข้าโพลเดอร์ raw ภายใต้การอนุญาตชัดเจน บันทึกไว้ใน raw_json_data_log.md และ No_touch_box/CLAUDE.md ผูกกับ commit 0029264 แล้วเอาการเช็ค "สองโพลเดอร์ต่างกัน 0 ไฟล์" เข้าเป็นส่วนหนึ่งของการปิดงาน

กฎที่ได้: ถ้ามีข้อมูลชุดเดียวกันอยู่ 2 ที่ ต้องมีขั้นตอน sync อยู่ใน workflow และมีตัวเช็คอัตโนมัติว่าเท่ากัน ไม่งั้นอีกฝั่งจะกลายเป็นของเก่าเจี๊ยบๆ และให้เอาการยืนยันว่า "สองโพลเดอร์ต่างกัน 0 ไฟล์" ไปไว้ในด้านตรวจ ไม่ใช่ในสรุปปิดงาน

สูง 2026-08-05

บ้าน 13 หายไป 10 หน้า แต่เพิ่งจับได้ตอน agent เบื่องหลังทั้ง 7 ตัวทำงานเสร็จหมดแล้ว

เกิดอะไรขึ้น: ตัวเช็คความครบของหน้าที่ตัวประสานงานเพิ่งพบว่าไฟล์หายไป 10 ไฟล์ (หน้า01/02/28-33 และการแก้ไข grid_ref บนหน้า34) ของบ้าน 13 หลังจาก agent เบื่องหลังทั้ง 7 ตัวส่งงานกลับมาหมดแล้ว ต้องรันตามอีกรอบเพื่อทำบ้านนี้ให้ครบ

ต้นเหตุจริง: ความครบของหน้าถูกเช็คตอนเป็นด้านสุดท้าย แทนที่จะเช็คตอนแจกงาน ไม่มีอะไรตรวจว่าช่วงหน้าที่แจกให้ agent ทุกตัวรวมกันแล้วครบทั้ง 92 หน้าหรือยังก่อนส่งเริ่ม

เสียหายอะไร: บ้านหลังหนึ่งถูกรายงานว่าสัปดาห์เสร็จแล้วทั้งที่ขาดไป 10 หน้า ต้องแจกงานรอบสอง และถ้าไม่ได้เช็ค ช่องโหว่นี้จะเข้า corpus ไปแบบเจี๊ยบๆ

แก้ยังไง: สกัดไฟล์ที่หายทั้ง 10 ในการ commit ตามอีกรอบจนบ้านครบ พร้อมบันทึกว่าตัวเช็คความครบควรรันให้เร็วกว่านี้

กฎที่ได้: ให้ตรวจความครบของหน้าตอนแจกงาน ไม่ใช่ตอนงานเสร็จ — ยืนยันว่าช่วงหน้าที่แจกไปทั้งหมดรวมกันแล้วเท่ากับจำนวนหน้าทั้งหมด ก่อนที่ agent ตัวแรกจะเริ่มทำงาน

สูง 2026-08-07

บ้าน 08 กับ 09 เซ็นรับว่า '100% เสร็จ' เมื่อ 07-30 แต่อีกสัปดาห์ต่อมาต้องมาแก้ทั้งคู่

เกิดอะไรขึ้น: บ้าน 08 ที่ประกาศเมื่อ 2026-07-30 ว่า '100% เสร็จ 63/63 หน้า ไฟล์ JSON ทั้ง 97 ไฟล์ถูกต้อง' ต้องแก้ 4 ไฟล์เมื่อ 08-07 (เส้นกริด, ผังคาน, ผังคานรัตรอบ, การขยายคาน-เสา) ส่วนบ้าน 09 ที่ประกาศ 100% เหมือนกัน ต้องแก้ 3 ไฟล์เมื่อ 08-08

ต้นเหตุจริง: คำว่า "เสร็จ" ถูกนิยามว่า "สัปดาห์ครบทุกหน้าและตัวเช็คฟอร์แมตผ่าน" ซึ่งวัดแค่ความครบของหน้าและความถูกต้องของรูปทรงข้อมูล ไม่ได้วัดความถูกต้องของเนื้อหา ไม่มีอะไรในเกณฑ์ปิดงานที่เอาค่าที่สัปดาห์ไปเทียบกับแบบจริง

เสียหายอะไร: บ้าน 2 หลังถูกบันทึกว่าเสร็จทั้งที่มีเนื้อหาผิดอยู่ และแปลว่าสถานะ "เสร็จ" ของบ้านอื่นๆ ก็เชื่อไม่ได้ด้วยเหตุผลเดียวกัน การแก้ครั้งนี้เกิดขึ้นเพราะบังเอิญมีคนกลับไปดูอีกรอบเท่านั้น

แก้ยังไง: ทำรอบแก้ไขไฟล์ของทั้งสองบ้าน

กฎที่ได้: แยก "สัปดาห์ครบแล้ว" กับ "ตรวจเนื้อหาแล้ว" ให้เป็นสองสถานะที่มีการตรวจคนละชุด ตัวเช็คฟอร์แมตผ่านไม่ใช่การรีวิวเนื้อหา อย่าปล่อยให้ ALL CHECKS PASS ถูกอ่านว่า "บ้านหลังนี้ถูกต้องแล้ว"

สูง 2026-08-18

branch King-try ตามหลัง main อยู่ 277 commit แคมพก schema สำเนาที่เก่าไปหลายสิบกฎ

เกิดอะไรขึ้น: ตอนรีวิว branch King-try ของ kingparadorn พบว่ามันแยกออกมาตั้งแต่ 2026-07-25 และตามหลัง main อยู่ 277 commit — คือเกิดก่อนแคมเปญ op01/op03 ทั้ง 22 บ้าน และก่อนการรื้อ schema \$0 FORMAT LOCK ทั้งชุด สำเนา primary_rawjson_schema.md ของมันเองก็อยู่ใต้โฟลเดอร์เก่า raw_json_ตัวที่ใช้งานจริง/ แบบ 5 บ้าน และเก่าไปหลายสิบกฎ

ต้นเหตุจริง: branch พิเจอร์ที่อยู่ยาวไม่เคย rebase หรือ sync เลย ในขณะที่สเปกที่มันพึ่งพากำลังถูกเขียนใหม่บน main และเพราะ branch พกสำเนาเอกสาร schema ของตัวเองแทนที่จะอ้างอิงไปที่ทีเดียว การเบี่ยงเบนเลยมองไม่เห็นจากทั้งสองฝั่งจนกว่าจะมีคนเอามา diff กัน

เสียหายอะไร: การ merge จริงกลายเป็นเรื่องทำไม่ได้ งานที่มีประโยชน์จริง 12 commit (ต้นแบบ overlay ของ Gemini, hooks, skills, agent) เลยค้างเต้ง และจนถึง 2026-08-20 ก็ยังไม่ได้ cherry-pick ออกมา เครื่องมืออะไรที่เขียนบน branch นั้นก็เล็งไปที่รูปทรง schema ที่ไม่มีอยู่แล้ว

แก้ยังไง: ยังไม่แก้ บันทึกคำแนะนำไว้ว่าให้ cherry-pick ต้นแบบ overlay พร้อม hooks/skills/agent ขึ้น main แทนที่จะพยายาม merge ซึ่งยังค้างอยู่ ณ 2026-08-20 ('เซสชันนี้ไม่ได้ cherry-pick อะไรขึ้น main เลย')

กฎที่ได้: sync branch พิเจอร์กับ main อย่างน้อยสัปดาห์ละครั้ง และห้ามให้ branch พกสำเนาสเปกที่ใช้ร่วมกันของตัวเอง — ให้อ้างอิงไปที่ไฟล์กลางไฟล์เดียว เวลาสเปกเปลี่ยนจะได้เกิด conflict ที่มองเห็นได้ แทนที่จะเบี่ยงเบนกันเสียๆ

สูง 2026-08-20

สร้างด่านรับ/ปฏิเสธแบบ checklist ตัวหนังสือ ทั้งที่โจทย์คือกดที่กรอบเขียวบนแบบ

เกิดอะไรขึ้น: สร้างด่านรับ/ปฏิเสธเต็มรูปแบบเข้าไปในเส้นทางนำเข้า raw JSON ของ Constaintant — เป็น checklist ของทุก element พร้อม มาร์ค/ประเภท/ชั้น/ตำแหน่งกริด/ความมั่นใจ บวกกับการแยก qt_importRawExtractionFiles() ออกเป็นชั้น preview กับ save ค่าตัดสินของมะขาม: ยังไม่ดีพอ โจทย์เดิมคือกดที่กรอบเขียวที่วาดทับบนผัง ไม่ใช่รายการตัวหนังสือ ตัวที่เป็นภาพจริงๆ ยังต้องสร้างอยู่

ต้นเหตุจริง: ติความโจทย์ 'กดเพื่อรับ element' เป็นปัญหาการรีวิวข้อมูล (แถวในตาราง) แทนที่จะเป็นปัญหาการรีวิวเชิงตำแหน่ง (มาร์คบนแบบ) และไม่ได้ทวนโจทย์กลับไปยืนยันก่อนลงมือ ความไม่ตรงกันเลยโผล่ตอนเดโม

เสียหายอะไร: เสียงานทั้งเซสชัน บวกกับ UI ที่แก้แล้วยังไม่ commit ซึ่งไม่ตอบโจทย์ ไฟเจอร์จริง (SVG ที่กดได้วางทับกริด วาดจาก GridReference ที่บันทึกไว้) ยังไม่ได้สร้าง ส่วนการแยก preview/save ที่ทำให้ยังเอาไปใช้ต่อได้

แก้ยังไง: เซสชันนี้ยังไม่แก้ — บันทึกว่าเวอร์ชัน overlay ที่เป็นภาพยังค้างอยู่ ส่วนการแยก qt_buildRawExtractionPreview / qt_saveRawExtractionPreview เก็บไว้เป็นฐานที่ใช้ต่อได้

กฎที่ได้: ก่อนสร้าง UI รีวอะไรก็ตาม ให้ทวนโจทย์กลับเป็นประโยคเดียวที่ระบุว่าผู้ใช้จะกดตรงไหน ('คุณกดที่กล่องเขียวบนแบบสแกน') แล้วขอคำยืนยัน — โจทย์เชิงตำแหน่งที่ตอบด้วยตาราง แปลว่าต้องรื้อสร้างใหม่ทั้งหมด ไม่ใช่แค่ปรับนิดหน่อย

สูง 2026-08-20

Run scope collapsed from 4 houses to 19 pages because per-page inference time was never measured first (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Real per-page timing came in at 1.5–15 minutes with huge variance, worse on unseen pages that hit the token cap. Scope was cut live several times: 4 houses → 2 houses → house 08 alone → capped at 25 of 63 pages → ended at 19/25 pages. Houses 09/10/11 never started.

ต้นเหตุจริง: The plan was built on an unmeasured per-page time estimate. The variance driver (unseen pages looping until max_new_tokens) was itself the undiagnosed repetition-loop bug, so the slowest pages were slow for a reason nobody had found yet.

เสียหายอะไร: A 4-house generalization test degraded to 19 pages of one house — the resulting recall number (11%) rests on a much thinner sample than planned, and houses 09/10/11 were still unstarted a day later.

แก้ยังไง: Scope cut repeatedly during the run; PAGE_FILTER gained a list-as-explicit-order mode to allow mid-run reordering. Houses 09/10/11 recorded as not started.

กฎที่ได้: Time two representative pages (one easy, one hardest-pattern) on the rented box before committing to a scope, and treat wildly variable page times as a bug signal to investigate, not just a budgeting problem.

สูง 2026-08-21

Two schema edits made silently all day, violating rule_of_tune Lesson #7's announce-and-log requirement (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: After reading rule_of_tune.md on Makham's instruction, it emerged that Lesson #7 had been violated all day: primary_rawjson_schema.md is exempt from Rule 1 (it is a .md doc) but Lesson #7 requires stating the exemption out loud before editing and logging every edit in primary_rawjson_schema_edit_log.md immediately. Both of the day's earlier schema edits were made with neither.

ต้นเหตุจริง: The governing rules file was not read at session start, so the edit protocol for the single most consequential document in the project was unknown while editing it. The exemption from Rule 1 was noticed; the additional obligations attached to that exemption were not.

เสียหายอะไร: Two schema changes to the project's canonical spec landed with no audit trail and no warning to the human — exactly the situation the edit log exists to prevent. Recovered retroactively, but the window existed.

แก้ยังง: Both edits logged retroactively and marked as rule violations in the log itself; the Rule 2 warning was given properly before the next two edits, which were warned first and logged immediately.

กฎที่ได้: Read rule_of_tune.md at the start of every session that will touch a spec file — and treat an exemption from one rule as a signal to read what other obligations attach to it, never as blanket permission.

สูง 2026-08-21

Two folders one letter apart held two contradictory designs, with nothing saying which was live (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: tune_ai/t03 contained both prompt/ (the 2-pass draft from 2026-08-04) and prompts/ (the 4-pass work from 2026-08-21). Nothing indicated which was current. Reorganized after midnight into one folder per pass, with prompt/ renamed to _old_2026-08-04_two_pass/ and given a 🚫 banner.

ต้นเหตุจริง: A superseded design was left in place under a name one character from the new one, with no deprecation marker. Directory naming was not treated as load-bearing.

เสียหายอะไร: A real chance of running the obsolete 2-pass prompts against the 4-pass folder tree. Also produced duplicated spec-gap text across two READMEs that would have drifted.

แก้ยังง: One folder per pass (pass0_classify/, pass1_organize/, pass2_used/, pass3_unused/); prompt/ → _old_2026-08-04_two_pass/ with a 🚫 banner; _common.md hoisted to t03 root; prompts/README.md deleted with its unique sections folded into t03/README.md. No prompt text changed — moves and links only.

กฎที่ได้: Mark a superseded design at the moment it is superseded — rename it with a date prefix and put a 🚫 banner in its README; never leave two candidate designs distinguishable only by a trailing 's'.

สูง 2026-08-21

Two critical adapter fixes and their regression tests left uncommitted in the Constistant repo (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The parseGridMaster nested-grid fix and the parseGridRef no-dash fix in js/drawing/raw-extraction-adapter.js, plus 4 new regression tests in tests/raw-extraction-grid.test.js, were left uncommitted at end of session. The 08-20 checklist UI change in raw-extraction-import.js was likewise left uncommitted.

ต้นเหตุจริง: Session ended without a commit step, and the files touch a pure module with a purity guard which the author wanted reviewed first — but 'needs review' was recorded as a pending item rather than as a commit on a branch.

เสียหายอะไร: Two fixes for critical silent bugs (every import producing empty grids and null spans since 2026-08-02) exist only in one working tree. A lost working tree, a branch switch, or a stash would erase them along with their tests. This repo's own history already records a plan and an approved mockup lost exactly this way.

แก้ยังไง: Not fixed — recorded as pending: 'The two adapter fixes are uncommitted in the Constistant repo — they touch a pure module with a purity guard; review and commit them.'

กฎที่ได้: Never end a session with a fix for a critical bug uncommitted — commit to a branch and request review there; 'needs review' is a reason to commit, not a reason to leave work in a working tree.

สูง 2026-08-24

Misdiagnosis: assumed xgrammar would prevent generation stalls — page 25 stalled anyway (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: After xgrammar fixed page 22, the assumption was that stalls were solved. Page 25 then stalled again with grammar attached, costing more rental time before the real distinction was understood.

ต้นเหตุจริง: Conflated two different failure modes. Grammar constrains token *validity* ("JSON won't close"), it does not bound output *length* ("generates too long"). Nothing in the runner enforced a wall-clock ceiling.

เสียหายอะไร: A second multi-tens-of-minutes stall on paid GPU time, and Makham ultimately cut every page t01 couldn't finish — shrinking the comparison from 34 → 22 → 12 → 6 usable pages.

แก้ยังไง: Added PAGE_TIMEOUT_S=25min as a StoppingCriteria, separate from grammar.

กฎที่ได้: Grammar fixes malformed JSON, not runaway length. Every generation loop needs an independent wall-clock stopping criterion regardless of what decoding constraints are attached.

สูง 2026-08-25

Whole t03 training round spent on plan_beam, which moved 0% → 0% (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Training round 3 (t03) was run specifically to fix plan_beam element recall. It improved schedule (0→21.9%) and plan_slab (0→20%), but plan_beam stayed at exactly 0%. Cost ~\$5 and 5 hours for zero movement on the one metric the round existed to fix.

ต้นเหตุจริง: The round was launched without first diagnosing WHY plan_beam was 0%. The failure mode was 'the model does not see the beams at all' (empty elements array), not 'the model misreads numbers' — retraining on the same silently-downscaled images could never fix a visibility problem.

เสียหายอะไร: ~\$5 GPU spend and 5 hours of wall-clock training time produced no progress on the round's primary objective; the team only learned the diagnosis afterwards.

แก้ยังไง: Diagnosed after the fact: recall 0% = model never emits any element, plus images were 100% downscaled (separate finding). Both root causes identified same day; retrain not yet redone.

กฎที่ได้: Never launch a paid training round to fix a metric until you have a written diagnosis of why that metric is bad. Diagnosis first, GPU spend second.

สูง 2026-08-25

Started building a fix for section 0% before diagnosing why it was 0% (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Asked whether pattern recognition could help the section subtask, work began immediately on applying the tool. Mid-work the author caught himself doing the steps in the wrong order, backed out, and diagnosed first — at which point the answer changed completely.

ต้นเหตุจริง: A request phrased as 'can tool X help problem Y' was taken as an instruction to apply X, skipping the step of establishing what Y actually is.

เสียหายอะไร: Partial wasted effort, caught before it became a full implementation. Had it not been caught, effort would have gone into a tool that provably cannot help this metric.

แก้ยังไง: Backed out and wrote measure_id_ceiling.py to diagnose the metric instead, which revealed the real problem was the evaluation, not the model.

กฎที่ได้: When asked 'can we use tool X here', answer the diagnostic question first: what is actually broken. A request to try a tool is not a diagnosis.

สูง 2026-08-25

Started building a fix for section 0% before diagnosing it; the diagnosis changed the answer completely (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Asked whether pattern recognition could rescue section's 0% id-recall, the work started by trying the tool, then stopped and went back to diagnose. The diagnosis found the real cause was the scoring metric, not the model — so the tool being built could not possibly have helped.

ต้นเหตุจริง: Jumping to a proposed solution because it was the tool already at hand, instead of first establishing what the failing number actually measures.

เสียหายอะไร: Time spent on three failed pattern-recognition experiments on section (HoughCircles, contour box detection, rebar-dot counting) that were never going to address the real cause. Caught before committing a whole tuning round.

แก้ยังไง: Went back and diagnosed first, building `measure_id_ceiling.py` to compute the achievable ceiling per subtask before proposing any fix.

กฎที่ได้: Diagnose before building, every time — especially when a tool you just built looks like it fits. If you cannot state what the failing number measures and what its theoretical maximum is, you are not ready to fix it.

สูง 2026-08-25

The professor's `max_frames=768` suggestion pointed at a knob this project never invokes (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `max_frames: 768` was found in `processor_config.json` and proposed as a tuning target. It is the hardcoded default in Qwen's video frame sampler. This project sends every drawing page as a still image content item, never as video, so the video processor path is never invoked and raising the value would have zero effect.

ต้นเหตุจริง: A configuration value was identified by name-matching in a config file without tracing whether the code path that reads it is ever executed.

เสียหายอะไร: Would have burned a tuning round changing a value with no effect. Instead the trace surfaced the real knobs (`MAX_PIXELS`, `MAX_LENGTH`) and led to the 100%-downscale discovery.

แก้ยังไง: Grepped the whole repo for `max_frames` (single hit, an auto-saved config), confirmed the training/inference code sends `{"type": "image"}`, and reported back that the real knobs are `MAX_PIXELS` and `MAX_LENGTH`.

กฎที่ได้: Before tuning any config value, trace the code path that reads it and prove it executes in your setup. A value present in an auto-saved config file is not evidence that your code uses it.

สูง 2026-08-25

A full day of tool, config, doc and metric changes left uncommitted at end of day (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: At day's close the uncommitted list included `measure_id_ceiling.py`, `infer_house_t03.py` (the new metric), `measure_capacity.py`, `train_t03.py` (7680/47104), the teaching `.md` + `.pdf` + image folder, `md_to_book_pdf.py` (the image fix), `t03_workflow.md`, and the day's diary. Only the first two commits of the day (pattern recognition) had been committed by Makham.

ต้นเหตุจริง: Commits were deferred pending Makham's review, so a whole day of interdependent changes accumulated in the working tree as a single unreviewed blob.

เสียหายอะไร: A crash, a stray ``git checkout``, or a branch switch loses the day. The related risk is already documented in the Constistant repo: an approved plan and mockup were lost precisely because they were never committed. It also makes the changes hard to review or bisect since they arrive as one lump.

แก้ยังไม: Not fixed — the uncommitted list was written into the diary so nothing is silently lost track of.

กฎที่ได้: Commit each self-contained change as it lands, on a branch if review is pending. "Waiting for review" is not a reason to leave work only in the working tree — an unreviewed commit is recoverable, an uncommitted file is not.

สูง 2026-08-25

pass0 page classification — the riskiest stage in the pipeline — had no cross-check of any kind (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Makham proposed Pass 0.5 (a CV template scan of every page producing a page fingerprint) specifically because pass0's page labels are "ด้านเสียงสุด เดิมไม่มีใครตรวจ" — the stage everything downstream depends on had zero verification. A mislabeled page silently routes to the wrong extraction prompt and its content disappears.

ต้นเหตุจริง: Verification effort went to the extraction outputs (element counts, ids, recall) and none to the routing decision upstream of them, so a whole class of failure could only surface as unexplained missing data much later.

เสียหายอะไร: Every page mislabeled by pass0 to date went uncaught — this is the same shape as the `roof_plan`→`plan` bug where mis-tagged pages made roof-beam BOQ quantities vanish with no error anywhere.

แก้ยังไม: `tools/cv_scan.py` built this evening as the Pass 0.5 cross-check; not yet wired into the real pipeline ("ต่อ `cv_scan` เข้าสายพาน: เริ่มจาก pass0 cross-check" is still pending).

กฎที่ได้: Verify the routing stage, not just the output stage — an unchecked classifier turns every downstream error into an unexplained absence.

สูง 2026-08-26

plan_beam 0% recall had been attributed to image downscaling — the real cause was answer length (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The teaching PDF documents the plan_beam 0%-recall investigation and its real cause: answer length causing one-bad-token JSON collapse, not image downscaling. The downscaling theory was disproven visually with a real before/after crop comparison from an actual training image.

ต้นเหตุจริง: Image resolution was the intuitive suspect (and had been a real bug in t01 at 512px), so it was assumed again without testing. The actual mechanism — long answers giving more chances for a single token to break the JSON — was only found by investigating rather than assuming.

เสียหายอะไร: Investigation time spent on the wrong hypothesis; if uncorrected, the fix would have been more VRAM for higher resolution rather than shorter answers.

แก้ยังไง: Proven with a real crop comparison from a training image and documented in the PDF. Recommended fix direction: split plan_beam into per-grid-zone sub-answers.

กฎที่ได้: A cause that was real in a previous round is the most seductive wrong hypothesis in the next one. Disprove it with a direct measurement before acting on it.

สูง 2026-08-28

Renumbering 22 house folders broke every reference in prior logs and desynced from rawjson_ (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: 22 folders under json_แก้ไขแล้ว were git-mv'd to renumber 01–38 (01–16 unchanged). The original gaps were 12 houses nobody had reviewed yet, not broken names. After the renumber, numbers no longer match the `rawjson\_` tree from 17 upward.

ต้นเหตุจริง: The number gaps were misread as a naming problem to fix rather than a meaningful signal (unreviewed houses). The renumber then broke the identity mapping between two parallel trees and invalidated every numeric reference in prior diary/log entries.

เสียหายอะไร: Every prior log entry referencing a house number is now ambiguous; the only way to read them is the old→new mapping table. Two trees now use different numbering for the same houses.

แก้ยังไง: An old→new mapping table lives at the head of json_แก้ไขแล้ว/README.md, marked ห้ามลบ (must not be deleted) — it is the only way to read old logs. Standing rule: reference houses by name when unsure.

กฎที่ได้: Never renumber identifiers that appear in historical records. Use stable names as the identity and treat numbers as display-only — and if you must renumber, the mapping table becomes permanent load-bearing documentation.

สูง 2026-08-29

Copying unchanged stage files between t03 and t04 previously caused silent drift between duplicated copies (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The project had previously experienced 'drift': the same file existing in two places, someone editing one copy, and the other going stale with nobody noticing. For t04 the team deliberately avoided copying unchanged pass 0/1/2 files and left small pointer READMEs instead.

ต้นเหตุจริง: Duplicating a file creates two sources of truth with no mechanism to keep them in sync; divergence is silent because both copies remain syntactically valid.

เสียหายอะไร: Previously caused stale copies to be used unknowingly; the cost is invisible until wrong behaviour surfaces far downstream.

แก้ยังไง: Adopted the rule that the real file lives in exactly one place and every other location holds only a pointer README.

กฎที่ได้: Never duplicate a file across training rounds or folders. One source of truth, many pointers — if a copy is unavoidable, add an automated check that fails when the copies diverge.

สูง 2026-08-29

jsonl rebuilt at least five times in one day — real risk of training on a stale set already on the rented machine (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: train/val.jsonl was rebuilt repeatedly through the day (prompt hygiene, glossary move, split change, span rule, house 51/52, house 08). Each time the diary had to repeat the same warning: upload the latest set before firing the GPU, do not use the stale one already on the rental.

ต้นเหตุจริง: There is no version/hash stamp on the dataset and no check on the training machine that the uploaded jsonl matches the local build. The only defence is a human remembering to re-upload — and the warning had to be written four separate times, which is the tell.

เสียหายอะไร: A single forgotten upload means paying for a full training run on the wrong data, discovered only after the fact if at all.

แก้ยังไง: Not fixed — mitigated by repeating the warning in the diary and RUNBOOK.

กฎที่ได้: Stamp the dataset with a content hash and have the training script assert it matches the expected hash before step 0. A warning repeated four times in one day is a missing automated check.

สูง 2026-08-29

t04 had been running for days with no Phase 0 round doc, which rule_of_tune §4 requires (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The t04_workflow.md Phase 0 document was created only at the point of the InternVL3 model switch, with the note that this round had never had one.

ต้นเหตุจริง: The t04/Purson conveyor work started as a continuation of t03 rather than as a declared new round, so the round-opening checklist was never triggered.

เสียหายอะไร: Days of pass-v2 design, prompt rewrites, CV template work and dataset rebuilds happened without the round's model/framework/GPU decisions written down — which is exactly why the model switch invalidated a whole toolchain that was still being extended the day before.

แก้ยังง: wrote t04_Purson/t04_workflow.md with a Phase 0 gap list (framework, GPU strategy, dataset converter, new train/infer scripts, xgrammar probe)

กฎที่ได้: A round that gets a new folder name is a new round — open its Phase 0 doc before writing any of its code. The decisions it forces (which model, which framework, which GPU) are the ones that invalidate work retroactively.

สูง 2026-08-29

InternVL3-78B does not fit the single 96GB card the entire rental workflow was built around (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Alongside the Unsloth incompatibility, the model switch also broke the hardware assumption: LoRA in bf16 on a dense 78B needs at least 2×H100-class cards, or an untried single-card QLoRA 4-bit path.

ต้นเหตุจริง: The model was chosen on capability grounds (OmniDocBench score, license) with the VRAM consequence discovered afterward, not as part of the decision.

เสียหายอะไร: Every rental script, onstart, price estimate and the whole \$1/hr single-card cost model become invalid; the round now needs either double the hardware or an unproven quantized path, both unbudgeted.

แก้ยังง: not fixed — recorded in t04_workflow.md Phase 0 as an open GPU-strategy decision

กฎที่ได้: Model selection is a hardware decision. Compute the VRAM requirement and the resulting hourly cost as part of the choice, not as a consequence discovered while writing the workflow doc.

The 25-minute `PAGE\_TIMEOUT\_S` safety net silently failed to cut off the hung request (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: `PAGE\_TIMEOUT\_S=25` นาที existed specifically to kill a runaway page job, but during the 7-hour server hang it never fired — the request was never terminated and no timeout error was raised.

ต้นเหตุจริง: Suspected (not yet confirmed) that the blocking `generate()` call prevented the event loop from ever getting control back to evaluate the deadline — a timeout implemented on the async side cannot preempt synchronous code holding the loop.

เสียหายอะไร: The one mechanism designed to bound the damage from a bad request did nothing, turning a 25-minute worst case into a 7-hour outage and wasted GPU rental.

แก้ยังง: Not fixed. Listed as item 2 of the pre-presentation must-do list: investigate why the timeout did not cut off the job.

กฎที่ได้: A timeout that lives in the same execution context as the thing it is supposed to interrupt is not a timeout. Test every safety net by actually triggering the failure it guards against, don't assume it works because the code exists.

Hand-written SKIP_SUBTASKS blacklist missed 3 subtasks with prompts but zero training examples — worker would have fired prompts the model never saw (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Makham caught that 'ระบบ pass ผิด'. The SKIP_SUBTASKS list had been written from memory rather than checked against `pass\_io\_table\_detailed.csv`, so it missed 3 subtasks that have prompts but 0 training examples: `plan\_column` (dead), `material\_list` (prompt never loaded into training) and `soil\_boring\_log` (no GT at all).

ต้นเหตุจริง: A blacklist was authored by guessing what to exclude instead of deriving from the authoritative training manifest. A blacklist fails open — anything forgotten gets executed — which is exactly the wrong failure direction here.

เสียหายอะไร: Would have sent prompts the model has never been trained on into production, producing plausible-looking garbage with no error.

แก้ยังง: Replaced with an allowlist `TRAINED\_SUBTASKS` containing only the 7 subtasks that were actually trained, derived from the real training table. Forgetting to add one now means skipping it, not firing blind.

กฎที่ได้: When the cost of a false positive is silent garbage, use an allowlist derived from the authoritative source file, never a hand-written blacklist. Fail closed.

สูง 2026-08-30

The 78B adapter the whole round paid for cannot be served at all — trained with LLaMA-Factory, but the only working server loads via Unsloth, so `--model t04` is rejected outright (ยังไม่ได้แปล —

ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: vLLM was ruled out because Unsloth's MoE LoRA layout matches neither format it accepts, so `serve\_purson.py` was written around Unsloth's load/generate path lifted from `infer\_house\_t03.py`. That server can only load Unsloth-trained adapters, so `--model t04` (the InternVL3-78B trained with LLaMA-Factory) is refused with a message saying the loader from `infer\_house\_t04.py` must be ported first. The round's actual output has no production path.

ต้นเหตุจริง: Training framework and serving framework were chosen independently at different times — LLaMA-Factory was picked for training because it was the only route to QLoRA on InternVL3-78B, and Unsloth for serving because it was the only proven load path — and nobody checked at decision time that the two could meet.

เสียหายอะไร: The 78B round produces artefacts that cannot be served without an unbudgeted loader port; the presentation stack falls back to the older t03 adapter instead.

แก้ยังไม: Not fixed. `--model t04` fails fast with an explicit message rather than breaking mid-demo — deliberately chosen over letting it fail live.

กฎที่ได้: Pick the serving path at the same time as the training framework and prove one loads the other's output before paying for a training run — 'we trained it' and 'we can run it' are two separate claims.

สูง 2026-08-31

Fixing one confirmed root cause was mistaken for unblocking the whole round (ยังไม่ได้แปล — ต้นฉบับ

อังกฤษ)

เกิดอะไรขึ้น: After Lesson 15 identified crop_to_patches as t04's root cause and it was verified correct, the team assumed the arm was fixed. A second disease (4-bit killing the ViT) sat underneath and would have produced identical symptoms — beautiful JSON, ~0% recall.

ต้นเหตุจริง: Treating 'root cause found and fixed' as terminal, rather than re-walking the remaining known risk list that the design dossier had already written down.

เสียหายอะไร: Would have burned another full multi-GPU night on a model that could not see, had the ~\$2 gate not caught it.

แก้ยังไม: Recorded as rule_of_tune Lesson 16 item 3: after fixing a root cause, walk the remaining risk list from the round's design doc and build a gate for each item.

กฎที่ได้: A confirmed root cause fix is not a green light — re-read the round's own risk list and put a cheap detection gate on every remaining item before spending money.

สูง 2026-08-31

Sequence-length headroom was only 2.8% and the guard against silent label truncation was only a comment (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Measured longest sequence was 45,839 against MAX_LENGTH 47,104 — a 2.8% margin, below the 5% that should exist. An audit had already noted that the t01-class bug (labels silently truncated mid-JSON) had comments but no actual runtime check.

ต้นเหตุจริง: Capacity was computed on paper and documented as an intention rather than enforced by a preflight assertion at data-load time.

เสียหายอะไร: Any slightly longer row would have silently truncated a training label mid-JSON, teaching the model malformed output with no error raised.

แก้ยังไง: Added preflight_seq_lengths() to the training script: measures visual tokens from real image dimensions plus text tokens from the real tokenizer for every row at load time and halts immediately if any row exceeds the cap.

กฎที่ได้: A documented limit with no runtime assertion is not a guard — enforce every capacity constraint with a hard fail at data-load time, before the first training step.

สูง 2026-08-31

MAX_PIXELS and MAX_LENGTH had been paper-calculated since 2026-08-25 and never actually run (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The workflow doc had recorded since 25 Aug that MAX_PIXELS=7680 / MAX_LENGTH=47104 with a 4.0 GB margin was a calculation on paper that had never been executed, and required TEST_STEPS=5 confirmation. This round was the first to actually run it — and it hit the ceiling.

ต้นเหตุจริง: A capacity budget was documented as validated-looking numbers when it was in fact an untested projection.

เสียหายอะไร: OOM at step 2 during dry-run, then real OOMs on 2 of 4 folds in the full run.

แก้ยังไง: Followed the pre-written fallback order: reduce gridline images 4->3 first, then MAX_PIXELS, never reduce MAX_LENGTH (which truncates labels).

กฎที่ได้: Label every untested capacity figure as untested in the doc, and require a real measured run before any spend that depends on it.

สูง 2026-08-31

Extracting the data tarball overwrote the already-fixed training script with an old copy inside it (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: folds4.tar.gz contained an old version of train_t05_courser.py. Extracting it after pushing the fixed script overwrote the fix, so gpu4 ran the probe with the old code and died with 401 (still pushing during dry-run).

ต้นเหตุจริง: A data bundle also carried source files, and the operation order was push-script-then-extract-tarball.

เสียหายอะไร: One machine failed its probe and time was lost diagnosing an already-fixed bug.

แก้ยังไง: Re-sent the new script and grep-verified a marker string on all four machines rather than trusting scp.

กฎที่ได้: Never let a data bundle carry source files; if it must, extract first then push code, and always grep-verify a version marker on every machine instead of assuming the copy landed.

สูง 2026-08-31

The demo launcher has still never completed end-to-end — all three evening rentals ended without a working server (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: Evening demo prep burned three instances in a row: 49408084 rented after Makham said stop, 49410094 started the wrong model (t03), 49411642 hit the pgrep self-match and never started the server. The fixed path is scheduled to be tried for the first time the next morning.

ต้นเหตุจริง: Each fix was verified only against the specific failure it addressed, never by running the whole GO.bat path once from a cold start.

เสียหายอะไร: Roughly three rentals of billed time with no working demo, and the presentation-day path is still unproven going into 1 Sep.

แก้ยังไง: not fixed — first full end-to-end run deferred to the next day

กฎที่ได้: After fixing a step in a chain, run the whole chain once from cold; three consecutive single-step fixes still leave the chain unproven.

สูง 2026-09-01

t04 was trained to completion with no cheap pre-flight check, so the failure was only found after paying (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: t04's fatal defect was detected only after the full training run finished and cost several thousand baht. The equivalent defect in t44 was caught for about \$2 because a go/no-go gate had by then been introduced.

ต้นเหตุจริง: No pre-spend validation step existed in the process at the time of t04. Verification was implicitly scheduled after the expensive step rather than before it.

เสียหายอะไร: Several thousand baht of GPU spend lost on t04 that a ~\$2 check would have prevented.

แก้ยังไง: Introduced a go/no-go gate: load the model and have it read 3 real drawing pages before committing to a full run. Pass → train; fail → stop.

กฎที่ได้: Put the cheapest possible correctness gate immediately before every expensive step. A \$2 check that can save \$70 must run every time, without exception, even when the setup 'looks obviously fine'.

สูง 2026-09-01

Live demo day consumed entirely by host failures and teardown/retry cycles, demo never completed (ยังไม่ได้แปล — ต้นฉบับอังกฤษ)

เกิดอะไรขึ้น: The whole session was spent on infrastructure problems: a dead tunnel, a bad host requiring destroy-and-re-rent, and two full teardown cycles ('presentation.py down', verified 0 billed instances both times, one of them just to redo the demo with screen recording on). The live demo itself was never completed and got pushed to the next day.

ต้นเหตุจริง: No pre-demo smoke test. The demo attempt itself was the first end-to-end check, so every latent infra fault (tunnel, host, key) surfaced during the demo window instead of before it.

เสียหายอะไร: An entire working session lost with no demo delivered; GPU rental costs incurred across multiple instances for zero demo output.

แก้ยังไง: None on the day — Makham's own call was to retry the next day.

กฎที่ได้: Run a full dry-run of the demo path (rent → tunnel → submit → one page) hours before the actual demo slot, on a cheap instance. Never let the demo be the first time the stack is exercised end-to-end.

เรื่องที่เหลือในหมวดนี้ (134)

ไม่มีบันทึกการทำงานเลยก่อนวันที่ 2026-07-05 และไม่เคຍย้อนเอา git history ช่วงแรกมาเขียนเป็นไดอารี่

กลาง 2026-07-01

เริ่มเขียนบันทึกการทำงานตั้งแต่วันแรกของโปรเจกต์ ถ้าเริ่มเข้าไปแล้ว ให้ย้อนไล่ git history ที่มีอยู่มาเขียนย้อนหลังทีเดียวทันที เพราะ commit message บอกแค่ว่าจะไรเปลี่ยน แต่ไม่เคยบอกว่าทำไม และ 'ทำไม' คือส่วนเดียวที่ย้อนกลับไปสร้างใหม่ไม่ได้

ย้ายเอกสารข้ามรีโปโดยไม่ได้เช็คว่ามีอะไรลิงก์มาที่ path เดิมบ้าง

กลาง 2026-07-05

ก่อนย้ายหรือเปลี่ยนชื่อไฟล์ที่มีเอกสารอื่นอ้างถึง ให้ grep หา path เดิมทั้ง workspace แล้วแก้ทุกจุดในการแก้ครั้งเดียวกัน การย้ายยังไม่ถือว่าเสร็จจนกว่าลิงก์ขาเข้าจะเสร็จด้วย

schema มีจุดที่ยังไม่ตัดสินใจ 5 จุดตั้งแต่เริ่มสกัดข้อมูล และหนึ่งในนั้นเพิ่งโผล่กลางงานจนงานติด

กลาง 2026-07-05

ให้สำรวจประเภทหน้าแบบคร่าวๆ ทั้งชุดเอกสารก่อนออกแบบ schema ไม่ใช่ดูแค่ตัวอย่างบางส่วน การเจอหน้าประเภทใหม่ทั้งกลุ่มกลางคันจะบิบบให้ตัดสินใจเรื่อง schema ตอนกำลังรีบ ซึ่งเป็นที่มาของถังขยะรวมที่ไม่มีใครกลับมาเก็บกวาดอีกเลย

คีย์ localStorage ถูกประกาศเองในโมดูล 14 ที่ พร้อมรายการคีย์แบบดูแลด้วยมือ ซึ่งคือต้นเหตุจริงของบั๊ก **deleteProject** ที่ทิ้งข้อมูลกำพรว้าไว้

กลาง 2026-07-05

รายการที่ควรจะได้มาจากข้อมูลอื่น แต่ดูแลด้วยมือ ยังไงก็ต้องเก๋าค้าง ให้ derive มันมาจากทะเบียนกลางแทน และเวลาบั๊กเป็นแบบ 'ลืมเพิ่ม X ลงในรายการ Y' ให้แก้ที่กลไกที่ยอมให้รายการนั้นแยกออกมาได้ ไม่ใช่แค่เติมรายการที่ขาด

เขียน config ของ Label Studio ส่งไปโดยไม่เคยเปิดดูเอง มะขามเลยต้องวางใหม่ 4 รอบในเย็นเดียว ตามที่แต่ละจุดฟังโผล่มาจากสกรีนช็อตของเขา

กลาง 2026-07-06

อย่าส่ง config ของ UI ให้คนอื่นเป็นคนทดสอบครั้งแรกแทน ให้โหลดดูเองสักรอบก่อน แม้จะเป็น task ปลอมอันเดียวก็น่าดี ค่อยบอกว่าพร้อม ทดสอบเองรอบเดียวจะยุบวงจรวางใหม่ 4 รอบให้เหลือรอบเดียว

หน้า BOQ 41-47 ค้างไม่ได้สักสองวัน หลังจากรอบก่อนถูกหยุดกลางคัน

กลาง 2026-07-07

เวลางานถูกหยุดกลางคัน ให้เขียนหน่วยสุดท้ายที่ทำเสร็จลงในรายการค้างทันที ช่วงที่ขาดหายไปถ้าอยู่แค่ในบทสนทนา จะถูกเอามาเกลี่ยกันใหม่ทุกเซสชัน

ข้อความ 'Issue loading URL from \$image value' ถูกสงสัยว่าเป็น CORS หรือยังไม่ push ขึ้น GitHub ทั้งที่ URL ปกติดีมาตลอด

กลาง 2026-07-08

เจอ error แบบ 'โหลด URL ไม่ได้' ให้ยิง URL นั้นด้วยตัวเองก่อนเลย (curl ทั้งแบบใส่และไม่ใส่ Origin) ก่อนจะไปตั้งทฤษฎี เพราะปัญหาฝั่ง server กับฝั่ง client ขึ้นข้อความ error ในแอปหน้าตาเหมือนกันเป๊ะ

สร้างเส้นทาง migration เพื่อประหยัด token ขึ้นมาทั้งชุด รันไปแล้ว แล้วลบทิ้งภายในไม่กี่นาที

กลาง 2026-07-08

ตัดสินใจให้ชั้ดก่อนว่ารอบนี้ทำเพื่อ 'จัดรูปข้อมูลใหม่' หรือ 'ตรวจสอบซ้ำ' ก่อนจะไปคิดเรื่องลดต้นทุน เพราะ migration ไม่มีทางเจอข้อผิดพลาดในข้อมูลที่มันกำลังย้ายอยู่ได้เลย

หน้าของ t3 ประมาณ 13 หน้าติด pattern type ผิด และเจอเพราะ t4 มาอ่านรูปใหม่ทุกหน้าเท่านั้น

กลาง 2026-07-08

คำศัพท์แบบปิดที่ไม่มีคำนิยามเขียนกำกับแต่ละคำ ไม่ใช่คำศัพท์ที่ควบคุมได้ มันเป็นแค่ข้อเสนอแนะ ให้นิยามทุกชนิดให้ครบก่อนสกัดข้อมูลรอบแรก ไม่ใช่หลังรอบที่สอง

คำสั่งย่อ op1 ถูกเขียนเป็นเอกสาร workflow 7 ขั้นตอนรูปแบบ ทั้งที่ยังไม่เคยรันสักครั้ง

กลาง 2026-07-10

อย่าเขียน workflow เป็นเอกสารหลักก่อนที่จะรันจริงจนจบสักรอบ เพราะการรันจริงรอบแรกคือสิ่งที่บอกว่ามีจุดฟังแบบไหนบ้างที่ spec ต้องรับมือ

agent ที่ถูกฆ่าเพราะชนลิมิตของเซสชัน รายงานจุดเช็คพอยต์ในข้อความสุดท้ายเก่ากว่าที่มันเขียนลงดิสก์ไปจริงๆ

กลาง 2026-07-12

หลัง agent ถูกฆ่าทุกครั้ง ไฟล์บันดิสก์คือแหล่งความจริงเดียวเรื่องความคืบหน้า อย่าทำงานต่อจากข้อความสุดท้ายของตัว agent เอง

แก้ไฟล์เกี่ยวกับการ tune โดยไม่ได้ทำขั้นตอนเตือนที่กฎข้อ 2 กำหนดไว้

กลาง 2026-07-13

เวลาถูกกำหนดขอบเขตด้วย path ให้เช็คก่อนว่าโพลเดอร์ใหม่เข้าข่ายเจตนาของกฎนั้นหรือเปล่า อย่าเหมารวมมันได้รับการยกเว้น โพลเดอร์งานใหม่ต้องเคาะขอบเขตของกฎก่อนจะแก้ไฟล์แรก ไม่ใช่มาเคาะทีหลัง

ไฟล์ task ของ Label Studio ถูก generate ไว้สำหรับบ้านหลังที่ 2-5 แต่ไม่เคยถูก import จริง

กลาง 2026-07-13

pipeline ที่ขั้นสุดท้ายต้องทำมือ จะหยุดค้างอยู่ที่ขั้นอัตโนมัติขั้นสุดท้ายเสมอ ให้เขียนสคริปต์ import ชะ หรือไม่ก็นิยามคำว่าเสร็จเป็น 'import แล้ว' ไม่ใช่ 'generate แล้ว'

Architecture-level finishing-material and door/window data extracted for five houses is used nowhere in the app

กลาง 2026-07-14

Before adding a field class to the extraction scope, name the downstream consumer that will read it. Extract-because-it-is-printed is how a dataset grows cost without growing value.

Beam B2 endpoints mistyped as D'2/D'3 instead of D2/D3

กลาง 2026-07-15

After adding a dummy grid, run a check that every grid_ref in the file resolves to a grid the element genuinely touches — hand edits near a newly added label are where transcription slips happen.

An external AI recommended a rebar grade change that would have corrupted correct data

กลาง 2026-07-15

Treat external AI advice as a hypothesis, never as a finding. Verify against the source image and the project's own convention rules before editing anything on its say-so.

A page12 dimension claims a 2.70m E-D span while the grid master says 2.0m

กลาง 2026-07-16

Add a validation pass that checks every per-file `dimension\_m` against the grid master's computed span. Contradictions between a file and the master should be a hard error, not a note.

House 4's dummy grid E' has three conflicting values across three structural sheets of the same house

กลาง 2026-07-16

Leaving a three-way conflict flagged rather than averaged is correct — but a grid-master value in conflict should block cascading into dependent files, not just carry a warning.

The conclusion 'the 1.30 eave is unrelated to 1'/4" was half wrong and had to be revisited

กลาง 2026-07-16

A negative conclusion about one feature does not generalise to the sheet — enumerate the distinct features on a sheet before ruling out a relationship, and delete the artefacts of any superseded reading in the same edit.

Grid refs were extracted with 'grid'/'dummy' prefixes and spelled-out primes, forcing a bulk normalization pass in every house

กลาง 2026-07-19

Fix the canonical string format in the schema before extraction, not after — retrofitting formats across houses costs a regex sweep per house and each sweep is a chance to corrupt an adjacent field.

An uncalibrated pixel measurement produced a false discrepancy flag against a correct grid value

กลาง 2026-07-20

Never record a measurement-based discrepancy without first calibrating px/m against two known grid centrelines on that same sheet. An uncalibrated measurement is not evidence.

A grid_ref written as 'A-B' contradicting the file's own description

กลาง 2026-07-20

Validate every grid_ref against its own file's description text — a file that contradicts itself is a detectable class of bug that a script can catch, and prime labels are easy to confuse.

Multi-line paste silently dropped commands (a `cd` never ran)

กลาง 2026-07-21

One command per paste, and wait for the prompt to return before the next one — never bulk-paste a command block into a live terminal.

Full training believed started, but the command was only queued behind the still-running test job

กลาง 2026-07-21

A command has started only when the prompt returns; verify with a process/log check before assuming a long job is running — never type ahead into a busy terminal.

Burned day-21 effort trying to export mmpoj when the stock vendor file was already valid

กลาง 2026-07-21

Before fighting a tool limitation, check whether your own config makes the problem moot — a frozen component does not need re-exporting, the upstream artifact is still valid.

House 5's dummy grid B' recorded as 12.5 before checking the source sheet corrected it to 12.2

กลาง 2026-07-24

Never place a dummy grid at whatever value makes a beam land — measure it from the source sheet first, then check whether the beam fits. Reverse-fitting a grid to an element hides the real discrepancy.

Both attempts at the big-house extraction run were killed, producing no output at all

กลาง 2026-07-25

A multi-hour inference job needs per-page output checkpointing and should run detached (tmux/nohup) rather than in an interactive foreground session, so a kill costs one page and not the whole run.

Assumed comparable model size implies comparable training time — Qwen3-VL ran 2.9x slower than Qwen3.6 with identical settings

กลาง 2026-07-28

Never estimate runtime or cost from parameter count. Measure step time on a short test run (TEST_STEPS) and extrapolate from the measurement before committing to the full run.

Re-verifying t01 on the same machine was not planned into the rental, forcing a second ~70GB base-model download on the clock

กลาง 2026-07-28

In a controlled A/B, both arms must be measured on the same machine in the same session — otherwise the control run becomes an expensive, unplanned add-on the moment anyone doubts the numbers.

Near-misdiagnosis: a silent model was almost declared hung

กลาง 2026-07-29

'No output' is never evidence of 'stopped'. Before killing or debugging a silent long-running job, confirm death on a second channel: GPU utilisation, file size growth, or a heartbeat log line the job emits on its own.

The extraction script written on the 28th lived in the scratchpad and was gone by the 29th

กลาง 2026-07-29

Anything you ran successfully once is a work product: move it out of the scratchpad and commit it before the session ends. Scratchpad is for output you will never want again, not for the script that produced it.

Original plan was a full extract of every page — ~16 hrs/house before anyone did the arithmetic

กลาง 2026-07-29

Multiply unit cost by unit count before starting any batch job, and compare against the time you are actually willing to spend. A five-second calculation prevents starting a sixteen-hour job you would have cancelled.

Same page-scoping mistake repeated on house 09 — 4 'criteria' pages, only 1 real

กลาง 2026-07-30

When a defect is found and fixed in one house, patch the process that generated it before starting the next house. Also: hash back-matter pages across houses in a series — byte-identical pages should be recognised, not re-read.

Six near-identical BOQ sets (174 sheets) were being re-read sheet by sheet instead of diffed

กลาง 2026-07-31

Before extracting a near-duplicate document set, diff it against the extracted one first and read only the differing sheets. Prove byte-identity, then reuse.

Item-55 lesson applied: don't trust the fix-log, verify against the real data before editing the spec

กลาง 2026-08-02

'The log says it's fixed' is a hypothesis, not a fact. Grep the actual data before making any decision that depends on a past fix having landed.

5 files still carried `phase\_note`, which by the schema's own rule means they are unfinished

กลาง 2026-08-02

Any 'this is unfinished' marker needs a checker that fails on it. A TODO nobody scans for is the same as no TODO.

A folder rename required editing 454 JSON files because full paths were embedded in data fields

กลาง 2026-08-02

Never embed a top-level folder name in a data field. Store references relative to a configured root so the root can be renamed without touching data.

เจ็ด session ติดกันไม่เคยบันทึกไดอารี่สดๆ ต้องมานั่งย้อนดูจาก git log ทีหลัง

กลาง 2026-08-05

เพิ่ม 'เขียนไดอารี่แล้ว' เข้าไปในด่านก่อนปิดเครื่องของ op03 ควบคู่กับการ commit เพราะ git log บอกได้แค่ว่าจะอะไรเปลี่ยน ไม่เคยบอกว่าตัดสินใจอะไรไป หรือเกือบพลาดตรงไหน

บ้านหลัง 15 ถูก commit ด้วยข้อความว่า 'บ้านต่อมา' และไม่ได้บันทึกผล check_format.py ไว้เลย

กลาง 2026-08-08

ทำให้ผลของตัวตรวจเป็นส่วนหนึ่งของ commit เลย (ผ่าน pre-commit hook หรือบังคับให้มีในข้อความ commit) ด่านคุณภาพที่ไม่ได้บันทึกผลไว้ ถือว่าไม่เคยผ่านด่านนั้น

งานสกัดข้อมูลบ้านเจ็ดวันไม่ได้บันทึกสดเลย ต้องมาย้อนสร้างจาก git log ทีหลัง

กลาง 2026-08-18

งาน op รอบหนึ่งยังไม่ถือว่าจบจนกว่าจะเขียนไดอารี่เสร็จ ให้เขียนใน session เดียวกับที่ทำงาน ก่อนจะทำลายเครื่องเข้าหรือปิด session เพราะการย้อนจาก git log ได้แค่ว่าจะอะไรเปลี่ยน ไม่มีทางได้ว่าทำไม

เพดานเวลา 90 นาทีของ op3 ทำให้บ้านหลังใหญ่ที่สุดไม่มีวันเสร็จเลยอย่างเป็นระบบ

กลาง 2026-08-18

ถ้างานที่จับเวลาไว้ทำงานชิ้นใหญ่สุดไม่ทัน ให้เพิ่มโหมดทำต่อได้หรือแบ่งงานเป็นท่อน อย่าปล่อยให้การเลือกงานเอียงไปหาของง่ายเสียๆ ไม่งั้นเคสยากจะไม่มีวันเข้า dataset เลยสักที

สคริปต์เปรียบเทียบและ overlay เขียนไว้ใน scratchpad ของ session ไม่เคย commit เลย

กลาง 2026-08-20

สคริปต์ไหนที่ผลิตตัวเลขที่คุณจะเอาไปอ้าง ต้อง commit ใน session เดียวกันนั้น โค้ดที่ใช้วัดผลคือของส่งมอบ ไม่ใช่ของชั่วคราว ถ้าพบว่าตัวเองต้องเขียนมันใหม่อีกรอบ แปลว่ารอบแรกก็ควร commit ไปแล้ว

แก้ repetition penalty แล้ว commit เลย โดยไม่เคยรันบน GPU สักครั้ง

กลาง 2026-08-21

การแก้พารามิเตอร์ decoding ยังถือว่าไม่ได้ทดสอบ จนกว่ามันจะสร้าง token จริบนหน้าที่เคยฟังให้ดู ให้ระบุชัดๆ ทั้งในโค้ดและไดอารี่ว่ายังไม่ได้พิสูจน์ และห้ามใช้มันปิดบั๊ก

pass1_organize.py เขียนเสร็จและส่งออกไปโดยไม่เคยรันกับรูปจริงเลย

กลาง 2026-08-21

ถ้าถูกสั่งให้ข้ามการทดสอบ ให้ส่งโค้ดพร้อมโน้ตตัวใหญ่ว่าอะไรที่ยังไม่เคยถูกใช้จริง และบอกไปพร้อมกันเลยว่าชุดทดสอบที่ถูกที่สุดคืออะไร เพราะโค้ด crop รูปที่ไม่ได้ทดสอบจะฟังแบบเสียบๆ ด้วยการให้ crop ที่ผิดแต่ดูน่าเชื่อ

ถ้าให้ Pass 0 จำแนกออกมาเป็น pattern 16 ค่า จะทำให้ Pass 1 ตัดสินใจส่งงานต่อไม่ได้เลย

กลาง 2026-08-21

เขียนไฟล์สัญญาาระหว่างสองขั้นตอนก่อนจะลงมือเขียนขั้นตอนไหนก็ตาม สัญญานั้นแหละคือจุดที่จะเห็นชัดว่าเป้าหมายการจำแนกละเอียดไม่พอ และเห็นตรงนั้นถูกกว่ามารู้ออนเทรนเสร็จเยอะ

รูปแบบผลลัพธ์ของ run_house_batch.py เข้ากันไม่ได้กับตัว import ของแพลตฟอร์ม

กลาง 2026-08-21

นิยามสัญญาว่าไฟล์ฝั่งผู้ใช้ข้อมูลต้องหน้าตาอย่างไร ก่อนจะเขียนรูปแบบผลลัพธ์ของฝั่งผู้ผลิต และถ้าเสี่ยงตัวแปลงไม่ได้จริงๆ ให้มันคัดลอกเนื้อหาแบบตรงตัวและบันทึกทุกฟิลด์ที่มันสร้างขึ้นเอง เพื่อไม่ให้มีใครเข้าใจผิดว่าไฟล์ที่ผ่านตัวแปลงคือไฟล์ที่คนตรวจแล้ว

skill ชื่อ op04 เขียนและลงทะเบียนไว้ โดยไม่เคยเอาไปรันกับบ้านสักหลัง

กลาง 2026-08-21

skill ที่ลงทะเบียนไว้ = คำสัญญาว่ามันใช้ได้ ห้ามลงทะเบียน workflow ที่ยังไม่ทดสอบโดยไม่มีโน้ตตัวใหญ่บนหัวไฟล์ skill ว่ายังไม่เคยรัน และต้องเอาไปรันกับของจริงหนึ่งชิ้นก่อนเริ่มแคมเปญถัดไป

สร้างเส้นทาง inference ด้วย GGUF ในเครื่องเสร็จ แล้วทิ้งทันทีที่วัดได้ว่าใช้เวลา 23 นาทีต่อหน้า

กลาง 2026-08-24

จับเวลาหน่วยงานตัวแทนหนึ่งชิ้นบนเครื่องที่จะใช้จริงก่อน แล้วค่อยสร้าง workflow รอบๆ มัน การวัดแค่ 60 วินาทีฆ่าเส้นทางที่ไม่เวิร์กได้ก่อนที่มันจะกินเวลาไปทั้งวัน

การตัดสินใจที่ได้รับมอบอำนาจไว้แล้วภายใต้ att1235 ถูกเขียนว่า "รอมะขาม" แทนที่จะตัดสินใจไปเลย

กลาง 2026-08-24

เมื่อได้รับคำสั่งยืมมาให้ตัดสินใจเอง ให้บันทึกผลว่า "ตัดสินใจแล้ว ☒ + เหตุผล + ความเสี่ยงที่รับไว้" ห้ามเขียนว่า "รออนุมัติ" การโยนการตัดสินใจที่มอบหมายมาแล้วกลับไป ตัวมันเองคือความผิดพลาด

ตารางเทียบ train_t03.py กับค่าเริ่มต้นของ t02 ไม่เคยทำงานครบ

กลาง 2026-08-24

ป้ายบอกที่มากับได้แค่เจตนา ตารางเทียบต่างหากที่จับของที่ตกหล่น ให้ทำทั้งสองอย่าง คือไล่ทุก argument ที่ส่งเข้าไปเทียบกับรอบที่ใช้ได้ล่าสุด ไม่ใช่เทียบแค่ตัวที่เราตั้งใจเปลี่ยน

สร้าง template matching หลายสเกลขึ้นมาแก้ปัญหาเรื่องขนาด ทั้งที่จริงๆ มันเป็นปัญหาเรื่องรูปทรง

กลาง 2026-08-25

ทดสอบสมมติฐานด้วยการเปิดข้อมูลจริงขึ้นมาดูก่อน แล้วค่อยเขียนโค้ด สิบวินาทีที่ใช้ดูด้วยตา ชนะการนั่งเขียนทฤษฎีผิดๆ เป็นชั่วโมง เหมือนบทเรียนข้อ 14 ใน rule_of_tune.md

Pixtral-Large was evaluated before its license was checked — commercial use is disallowed

กลาง 2026-08-25

Put the cheapest hard filter first. License and commercial-use terms take seconds to check and eliminate candidates before you spend time on weight sizes and framework support.

Claude misunderstood pass3 as "discard" when it is a real extraction stage with 8 subtasks

กลาง 2026-08-25

Read a pipeline stage's own spec before reasoning about it. Write down the corrected understanding immediately — an unrecorded correction gets re-made by the next session.

The training set was rebuilt AFTER the TEST_STEPS sanity check ran, so the recorded check described data that no longer existed

กลาง 2026-08-25

Stamp every recorded measurement with the dataset build it came from, and re-run the sanity checks after any rebuild — otherwise a rebuild silently orphans every number recorded before it.

House 42 deleted locally (uncommitted) while origin held newer teammate edits — merge conflict

กลาง 2026-08-26

Never resolve a deletion-vs-edit conflict by deleting. Park the disputed data in a holding area and decide later — and announce a house's removal before deleting it locally.

None of the GPU-research recommendations were acted on

กลาง 2026-08-26

Convert every research recommendation into a concrete next action with a cost estimate the same session, or it will not happen. A recommendation buried in prose is a recommendation not made.

train/val/test.jsonl regenerated — the previous set is gone with no snapshot

กลาง 2026-08-26

Stamp each dataset build with a hash/version and keep the manifest, so any result can be tied to an exact dataset. Do not bundle multiple independent changes into one regenerate — you lose the ability to attribute a metric change to a cause.

GT correction commits say only "final"/a house number — reasoning is unrecoverable

กลาง 2026-08-26

A GT data correction commit must name what changed and why (which drawing, which value, what it was before). "final" tells a future reader nothing and destroys the audit trail on data that feeds a model.

A 156-line GT page deleted from house 37 with no recorded reason

กลาง 2026-08-27

Deleting GT is a data decision, not a cleanup. It needs its own commit and a one-line reason, always.

สัญญาของ hint block อยู่สองที่ (prompt.md กับ cv_hint_text()) มีแค่คอมเมนต์กันไม่ให้เขียน

กลาง 2026-08-29

เวลาที่เอกสารสเปกกับโค้ดต้องตรงกัน ให้เขียนเฮสต์อัตโนมัติตรวจผลลัพธ์ของโค้ดเทียบกับสเปก คอมเมนต์ที่เขียนว่า 'อย่าลืมแก้ไขให้ตรงกัน' ไม่ใช่ตัวควบคุมอะไรเลย

prompt ของ pass 4 เขียนเสร็จทั้งชุด แล้วถูกยกเลิกเพราะไม่จำเป็น

กลาง 2026-08-29

ก่อนสร้างขั้นตอนสกัดข้อมูลใหม่ ให้ยืนยันก่อนว่าปลายทางเอาไปใช้จริงไหม ไล่ดูว่าการคำนวณตอนท้ายอ่านฟิลต์ไหนบ้าง แล้วสร้างเฉพาะ pass ที่ป้อนฟิลต์เหล่านั้น

แผนการทดลอง 2.4b (ภาพที่มาร์คเลข) ถูกสร้างเข้าไปใน pipeline แล้วก็ถูกยกเลิก

กลาง 2026-08-29

ตัดสินใจว่าการทดลองจะมีก็แผนก่อนลงมือเขียนกลไกสลับแผน เริ่มจากตัวควบคุมบวกอีกหนึ่งแผนพอ แล้วค่อยเพิ่มเมื่อผลเปรียบเทียบกับรอบแรกบอกว่าต้องเพิ่มจริง

prompt ถอดปริมาณของ **pass 3** เขียนเสร็จเต็มรูปแบบแต่ไม่เคยรันเลย และอาจต้องเขียนใหม่ทั้งอัน

กลาง 2026-08-29

อย่าสร้างขั้นตอนที่การออกแบบขึ้นอยู่กับคำถามต้นน้ำที่ยังไม่มีคำตอบให้เสร็จ ทดลองแบบถูกๆ เพื่อเคลียร์ข้อสันนิษฐานนั้นก่อน แล้วค่อยสร้าง

หัวข้อเชาการ์ด Vast ใน RUNBOOK มีกฎเหล็กสามข้อ — การ์ดถูกก่อน แยก venv ดึงผลลัพธ์ก่อนทำลาย — ทุกข้อเขียนหลังจากเสียหายไปแล้ว

กลาง 2026-08-29

กฎที่อยู่แค่ใน runbook จะถูกข้ามพอดีตอนที่มันสำคัญที่สุด โดยเฉพาะขั้นตอนทำลายเครื่อง ควรให้มันปฏิเสธที่จะรันจนกว่าจะยืนยันแล้วว่าไฟล์ผลลัพธ์ออกจากเครื่องมาแล้วจริง

ย้าย prompt และสคริปต์จาก t03 ไป t04_Purson แล้ว path แบบ relative พังไปหลายไฟล์

กลาง 2026-08-29

พิสูจน์ว่าการย้ายโฟลเดอร์สำเร็จด้วยการสร้างไฟล์ผลลัพธ์ใหม่แล้วเทียบจำนวนกับก่อนย้าย — import ผ่านไม่ใช่หลักฐาน และที่ดีกว่านั้นคือให้อ้าง path ข้ามไฟล์ผ่านค่าคงที่ตัวเดียว การย้ายจะได้แค่แคบรทัดเดียว

InternVL3 ถูกปิดตกในแคตตาล็อกวันที่ 2026-08-26 แล้วเอากลับมาใช้อีกสามวันถัดมา — เหตุผลที่ปิดตกใช้ตัดสินใจอะไรไม่ได้เลย

กลาง 2026-08-29

การปิดตกโมเดลต้องอ้างข้อบกพร่องที่วัดได้ หรือข้อจำกัดเชิงๆ (license, framework รองรับไหม, VRAM) ไม่ใช่ความเห็นเรื่องสถาปัตยกรรม จัดอันดับตัวเลือกด้วยเกณฑ์ที่ทดสอบได้ เวลากลับค่าจะได้เป็นเรื่องข้อมูล ไม่ใช่เรื่องความชอบ

prompt ของ pass 3 กับ merge_guard เขียนเสร็จแต่ไม่เคยรันกับโมเดลจริง — และถูกห้ามลองด้วย

กลาง 2026-08-29

โค้ดที่ยังไม่เคยรัน ให้ติดป้ายว่ายังไม่ผ่านการตรวจสอบไว้ตรงจุดที่คนจะกดรันมัน ไม่ใช่แค่ในเอกสาร workflow เขียนเสร็จแล้วตัวเองไม่เท่ากับเคยรันจริง

โครงสร้าง k-fold CV ถูกสร้างขึ้นทั้งที่ตัดสินใจไปแล้วว่าไม่คุ้ม GPU-hours และไม่เคยรันเลย

กลาง 2026-08-29

สร้างตามที่ถูกสั่งได้ แต่ต้องผูกเงื่อนไขไว้ตั้งแต่ตอนสร้างว่า 'รันเมื่อ X' ระบบที่ไม่มีเงื่อนไขว่าจะรันเมื่อไรและแพงกว่าเดิม 5 เท่า แยกไม่ออกจากระบบที่จะไม่ถูกรันเลย และมันจะผูกเมื่อ toolchain ข้างล่างเปลี่ยน

ยังมีความเสี่ยงเดิมคือคืนแค่การ์ดใบสุดท้าย และสับสนระหว่าง stop กับ destroy (บทเรียน 2026-08-24) — รอบนี้มั้งตั้ง 4 ใบ

กลาง 2026-08-30

เช็คลิสต์รื้อระบบต้องไล่ทรัพยากรทีละ ID และปิดท้ายด้วยการถามฝั่งผู้ให้บริการว่าเหลือศูนย์จริง ห้ามเขียนลอยๆ ว่า 'คืนการ์ด'

รายงานว่า 'อัปเดต workflow แล้ว' ทั้งที่ยังไม่ได้แก้ไฟล์จริง

กลาง 2026-08-30

ห้ามรายงานว่าแก้ไฟล์เสร็จถ้ายังไม่ได้ส่งเขียนจริง ถ้าไม่แน่ใจให้เปิดไฟล์อ่านซ้ำแล้วกบรทัดที่แก้มาเป็นหลักฐาน

ตัวแปลง GGUF ของ llama.cpp รองรับเฉพาะ checkpoint InternVL3 แบบ non-hf แต่ทั้ง pipeline ดันบังคับเทรนบนตัว '-hf'

กลาง 2026-08-30

ตรวจสอบความเข้ากันได้ของฟอร์แมตตลอดเส้นทาง (train → merge → export → serve) บนโมเดลเล็กที่สุดก่อนทุ่มเงินรันตัวใหญ่ อย่าปล่อยให้ขั้นตอน export เป็นสิ่งสุดท้ายที่ทดสอบ

แผนเสิร์ฟ adapter ของ t03 ผ่าน vLLM ใช้ไม่ได้ — layout MoE LoRA ของ Unsloth ไม่ตรงกับฟอร์แมตของ vLLM สักแบบ

กลาง 2026-08-30

เปิดดูไฟล์จริง (ชื่อ key, shape) เทียบกับฟอร์แมตที่ปลายทางคาดหวัง บนเครื่องตัวเองก่อนเข้า GPU สักใบ — 'เช็คของถูกก่อนของแพง'

งานเชื่อม Purson→web เขียนเสร็จทั้งเส้น แต่ไม่มีขั้นตอนไหนถูกรันเลย — migration ยังไม่ apply, function ยังไม่ deploy, ปุ่มไม่เคยถูกกดกับ GPU จริง

กลาง 2026-08-30

งานที่มีวันเดโมตายตัว ต้องซ่อมเส้นทางเติมตั้งแต่ต้นจนจบอย่างน้อยหนึ่งรอบก่อนถึงวันจริงนานๆ เขียนให้ชัดว่าขั้นตอนไหนยังไม่เคยรัน แล้วใช้รายการนั้นเป็นตารางงาน

`test\_browser\_path.py` (ใช้ JWT ของผู้ใช้จริง ไม่ใช่ service key) ไม่เคยรันจบกับโมเดลจริงเลย

กลาง 2026-08-30

จัดลำดับการตรวจสอบให้เคลมที่เสี่ยงที่สุดแบบต้นจนจบถูกพิสูจน์ก่อน ตอนที่เครื่องเขายังปกติดี อย่าเก็บเทสต์ที่สำคัญที่สุดไว้ท้ายสุดหลังการทดลองที่อาจทำให้ environment พัง

บ้านหนึ่งหลังใช้เวลา ~3 ชั่วโมงและค่า GPU ~\$3.2 — รันสดตอนฟรีเซนต์ไม่ได้

กลาง 2026-08-30

วัดเวลาต่อหน่วยของงานที่จะเดโม แล้วคูณด้วยขนาดงานจริงก่อนออกแบบรูปแบบการเดโม อะไรที่เกินสองสามนาที่ที่ต้องคำนวณไว้ล่วงหน้า

แนะนำให้ใช้ split เดิมต่อ 'เพื่อประหยัดบิไว้ทำ eval' โดยไม่เคยตรวจว่า val ของ split นั้นปนเปื้อนหรือเปล่า

กลาง 2026-08-30

อย่าให้คำแนะนำเรื่องบหรือตารางงานที่เกี่ยวกับ dataset จนกว่าจะพิสูจน์ความสะอาดของ split ก่อน สคริปต์ถูกๆ ตัวเดียวเล็กกว่าแผนทั้งแผนที่สร้างทับข้อสันนิษฐานนั้น

ลองจับคู่หน้าคานด้วยตาเปล่า แล้วพบว่าทำไม่ได้จริงๆ

กลาง 2026-08-31

ทดลองวิธีทำมือกับเคสจริงที่ยากที่สุดหนึ่งเคสก่อนจะจ้อคิวทำทั้งชุด และถ้ามีพิกัดอยู่แล้ว ให้เลือกวิธีเชิงเรขาคณิตที่คำนวณได้แน่นอน ดีกว่าให้คนมานั่งดู

Training ETA was off by a factor of two because InternVL-78B step times were applied to Qwen 35B-A3B

กลาง 2026-08-31

Never carry a step-time constant across model architectures — treat it as unknown until a real dry-run on the target model produces a number.

Destroyed three GPUs about two minutes before the 'don't return the cards yet' message arrived

กลาง 2026-08-31

Treat destroy as irreversible: when a human is awake and in the loop, pause briefly before destroying — the cost of waiting minutes is far below the cost of a lost machine.

destrier (t05) test results were written under t04_Purson, making them look like results from the dead t04 arm

กลาง 2026-08-31

Store cross-round benchmark assets in a round-neutral location from the start, and never write a new round's outputs into a previous round's folder just because the inputs happen to live there.

op04_run.py writes to fixed output filenames, so a comparison round would have destroyed the first round's results

กลาง 2026-08-31

Give every experiment run its own output directory keyed by run identifier — never rely on remembering to rename before a comparison run.

The U/P proportion fallback is assigned to the VLM, which research proved cannot measure at takeoff accuracy

กลาง 2026-08-31

Every place the pipeline asks the model for a number rather than a name is a place to audit against the symbolic/metric split, not just the one path being designed at the time.

t44 was built on InternVL3-78B, a model the team's own catalogue had disqualified days earlier

กลาง 2026-09-01

When research produces a disqualification list, make it the binding input to the next selection decision and record which shortlist entry was chosen and why. A catalogue nobody consults at decision time is wasted research.

pass3 can now measure unresolved beam spans but was never wired into the adapter to replace the nulls

กลาง 2026-09-01

Building the capability is half the fix. Track 'wired into the consumer' as a separate, explicit completion criterion, or the original bug survives untouched behind a solved-looking component.

Whole demo environment torn down and re-set-up a second time because the first run was not screen-recorded

กลาง 2026-09-01

Anything whose only purpose is to produce an artifact for someone else (recording, screenshot, exported file) must be armed and verified BEFORE the expensive run starts, not after — a GPU run cannot be replayed for free.

Codebase could not be reorganised because scripts hard-code paths, so a 115 MB duplicate copy was made instead — two sources that will drift

กลาง 2026-09-01

Duplicating files to organise them creates a second source of truth that WILL drift; this repo has already deleted one such folder for that exact reason. Fix the hard-coded paths, or use a single copy plus a manifest, before copying anything again.

Truncated-JSON recovery is a manual salvage step, not real automation

กลาง 2026-09-02

A salvage script relieves pain but hides the defect — log it as debt with the real fix named, and do not count the pipeline as automated while a human step remains in the normal path.

Pages mixing two views were a known warned-about risk that was only confirmed by a pilot read after the schema was designed

ต้า 2026-07-05

Keep doing the pilot read — it worked here and is the cheapest possible validation. But when a spec document warns about a structural risk, resolve it as a decision before extraction rather than carrying it as a warning; a warning that survives into the run becomes a mid-run schema change.

The spec doc was moved out of the Training repo, then quietly moved back — the cross-repo relocation was churn, not a decision that held

ต้า 2026-07-05

Keep a spec in the same repo as the data it governs. If a move is reversed later, record the reversal in the diary at the point it happens, otherwise the log leaves two contradictory locations on record.

The first condensed schema (45KB to 10KB JSON) was rejected as too short and redone

📅 2026-07-10

Before condensing a spec, write down what class of content must survive verbatim — optimising for a byte target first guarantees a second attempt.

The production folder was created without its 00 sort prefix and had to be renamed mid-move

📅 2026-07-10

Settle the naming convention for a folder before putting anything in it — renaming a directory mid-move is where stale path references get created.

The same source-document price anomaly re-flagged as a reading error on four consecutive houses

📅 2026-07-12

The first time a source-document anomaly is confirmed as genuine, write it into a project-level known-anomalies list — otherwise every reviewer pays to rediscover it.

Garbled terminal output (^[[B^[A) mistaken for a broken/stuck job

📅 2026-07-21

Judge whether a GPU job is alive with ``nvidia-smi`` (utilisation + VRAM), never with what the log text looks like.

First environment-repair attempt (reinstall unsloth) fixed nothing

📅 2026-07-21

To repair a broken environment, diff what actually changed (pip log / run log versions) and restore those exact packages — don't reinstall the package whose import failed.

First OOM fix attempt pulled the wrong lever (allocator tuning instead of LoRA rank)

📅 2026-07-21

Before applying a generic OOM remedy, itemize what is actually holding VRAM (weights / adapters / activations) and attack the largest term — a fix that saves 1% costs a whole restart cycle.

Workflow document status markers contradicted their own body text

📅 2026-07-25

Update the status marker in the same edit as the content it summarises, and re-verify a tracking document against the real artifacts periodically instead of trusting it.

House 08 (op2) work carried over from the 28th and still not started

📅 2026-07-29

Keep incident work and planned work on separate ledgers. An item that carries over twice should be either scheduled explicitly or dropped on purpose — silent carry-over is how a backlog becomes invisible.

Label Studio task generation was run as a mandatory op1 step on houses 08/09/10 three days before the whole thing was cancelled

📅 2026-07-30

When a downstream feature is under review, suspend its step in the upstream workflow rather than continuing to feed it. The cheapest artifact is the one not generated.

A parallel English diary folder had been stale since 2026-07-21 — two folders disagreeing

📅 2026-08-02

Never maintain two copies of the same document in different languages/locations. One source of truth — a stale duplicate is worse than no duplicate.

Two documentation links in No_touch_box/CLAUDE.md were already broken — missing a docs/ path segment

ตำ 2026-08-04

Run a link checker over the .md files. Broken doc links in an orientation file cost every new reader time and go undetected for months.

Pass 1 designed as an ordering/queue when it is actually a physical folder reorganisation

ตำ 2026-08-21

For any pipeline stage, write down what the next stage must find on disk before designing the stage — an ambiguous verb like 'organize' or 'queue' should be resolved into a concrete directory layout first.

Teaching chapter written in Thai against the diary's own English-only rule

ตำ 2026-08-21

When a rule and the reader's actual needs conflict, state the deviation and its reasoning in the same document — then amend the rule to carve out the case, so the next person does not have to re-argue it.

Two sessions edited run_house_batch.py the same day without knowing about each other

ตำ 2026-08-21

When two sessions work the same bug the same day, say so in the diary before editing, not after — otherwise the same file gets two independent fixes for one symptom and the second makes the first redundant.

Training run killed at step 20 (~\$0.35 burned) because the benchmark house was chosen wrong

ตำ 2026-08-24

Freeze train/val/test house assignment as a Phase 0 gate before any GPU is rented. Changing the split after step 0 always means paying for the run twice.

Workflow doc filename kept referencing houses 08/09/10 after the scope pivoted to house 11

ตำ 2026-08-24

When an experiment's scope changes, rename the artifact and fix the links. A filename that lies is worse than a broken link because it fails silently.

Advisor request #1 (Thai-English dictionary) was researched in full, then parked because nobody had defined which problem it was meant to solve

ตำ 2026-08-25

Before researching a suggestion, ask which measured failure it is aimed at. Item 3 in the same meeting pointed at a wrong number but had a real target and paid off; item 1 had the reverse shape and stalled.

The combined teaching book is named for July but already carries an August chapter, blocking the ตอนที่ 4 append

ตำ 2026-08-25

Never put a date range in the filename of an append-only document; rename at the first append, not after three.

The professor's 'pattern_recognition.py' was searched for across three repos and full git history — it exists nowhere

ตำ 2026-08-25

Confirm whether a referenced file is ours or a lecture artifact before searching for it — and the same class as the max_frames finding: a professor's pointer names a concept, not necessarily a path in our tree.

Teaching PDF written condensed, then rewritten from scratch at 13 pages after "ขอละเอียดกว่านี้"

ตำ 2026-08-26

For teaching material, ask the depth level up front, and default to showing rejected options with their reasons — the discarded candidates are usually the most useful part.

Commit named "t04" as a joke, containing no t04 module — a naming landmine

ตำ 2026-08-26

Commit messages are read months later by people with no context. Never use a joke name that collides with the project's real naming scheme.

CLAUDE.md merge conflict because two people appended work-log entries at the same spot

ตำ 2026-08-26

An append-at-top shared log conflicts on every parallel day. Either append at the bottom, or give each day its own file the way workmen's_diary already does.

26 OCR test sidecar files left sitting in the real image/ directory

ตำ 2026-08-29

Test runs of any sidecar-writing tool must target a scratch output root. Sidecars written next to real data are indistinguishable from production output on the next pass.

Schema §1 carried a stale claim that the prompt didn't teach split — it already did

ตำ 2026-08-29

Avoid documenting what another file does **not** do. If you must, put a pointer in the other file so a change there triggers the doc update.

Pass 4 cancelled after being built and wired into the pipeline

ตำ 2026-08-29

Confirm a pipeline stage is needed before building and wiring it. When cancelling, park the artifact rather than deleting — but count the round trip as evidence that scope needed settling earlier.

Experiment arm 2.4b built with marked-image swapping, then cancelled before ever running

ตำ 2026-08-29

Decide the experiment matrix before implementing arms. Building three arms and cancelling one is cheaper than building none — but only if the arms were genuinely undecidable in advance.

A plan_column prompt was rendered as a real file even though the subtask has no training config

ตำ 2026-08-29

Do not render a live-looking artifact for a dead target. If it must exist for reference, put it in a clearly separate folder rather than beside the three files that are real.

An op04 order for 'houses never done before' was given against an already fully-extracted 50-house library

ตำ 2026-08-29

Keep an extracted/remaining counter where orders get given. Checking the backlog before accepting the order is what turned a duplicate-download risk into a clean new series.

Recorded that HF login was done on all 4 cards when it was actually 3/4

ตำ 2026-08-30

Record per-machine setup state only after running a verification command on that specific machine (e.g. `hf whoami`), never from an assumption that a repeated step was completed everywhere.

A `testfile.txt` left over from debugging the upload bug ended up in a public HF repo

📅 2026-08-30

When debugging against a production or public destination, name artifacts distinctively and delete them in the same step that created them — never leave cleanup to memory.

The output directory was 5.6GB of which only 842MB was the real adapter — naive upload would have pushed redundant checkpoints

📅 2026-08-30

Separate the artifact you intend to publish into a staging directory before uploading; do not filter a mixed directory at upload time.

Fold numbering differs between HF repo names (1-based) and internal files (0-based) — a live mis-mapping hazard

📅 2026-08-30

Do not maintain two numbering bases for the same entity. If it cannot be avoided, write the full mapping table in the checklist and verify each artifact against it before publishing.

Supabase service key pasted into the wrong config field and with its first character missing (`b\_secret\_` instead of `sb\_secret\_`)

📅 2026-08-30

Validate secrets at load time — assert the expected prefix (`sb\_secret\_`) and reject unknown config keys loudly, so a mis-pasted or misplaced value fails with a clear message instead of an auth error.

146 template candidates stuck in staging awaiting human review — the bottleneck is people, not tooling, and auto-promotion is forbidden by a prior lesson

📅 2026-08-30

When a step must stay manual for correctness, build tooling that makes the manual step fast rather than letting the queue grow — and keep the tool advisory (it produces a command) rather than actuating.

Vast.ai credit balance was recorded as \$16.66 when it was actually \$84.82

📅 2026-08-31

Re-check the live account balance from the provider at the moment of the spend decision — never plan against a number copied from an earlier note.

Benchmark house images remain under t04_Purson — carried over as unfinished for a second day

📅 2026-08-31

When cleanup is deferred because a job is running, schedule it to the job's completion rather than to a generic to-do list, or it rolls over indefinitely.

Epoch count was changed in both arms' configs before the verification research had returned

📅 2026-08-31

When an instruction pairs a change with a verification step, complete the verification before committing the change unless the instruction explicitly says otherwise.

The CV batch was estimated at 70s per page from a cold-start first page; the real rate was ~13s

📅 2026-08-31

Discard the first sample when estimating a throughput rate — measure steady state across several items before projecting a batch duration.

Suspected the rasterizer was producing inconsistent DPI; measurement showed DPI was fine and the variance came from one house

📅 2026-08-31

Measure the distribution before naming a cause — "the sizes vary" and "the tool is broken" are different claims.

Renaming t05_Asclepius to t05_Courser invalidated both datasets because paths were baked into the generated JSON

📅 2026-08-31

Keep paths out of generated data — resolve them at load time from a single root — or a rename becomes a rebuild.

Opening the worker window is the step most often forgotten when starting the demo by hand

📅 2026-08-31

A startup step that is easy to skip and silent when skipped should be automated or asserted, not documented.

pass2.5 self-harvest output written to results with no consumer anywhere

📅 2026-09-01

Do not ship a producer stage before its consumer exists. If you must, mark it loudly in code and in the diary so nobody mistakes 'the file exists' for 'the feature works'.

An entire greedy-decoding experiment round failed and had to be archived as a dead result

📅 2026-09-01

Name experiment output folders with their outcome and order at creation time ('01_usable', '02_failed_greedy'), not afterwards — an ambiguous results directory costs a cleanup pass and risks someone quoting a dead run.

INVENTORY.csv was 70 rows behind reality — six whole categories of work were untracked

📅 2026-09-01

An inventory updated in batches is an inventory that is wrong most of the time; a new model/fold/result set should get its row when it is created, not at the next cleanup.

npm test was skipped for the session on the reasoning that only an untested file changed

📅 2026-09-02

'This file has no tests' is a reason to add one, not to skip the suite. Run the suite anyway — it is cheap and it guards the rest of the day's work.